

Custom Compiler™ Python Reference Manual

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	55
Related Products, Publications, and Trademarks	55
Conventions	55
Customer Support	56
Statement on Inclusivity and Diversity	57

1. Introduction to Custom Compiler Python	58
Python Packages	58
Python Modules	60
Adding Python Modules	60
Python Reserved Words	61

2. snps.custom.utils	63
snps.custom.utils.macros	63
source	63
ed	64
Using snps.custom.utils.debug	64
enable_breakpoints	65
start	65

3. snps.custom.cmd	66
object	68
PyExc_UserInterrupt	69
snps.custom.cmd.cm	69
checkIntents	71
createAffinity	72
createAlignment	73
createBlockSize	73
createBus	74
createCapacitance	75
createCluster	76

Contents

createCMOS	77
createColorMatching	79
createColumn	79
createDifferentialGroup	80
createEdgeAssign	82
createEnclosingShape	82
createGuardRing	84
createHalo	85
createHierNetSpacing	86
createInstanceMatching	87
createInstMatching	89
createIntent	91
createIntentDef	92
createIntentParamDef	93
createLength	94
createMaxCommonResistance	95
createNetPriority	96
createNetShielding	96
createOrientations	98
createPatternRoute	99
createPlacementGrid	100
createRelative	101
createResistance	101
createResistorBank	103
createRoutingStripes	104
createRows	105
createShieldOverhang	107
createStandardCells	107
createSymmetric	109
createTask	110
createTemplate	110
createWireMatching	111
createWireStriping	112
exportIntents	113
getBacksideMetalLayers	114
getConstraints	114
getFrontsideMetalLayers	115
getIntentDefs	116
getPlacement	117

Contents

getRowPlacement	118
getShieldOverhangData	119
getSizePolicy	120
getSpacingData	120
getTasks	121
importIntents	122
showConstraintManager	122
showExportIntents	123
showImportIntents	123
snps.custom.cmd.db	124
add	136
addAttr	137
addFigGroupMembers	138
addGroupMembers	138
addInit	139
addListener	140
addMarkerObject	141
append	141
attachTech	142
batch	143
checkDRD	143
checkFeature	144
checkForInterrupt	145
checkLicenseTier	145
checkoutFeature	146
checkVersion	147
cleanPcellRepository	147
closeCDF	148
closeDesign	149
copyParams	149
copyPrefs	150
createAssignAssignment	151
createAssignValue	151
createCallback	152
createCircularRuler	153
createCircularRulerDef	154
createCollection	155
createDesign	155

Contents

createDeviceMappingFileDialog	156
createEnum	156
createEvaluator	157
createExpressionMap	158
createFigGroup	159
createGenOptionSet	160
createGroup	160
createHistogram	162
createImage	163
createIncludeFileControl	164
createInstCache	165
createInstTerm	165
createInterfaceParam	166
createInterfacePin	167
createInterfaceSet	168
createInterruptHandler	169
createLibListInput	169
createLinearRuler	170
createMarker	171
createNet	172
createNetlistInst	173
createNotifier	174
createObject	174
createPaperSize	175
createParamDef	176
createParamSandbox	178
createParasiticStencil	178
createPcell	179
createPcellDef	180
createPin	181
createPref	182
createPrinter	183
createProgress	184
createProp	185
createScope	186
createScopeType	187
createScratchDesign	187
createStdViaVariant	188
createStreamNameMap	190

Contents

createSubCircularRuler	190
createWindowAnnotationSet	191
defineUdvDevice	192
defineVflagMaxDevice	192
defineVflagMaxMinDevice	193
defineVflagMinDevice	194
defineVsyncDevice	194
deleteBuiltInVias	195
deprecateAttr	196
destroy	197
detectNetlistType	197
disableUndo	198
enableTk	198
enableUndo	199
encrypt	200
engToSci	200
eval	201
evalCallback	202
evalEnvironment	202
evalParam	203
exportImage	204
exportMCellFile	205
exportParams	206
exportStream	207
exportTechFile	213
expr	214
findCircuitMatches	215
findConnections	216
findConstraint	217
findContextConnectionDef	217
findContextConnections	218
findGenerator	219
findInterfaceReader	219
flushPcellCache	220
freezeRefs	221
generateAnalogSchematics	221
generateBuiltInVias	223
generateCDL	223
generateCellView	224

Contents

generateConstraintPatterns	225
generateDeviceMappingTemplate	226
generateLayout	227
generateRule	228
generateSchematic	229
generateSPICE	232
generateSystemVerilog	232
generateVerilog	233
generateVerilogA	234
generateVerilogAMS	234
generateVHDL	235
getAnnotationSets	236
getAssignmentDefs	236
getAssignments	237
getBuiltinNetlists	238
getCallbacks	238
getCellViewInterface	239
getCircularRulerDefs	240
getComponents	240
getConnectionDefState	241
getContextConnectionDefs	241
getCurrentContext	242
getCurrentEvalText	243
getCurrentParam	243
getCurrentRef	244
getDeviceDisplays	244
getEffectiveNet	245
getEnums	245
getEquivalentNets	246
getEvaluators	247
getExportOasisOptions	248
getExportStreamOptions	248
getFeatures	249
getFigGroupMembers	249
getFigGroups	250
getFlightlineCoordinates	251
getGenOptionSets	251
getGroupMembers	252
getGroups	252

Contents

getInstHeaders	253
getInsts	254
getInstTerms	254
getLayers	255
getLicenseTier	256
getLPPHeaders	256
getMarkers	257
getNetlistInfo	258
getNets	259
getNotifiers	260
getOpenDesigns	261
getPaperSizes	261
getParamDefs	262
getParamFilter	263
getParams	263
getParamValue	264
getPcellCacheLocation	265
getPcellCacheSize	266
getPcellDefs	266
getPcells	267
getPins	268
getPrefs	268
getPrefValue	269
getPrinters	270
getProcessInfo	270
getProps	271
getRegisteredNetlistInfo	271
getRulers	272
getScopes	273
getScopeTypes	273
getShapes	274
getTableValue	274
getTerms	275
getType	276
getVDRCDeviceList	276
getViaDefs	277
getViaHeaders	278
getVias	278
getVirtualInstTerms	279

Contents

getWindowAnnotationSets	280
hideProps	280
importEDIF	281
importMCellFile	283
importSpice	283
importTechFile	284
importText	285
includes	287
init	288
instCacheTriggerParamCallbacks	289
interrupt	290
isEncrypted	290
isObject	291
isProhibitedName	291
listCallbackTypes	292
loadDeviceDisplays	292
loadGenOptionSets	293
loadMCells	294
loadModelFile	294
loadPrefs	295
loadStreamNameMap	296
localizePrefs	296
mapStreamName	297
match	298
notify	298
openDesign	299
openDesignHierarchy	300
openParamDefEditor	300
openScope	301
pdaExplainCell	302
pdaShowCell	302
print	303
purgeCDF	306
purgePrefs	306
query	307
queryFeature	307
raiseLicenseTier	308
refreshICCCache	309
refreshMCells	309

Contents

registerCallbackRegsub	310
registerGenerator	311
registerInterfaceReader	312
registerNetlistInfo	312
registerParamEvalFunction	313
relativePath	314
releaseFeature	315
remove	315
removeFigGroupMembers	316
removeMarkerObject	317
renderTemplatesParasitics	318
reopenDesign	318
reportCallbacks	319
reset	320
resetVDRCDefinedDevices	320
resolve	321
resolvePath	322
revertParam	322
revertPrefs	323
saveCDF	324
saveDesign	324
saveDesignAs	325
saveDeviceDisplays	326
saveGenOptionSets	326
savePrefs	327
sciToEng	328
setAttr	328
setDeviceDisplay	329
setInstName	330
setLicenseTier	331
setMaster	331
setNetlistInfo	332
setParamFilter	333
setParamValue	334
setPrefValue	335
setProgress	335
setViaDef	336
showAbstractGeneratorOptions	337
showAdvancedPrintOptions	337

Contents

showCellViewFromCellView	338
showCellViewFromPinList	339
showExportDef	339
showExportImage	340
showExportLef	341
showExportMCellFile	341
showExportOasis	342
showExportStream	342
showFindReplace	343
showGenerateAbstract	344
showGenerateLibertyFile	344
showGenerateSchematic	345
showGenericPrint	345
showImportDef	346
showImportLef	346
showImportMCellFile	347
showImportOasis	347
showImportSimulationDataDialog	348
showImportStream	349
showImportText	349
showImportVerilog	350
showLoadPreferences	350
showManageTechnology	351
showPrint	351
showPrintQueueStatus	352
showPurgePreferences	352
showRefreshMCells	353
showSavePreferences	354
showUnitDisplayScale	354
snapPoints	355
sort	356
source	356
sourceEnc	357
startStatistics	357
stopStatistics	358
transform	359
triggerParamCallback	360
unregisterGenerator	360
unregisterInterfaceReader	361

Contents

updateCDFParams	361
vdrcMgrAddNet	362
vdrcMgrAddNetPair	363
vdrcMgrAnnotateNet	363
vdrcMgrAnnotateNetPair	364
vdrcMgrCheckHierarchyMismatch	365
vdrcMgrCheckSymbolPlacement	365
vdrcMgrClearAllData	366
vdrcMgrDeleteNet	367
vdrcMgrDeleteNetPair	367
vdrcMgrImportLayout	368
vdrcMgrImportSchematic	369
vdrcMgrRefresh	369
viaDefExists	370
viewExists	371
vwait	371
snps.custom.cmd.idb	372
query	372
snps.custom.cmd.de	373
abortCommand	381
activateTools	382
addCapability	383
addObject	383
addPoint	384
addTemplateView	385
addTool	386
applyOLPPreset	387
audit	387
cancelCommand	388
cancelTransaction	389
clearHighlights	389
clearUnusedDesigns	390
clone	390
close	391
CommandInterface	392
commandOption	392
completeShape	393
copy	393

Contents

copyBookmark	394
createActivationEngine	395
copyClipboardToLib	396
createApplication	397
createArgument	398
createArgumentGroup	399
createAssociation	399
createBookmark	400
createCanvas	401
createColorExpression	402
createCommand	403
createContext	405
createCrossProbeTool	405
createErrorViewerDatabase	406
createErrorViewerMarker	407
createFavoriteDevice	408
createFavoriteDeviceCategory	409
createGradient	410
createHierObject	411
createHighlightSelector	411
createHighlightSet	412
createInterfaceElement	413
createLPPInput	414
createLPPSet	416
createNetHighlightSet	416
createObjectFilter	417
createOLPPreset	418
createOnCanvasWidget	419
createOverlay	420
createProbe	421
createSelectionEngine	422
createShapeEngine	423
createTemplate	424
createTool	425
crossCheckViews	426
cut	427
cycleActiveFigure	428
cycleTapObject	429
deactivateTools	430

Contents

defineOrigin	430
deleteBookmarks	431
deprecateCommand	431
descend	432
descendInPlace	433
descendIntoGroup	434
deselect	434
deselectAll	435
deselectAllLogic	435
deselectLogic	436
enableColorAnnotator	437
enablePartitioningDisplay	438
enableRotationAndScaling	439
endDrag	439
endLog	440
endTransaction	441
errorViewerLoadResult	441
exportBookmarks	442
exportProbeCellSettings	443
extractInterfaceElements	444
findMessage	444
finishEvalTextRendering	445
fit	446
getActiveCommand	447
getActiveContext	447
getActiveFigure	448
getActiveLPP	448
getApplications	449
getAssociations	450
getBackgroundImporters	450
getBBox	451
getBinsForGradient	451
getBookmarks	452
getBreakpoints	453
getCapabilities	453
getClipboardLibs	454
getClips	454
getColorAnnotators	455
getCommands	455

Contents

getContexts	456
getCrossProbeTools	457
getCursor	457
getCursorSnapPoint	458
getDBLimits	458
getDesignTemplates	459
getEditors	460
getErrorViewerDatabases	460
getFavoriteDeviceCategories	461
getFavoriteDevices	461
getFigures	462
getHighlighted	463
getHighlightSets	463
getInterfaceElements	464
getLayers	465
getLookaheadMessages	466
getLPPs	466
getLPPSets	467
getObjectFilters	467
getOLPPresets	468
getOpenFiles	469
getOutdatedDesigns	469
getOutdatedDMData	470
getOverlays	470
getPartitioner	471
getProbeBitNames	472
getProbeFigures	472
getProbeNets	473
getProbes	474
getRecentFiles	474
getSelected	475
getSelectedLogic	475
getSelectedVertex	476
getTemplates	477
getTools	477
getTransactions	478
getViewport	479
gotoViewport	479
help	480

Contents

highlight	481
importBookmarks	481
importProbeCellSettings	482
isEditable	483
isInUndo	483
isModified	484
linkContext	484
loadFavoriteDevices	485
loadInterfaceElements	486
loadOLPPresets	486
loadProbes	487
navigateTo	488
next	488
open	489
openCanvasPropertyEditor	490
openClipboardLib	490
openEditor	491
openFile	492
pan	492
paste	493
redoSelection	495
redoTransaction	495
redraw	496
registerHelp	496
removeBreakpoints	497
removeObject	498
removePoint	499
removeTool	499
renumberInstances	500
repeatCommand	501
replay	501
replayCommand	502
reportOpenTransactions	503
restoreBookmark	504
return_	504
revert	505
run	506
runBackgroundImporter	506
save	507

Contents

saveAs	507
saveFavoriteDevices	508
saveInterfaceElements	509
saveOLPPresets	509
saveProbes	510
select	511
selectAll	512
selectLogic	512
sendComplete	513
sendMessage	514
setActiveLPP	514
setBreakpoint	515
setCursor	515
setEditMode	516
setErrorViewerCategory	517
setErrorViewerDetailInfo	518
setErrorViewerFieldName	518
setErrorViewerTip	519
setViewport	520
shiftCursor	520
showAbout	521
showBookmarkManager	522
showCloseDesigns	522
showConnectionsNavigator	523
showConnectionsNavigatorOptions	523
showCreateBookmark	524
showCrossCheckOptions	525
showDescendDialog	525
showDesignsPrompt	526
showGeneralOptions	527
showMakeDesignsReadOnly	528
showOpenDesign	528
showParamFilter	529
showProbeBitSelectionDialog	530
showProperties	530
showRefreshDesigns	531
showRenumberInstances	532
showReturnToLevelDialog	532
showSaveDesignAs	533

Contents

sleep	533
snapToObject	534
startDrag	535
startLog	535
startRendererStatistics	536
startTransaction	537
stop	538
stopBackgroundImporter	538
stopRendererStatistics	539
suppressMessage	539
transformPoint	540
undoSelection	540
undoTransaction	541
unhighlight	542
uniquify	542
zoom	543
snps.custom.cmd.ide	544
createProbe	545
defineOrigin	545
descend	546
descendIntoGroup	547
editCanvasText	548
enterBox	548
enterLine	549
enterPoint	550
enterPolygon	551
highlight	552
pan	552
pick	553
selectByName	554
selectByRegion	555
setViewport	555
zoom	556
snps.custom.cmd.di	557
closeResultsCellView	558
createResultsCellView	559
getCapacitors	560
getCapAttr	561

Contents

getCorners	561
getElmoreDelayResult	562
getElmoreDelayResultAttr	562
getFanoutResult	563
getFanoutResultAttr	564
getInstAttr	564
getInsts	565
getLayers	566
getModels	566
getNetAttr	567
getNets	568
getNodeAttr	569
getResAttr	569
getResistanceContributionAttr	570
getResistanceResult	571
getResistanceResultAttr	572
getResistors	572
getTermAttr	573
getTerms	574
getViewTypes	574
listAttrs	575
loadResultsCellView	576
runElmoreDelayAnalysis	576
runFanoutAnalysis	577
runResistanceAnalysis	578
snps.custom.cmd.dm	579
addICCRRefLibs	582
addToCellCategory	583
addToLibDefs	584
associateVersionControl	585
checkIn	585
checkLock	586
checkOut	587
checkStatus	587
compress	588
convertDMSystem	589
copyCellCategory	589
copyCells	590

Contents

copyCellsUnique	592
copyCellViews	593
copyLib	595
copyProps	597
createCell	598
createCellCategory	598
createCellInput	599
createCellView	601
createCellViewInput	601
createLib	603
createLibInput	604
createMilkywayLib	605
createMultiCellViewInput	606
deleteFromCellCategory	607
findCell	608
findCellView	609
findDMData	610
findDMFile	610
get3DICLayerMap	611
get3DICRefLibs	611
getCategoriesFromCategory	612
getCategoriesOfMember	613
getCellCategories	613
getCells	614
getCellsFromCategory	615
getCellViews	615
getDMFileFollowers	616
getDMFileLeader	617
getDMFiles	617
getFCLayerMap	618
getFCRefLibs	619
getFollowers	619
getICC2LayerMap	620
getICC2RefLibs	620
getLeaders	621
getLibDefPath	622
getLibs	622
getMilkywayRefLibs	623
getOpenDesigns	623

Contents

getSelected	624
getStandardLibDefPath	624
getTopLibDefPath	625
getVersionControl	625
getVersionControlSystems	626
isDMFileLeader	627
isHidden	627
moveCells	628
moveCellViews	629
moveLib	630
openLibraryManager	631
openLibs	632
refreshLibraryManager	632
refreshLibs	633
registerCopyProps	633
registerVersionControl	634
removeVersionControl	636
renameCellCategory	637
revert	637
saveAsFCRefLib	638
saveAsICC2RefLib	638
set3DICLCVMap	639
set3DICRefLibs	640
setFCLCVMap	640
setFCRefLibs	641
setFollowers	642
setICC2LCVMap	643
setICC2RefLibs	643
setICCTechSync	644
setSelected	645
showCopyCell	646
showCopyCellView	646
showCopyLibrary	647
showCopyUnique	647
showDeleteViewsByName	648
showEditLibraryCompression	648
showLibraryManager	649
showMoveCell	650
showMoveCellView	650

Contents

showMoveLibrary	651
showNewCell	651
showNewCellView	652
showNewLibrary	652
showRenameCellCategory	653
syncToVersion	654
uncompress	654
updateLib	655
snps.custom.cmd.dr	656
createColor	657
createColorSelector	658
createDisplay	659
createLineStyle	660
createLineStyleSelector	661
createPacket	662
createPacketAlias	663
createStipple	663
createTextPacket	664
deleteColor	665
deleteLineStyle	666
deleteStipple	666
getColors	667
getDisplays	668
getLineStyles	668
getPacketAliases	669
getPackets	670
getStipples	670
getTextPackets	671
loadDisplayResources	672
mergeDisplayResources	672
renameColor	673
renameLineStyle	674
renameStipple	675
saveDisplayResources	675
showDisplayResourceEditor	676
snps.custom.cmd.dspf	677
getLines	677
patch	678

Contents

snps.custom.cmd.gi	679
addAction	683
addAssistant	684
addMenu	685
addToolBar	686
addWindow	687
applyWorkspace	688
closeFrames	688
closeWindows	689
collapse	689
configureTable	690
createAction	691
createActionCategory	693
createAppGrid	693
createApplcon	694
createAssistant	695
createBinding	696
createBindingSet	698
createBooleanInput	698
createCarousel	699
createCarouselItem	700
createCheckableGroup	701
createColumn	703
createDataType	704
createDbTable	705
createDialog	707
createDrag	709
createDynamicList	710
createFileInput	711
createFrame	713
createGroup	714
createInlineGroup	715
createLabel	715
createListInput	716
createMenu	718
createMutexInput	719
createNumberInput	721
createPointListInput	723
createPushButton	724

Contents

createRow	725
createScrollArea	726
createSplitterGroup	727
createTabGroup	728
createTable	729
createTextInput	731
createTheme	733
createToolBar	733
createWebView	734
createWindow	735
createWindowType	736
createWorkspace	737
createX11Window	738
drop	739
execDialog	740
executeAction	740
expand	741
exportTable	742
findChild	742
getActionCategories	743
getActions	744
getActiveDialog	744
getActiveTab	745
getActiveWindow	746
getAppIcons	746
getAssistants	747
getBindings	747
getBindingSets	748
getCells	748
getChildren	749
getColumns	750
getContextMenu	750
getDataTypes	751
getDialogs	751
getField	752
getFrames	753
getMenus	753
getRows	754
getSectionSizes	755

Contents

getThemes	755
getToolbars	756
getWindows	757
getWindowTypes	757
getWorkspaces	758
getXidUsage	758
hideContextMenu	759
layout	759
loadBindings	761
loadWorkspaces	761
moveColumns	762
moveRows	762
openTerminal	763
pickWindow	764
pressButton	765
prompt	765
registerWebViewScheme	766
remove	767
reportWidgetState	768
saveBindings	768
saveWorkspaces	769
scrollTo	770
sendGUIEvent	770
setActiveDialog	771
setActiveTab	771
setActiveWindow	772
setContextMenu	773
setCurrentIndex	774
setDropAction	774
setField	775
setFocus	776
setItemSelection	776
setSectionSizes	777
showBusyCursor	778
showConsole	778
showContextMenu	779
showCustomizeWindow	780
showHideAppIcons	780
showKeyboardShortcuts	781

Contents

showSaveHistoryAs	781
sortItems	782
unregisterWebViewScheme	783
snps.custom.cmd.he	783
addRule	785
addViewGroup	786
clone	786
closeConfig	787
closePropDB	788
computeId	788
createConfig	789
createConfigTemplate	790
createContext	791
createLogicalSelectionEngine	791
createNetInput	792
createOverlayConfig	794
createOverlayView	795
createRule	795
createViewGroup	797
deleteRule	797
deleteViewGroup	798
expandViewList	799
findPropDB	799
findSwitchedMaster	800
foreach	801
getConfigTemplates	802
getConfigView	802
getExternalFileName	803
getInstHeaders	803
getInsts	804
getProps	805
getPropValue	805
getRules	806
getSelected	808
getViewGroups	808
isExternalFile	809
isLeaf	809
listPropDBs	810

Contents

openConfig	811
openPropDB	812
popContext	812
pushContext	813
queryViewSelection	814
saveAsPropDB	814
saveConfig	815
savePropDB	816
setPropValue	816
setRuleFromDE	817
showExportToFile	818
showHierarchyEditor	819
showObject	819
undoRedoHistory	820
snps.custom.cmd.il	821
evalNamedArgFuncCall	821
sprintf	822
toTcl	822
snps.custom.cmd.le	823
addDummy	831
addToFigGroup	832
align	833
alignAngle	834
autoFix	835
autoVia	836
boolean	837
breakSync	838
changePinRelationship	839
checkCapacitance	839
checkDRCViolations	840
checkDRCViolationsByICV	841
checkEMResConstraints	842
checkOpens	843
checkPinConnectivity	844
checkShapeEM	844
checkShield	846
checkShorts	846
checkTrackSnapping	847

Contents

checkVias	848
checkVoltageAnnotations	849
chop	851
clearFlightlines	852
clearPinConnections	853
clearVoltagePhysicalAnnotations	854
clearVoltages	854
cloneMaster	855
closeICVLManager	855
compactBus	856
connectICNet	857
convertToAliasLayer	858
convertToArray	859
convertToPath	860
convertToPolygon	860
copy	861
copyHighlighted	863
copyLook	864
copyTrackRegionDef	864
correctOffgrid	865
createAttributeLabel	866
createBlockage	867
createBoundary	868
createCellViewFromSelection	869
createCoordinateMark	870
createCorridorArea	871
createDRDLineEdit	872
createDRDTreeWidget	873
createEllipse	874
createFlightlines	875
createGlobalTrackRegion	876
createGuardRing	877
createICInst	878
createICNet	879
createInst	879
createLabel	881
createMPP	882
createMPPDef	883
createMPPEnclosedPathDef	884

Contents

createMPPEnclosedShapeDef	885
createMPPMasterPathDef	886
createMPPOffsetPathDef	888
createMPPRingDef	889
createMPPSubRectDef	891
createNetGroup	893
createOffTrackInfo	893
createPath	894
createPathSeg	895
createPin	896
createPinsFromText	897
createPolygon	898
createRectangle	898
createRouteCorridor	899
createRow	900
createSiteDef	901
createStripedWire	902
createStripeSegment	904
createSyncFigGroup	905
createSyncGroup	906
createTerm	906
createTrack	907
createTrackGroupDef	909
createTrackPattern	910
createTrackPatternDef	911
createTrackRegion	913
createTrackRegionDef	914
createVia	916
createVoltagePhysicalAnnotations	917
cutCorner	919
cycleDensity	919
delete	920
deleteCutMetal	921
deleteDensityResult	922
deleteGeneratedPins	922
designSummary	923
displayFlightlines	923
displayPinConnections	924
distribute	924

Contents

evalViaParams	926
exportResistanceQuery	927
externalizeColor	928
extractLayoutContext	929
fill	929
fixTrackSnapping	931
flatten	932
generateAbstract	933
generateAntennaRunset	935
generateCutMetal	936
generateGuardRing	938
generateLabel	940
generateLayout	942
generateLayoutContext	943
generatePGMarkers	944
generatePins	945
generateShapes	948
generateSoftAbstract	949
getAbuttedInsts	950
getAbuttedMPPs	951
getAreaAndPerimeter	952
getBoundaryTrackRegionDefs	952
getCommonPathPoints	953
getCoordinateMarks	954
getFlightlines	954
getGateDistance	955
getGateRectangles	956
getGlobalTrackRegions	957
getIncompleteNets	958
getIntersections	958
getMPPDefs	959
getMPPs	960
getNearestTracks	961
getNetGroups	962
getNetIslands	962
getPseudoParallelNets	963
getRouteCorridors	964
getRuleInfoByICV	964
getStripedWires	965

Contents

getSyncGroups	966
getTrackGroupDefs	967
getTrackPatternDefs	967
getTrackPatterns	968
getTrackRegionDefs	969
getTrackRegions	969
getTracks	970
getTracksInRegion	971
getUnboundGlobalTrackRegions	972
getUnboundTrackRegions	973
getUnboundTracks	973
getUnnettedFigs	974
highlightConnected	974
importCommandBindings	976
importResistanceQuery	976
importVoltages	977
isInSync	978
snps.custom.cmd.le	978
merge	979
move	980
padLabel	981
pasteLook	982
propagateConnectivity	983
refreshDensityResult	983
regenerateMPP	984
remaster	985
removeFromFigGroup	985
removeOverlappingCuts	986
removeUnboundInstanceTerms	987
repeatCopy	987
replaceVia	988
reportCapacitance	989
reportCELayers	990
reportConnectivityPoints	990
reportEM	991
reportNets	992
reportRC	993
reportResistance	995
reportShield	997

Contents

resetVoltageOverrides	997
reshape	998
resize	999
rotate	1000
scalarizeArrayInst	1001
setNetAnchor	1002
showAlignAssistOptions	1002
showCommandBindings	1003
showCreateTrack	1003
showCreateTrackGroupDefinition	1004
showCreateTrackRegionDefinition	1004
showDefineTerminals	1005
showDesignOptions	1006
showDRDOptions	1006
showDRDRuleSelection	1007
showEditTrack	1007
showEditTrackGroupDefinition	1008
showEditTrackRegion	1008
showImportLayout	1009
showLayoutGeneratorOptions	1010
showOLPOptions	1010
showTrackRegionOptions	1011
split	1012
splitArray	1013
splitIntoBoxes	1013
stretch	1014
syncTo	1015
toggleDRDTreeWidget	1016
trimWire	1016
validateColor	1018
yank	1019
snps.custom.cmd.le.pd	1020
drawMap	1020
clearMap	1021
createMap	1022
deleteMap	1022
drawLegend	1023
snps.custom.cmd.le.pkg	1024
highlightPlaneBoundary	1025
realizePlane	1026

Contents

replacePlaneBoundary	1027
reportDifferentialPairConsistency	1028
reportRedundantVias	1029
reportShieldingRate	1029
checkPlaneBoundary	1030
createDegassingHole	1031
createPlaneBoundary	1032
deleteDegassingHole	1034
deleteFloatingShapes	1034
deleteRedundantVias	1035
generatePlaneBoundary	1036
generateRedundantVias	1039
generateSealRing	1040
generateZigzag	1040
getPlaneBoundaries	1041
snps.custom.cmd.ile	1042
addDummy	1047
addToFigGroup	1047
adjustCursor	1048
align	1048
alignAngle	1049
applyColor	1049
attachDetach	1050
autoChopGuardRing	1051
autoFix	1051
autoFlipHorizontal	1052
autoFlipVertical	1052
autoRotate90	1053
boolean	1053
bridgeInterconnect	1054
changePinRelationship	1054
checkCongestion	1055
checkDensity	1056
checkVias	1056
checkVoltageAnnotations	1057
chop	1057
clearDensityDisplay	1058
clearVoltagePhysicalAnnotations	1058
clearVoltages	1059
cloneMaster	1060

Contents

collapseDensityTree	1060
color	1061
compactBus	1061
continueBus	1062
convertToArray	1062
convertToPath	1063
convertToPolygon	1063
copy	1064
copyLook	1065
copyToClipboard	1065
correctOffGrid	1066
createAttributeLabel	1066
createBlockage	1067
createBoundary	1067
createBus	1068
createCellViewFromSelection	1069
createCoordinateMark	1069
createEllipse	1070
createFigGroup	1070
createGlobalTrackRegion	1071
createGuardRing	1071
createInst	1072
createInterconnect	1073
createLabel	1073
createMPP	1074
createPin	1074
createPinsFromSelection	1075
createPolygon	1075
createRectangle	1076
createRouteCorridor	1076
createRow	1077
createRuler	1078
createTopology	1078
createTrackRegion	1079
createTrackRegionPCell	1079
createTransmissionLine	1080
createVia	1080
createVoltageConfig	1081
createVoltagePhysicalAnnotations	1082

Contents

cut	1082
cutCorner	1083
delete	1083
deleteCutMetal	1084
deleteFigGroup	1084
distributeAlign	1085
eraseLayer	1086
expandDensityTree	1086
fill	1087
flatten	1087
generateCutMetal	1088
generateGuardRing	1088
generateLabel	1089
generateLayoutPins	1089
generateMPP	1090
generatePins	1090
generatePlacementTrackRegion	1091
generateRoutingTrackRegion	1092
generateShapes	1092
heal	1093
hierarchicalCopy	1093
highlightConnected	1094
highlightPath	1094
importVoltages	1095
inputCoord	1095
lineSelect	1096
measureAngle	1096
measureDistance	1097
measurePath	1098
merge	1098
move	1099
padLabel	1099
partialToFull	1100
paste	1101
pasteLook	1101
reassignNet	1102
removeFromFigGroup	1102
removeOverlappingCuts	1103
replaceVia	1103

Contents

reportCapacitance	1104
reportEM	1104
reportParasitic	1105
reportResistance	1106
reshape	1106
reshapeInterconnect	1107
resize	1107
rotate	1108
scalarizeArrayInst	1108
selectByLPP	1109
selectObjects	1109
selectRouteObjects	1110
showDensityDisplaySetting	1110
showEMCalculator	1111
split	1112
splitArray	1112
stackInterconnect	1113
stretch	1113
stretchArray	1114
stretchWidth	1114
stripeWire	1115
tap	1116
trimWire	1116
validateColor	1117
yank	1117
snps.custom.cmd.ile.pkg	1118
createDegassingHole	1118
createPlaneBoundary	1119
definePackagingLibrary	1119
generateRedundantVias	1120
generateZigzag	1120
realizePlane	1121
replacePlaneBoundary	1122
snps.custom.cmd.lp	1122
align	1123
applyTemplates	1124
autoPinPlace	1125
autoPlace	1126
checkPlacementRule	1126

Contents

cngenlib	1127
copyResult	1128
createGraph	1129
doDeviceGrouping	1130
doStdCellGrouping	1131
exportPinPlacement	1132
extractRelativeString	1133
extractTopology	1133
fixPlacementRule	1134
fixRelativeConstraint	1135
generateConstraint	1135
group	1136
initCellInfo	1137
insertBoundary	1138
insertColumn	1139
insertDecapFiller	1140
insertDiode	1141
insertTap	1142
optimizePins	1143
placePins	1144
showOptions	1146
snps.custom.cmd.ilp	1146
autoPinPlace	1147
autoPlace	1147
copyResult	1148
extractTopology	1148
optimizePins	1149
placeToRows	1149
snps.custom.cmd.lx	1150
addConditionalDummy	1153
addPenalty	1153
applyPresetParams	1154
changeInstanceMapping	1155
clearAnchors	1155
clearCrossHighlight	1156
clearHighlight	1157
clearNetVoltage	1157
clearRuler	1158

Contents

createSession	1158
establishCorrespondence	1159
establishCorrespondenceFromLVS	1160
expandSchematic	1161
exportSdlNetlist	1163
findCloneCandidates	1163
generatePlacementCatalog	1164
getAssociatedObject	1165
getAssociatedObjectByName	1166
getDeviceInfo	1167
getFoldingParam	1167
getFoldingThreshold	1168
getMismatchedConstraints	1168
getPlacement	1169
getSession	1170
getSessions	1170
highlightFlightline	1171
importSDLMappingFile	1171
loadAddShapeSettings	1172
loadCellTemplate	1173
snps.custom.cmd.lx	1173
preview	1174
realize	1174
realizeConstraint	1175
realizeTemplate	1176
rechain	1177
registerLayoutTemplate	1177
reloadSymbolicEditor	1178
removePeriphery	1179
reopenLogicalSource	1179
replacePlacement	1180
saveAddShapeSettings	1181
saveCellTemplate	1181
saveConstraint	1182
sedGetDesign	1182
setPlacement	1183
setupSDL	1184
showAddShapeSettingsDialog	1184
showAllNet	1185

Contents

showBatchCreateTemplate	1186
showCellTemplate	1186
showCheckerDialog	1187
showCheckOptions	1187
showCreateTemplate	1188
showDefineSource	1188
showDefineTarget	1189
showExpandSchematic	1189
showGateConnected	1190
showGateNet	1191
showGenerateLayout	1191
showImportLayout	1192
showODNet	1192
showOptions	1193
showPresetSetup	1193
showSDLCheckAndRepairWindow	1194
showSDLMapping	1194
showSymbolicDesignOptions	1195
showUpdateSchematic	1196
splitDevice	1196
unregisterLayoutTemplate	1197
updateConstraints	1198
updateLayout	1198
verify	1199
verifyNetVoltage	1200
snps.custom.cmd.ilx	1200
add	1203
addDummy	1204
addIncremental	1204
addPeripheralDummy	1205
alignGates	1206
alignIndividual	1206
alignRow	1207
applyPresetParams	1208
associateSpareDummy	1208
calculateGridSize	1209
chainSerialRes	1209
clone	1210

Contents

copy	1211
customizeSpacing	1211
delete	1212
deleteTrack	1213
distance	1213
establishCorrespondence	1214
flip	1214
flipRow	1215
foldGates	1216
generateGroupPlacements	1216
generateGuardRings	1217
generatePattern	1218
generatePlacement	1218
highlightFlightline	1219
insertColumn	1220
insertRow	1220
insertShift	1221
insertTrack	1222
manualCorrespondence	1222
merge	1223
mergeGates	1223
mirror	1224
modifyDummy	1225
modifyFlightline	1225
modifyPode	1226
modifyPreset	1227
move	1227
moveFlightline	1228
optimizeOD	1229
pickAndPlace	1229
reassignNet	1230
rechain	1231
removeColumn	1231
removeLayoutParam	1232
removeRow	1233
removeShift	1233
reportSummary	1234
resetCustomizedSpacing	1235
resetIndividualAlignment	1235

Contents

rotate	1236
sedRealize	1237
selectEmpty	1237
split	1238
splitDevice	1239
swapDevice	1239
swapInterdigitate	1240
swapSD	1240
swapTerm	1241
symbolicEditor	1242
transpose	1242
unalignGates	1243
unsetFingerGroup	1244
updateLayout	1244
viaTrackPlanner	1245
snps.custom.cmd.ne	1246
getLogicals	1246
snps.custom.cmd.nl	1247
addMapping	1250
addVariableReference	1251
addViewStopList	1252
append	1252
calculateMapping	1253
checkInstHeader	1254
checkVHDLPortConsistency	1255
checkVHDLUniqueEntityDefinitions	1255
clearBuffer	1256
clearModuleScope	1256
createAMSNetlister	1257
createHierMap	1258
createHSPICENameMap	1258
createNameMap	1259
createNetlister	1260
createOANameMap	1260
createParameterizedNameMap	1261
createPartitioner	1263
createTerm	1263
extractDigitalPowerDomains	1264

Contents

finalizeNameMap	1265
findAMSDomain	1266
findAMSDomainObjects	1266
findInstNamePrefix	1267
findModuleDir	1268
findModuleName	1268
findNetlistProcedure	1269
findParamMap	1270
findParamValue	1270
flattenDesign	1271
generateEvent	1272
getAlwaysWriteParamDefault	1273
getAMSNetlisters	1274
getBuffer	1274
getBufferLastPart	1275
getEffectiveNetlistInfo	1275
getExternalEntities	1276
getInstTerms	1277
getMappedContext	1277
getMappings	1278
getModuleDescription	1279
getModuleNameMapForAMS	1279
getNamesForMapping	1280
getNetlisters	1281
getNetMembers	1281
getNetType	1282
getParamNames	1283
getReferencedVariables	1283
getShortedNets	1284
getTerms	1285
getVirtualTerms	1286
getWireBitOrder	1286
getWorkerNetlisters	1287
indent	1288
instHeaderHandler	1288
isLeaf	1289
isModuleUpToDate	1290
isNetNameGlobal	1290
isStringParam	1291

Contents

isTextModule	1292
lookup	1292
map	1293
mapPath	1294
mapTerminalNames	1294
message	1295
moduleBodyHandler	1296
moduleHandler	1296
modulePreprocessHandler	1297
netlistBeginHandler	1297
netlistBodyHandler	1298
netlistEndHandler	1299
netlistMainHandler	1299
openNameMap	1300
preloadModuleNames	1300
printSpectreInst	1301
printSpiceInst	1302
printSpiceInstTerms	1303
refreshPartitioner	1303
revertOutdatedDesigns	1304
runNetlister	1304
saveModuleState	1305
saveNameMap	1306
setTermOrder	1307
setWireBitOrder	1307
uniqueify	1308
snps.custom.cmd.po	1309
and_	1311
angle	1312
append	1313
area	1314
chop	1314
coinEdge	1315
compress	1316
convexEdge	1316
createCollection	1317
createPointSet	1318
cut	1319

Contents

cutCorner	1320
density	1321
donut	1322
enclose	1323
encloseRect	1324
enclosure	1325
expandEdge	1327
extent	1328
extents	1328
external	1329
getAngle	1331
getArea	1332
getContours	1332
getDistance	1333
getHoles	1333
getIntersection	1334
getMaxBox	1335
getPerimeter	1335
getPoints	1336
getSpecifiedBoxes	1336
inside	1337
insideEdge	1338
interact	1339
internal	1340
isClockwise	1342
isHorizontal	1342
isVertical	1343
length	1344
makeClockwise	1344
manhattanContour	1345
not_	1346
offGrid	1346
or_	1347
outside	1348
outsideEdge	1349
pathLength	1349
perimeter	1350
polyToTrap	1351
rectangle	1351

Contents

rectangles	1352
rectEnclosure	1353
shapePoints	1354
shift	1355
size	1356
snapGrid	1357
straddle	1357
touch	1358
touchEdge	1359
transform	1360
vertex	1361
withEdge	1362
withNeighbor	1362
withWidth	1363
xor	1364
snps.custom.cmd.ps	1365
getOptoDesigner	1365
startOptoDesigner	1365
terminateOptoDesigner	1366
snps.custom.cmd.rc	1366
getCells	1367
getCouplingCapacitors	1368
getElmoreDelay	1369
getGPDCorners	1370
getGPDLayers	1370
getGroundCapacitors	1371
getNets	1372
getPins	1373
getPointToPointResistance	1373
getPorts	1374
getResistors	1375
readParasitics	1376
removeAnnotatedParasitics	1376
reportGPDCConfig	1377
reportGPDPProperties	1378
resetGPDCConfig	1378
setGPDCConfig	1379
snps.custom.cmd.rt	1380

Contents

connectCellRails	1381
createFreeFormTopology	1382
createGroupPattern	1383
createPowerStaples	1384
createPowerStripes	1386
createTrunkTopology	1388
createUniblock	1389
deleteTopology	1390
deleteTrunkAndShield	1391
deleteTrunkBox	1392
exportTopology	1392
forceConnect	1393
generateTrunk	1394
getTopologies	1397
getTopologyInfo	1398
getTopologyNets	1399
importTopology	1399
insertVias	1400
powerHookup	1401
reportSimilarity	1403
routeHierarchy	1403
snps.custom.cmd.rt.pkg	1404
autoTaper	1404
pushRoute	1405
reportD2dRoutingBottleneck	1406
reportNetLength	1407
reportPlaneBoundaryViaConnections	1408
route	1408
snps.custom.cmd.irt	1409
createRoute	1409
snps.custom.cmd.sdp	1410
MIU	1410
snps.custom.cmd.se	1411
activateSheet	1414
addSheets	1414
addToOverlayRendering	1415
align	1416
attachText	1417
check	1417

Contents

chopWires	1419
clearNetColorizations	1419
colorizeNets	1420
connectWires	1421
copy	1421
createArc	1423
createBlock	1424
createCheckOption	1425
createConnectDef	1425
createEllipse	1426
createInst	1427
createLine	1428
createMask	1429
createNetColorizationDefinition	1430
createPin	1430
createPolygon	1432
createRectangle	1432
createRule	1433
createSelectionShape	1434
createTable	1435
createText	1437
createViolation	1438
createWire	1439
createWireName	1440
createWireStubs	1442
delete	1443
disconnectWires	1443
enableDynamicNetLabels	1444
extractConnectivity	1445
findReferenceDifferences	1445
generateEstRCSchematic	1446
generateSchematic	1447
generateSymbol	1448
getActiveSheet	1449
getBillOfMaterials	1449
getColorizationDefinitions	1450
getDynamicNetLabels	1450
getNetComponents	1451
getPinWiringPoint	1452

Contents

getRules	1452
getShapeStyle	1453
getSheets	1453
getTables	1454
importConfig	1455
importSymbol	1455
isNetlistable	1456
move	1457
moveSheetToPosition	1458
populateDesignSummary	1458
propagatePortOrder	1459
registerLabelFunction	1460
registerParasiticTopology	1461
registerPinShape	1462
renameNet	1462
resizeSheets	1463
setConnectivityValid	1464
setShapeStyle	1464
setSheetColumns	1465
showCheckOptions	1466
showEditTable	1466
showLibraryCheck	1467
showSchDesignOptions	1468
showSymDesignOptions	1468
showSynchronizePortOrder	1469
stretch	1469
syncOverlayDesign	1470
unregisterLabelFunction	1471
updatePinLabels	1472
snps.custom.cmd.ise	1472
align	1474
alignToColumn	1474
attachText	1475
check	1476
chopWires	1476
colorizeNets	1477
connectWires	1477
copy	1478

Contents

copyToClipboard	1479
createBlock	1479
createColumn	1480
createConnectDef	1480
createInst	1481
createLabel	1482
createMask	1482
createNote	1483
createOverlayInstanceWithMask	1484
createRuler	1484
createSchematicPin	1485
createSelectionShape	1485
createShape	1486
createSymbolPin	1487
createTable	1487
createTemplate	1488
createWire	1488
createWireName	1489
createWireStubs	1490
cut	1490
delete	1491
disconnectWires	1491
ignoreInst	1492
importSymbol	1493
manageLabels	1493
move	1494
paste	1494
renameInst	1495
route	1496
stretch	1496
snps.custom.cmd.slr	1497
flush	1497
isActive	1498
start	1498
stop	1499
snps.custom.cmd.te	1499
bindExternalNetlistView	1500
getActiveObject	1501

Contents

getCursor	1501
getTokens	1502
insertText	1503
removeText	1503
setCursor	1504
snps.custom.cmd.ue	1505
cndbg	1505
compile	1506
getDesign	1506
preview	1507
searchAndReplace	1508
showOptions	1508
showSearchReplaceDialog	1509
updateCanvas	1509
snps.custom.cmd.iue	1510
align	1511
boolean	1511
copy	1512
debug	1512
fill	1513
move	1513
remove	1514
setOrigin	1514
size	1515
stretch	1515
snps.custom.cmd.xt	1516
addJobClassPrefix	1520
addPlotPacket	1521
annotateParasitics	1522
clearPlotPackets	1522
clearViolations	1523
closeTextViewerTab	1523
configureQueueEngine	1524
createCrossRef	1525
createDebugSession	1526
createIntegration	1527
createJob	1527
createJobClass	1529

Contents

createJobGroup	1530
createParDBfromQuery	1531
createPDKConfig	1532
createPlot	1533
createPlotQueueItem	1533
createPlotViewer	1534
createPlotViewerDef	1535
createPlotViewerInst	1535
createPveJob	1536
createSimRun	1537
createSimulator	1537
createTool	1538
crossProbe	1539
findExecutable	1540
findFeature	1540
findFeatureManifest	1541
findJobGroup	1542
getCallingJob	1543
getCrossRefs	1543
getDebugSessions	1544
getGPDCorners	1544
getGPDCouplingCapacitors	1545
getGPDGroundCapacitors	1546
getGPDLayers	1546
getGPDOObjects	1547
getGPDRefs	1548
getGPDResistors	1548
getInputFiles	1549
getInputFilesTimeStamps	1550
getIntegrations	1550
getJobClasses	1551
getJobClassOptionPrefName	1552
getJobGroups	1552
getJobs	1553
getNetlistInputFiles	1554
getOpenFile	1554
getOutOfDateFiles	1555
getPackages	1556
getPDKConfigs	1556

Contents

getPDKLibs	1557
getPveJobs	1557
getQueueEngines	1558
getSaveFile	1558
getSelectedJobGroups	1559
getSelectedJobs	1560
getSimRuns	1561
getSimulators	1561
getTextViewerActiveTab	1562
getTextViewerTabs	1562
getTools	1563
getWalkupPort	1563
hideJobMonitorColumn	1564
highlightViolation	1565
initializeQueueEngine	1565
initIntegration	1566
isValidRunDir	1567
launchJobs	1567
loadPackage	1568
loadPlotSet	1568
loadPlugin	1569
loadSimRun	1569
openFeatureManifest	1570
openTextViewer	1571
openTextViewerTab	1571
processMarkupTags	1572
queryParasitics	1573
readGPDParasitics	1574
readJob	1575
refreshTextViewer	1576
reloadJobs	1576
removeGPDParasitics	1577
removeJobClassPrefix	1577
reportGPDConfig	1578
reportGPDPProperties	1579
resetGPDConfig	1579
runNetlisterCbk	1580
savePlotSet	1581
saveSimRun	1581

Contents

sendCommand	1582
setGPDCConfig	1583
setSaveRequiredValue	1584
showInitLoad	1584
showInitNew	1585
showJobMonitor	1585
showNetlistAndSimulate	1586
showQueryParasiticsReport	1586
showSetCalibreHighlightColorDialog	1587
signalJob	1588
testPackage	1588
unloadPlugin	1589
updateJob	1590
wait	1591
writeJob	1591
writeParasitics	1592
snps.custom.cmd.xt.plotQueue	1593
removeFirstItem	1593
addNewItem	1594
clear	1595
getPlotObj	1595
hasNoItems	1596
peekFirstItem	1596
snps.custom.cmd.xt.utils	1597
showNetlistChanged	1597
freshPlotPacket	1598
getLineage	1598
getPacketRGB	1599
loadNameMap	1599
removeFromRegistry	1600
snps.custom.cmd.ixt	1600
crossProbe	1601
editPlotSet	1602
plotWaveform	1602
queryGPDParasitic	1603
showGPDPParasiticsOptions	1603

About the Python Reference Manual

The Custom Compiler™ Python Reference Manual covers Custom Compiler Python utilities, commands, and objects by using Python modules and submodules.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Custom Compiler Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the Custom Compiler tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

- Synopsys® Custom Compiler™
- Synopsys® PrimeWave™ Design Environment
- PrimeSim™ Continuum

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
<code>Courier</code>	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction to Custom Compiler Python

The Custom Compiler Python modules and submodules supplement for designing and interacting with the Custom Compiler environment. Using the Custom Compiler Python commands, you can customize the user interfaces or automate design tasks.

This chapter covers the following topics:

- [Python Packages](#)
- [Python Modules](#)
- [Python Reserved Words](#)

Python Packages

Python interfaces are grouped into loadable modules. The module name doubles as a namespace for functions and classes contained within. For example, the `os` module contains the `environ` dictionary, which consists of the environment variables.

A hierarchical grouping of modules is called a *package*. The Custom Compiler tool distributes its modules within the `snps.custom` package. [Table 1](#) lists available modules where to find more information in this manual.

Table 1 Custom Compiler Python Modules

Submodules	For details, see:
<code>snps.custom.utils.macros</code>	Macros
<code>snps.custom.utils.debug</code>	Debug
<code>snps.custom.cmd.cm</code>	Constraint Management
<code>snps.custom.cmd.db</code> , <code>snps.custom.cmd.idb</code>	Database (db and idb)
<code>snps.custom.cmd.de</code> , <code>snps.custom.cmd.ide</code>	Design Editing (de and ide)

Table 1 *Custom Compiler Python Modules (Continued)*

Submodules	For details, see:
<code>snps.custom.cmd.di</code>	Design Insights
<code>snps.custom.cmd.dm</code>	Database Management
<code>snps.custom.cmd.dr</code>	Display Resource
<code>snps.custom.cmd.dspf</code>	Detailed Standard Parasitic Format
<code>snps.custom.cmd.gi</code>	Graphical Infrastructure
<code>snps.custom.cmd.he</code>	Hierarchy Editing
<code>snps.custom.cmd.il</code>	Process Design Kit
<code>snps.custom.cmd.le</code> , <code>snps.custom.cmd.ile</code>	Layout Editing (snps.custom.cmd.le and snps.custom.cmd.ile)
<code>snps.custom.cmd.lp</code> , <code>snps.custom.cmd.ilp</code>	Placement (snps.custom.cmd.lp and snps.custom.cmd.ilp)
<code>snps.custom.cmd.lx</code> , <code>snps.custom.cmd.ilx</code>	Schematic Driven Layout (snps.custom.cmd.lx and snps.custom.cmd.ilx)
<code>snps.custom.cmd.ne</code>	Netlist Editing
<code>snps.custom.cmd.nl</code>	Netlist Tool
<code>snps.custom.cmd.po</code>	Geometry Manipulation
<code>snps.custom.cmd.ps</code>	OptoDesigner
<code>snps.custom.cmd.rc</code>	StarRC
<code>snps.custom.cmd.rt</code> , <code>snps.custom.cmd.irt</code>	Routing (snps.custom.cmd.rt and snps.custom.cmd.irt)
<code>snps.custom.cmd.sdp</code>	SDP
<code>snps.custom.cmd.se</code> , <code>snps.custom.cmd.ise</code>	Schematic Editing (snps.custom.cmd.se and snps.custom.cmd.ise)
<code>snps.custom.cmd.slr</code>	Synopsys Live Recorder
<code>snps.custom.cmd.te</code>	Text Editing
<code>snps.custom.cmd.ue</code>	UDD Editing

Table 1 Custom Compiler Python Modules (Continued)

Submodules	For details, see:
snps.custom.cmd.xt, snps.custom.cmd.ixt	Integrated Tool (snps.custom.cmd.xt and snps.custom.cmd.ixt)

Python Modules

Python modules are distributed in directories that are added to a search path and imported.

To use the interface within a user-defined Python module, it must be imported into the module. The following example imports the db and gi namespaces into the module:

```
from snps.custom.cmd import db, gi
```

To add directories to the Custom Compiler Python search path, set the path as shown in the following example:

```
SYNOPSIS_CUSTOM_PYTHONPATH=path1:path2:...
```

For example, if the directory is as follows:

```
# cd /python
# ls
script1.py
script2.py
```

You must first add this to SYNOPSIS_CUSTOM_PYTHONPATH and then import modules, as shown in the following example:

```
# export SYNOPSIS_CUSTOM_PYTHONPATH=/python
# custom_compiler &
python> import script1
python> script1.func()
```

Paths are not implicitly added to SYNOPSIS_CUSTOM_PYTHONPATH by default.

Adding Python Modules

To add a Python module, for example *my_module/* in */my/module/path*, you must set the path with SYNOPSIS_CUSTOM_PYTHONPATH:, as shown in the following example:

```
$ SYNOPSIS_CUSTOM_PYTHONPATH=/my/module/path:$SYNOPSIS_CUSTOM_PYTHONPATH
custom_compiler
```


Then in Python console, you can either import the module, *my_module* ([Example 1](#)), or add path ([Example 2](#)).

Example 1 *Importing Python Module in Python Console*

```
Python> import my_module
```

Example 2 *Adding Path in Python Console*

```
Python> import sys; sys.path.append('/my/module/path')  
Python> import my_module
```

Python Reserved Words

Python has a set of keywords that are reserved words. These reserved words cannot be used as command names, variable names, option names, argument names, function names, or any other identifiers

Any of the reserved words occurs in any options or arguments of a command, these options or arguments are post-fixed with an underscore (_) in Python. [Table 2](#) lists the reserved keywords, which are used in the options, or arguments, that are post-fixed with underscore in the Custom Compiler Python environment.

Table 2 *Custom Compiler Python Modules*

Reserved Keywords	With Underscore	With Underscore
and	A logical operator	and_
as	To create an alias	as_
assert	For debugging	assert_
break	To break out of a loop	break_
class	To define a class	class_
continue	To continue to the next iteration of a loop	continue_
def	To define a function	def_
del	To delete an object	del_
elif	Used in conditional statements, same as else if	elif_
else	Used in conditional statements	else_
except	Used with exceptions, what to do when an exception occurs	except_
False	Boolean value, result of comparison operations	False_

Table 2 Custom Compiler Python Modules (Continued)

Reserved Keywords	With Underscore	With Underscore
finally	Used with exceptions, a block of code that can be executed no matter if there is an exception or not	finally_
for	To create a for loop	for_
from	To import specific parts of a module	from_
global	To declare a global variable	global_
if	To make a conditional statement	if_
import	To import a module	import_
in	To check if a value is present in a list, tuple, and so on	in_
is	To test if two variables are equal	is_
lambda	To create an anonymous function	lambda_
None	Represents a null value	None_
nonlocal	To declare a non-local variable	nonlocal_
not	A logical operator	not_
or	A logical operator	or_
pass	A null statement, a statement that does nothing	pass_
raise	To raise an exception	raise_
return	To exit a function and return a value	return_
True	Boolean value, result of comparison operations	True_
try	To make a try...except statement	try_
while	To create a while loop	while_
with	Used to simplify exception handling	with_
yield	To return a list of values from a generator	yield_

2

snps.custom.utils

The Custom Compiler Python snps.custom.utils modules includes macros and debug utilities as Python functions.

- [snps.custom.utils.macros](#)
- [Using snps.custom.utils.debug](#)

snps.custom.utils.macros

This chapter lists the modules available in macros utilities. Macros are short functions useful in interactive sessions.

This section covers the following topics:

- [source](#)
- [ed](#)

source

Syntax

```
source(  
  path: str  
)  
Returns: None
```

Source one file.

Arguments

path

File to be sourced.

ed

Syntax

`ed()`

Get current design.

Using snps.custom.utils.debug

Python debugger is also known as `debugpy` that supports remote debugging, developed by Microsoft and used in various Integrated Development Environments (IDEs), such as isual Studio Code. This helps you to debug a Python application running on different machines.

To start debugging and establish a connection with the Python debugger, use following commands in the Python console. You can use the integrated development environment, such as Visual Studio Code, to establish a connection.

```
Python> import snps.custom.utils.debug as dbg
Python> dbg.start()
```

After you use the commands to start debugging, the Custom Compiler tool stops and waits to establish the connection with the debugger. The debug port is automatically allocated and its value is displayed in the terminal, as shown in the following example.

```
~: cdesigner
[snps.custom.utils.debug] Open debugger and connect to port 44953
snps.custom.utils.debug] Waiting debugger to connect...
```

You can also specify the port number manually, as shown in the following example:

```
Python> import snps.custom.utils.debug as dbg
Python> dbg.start (port=5678)
```

The following topics cover the command available with `snps.custom.utils.debug`:

- [enable_breakpoints](#)
- [start](#)

enable_breakpoints

Syntax

```
enable_breakpoints(  
    enabled: bool = True)  
Returns: None
```

Enable/disable breakpoints

Arguments

enabled

True to enable. False to disable.

start

Syntax

```
start(  
    type: Type = Ellipsis,  
    wait: bool = True,  
    *,  
    port=0)  
Returns: None
```

Start debug daemon.

Arguments

type

wait

port

Debug Type. Wait for debugger connection. If true, CC is blocked and output port number to log file instead of console. port number. Auto allocating unused port if 0 or not specified.

3

snps.custom.cmd

The Custom Compiler Python `snps.custom.cmd` modules exposes the commands to Python, as Python functions.

Note that the TCL commands are automatically exported to Python for making complete functionalities available in the Custom Compiler Python environment.

Importing `snps.custom.cmd` and Using Python Code

To import commands in `snps.custom.cmd` and use the python code,

```
Python> import snps.custom.cmd as c
Python> c.db.eval('1+1')
2
$ cdesigner -pycommand 'print("Hello, World!")'
```

To execute the batch scrip to use Python code:

```
$ cdesigner -pycommand 'exec(open("path/to/script.py").read())'
```

[Example 3](#) shows how you can use the `help()` function to display detailed usage information for APIs.

The TCL argument `in` of `gi::findChild` is changed to `in_` in Python. This change was made because `in` is a reserved word in Python. Reserved words in Python are postfixed with `_`. For information, see [Python Reserved Words](#).

Example 3 Using `help()` for `gi::findChild`

```
Python> help(c.gi.findChild)
Help on function findChild in module snps.custom.cmd:
```

```
findChild(none: str, /, *, in_: typing.Union[snps.custom.cmd.giAssistant,
snps.custom.cmd.giContainer, snps.custom.cmd.giDialog,
snps.custom.cmd.giWindow]) -> typing.Union[snps.custom.cmd.dbCore, str]
    findChild(none: str, /, *, in_: Union[giAssistant, giContainer,
giDialog, giWindow]) -> Union[dbCore, str]
```

Summary:

Find and returns a child widget object from a container An absolute path relative to the container may be specified by using a `/` for the first character. Relative paths may also be specified. The command returns an error if more than one match is found within the container.

```
Category:
    Graphical Infrastructure

Arguments:
    none : Widget name to find.
    in_  : Container object to search.

Returns:
    Object or attribute value with specified name or "" if it
could not be found

Licenses:
    None
```

This section covers the following topics:

- [object](#)
- [PyExc_UserInterrupt](#)
- [snps.custom.cmd.cm](#)
- [snps.custom.cmd.db](#)
- [snps.custom.cmd.idb](#)
- [snps.custom.cmd.de](#)
- [snps.custom.cmd.ide](#)
- [snps.custom.cmd.di](#)
- [snps.custom.cmd.dm](#)
- [snps.custom.cmd.dr](#)
- [snps.custom.cmd.dspf](#)
- [snps.custom.cmd.gi](#)
- [snps.custom.cmd.he](#)
- [snps.custom.cmd.il](#)
- [snps.custom.cmd.le](#)
- [snps.custom.cmd.ile](#)
- [snps.custom.cmd.lp](#)
- [snps.custom.cmd.ilp](#)
- [snps.custom.cmd.lx](#)

- [snps.custom.cmd.ilx](#)
- [snps.custom.cmd.ne](#)
- [snps.custom.cmd.nl](#)
- [snps.custom.cmd.po](#)
- [snps.custom.cmd.ps](#)
- [snps.custom.cmd.rc](#)
- [snps.custom.cmd.rt](#)
- [snps.custom.cmd.irt](#)
- [snps.custom.cmd.sdp](#)
- [snps.custom.cmd.se](#)
- [snps.custom.cmd.ise](#)
- [snps.custom.cmd.slr](#)
- [snps.custom.cmd.te](#)
- [snps.custom.cmd.ue](#)
- [snps.custom.cmd.iue](#)
- [snps.custom.cmd.xt](#)
- [snps.custom.cmd.ixt](#)

object

This section covers the following topics:

Class object

Bases: `object`

PyExc_UserInterrupt

This section covers the following topics:

Exception PyExc_UserInterrupt

```
snps.custom.cmd.PyExc_UserInterrupt()
```

snps.custom.cmd.cm

This chapter describes how to use Python submodules to set design constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform constraint management operations.

For details, see:

- [checkIntents](#)
- [createAffinity](#)
- [createAlignment](#)
- [createBlockSize](#)
- [createBus](#)
- [createCapacitance](#)
- [createCluster](#)
- [createCMOS](#)
- [createColorMatching](#)
- [createColumn](#)
- [createDifferentialGroup](#)
- [createEdgeAssign](#)
- [createEnclosingShape](#)
- [createGuardRing](#)
- [createHalo](#)
- [createHierNetSpacing](#)

- [createInstanceMatching](#)
- [createInstMatching](#)
- [createIntent](#)
- [createIntentDef](#)
- [createIntentParamDef](#)
- [createLength](#)
- [createMaxCommonResistance](#)
- [createNetPriority](#)
- [createNetShielding](#)
- [createOrientations](#)
- [createPatternRoute](#)
- [createPlacementGrid](#)
- [createRelative](#)
- [createResistance](#)
- [createResistorBank](#)
- [createRoutingStripes](#)
- [createRows](#)
- [createShieldOverhang](#)
- [createStandardCells](#)
- [createSymmetric](#)
- [createTask](#)
- [createTemplate](#)
- [createWireMatching](#)
- [createWireStriping](#)
- [exportIntents](#)
- [getBacksideMetalLayers](#)
- [getConstraints](#)

- [getFrontsideMetalLayers](#)
- [getIntentDefs](#)
- [getPlacement](#)
- [getRowPlacement](#)
- [getShieldOverhangData](#)
- [getSizePolicy](#)
- [getSpacingData](#)
- [getTasks](#)
- [importIntents](#)
- [showConstraintManager](#)
- [showExportIntents](#)
- [showImportIntents](#)

checkIntents

Syntax

```
checkIntents(  
  intents: dbCollection)  
Returns: None
```

Summary:

Invokes the registered check procedure on the given intents.

Category:

Constraints

Licenses:

None

Parameters

intents – Intent collection to be checked

Returns

None

createAffinity

Syntax

```
createAffinity(  
name: str = None,  
/,  
*,  
for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia] = None,  
insts: list[str] = None,  
in_: oaDesign = None,  
strength: int)  
Returns: cmAaffinity
```

Summary:

Creates an Affinity intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Optional name of new intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- strength – Repulsive (negative value) or attractive force (positive value) to apply between instances

Returns

Affinity intent

createAlignment

Syntax

```
createAlignment(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
         | list[oaVia],  
/,  
*,  
align: cm.createAlignment.Align,  
groupName: str = None,  
lpp: lpp = None)  
Returns: None
```

Summary:

Creates an align constraint for the group of objects given

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- objects – Collection of objects to align
- align – Alignment geometry ({ bottomEdge | center | leftEdge | rightEdge | topEdge })
- groupName – Name for created oaGroup
- lpp – Layer and optional purpose for the shape. Default purpose is drawing.

Returns

None

createBlockSize

Syntax

```
createBlockSize(  
*,  
for_: oaDesign,  
sizePolicy: cmSizePolicy)  
Returns: cmBlockSize
```

Summary:

Creates an Block Size intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- `for_` – Collection of instances
- `sizePolicy` – Size policy for the design/block

Returns

Block Size intent

createBus

Syntax

```
createBus(  
  groupName: str,  
  /,  
  *,  
  for_: list[oaGroup] | list[oaNet],  
  shieldPlacement: cm.createBus.ShieldPlacement = None,  
  validLayers: list[str] = None,  
  layerSpacing: str = None,  
  layerWidth: str = None,  
  gap: float = None,  
  cornerType: cm.createBus.CornerType = None,  
  offGridLayers: list[str] = None,  
  optimizeOrder: bool = None,  
  extendTrunk: bool = None)  
Returns: None
```

Summary:

Creates a bus intent and uses the ordered collection of nets ornet groups to define the nets in the bus and their order

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- `groupName` – Name of bus intent
- `for_` – Collection of nets or net groups
- `shieldPlacement` – Where to add shields if required ({ `default` | `doubleInterleave` | `interleave` | `outside` }) (Default is: 'default')
- `validLayers` – Optional list of layer names to use for the bus trunk
- `layerSpacing` – Optional map of layer name and spacing pairs
- `layerWidth` – Optional map of layer name and width pairs
- `gap` – Optional gap spacing to use between the outermost bus bits and the other shapes in the design (Default is: 0)
- `cornerType` – Optional corner type override to use for the bus ({ `auto` | `cross` | `default` | `river` }) (Default is: 'default')
- `offGridLayers` – Optional list of layers allowed to go off grid for the bus trunk.
- `optimizeOrder` – Optional setting to control whether order reversing is allowed. (Default is: True)
- `extendTrunk` – Optional setting to control extension of bus trunk. (Default is: True)

Returns

None

createCapacitance

Syntax

```
createCapacitance(  
*,  
    minValue: float = None,  
    maxValue: float,  
    nets: list[oaNet] = None)  
Returns: cmCapacitance
```

Summary:

Creates a capacitance intent for the given net(s)

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- `minValue` – Minimum value (Default is: 0)
- `maxValue` – Maximum value
- `nets` – Collection of net

Returns

Capacitance intent

createCluster

Syntax

```
createCluster(  
    name: str = None,  
    /,  
    *,  
    for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
    list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
    | list[oaVia] = None,  
    insts: list[str] = None,  
    in_: oaDesign = None,  
    style: cm.createCluster.Style = None,  
    sizePolicy: cm.SizePolicy = None)  
Returns: cm.Cluster
```

Summary:

Creates a Cluster intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Optional name of new intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- style – Style of cluster placement ({ normal | packed | symmetricX | symmetricXorY | symmetricY } (Default is: 'normal'))
- sizePolicy – Size policy to use

Returns

Cluster intent

createCMOS

Syntax

```
createCMOS(  
name: str = None,  
/  
*,  
for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia] = None,  
insts: list[str] = None,  
in_: oaDesign = None,  
gateAlignment: cm.createCMOS.GateAlignment = None,  
rowSpacings: list[float] = None,  
rowSeparations: list[int] = None,  
leftDummies: int = None,  
rightDummies: int = None,  
sizePolicy: cm.SizePolicy = None,  
defaultPreset: str = None,  
rowPattern: cm.createCMOS.RowPattern = None,  
userPattern: str = None,  
abutmentStyle: cm.createCMOS.AbutmentStyle = None,  
separation: cm.createCMOS.Separation = None,  
sameRowSpacing: float = None,  
boundaryStyle: cm.createCMOS.BoundaryStyle = None,  
sameRowLPP: str = None,  
diffRowLPP: str = None)  
Returns: cmCmos
```

Summary:

Creates a CMOS intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Optional name of new intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- gateAlignment – Alignment of gates for same net ({ center | left | none | right } (Default is: 'center'))
- rowSpacings – List of row spacing values
- rowSeparations – List of row separation values
- leftDummies – Number of dummy devices to be included on the left (Default is: 0)
- rightDummies – Number of dummy devices to be included on the right (Default is: 0)
- sizePolicy – Size policy for the design/block
- defaultPreset – Optional preset cell name to use by default (Default is: "")
- rowPattern – Defines the pattern of row types ({ NP | NPPN | PN | PNNP | custom } (Default is: 'PN'))
- userPattern – Defines a user entered row pattern (Default is: "")
- abutmentStyle – Type of abutment performed on instances in the same row spacing values ({ abut | none | optimal | withContacts } (Default is: 'abut'))
- separation – Type of separation for same row spacing ({ drc | microns | polyPitch } (Default is: 'drc'))
- sameRowSpacing – When defaultSeparation is fixed, defines the explicit spacing between adjacent instances in a row (Default is: 0)
- boundaryStyle – Type of device boundary ({ activeLayer | instance | specify } (Default is: 'activeLayer'))
- sameRowLPP – When boundary is specify defines the same row lpp
- diffRowLPP – When boundary is specify defines the diff row lpp

Returns

CMOS intent

createColorMatching

Syntax

```
createColorMatching(  
    name: str = None,  
    /,  
    *,  
    for_: list[oaGroup] | list[oaNet],  
    color: str)  
Returns: cmColorMatching
```

Summary:

Creates color match intent for the collection of nets or net groups given

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Name of color matching intent
- for_ – Collection of nets or net groups
- color – Name of color mask or color make enumeration

Returns

Color Matching intent

createColumn

Syntax

```
createColumn(  
    name: str = None,  
    /,  
    *,  
    for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
    list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
    | list[oaVia] = None,
```

```
insts: list[str] = None,  
in : oaDesign = None,  
alignment: cm.createColumn.Alignment = None,  
ordering: cm.createColumn.Ordering = None,  
separation: cm.createColumn.Separation = None,  
spacing: float = None)  
Returns: cmColumn
```

Summary:

Creates a Column intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Optional name of new intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- alignment – Alignment of instances in the column ({ center | left | right } (Default is: 'center'))
- ordering – Order of instances in column ({ asIs | autoOrdering } (Default is: 'autoOrdering'))
- separation – Type of separation between instances ({ abut | drc | fixed } (Default is: 'abut'))
- spacing – If separation is fixed, then amount of separation in user units (Default is: 0)

Returns

Column intent

createDifferentialGroup

Syntax

```
createDifferentialGroup(  
groupName: str,  
/,
```

```
*,
nets: list[oaNet],
twistStyle: cm.createDifferentialGroup.TwistStyle = None,
gap: float = None,
twistInterval: float = None,
twistOffset: float = None,
shieldPlacement: cm.createDifferentialGroup.ShieldPlacement = None,
validLayers: list[str] = None,
layerSpacing: str = None,
layerWidth: str = None)
Returns: None
```

Summary:

Creates Differential Group intent for two or three nets given. An optional third net can be specified to produce a differential triplet.

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- groupName – Name for created oaGroup holding given nets
- nets – Two or three nets to group as a differential group
- twistStyle – Defines how the nets in the group will be twisted ({ default | diagonal | none | orthogonal } (Default is: 'default'))
- gap – Optional gap spacing to use between (Default is: 0)
- twistInterval – Optional setting for twist interval, only valid if twist style is not none (Default is: 0)
- twistOffset – Optional setting for twist offset, only valid if twist style is not none (Default is: 0)
- shieldPlacement – Where to add shields if required ({ default | doubleInterleave | interleave | outside } (Default is: 'default'))
- validLayers – Optional list of layer names to use for the trunk
- layerSpacing – Optional map of layer name and spacing pairs
- layerWidth – Optional map of layer name and width pairs

Returns

None

createEdgeAssign

Syntax

```
createEdgeAssign(  
    name: str = None,  
    /,  
    *,  
    for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
    list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
    | list[oaVia] = None,  
    insts: list[str] = None,  
    in_: oaDesign = None)  
Returns: cmEdgeAssign
```

Summary:

Creates an Edge Assign intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Optional name of new intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns

Returns

Edge Assign intent

createEnclosingShape

Syntax

```
createEnclosingShape(  
    name: str = None,
```

```
/,
*,
for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]
| list[oaVia] = None,
insts: list[str] = None,
in_: oaDesign = None,
lpp: lpp,
netName: str = None,
leftEnclose: cm.createEnclosingShape.LeftEnclose = None,
leftMinimum: float = None,
bottomEnclose: cm.createEnclosingShape.BottomEnclose = None,
bottomMinimum: float = None,
rightEnclose: cm.createEnclosingShape.RightEnclose = None,
rightMinimum: float = None,
topEnclose: cm.createEnclosingShape.TopEnclose = None,
topMinimum: float = None)
Returns: cmEnclosingShape
```

Summary:

Creates an Enclosing Shape intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Name of intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- lpp – Layer and optional purpose for the shape. Default purpose is drawing.
- netName – Optional net name for shape
- leftEnclose – Defines the enclosure for the left edge ({ intent | layer } (Default is: 'intent')
- leftMinimum – Minimum amount of enclosure for the left edge (Default is: 0)
- bottomEnclose – Defines the enclosure for the bottom edge ({ intent | layer } (Default is: 'intent')

- bottomMinimum – Minimum amount of enclosure for the bottom edge (Default is: 0)
- rightEnclose – Defines the enclosure for the right edge ({ intent | layer } (Default is: 'intent')
- rightMinimum – Minimum amount of enclosure for the right edge (Default is: 0)
- topEnclose – Defines the enclosure for the top edge ({ intent | layer } (Default is: 'intent')
- topMinimum – Minimum amount of enclosure for the top edge (Default is: 0)

Returns

Enclosing Shape intent

createGuardRing

Syntax

```
createGuardRing(  
name: str = None,  
/,  
*,  
for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia] = None,  
insts: list[str] = None,  
in_: oaDesign = None,  
netName: str = None,  
cellView: dmCellView = None,  
style: cm.createGuardRing.Style = None,  
minNotch: float = None,  
cutRows: int = None,  
bodyWidth: float = None,  
individualWrap: bool = None,  
mergeDistance: float = None,  
params: str = None,  
offset: list[float] = None)  
Returns: cmGuardRing
```

Summary:

Creates a Guard Ring intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Optional name of new intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- netName – Net name to assign to rings
- cellView – lib/cell/view names for guardring pcell
- style – Style of guardring ({ bottom | conformal | horizontal | left | rectangular | right | top | vertical }) (Default is: 'rectangular')
- minNotch – Minimum size notch allowed for the body centerline, (Default is: 0)
- cutRows – Number of cut rows to include for MCell guardrings (Default is: 0)
- bodyWidth – Minimum width of body for MCell guardrings (Default is: 0)
- individualWrap – Indicates whether individual guard ring wrap should be applied to each instance separately. (Default is: False)
- mergeDistance – Merge distance (Default is: 0)
- params – Optional list of pairs of parameter names and values in the format of {{<name> <value>}}
- offset – List of 1, 2, or 4 spacing values

Returns

Guard Ring intent

createHalo

Syntax

```
createHalo(  
name: str = None,  
/,  
*,  
for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia] = None,  
insts: list[str] = None,  
in_: oaDesign = None,  
left: float = None,  
right: float = None,  
)
```

```
top: float = None,  
bottom: float = None)  
Returns: cmHalo
```

Summary:

Creates a Halo intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Optional name of new intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- left – Distance from left edge (Default is: 0)
- right – Distance from bottom edge (Default is: 0)
- top – Distance from right edge (Default is: 0)
- bottom – Distance from top edge (Default is: 0)

Returns

Halo intent

createHierNetSpacing

Syntax

```
createHierNetSpacing(  
name: str = None,  
/,  
*,  
for_: oaDesign,  
nets1: str,  
nets2: str,  
spacings: dbCollection)  
Returns: cmHierNetSpacing
```

Summary:

Creates a hierNetSpacing intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Optional name for the intent, if not given the intent will be auto named
- for_ – Option to specify design for new intent
- nets1 – List of 1 or more net names for first set
- nets2 – List of 1 or more net names for second set
- spacings – Collection of cmSpacingData objects which define the spacings between nets of the two sets

Returns

Hier Net Spacing intent

createInstanceMatching

Syntax

```
createInstanceMatching(  
name: str = None,  
/,  
*,  
for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia] = None,  
insts: list[str] = None,  
in_: oaDesign = None,  
style: cm.createInstanceMatching.Style = None,  
groupCount: int = None,  
firstGroup: int = None,  
sizePolicy: cm.SizePolicy = None,  
pattern: str = None,  
rowSpacing: float = None,  
defaultSeparation: cm.createInstanceMatching.DefaultSeparation =  
None,  
spacingType: cm.createInstanceMatching.SpacingType = None,  
gap: float = None,  
)
```

```
defaultPreset: str = None)  
Returns: None
```

Summary:

Creates an Instance Matching intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Name of intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- style – Defines how the nets in the group will be twisted ({ centralSymmetric | commonCentroid | custom | definedPattern | deviceGroup | interdigitate } (Default is: 'interdigitate')
- groupCount – Number of fingers from same instance used for a pattern grouping (Default is: 1)
- firstGroup – Number of fingers from same instance used in first pattern group (Default is: 1)
- sizePolicy – Size policy for the design/block
- pattern – When style is custom, defines the custom pattern
- rowSpacing – Defines explicit spacing to use between rows (Default is: 0)
- defaultSeparation – Defines how adjacent instances in a row are separated ({ abut | abutKeepContacts | drc | fixed | forceAbut | forceAbutKeepContacts } (Default is: 'abut')
- spacingType – Indicates the units of the row spacing values ({ microns | tracks } (Default is: 'microns')
- gap – When defaultSeparation is fixed, defines the explicit spacing between adjacent instances in a row (Default is: 0)
- defaultPreset – Optional preset cell name to use by default (Default is: "")

Returns

None

createInstMatching

Syntax

```
createInstMatching(  
name: str = None,  
/,  
*,  
for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia] = None,  
insts: list[str] = None,  
in_: oaDesign = None,  
style: cm.createInstMatching.Style = None,  
groupCount: int = None,  
firstGroup: int = None,  
rowOrientations: cm.createInstMatching.RowOrientations = None,  
sizePolicy: cmSizePolicy = None,  
pattern: str = None,  
differentRowSpacings: list[float] = None,  
rowSeparations: list[int] = None,  
separation: cm.createInstMatching.Separation = None,  
sameRowSpacing: float = None,  
defaultPreset: str = None,  
chainStyle: cm.createInstMatching.ChainStyle = None,  
startInstance: str = None,  
spacingType: cm.createInstMatching.SpacingType = None,  
abutmentStyle: cm.createInstMatching.AbutmentStyle = None,  
boundaryStyle: cm.createInstMatching.BoundaryStyle = None,  
sameRowLPP: str = None,  
diffRowLPP: str = None)  
Returns: cmInstMatching
```

Summary:

Creates an Instance Matching intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Name of intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- style – Defines how the nets in the group will be twisted ({ balancedPattern | centralSymmetric | commonCentroid | custom | definedPattern | deviceGroup | interdigitate | seriesChain | textPattern } (Default is: 'deviceGroup'))
- groupCount – Number of fingers from same instance used for a pattern grouping (Default is: 1)
- firstGroup – Number of fingers from same instance used in first pattern group (Default is: 1)
- rowOrientations – Defines default row orientations ({ evenFlipped | flipped | none | oddFlipped } (Default is: 'none'))
- sizePolicy – Size policy for the design/block
- pattern – When style is custom, defines the custom pattern
- differentRowSpacings – List of row spacing values
- rowSeparations – List of row separation values
- separation – Type of separation for same row spacing ({ abut | abutKeepContacts | drc | fixed | microns | polyPitch } (Default is: 'drc'))
- sameRowSpacing – When defaultSeparation is fixed, defines the explicit spacing between adjacent instances in a row (Default is: 0)
- defaultPreset – Optional preset cell name to use by default (Default is: "")
- chainStyle – Chaining style ({ lowerLeftHorizontalSnake | lowerLeftHorizontalZigzag | lowerLeftVerticalSnake | lowerLeftVerticalZigzag | lowerRightHorizontalSnake | lowerRightHorizontalZigzag | lowerRightVerticalSnake | lowerRightVerticalZigzag | none | upperLeftHorizontalSnake | upperLeftHorizontalZigzag | upperLeftVerticalSnake | upperLeftVerticalZigzag | upperRightHorizontalSnake | upperRightHorizontalZigzag | upperRightVerticalSnake | upperRightVerticalZigzag } (Default is: 'none'))
- startInstance – Optional start instance (Default is: 'A')
- spacingType – Indicates the units of the row spacing values ({ microns | tracks } (Default is: 'microns'))

- `abutmentStyle` – Type of abutment performed on instances in the same row spacing values ({ `abut` | `none` | `optimal` | `withContacts` }) (Default is: `'abut'`)
- `boundaryStyle` – Type of device boundary ({ `activeLayer` | `instance` | `specify` }) (Default is: `'activeLayer'`)
- `sameRowLPP` – When boundary is `specify` defines the same row lpp
- `diffRowLPP` – When boundary is `specify` defines the diff row lpp

Returns

Instance Matching intent

createIntent

Syntax

```
createIntent(  
name: str = None,  
/,  
*,  
defName: str,  
for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia] = None,  
insts: list[str] = None,  
in_: oaDesign = None,  
nets: list[oaNet] = None,  
groups: list[str] = None,  
params: list[str] = None)  
Returns: cmCustom
```

Summary:

Creates an intent according to the given definition and objects

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- `name` – Name for new intent
- `defName` – Definition name of custom intent
- `for_` – Collection of instances

- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- nets – One or more nets if intent is net based
- groups – Tcl list of group names representing custom intent group name
- params – Tcl list of paramName/paramValue pairs, for example: { {name1 val1 }
{ name2 val2 } }

Returns

Newly created User Defined Intent

createIntentDef

Syntax

```
createIntentDef(  
name: str,  
/,  
*,  
groupDef: oaGroupDef = None,  
category: cm.createIntentDef.Category = None,  
title: str = None,  
icon: str = None,  
validateProc: str = None,  
checkProc: str = None,  
menu: str = None,  
memberType: list[str],  
minMembersCount: int = None,  
maxMembersCount: int = None,  
allowSharedMember: bool = None,  
visibleInMenu: bool = None,  
paramDefs: list[cm.IntentParamDef] = None)  
Returns: cm.IntentDef
```

Summary:

Creates a constraint definition object for use in a session. The name of the intent constraint definition must be unique in the session.

Category:

Constraints

Licenses:

None

Parameters

- name – Name of new intent definition, should be unique in the system
- groupDef – Group definition object
- category – Category of the intent definition ({ placement | routing })
- title – Title of the intent definition, used for display purposes in constraint editor
- icon – Icon of the intent definition; icon should be pre-loaded in system
- validateProc – Tcl procedure to validate if the intent is in correct state, returning “true” from proc indicates successful checking
- checkProc – Tcl procedure to check if the intent is satisfied in design
- menu – Menu entry name of the intent definition in constraint editor
- memberType – Tcl list of allowed intent member types, combination are possible
- minMembersCount – Minimum members count of the intent definition
- maxMembersCount – Maximum members count of the intent definition
- allowSharedMember – Indicates if intents can share the same object as member item (Default is: False)
- visibleInMenu – Indicates if intent name will be visible in Constraint Editor creation menu (Default is: True)
- paramDefs – Parameter definitions list of the intent definition

Returns

Newly created Intent Definition Object

createIntentParamDef

Syntax

```
createIntentParamDef(  
name: str,  
/,  
*,  
title: str = None,  
defaultValue: str,  
type: cm.createIntentParamDef.Type,  
validValues: list[str] = None,  
visible: bool = None)  
Returns: cmIntentParamDef
```

Summary:

Creates an Intent Parameter Definition

Category:

Constraints

Licenses:

None

Parameters

- name – Name of intent parameter definition
- title – Title of intent parameter definition
- defaultValue – Default Value of parameter
- type – Name of the value type ({ bool | enum | float | integer | multiEnum | multiString | string | userUnit })
- validValues – List of string values, parameter will be exposed as enumeration
- visible – Indicates if parameter is shown or not on Constraint Editor (Default is: True)

Returns

Intent Parameter Definition

createLength

Syntax

```
createLength(  
  groupName: str = None,  
  /,  
  *,  
  first: oaInstTerm | oaPin | oaPinFig | oaTerm = None,  
  second: oaInstTerm | oaPin | oaPinFig | oaTerm = None,  
  maxValue: float,  
  minValue: float = None,  
  nets: list[oaGroup] | list[oaNet] | list[oaRoute] = None)  
Returns: cmLength
```

Summary:

Creates a length constraint for the collection of nets or between the two connections given

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- groupName – Name of group if needed
- first – First connection point
- second – Second connection point
- maxValue – Maximum value
- minValue – Minimum value (Default is: 0)
- nets – Nets to add to match constraint

Returns

Length intent

createMaxCommonResistance

Syntax

```
createMaxCommonResistance(  
    resistance: float,  
    /,  
    *,  
    of: oaBitTerm | oaInstTerm,  
    exclude: list[str] = None)  
Returns: cmMaxCommonResistance
```

Summary:

Creates a maximum common resistance intent for the given terminal or instance terminal

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- resistance – Resistance value in ohms.
- of – Terminal or instance terminal to constrain
- exclude – List of space separated names of terminals and/or instance terminals

Returns

Max Common Resistance intent

createNetPriority

Syntax

```
createNetPriority(  
    *,  
    for_: list[oaGroup] | list[oaNet],  
    priority: int)  
Returns: cmNetPriority
```

Summary:

Creates a netPriority intent for the given net(s) or net group. The netPriority intent assigns the given priority to the set of nets.

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- for_ – Net or net group.
- priority – Priority value.

Returns

Newly created intent

createNetShielding

Syntax

```
createNetShielding(  
    groupName: str = None,  
    /,  
    *,  
    nets: list[oaGroup] | list[oaNet],  
    shieldNet: str = None,  
    secondShieldNet: str = None,  
    style: cm.createNetShielding.Style = None,  
    gap: float = None,  
    width: float = None,
```

```
layerWidths: str = None,  
layerGaps: str = None,  
layerMaxGaps: str = None,  
viaDefs: list[str] = None,  
disabledLayers: list[str] = None,  
encloseVia: cm.createNetShielding.EncloseVia = None,  
enclosePin: cm.createNetShielding.EnclosePin = None,  
minimumSegment: float = None,  
sharing: cm.createNetShielding.Sharing = None,  
defaultMaxGap: float = None,  
groupShield: bool = None,  
topOffset: int = None,  
bottomOffset: int = None)  
Returns: cmNetShielding
```

Summary:

Creates net shield intent for the group of nets given

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- groupName – Name of the oaGroup if needed.
- nets – Nets or net groups to add shield constraint, nets are only supported on OA designs
- shieldNet – Name of net to connect to the shield
- secondShieldNet – Optional second shield net
- style – Shielding style that must be used ({ above | aboveCenterCoaxial | aboveSolidCoaxial | aboveSplitCoaxial | below | belowCenterCoaxial | belowSolidCoaxial | belowSplitCoaxial | centerCoaxial | coaxial | offsetAbove | offsetAboveCenterCoaxial | offsetAboveSolidCoaxial | offsetAboveSplitCoaxial | offsetBelow | offsetBelowCenterCoaxial | offsetBelowSolidCoaxial | offsetBelowSplitCoaxial | offsetCenterCoaxial | offsetCoaxial | offsetSplitCoaxial | offsetTandem | parallel | parallelMultiLayer | splitCoaxial | tandem } (Default is: 'parallel')
- gap – Layer independent spacing between route objects and shield in user units
- width – Layer independent width for shielding material in user units

- **layerWidths** – Optional list of layer specific width values in the format of `{{<layerName> <width>}}`
- **layerGaps** – Optional list of layer specific gap values in the format of `{{<layerName> <gap>}}`
- **layerMaxGaps** – Optional list of layer specific max gap values in the format of `{{<layerName> <maxGap>}}`
- **viaDefs** – Optional list of viaDef names. If given, only specified viaDefs will be used to connect to the shield, otherwise all viaDefs are valid.
- **disabledLayers** – Optional list of layer names to be disabled from use for shielding
- **encloseVia** – Optional override to indicate if vias are enclosed by the shield (`{ default | false | true }` (Default is: 'default'))
- **enclosePin** – Optional override to indicate if pins are enclosed by the shield (`{ default | false | true }` (Default is: 'default'))
- **minimumSegment** – Optional override defining the minimum segment length to shield
- **sharing** – Shielding style that must be used (`{ default | false | true }` (Default is: 'default'))
- **defaultMaxGap** – Layer independent max spacing between route objects and shield in user units
- **groupShield** – Indicates whether all nets in the shielding group should be shielded with a single set of shield shapes. (Default is: False)
- **topOffset** – Top layer offset used in tandem shield
- **bottomOffset** – Bottom layer offset used in tandem shield

Returns

Net Shielding intent

createOrientations

Syntax

```
createOrientations(  
name: str = None,  
/,  
*,  
for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia] = None,  
insts: list[str] = None,  
in_: oaDesign = None,
```

```
orientations: str)
Returns: cmOrientations
```

Summary:

Creates an Orientations intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Optional name of new intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- orientations – Tcl list of orientations

Returns

Orientations intent

createPatternRoute

Syntax

```
createPatternRoute(
description: str,
/,
*,
net: oaNet | str = None,
for_: oaFigGroup | oaGroup | oaNet)
Returns: cmPatternRoute
```

Summary:

Creates a Pattern Route intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- description – Description of route pattern
- net – Name of net or net object to route (Default is: “)
- for_ – Required if -net is also given, defines the owner of the new intent

Returns

Pattern Route intent

createPlacementGrid

Syntax

```
createPlacementGrid(  
name: str = None,  
/  
*,  
insts: list[str] = None,  
in_: oaDesign,  
gridNames: list[str])  
Returns: cmPlacementGrid
```

Summary:

Creates a Placement Grid decorator intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Name of intent
- insts – Tcl list of instance names
- in_ – Design for new intent, if -insts not given, new intent applies to design
- gridNames – List of track region definition names to use for placement grid

Returns

Placement Grid intent

createRelative

Syntax

```
createRelative(  
    name: str = None,  
    /,  
    *,  
    for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
    list[oaFigGroup] | list[oaInst],  
    aspectRatio: float = None)  
Returns: cmRelative
```

Summary:

Creates an Relative intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Name of intent
- for_ – Collection of instances and/or lePlacementGroups to include in placement intent
- aspectRatio – Aspect Ratio, non-negative value in user unit (Default is: 0)

Returns

Relative intent

createResistance

Syntax

```
createResistance(  
    groupName: str = None,  
    /,  
    *,  
    maxValue: float,  
    minValue: float = None,
```

```
nets: list[oaNet] = None,  
sources: list[str] = None,  
destinations: list[str] = None,  
in_: oaDesign = None,  
sourcesShorted: bool = None,  
destinationsShorted: bool = None)  
Returns: cmResistance
```

Summary:

Creates a resistance constraint for the collection of nets or between the two connections given

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- groupName – Name of group if needed
- maxValue – Maximum value
- minValue – Minimum value (Default is: 0)
- nets – Object(s) to add to match constraint
- sources – Tcl list of instance pins on the same net as -destinations
- destinations – List of space separated names of terminals to target on same net
- in_ – The design
- sourcesShorted – Optional boolean indicating how to treat multiple sources during calculation (Default is: True)
- destinationsShorted – Optional boolean indicating how to treat multiple destinations during calculation (Default is: True)

Returns

Resistance intent

createResistorBank

Syntax

```
createResistorBank(  
name: str = None,  
/,  
*,  
for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia] = None,  
insts: list[str] = None,  
in_: oaDesign = None,  
unitResistance: float,  
unitHeight: float = None,  
unitLength: float = None,  
tolerance: float = None,  
rowSpacings: list[float] = None,  
rowSeparations: list[int] = None,  
defaultSeparation: cm.createResistorBank.DefaultSeparation = None,  
gap: float = None,  
sizePolicy: cmSizePolicy = None,  
defaultPreset: str = None,  
chainStyle: cm.createResistorBank.ChainStyle = None,  
startInstance: str = None,  
boundaryStyle: cm.createResistorBank.BoundaryStyle = None,  
sameRowLPP: str = None,  
diffRowLPP: str = None)  
Returns: cmResistorBank
```

Summary:

Creates a Resistor Bank intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Optional name of new intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- unitResistance – Resistance of unit resistor piece

- unitHeight – (Default is: 0)
- unitLength – (Default is: 0)
- tolerance – (Default is: 0)
- rowSpacings – List of row spacing values
- rowSeparations – List of row separation values
- defaultSeparation – Defines how adjacent instances in a row are separated ({ abut | abutKeepContacts | drc | fixed | forceAbut | forceAbutKeepContacts } (Default is: 'abut'))
- gap – When defaultSeparation is fixed, defines the explicit spacing between adjacent instances in a row (Default is: 0)
- sizePolicy – Size policy for the design/block
- defaultPreset – Optional preset cell name to use by default (Default is: "")
- chainStyle – Chaining style ({ lowerLeftHorizontalSnake | lowerLeftHorizontalZigzag | lowerLeftVerticalSnake | lowerLeftVerticalZigzag | lowerRightHorizontalSnake | lowerRightHorizontalZigzag | lowerRightVerticalSnake | lowerRightVerticalZigzag | none | upperLeftHorizontalSnake | upperLeftHorizontalW | upperLeftHorizontalZigzag | upperLeftVerticalSnake | upperLeftVerticalZigzag | upperRightHorizontalSnake | upperRightHorizontalZigzag | upperRightVerticalSnake | upperRightVerticalZigzag } (Default is: 'none'))
- startInstance – Optional start instance (Default is: 'A')
- boundaryStyle – Type of device boundary ({ activeLayer | instance | specify } (Default is: 'activeLayer'))
- sameRowLPP – When boundary is specify defines the same row lpp
- diffRowLPP – When boundary is specify defines the diff row lpp

Returns

Resistor Bank intent

createRoutingStripes

Syntax

```
createRoutingStripes(  
name: str = None,  
/,  
*,
```

```
for_ : list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia] = None,  
insts: list[str] = None,  
in_ : oaDesign = None,  
locations: str = None,  
order: cm.createRoutingStripes.Order = None)  
Returns: cmRoutingStripes
```

Summary:

Creates a Routing Stripes intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Optional name of new intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- locations – Sides to add routing stripes (Default is: 'top bottom')
- order – Order to use for creating multiple stripes ({ auto | inward | outward } (Default is: 'auto'))

Returns

Routing Stripes intent

createRows

Syntax

```
createRows(  
name: str = None,  
/  
*,  
for_ : list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia] = None,  
insts: list[str] = None,  
in_ : oaDesign = None,
```

```
sizePolicy: cmSizePolicy = None,  
ordering: cm.createRows.Ordering = None,  
fillStyle: cm.createRows.FillStyle = None,  
colAlignment: cm.createRows.ColAlignment = None,  
separation: cm.createRows.Separation = None,  
spacing: float = None,  
rowAlignment: cm.createRows.RowAlignment = None,  
rowSpacings: list[float] = None,  
rowSeparations: list[int] = None)  
Returns: cmRows
```

Summary:

Creates a Rows intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Optional name of new intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- sizePolicy – Size policy for the design/block
- ordering – Order of instances in rows ({ asls | auto } (Default is: 'auto'))
- fillStyle – Defines how the rows are filled when ordering is asls ({ bottomThenTop | bottomToTop | leftThenRight | leftToRight | optimal | rightThenLeft | rightToLeft | topThenBottom | topToBottom } (Default is: 'leftToRight'))
- colAlignment – Alignment of columns when rows is greater than 1 ({ center | left | right } (Default is: 'center'))
- separation – Separation of instances in a row ({ abut | drc | fixed } (Default is: 'abut'))
- spacing – When separation is fixed, the minimum separation distance in user units (Default is: 0)
- rowAlignment – Alignment of instances in a row ({ bottom | center | top } (Default is: 'center'))

- rowSpacings – List of row spacing values
- rowSeparations – List of row separation values

Returns

Rows intent

createShieldOverhang

Syntax

```
createShieldOverhang(  
*,  
  for_: oaDesign | oaGroup | oaNet,  
  shieldOverhangs: dbCollection)  
Returns: cmShieldOverhang
```

Summary:

Creates a shieldOverhang intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- for_ – oaNet, oaGroup or oaDesign object
- shieldOverhangs – Collection of cmShieldOverhangData objects

Returns

Shield Overhang intent

createStandardCells

Syntax

```
createStandardCells(  
name: str = None,  
/,  
*,  
  for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
  list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
  | list[oaVia] = None,
```

```
insts: list[str] = None,
in_: oaDesign = None,
rowSpacing: float = None,
powerMode: cm.createStandardCells.PowerMode = None,
UPFFile: str = None,
powerNetName: str = None,
groundNetName: str = None,
targetDensity: float = None,
rowFlip: cm.createStandardCells.RowFlip = None,
auxCellSetName: str = None,
tapCellDistance: float = None,
tapCellEveryRow: bool = None,
tapCellDefaultLocation: cm.createStandardCells.TapCellDefaultLocation
= None,
sizePolicy: cmSizePolicy = None,
spacingType: cm.createStandardCells.SpacingType = None,
allowYAxisFlip: bool = None)
Returns: None
```

Summary:

Creates a Standard Cells intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Name of intent
- for_ – Collection of instances
- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns
- rowSpacing – Minimum row separation distance in user units (Default is: 0)
- powerMode – Type of power mode ({ UPF | singleSupply }) (Default is: 'singleSupply')
- UPFFile – If powerMode is UPF, specifies the path to the UPF file
- powerNetName – The power net name
- groundNetName – The ground net name
- targetDensity – A value between 0 and 1 defining the cell density (Default is: 0)
- rowFlip – Indicates how rows are flipped ({ all | even | none | odd }) (Default is: 'none')

- `auxCellSetName` – Name of auxillary cell set to reference
- `tapCellDistance` – Tap cell distance (Default is: 0)
- `tapCellEveryRow` – If true, a tap cell is required in every row (Default is: False)
- `tapCellDefaultLocation` – Relative location for tap cell ({ center | left | right }) (Default is: 'center')
- `sizePolicy` – Size policy for the design/block
- `spacingType` – Specifies the type of value given in the Row Spacings parameter ({ microns | tracks }) (Default is: 'microns')
- `allowYAxisFlip` – Control of Y-axis flipping of instances in a row (Default is: True)

Returns

None

createSymmetric

Syntax

```
createSymmetric(  
name: str = None,  
/,  
*,  
for_: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia] = None,  
insts: list[str] = None,  
in_: oaDesign = None)  
Returns: cmSymmetric
```

Summary:

Creates a Symmetric intent

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- `name` – Optional name of new intent
- `for_` – Collection of instances

- insts – Tcl list of instance names
- in_ – Design or cell view to search for patterns

Returns

Symmetric intent

createTask

Syntax

```
createTask(  
name:  str,  
/,  
*,  
title: str,  
intents: dbCollection = None)  
Returns: cmTask
```

Summary:

Creates new task

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Name of the task
- title – User visible title string for the task
- intents – Tcl list of intent definition objects to add to task

Returns

New task object

createTemplate

Syntax

```
createTemplate(  
name:  str = None,  
/,  
*,
```

```
insts: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia],  
template: dbCore,  
viewName: str = None,  
staticClone: bool = None)  
Returns: cmTemplate
```

Summary:

Creates template intent.

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Optional name for the intent, if not given the intent will be auto named
- insts – Collection of instances that match the given template netlist
- template – The template being referenced
- viewName – Optional view name in the template
- staticClone – Optional setting defining whether the template should use the selected view verbatim

Returns

The newly created template.

createWireMatching

Syntax

```
createWireMatching(  
groupName: str,  
/,  
*,  
for_: list[oaGroup] | list[oaInstTerm] | list[oaNet] |  
list[oaRoute],  
relative: bool = None,  
matchType: cm.createWireMatching.MatchType = None,  
tolerance: float)  
Returns: cmWireMatching
```

Summary:

Creates a wire match intent for the collection of objects given

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- groupName – Name for created oaGroup holding given nets.
- for_ – Objects to add to match constraint
- relative – Indicates if tolerance value is relative or absolute. (Default is: False)
- matchType – What characteristic or property should be matched ({ capacitance | length | lengthPerLayer | resistance } (Default is: 'length'))
- tolerance – Percent tolerance for consider when nets match

Returns

Wire Matching intent

createWireStriping

Syntax

```
createWireStriping(  
name: str = None,  
/,  
*,  
for_: list[oaGroup] | list[oaNet],  
maxWidth: str,  
spacing: str = None)  
Returns: dbCollection
```

Summary:

Creates wire striping intent for the collection of nets or net groups

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- name – Name of intent
- for_ – Collection of nets or net groups
- maxWidth – Maximum width allowed for stripes per layer
- spacing – Minimum spacing required between stripes per layer. If not given, foundry minimum spacing is used

Returns

Collection of newly created intents

exportIntents

Syntax

```
exportIntents(  
    filePath: str,  
    /,  
    *,  
    intents: dbCollection = None,  
    design: dmCellView | oaDesign = None,  
    overwrite: bool = None,  
    category: cm.exportIntents.Category = None)  
Returns: None
```

Summary:

Export Intents

Category:

Constraints

Licenses:

None

Parameters

- filePath – Name of file to store intents
- intents – Intent collection to be exported; if given - XML file format will be used to store intents
- design – The cellview or opened design from which to export intents

- **overwrite** – Whether to overwrite existing file (Default is: True)
- **category** – Category type to export intents ({ all | placement }) (Default is: 'all')

Returns

None

getBacksideMetalLayers

Syntax

```
getBacksideMetalLayers(  
*,  
  of: oaDesign | oaNet | oaTech)  
Returns: oaPhysicalLayer
```

Summary:

Gets backside metal layers

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

of – Scope to use to query the backside metal layers.

Returns

Collection of backside metal layer

getConstraints

Syntax

```
getConstraints(  
expression: list[str] = None,  
/,  
*,  
  lpps: list[lpp | str] = None,  
  of: oaDesign | oaShape | oaTech | list[deFigure] |  
  list[deSelectionSet] | list[oaFig] | list[oaFigGroup] |  
  list[oaGroup] | list[oaInst] | list[oaMarker] | list[oaNet] |  
  list[oaRoute] | list[oaShape] | list[oaVia],  
  local: bool = None,
```

```
constraintGroupType: cm.getConstraints.ConstraintGroupType = None,  
match: cm.getConstraints.Match = None)  
Returns: dbCollection
```

Summary:

Returns a collection of cmConstraint objects for the given owner, lpps and constraint names

Category:

Constraints

Licenses:

None

Parameters

- expression – Tcl list of constraint type names
- lpps – List containing lpp or string
- of – Object to specify the scope of interest
- local – Indicates whether constraints just from the -of object should be returned or if all child objects should also be considered. This option is applicable in all cases except for oaTech objects (Default is: True)
- constraintGroupType – This argument further qualifies which constraint group type to consider depending on the object given with -of ({ default | interChild | shielding | taper }) (Default is: 'default')
- match – Defines whether all associated objects must exists in the -of collection (full) or just one (partial) for a constraint to be considered a match and returned ({ full | partial }) (Default is: 'partial')

Returns

Collection of cmConstraint objects

getFrontsideMetalLayers

Syntax

```
getFrontsideMetalLayers (  
*,  
of: oaDesign | oaNet | oaTech)  
Returns: oaPhysicalLayer
```

Summary:

Gets frontside metal layers

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

of – Scope to use to query the frontside metal layers.

Returns

Collection of frontside metal layer

getIntentDefs

Syntax

```
getIntentDefs(  
    expression: str = None,  
    /,  
    *,  
    category: cm.getIntentDefs.Category = None)  
Returns: dbCollection
```

Summary:

Gets all intent definition objects that are registered, either explicitly from Tcl or internally

Category:

Constraints

Licenses:

None

Parameters

- expression – Glob expression to match against intent definition names
- category – Only return intents definition for the given category ({ electrical | placement | process | routing })

Returns

Collection of intent definition objects

getPlacement

Syntax

```
getPlacement(  
    item: str,  
    /,  
    *,  
    orient: cm.getPlacement.Orient = None,  
    fingers: int = None,  
    abutted: bool = None,  
    splitDiffusion: bool = None,  
    conditionalDummy: bool = None,  
    grouping: int = None,  
    gateAlign: list[int] = None,  
    horizontalSpacing: float = None,  
    gateWidth: float = None,  
    verticalAlignment: cm.getPlacement.VerticalAlignment = None,  
    preset: str = None,  
    dummyNets: list[str] = None)  
Returns: cmPlacement
```

Summary:

Returns a cmPlacement data object

Category:

Constraints

Licenses:

None

Parameters

- item – Specifies the item used in this placement slot
- orient – Orientation of the placement; R0, R90, MX, and MY ({ MX | MY | R0 | R90 } (Default is: 'R0'))
- fingers – Number of fingers, if zero, then default number of fingers applies (Default is: 0)
- abutted – Indicates if this placement abut with the next placement (Default is: False)
- splitDiffusion – Indicates that the diffusion is split immediately after this placement slot (Default is: False)
- conditionalDummy – Indicates that the dummy is conditional dummy (Default is: False)
- grouping – Grouping number (Default is: 0)

- `gateAlign` – Two integers representing group number and gate index in the group
- `horizontalSpacing` – Horizontal Spacing (Default is: 3.40282e+38)
- `gateWidth` – Gate Width (Default is: 0)
- `verticalAlignment` – Vertical Alignment ({ bottom | center | none | top }) (Default is: 'none')
- `preset` – Name of preset
- `dummyNets` – List of net names for left, gate, right, and bulk nets. Only valid if the item attribute is an asterisk

Returns

Placement Data

getRowPlacement

Syntax

```
getRowPlacement(  
*,  
    alignment: cm.getRowPlacement.Alignment = None,  
    placements: list[dbCollection],  
    preset: str = None,  
    dummySize: str = None,  
    height: float = None)  
Returns: cmRowPlacement
```

Summary:

Returns a `cmRowPlacement` data object

Category:

Constraints

Licenses:

None

Parameters

- `alignment` – Vertical alignment of instances in the row ({ bottom | center | top }) (Default is: 'center')
- `placements` – Collection of placements for the row
- `preset` – Optional name of default preset to use for the row

- **dummySize** – Indicates the size of dummy devices for the row with values of (unset), 'm', 'M', or an explicit float value in user units
- **height** – Height of the row, a value of zero means unset (Default is: 0)

Returns

Row Placement Data

getShieldOverhangData

Syntax

```
getShieldOverhangData(  
    layer: str,  
    /,  
    *,  
    width: float = None,  
    lowerOverhang: float = None,  
    upperOverhang: float = None)  
Returns: cmShieldOverhangData
```

Summary:

Returns a cmShieldOverhangData data object

Category:

Constraints

Licenses:

None

Parameters

- **layer** – Layer name
- **width** – Width (Default is: 0)
- **lowerOverhang** – Lower layer shield overhang (Default is: 0)
- **upperOverhang** – Upper layer shield overhang (Default is: 0)

Returns

Shield Overhang Data

getSizePolicy

Syntax

```
getSizePolicy(  
sizePolicy: cm.getSizePolicy.SizePolicy,  
/,  
*,  
target: float = None,  
minimum: float = None,  
maximum: float = None)  
Returns: cmSizePolicy
```

Summary:

Returns a cmSizePolicy object with the given attribute values

Category:

Constraints

Licenses:

None

Parameters

- sizePolicy – Size policy ({ aspectRatio | columns | height | none | rowPairs | rows | width })
- target – Target value (Default is: 0)
- minimum – Minimum value (Default is: 0)
- maximum – Maximum value (Default is: 0)

Returns

Size policy

getSpacingData

Syntax

```
getSpacingData(  
*,  
layer1: int | str,  
layer2: int | str = None,  
spacing: float,  
runLengthThreshold: float = None)  
Returns: cmSpacingData
```

Summary:

Returns a cmSpacingData data object

Category:

Constraints

Licenses:

None

Parameters

- layer1 – Layer number or name for layer1
- layer2 – Layer number or name for second layer if different than layer1
- spacing – Spacing value
- runLengthThreshold – Run length threshold value

Returns

Spacing Data

getTasks

Syntax

```
getTasks(  
expression: str = None)  
Returns: dbCollection
```

Summary:

Gets all task objects that are registered

Category:

Constraints

Licenses:

None

Parameters

expression – Glob expression to match against task names

Returns

Collection of task objects

importIntents

Syntax

```
importIntents(  
    filePath: str,  
    /,  
    *,  
    design: dmCellView | oaDesign = None,  
    merge: bool = None,  
    category: cm.importIntents.Category = None)  
Returns: dbCollection
```

Summary:

Import Intents

Category:

Constraints

Licenses:

Backend or Frontend

Parameters

- filePath – Name of file containing intents to import
- design – Design to receive intents
- merge – Merge constraints from file, otherwise replace (Default is: False)
- category – Catigory type to import intents ({ all | placement } (Default is: 'all'))

Returns

Collection of cmConstraint objects

showConstraintManager

Syntax

```
showConstraintManager(  
    *,  
    window: giWindow | int)  
Returns: giWindow
```

Summary:

Creates a Constraint Manager Window

Category:

Constraints

Licenses:

None

Parameters

window – Window or window id to which the UCM will be associated.

Returns

Newly created Constraint Manager Window

showExportIntents

Syntax

```
showExportIntents(  
*,  
    window: giWindow | int)  
Returns: giDialog
```

Summary:

Shows Export Intents Dialog

Category:

Constraints

Licenses:

None

Parameters

window – Window or window id parent window.

Returns

New export intents dialog object

showImportIntents

Syntax

```
showImportIntents(  
*,
```

```
    window: giWindow | int)
Returns: giDialog
```

Summary:

Shows Import Intents Dialog

Category:

Constraints

Licenses:

None

Parameters

window – Window or window id parent window.

Returns

New import intents dialog object

snps.custom.cmd.db

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform database operations.

For details, see:

- [add](#)
- [addAttr](#)
- [addFigGroupMembers](#)
- [addGroupMembers](#)
- [addInit](#)
- [addListener](#)
- [addMarkerObject](#)
- [append](#)
- [attachTech](#)
- [batch](#)

- [checkDRD](#)
- [checkFeature](#)
- [checkForInterrupt](#)
- [checkLicenseTier](#)
- [checkoutFeature](#)
- [checkVersion](#)
- [cleanPcellRepository](#)
- [closeCDF](#)
- [closeDesign](#)
- [copyParams](#)
- [copyPrefs](#)
- [createAssignAssignment](#)
- [createAssignValue](#)
- [createCallback](#)
- [createCircularRuler](#)
- [createCircularRulerDef](#)
- [createCollection](#)
- [createDesign](#)
- [createDeviceMappingFileDialog](#)
- [createEnum](#)
- [createEvaluator](#)
- [createExpressionMap](#)
- [createFigGroup](#)
- [createGenOptionSet](#)
- [createGroup](#)
- [createHistogram](#)
- [createImage](#)

- [createIncludeFileControl](#)
- [createInstCache](#)
- [createInstTerm](#)
- [createInterfaceParam](#)
- [createInterfacePin](#)
- [createInterfaceSet](#)
- [createInterruptHandler](#)
- [createLibListInput](#)
- [createLinearRuler](#)
- [createMarker](#)
- [createNet](#)
- [createNetlistInst](#)
- [createNotifier](#)
- [createObject](#)
- [createPaperSize](#)
- [createParamDef](#)
- [createParamSandbox](#)
- [createParasiticStencil](#)
- [createPcell](#)
- [createPcellDef](#)
- [createPin](#)
- [createPref](#)
- [createPrinter](#)
- [createProgress](#)
- [createProp](#)
- [createScope](#)
- [createScopeType](#)

- [createScratchDesign](#)
- [createStdViaVariant](#)
- [createStreamNameMap](#)
- [createSubCircularRuler](#)
- [createWindowAnnotationSet](#)
- [defineUdvDevice](#)
- [defineVflagMaxDevice](#)
- [defineVflagMaxMinDevice](#)
- [defineVflagMinDevice](#)
- [defineVsyncDevice](#)
- [deleteBuiltInVias](#)
- [deprecateAttr](#)
- [destroy](#)
- [detectNetlistType](#)
- [disableUndo](#)
- [enableTk](#)
- [enableUndo](#)
- [encrypt](#)
- [engToSci](#)
- [eval](#)
- [evalCallback](#)
- [evalEnvironment](#)
- [evalParam](#)
- [exportImage](#)
- [exportMCellFile](#)
- [exportParams](#)
- [exportStream](#)

- [exportTechFile](#)
- [expr](#)
- [findCircuitMatches](#)
- [findConnections](#)
- [findConstraint](#)
- [findContextConnectionDef](#)
- [findContextConnections](#)
- [findGenerator](#)
- [findInterfaceReader](#)
- [flushPcellCache](#)
- [freezeRefs](#)
- [generateAnalogSchematics](#)
- [generateBuiltInVias](#)
- [generateCDL](#)
- [generateCellView](#)
- [generateConstraintPatterns](#)
- [generateDeviceMappingTemplate](#)
- [generateLayout](#)
- [generateRule](#)
- [generateSchematic](#)
- [generateSPICE](#)
- [generateSystemVerilog](#)
- [generateVerilog](#)
- [generateVerilogA](#)
- [generateVerilogAMS](#)
- [generateVHDL](#)
- [getAnnotationSets](#)

- [getAssignmentDefs](#)
- [getAssignments](#)
- [getBuiltinNetlists](#)
- [getCallbacks](#)
- [getCellViewInterface](#)
- [getCircularRulerDefs](#)
- [getComponents](#)
- [getConnectionDefState](#)
- [getContextConnectionDefs](#)
- [getCurrentContext](#)
- [getCurrentEvalText](#)
- [getCurrentParam](#)
- [getCurrentRef](#)
- [getDeviceDisplays](#)
- [getEffectiveNet](#)
- [getEnums](#)
- [getEquivalentNets](#)
- [getEvaluators](#)
- [getExportOasisOptions](#)
- [getExportStreamOptions](#)
- [getFeatures](#)
- [getFigGroupMembers](#)
- [getFigGroups](#)
- [getFlightlineCoordinates](#)
- [getGenOptionSets](#)
- [getGroupMembers](#)
- [getGroups](#)

- [getInstHeaders](#)
- [getInsts](#)
- [getInstTerms](#)
- [getLayers](#)
- [getLicenseTier](#)
- [getLPPHeaders](#)
- [getMarkers](#)
- [getNetlistInfo](#)
- [getNets](#)
- [getNotifiers](#)
- [getOpenDesigns](#)
- [getPaperSizes](#)
- [getParamDefs](#)
- [getParamFilter](#)
- [getParams](#)
- [getParamValue](#)
- [getPcellCacheLocation](#)
- [getPcellCacheSize](#)
- [getPcellDefs](#)
- [getPcells](#)
- [getPins](#)
- [getPrefs](#)
- [getPrefValue](#)
- [getPrinters](#)
- [getProcessInfo](#)
- [getProps](#)
- [getRegisteredNetlistInfo](#)

- [getRulers](#)
- [getScopes](#)
- [getScopeTypes](#)
- [getShapes](#)
- [getTableValue](#)
- [getTerms](#)
- [getType](#)
- [getVDRCDDeviceList](#)
- [getViaDefs](#)
- [getViaHeaders](#)
- [getVias](#)
- [getVirtualInstTerms](#)
- [getWindowAnnotationSets](#)
- [hideProps](#)
- [importEDIF](#)
- [importMCellFile](#)
- [importSpice](#)
- [importTechFile](#)
- [importText](#)
- [includes](#)
- [init](#)
- [instCacheTriggerParamCallbacks](#)
- [interrupt](#)
- [isEncrypted](#)
- [isObject](#)
- [isProhibitedName](#)
- [listCallbackTypes](#)

- [loadDeviceDisplays](#)
- [loadGenOptionSets](#)
- [loadMCells](#)
- [loadModelFile](#)
- [loadPrefs](#)
- [loadStreamNameMap](#)
- [localizePrefs](#)
- [mapStreamName](#)
- [match](#)
- [notify](#)
- [openDesign](#)
- [openDesignHierarchy](#)
- [openParamDefEditor](#)
- [openScope](#)
- [pdaExplainCell](#)
- [pdaShowCell](#)
- [print](#)
- [purgeCDF](#)
- [purgePrefs](#)
- [query](#)
- [queryFeature](#)
- [raiseLicenseTier](#)
- [refreshICCCache](#)
- [refreshMCells](#)
- [registerCallbackRegsub](#)
- [registerGenerator](#)
- [registerInterfaceReader](#)

- [registerNetlistInfo](#)
- [registerParamEvalFunction](#)
- [relativePath](#)
- [releaseFeature](#)
- [remove](#)
- [removeFigGroupMembers](#)
- [removeMarkerObject](#)
- [renderTemplatesParasitics](#)
- [reopenDesign](#)
- [reportCallbacks](#)
- [reset](#)
- [resetVDRCDefinedDevices](#)
- [resolve](#)
- [resolvePath](#)
- [revertParam](#)
- [revertPrefs](#)
- [saveCDF](#)
- [saveDesign](#)
- [saveDesignAs](#)
- [saveDeviceDisplays](#)
- [saveGenOptionSets](#)
- [savePrefs](#)
- [sciToEng](#)
- [setAttr](#)
- [setDeviceDisplay](#)
- [setInstName](#)
- [setLicenseTier](#)

- [setMaster](#)
- [setNetlistInfo](#)
- [setParamFilter](#)
- [setParamValue](#)
- [setPrefValue](#)
- [setProgress](#)
- [setViaDef](#)
- [showAbstractGeneratorOptions](#)
- [showAdvancedPrintOptions](#)
- [showCellViewFromCellView](#)
- [showCellViewFromPinList](#)
- [showExportDef](#)
- [showExportImage](#)
- [showExportLef](#)
- [showExportMCellFile](#)
- [showExportOasis](#)
- [showExportStream](#)
- [showFindReplace](#)
- [showGenerateAbstract](#)
- [showGenerateLibertyFile](#)
- [showGenerateSchematic](#)
- [showGenericPrint](#)
- [showImportDef](#)
- [showImportLef](#)
- [showImportMCellFile](#)
- [showImportOasis](#)
- [showImportSimulationDataDialog](#)

- [showImportStream](#)
- [showImportText](#)
- [showImportVerilog](#)
- [showLoadPreferences](#)
- [showManageTechnology](#)
- [showPrint](#)
- [showPrintQueueStatus](#)
- [showPurgePreferences](#)
- [showRefreshMCells](#)
- [showSavePreferences](#)
- [showUnitDisplayScale](#)
- [snapPoints](#)
- [sort](#)
- [source](#)
- [sourceEnc](#)
- [startStatistics](#)
- [stopStatistics](#)
- [transform](#)
- [triggerParamCallback](#)
- [unregisterGenerator](#)
- [unregisterInterfaceReader](#)
- [updateCDFParams](#)
- [vdrcMgrAddNet](#)
- [vdrcMgrAddNetPair](#)
- [vdrcMgrAnnotateNet](#)
- [vdrcMgrAnnotateNetPair](#)
- [vdrcMgrCheckHierarchyMismatch](#)

- [vdrcMgrCheckSymbolPlacement](#)
- [vdrcMgrClearAllData](#)
- [vdrcMgrDeleteNet](#)
- [vdrcMgrDeleteNetPair](#)
- [vdrcMgrImportLayout](#)
- [vdrcMgrImportSchematic](#)
- [vdrcMgrRefresh](#)
- [viaDefExists](#)
- [viewExists](#)
- [vwait](#)

add

Syntax

```
add(  
  collection: dbIterator,  
  /,  
  *,  
  to: dbIterator)  
Returns: dbIterator
```

Summary:

Adds an object to a dbCollection.

Category:

Foundation

Licenses:

None

Parameters

- `collection` – An iterator over the dbCollection.
- `to` – A dbCollection to which to add elements.

Returns

A dbCollection on which operated.

addAttr

Syntax

```
addAttr(  
  name: dbVarAttrName,  
  /,  
  *,  
  of: dbCollection | dbIterator = None,  
  valueType: db.addAttr.ValueType = None,  
  value: variant,  
  type: db.addAttr.Type = None)  
Returns: None
```

Summary:

Adds the specified attribute on the given object.

Category:

Foundation

Licenses:

None

Parameters

- name – The attribute name or hierarchical sequence of names separated by periods to add.
- of – The objects on which the attribute is added.
- valueType – The type of the attribute value to create. Only used for prop attribute type. ({ boolean | double | float | hierarchical | integer | list | string } (Default is: 'string'))
- value – The attribute value to add.
- type – Forces only attributes of the specified type to be added. ({ all | appDef | intrinsic | param | prop } (Default is: 'all'))

Returns

None

addFigGroupMembers

Syntax

```
addFigGroupMembers(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
         | list[oaVia],  
/,  
*,  
group: deFigure | oaFigGroup | str,  
design: oaDesign = None)  
Returns: None
```

Summary:

Creates a new FigGroupMem object making the Figure a member of the FigGroup.

Category:

Database

Licenses:

None

Parameters

- objects – The figure to add.
- group – The FigGroup in which the figure should be added.
- design – The Design in which the group belongs.

Returns

None

addGroupMembers

Syntax

```
addGroupMembers(  
*,  
group: oaGroup,  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig]  
         | list[oaFigGroup] | list[oaGroup] | list[oaInst] |  
         list[oaInstHeader] | list[oaInstTerm] | list[oaLPPHeader] |  
         list[oaMarker] | list[oaNet] | list[oaPin] | list[oaProp] |  
         list[oaShape] | list[oaTerm] | list[oaVia] | list[oaViaHeader],  
isLeader: bool = None)  
Returns: None
```

Summary:

Creates group members for the specified objects in the specified group.

Category:

Layout Editing

Licenses:

None

Parameters

- `group` – The group to add object to.
- `objects` – The object to add to group. If multiple objects are passed, only the first one is created.
- `isLeader` – Specify this group member as the leader of the group. (Default is: False)

Returns

None

addInit

Syntax

```
addInit(  
*,  
    type: str,  
    id: str,  
    eval: script)  
Returns: None
```

Summary:

Evaluate a TCL expression when specific object is requested.

Category:

Foundation

Licenses:

None

Parameters

- type – Object type.
- id – Object identifier, frequently its name.
- eval – TCL expression to evaluate when the object is requested.

Returns

None

addListener

Syntax

```
addListener(  
proc: list[variant],  
/,  
*,  
to: dbNotifier)  
Returns: dbListener
```

Summary:

Adds a listener to a notifier.

Category:

Foundation

Licenses:

None

Parameters

- proc – Procedure to invoke
- to – Notifier to listen to.

Returns

Newly created listener

addMarkerObject

Syntax

```
addMarkerObject(  
objects: oaBlockObject | list[deFigure] | list[deSelectionSet] |  
list[oaFig] | list[oaFigGroup] | list[oaInst] | list[oaMarker] |  
list[oaShape] | list[oaVia],  
/,  
*,  
marker: deFigure | oaMarker)  
Returns: None
```

Summary:

Adds the specified object to the list of objects associated with the marker.

Category:

Layout Editing

Licenses:

None

Parameters

- objects – The object(s) to add to marker
- marker – The marker for the object(s)

Returns

None

append

Syntax

```
append(  
collection: dbCollection | dbIterator,  
/,  
*,  
to: dbCollection | dbIterator)  
Returns: dbCollection | dbIterator
```

Summary:

Merges two dbCollections into a single dbCollection.

Category:

Foundation

Licenses:

None

Parameters

- `collection` – An iterator over the `dbCollection`.
- `to` – A `dbCollection` to which to append elements.

Returns

The merged `dbCollection`.

attachTech

Syntax

```
attachTech(  
lib: oaLib | str,  
/,  
*,  
refLib: oaLib = None,  
refLibName: str = None)  
Returns: None
```

Summary:

Set a reference from a library to technology library

Category:

Data Management

Licenses:

None

Parameters

- `lib` – OA library, for which reference should be set.
- `refLib` – OA library, to which reference should be set.
- `refLibName` – Name of OA library, to which reference should be set.

Returns

None

batch

Syntax

```
batch(  
    script: variant,  
    /,  
    **kwargs)  
Returns: None
```

Summary:

Evaluates a Tcl expression in the scope of one design transaction

Category:

Foundation

Licenses:

None

Parameters

script – Tcl expression to evaluate.

Returns

None

checkDRD

Syntax

```
checkDRD(  
    *,  
    rule: str,  
    lib: str | None = None,  
    cell: str | None = None,  
    view: str | None = None,  
    map: str | None = None,  
    objectMap: str | None = None)  
Returns: None
```

Summary:

DRC

Category:

DRD

Licenses:

Backend

Parameters

- rule – Rule File
- lib – Library Name
- cell – Cell Name
- view – View Type
- map – Layer Mapping File
- objectMap – Object Mapping File

Returns

None

checkFeature

Syntax

```
checkFeature(  
name: str,  
/,  
*,  
exists: script = None,  
doesNotExist: script = None)  
Returns: None
```

Summary:

Determines whether a feature is defined on the license server.

Category:

Foundation

Licenses:

None

Parameters

- name – Name of the licensing feature to check.
- exists – TCL expression to evaluate if the specified feature exists.
- doesNotExist – TCL expression to evaluate if the specified feature does not exist.

Returns

None

checkForInterrupt

Syntax

```
checkForInterrupt(  
    id: str = None,  
    /,  
    *,  
    handler: dbInterruptHandler | dbProgress)  
Returns: None
```

Summary:

Breaks the current loop if an interrupt (Ctrl-c) was detected while evaluating a script with `db::eval`.

Category:

Foundation

Licenses:

None

Parameters

- `id` – Optional loop id for replaying the interrupt. (Default is: “)
- `handler` – Interrupt handler object.

Returns

None

checkLicenseTier

Syntax

```
checkLicenseTier(  
    *,  
    atLeast: db.checkLicenseTier.AtLeast)  
Returns: bool
```

Summary:

Check that the current tier is satisfied.

Category:

Foundation

Licenses:

None

Parameters

atLeast – Required license tier. ({ advanced | advancedPlus | apex | backend | basic | elite | frontend | mtm | router | schematic })

Returns

True if the current tier is sufficient, false otherwise.

checkoutFeature

Syntax

```
checkoutFeature(  
    name: str,  
    /,  
    *,  
    implicit: bool | None = None)  
Returns: None
```

Note that the license name is an internally defined license name which may refer to one or more features on the actual license server. Use `db::getFeatures` to determine which features can be checked out.

Summary:

Checks out an available licensing feature from the license pool. If the feature could not be checked out, the command throws an error. If the feature is already checked out, this command does nothing.

Category:

Foundation

Licenses:

None

Parameters

- name – Name of the licensing feature to check out.
- implicit – If true the implicit check-out count is incremented. (Default is: False)

Returns

None

checkVersion

Syntax

```
checkVersion(  
*,  
    is_: str | None = None,  
    exact: str | None = None,  
    atLeast: str | None = None)  
Returns: bool
```

Summary:

Determines that the program version is at least the specified version.

Category:

Foundation

Licenses:

None

Parameters

- `is_` – Requested product version, allows for additional patches to the specified release.
- `exact` – Requested product version, must be an exact match.
- `atLeast` – Requested product version, must be at least the specified version.

Returns

True if the version is sufficient, false otherwise.

cleanPcellRepository

Syntax

```
cleanPcellRepository(  
object: dmCellView | dmCellViewName | oaDesign,  
/,  
*,  
    saveBackup: bool = None,  
    listDetails: bool = None,  
    checkOnly: bool = None)  
Returns: None
```

Summary:

Clean Pcell Repository.

Category:

Foundation

Licenses:

None

Parameters

- **object** – dmCellView | oaDesign or dmCellViewName that contains a pcell repository to cleanup.
- **saveBackup** – If true, backup the old pcell repository. (Default is: False)
- **listDetails** – If true, information about cleanup is shown for each repository superMaster (Default is: False)
- **checkOnly** – If true, it will do a check only on the repository without changing anything. Nothing will be written or backed up. (Default is: False)

Returns

None

closeCDF

Syntax

```
closeCDF(  
data: dmCell | oaLib)  
Returns: None
```

Summary:

`Close CDF for a cell or library

Category:

Foundation

Licenses:

None

Parameters

data – Specifies the location to store the CDF as oaDMDData

Returns

None

closeDesign

Syntax

```
closeDesign(  
    design: oaDesign,  
    /,  
    *,  
    force: bool = None)  
Returns: None
```

Summary:

Close design

Category:

Database

Licenses:

None

Parameters

- design – OA Design object
- force – Force to close design. (Default is: False)

Returns

None

copyParams

Syntax

```
copyParams(  
    parameters: oaProp | list[dbParam],  
    /,  
    *,  
    to: dmCell | oaLib | oaStdViaDef | oaTech)  
Returns: None
```

Summary:

Merges parameters from the source to the destination

Category:

Foundation

Licenses:

None

Parameters

- `parameters` – The source parameters to copy/merge to the destination.
- `to` – The destination container.

Returns

None

copyPrefs

Syntax

```
copyPrefs(  
  dbpref: dbCollection)  
Returns: dbCollection
```

Summary:

Copies a collection of preferences to a new environment or copy the collection to the active environment.

Category:

Foundation

Licenses:

None

Parameters

`dbpref` – Collection of preferences to copy

Returns

Collection of copied preferences

createAssignAssignment

Syntax

```
createAssignAssignment(  
    name: str,  
    /,  
    *,  
    newName: str,  
    inst: oaInst = None,  
    design: oaDesign = None,  
    defaultNetName: str)  
Returns: dbAssignAssignment
```

Summary:

This command creates an assignAssignment on an instance or a design.

Category:

Design Editing

Licenses:

Basic or Frontend

Parameters

- name – The name of a connection definition, which should be assigned.
- newName – A new name for connection definition
- inst – Instance on which to create the assign assignment.
- design – Design on which to create the assign assignment.
- defaultNetName – The name of default net

Returns

Object describing assignment on the instance or design

createAssignValue

Syntax

```
createAssignValue(  
    name: str,  
    /,  
    *,  
    inst: oaInst = None,  
    design: oaDesign = None,
```

```
netName: str)
Returns: dbAssignValue
```

Summary:

This command creates an assignValue on an instance or a design.

Category:

Design Editing

Licenses:

Basic or Frontend

Parameters

- name – The name of a connection definition, which should be assigned.
- inst – Instance on which to create the assign value.
- design – Design on which to create the assign value.
- netName – The name of default net

Returns

Object describing value assignment on the instance or design

createCallback

Syntax

```
createCallback(
callbackName: str,
/,
*,
callbackType: dbEventName,
procedure: dbExecObject | str,
priority: int = None,
viewType: str = None,
clientData: str = None,
enableInUndo: bool = None)
Returns: dbCallback
```

Summary:

Registers a user callback for any registered callbackType

Category:

Foundation

Licenses:

None

Parameters

- callbackName – callback name
- callbackType – name of the event which will trigger the callback
- procedure – Tcl procedure to call when executing the callback
- priority – priority of execution relative to other callbacks for this event (Default is: 10000)
- viewType – view type name (Default is: “)
- clientData – any additional parameters to pass to the callback (Default is: “)
- enableInUndo – Enable this callback during undo operations for database events (Default is: False)

Returns

New callback object

createCircularRuler

Syntax

```
createCircularRuler(  
point: point,  
/,  
*,  
design: oaDesign,  
rulerDef: dbCircularRulerDef | str = None,  
subRulers: list[dbSubCircularRuler] = None,  
angle: float = None,  
persistent: bool = None)  
Returns: dbCircularRuler
```

Summary:

Creates a circular ruler in the specified layout, symbol or schematic design.

Category:

Design Editing

Licenses:

None

Parameters

- point – The center point for all circles.
- design – Design in which to create the ruler.
- rulerDef – A ruler definition or definition name to define circular ruler instance.
- subRulers – List of sub-rulers which define the circular ruler instance.
- angle – Radial measure scribe direction [0...360]. (Default is: 0)
- persistent – Enable to keep linear ruler in the design. (Default is: True)

Returns

Circular ruler created.

createCircularRulerDef

Syntax

```
createCircularRulerDef(  
    name: str,  
    /,  
    *,  
    subRulers: list[dbSubCircularRuler],  
    angle: float = None)  
Returns: dbCircularRulerDef
```

Summary:

Defines a circular ruler definition.

Category:

Design Editing

Licenses:

None

Parameters

- name – Ruler definition name.
- subRulers – List of sub-rulers associated with this definition object.
- angle – Radial measure scribe direction [0...360]. (Default is: 0)

Returns

Ruler definition created.

createCollection

Syntax

```
createCollection(  
  object: list[attrObject] = None  
Returns: dbCollection | dbIterator
```

Summary:

Converts a Tcl list to a dbCollection.

Category:

Foundation

Licenses:

None

Parameters

object – A list of objects which will be converted.

Returns

A newly created dbCollection from list.

createDesign

Syntax

```
createDesign(  
  cellView: dmCellView | dmCellViewName,  
  /,  
  *,  
  viewType: db.createDesign.ViewType,  
  overwrite: bool = None)  
Returns: oaDesign
```

Summary:

Create design

Category:

Database

Licenses:

None

Parameters

- `cellView` – CellView for which to create design
- `viewType` – View type of design. ({ layout | schematic | symbol })
- `overwrite` – Overwrite existing design. (Default is: False)

Returns

Newly created design

createDeviceMappingFileDialog

Syntax

```
createDeviceMappingFileDialog(  
*,  
    shown: bool = None)  
Returns: giDialog
```

Summary:

Create Device Mapping File Template Generation dialog

Category:

Data Translation

Licenses:

None

Parameters

`shown` – Initial shown state for dialog (Default is: True)

Returns

Device Mapping Template Generation Dialog

createEnum

Syntax

```
createEnum(  
    name: str,  
    /,  
    *,  
    values: list[str] = None,  
    valueDescPairs: list[str] = None,  
    valueProc: list[variant] = None,
```



```
valueDescProc: list[variant] = None,  
caseSensitive: bool = None,  
exclusive: bool = None,  
validator: list[variant] = None)  
Returns: dbEnum
```

Summary:

Create and return a new dbEnum object

Category:

Foundation

Licenses:

None

Parameters

- name – Enum Name
- values – List of valid values.
- valueDescPairs – List of {value description} pairs.
- valueProc – Procedure which returns a list of valid values.
- valueDescProc – Procedure which returns a list of {value description} pairs.
- caseSensitive – Case matching behavior against valid values (Default is: False)
- exclusive – If false, permit values other than valid values (Default is: True)
- validator – Validator procedure for use with -exclusive false and an unrecognized value.

Returns

New dbEnum object

createEvaluator

Syntax

```
createEvaluator(  
name: str,  
/,  
*,  
annotationSet: str,  
type: db.createEvaluator.Type,  
evaluatorProc: str = None,  
paramListProc: str = None,
```

```
paramLabelSetProc: str = None)  
Returns: dbLabelEvaluator
```

Summary:

This command creates evaluator.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- name – Name of evaluator object, which will be shown in Device Label Editor.
- annotationSet – Annotation set name to which the evaluator should belong.
- type – Display type. ({ name | param | term })
- evaluatorProc – Name of evaluator procedure.
- paramListProc – Name of tcl procedure for instance parameters retrieving.
- paramLabelSetProc – Name of tcl procedure used for default parameter label set retrieving.

Returns

Evaluator object used for cds term evaluation.

createExpressionMap

Syntax

```
createExpressionMap()  
Returns: dbExpressionMap
```

Arguments:

Summary:

Create expression map

Category:

Foundation

Licenses:

None

Returns

The expression map

createFigGroup

Syntax

```
createFigGroup(  
    name: str,  
    /,  
    *,  
    design: oaDesign,  
    origin: point = None,  
    orient: db.createFigGroup.Orient = None,  
    members: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
             list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
             | list[oaVia] = None)  
Returns: oaFigGroup
```

Summary:

Creates a new FigGroup in the given design with the given attributes.

Category:

Database

Licenses:

None

Parameters

- name – The name of the FigGroup to be created.
- design – Design in which to create the fig group.
- origin – Location of the origin of fig group.
- orient – Orientation of the fig group. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 } (Default is: 'R0'))
- members – Creates figure group members for the specified figures to this FigGroup.

Returns

Newly created figure group

createGenOptionSet

Syntax

```
createGenOptionSet(  
    name: str,  
    /,  
    *,  
    type: str,  
    value: str = None,  
    loadDefault: bool = None,  
    src: oaDesign = None,  
    fromTech: bool = None)  
Returns: dbGenOptionSet
```

Summary:

Create an generator option set object.

Category:

Data Translation

Licenses:

None

Parameters

- name – The name of the set.
- type – The type of the set.
- value – The contents of the set.
- loadDefault – Load default values for abstractGenerator (Default is: False)
- src – The source design.
- fromTech – Get constraints from technology database, otherwise get them from the design. (Default is: False)

Returns

If successful, newly created dbGenOptionSet object

createGroup

Syntax

```
createGroup(  
    *,
```

```
name: str = None,
design: oaDesign,
isUniqueName: bool = None,
deleteWhen: db.createGroup.DeleteWhen = None,
members: list[deFigure] | list[deSelectionSet] | list[oaFig]
| list[oaFigGroup] | list[oaGroup] | list[oaInst] |
list[oaInstHeader] | list[oaInstTerm] | list[oaLPPHeader] |
list[oaMarker] | list[oaNet] | list[oaPin] | list[oaProp] |
list[oaShape] | list[oaTerm] | list[oaVia] | list[oaViaHeader] =
None,
leader: oaFig | oaFigGroup | oaGroup | oaInst | oaInstHeader |
oaInstTerm | oaLPPHeader | oaMarker | oaNet | oaPin | oaProp
| oaShape | oaTerm | oaVia | oaViaHeader = None)
Returns: oaGroup
```

Summary:

Creates a new Group in the given design with the given attributes

Category:

Database

Licenses:

None

Parameters

- name – The name of the group to be created. (Default is: ‘__CDBA_PARENTCHILD_ONLY_GROUP’)
- design – Design in which to create the group.
- isUniqueName – Specifies whether the group name must be unique. (Default is: False)
- deleteWhen – Sets the event type that determines when, if ever, this group will be deleted. ({ never | onFirst | onLast } (Default is: ‘onLast’)
- members – Creates figure group members for the specified figures to this FigGroup.
- leader – Specify the leader for the group. If multiple objects are passed, the first one is set as the leader.

Returns

Newly created oaGroup

createHistogram

Syntax

```
createHistogram(  
    _none_: str = None,  
    /,  
    *,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    xAxisLabel: str = None,  
    yAxisLabel: str = None,  
    bins: dbCollection = None,  
    precision: int = None)  
Returns: dbHistogram
```

Summary:

Creates a new dbHistogram widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: "")
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `xAxisLabel` – x-axis label (Default is: "")
- `yAxisLabel` – y-axis label (Default is: "")
- `bins` – Collection of bin objects
- `precision` – The display precision (Default is: 5)

Returns

New object.

createImage

Syntax

```
createImage(  
    filePath: str,  
    /,  
    *,  
    design: oaDesign,  
    display: db.createImage.Display = None,  
    attachTo: oaFig = None,  
    origin: point,  
    just: db.createImage.Just = None,  
    size: float)  
Returns: oaRect
```

Summary:

Creates an image object

Category:

Foundation

Licenses:

Frontend

Parameters

- filePath – Image file path.
- design – Design in which to create the image.
- display – Image display mode ({ collapsed | expanded }) (Default is: 'expanded')
- attachTo – Parent figure to attach the image to.
- origin – Location of the origin of image.
- just – Justification of the image with respect to the origin. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight }) (Default is: 'lowerCenter')
- size – Image size

Returns

The new image added to the given design

createIncludeFileControl

Syntax

```
createIncludeFileControl(  
    _none_: str = None,  
    /,  
    *,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    prefName: dbPrefName = None,  
    tooltip: str = None,  
    exceptionsEnabled: bool = None,  
    includeModeEnabled: bool = None)  
Returns: dbIncludeFileControl
```

Summary:

Creates a new include file control widget

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- `tooltip` – Optional widget tooltip. (Default is: “)
- `exceptionsEnabled` – Flag indicating whether exceptions for specific files should be supported (Default is: False)
- `includeModeEnabled` – Flag indicating whether include mode should be supported (Default is: True)

Returns

New object.

createInstCache

Syntax

```
createInstCache()  
Returns: dbInstCache
```

Arguments:

Summary:

Create cache for parameters.

Category:

Foundation

Licenses:

None

Returns

The newly created dbInstCache object

createInstTerm

Syntax

```
createInstTerm(  
    terminal: oaTerm | str,  
    /,  
    *,  
    net: oaNet,  
    inst: oaInst = None,  
    instName: str = None)  
Returns: oaInstTerm
```

Summary:

Create instance terminal

Category:

Layout Editing

Licenses:

None

Parameters

- terminal – Name of terminal or oaTerm inside instance master.
- net – oaNet
- inst – oaInst.
- instName – instName.

Returns

Newly created instTerm

createInterfaceParam

Syntax

```
createInterfaceParam(  
name: str,  
/,  
*,  
type: db.createInterfaceParam.Type,  
defValue: str = None,  
isCDF: bool = None,  
minValue: str = None,  
maxValue: str = None,  
enums: list[str] = None)  
Returns: dbInterfaceParam
```

Summary:

Create an interface param object.

Category:

Data Translation

Licenses:

None

Parameters

- name – The name of the parameter.
- type – The type of value for this parameter. ({ bool | double | doubleRange | enum | float | floatRange | int | intRange | string | time | timeRange })

- defValue – Default value.
- isCDF – Indicator for creation of CDF parameters. (Default is: False)
- minValue – Minimum value for the range type property.
- maxValue – Maximum value for the range type property.
- enums – List of strings for the property of type enum.

Returns

The created interface parameter object

createInterfacePin

Syntax

```
createInterfacePin(  
    string: str,  
    /,  
    *,  
    type: db.createInterfacePin.Type,  
    side: db.createInterfacePin.Side = None,  
    position: int = None,  
    assignmentName: str = None,  
    defaultConnection: str = None,  
    **kwargs)  
Returns: dbInterfacePin
```

Summary:

Create an interface pin object.

Category:

Data Translation

Licenses:

None

Parameters

- string – The name of the pin.
- type – The type of the pin. ({ input | inputOutput | output | switch | tristate })
- side – The side of the core rectangle to place the pin ({ bottom | left | right | top | unknown } (Default is: 'unknown'))
- position – The relative order of the pin on the specified side.

- `assignmentName` – Assignment name for connect definition to be added to pin.
- `defaultConnection` – Name of default net for connection definition.

Returns

The created interface pin object

createInterfaceSet

Syntax

```
createInterfaceSet(  
*,  
    pins: dbCollection = None,  
    parameters: dbCollection = None,  
    cellview: dmCellView = None,  
    cellView: dmCellView | dmCellViewName = None,  
    filePath: str = None)  
Returns: dbInterfaceSet
```

Summary:

Create an interface set object.

Category:

Data Translation

Licenses:

None

Parameters

- `pins` – Collection of interface pins.
- `parameters` – Collection of interface parameters.
- `cellview` – Source cellview.
- `cellView` – Source cellview.
- `filePath` – Source file.

Returns

The created interface set object

createInterruptHandler

Syntax

```
createInterruptHandler(  
    name: str = None,  
    /,  
    *,  
    safe: bool = None)  
Returns: dbInterruptHandler
```

Summary:

Creates a new interrupt handler which processes Ctrl-c events during the evaluation of a script with `db::eval`.

Category:

Foundation

Licenses:

None

Parameters

- name – Optional identifier for replaying the interrupt. (Default is: “")
- safe – Safe mode will not process events when checking for interrupts (Default is: True)

Returns

Interrupt handler object

createLibListInput

Syntax

```
createLibListInput(  
    _none_: str = None,  
    /,  
    *,  
    label: str = None,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    prefName: dbPrefName = None,  
    toolTip: str = None)  
Returns: dbLibraryListInput
```

Summary:

Creates a library list input widget

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `label` – Widget label, shown in the left column. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- `toolTip` – Optional widget toolTip. (Default is: “)

Returns

New object.

createLinearRuler

Syntax

```
createLinearRuler(  
    points: list[point],  
    /,  
    *,  
    design: oaDesign,  
    vectorOnly: bool = None,  
    persistent: bool = None,  
    **kwargs)  
Returns: dbLinearRuler
```

Summary:

Creates a linear ruler in the specified layout, symbol or schematic design.

Category:

Design Editing

Licenses:

None

Parameters

- points – List of points for create linear ruler.
- design – Design in which to create linear ruler.
- vectorOnly – Draw all measure values or only end measure value. (Default is: False)
- persistent – Enable to keep linear ruler in the design. (Default is: True)

Returns

Linear ruler created.

createMarker

Syntax

```
createMarker(  
points: list[point],  
/,  
*,  
design: oaDesign,  
message: str = None,  
shortMessage: str = None,  
tool: str = None,  
severity: db.createMarker.Severity = None,  
deleteWhen: db.createMarker.DeleteWhen = None,  
closed: bool = None)  
Returns: oaMarker
```

Summary:

Creates a new marker in the specified design with the given attributes. There is no check if a marker already exists with the specified attributes.

Category:

Layout Editing

Licenses:

None

Parameters

- points – The list of points that form marker.
- design – Design in which to create the marker.
- message – Sets the message string associated with this marker.
- shortMessage – Sets the short message string associated with this marker.
- tool – Sets the tool string associated with this marker.
- severity – Sets the severity attribute of this marker. ({ acknowledgedWarning | annotation | criticalError | error | fatalError | info | signedOffCriticalError | signedOffError | warning } (Default is: 'error'))
- deleteWhen – Sets the deleteWhen attribute of this marker to the specified deleteWhen value. ({ deleteFirst | deleteLast | deleteModified | deleteNever } (Default is: 'deleteFirst'))
- closed – Used to indicate whether the pointArray associated with this marker is closed or not. (Default is: True)

Returns

Newly created marker

createNet

Syntax

```
createNet(  
name: str,  
/,  
*,  
design: oaDesign,  
global_: bool = None,  
sigType: db.createNet.SigType = None)  
Returns: oaNet
```

Summary:

Create net

Category:

Database

Licenses:

None

Parameters

- name – Net name
- design – Design in which to create the net.
- global_ – Global (Default is: False)
- sigType – Net signal type ({ analog | clock | ground | power | reset | scan | signal | tieHi | tieLo | tieoff }) (Default is: 'signal')

Returns

Newly created oaNet

createNetlistInst

Syntax

```
createNetlistInst(  
    name: str,  
    /,  
    *,  
    deviceType: db.createNetlistInst.DeviceType,  
    connections: list[map_],  
    otherDeviceName: str = None)  
Returns: dbNetlistInst
```

Summary:

Creates dbNetlistInst object.

Category:

Templates Category

Licenses:

None

Parameters

- name – Instance name
- deviceType – Type of device ({ cap | diode | ind | nmos | npn | other | pmos | pnp | res })
- connections – List of net/port pairs
- otherDeviceName – Name of other device

Returns

Netlist instance object.

createNotifier

Syntax

```
createNotifier(  
  name: str,  
  /,  
  *,  
  to: dbObject,  
  args: list[str] = None,  
  description: str)  
Returns: dbNotifier
```

Summary:

Add a notifier to an object created with db::createObject.

Category:

Foundation

Licenses:

None

Parameters

- name – Notifier name
- to – Object to which to attach the new notifier.
- args – TCL list of argument types.
- description – Description of this notifier.

Returns

Newly created notifier

createObject

Syntax

```
createObject()  
Returns: dbObject
```

Arguments:

Summary:

Creates an empty dbObject and returns it

Category:

Foundation

Licenses:

None

Returns

New empty dbObject

createPaperSize

Syntax

```
createPaperSize(  
    name: str,  
    /,  
    *,  
    width: str,  
    height: str,  
    offsetX: str = None,  
    offsetY: str = None,  
    spoolProcOptions: str = None)  
Returns: dbPaperSize
```

Summary:

Create PaperSize Object

Category:

Printing and Images

Licenses:

None

Parameters

- name – The name of the paper size to create
- width – The width of the paper
- height – The height of the paper
- offsetX – The X offset of the origin. (Default is: '0')

- offsetY – The Y offset of the origin. (Default is: '0')
- spoolProcOptions – Options to be passed to the spool proc when this paper size is used (e.g. if using lpr as a spool proc then for Letter paper size this attribute can be “-o media=Letter”). (Default is: “)

Returns

The paper size that was created

createParamDef

Syntax

```
createParamDef(  
name: str,  
/,  
*,  
data: dmCell | oaCellDMDData | oaLib | oaLibDMDData | oaStdViaDef  
| oaTech,  
prompt: str = None,  
defValue: str = None,  
type: db.createParamDef.Type = None,  
callback: str = None,  
units: str = None,  
choices: str = None,  
parseAsNumber: bool = None,  
parseAsPEL: bool = None,  
editable: str = None,  
display: str = None,  
use: str = None,  
storeDefault: bool = None,  
propList: str = None,  
mode: str = None,  
filePrompt: str = None,  
relativePath: bool = None,  
fileType: str = None,  
fileMasks: str = None,  
includeQuotes: bool = None,  
includeParenthesis: bool = None,  
level: db.createParamDef.Level = None)  
Returns: dbParamDef
```

Summary:

Create CDF Parameter at cell or library level

Category:

Foundation

Licenses:

None

Parameters

- name – The name of dbParamDef object.
- data – The storage location of dbParamDef.
- prompt – Prompt in property editor. (Default is: “)
- defValue – Default value of the parameter. (Default is: “)
- type – Parameter Value Type. ({ bool | boolean | button | cyclic | filePath | float | int | integer | lpp | netSet | radio | real | string }) (Default is: ‘string’)
- callback – Callback for the parameter. (Default is: “)
- units – Units for the parameter. (Default is: “)
- choices – Choices for the parameter. (Default is: “)
- parseAsNumber – Is number. (Default is: True)
- parseAsPEL – Is PEL expression. (Default is: False)
- editable – Is parameter editable. (Default is: ‘t’)
- display – Is parameter visible. (Default is: ‘t’)
- use – Is parameter visible. (Default is: ‘t’)
- storeDefault – Is parameter storeDefault. (Default is: False)
- propList – Prop list (Default is: ‘nil’)
- mode – File mode (Default is: ‘open’)
- filePrompt – File prompt (Default is: “)
- relativePath – Relative path (Default is: False)
- fileType – File type (Default is: ‘file’)
- fileMasks – File masks (Default is: “)
- includeQuotes – Include quotes (Default is: True)
- includeParenthesis – Include parenthesis (Default is: False)
- level – The CDF level in which a new parameter will be created ({ base | memory | overlay }) (Default is: ‘overlay’)

Returns

The newly created dbParamDef object.

createParamSandbox

Syntax

```
createParamSandbox()  
Returns: dbParamSandbox
```

Arguments:

Summary:

Create sandbox for parameters.

Category:

Foundation

Licenses:

None

Returns

The newly created dbParamSandbox object

createParasiticStencil

Syntax

```
createParasiticStencil(  
*,  
  fromFile: str,  
  cellView: dmCellView | dmCellViewName,  
  instanceMap: str = None,  
  netMap: str = None,  
  lineage: str = None)  
Returns: str
```

Summary:

Creates parasitic stencil file for template from DSPF file.

Category:

Templates Category

Licenses:

None

Parameters

- `fromFile` – Path of the source DSPF file.
- `cellView` – Template cell view for which the stencil is generated.
- `instanceMap` – Instance names map between DSPF file and target template design. (Default is: "")
- `netMap` – Net names map between DSPF file and target template design. (Default is: "")
- `lineage` – Hierarchy lineage to the module in DSPF file in which the template is located. (Default is: "")

Returns

The path of newly created file.

createPcell

Syntax

```
createPcell(  
    name: str,  
    /,  
    *,  
    def_: dbPcellDef,  
    cellName: str,  
    viewName: str = None,  
    viewType: oaReservedViewType | str = None,  
    overwrite: bool = None)  
Returns: oaDesign
```

Summary:

Creates a pcell with the specified name, using the specified pcell definition.

Category:

Foundation

Licenses:

None

Parameters

- name – The name of the pcell
- def_ – The pcell definition this pcell uses.
- cellName – The cell in which to create the pcell
- viewName – The view in which to create the pcell (Default is: 'layout')
- viewType – The view type of the pcell being defined (Default is: 'maskLayout')
- overwrite – If true, overwrites any existing data, otherwise, the command fails (Default is: False)

Returns

The pcell superMaster design

createPcellDef

Syntax

```
createPcellDef(  
name: str,  
/,  
*,  
libName: str,  
def: list[str],  
plugin: db.createPcellDef.Plugin = None,  
pcellNames: list[str] = None,  
support: list[str] = None,  
overwrite: bool = None)  
Returns: dbPcellDef
```

Summary:

Creates a pcell definition with the specified name in the specified library.

Category:

Foundation

Licenses:

None

Parameters

- name – The name of the pcell
- libName – The library in which to create the pcell definition.

- `def_` – The list of definition file paths to store in the supplied library
- `plugin` – The pcell plug-in associated with this definition (`{ oaCPP | oaTcl }`) (Default is: `'oaTcl'`)
- `pcellNames` – The list of pcell names this definition supports
- `support` – The list of support file paths to store in the supplied library
- `overwrite` – If true, overwrites any existing data, otherwise, the command fails (Default is: False)

Returns

The pcell definition object

createPin

Syntax

```
createPin(  
name: str = None,  
/,  
*,  
term: oaTerm,  
accessDir: int = None)  
Returns: oaInstTerm
```

Summary:

Create Pin

Category:

Database

Licenses:

None

Parameters

- `name` – Pin name (Default is: `'P_0'`)
- `term` – `oaTerm` on which the pin will be created.
- `accessDir` – access direction. (Default is: 0)

Returns

Newly created `instTerm`

createPref

Syntax

```
createPref(  
name: str,  
/,  
*,  
value: str,  
description: str = None,  
validationProc: str = None,  
labels: list[str] = None,  
type: enPrefType = None,  
defaultScope: dbScopeType | str = None,  
autoLocalizeUser: bool = None,  
deprecated: bool = None,  
defunct: bool = None,  
replacement: str = None,  
valueChangeProc: list[variant] = None)  
Returns: dbPref
```

Summary:

Creates a new preference within all environments.

Category:

Foundation

Licenses:

None

Parameters

- name – Preference name.
- value – Preference default value.
- description – Preference description. (Default is: "")
- validationProc – Validation procedure, which will be called to validate preference values as they are set. (Default is: "")
- labels – TCL list of arbitrary string labels for this preference.
- type – Preference's type. ({ bool | float | int | lcv | multilcv | point | string | enum }, default is: string)
- defaultScope – Scope type, where preference will be automatically localized

- `autoLocalizeUser` – If true, this preference will be automatically localized to the user scope when created and on application startup. This option should only be set to false when another interface controls the localizations. (Default is: True)
- `deprecated` – True if the preference is deprecated. (Default is: False)
- `defunct` – True if the preference is defunct. (Default is: False)
- `replacement` – Name of replacement preference to see regarding deprecating or defuncting of this preference (Default is: “)
- `valueChangeProc` – Optional procedure to invoke when this preference is modified

Returns

Newly created preference object at the install scope regardless of any default localizations.

createPrinter

Syntax

```
createPrinter(  
    name: str,  
    /,  
    *,  
    resolution: int,  
    spoolProc: str,  
    queryProc: str,  
    removeProc: str,  
    paperSizes: list[str | list[dbPaperSize]] = None,  
    description: str = None,  
    doubleSided: bool = None,  
    color: bool = None,  
    display: str = None,  
    replace: bool = None,  
    **kwargs)  
Returns: dbPrinter
```

Summary:

Create Printer Object

Category:

Printing and Images

Licenses:

None

Parameters

- name – The name of the printer to create
- resolution – The number of addressable units per inch (aka, DPI or PPI)
- spoolProc – The command used to send the print request to the printer queue
- queryProc – The command used to query the printer's queue
- removeProc – The command used to remove a queued print job
- paperSizes – The collection of available paper sizes of the printer
- description – A description of the printer (Default is: "")
- doubleSided – Indicates if the printer supports double-sided printing (Default is: False)
- color – Indicates if the printer supports color printing (Default is: False)
- display – Indicates the default DRF Display to use with this printer (Default is: "")
- replace – Indicates if an existing printer definition should be replaced with this one (Default is: False)

Returns

The printer that was created

createProgress

Syntax

```
createProgress(  
    name: str,  
    /,  
    *,  
    title: str,  
    label: str = None,  
    maxValue: int = None,  
    shown: bool = None,  
    parent: dbProgress = None,  
    safe: bool = None,  
    hideCancel: bool = None,  
    minDuration: int = None)  
Returns: dbProgress
```

Summary:

Creates a new progress dialog which processes Ctrl-c events.

Category:

Foundation

Licenses:

None

Parameters

- name – Unique identifier, used for logging and replay.
- title – Title for dialogs.
- label – Current suboperation label (Default is: “)
- maxValue – Maximum value, if this task is bounded. (Default is: 0)
- shown – Show or hide the progress dialog. (Default is: True)
- parent – Optional parent progress.
- safe – Safe mode will not process events when checking for interrupts. (Default is: True)
- hideCancel – Hide the cancel button on the progress dialog. (Default is: False)
- minDuration – The time (milliseconds) that must pass before dialog appears. (Default is: 4000)

Returns

New progress object

createProp

Syntax

```
createProp(  
    object: oaObject,  
    /,  
    *,  
    name: str,  
    value: str,  
    type: db.createProp.Type = None,  
    replace: bool = None)  
Returns: oaProp
```

Summary:

Create Prop

Category:

Database

Licenses:

None

Parameters

- object – An object for which property should be created.
- name – Property name
- value – Property value
- type – propEnum. ({ bool | double | float | int | string } (Default is: 'string'))
- replace – Replace already existed property with name (Default is: True)

Returns

Newly created oaProp

createScope

Syntax

```
createScope(  
name: str,  
/  
*,  
type: dbScopeType = None,  
inherits: dbScope = None,  
filePath: str = None)  
Returns: dbScope
```

Summary:

Creates a new named scope in all environments

Category:

Foundation

Licenses:

None

Parameters

- name – Scope name.
- type – Specifies the scope type.
- inherits – A scope from which values are inherited.
- filePath – Default save/load target for this scope. (Default is: “)

Returns

Newly created scope

createScopeType

Syntax

```
createScopeType(  
    _name_: str  
Returns: dbScopeType
```

Summary:

Creates a new scope type from which scopes may be created.

Category:

Foundation

Licenses:

None

Parameters

name – Scope type name (unique, case insensitive).

Returns

Newly created scope type.

createScratchDesign

Syntax

```
createScratchDesign(  
    *,  
    lib: oaLib | str,  
    viewType: db.createScratchDesign.ViewType)  
Returns: oaDesign
```

Summary:

Create scratch design

Category:

Database

Licenses:

None

Parameters

- lib – Lib in which scratch design will be created
- viewType – View type of scratch design. ({ layout | schematic | symbol })

Returns

Newly created scratch design

createStdViaVariant

Syntax

```
createStdViaVariant(  
name: str,  
/,  
*,  
definition: oaStdViaDef | str,  
design: oaDesign,  
layer1Enc: point = None,  
layer2Enc: point = None,  
implant1Enc: point = None,  
implant2Enc: point = None,  
layer1Offset: point = None,  
layer2Offset: point = None,  
cutSpacing: point = None,  
originOffset: point = None,  
cutWidth: float = None,  
cutHeight: float = None,  
cutColumns: int = None,  
cutRows: int = None)  
Returns: oaStdViaVariant
```

Summary:

Create oaStdViaVariant

Category:

Database

Licenses:

None

Parameters

- name – The name of the oaStdViaVariant to be created.
- definition – The definition of the oaStdViaVariant to be created.
- design – Design in which to create the std_via_variant.
- layer1Enc – Specifies the width and height of the layer1 enclosure.
- layer2Enc – Specifies the width and height of the layer2 enclosure.
- implant1Enc – Specifies the width and height of the implant enclosure for the implant1 layer.
- implant2Enc – Specifies the width and height of the implant enclosure for the implant2 layer.
- layer1Offset – Specifies the current x and y offsets for the layer1 enclosure rectangle.
- layer2Offset – Specifies the current x and y offsets for the layer2 enclosure rectangle.
- cutSpacing – Specifies the x and y spacing between the cuts in the via cut matrix for this object.
- originOffset – Specifies the originOffset for this oaViaParam object.
- cutWidth – Specifies the width to be used for each cut in the via cut matrix on the cut layer for this oaViaParam object.
- cutHeight – Specifies the height to be used for each cut in the via cut matrix on the cut layer for this oaViaParam object.
- cutColumns – Specifies the number of columns for the cut matrix on the cutLayer for this oaViaParam object.
- cutRows – Specifies the number of rows for the cut matrix on the cutLayer for this oaViaParam object.

Returns

Newly created oaStdViaVariant

createStreamNameMap

Syntax

```
createStreamNameMap()  
Returns: dbStreamNameMap
```

Arguments:

Summary:

Create a new, empty dbStreamNameMap object.

Category:

Data Translation

Licenses:

None

Returns

New, empty dbStreamNameMap object

createSubCircularRuler

Syntax

```
createSubCircularRuler(  
*,  
    radius: float,  
    packet: str = None,  
    label: str = None)  
Returns: dbSubCircularRuler
```

Summary:

Creates a disconnected sub-ruler object.

Category:

Design Editing

Licenses:

None

Parameters

- radius – Sub-ruler radius.
- packet – Display packet or packet alias name to draw the circle. (Default is: 'ruler')
- label – Sub-ruler label.

Returns

Sub circular ruler created

createWindowAnnotationSet

Syntax

```
createWindowAnnotationSet(  
    string: str,  
    /,  
    *,  
    window: giWindow | int,  
    **kwargs)  
Returns: dbWindowAnnotationSet
```

Summary:

This command associates an annotation set with a window. To remove the annotation set from the window destroy the associated dbWindowAnnotationSet object using db::destroy.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- string – Name of Annotation set object that should be enabled in window.
- window – Window for which annotation should be enabled.

Returns

Created window annotation set object.

defineUdvDevice

Syntax

```
defineUdvDevice (  
*,  
    deviceLCV:  dmCellViewName,  
    targetLib:  oaLib | str,  
    deltaVoltageParam: str)  
Returns: None
```

Summary:

Define user-defined device for udv

Category:

Database

Licenses:

None

Parameters

- deviceLCV – The LCV of the defined udv
- targetLib – The target library which applies to this defined udv
- deltaVoltageParam – The mapping name of delta voltage parameter for this defined udv

Returns

None

defineVflagMaxDevice

Syntax

```
defineVflagMaxDevice (  
*,  
    deviceLCV:  dmCellViewName,  
    targetLib:  oaLib | str,  
    voltageParam: str)  
Returns: None
```

Summary:

Define user-defined device for vflagMax

Category:

Database

Licenses:

None

Parameters

- deviceLCV – The LCV of the defined vflagMax
- targetLib – The target library which applies to this defined vflagMax
- voltageParam – The mapping name of voltage parameter for this defined vflagMax

Returns

None

defineVflagMaxMinDevice

Syntax

```
defineVflagMaxMinDevice (  
*,  
    deviceLCV:  dmCellViewName,  
    targetLib:  oaLib | str,  
    vmaxParam:  str,  
    vminParam:  str)  
Returns: None
```

Summary:

Define user-defined device for vflagMaxMin or vflaghl

Category:

Database

Licenses:

None

Parameters

- deviceLCV – The LCV of the defined vflaghl
- targetLib – The target library which applies to this defined vflaghl
- vmaxParam – The mapping name of vmax parameter for this defined vflaghl
- vminParam – The mapping name of vmin parameter for this defined vflaghl

Returns

None

defineVflagMinDevice

Syntax

```
defineVflagMinDevice(  
    *,  
    deviceLCV: dmCellViewName,  
    targetLib: oaLib | str,  
    voltageParam: str)  
Returns: None
```

Summary:

Define user-defined device for vflagMin

Category:

Database

Licenses:

None

Parameters

- deviceLCV – The LCV of the defined vflagMin
- targetLib – The target library which applies to this defined vflagMin
- voltageParam – The mapping name of voltage parameter for this defined vflagMin

Returns

None

defineVsyncDevice

Syntax

```
defineVsyncDevice(  
    *,  
    deviceLCV: dmCellViewName,  
    targetLib: oaLib | str)  
Returns: None
```

Summary:

Define user-defined device for vsync

Category:

Database

Licenses:

None

Parameters

- deviceLCV – The LCV of the defined vsync
- targetLib – The target library which applies to this defined vsync

Returns

None

deleteBuiltInVias**Syntax**

```
deleteBuiltInVias(  
*,  
    lib: oaLib = None,  
    libName: dmLibName = None)  
Returns: None
```

Summary:

Delete all built-in custom vias

Category:

Data Management

Licenses:

Basic

Parameters

- lib – OA library where the built-in custom via definitions and superMasters will be removed
- libName – Name of OA library where the built-in custom via definitions and superMasters will be removed

Returns

None

deprecateAttr

Syntax

```
deprecateAttr(  
    name: str,  
    /,  
    *,  
    of: str,  
    type: db.deprecateAttr.Type = None,  
    replacement: str = None,  
    defunct: bool = None)  
Returns: None
```

Summary:

Marks an attribute as deprecated or defunct.

Category:

Foundation

Licenses:

None

Parameters

- name – Attribute name to deprecate or make defunct.
- of – The type of object for which to deprecate or make defunct the attribute.
- type – The attribute type to deprecate or make defunct. ({ appDef | intrinsic } (Default is: 'intrinsic'))
- replacement – Name of replacement attribute to see regarding deprecating or defuncting of this attribute. (Default is: "")
- defunct – Mark the attribute defunct. (Default is: False)

Returns

None

destroy

Syntax

```
destroy(  
    tclObj: attrObject)  
Returns: None
```

Summary:

Destroy tcl object

Category:

Foundation

Licenses:

None

Parameters

tclObj – any object to delete

Returns

None

detectNetlistType

Syntax

```
detectNetlistType(  
    *,  
    netlist: dbCollection = None,  
    design: oaDesign = None,  
    data: list[deFigure] | list[oaFig] = None)  
Returns: None
```

Summary:

Command to detect default analog library component from netlist.

Category:

Templates Category

Licenses:

None

Parameters

- netlist – Netlist type detection
- design – Design type detection
- data – Data type detection

Returns

None

disableUndo

Syntax

```
disableUndo(  
*,  
    design: oaDesign = None,  
    config: heConfig = None)  
Returns: None
```

Summary:

Disable undo

Category:

Foundation

Licenses:

None

Parameters

- design – oaDesign option
- config – Config object

Returns

None

enableTk

Syntax

```
enableTk()  
Returns: None
```

Arguments:

Summary:

Enables tk framework.

Category:

Foundation

Licenses:

None

Returns

None

enableUndo

Syntax

```
enableUndo(  
    *,  
    design: oaDesign = None,  
    config: heConfig = None)  
Returns: None
```

Summary:

Enable undo

Category:

Foundation

Licenses:

None

Parameters

- design – oaDesign option
- config – Config object

Returns

None

encrypt

Syntax

```
encrypt(  
    filename: str,  
    /,  
    *,  
    target: str)  
Returns: None
```

Summary:

encrypts a file using the Laker format

Category:

DRD

Licenses:

None

Parameters

- filename – Source file
- target – Destination file

Returns

None

engToSci

Syntax

```
engToSci(  
    input: str,  
    /,  
    *,  
    precision: int | None = None)  
Returns: str
```

Summary:

Convert double/string values to scientific notation

Category:

Foundation

Licenses:

None

Parameters

- input – input number in engineering notation
- precision – Precision used for result (Default is: 6)

Returns

Result of converting input to a string using Scientific notation

eval

Syntax

```
eval(  
  script:  variant,  
  /,  
  *,  
  log:  bool  =  None)  
Returns: None
```

Summary:

Evaluates a Tcl expression with support for block-scoped objects

Category:

Foundation

Licenses:

None

Parameters

- script – Tcl expression to evaluate.
- log – Log the expression. (Default is: False)

Returns

None

evalCallback

Syntax

```
evalCallback(  
  callback: str,  
  /,  
  *,  
  of: oaRef = None,  
  param: str = None,  
  text: oaEvalText = None,  
  context: heContext = None)  
Returns: str
```

Summary:

Evaluate callback syntax

Category:

Foundation

Licenses:

None

Parameters

- callback – String to be evaluated
- of – The current instance, custom via, or standard via.
- param – Name of current parameter (Default is: “”)
- text – oaEvalText object
- context – Hierarchy context

Returns

The evaluated value of the callback string

evalEnvironment

Syntax

```
evalEnvironment(  
  *,  
  ref: oaRef = None,  
  text: oaEvalText = None,  
  context: heContext = None,
```

```
script: str)
Returns: None
```

Summary:

Sets the values for db::getCurrentRef, db::getCurrentEvalText and db::getCurrentContext for the duration of the script.

Category:

Foundation

Licenses:

None

Parameters

- ref – The current instance, custom via, or standard via.
- text – oaEvalText object
- context – Hierarchy context
- script – String to be evaluated

Returns

None

evalParam

Syntax

```
evalParam(
param: dbParam,
/,
*,
context: heContext = None,
evalType: db.evalParam.EvalType = None,
refDesignVars: str = None,
refParentParams: str = None)
Returns: str
```

Summary:

Calculate evaluated value of a parameter

Category:

Foundation

Licenses:

None

Parameters

- param – Parameter to be evaluated
- context – Evaluation context
- evalType – Parameter Evaluation Type ({ debug | full | netlist } (Default is: 'full'))
- refDesignVars – Name of Tcl array for storing design variables
- refParentParams – Name of Tcl array for storing parent references

Returns

The parameters

exportImage

Syntax

```
exportImage(  
context: dmCellView | giWindow,  
/,  
*,  
depth: int = None,  
width: int = None,  
height: int = None,  
file: str = None,  
overwrite: bool = None,  
display: str = None,  
omitBackground: bool = None,  
includeBackground: bool = None,  
format: db.exportImage.Format = None,  
colorMode: db.exportImage.ColorMode = None)  
Returns: oaFile
```

Summary:

Outputs the specified CellView or window contents to the specified format.

Category:

Printing and Images

Licenses:

None

Parameters

- context – The CellView or window to output
- depth – The start level from which the hierarchy should be outputted
- width – Exported image width (Default is: 0)
- height – Exported image height (Default is: 0)
- file – The file to output to
- overwrite – Controls if existing file is overwritten
- display – The display to use when outputting
- omitBackground – Whether to omit the background during outputting
- includeBackground – Whether to include the background during outputting
- format – The format to use when outputting the data ({ BMP | JPEG | PNG | SVG })
- colorMode – Whether to output in color, grayscale or black & white ({ blackAndWhite | color | grayscale })

Returns

The file that was created

exportMCellFile

Syntax

```
exportMCellFile(  
    path: str,  
    /,  
    *,  
    lib: oaLib = None,  
    libName: str = None,  
    cdfFilePath: str = None,  
    overwrite: bool = None)  
Returns: None
```

Summary:

Export MCell definition file.

Category:

Data Management

Licenses:

Basic

Parameters

- `path` – Path including the file name of MCell definition file to export.
- `lib` – OA library, from where MCell superMaster data should be exported
- `libName` – Name of OA library, from where MCell superMaster data should be exported
- `cdfFilePath` – Path including the file name of MCell CDF file to export. (Default is: “”)
- `overwrite` – Whether to overwrite existing MCell definition and CDF file. (Default is: False)

Returns

None

exportParams

Syntax

```
exportParams(  
    *,  
    of: dmCell | oaCellDMDData | oaLib | oaLibDMDData | str,  
    filePath: str,  
    level: db.exportParams.Level = None)  
Returns: None
```

Summary:

Writes CDF information to a text file.

Category:

Foundation

Licenses:

None

Parameters

- of – DMDData (or a cell, library, or library name that contains one) that supplies the parameters to export.
- filePath – Output Tcl file.
- level – The CDF level from which parameters will be exported ({ base | memory | overlay }) (Default is: 'base')

Returns

None

exportStream

Syntax

```
exportStream(  
*,  
gds: str,  
lib: str,  
batchMode: bool = None,  
showResult: bool = None,  
libDefFile: str = None,  
acutePathToPolygon: bool = None,  
autoMapObjects: int = None,  
blockageType: int = None,  
cell: list[str] = None,  
cellMap: str = None,  
cellNamePrefix: str = None,  
cellNameSuffix: str = None,  
charLimit: bool = None,  
colorMapping: db.exportStream.ColorMapping = None,  
convertPinToText: bool = None,  
convertNetFromPinToText: bool = None,  
convertTextBrackets: db.exportStream.ConvertTextBrackets = None,  
dbuPerUU: int = None,  
donutNumSides: int = None,  
dumpDesignTree: bool = None,  
dumpRefLib: bool = None,  
duplicateCell: db.exportStream.DuplicateCell = None,  
ellipseNumSides: int = None,  
expandArrayInsts: bool = None,  
filterAcutePaths: bool = None,  
filterDots: bool = None,  
filterEmptyCells: bool = None,  
filterReentrantPgns: bool = None,  
filterSelfIntersectingPgns: bool = None,  
filterText: bool = None,  
filterUnboundInsts: bool = None,  
filterZeroWidth: bool = None,  

```

```
flattenPcells: bool = None,
flattenVias: bool = None,
fontMap: str = None,
hierDepth: int = None,
ignoreCellsAffix: str = None,
ignoreTopCellsAffix: bool = None,
keepAbuts: bool = None,
keepFullCellName: bool = None,
keepViewName: bool = None,
keepHalfWidthPaths: bool = None,
keepPcells: bool = None,
keepVias: bool = None,
labelDepth: int = None,
labelMap: str = None,
lakerLicense: str = None,
layerMap: list[str] = None,
logFile: str = None,
mapAllText: bool = None,
maxVertices: int = None,
mergeCellviews: str = None,
newMaxVerticesFunction: bool = None,
newMaxVerticesRule: str = None,
noAutoBackupFile: bool = None,
noConvertPath: bool = None,
noExportRefLibCells: bool = None,
noInfo: list[int] = None,
noOutputBlockages: bool = None,
noOutputBoundaries: bool = None,
noOutputTextDisplays: bool = None,
noRoundPathToPolygon: bool = None,
noWarning: list[int] = None,
notExternalizeShapes: bool = None,
outputBBoxInfo: bool = None,
sortLayer: bool = None,
objectMap: str = None,
pathToPolygon: bool = None,
propMap: str = None,
propValueOnly: bool = None,
propertyInst: int = None,
propertyLogic: int = None,
propertyNet: int = None,
propertyWideAlignment: int = None,
propertyVia: int = None,
rectAsBoundary: bool = None,
refLibList: str = None,
refLibs: list[str] = None,
referenceMap: str = None,
roundPathToPolygon: bool = None,
snapToMfgGrid: bool = None,
tclFile: str = None,
tclFunction: list[str] = None,
techLib: str = None,
text: str = None,
```

```
textDepth: int = None,  
textHeight: db.exportStream.TextHeight = None,  
textMap: list[str] = None,  
toLower: bool = None,  
toLowerLabel: bool = None,  
toUpper: bool = None,  
toUpperLabel: bool = None,  
unmappedLayersUseOALayers: bool = None,  
useTechLayerMap: bool = None,  
useTechObjectMap: bool = None,  
ver: int = None,  
view: str = None)  
Returns: bool
```

Summary:

Execute Export Stream

Category:

Data Translation

Licenses:

Basic or Integrator

Parameters

- gds – Output stream file
- lib – Library name
- batchMode – Specifies if exportStream is run with interactive mode (Default is: True)
- showResult – Specifies if display notification when complete in batch mode (Default is: False)
- libDefFile – Use a specific library definitions file
- acutePathToPolygon – Convert acute paths to polygons (Default is: False)
- autoMapObjects – Use layer map entries to write prBoundaries, snapBoundaries, and layer/area Blockages (Default is: 99)
- blockageType – Datatype for output blockages (Default is: 0)
- cell – Input cell name
- cellMap – Cell map file
- cellNamePrefix – Add this prefix to all the structure names in the stream file
- cellNameSuffix – Add this suffix to all the structure names in the stream file

- `charLimit` – Use the 32-character limit on structure names (Default is: `False`)
- `colorMapping` – Specifies layer mapping rule (`{ allVias | full | honor | ignore | lockOnly | strict }`) (Default is: `'honor'`)
- `convertPinToText` – Convert pin's term name to label (Default is: `False`)
- `convertNetFromPinToText` – Convert pin's net name to label (Default is: `False`)
- `convertTextBrackets` – Convert text brackets to angle brackets or square brackets (`{ angle | square }`)
- `dbuPerUU` – dbuPerUU value (Default is: 1000)
- `donutNumSides` – Number of sides for the boundary of donut objects (Default is: 64)
- `dumpDesignTree` – Dump design tree to log file (Default is: `False`)
- `dumpRefLib` – Dump reference library list to log file (Default is: `False`)
- `duplicateCell` – Handling for duplicate cells (`{ skip | withLibName | withNumber }`) (Default is: `'withNumber'`)
- `ellipseNumSides` – Number of sides for the boundary of ellipse objects (Default is: 64)
- `expandArrayInsts` – Expand Arrays to Instances (Default is: `False`)
- `filterAcutePaths` – Filter acute paths (Default is: `False`)
- `filterDots` – Filter dots (Default is: `False`)
- `filterEmptyCells` – Specifies if translator filter empty cells (Default is: `False`)
- `filterReentrantPgns` – Filter reentrant polygons (Default is: `False`)
- `filterSelfIntersectingPgns` – Filter Self-Intersecting polygons (Default is: `False`)
- `filterText` – Filter text (Default is: `False`)
- `filterUnboundInsts` – Specifies if translator filter unbound instances (Default is: `False`)
- `filterZeroWidth` – Filter zero width paths (Default is: `False`)
- `flattenPcells` – Flatten any pcell into their geometry (Default is: `False`)
- `flattenVias` – Flatten standard via into their geometry (Default is: `False`)
- `fontMap` – Font map file
- `hierDepth` – Hierarchical depth to translate to (Default is: 256)
- `ignoreCellsAffix` – Do not add prefix/suffix to these cells
- `ignoreTopCellsAffix` – Do not add prefix/suffix to top cells (Default is: `False`)

- keepAbuts – Specifies if exportStream translator keep abutments (Default is: False)
- keepFullCellName – Use original length of cell submaster(and stdVia) name (Default is: False)
- keepViewName – Specifies if translator keep view name (Default is: False)
- keepHalfWidthPaths – Specifies if exportStream translator keep half width paths (Default is: False)
- keepPcells – Specifies if exportStream translator keep Pcells (Default is: False)
- keepVias – Specifies if exportStream translator keep Vias (Default is: False)
- labelDepth – Hierarchical depth to add labels to (Default is: 1)
- labelMap – Label map file
- lakerLicense – Permit using Laker licenses to access some Custom Compiler features. Possible values are: schematic, layout, both (Default is: “)
- layerMap – Dump the parsing info of layer mapping to log file for debugging
- logFile – Name of log file (Default is: ‘exportStream.log’)
- mapAllText – Map all texts from CDBA to the namespace specified with -text (Default is: False)
- maxVertices – Split polygons and paths with more than this number of vertices. Range 200 - 8190 (Default is: 8190)
- mergeCellviews – Specifies merge cellviews file
- newMaxVerticesFunction – Set use new maxvertices function (Default is: False)
- newMaxVerticesRule – Set new maxvertices rule. Possible values are: SwapXY, noSwap, debug (Default is: ‘SwapXY’)
- noAutoBackupFile – Do not auto backup output GDS file if file exists (Default is: False)
- noConvertPath – Do not convert half width paths to boundaries (Default is: False)
- noExportRefLibCells – Only output cells from the library specified by -lib (Default is: False)
- noInfo – Suppress the given INFO message IDs
- noOutputBlockages – Do not output blockage objects (Default is: False)
- noOutputBoundaries – Do not output boundaries objects (Default is: False)
- noOutputTextDisplays – Do not output textDisplay objects (Default is: False)

- noRoundPathToPolygon – Do not convert round paths to polygons (Default is: False)
- noWarning – Suppress the given WARNING message IDs
- notExternalizeShapes – Do not externalize shapes (Default is: False)
- outputBBoxInfo – Output BBox information for cell in log file (Default is: False)
- sortLayer – Sort layer statistics in log file (Default is: False)
- objectMap – Object map file
- pathToPolygon – Convert paths to polygons (Default is: False)
- propMap – Prop map file
- propValueOnly – Output property values only (Default is: False)
- propertyInst – convert property in (1-127) to instance name
- propertyLogic – convert property in (1-127) to logic information
- propertyNet – convert property in (1-127) to net name
- propertyWideAlignment – convert property in (1-127) to wide alignment information
- propertyVia – convert property in (1-127) to via information
- rectAsBoundary – Output rectangles as boundary records (Default is: False)
- refLibList – RefLibList file
- refLibs – Specifies a list of libraries that contain reference cells
- referenceMap – Output complete mapping of designs to stream structures
- roundPathToPolygon – Convert round paths to polygons (Default is: False)
- snapToMfgGrid – Snap the vertices to the manufacturing grid (Default is: False)
- tclFile – tcl file
- tclFunction – tcl founction
- techLib – Technology library
- text – Text namespace (Default is: 'cdba')
- textDepth – Text depth to translate to (Default is: 256)
- textHeight – Text height mode ({ default | matchOABBox | matchObjectHeight }) (Default is: 'matchOABBox')
- textMap – Quoted list of text map files

- toLower – Convert cell and instance names to lower case (Default is: False)
- toLowerLabel – Convert Label names to lower case (Default is: False)
- toUpper – Convert cell and instance names to upper case (Default is: False)
- toUpperLabel – Convert Label names to upper case (Default is: False)
- unmappedLayersUseOALayers – Use OA layer numbers if not mapped (Default is: False)
- useTechLayerMap – Use the layer map file that is in the source tech library (Default is: False)
- useTechObjectMap – Use the object map file that is in the source tech library (Default is: False)
- ver – Stream version to write out (Default is: 3)
- view – Input view name

Returns

Execute exportStream

exportTechFile

Syntax

```
exportTechFile(  
    path: str,  
    /,  
    *,  
    lib: oaLib = None,  
    libName: str = None,  
    overwrite: bool = None,  
    local: bool = None,  
    includeEmptySections: bool = None,  
    sections: str = None,  
    **kwargs)  
Returns: None
```

Summary:

Export technology file

Category:

Data Management

Licenses:

None

Parameters

- path – Path including the file name of technology file to export.
- lib – OA library, from where technology data should be exported.
- libName – Name of OA library, from where technology data should be exported.
- overwrite – Whether to overwrite or not existing technology file. (Default is: False)
- local – Whether to export local or global. (Default is: False)
- includeEmptySections – Whether to export include empty sections. (Default is: True)
- sections – Sections or subsections to be exported. (Default is: 'all')

Returns

None

expr

Syntax

```
expr(  
string: str,  
/,  
*,  
of: oaCustomVia | oaInst = None,  
evalType: db.expr.EvalType = None,  
context: heContext = None,  
map: db.ExpressionMap = None,  
refLineageParams: str = None,  
refParentParams: str = None,  
refDesignVars: str = None,  
designVars: list[map_] = None)  
Returns: float
```

Summary:

Evaluates an arithmetic expression.

Category:

Foundation

Licenses:

None

Parameters

- `string` – The input arithmetic expression.
- `of` – Object to be queried.
- `evalType` – Parameter Evaluation Type (`{ debug | full | netlist | none }`) (Default is: `'none'`)
- `context` – Context for full evaluation
- `map` – Expression Map for evaluation
- `refLineageParams` – Variable storage reference for lineage parameters
- `refParentParams` – Variable storage reference for parent parameters
- `refDesignVars` – Variable storage reference for design variables
- `designVars` – set of design variable/values

Returns

The resultant value of the input expression.

findCircuitMatches

Syntax

```
findCircuitMatches(  
  none: dbCollection,  
  /,  
  *,  
  of: oaDesign = None,  
  in_: dbCollection = None,  
  full: bool = None,  
  postFilter: oaDesign = None)  
Returns: dbCollection
```

Summary:

Returns a collection of `dbNetlistMatch` objects for each unique circuit structure that matches the given template circuit structure.

Category:

Templates Category

Licenses:

None

Parameters

- none – Collection of ordered templates instances representing a single circuit structure.
- of – Design to search.
- in_ – Circuit structure to search.
- full – Indicates whether the source and target circuits structures must fully match (true) or partly (false). (Default is: False)
- postFilter – if exist oaDesign then post process all possible matches and separate into two setsfirst set contains best matches(non exist in second set), second set contain all other matches(non exist in first set) otherwise use default way (Default is: False)

Returns

Collection of dbNetlistMatch objects.

findConnections

Syntax

```
findConnections(  
  none: heContext | oaDesign)  
Returns: dbConnections
```

Summary:

This command collects all connect definitions in the hierarchy defined by the context or design.

Category:

Design Editing

Licenses:

None

Parameters

none – Top level context or design to use to collect all defined connection information.

Returns

dbConnections object describing all defined connects in the hierarchy

findConstraint

Syntax

```
findConstraint(  
definition: dbConstraintDefName,  
/,  
*,  
layers: list[int | str] = None,  
from_: oaDesign | oaTech = None,  
of: oaDesign | oaTech = None)  
Returns: dbConstraint
```

Summary:

Returns first found oaConstraint of the given definition, else none

Category:

Constraints

Licenses:

None

Parameters

- definition – Constraint definition name to find
- layers – Tcl list of layer numbers or names
- of – Database to begin search, if oaDesign is given and no constraint is found, the associated oaTech will be searched

Returns

dbConstraint object

findContextConnectionDef

Syntax

```
findContextConnectionDef(  
none: str,  
/,  
*,  
of: dbContextConnections)  
Returns: dbContextConnectionDef
```

Summary:

This command will retrieve information about a specific connect definition with respect to the given context

Category:

Design Editing

Licenses:

None

Parameters

- none – Connection definition name to use when finding the object
- of – Context connections in which specific information should be found

Returns

Connection obj describing all defined connects in context hierarchy

findContextConnections

Syntax

```
findContextConnections(  
  none: heContext,  
  /,  
  *,  
  of: dbConnections)  
Returns: dbContextConnections
```

Summary:

This command will retrieve the connection information for a specific context

Category:

Design Editing

Licenses:

None

Parameters

- none – Context which connection information should be retrieved
- of – Defined connections object

Returns

Object describing all defined connects in the design context

findGenerator

Syntax

```
findGenerator(  
*,  
  for_: str,  
  from_: str | None = None,  
  of: str | None = None)  
Returns: str
```

Summary:

Get the specified cellview from cellview generator Tcl procedure name.

Category:

Data Translation

Licenses:

None

Parameters

- for_ – The destination viewtype name for the generator.
- of – The source viewtype name for the generator.

Returns

Name of registered Tcl proc implementing the specified generator

findInterfaceReader

Syntax

```
findInterfaceReader(  
*,  
  viewType: str)  
Returns: str
```

Summary:

Gets the specified interface reader Tcl proc name.

Category:

Data Translation

Licenses:

None

Parameters

viewType – The viewtype name for the reader.

Returns

Name of registered Tcl procedure implementing the specified reader

flushPcellCache

Syntax

```
flushPcellCache(  
*,  
  cellViews: list[dmCellView] = None,  
  param: list[oaParamArray] = None)  
Returns: None
```

Summary:

Flushes the persistent cache for the supplied pcell superMasters. If omitted, then all pcell superMasters are flushed for all libraries where caching is enabled.

Category:

Foundation

Licenses:

None

Parameters

- cellViews – Set of pcell superMasters for which to flush cached subMasters.
- param – Specifies a set of subMasters to flush for each target dmCellView.

Returns

None

freezeRefs

Syntax

```
freezeRefs(  
*,  
  of: oaDesign,  
  freeze: bool = None)  
Returns: None
```

Summary:

Freeze or unfreeze all references in design

Category:

Foundation

Licenses:

None

Parameters

- of – The design which references to freeze / thaw
- freeze – Freeze(true) or thaw(false) all references in design. (Default is: True)

Returns

None

generateAnalogSchematics

Syntax

```
generateAnalogSchematics(  
destination: dmCellView | dmCellViewName = None,  
/,  
*,  
src: dmCellView | dmCellViewName,  
overwrite: bool = None,  
makeFormGate: bool = None,  
withoutWiring: bool = None,  
mergeSubPorts: db.generateAnalogSchematics.MergeSubPorts = None,  
labelAllSegments: bool = None,  
autoCorrectIOPorts: bool = None,  
regenerateSymbols: bool = None,  
viewSearchList: str = None,  
viewStopList: str = None,  
symbolGeneratorOptionsSet: str = None,  
deviceMappingFile: str = None,
```

```
**kwargs)  
Returns: list[dmCellView]
```

The first element of the collection is the top cell view of the generated hierarchy.

Summary:

Generates schematic representation of analog cellviews.

Category:

Data Translation

Licenses:

None

Parameters

- destination – Destination hierarchy top cellview. The view name of destination cellview will be used while creating all new schematics in hierarchy. If not specified, by default a schematic view will be created for source cells. If the destination library does not exist, the command will create a new one.
- src – Top cellview of source hierarchy.
- overwrite – Overwrite the targets if exist. (Default is: True)
- makeFormGate – Wrap the recognized part of schematic into one additional level of hierarchy. (Default is: False)
- withoutWiring – No routing, just placement. The -labelAllSegments option will be set automatically. (Default is: False)
- mergeSubPorts – Merge related bit terminals into busTerm in specified order. ({ 0 | 1 | asc | desc | false | off | true } (Default is: '0'))
- labelAllSegments – Create wire name labels on all wire segments. (Default is: False)
- autoCorrectIOPorts – Try to automatically correct I/O ports directions. (Default is: True)
- regenerateSymbols – Regenerate the symbol if a cell already has a symbol view. (Default is: False)
- viewSearchList – The view search list to use while traversing the source hierarchy. (Default is: 'spice cdl')
- viewStopList – The view stop list to use while traversing the source hierarchy. (Default is: "")

- `symbolGeneratorOptionsSet` – The symbol generator option set to use while generating common symbols. (Default is: 'default')
- `deviceMappingFile` – The devices mapping file to use for device resolution. If not specified, the default `$SYNOPSYS_CUSTOM_INSTALL/samples/device_map.xml` file is taken.

Returns

Collection of new generated cell views.

generateBuiltInVias

Syntax

```
generateBuiltInVias(  
*,  
    lib: oaLib = None,  
    libName: dmLibName = None)  
Returns: list[dmCellView]
```

Summary:

Generate built-in custom vias based on standard via definitions

Category:

Data Management

Licenses:

Basic

Parameters

- `lib` – OA library where the built-in custom via definitions and superMasters will be created
- `libName` – Name of OA library where the built-in custom via definitions and superMasters will be created

Returns

the collection of new created built-in custom via cell views

generateCDL

Syntax

```
generateCDL(  
*,
```

```
interface: dbInterfaceSet,  
dest: dmCellView | dmCellViewName,  
options: dbGenOptionSet = None)  
Returns: None
```

Summary:

Generate CDL cellview.

Category:

Data Translation

Licenses:

None

Parameters

- interface – The interface set object.
- dest – Destination cellview.
- options – Object containing the option values.

Returns

None

generateCellView

Syntax

```
generateCellView(  
*,  
dest: dmCellView | dmCellViewName,  
cellview: dmCellView = None,  
cellView: dmCellView | dmCellViewName = None,  
interface: dbInterfaceSet = None,  
viewType: str = None,  
update: bool = None,  
options: dbGenOptionSet = None,  
pinMap: list[str] = None)  
Returns: None
```

Summary:

Generates a cellview from the given cellview or interface set.

Category:

Data Translation

Licenses:

None

Parameters

- `dest` – The destination cellview.
- `cellview` – The source cellview.
- `cellView` – The source cellview.
- `interface` – Tcl list of pin definitions. A pin definition is an ordered list of name, type, and side.
- `viewType` – The destination cellview viewType name, if the `-dest` option is not specified.
- `update` – Indicates if the pre-existing destination cellview should be re-generated or just updated. (Default is: False)
- `options` – Optional option set that applies to the corresponding generator. The option set type must match the type registered with the generator.
- `pinMap` – The pin name mapping items.

Returns

None

generateConstraintPatterns

Syntax

```
generateConstraintPatterns(  
*,  
    techLibName: str,  
    outputLibName: str,  
    constrDefList: str,  
    constraintGroupName: str | None = None,  
    verboseLevel: int | None = None,  
    forAutoVia: bool | None = None)  
Returns: None
```

Category:

Summary:

Generate constraint patterns for foundry/user-specified constraint group of input library

Licenses:

None

Parameters

- techLibName – Technology library name
- outputLibName – Output library name
- constrDefList – Constraint definitions to generate
- constraintGroupName – Constraint group name to generate, default: foundry
- verboseLevel – Verbose level for report file (0 or 1), default: 0
- forAutoVia – Flag for generating patterns for AutoVia, default: false

Returns

None

generateDeviceMappingTemplate

Syntax

```
generateDeviceMappingTemplate(  
*,  
    filePath: str,  
    libName: str,  
    viewSearchList: str | None = None,  
    cellName: str | None = None,  
    mapModules: bool | None = None,  
    force: bool | None = None,  
    simInfo: str | None = None)  
Returns: None
```

Summary:

The command will generate the device mapping file template for cells from the specified library filtered based on cell names pattern and views search list.

Category:

Data Translation

Licenses:

None

Parameters

- filePath – The file path to be created (relative or absolute).
- libName – The name of the library to parse.

- `viewSearchList` – The separated by spaces list of view names to be looked for. (Default is: 'symbol')
- `cellName` – The wildcard style pattern to use for cell names filtering. (Default is: '*')
- `mapModules` – The option is queried when the `namePrefix` prop is not found, if `true devType` is set to X, otherwise - "?". (Default is: True)
- `force` – Force to overwrite the file if it already exists. (Default is: False)
- `simInfo` – The name of the `simInfo` section using which device mapping file template should be generated. (Default is: 'hspiceD')

Returns

None

generateLayout

Syntax

```
generateLayout(  
cellView: dmCellView | dmCellViewName,  
/,  
*,  
src: dmCellView | dmCellViewName,  
mFactorNames: list[str] = None,  
toolSearchList: list[str] = None,  
template: db.generateLayout.Template = None,  
force: bool = None,  
xScale: float = None,  
yScale: float = None)  
Returns: None
```

Summary:

Generates layout from given schematic

Category:

Layout Editing

Licenses:

Backend

Parameters

- `cellView` – Destination cellview
- `src` – Source cellview

- mFactorNames – Space delimited string of parameter/property names to recognize as M-Factor for placeLikeSchematic template
- toolSearchList – Space delimited string of CDF tool names to search for terminal and property mapping and/or terminal order for placeLikeSchematic template
- template – Template that contains the algorithms used as the basis for generating the layout ({ floorPlan | placeLikeSchematic } (Default is: 'placeLikeSchematic'))
- force – Flag to write over existing top design (Default is: False)
- xScale – Scale factor for x-axis for floorPlan template
- yScale – Scale factor for y-axis for floorPlan template

Returns

None

generateRule

Syntax

```
generateRule(  
*,  
    design: oaDesign = None,  
    lib: oaLib = None,  
    cg: oaConstraintGroup = None,  
    cg_name: str = None,  
    output: str = None,  
    genConstDef: str = None,  
    dumpConstDef: bool = None,  
    debug: bool = None)  
Returns: str
```

Summary:

convert rule

Category:

DRD

Licenses:

None

Parameters

- design – Cell View Id
- lib – Lib Id

- cg – Constraint Group Id
- cg_name – Constraint Group Name
- output – Output File Name
- genConstDef – Generate tfConstDef.file as C++ codes (.cpp)
- dumpConstDef – Dump system constraint definitions to dumpConstDef.txt
- debug – Turn on/off debug mode

Returns

generateRule

generateSchematic

Syntax

```
generateSchematic(  
cellView: dmCellView | dmCellViewName = None,  
/,  
*,  
src: dmCellView | dmCellViewName | oaDesign | list[dmCellView],  
libName: str = None,  
viewName: str = None,  
overwrite: db.generateSchematic.Overwrite = None,  
connectByName: db.generateSchematic.ConnectByName = None,  
netNames: str = None,  
symbolViewList: str = None,  
generateSymbols: bool = None,  
symbolOptionSet: str = None,  
labelAllSegments: bool = None,  
createPortOrder: bool = None,  
ignoreInsts: str = None,  
unroutableConnections: db.generateSchematic.UnroutableConnections =  
None,  
minVerticalSpacing: int = None,  
minHorizontalSpacing: int = None,  
instanceAlignment: db.generateSchematic.InstanceAlignment = None,  
makeFormGate: bool = None,  
mergeSubPorts: db.generateSchematic.MergeSubPorts = None,  
autoCorrectIOPorts: bool = None,  
viewSearchList: str = None,  
viewStopList: str = None,  
deviceMappingFile: str = None,  
labelAllWires: bool = None,  
mergeInstBits: db.generateSchematic.MergeInstBits = None,  
createVirtualTerms: db.generateSchematic.CreateVirtualTerms = None,  
bitFormat: str = None,  
byNameNetsPattern: db.generateSchematic.ByNameNetsPattern = None,
```

```
byNameNetsCaseSensitive: bool = None)  
Returns: dmCellView | list[dmCellView]
```

Summary:

Generates schematic representation of verilog cellview.

Category:

Data Translation

Licenses:

None

Parameters

- cellView – Destination cellview. If not specified - schematic design will be created in source cell.
- src – Source cellview.
- libName – Destination library name.
- viewName – Destination view name.
- overwrite – Flag to control schematics and symbols overwrite if targets exist. ({ false | schematicsOnly | true } (Default is: 'false'))
- connectByName – Indicates whether connection will be created by name or physically ({ all | none | selected } (Default is: 'selected'))
- netNames – Specifies nets for by name connection (Default is: '')
- symbolViewList – List of views for instances placed masters (Default is: 'symbol')
- generateSymbols – Indicates whether symbol views must be generated if they are absent for instances placed masters (Default is: True)
- symbolOptionSet – Options set for symbol generation (Default is: 'default')
- labelAllSegments – Create wire name labels on all wire segments. (Default is: False)
- createPortOrder – Indicates whether portOrder property should be set on generated schematics. (Default is: True)
- ignoreInsts – Ignore instances with regular expression. Ex: ".*(Dummy).*", which is to ignore instances with "Dummy" (Default is: '')
- unroutableConnections – Specifies the way to realize unroutable connections (with flightlines or by name). ({ byName | flightline } (Default is: 'byName'))

- `minVerticalSpacing` – Minimum vertical spacing in grids between symbols during placement (Default is: 10)
- `minHorizontalSpacing` – Minimum horizontal spacing in grids between symbols during placement (Default is: 10)
- `instanceAlignment` – Alignments of symbols in each column ({ center | left | right } (Default is: 'left'))
- `makeFormGate` – Wrap the recognized part of schematic into one additional level of hierarchy. (Default is: False)
- `mergeSubPorts` – Merge related bit terminals into busTerm in specified order. ({ 0 | 1 | asc | desc | false | off | true } (Default is: '0'))
- `autoCorrectIOPorts` – Try to automatically correct I/O ports directions. (Default is: True)
- `viewSearchList` – The view search list to use while traversing the source hierarchy. By default will not traverse the hierarchy. (Default is: '<unbound>')
- `viewStopList` – The view stop list to use while traversing the source hierarchy. (Default is: "")
- `deviceMappingFile` – The devices mapping file to use for device resolution. If not specified, the default \$SYNOPSYS_CUSTOM_INSTALL/samples/device_map.xml file is taken.
- `labelAllWires` – Create wire labels on all wire segments. (Default is: False)
- `mergeInstBits` – Merge related vector bit instances into vector instance in specified order. ({ asc | desc | off } (Default is: 'off'))
- `createVirtualTerms` – Controls what terminals to create for a corresponding virtual terminals found in source. ({ asPhysicals | asPhysicalsWithConnectDef | asVirtuals } (Default is: 'asVirtuals'))
- `bitFormat` – The bus bit format used in source cellview. For verilog cellviews the value is ignored. (Default is: '[d]')
- `byNameNetsPattern` – Pattern type to use for "byName" net names matching. ({ regexp | wildcard } (Default is: 'wildcard'))
- `byNameNetsCaseSensitive` – Case sensitivity to use while matching net names for "byName" connection. (Default is: True)

Returns

Generated cell view(s)

generateSPICE

Syntax

```
generateSPICE(  
    *,  
    interface: dbInterfaceSet,  
    dest: dmCellView | dmCellViewName,  
    options: dbGenOptionSet = None)  
Returns: None
```

Summary:

Generate spice cellview.

Category:

Data Translation

Licenses:

None

Parameters

- interface – The interface set object.
- dest – Destination cellview.
- options – Object containing the option values.

Returns

None

generateSystemVerilog

Syntax

```
generateSystemVerilog(  
    *,  
    interface: dbInterfaceSet,  
    dest: dmCellView | dmCellViewName,  
    options: dbGenOptionSet = None)  
Returns: None
```

Summary:

Generate systemVerilog cellview.

Category:

Data Translation

Licenses:

None

Parameters

- interface – The interface set object.
- dest – Destination cellview.
- options – Object containing the option values.

Returns

None

generateVerilog

Syntax

```
generateVerilog(  
*,  
    interface: dbInterfaceSet,  
    dest: dmCellView | dmCellViewName,  
    options: dbGenOptionSet = None)  
Returns: None
```

Summary:

Generate verilog cellview.

Category:

Data Translation

Licenses:

None

Parameters

- interface – The interface set object.
- dest – Destination cellview.
- options – Object containing the option values.

Returns

None

generateVerilogA

Syntax

```
generateVerilogA(  
    *,  
    interface: dbInterfaceSet,  
    dest: dmCellView | dmCellViewName,  
    options: dbGenOptionSet = None)  
Returns: None
```

Summary:

Generate veriloga cellview.

Category:

Data Translation

Licenses:

None

Parameters

- interface – The interface set object.
- dest – Destination cellview.
- options – Object containing the option values.

Returns

None

generateVerilogAMS

Syntax

```
generateVerilogAMS(  
    *,  
    interface: dbInterfaceSet,  
    dest: dmCellView | dmCellViewName,  
    options: dbGenOptionSet = None)  
Returns: None
```

Summary:

Generate verilogAMS cellview.

Category:

Data Translation

Licenses:

None

Parameters

- interface – The interface set object.
- dest – Destination cellview.
- options – Object containing the option values.

Returns

None

generateVHDL**Syntax**

```
generateVHDL(  
*,  
    interface: dbInterfaceSet,  
    dest: dmCellView | dmCellViewName,  
    options: dbGenOptionSet = None)  
Returns: None
```

Summary:

Generate vhdl cellview.

Category:

Data Translation

Licenses:

None

Parameters

- interface – The interface set object.
- dest – Destination cellview.
- options – Object containing the option values.

Returns

None

getAnnotationSets

Syntax

```
getAnnotationSets()  
Returns: str
```

Arguments:

Summary:

This command gets annotation sets.

Category:

Schematic and Symbol Editing

Licenses:

None

Returns

Collection of registered annotation sets names.

getAssignmentDefs

Syntax

```
getAssignmentDefs(  
*,  
  of: dbContextConnectionDef,  
  instName: str)  
Returns: list[dbAssignmentDef]
```

Summary:

This command will retrieve the assignmnet definitions found in the specified instance in the given context.

Category:

Design Editing

Licenses:

None

Parameters

- of – Context connection definition
- instName – Name of instance in the given context

Returns

Collection of assignment defs found in the given instance hierarchy

getAssignments

Syntax

```
getAssignments(  
  none: str = None,  
  /,  
  *,  
  from_: oaDesign | oaInst = None,  
  of: oaDesign | oaInst = None)  
Returns: list[dbAssignmentDef]
```

Summary:

This command will retrieve the assignment definitions found in the specified instance or design.

Category:

Design Editing

Licenses:

None

Parameters

- none – Glob-style pattern for matching assignment name (Default is: "")
- of – Design or instance in which find assignments

Returns

dbCollection of design of instance assignments

getBuiltinNetlists

Syntax

```
getBuiltinNetlists(  
  glob: str | None = None  
Returns: None
```

Summary:

Command to return builtin analog block component netlists.

Category:

Templates Category

Licenses:

None

Parameters

glob – Glob-style name filter. (Default is: '*')

Returns

None

getCallbacks

Syntax

```
getCallbacks(  
  pattern: str = None,  
  /,  
  *,  
  callbackType: dbEventName = None,  
  viewType: str = None)  
Returns: list[dbCallback]
```

Summary:

Gets a filtered list of callbacks, based on the matches defined

Category:

Foundation

Licenses:

None

Parameters

- **pattern** – Pattern to match name of callback (Default is: ‘*’)
- **callbackType** – Callback type for which to return the registered callbacks. The default value, the empty string (“”) indicates that callbacks of all types should be returned. (Default is: “”)
- **viewType** – View type name, if applicable, for which to return the registered callbacks. The default value, the empty string (“”) indicates that callbacks of all view types should be returned. (Default is: “”)

Returns

Filtered list of callbacks

getCellViewInterface

Syntax

```
getCellViewInterface(  
    *,  
    src: dmCellView | dmCellViewName,  
    ignoreParams: bool = None,  
    processPortOrder: bool = None)  
Returns: dbInterfaceSet
```

Summary:

Creates an interface set from the source cellview.

Category:

Data Translation

Licenses:

None

Parameters

- **src** – Cellview from which to read interface information.
- **ignoreParams** – If true - parameter interface information will not be read from source cell view (Default is: True)
- **processPortOrder** – If true - the port list in text-type cellviews will be generated according to portOrder list of source cell view (Default is: False)

Returns

Interface set information from the specified cellview

getCircularRulerDefs

Syntax

```
getCircularRulerDefs(  
  none: str = None)  
Returns: list[dbCircularRulerDef]
```

Summary:

Returns a collection of circular ruler definitions.

Category:

Design Editing

Licenses:

None

Parameters

none – Pattern to match definition name.

Returns

Collection of ruler definition objects.

getComponents

Syntax

```
getComponents(  
  *,  
  of: dmCell | oaDesign | oaLib | oaRef | oaTech)  
Returns: dbCollection
```

Summary:

Retrieves a collection of components for the given object.

Category:

Foundation

Licenses:

None

Parameters

of – Collection of dbComponent objects

Returns

Collection of dbComponent objects

getConnectionDefState

Syntax

```
getConnectionDefState(  
    *,  
    of: dbContextConnectionDef,  
    instName: str)  
Returns: None
```

Summary:

This command will return the state of assignment for the given connection definition in the instance's master

Category:

Design Editing

Licenses:

None

Parameters

- of – Context connection definition
- instName – Name of instance in the given context

Returns

None

getContextConnectionDefs

Syntax

```
getContextConnectionDefs(  
    none: str = None,  
    /,  
    *,  
    from_: dbContextConnections = None,  
    of: dbContextConnections = None,  
    instName: str)  
Returns: list[dbContextConnectionDef]
```

Summary:

This command will return a collection of dbContextConnectionDef objects which exist in the switched master associated with the given instance name

Category:

Design Editing

Licenses:

None

Parameters

- none – Glob-style pattern for matching parameter name (Default is: “")
- of – Context connections
- instName – Name of instance in the given context

Returns

Collection of connect definition objects relative to given instance

getCurrentContext**Syntax**

```
getCurrentContext()  
Returns: None
```

Arguments:

Summary:

Returns the currently active context. This method should only be called from within parameter change callbacks or interpreted label evaluation.

Category:

Foundation

Licenses:

None

Returns

None

getCurrentEvalText

Syntax

```
getCurrentEvalText()  
Returns: oaEvalText
```

Arguments:

Summary:

Returns the currently active eval text. This method should only be called from within parameter change callbacks or interpreted label evaluation.

Category:

Foundation

Licenses:

None

Returns

The currently rendered or queried oaEvalText object

getCurrentParam

Syntax

```
getCurrentParam()  
Returns: str
```

Arguments:

Summary:

Returns the current parameter. This method should only be called from within parameter change callbacks or interpreted label evaluation.

Category:

Foundation

Licenses:

None

Returns

The name of the parameter

getCurrentRef

Syntax

```
getCurrentRef()  
Returns: oaRef
```

Arguments:

Summary:

Returns the currently active reference. This method should only be called from within parameter change callbacks or interpreted label evaluation.

Category:

Foundation

Licenses:

None

Returns

Reference associated with the current IL label or dbParam evaluation

getDeviceDisplays

Syntax

```
getDeviceDisplays(  
*,  
    window: int = None)  
Returns: list[dbDeviceDisplay]
```

Summary:

This command retrieves settings from the dbDeviceDisplay object(s) associated with a given instance, master, library or design.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

window – Window Id.

Returns

Objects defining state of device label displays

getEffectiveNet

Syntax

```
getEffectiveNet(  
*,  
  of: oaBitNet | oaBitTerm | str,  
  context: heContext)  
Returns: None
```

Summary:

This command returns a context where the effective net is found and the effective net name.

Category:

Netlist Tools

Licenses:

None

Parameters

- of – An object for which the effective net to search.
- context – A hierarchy context where the source object is located.

Returns

None

getEnums

Syntax

```
getEnums(  
pattern: str = None)  
Returns: dbCollection
```

Summary:

Collection of registered dbEnum objects, optionally filtered by name

Category:

Foundation

Licenses:

None

Parameters

pattern – Optional wildcard pattern to match against the name of returned enums. (Default is: '*')

Returns

Collection of dbEnum objects.

getEquivalentNets

Syntax

```
getEquivalentNets(  
    bitNet: oaBitNet | str,  
    /,  
    *,  
    context: heContext,  
    direction: db.getEquivalentNets.Direction = None,  
    signalDirection: db.getEquivalentNets.SignalDirection = None,  
    includeVirtualTerminals: bool = None,  
    includeSymbolPins: bool = None,  
    traceProc: list[variant] = None,  
    sortByDepth: bool = None,  
    depth: int = None)  
Returns: None
```

Summary:

Get hierarchically equivalent nets

Category:

Hierarchy Editing

Licenses:

None

Parameters

- bitNet – The oaBitNet or net name to trace.
- context – The queried context to determine child context

- **direction** – The hierarchy direction(s) to search ({ both | down | up } (Default is: 'both')
- **signalDirection** – The signal direction to trace ({ backward | both | forward } (Default is: 'both')
- **includeVirtualTerminals** – Include or not the nets connected by virtual connections (Default is: True)
- **includeSymbolPins** – Include connections to symbol pins in results (Default is: True)
- **traceProc** – Callback for computing additional nets to trace. The specified proc is called for each oaInstTerm the traced bit net. The proc should return the list of additional oaBitNets to trace. Signature: <list of oaBitNets> traceProc <heContext> <oaInstTerm>. Available example is 'de::probeTraceThroughLvlsIgnore'
- **sortByDepth** – If true sort results by hierarchy depth. And sort contexts with the same depth alphanumerically. (Default is: False)
- **depth** – Maximum levels to search bellow the given context. Negative value searches all levels. (Default is: -1)

Returns

None

getEvaluators

Syntax

```
getEvaluators(  
*,  
    annotationSet: str = None)  
Returns: list[dbLabelEvaluator]
```

Summary:

This command gets evaluators.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

annotationSet – Name of Annotation set object the evaluators of which should be returned.

Returns

Collection of dbLabelEvaluator objects.

getExportOasisOptions

Syntax

```
getExportOasisOptions()  
Returns: str
```

Arguments:

Summary:

Gets ExportOasis options

Category:

Data Translation

Licenses:

None

Returns

Get exportOasis command

getExportStreamOptions

Syntax

```
getExportStreamOptions(  
*,  
    tclOpt: bool | None = None)  
Returns: str
```

Summary:

Gets ExportStream options

Category:

Data Translation

Licenses:

None

Parameters

tclOpt – get db::exportStream options

Returns

Get exportStream options

getFeatures

Syntax

```
getFeatures()  
Returns: str
```

Note that the license name is an internally defined license name which may refer to one or more features on the actual license server. Use db::getFeatures to determine which features can be checked out. Arguments:

Summary:

Returns a Tcl list of all registered license features. Features returned may or may not be checked out.

Category:

Foundation

Licenses:

None

Returns

List of names of all registered features.

getFigGroupMembers

Syntax

```
getFigGroupMembers(  
*,  
  group: deFigure | oaFigGroup | str,  
  design: oaDesign = None)  
Returns: list[oaFig]
```

Summary:

Returns a collection containing of FigGroupMem in an FigGroup. Each FigGroupMem references one Fig that belongs to the FigGroup.

Category:

Database

Licenses:

None

Parameters

- group – The FigGroup from which to get members.
- design – The Design in which the group belongs.

Returns

Collection of fig group members

getFigGroups

Syntax

```
getFigGroups(  
    pattern: str = None,  
    /,  
    *,  
    of: oaDesign)  
Returns: list[oaFigGroup]
```

Summary:

Returns a collection of fig groups.

Category:

Foundation

Licenses:

None

Parameters

- pattern – Pattern to match object name. (Default is: “")
- of – Design to fetch the groups from.

Returns

Collection of fig groups

getFlightlineCoordinates

Syntax

```
getFlightlineCoordinates(  
    net: oaNet)  
Returns: oaPoint
```

Summary:

Get flightline coordinates.

Category:

Foundation

Licenses:

Basic

Parameters

net –

Returns

List of point pairs.

getGenOptionSets

Syntax

```
getGenOptionSets(  
    name: str = None)  
Returns: list[dbGenOptionSet]
```

Summary:

Get the generator option set objects with specified name, or appropriate to filter.

Category:

Data Translation

Licenses:

None

Parameters

name – The name of the set.

Returns

Collection of dbGenOptionSet objects, unordered

getGroupMembers

Syntax

```
getGroupMembers(  
*,  
  from_: list[oaGroup] = None,  
  of: list[oaGroup] = None,  
  types: db.getGroupMembers.Types = None)  
Returns: list[oaFig]
```

Summary:

Returns a collection of member figures

Category:

Layout Editing

Licenses:

None

Parameters

- of – The group to get members for
- types – Specify the type of members to return from the group. ({ all | exceptLeader | leader })

Returns

Collection of oaFig

getGroups

Syntax

```
getGroups(  
pattern: str = None,  
/,  
*,  
  of: oaDesign,  
  groupDef: oaGroupDef = None)  
Returns: list[oaGroup]
```


Summary:

Returns a collection of groups.

Category:

Foundation

Licenses:

None

Parameters

- pattern – Pattern to match object name. (Default is: “)
- of – Design to fetch the groups from.
- groupDef – Groups in the design that are associated with the specified group definition.

Returns

Collection of groups

getInstHeaders**Syntax**

```
getInstHeaders(  
dmCellViewName: dbCellViewName = None,  
/,  
*,  
of: oaDesign)  
Returns: list[oaInstHeader]
```

Summary:

Collection of oaInstHeader objects.

Category:

Database

Licenses:

None

Parameters

- dmCellViewName – Cell view name from which instHeaders will be returned (Default is: “)
- of – oaDesign to search.

Returns

Collection of instHeaders

getInsts

Syntax

```
getInsts(  
pattern: str = None,  
/,  
*,  
of: oaDesign | list[oaInstHeader] | list[oaInstTerm] |  
list[oaPin] | list[oaInstHeader])  
Returns: list[oaInst]
```

Summary:

Returns a collection of insts.

Category:

Foundation

Licenses:

None

Parameters

- pattern – Pattern to match object name. (Default is: “)
- of – Design or objects from which to collect instances.

Returns

Collection of instances

getInstTerms

Syntax

```
getInstTerms(  
pattern: oaTerm | str = None,  
/,  
*,  
of: oaDesign | list[oaFig] | list[oaInst] | list[oaNet],  
includeImplicit: bool = None,  
singleBit: bool = None)  
Returns: list[oaInstTerm]
```

Summary:

Returns a collection of instance terminals.

Category:

Foundation

Licenses:

None

Parameters

- `pattern` – Pattern to match instTerm name or oaTerm. (Default is: “)
- `of` – Design or objects from which to collect instance terminals.
- `includeImplicit` – Include implicit nets in the return collection (Default is: False)
- `singleBit` – Whether return single bit nets or not. (Default is: False)

Returns

Collection of instance terminals

getLayers

Syntax

```
getLayers(  
name: str | int = None,  
/,  
*,  
of: oaTech = None,  
local: bool = None,  
above: oaPhysicalLayer = None,  
below: oaPhysicalLayer = None,  
material: str = None)  
Returns: list[oaLayer]
```

Summary:

Returns specified layers from specified tech.

Category:

Database

Licenses:

None

Parameters

- name – Glob-style pattern to match against layer name or a number to filter the returned collection.
- of – The tech to get layers from.
- local – Return local layers (Default is: False)
- above – Retrieve layers above the specified layer.
- below – Retrieve layers below the specified layer.
- material – The material to match against layers and filter the returned collection.

Returns

Collection of oaLayer

getLicenseTier

Syntax

```
getLicenseTier()  
Returns: None
```

Arguments:

Summary:

Get the current tier.

Category:

Foundation

Licenses:

None

Returns

None

getLPPHeaders

Syntax

```
getLPPHeaders(  
lpp: lpp = None,  
/,  
*,
```

```
includeEmpty: bool = None,  
of: oaDesign)  
Returns: list[oaLPPHeader]
```

Summary:

Collection of oaLPPHeader objects.

Category:

Database

Licenses:

None

Parameters

- lpp – Layer Purpose Pair (Default is: '* *')
- includeEmpty – Search for LPP headers in design hierarchy by filtering headers containing empty shape option_collection (Default is: False)
- of – oaDesign to search.

Returns

Collection of lppHeaders

getMarkers

Syntax

```
getMarkers(  
*,  
of: oaDesign = None,  
design: oaDesign = None,  
tool: str = None,  
points: oaPointArray = None,  
message: str = None,  
shortMessage: str = None,  
regexp: bool = None,  
isVisible: bool = None,  
isClosed: bool = None,  
severity: oaMarkerSeverity = None)  
Returns: list[oaMarker]
```

Summary:

Finds the markers with given parameters

Category:

Foundation

Licenses:

None

Parameters

- of – Design in which markers are searched.
- design – Design in which markers are created.
- tool – Search pattern of the tool that created the marker.
- points – List of marker vertices.
- message – Search pattern of the description.
- shortMessage – Search pattern of the short description.
- regexp – Message, shortMessage and tool strings are regular expressions, and not glob-style expressions (Default is: False)
- isVisible – Defines if the marker is drawn on the canvas (Default is: True)
- isClosed – Specifies whether there is an edge between the first and the last points in the array. (Default is: True)
- severity – Severity type of marker (error,info).

Returns

Collection of Markers

getNetlistInfo

Syntax

```
getNetlistInfo(  
    name: str,  
    /,  
    *,  
    data: dmCell | oaCellDMDData | oaLib | oaLibDMDData,  
    netlist: str = None,  
    info: str = None,  
    section: db.getNetlistInfo.Section = None,  
    level: db.getNetlistInfo.Level = None)  
Returns: str
```

Summary:

Get Netlist info Property for a cell

Category:

Foundation

Licenses:

None

Parameters

- **name** – The name of Netlist Info.
- **data** – DMDData (or a cell or library that contains one) that supplies the requested Netlist Info.
- **netlister** – Netlister name, required only when section is 'netlist'. (Default is: "")
- **info** – Name of the subsection, only allowed when section is 'netlist'. (Default is: 'simInfo')
- **section** – Section of the netlist info that supplies the requested Netlist Info. ({ display | form | netlist }) (Default is: 'netlist')
- **level** – The CDF level from which the value will be obtained ({ base | memory | overlay }) (Default is: 'overlay')

Returns

The data stored in the Netlist info for the given netlister/name.

getNets

Syntax

```
getNets(  
    pattern: str = None,  
    /,  
    *,  
    of: oaDesign | list[oaConnFig] | list[oaInstTerm] | list[oaPin]  
    | list[oaTerm],  
    includeImplicit: bool = None,  
    equivNets: bool = None,  
    singleBit: bool = None)  
Returns: list[oaNet]
```

Summary:

Returns a collection of nets.

Category:

Foundation

Licenses:

None

Parameters

- `pattern` – Pattern to match object name. (Default is: “”)
- `of` – Design or objects from which to collect nets.
- `includeImplicit` – Include implicit nets in the return collection (Default is: False)
- `equivNets` – Whether return the logical equivalents in the database or not. (Default is: False)
- `singleBit` – Whether return single bit nets or not. (Default is: False)

Returns

Collection of Nets

getNotifiers

Syntax

```
getNotifiers(  
    pattern: str = None,  
    /,  
    *,  
    of: dbCollection)  
Returns: dbCollection
```

Summary:

Collection of notifiers supported by an object, optionally filtered by name

Category:

Foundation

Licenses:

None

Parameters

- `pattern` – Optional wildcard pattern to match against the name of returned notifiers. (Default is: `'*`)
- `of` – Object to fetch notifiers from (supported by).

Returns

Collection of notifiers.

getOpenDesigns

Syntax

```
getOpenDesigns(  
    cellViewName: dbCellViewName = None,  
    /,  
    *,  
    type: db.getOpenDesigns.Type = None)  
Returns: list[oaDesign]
```

Summary:

Get open designs

Category:

Database

Licenses:

None

Parameters

- `cellViewName` – Cell view name for which to get open designs
- `type` – Design type. ({ `all` | `pcell` | `regular` | `scratch` | `stdVia` } (Default is: `'all'`)

Returns

Opened designs

getPaperSizes

Syntax

```
getPaperSizes(  
    pattern: str = None)  
Returns: list[dbPaperSize]
```

Summary:

Returns a collection of paper sizes matching the specified criteria.

Category:

Printing and Images

Licenses:

None

Parameters

pattern – A glob-style wildcard pattern to match follower names against (Default is: '*')

Returns

A collection of paper sizes

getParamDefs

Syntax

```
getParamDefs(  
    pattern: str = None,  
    /,  
    *,  
    of: dmCell | oaCellDMDData | oaLib | oaLibDMDData | oaRef |  
    oaStdViaDef | oaTech,  
    level: db.getParamDefs.Level = None)  
Returns: list[dbParamDef]
```

Summary:

Get dbParamDef by name

Category:

Foundation

Licenses:

None

Parameters

- pattern – Glob-style pattern to match param name. (Default is: '*')
- of – Object that supplies the matching parameter definitions.
- level – The CDF level from which the value will be obtained ({ base | memory | overlay } (Default is: 'overlay'))

Returns

Collection of parameter definitions for the given object that match the supplied pattern.

getParamFilter

Syntax

```
getParamFilter(  
    *,  
    windowType: giWindowType | str  
Returns: str
```

Summary:

Get list of Netlists active in Parameter registry

Category:

Foundation

Licenses:

None

Parameters

windowType – Window Type

Returns

The list of registered Netlist Info identifiers

getParams

Syntax

```
getParams(  
    pattern: str = None,  
    /,  
    *,  
    of: dmCell | oaCellDMDData | oaLib | oaLibDMDData | oaRef |  
    oaStdViaDef | oaTech,  
    level: db.getParams.Level = None)  
Returns: list[dbParam]
```

Summary:

Get an iterator over CDF Parameters for given instance

Category:

Foundation

Licenses:

None

Parameters

- `pattern` – Glob style pattern to match param name. (Default is: `'*'`)
- `of` – Object that supplies the matching parameters.
- `level` – The CDF level from which the value will be obtained (`{ base | memory | overlay }`) (Default is: `'overlay'`)

Returns

Collection of parameters for the given object that match the supplied pattern.

getParamValue

Syntax

```
getParamValue(  
  name: str,  
  /,  
  *,  
  of: oaRef,  
  evalType: db.getParamValue.EvalType = None,  
  context: heContext = None,  
  useExponents: bool = None,  
  map: db.ExpressionMap = None,  
  refLineageParams: str = None,  
  refParentParams: str = None,  
  refDesignVars: str = None,  
  refParentListParams: str = None)  
Returns: str
```

Summary:

Get parameter value

Category:

Foundation

Licenses:

None

Parameters

- name – Parameter name
- of – Object to be queried.
- evalType – Parameter Evaluation Type ({ debug | full | netlist | none } (Default is: 'none')
- context – Context for full evaluation
- useExponents – Convert engineering suffixes to scientific notation. (Default is: True)
- map – Expression Map for evaluation
- refLineageParams – Variable storage reference for lineage parameters
- refParentParams – Variable storage reference for parent parameters
- refDesignVars – Variable storage reference for design variables
- refParentListParams – Variable storage reference for parent list parameters

Returns

The value of the named property/parameter/oaParam

getPcellCacheLocation

Syntax

```
getPcellCacheLocation(  
  CellView:  dmCellView)  
Returns: str
```

Summary:

Returns the on disk pathname of the cache file used to cache the subMaster of the passed in dmCellview as specified by an ASSIGN 'PcellCachePath' in the lib.defs file

Category:

Foundation

Licenses:

None

Parameters

CellView – A pcell dmCellView for which to determine the subMaster cache path on disk

Returns

The file path to the cache location of the supplied dmCellView or empty if not cached

getPcellCacheSize

Syntax

```
getPcellCacheSize(  
    CellView: dmCellView)  
Returns: int
```

Summary:

Returns the on disk size of the cache file used to cache the subMaster of the passed in dmCellview.

Category:

Foundation

Licenses:

None

Parameters

CellView – A pcell dmCellView for which to determine the subMaster cache size on disk

Returns

The size of the on disk subMaster cache. (Up to an 8 byte integer)

getPcellDefs

Syntax

```
getPcellDefs(  
    name: str = None,  
    /,  
    *,  
    lib: list[oaLib] = None,  
    libName: str = None)  
Returns: list[dbPcellDef]
```

Summary:

Returns a collection of pcell definitions defined within Custom Designer from the specified libraries whose names match the specified pattern.

Category:

Foundation

Licenses:

None

Parameters

- name – The glob-style wildcard pattern to match pcell definition names against. (Default is: ‘*’)
- lib – The libraries from which to retrieve pcell definitions
- libName – The library from which to retrieve pcell definitions

Returns

A collection of pcell definitions

getPcells

Syntax

```
getPcells(  
*,  
    lib: list[oaLib] = None,  
    libName: str = None)  
Returns: list[dmCellView]
```

Summary:

Returns a collection of dmCellViews that contains Custom Designer defined pcell superMasters in the supplied library, optionally filtered by the attributes of a dmCellview.

Category:

Foundation

Licenses:

None

Parameters

- lib – The libraries from which to retrieve pcells
- libName – The library for which pcells are desired.

Returns

A collection of dmCellViews that contain pcell superMaster oaDesigns

getPins

Syntax

```
getPins(  
    pattern: str = None,  
    /,  
    *,  
    of: oaDesign | list[oaNet] | list[oaTerm])  
Returns: list[oaPin]
```

Summary:

Returns a collection of pins.

Category:

Foundation

Licenses:

None

Parameters

- pattern – Pattern to match object name. (Default is: “”)
- of – Design or objects from which to collect pins.

Returns

Collection of Pins

getPrefs

Syntax

```
getPrefs(  
    name: str = None,  
    /,  
    *,  
    scope: dbCollection = None,  
    label: str = None,  
    localized: bool = None,  
    dialog: giDialog = None,  
    includeDeprecated: bool = None)  
Returns: dbPref
```

Summary:

Returns a collection of preference values from the specified scopes.

Category:

Foundation

Licenses:

None

Parameters

- name – Optional glob-style pattern to match against preference name
- scope – Optional collection of scope specifications to return preference values from these scopes
- label – Optional glob-style pattern to match against preference label
- localized – Return only preferences localized in the given scope (Default is: False)
- dialog – Optionally restrict list to preferences shown within a dialog
- includeDeprecated – Whether to include deprecated preferences in result (Default is: False)

Returns

Collection of dbPref objects matching criteria

getPrefValue

Syntax

```
getPrefValue(  
name: dbPrefName,  
/  
*,  
scope: dbScope | deContext | dmCell | dmCellView | oaLib |  
str = None)  
Returns: str
```

Summary:

Returns the value of a preference from the specified scope

Category:

Foundation

Licenses:

None

Parameters

- name – Preference name.
- scope – The scope from which to retrieve the preference value.

Returns

Preference value

getPrinters

Syntax

```
getPrinters(  
pattern: str = None)  
Returns: list[dbPrinter]
```

Summary:

Returns a collection of printers matching the specified criteria.

Category:

Printing and Images

Licenses:

None

Parameters

pattern – A glob-style wildcard pattern to match follower names against (Default is: '*')

Returns

A collection of printers

getProcessInfo

Syntax

```
getProcessInfo()  
Returns: dbProcessInfo
```

Arguments:

Summary:

Returns the program related information.

Category:

Foundation

Licenses:

None

Returns

Process Information

getProps

Syntax

```
getProps(  
    pattern: str = None,  
    /,  
    *,  
    of: oaObject)  
Returns: list[oaProp]
```

Summary:

Collection of oaProp objects.

Category:

Database

Licenses:

None

Parameters

- pattern – Property name (Default is: '*')
- of – OA object.

Returns

Collection of props

getRegisteredNetlistInfo

Syntax

```
getRegisteredNetlistInfo(  
    none: str | None = None)  
Returns: str
```

Summary:

Get Registered Netlists

Category:

Foundation

Licenses:

None

Parameters

none – The name of the netlist from which to retrieve its registered default cdf properties (Default is: “)

Returns

The list of registered Netlist Info identifiers

getRulers

Syntax

```
getRulers(  
*,  
    design: oaDesign,  
    addOverlay: bool = None,  
    editableDesignOnly: bool = None)  
Returns: list[dbRuler]
```

Summary:

Returns a collection of liner and circular rulers.

Category:

Design Editing

Licenses:

None

Parameters

- design – Design for which the rulers are required.
- addOverlay – Get additional objects from the overlay design
- editableDesignOnly – Ignore the persistent rulers in the read-only design

Returns

Collection of rulers

getScopes

Syntax

```
getScopes(  
    scope: deContext | dmCell | dmCellView | oaLib | str = None)  
Returns: dbScope
```

Summary:

Returns a collection of scopes.

Category:

Foundation

Licenses:

None

Parameters

scope – The scope from which to retrieve the preference value

Returns

Collection of scopes matching optional qualifiers

getScopeTypes

Syntax

```
getScopeTypes(  
    _pattern_: str = None)  
Returns: dbScopeType
```

Summary:

Returns a collection of all scope types.

Category:

Foundation

Licenses:

None

Parameters

`_pattern_` – Optional glob-style pattern to match against scope type names

Returns

Collection of scope types

getShapes

Syntax

```
getShapes(  
*,  
  of: oaDesign | list[oaLPPHeader] | list[oaNet] | list[oaPin],  
  lpp: lpp = None)  
Returns: list[oaShape]
```

Summary:

Returns a collection of shapes.

Category:

Foundation

Licenses:

None

Parameters

- `of` – Design or objects from which to collect shapes.
- `lpp` – Restricts shapes to be returned from a given LPP.

Returns

Collection of Shapes

getTableValue

Syntax

```
getTableValue(  
values: list[str],  
/,  
*,  
  from_: dbTable = None,  
  of: dbTable = None)  
Returns: dbCore
```

Summary:

Returns string representation of the oaValue stored in the lookup table

Category:

Constraints

Licenses:

None

Parameters

- values – Tcl list of column and row values to use for the table lookup. If the table is one-dimensional, only the first value from the list is used
- of – Table object holding the lookup table to search

Returns

String representation of the oaValue stored in the lookup table

getTerms**Syntax**

```
getTerms(  
  pattern: str = None,  
  /,  
  *,  
  of: oaDesign | list[oaNet],  
  includeImplicit: bool = None,  
  singleBit: bool = None)  
Returns: list[oaTerm]
```

Summary:

Returns a collection of terminals.

Category:

Foundation

Licenses:

None

Parameters

- pattern – Pattern to match object name. (Default is: "")
- of – Design or objects from which to collect terminals.

- includeImplicit – Include implicit terms in the return collection (Default is: False)
- singleBit – Whether return single bit nets or not. (Default is: False)

Returns

Collection of Terminals

getType

Syntax

```
getType(  
*,  
  of: attrObject)  
Returns: variant
```

Summary:

Returns specific object type.

Category:

Layout Editing

Licenses:

None

Parameters

of – The object to get type of.

Returns

Type of the object.

getVDRCDDeviceList

Syntax

```
getVDRCDDeviceList(  
*,  
  targetLib: oaLib | str,  
  type: db.getVDRCDDeviceList.Type)  
Returns: str
```

Summary:

Get the list of names of vdrc device with the specified type

Category:

Database

Licenses:

None

Parameters

- targetLib – Query VDRC devices applied in the target library
- type – Device type ({ udv | vflagMax | vflagMin | vflaghl | vsync })

Returns

List of LCV of VDRC devices

getViaDefs

Syntax

```
getViaDefs(  
    name: str = None,  
    /,  
    *,  
    of: oaTech,  
    local: bool = None)  
Returns: list[oaViaDef]
```

Summary:

Collection of oaViaDef objects.

Category:

Database

Licenses:

None

Parameters

- name – glob-style pattern to match against via def name to filter the returned collection.
- of – The tech to get via defs from.
- local – Return local via defs (Default is: False)

Returns

Collection of oaViaDef

getViaHeaders

Syntax

```
getViaHeaders(  
  viaDefName: str = None,  
  /,  
  *,  
  of: oaDesign)  
Returns: list[oaViaHeader]
```

Summary:

Collection of oaViaHeader objects.

Category:

Database

Licenses:

None

Parameters

- viaDefName – oaViaDef name (Default is: “”)
- of – oaDesign to search.

Returns

Collection of viaHeaders

getVias

Syntax

```
getVias(  
  pattern: str = None,  
  /,  
  *,  
  of: oaDesign | oaPin | list[oaNet] | list[oaViaHeader])  
Returns: list[oaVia]
```

Summary:

Returns a collection of vias.

Category:

Foundation

Licenses:

None

Parameters

- pattern – Pattern to match object name. (Default is: “")
- of – A design, pin or a collection of viaHeaders or nets to fetch the vias from.

Returns

Collection of Vias

getVirtualInstTerms

Syntax

```
getVirtualInstTerms(  
  none: dbContextConnections,  
  /,  
  *,  
  instName: str)  
Returns: str
```

Summary:

This command will return a tcl list of virtual instance terminals names

Category:

Design Editing

Licenses:

None

Parameters

- none – Context connections
- instName – Name of instance in the given context

Returns

Tcl list of strings representing virtual terminal names

getWindowAnnotationSets

Syntax

```
getWindowAnnotationSets(  
*,  
    window: giWindow | int)  
Returns: list[dbWindowAnnotationSet]
```

Summary:

This command gets window annotation sets.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

window – Window the dbWindowAnnotationSet objects of which should be retrieved.

Returns

Collection of dbWindowAnnotationSet objects.

hideProps

Syntax

```
hideProps(  
    names: list[str],  
    /,  
    *,  
    objType: str = None)  
Returns: None
```

Summary:

Tells the Property Editor to hide/ignore properties matching specific names.

Category:

Foundation

Licenses:

None

Parameters

- names – List of property names to ignore.
- objType – Type of object for which the named properties should be hidden. (Default is: 'Design')

Returns

None

importEDIF

Syntax

```
importEDIF(  
*,  
    inputFile: str,  
    outputLibName: str = None,  
    topCellName: str = None,  
    preparse: str = None,  
    importResultLogFile: str = None,  
    overwrite: bool = None,  
    ignoreUnusedParams: bool = None,  
    putLabelOnAllSegments: bool = None,  
    case: db.importEDIF.Case = None,  
    cellNamesCase: db.importEDIF.CellNamesCase = None,  
    instsNamesCase: db.importEDIF.InstsNamesCase = None,  
    netsNamesCase: db.importEDIF.NetsNamesCase = None,  
    paramsNamesCase: db.importEDIF.ParamsNamesCase = None,  
    symbolsOnly: bool = None,  
    cdfFilesDir: str = None,  
    notSearchExternalSymbol: bool = None,  
    dataSource: db.importEDIF.DataSource = None)  
Returns: None
```

Summary:

Import Edif file

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- inputFile – The input edif file.
- outputLibName – The output library name. This is only used with -symbolsOnly (Default is: “)
- topCellName – The top cell name of importing EDIF file. (Default is: “)
- preparse – Specify file to return the list of LCVs with parsed data dump, does not do importing. (Default is: “)
- importResultLogFile – The import results log file name. (Default is: “)
- overwrite – Overwrite the existing cellviews. (Default is: True)
- ignoreUnusedParams – Auto remove unused parameters. (Default is: False)
- putLabelOnAllSegments – Create wire labels on all segments of a net. (Default is: False)
- case – The case for all names to use. ({ lower | preserve | upper } (Default is: ‘preserve’)
- cellNamesCase – The case for cell names to use. ({ lower | preserve | upper })
- instsNamesCase – The case for instance names to use. ({ lower | preserve | upper })
- netsNamesCase – The case for net/terminal names to use. ({ lower | preserve | upper })
- paramsNamesCase – The case for parameter names/values to use. ({ lower | preserve | upper })
- symbolsOnly – Extract the symbol library only. (Default is: False)
- cdfFilesDir – The root directory where to lookup the ‘cdfData’ specified file. If not specified the input file directory is used.
- notSearchExternalSymbol – Do not search for external symbols. (Default is: False)
- dataSource – The producer tool of source data(leave default if not sure). ({ Composer | ViewDraw } (Default is: ‘Composer’)

Returns

None

importMCellFile

Syntax

```
importMCellFile(  
*,  
    lib: oaLib = None,  
    libName: dmLibName = None,  
    filePath: str,  
    cdfFilePath: str = None,  
    overwrite: bool = None)  
Returns: list[dmCellView]
```

Summary:

Import MCell Definition

Category:

Data Management

Licenses:

Basic

Parameters

- lib – OA library that the MCell devices will be created into
- libName – Name of OA library that the MCell devices will be created into
- filePath – The MCell definition file path
- cdfFilePath – The MCell CDF file path (Default is: "")
- overwrite – To overwrite existing cell views (Default is: False)

Returns

the collection of new created MCell cell views

importSpice

Syntax

```
importSpice(  
*,  
    filePath: str,  
    dest: dmCellView,  
    portOrder: list[str],  
    subcircuit: str)  
Returns: None
```

Summary:

Imports Spice text file within the OA library structure

Category:

Data Translation

Licenses:

None

Parameters

- `filePath` – Path of the Spice text file
- `dest` – The destination cellview
- `portOrder` – Port order list
- `subcircuit` – The subcircuit name

Returns

None

importTechFile**Syntax**

```
importTechFile(  
    path: str,  
    /,  
    *,  
    lib: oaLib = None,  
    libName: str = None,  
    merge: bool = None,  
    mode: str = None,  
    sections: str = None)  
Returns: oaTech
```

Summary:

Import technology file

Category:

Data Management

Licenses:

None

Parameters

- path – Path including the file name of technology file to import.
- lib – OA library, where technology data should be loaded.
- libName – Name of OA library, where technology data should be loaded.
- merge – Whether to merge or override existing technology. (Default is: False)
- mode – Importing technology mode: merge, overwrite or replace. (Default is: 'overwrite')
- sections – Sections or subsections to be imported. (Default is: "")

Returns

Technology object filled after reading the file

importText

Syntax

```
importText(  
filePath: list[str],  
/,  
*,  
libName: str,  
viewName: str = None,  
libSearchList: list[str] = None,  
viewSearchList: list[str] = None,  
interfaceOnly: bool = None,  
generateSymbols: str = None,  
topSubckt: str = None,  
digitalTop: str = None,  
overwrite: db.importText.Overwrite = None,  
overwriteSymbols: db.importText.OverwriteSymbols = None,  
viewType: db.importText.ViewType = None,  
deviceMappingFile: str = None,  
includeFiles: str = None,  
defines: str = None,  
parserOptions: str = None,  
preserveDirectives: bool = None,  
importModules: str = None,  
moduleMatchType: db.importText.ModuleMatchType = None,  
excludeModules: str = None,  
excludeModuleMatchType: db.importText.ExcludeModuleMatchType = None,  
configTemplate: heConfigTemplate | str = None,  
singleSource: bool = None,  
virtualTermMapping: list[str] = None,  
virtualTermsReferenceView: list[str] = None,  
skipModDefGeneration: bool = None,
```

```
**kwargs)  
Returns: list[dmCellView]
```

Summary:

Imports netlist text file within the OA library structure

Category:

Data Translation

Licenses:

None

Parameters

- `filePath` – List of netlist text files to import
- `libName` – Destination library name for imported text views
- `viewName` – Destination view name for imported text views
- `libSearchList` – Library search list for instance binding
- `viewSearchList` – View search list for instance binding
- `interfaceOnly` – Flag indicating whether only the interfaces of imported text views should be extracted (Default is: `False`)
- `generateSymbols` – Generate symbols using this option set: if not given symbols will not be generated (Default is: `""`)
- `topSubckt` – Indicates whether top level SPICE/CDL statements should be imported into a cell and what cell name to use for it. (Default is: `""`)
- `digitalTop` – Specifies the top module name, if digital is the top of the imported hierarchy: config will be created on corresponding cell. (Default is: `""`)
- `overwrite` – Cellview overwriting mode to use during importing. (`{ always | never }`)
- `overwriteSymbols` – Cellview overwriting mode to use during importing for symbols to be generated. (`{ always | never | portOrderOnly }`)
- `viewType` – View type for imported text views: if not specified will be deducted from the view name or file extensions (`{ CDLText | SPECTREText | SPICEText | VHDLText | systemVerilogText | verilogAMSText | verilogAText | verilogText }`) (Default is: `""`)
- `deviceMappingFile` – Device mapping file path to be used for instance binding: for analog languages only (Default is: `""`)
- `includeFiles` – Filter specifying processing mode for include files. The acceptable format is `'<action>'` or `'<action> {<action> <file_names> <match_mode>} ...'` where:

<action> is one of the set: {parse, include, ignore}, <file_names> is single name or Tcl list of names, <match_mode> is one of the set: {exact, regexp, wildcard}. (Default is: 'include')

- defines – Additional definitions list to be predeclared for verilog/A/AMS views (Default is: "")
- parserOptions – Parser specific options (Default is: "")
- preserveDirectives – Flag indicating whether directives should be preserved in imported text views (Default is: True)
- importModules – The names of the modules(or module name pattern) to be imported (Default is: "")
- moduleMatchType – The module matching type to filter the module names using the importModules option value. ({ exact | regexp | wildcard }) (Default is: 'exact')
- excludeModules – The names of the modules(or module name pattern) to be excluded during import (Default is: "")
- excludeModuleMatchType – The module matching type to filter the module names using the excludeModules option value. ({ exact | regexp | wildcard }) (Default is: 'exact')
- configTemplate – Hierarchy config template to be used for config creation, when -topSubckt or -digitalTop is specified
- singleSource – Indicates whether text importing should be done in single source mode (Default is: False)
- virtualTermMapping – Terminal to connection definition mapping
- virtualTermsReferenceView – Prioritized list of views for determining if a terminal should be virtual or not, any terminal that is not found in a reference view will be created as virtual.
- skipModDefGeneration – Flag indicating whether to generate module definition file from reference library (Default is: False)

Returns

A collection of created CellViews

includes

Syntax

```
includes (  
  collection: dbCollection,  
  /,
```

```
*,  
object: attrObject)  
Returns: None
```

Summary:

Checks whether an object is included in a given dbCollection.

Category:

Foundation

Licenses:

None

Parameters

- collection – Collection.
- object – An object to be checked.

Returns

None

init

Syntax

```
init(  
path: str | None = None,  
/,  
*,  
echo: bool | None = None,  
noWarning: bool | None = None)  
Returns: None
```

Summary:

Given a path relative to any startup directory, sources matches from any startup directory.

Category:

Foundation

Licenses:

None

Parameters

- path – Path relative to any startup directory. (Default is: '.')
- echo – Display content of sourced files. (Default is: False)
- noWarning – Do not issue a warning when no file found. (Default is: False)

Returns

None

instCacheTriggerParamCallbacks

Syntax

```
instCacheTriggerParamCallbacks(  
    cache: dbCore,  
    /,  
    *,  
    of: oaRef,  
    triggerParamList: str)  
Returns: None
```

Summary:

Triggers parameter callbacks by using cache

Category:

Foundation

Licenses:

None

Parameters

- cache – The inst cache.
- of – The instance used to trigger parameter callbacks.
- triggerParamList – The ordered list of parameter names to trigger callbacks for.

Returns

None

interrupt

Syntax

```
interrupt()  
Returns: None
```

Arguments:

Summary:

Throws a dbInterruptException Tcl exception.

Category:

Foundation

Licenses:

None

Returns

None

isEncrypted

Syntax

```
isEncrypted(  
  filename: str  
)  
Returns: None
```

Summary:

checks if a file is encrypted in the Laker format

Category:

DRD

Licenses:

None

Parameters

filename – file to check if encrypted

Returns

None

isObject

Syntax

```
isObject(  
  object: variant  
Returns: bool
```

Summary:

Checks whether a TCL variable holds a dbObject or an oaObject.

Category:

Foundation

Licenses:

None

Parameters

object – The object to be tested.

Returns

Returns true if the given variable holds a dbObject or oaObject.

isProhibitedName

Syntax

```
isProhibitedName(  
  name: str  
Returns: bool
```

Summary:

Checks whether the name contains prohibited characters defined by preference dbProhibitedCharacters.

Category:

Foundation

Licenses:

None

Parameters

name – Object name, to check

Returns

Returns true if given name contains prohibited character defined by preference dbProhibitedCharacters, otherwise false.

listCallbackTypes

Syntax

```
listCallbackTypes(  
    pattern: str | None = None  
)  
Returns: str
```

Summary:

Generates a report that lists all the defined callback types and their argument lists

Category:

Foundation

Licenses:

None

Parameters

pattern – Glob-type pattern to match callback names (Default is: '*')

Returns

Report of all the defined callback types

loadDeviceDisplays

Syntax

```
loadDeviceDisplays(  
    *,  
    window: int = None,  
    filePath: str  
)  
Returns: list[dbDeviceDisplay]
```

Summary:

This command loads the label display information.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- `window` – Window Id.
- `filePath` – File from which to load the display information. Assumed relative to current working directory if not absolutely specified.

Returns

Collection of `dbDeviceDisplay` objects created from the file

loadGenOptionSets

Syntax

```
loadGenOptionSets(  
*,  
    filePath: str,  
    force: bool = None,  
    loadDefault: bool = None,  
    src: oaDesign = None,  
    fromTech: bool = None)  
Returns: dbGenOptionSet
```

Summary:

Loads option sets from a file.

Category:

Data Translation

Licenses:

None

Parameters

- `filePath` – Name of file to read and load into the named option sets.
- `force` – If true, existing option sets with the same name as a set from the file will be overwritten. (Default is: False)
- `loadDefault` – Load default values for `abstractGenerator` (Default is: False)

- src – The source design.
- fromTech – Get constraints from technology database, otherwise get them from the design. (Default is: False)

Returns

If successful, collection of newly created dbGenOptionSet objects

loadMCells

Syntax

```
loadMCells(  
*,  
    lib: oaLib = None,  
    libName: str = None)  
Returns: None
```

Summary:

Load MCell rules from disk.

Category:

Data Management

Licenses:

Basic

Parameters

- lib – OA library, from where MCell superMaster data should be loaded
- libName – Name of OA library, from where MCell superMaster data should be loaded

Returns

None

loadModelFile

Syntax

```
loadModelFile(  
fileSection: list[str],  
/,  
*,  
    format: db.loadModelFile.Format = None)  
Returns: dbGndbProfile
```

Summary:

Load simulation model files and sections by SPICE parser.

Category:

Data Translation

Licenses:

None

Parameters

- fileSection – List of model files and sections to load
- format – Format of model files. Default is spice. ({ spectre | spice })

Returns

Sessions matching the search criteria

loadPrefs

Syntax

```
loadPrefs(  
collection: dbCollection = None,  
/,  
*,  
filePath: str = None)  
Returns: dbPref
```

Summary:

Load preferences from the supplied scope or a file

Category:

Foundation

Licenses:

None

Parameters

- collection – Collection of preferences to modify or a scope into which these values are loaded and localized
- filePath – Load preferences from supplied file. If provided, matching preferences given in the unnamed argument are modified (Default is: “)

Returns

Collection of preferences which were loaded

loadStreamNameMap

Syntax

```
loadStreamNameMap(  
    StreamObjMap: dbStreamNameMap,  
    /,  
    *,  
    filename: str)  
Returns: None
```

Summary:

Load XML file into dbStreamNameMap object

Category:

Data Translation

Licenses:

None

Parameters

- StreamObjMap – Cell name to stream object map
- filename – XML file to load into the map.

Returns

None

localizePrefs

Syntax

```
localizePrefs(  
    dbpref: dbCollection,  
    /,  
    *,  
    scope: dbScope | deContext | dmCell | dmCellView | oaLib |  
    str)  
Returns: dbCollection
```

Summary:

Localizes preferences in the scope.

Category:

Foundation

Licenses:

None

Parameters

- dbpref – Collection of preference objects which were localized in the given scope
- scope – Scope where the preference will be localized.

Returns

Collection of preference objects localized in the given scope

mapStreamName

Syntax

```
mapStreamName(  
    streamName: str,  
    //,  
    *,  
    map: dbStreamNameMap)  
Returns: dbStreamedObject
```

Summary:

Map the given stream name to a streamed object

Category:

Data Translation

Licenses:

None

Parameters

- streamName – Stream name to map
- map – The stream name to streamed object map that will be used

Returns

Mapped streamed object

match

Syntax

```
match(  
  text: str | None = None,  
  /,  
  *,  
  pattern: str | None = None,  
  noCase: bool | None = None)  
Returns: bool
```

Summary:

Determine whether pattern matches string, returning return 1 if it does, 0 if it doesn't.

Category:

Foundation

Licenses:

None

Parameters

- text – the text (Default is: "")
- pattern – Space delimited wildcard pattern (Default is: '*')
- noCase – is true, then the pattern attempts to match against the string in a case insensitive manner (Default is: False)

Returns

Returns true if the given pattern matches string

notify

Syntax

```
notify(  
  notifier: dbNotifier,  
  /,  
  *,  
  args: list[str] = None)  
Returns: None
```

Summary:

Call listeners of a notifier.

Category:

Layout Editing

Licenses:

None

Parameters

- `notifier` – Notifier whose listeners to call
- `args` – Arguments to pass to listeners.

Returns

None

openDesign**Syntax**

```
openDesign(  
  cellView: dmCellView | dmCellViewName,  
  /,  
  *,  
  enableUndo: bool = None,  
  readOnly: bool = None)  
Returns: oaDesign
```

Summary:

Open design

Category:

Database

Licenses:

None

Parameters

- `cellView` – CellView for which to open design
- `enableUndo` – Enable undo flag (Default is: False)
- `readOnly` – Read-only mode (Default is: False)

Returns

Newly opened design

openDesignHierarchy

Syntax

```
openDesignHierarchy(  
    design: oaDesign,  
    /,  
    *,  
    depth: int = None)  
Returns: None
```

Summary:

Open design hierarchy

Category:

Database

Licenses:

None

Parameters

- design – OA Design open depth
- depth – Depth to open design. (Default is: 32)

Returns

None

openParamDefEditor

Syntax

```
openParamDefEditor(  
    *,  
    cell: dmCell = None,  
    libName: str = None,  
    readOnly: bool = None)  
Returns: giWindow
```

Summary:

Open Parameter Definition Editor

Category:

Foundation

Licenses:

None

Parameters

- `cell` – Opens the parameter definitions editor for the `dmCell`
- `libName` – Opens the parameter definition editor for the library
- `readOnly` – Opens the parameter definitions in read-only mode (Default is: `False`)

Returns

New Parameter Definition Editor

openScope

Syntax

```
openScope(  
  object: dmCell | dmCellName | dmCellView | dmCellViewName |  
          oaCell | oaCellView | oaLib)  
Returns: dbScopeType
```

Summary:

Creates the OA scope and returns it. The scope is guaranteed to exist until all holders of this scope release their objects referencing it.

Category:

Foundation

Licenses:

None

Parameters

`object` – OA or DM object for which the scope will be opened.

Returns

Newly opened scope.

pdaExplainCell

Syntax

```
pdaExplainCell(  
    dialog: giDialog,  
    /,  
    *,  
    number: int = None)  
Returns: None
```

Summary:

Explain's cell inherited infos of tree widget in PowerDomainAnalyzer dialog

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- dialog – PDA dialog.
- number – indicates inherited info for expalin. (Default is: 0)

Returns

None

pdaShowCell

Syntax

```
pdaShowCell(  
    dialog: giDialog,  
    /,  
    *,  
    number: int = None)  
Returns: None
```

Summary:

Show's cell of tree widget

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- dialog – PDA dialog.
- number – indicates inherited info for show. (Default is: 0)

Returns

None

print

Syntax

```
print(  
context: str | list[dmCell] | list[dmCellView] | list[giWindow] |  
    list[oaLib],  
/,  
*,  
printer: dbPrinter,  
depth: int = None,  
hierMode: db.print.HierMode = None,  
viewlist: str = None,  
viewSwitchList: str = None,  
libFilter: str = None,  
cellFilter: str = None,  
viewFilter: str = None,  
file: str = None,  
fileType: db.print.FileType = None,  
overwrite: bool = None,  
copies: int = None,  
orientation: db.print.Orientation = None,  
paperSize: str = None,  
doubleSided: bool = None,  
leftMargin: float = None,  
rightMargin: float = None,  
topMargin: float = None,  
bottomMargin: float = None,  
display: str = None,  
colorMode: db.print.ColorMode = None,  
omitBackground: bool = None,  
includeBackground: bool = None,  
invertBrightness: list[bool] = None,  
scaleFit: db.print.ScaleFit = None,  
scaleFactor: float = None,  
fontScale: float = None,  
title: bool = None,  
legend: db.print.Legend = None,
```

```
titlePosition: db.print.TitlePosition = None,  
fontSize: int = None,  
grid: bool = None,  
borderRulers: bool = None,  
bookmarks: bool = None,  
window: list[giWindow] = None,  
sheets: list[str] = None)  
Returns: xtJob
```

Summary:

Outputs the specified CellView or window contents to the specified file or printer.

Category:

Printing and Images

Licenses:

None

Parameters

- context – The CellView or window or file path to output
- printer – The printer to which the output should be sent
- depth – The hierarchy depth
- hierMode – Mode to use when printing hierarchical data ({ all | inContext })
- viewlist – The viewlist to use when printing a non-config CellView hierarchically
- viewSwitchList – The viewSwitchList to use when printing a non-config CellView hierarchically
- libFilter – Library filter for cellviews printed indirectly (i.e., when a cellview is printed due to being referenced from a directly printed design).
- cellFilter – Cell filter for cellviews printed indirectly (i.e., when a cellview is printed due to being referenced from a directly printed design).
- viewFilter – View filter for cellviews printed indirectly (i.e., when a cellview is printed due to being referenced from a directly printed design).
- file – The file to output to (Default is: “)
- fileType – Output file format ({ pdf | pdfWithLinks | ps })
- overwrite – Controls whether existing file will be overwritten
- copies – The number of copies to output

- orientation – The orientation to use ({ auto | landscape | portrait })
- paperSize – The paper size to use
- doubleSided – Enabling or disabling double sided printing
- leftMargin – The left margin to use
- rightMargin – The right margin to use
- topMargin – The top margin to use
- bottomMargin – The bottom margin to use
- display – The display to use when outputting (Default is: “”)
- colorMode – Whether to print in color, grayscale or black & white ({ blackAndWhite | color | grayscale })
- omitBackground – Whether to omit background when printing
- includeBackground – Whether to include background when printing
- invertBrightness – Whether to darken light colors and brighten dark colors
- scaleFit – The fit mode to use. ({ height | page | width })
- scaleFactor – The scaling factor to use
- fontScale – An additional scale factor for fonts
- title – Whether to print a title
- legend – Enable/disable the legend and controls its location ({ bottom | disabled | left | right | separatePage | top })
- titlePosition – Enable/disable the title and control its location ({ bottom | disabled | left | right | top })
- fontSize – The font size for the text
- grid – Enable or disable the grid
- borderRulers – Enable/disable border rulers
- bookmarks – Enable/disable bookmark printing in hierarchy mode
- window – The design window that is used to print selected cellviews
- sheets – Sheets of multisheet design to print

Returns

The print job created

purgeCDF

Syntax

```
purgeCDF(  
  data: dmCell | oaLib,  
  /,  
  *,  
  removeCDF: bool = None,  
  level: db.purgeCDF.Level = None)  
Returns: None
```

Summary:

Purge CDF for a cell or library

Category:

Foundation

Licenses:

None

Parameters

- data – Specifies the location to purge the CDF from the oaDMDData
- removeCDF – Specifies whether the 'cdfData' property is to be removed from the oaDMDData (Default is: False)
- level – The CDF level to be purged ({ base | memory | overlay } (Default is: 'overlay'))

Returns

None

purgePrefs

Syntax

```
purgePrefs(  
  argument: dbCollection | str)  
Returns: None
```

Summary:

Removes any active and saved preference data at the supplied scope or their scopes

Category:

Foundation

Licenses:

None

Parameters

argument – Collection of preferences, scopes or an environment to purge.

Returns

None

query

Syntax

```
query(  
    object: list[dbCollection],  
    /,  
    *,  
    verbose: bool = None)  
Returns: str
```

Summary:

Reports objects.

Category:

Foundation

Licenses:

None

Parameters

- object – The object(s) to query.
- verbose – Query collections. (Default is: False)

Returns

The report.

queryFeature

Syntax

```
queryFeature(  
    name: str)  
Returns: None
```

Note that the license name is an internally defined license name which may refer to one or more features on the actual license server. Use `db::getFeatures` to determine which features can be checked out.

Summary:

Returns true if license feature is checked out. If the feature is not recognized, an error is thrown.

Category:

Foundation

Licenses:

None

Parameters

name – Name of the licensing feature to check out.

Returns

None

raiseLicenseTier

Syntax

```
raiseLicenseTier(  
    tier: db.raiseLicenseTier.Tier)  
Returns: None
```

Summary:

Raise the current tier.

Category:

Foundation

Licenses:

None

Parameters

tier – Tier name to check. ({ advanced | advancedPlus | apex | backend | basic | elite | frontend | mtm | router | schematic })

Returns

None

refreshICCCache

Syntax

```
refreshICCCache(  
    cellViews: list[dmCellView] = None,  
    *,  
    force: bool = None)  
Returns: list[dmCellView]
```

Summary:

Refreshes (creates or updates) the OpenAccess translation cache for layout designs in an ICC library

Category:

Data Translation

Licenses:

None

Parameters

- cellViews – Collection of cellViews to refresh. (Default is: All cellViews in all open libraries)
- force – When true, all cellViews are retranslated even if the cache is current. (Default is: False)

Returns

Collection of cellViews which were refreshed.

refreshMCells

Syntax

```
refreshMCells(  
    *,  
    masters: list[dmCellView],  
    lib: list[oaLib] = None,  
    libName: dmLibName = None)  
Returns: None
```

Summary:

Refresh subMasters of MCell

Category:

Data Management

Licenses:

Basic

Parameters

- `masters` – The superMaster dmCellViews
- `lib` – OA libraries that their instances of MCell and editable pcell with specified superMasters will be refreshed
- `libName` – Names of OA libraries that their instances of MCell and editable pcell with specified superMasters will be refreshed

Returns

None

registerCallbackRegsub

Syntax

```
registerCallbackRegsub(  
    regexp: str,  
    /,  
    *,  
    substitute: str)  
Returns: None
```

Summary:

Register regular expression substitution for callback evaluation

Category:

Foundation

Licenses:

None

Parameters

- `regexp` – Regular expression to use for pattern
- `substitute` – Pattern to use for substitution

Returns

None

registerGenerator

Syntax

```
registerGenerator(  
    name: str,  
    /,  
    *,  
    for_: str,  
    from_: str | None = None,  
    of: str | None = None,  
    optionSetType: str | None = None,  
    defaultOptionSetPref: str | None = None,  
    optionEditCommand: str | None = None,  
    supportsUpdate: bool | None = None,  
    usesPinSides: bool | None = None)  
Returns: None
```

Summary:

Register cellview from cellview generator.

Category:

Data Translation

Licenses:

None

Parameters

- `name` – The name of the cellview generator Tcl procedure.
- `for_` – The destination viewtype name for the generator.
- `of` – The source viewtype name for the generator.
- `optionSetType` – Type option set name.
- `defaultOptionSetPref` – Name of the preference which holds the default option set name.

- optionEditCommand – Command name which can be invoked from the dialog.
- supportsUpdate – Indicates that the generator supports updating an existing destination design. (Default is: False)
- usesPinSides – Indicates whether this generator uses the side information from pin specs. (Default is: False)

Returns

None

registerInterfaceReader

Syntax

```
registerInterfaceReader(  
    name: str,  
    /,  
    *,  
    viewType: str)  
Returns: None
```

Summary:

Register the interface reader for the specified view type

Category:

Data Translation

Licenses:

None

Parameters

- name – Tcl procedure name to call to retrieve the interface set object.
- viewType – The source viewtype name supported by the reader.

Returns

None

registerNetlistInfo

Syntax

```
registerNetlistInfo(  
    netlister: str,
```

```
/*  
*,  
properties: str | None = None)  
Returns: None
```

Summary:

Register Netlist

Category:

Foundation

Licenses:

None

Parameters

- netlist – Name of netlist to register
- properties – Registered set of default cdf properties (Default is: 'componentName instParameters namePrefix netlistProcedure permuteRule propMapping termOrder')

Returns

None

registerParamEvalFunction

Syntax

```
registerParamEvalFunction(  
none: str,  
/*  
*,  
cmdName: str,  
numArgs: int | None = None,  
numRequiredArgs: int | None = None,  
evalArgs: bool | None = None)  
Returns: None
```

Summary:

Register user defined PEL function

Category:

Foundation

Licenses:

None

Parameters

- none – Name of Tcl command implementing the PEL command
- cmdName – Name of PEL command
- numArgs – Maximum number of arguments (Default is: 0)
- numRequiredArgs – Number of required arguments
- evalArgs – Specifies if the arguments should be evaluated (Default is: True)

Returns

None

relativePath

Syntax

```
relativePath(  
  Path:  str,  
  /,  
  *,  
  base:  str)  
Returns: str
```

Summary:

Makes paths relative to the base path

Category:

Foundation

Licenses:

None

Parameters

- Path – Path, which should be made relative
- base – Path, which the result path should be relative to

Returns

Non-common suffix portion of the given paths, which is relative to the base path

releaseFeature

Syntax

```
releaseFeature(  
    name: str,  
    /,  
    *,  
    implicit: bool | None = None)  
Returns: None
```

Note that the license name is an internally defined license name which may refer to one or more features on the actual license server. Use `db::getFeatures` to determine which features can be checked out.

Summary:

Releases a licensing feature back to the license pool. If the feature was not previously checked out, the command does nothing. If the feature is not recognized, an error is thrown.

Category:

Foundation

Licenses:

None

Parameters

- `name` – Name of the licensing feature to release.
- `implicit` – If true the implicit check out count is decremented. If the count is reduced to zero the feature is released. (Default is: False)

Returns

None

remove

Syntax

```
remove(  
    object: attrObject | dbIterator,  
    /,  
    *,  
    from_: dbIterator = None,  
    of: dbIterator = None)  
Returns: dbIterator
```

Summary:

Removes an object from collection.

Category:

Foundation

Licenses:

None

Parameters

- object – An object or collection to remove.
- of – A dbCollection from which to remove element.

Returns

A dbCollection on which operated.

removeFigGroupMembers

Syntax

```
removeFigGroupMembers (  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia],  
/  
*,  
group: deFigure | oaFigGroup | str,  
design: oaDesign = None)  
Returns: None
```

Summary:

Destroys the FigGroupMem thereby removing its Fig from its FigGroup. It does not destroy the Figure.

Category:

Database

Licenses:

None

Parameters

- objects – The figure to be removed.
- group – The FigGroup from which the figure should be removed.
- design – The Design in which the group belongs.

Returns

None

removeMarkerObject

Syntax

```
removeMarkerObject(  
objects: oaBlockObject | list[deFigure] | list[deSelectionSet] |  
list[oaFig] | list[oaFigGroup] | list[oaInst] | list[oaMarker] |  
list[oaShape] | list[oaVia],  
/,  
*,  
marker: deFigure | oaMarker)  
Returns: None
```

Summary:

Removes the specified objects from the list of objects associated with this marker.

Category:

Layout Editing

Licenses:

None

Parameters

- objects – The object(s) to remove from marker
- marker – The marker of the object(s)

Returns

None

renderTemplatesParasitics

Syntax

```
renderTemplatesParasitics(  
    *,  
    toFolder: str,  
    design: oaDesign,  
    templates: str = None)  
Returns: str
```

Summary:

Creates parasitic DSPF files for all templates in the design.

Category:

Templates Category

Licenses:

None

Parameters

- toFolder – Folder to output the DSPF files.
- design – Design for which the files are generated.
- templates – List of templates 'l/c/v' for which must be rendered parasitic templates

Returns

Return generated parasitic template files

reopenDesign

Syntax

```
reopenDesign(  
    design: oaDesign,  
    /,  
    *,  
    editable: bool)  
Returns: None
```

Summary:

Reopen design

Category:

Database

Licenses:

None

Parameters

- design – oaDesign option
- editable – Open design in read/write mode

Returns

None

reportCallbacks

Syntax

```
reportCallbacks(  
pattern: str | None = None,  
/,  
*,  
regexp: bool | None = None,  
event: str | None = None,  
disable: bool | None = None,  
file: str | None = None)  
Returns: None
```

Summary:

reportCallbacks

Category:

Foundation

Licenses:

None

Parameters

- pattern – Pattern to match callback names (Default is: '*')
- regexp – If true, pattern is regular expression. If false, it is a glob-type pattern (Default is: False)
- event – Event for which to return registered callback names (Default is: 'all')

- `disable` – If true, reporting is disabled for specified events (Default is: False)
- `file` – Output file for reporting (Default is: “”)

Returns

None

reset

Syntax

```
reset(  
    collection: dbIterator)  
Returns: None
```

Summary:

Resets the iterator of a dbCollection to the start of the associated collection.

Category:

Foundation

Licenses:

None

Parameters

`collection` – An iterator over the dbCollection.

Returns

None

resetVDRCDefinedDevices

Syntax

```
resetVDRCDefinedDevices(  
    *,  
    targetLib: oaLib | str)  
Returns: None
```

Summary:

Reset all VDRC user-defined devices

Category:

Database

Licenses:

None

Parameters

targetLib – The target library to reset all user-defined devices

Returns

None

resolve

Syntax

```
resolve(  
    path: str | None = None,  
    /,  
    *,  
    includeResources: bool | None = None)  
Returns: None
```

Summary:

Given a path relative to any startup directory, returns all matching absolute paths from any startup directory

Category:

Foundation

Licenses:

None

Parameters

- path – Path relative to any startup directory. (Default is: '.')
- includeResources – Return resource path if available instead of absolute path (Default is: False)

Returns

None

resolvePath

Syntax

```
resolvePath(  
  string: str,  
  //,  
  *,  
  relative: str | None = None,  
  resolveEnvVars: bool | None = None,  
  resolveSymLinks: bool | None = None)  
Returns: str
```

Summary:

Resolves the relative path to absolute

Category:

Foundation

Licenses:

None

Parameters

- string – Given path to resolve
- relative – initial path when a relative path is specified
- resolveEnvVars – Whether environment variable needs to be resolved (Default is: True)
- resolveSymLinks – Whether symbolic link needs to be resolved (Default is: False)

Returns

Resolved path

revertParam

Syntax

```
revertParam(  
  param: dbParam | str,  
  //,  
  *,  
  of: oaRef = None)  
Returns: None
```

Summary:

Remove local override value for given parameter

Category:

Foundation

Licenses:

None

Parameters

- param – The parameter or name of parameter to revert
- of – The named parameter will be reverted for this object (Default is: 0)

Returns

None

revertPrefs

Syntax

```
revertPrefs(  
  dbPref: dbCollection)  
Returns: dbCollection
```

Summary:

Reverts the set of preferences to their startup values.

Category:

Foundation

Licenses:

None

Parameters

dbPref – Collection of preferences to revert

Returns

Collection of modified preferences

saveCDF

Syntax

```
saveCDF(  
  data: dmCell | oaLib,  
  /,  
  *,  
  close: bool = None,  
  level: db.saveCDF.Level = None)  
Returns: None
```

Summary:

Save and Close CDF for a cell or library

Category:

Foundation

Licenses:

None

Parameters

- data – Specifies the location to store the CDF as oaDMDData
- close – Specifies whether to close the oaDMDData after saving the CDF (Default is: True)
- level – The CDF level to be saved ({ base | overlay } (Default is: 'overlay'))

Returns

None

saveDesign

Syntax

```
saveDesign(  
  design: oaDesign)  
Returns: None
```

Summary:

Save design

Category:

Database

Licenses:

None

Parameters

design – OA Design object

Returns

None

saveDesignAs

Syntax

```
saveDesignAs (  
    design: oaDesign,  
    /,  
    *,  
    destination: dmCellView | dmCellViewName,  
    overwrite: bool = None)  
Returns: dmCellView
```

Summary:

Save design as

Category:

Database

Licenses:

None

Parameters

- design – OA Design object
- destination – Destination where to save design
- overwrite – Overwrite existing design. (Default is: True)

Returns

Saved design

saveDeviceDisplays

Syntax

```
saveDeviceDisplays(  
    empty: list[dbDeviceDisplay],  
    /,  
    *,  
    filePath: str,  
    force: bool = None)  
Returns: None
```

Summary:

This command saves the label display information of the current session to an XML file.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- empty – List of objects defining state of device label display to save.
- filePath – File in which to save the display information. Assumed relative to current working directory if not absolutely specified.
- force – Overwrite existing files. (Default is: False)

Returns

None

saveGenOptionSets

Syntax

```
saveGenOptionSets(  
    none: list[dbGenOptionSet],  
    /,  
    *,  
    filePath: str,  
    force: bool = None)  
Returns: None
```

Summary:

Saves the option set.

Category:

Data Translation

Licenses:

None

Parameters

- none – Collection of generator option set objects to save to disk.
- filePath – Name of file to store option set information.
- force – Overwrite the existing file. (Default is: True)

Returns

None

savePrefs**Syntax**

```
savePrefs(  
    preferences: dbCollection,  
    /,  
    *,  
    filePath: str = None)  
Returns: None
```

Summary:

Saves a collections of supplied preferences.

Category:

Foundation

Licenses:

None

Parameters

- preferences – Collection of preference objects which were localized in the given scope
- filePath – A file to save preferences to. (Default is: “")

Returns

None

sciToEng

Syntax

```
sciToEng(  
    input: str,  
    /,  
    *,  
    precision: int = None,  
    suffix: db.sciToEng.Suffix = None,  
    **kwargs)  
Returns: str
```

Summary:

Convert double/string values to engineering notation

Category:

Foundation

Licenses:

None

Parameters

- input – input number or string
- precision – Precision used for result (Default is: 6)
- suffix – Suffix for the converted value ({ E | G | K | M | MEG | Meg | N | P | T | U | X | Y | Z | a | auto | c | f | g | k | m | meg | n | p | u | y | z | {} })

Returns

Result of converting input to a string using Engineering suffixes

setAttr

Syntax

```
setAttr(  
    name: dbVarAttrName,  
    /,  
    *,  
    of: dbCollection | dbIterator = None,  
    value: variant,  
    type: db.setAttr.Type = None)  
Returns: None
```

Summary:

Sets the value of the specified attribute on the given object.

Category:

Foundation

Licenses:

None

Parameters

- **name** – The attribute name or hierarchical sequence of names separated by periods to set.
- **of** – The objects on which the attribute is set.
- **value** – The attribute value to set.
- **type** – Filters attributes to only those of the specified type. ({ all | appDef | intrinsic | param | prop }) (Default is: 'all')

Returns

None

setDeviceDisplay

Syntax

```
setDeviceDisplay(  
*,  
  window: int = None,  
  of: dmCellView | oaDesign | oaInst | oaInstHeader | oaLib,  
  in_: oaDesign = None,  
  terms: str = None,  
  name: str = None,  
  params: str = None)  
Returns: dbDeviceDisplay
```

Summary:

This command creates the dbDeviceDisplay object within the current session. If there is an existing dbDeviceDisplay object that references the current instance/master/library/cellview/design, it will be overwritten by the newly created object

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- window – Window Id.
- of – Indicates the object associated with the given display object: oaInst - instance scope; oaInstHeader - design scope; oaLib - library scope dmCellView or oaDesign with '-in' argument - design scope, dmCellView or oaDesign without '-in' argument - master scope.
- in_ – Parent design used for design scoping when -of argument is a dmCellView or oaDesign.
- terms – Indicates if the cdsTerm labels of the instance(s) evaluate to the corresponding instTerm's name or net attribute.
- name – Indicates if the cdsName label of the instance(s) evaluate to the instances name or cellName attribute.
- params – Indicates the list of parameters for cdsParam evaluation. If the parameter name is prepended with '-', the value only will be displayed.

Returns

Object defining state of device label display

setInstName

Syntax

```
setInstName(  
    name: str,  
    /,  
    *,  
    of: oaInst)  
Returns: None
```

Summary:

Sets the name of instance.

Category:

Design Editing

Licenses:

Basic or Frontend

Parameters

- name – A new name to be set.
- of – The instance whose name will be changed.

Returns

None

setLicenseTier

Syntax

```
setLicenseTier(  
  tier: db.setLicenseTier.Tier)  
Returns: None
```

Summary:

Change the current tier.

Category:

Foundation

Licenses:

None

Parameters

tier – Tier name to set. ({ advanced | advancedPlus | apex | backend | basic | elite | foundation | frontend | mtm | router | schematic })

Returns

None

setMaster

Syntax

```
setMaster(  
  cell: dmCellView,  
  /,  
  *,  
  of: oaInst)  
Returns: None
```

Summary:

Sets the master cell of the instance.

Category:

Design Editing

Licenses:

Basic or Frontend

Parameters

- cell – A new cellview to be set.
- of – The instance whose master will be changed.

Returns

None

setNetlistInfo**Syntax**

```
setNetlistInfo(  
name: str,  
/,  
*,  
data: dmCell | oaCellDMDData | oaLib | oaLibDMDData,  
netlist: str = None,  
info: str = None,  
section: db.setNetlistInfo.Section = None,  
value: str = None,  
type: db.setNetlistInfo.Type = None,  
clear: bool = None,  
level: db.setNetlistInfo.Level = None)  
Returns: None
```

Summary:

Set Netlist info Property for a cell

Category:

Foundation

Licenses:

None

Parameters

- **name** – The name used to specify the data field.
- **data** – DMDData (or a cell or library that contains one) that receives the specified Netlist Info.
- **netlister** – Netlister name, required only when -section is 'netlist'. (Default is: "")
- **info** – Name of the subsection, only allowed when -section is 'netlist'. (Default is: 'simInfo')
- **section** – Section of the netlist info that receives the specified Netlist Info. ({ display | form | netlist }) (Default is: 'netlist')
- **value** – The value to be associated with the specified name.
- **type** – The type of value. ({ floating | integer | list | real | string | symbol }) (Default is: 'symbol')
- **clear** – Whether to clear the Netlist Info value.
- **level** – The CDF level at which the value will be set ({ base | memory | overlay }) (Default is: 'overlay')

Returns

None

setParamFilter

Syntax

```
setParamFilter(  
tools: str,  
/,  
*,  
windowType: giWindowType | str)  
Returns: None
```

Summary:

Set list of Netlisters active in Parameter registry

Category:

Foundation

Licenses:

None

Parameters

- tools – Names of netlisters to register
- windowType – Window Type

Returns

None

setParamValue

Syntax

```
setParamValue(  
  name: str,  
  /,  
  *,  
  value: str,  
  type: db.setParamValue.Type = None,  
  of: oaRef,  
  evalCallbacks: bool = None)  
Returns: oaAssignValue | oaParam | oaProp
```

Summary:

Set parameter value

Category:

Foundation

Licenses:

None

Parameters

- name – Parameter name
- value – Parameter value
- type – Parameter Value Type ({ application | boolean | float | integer | real | string | time } (Default is: 'string'))
- of – Object for which parameter value is to be set
- evalCallbacks – Should callback be triggered or no (Default is: True)

Returns

The object created/modified by db::setParamValue.

setPrefValue

Syntax

```
setPrefValue(  
name: dbDeferredPrefName,  
/,  
*,  
value: str,  
scope: dbScope | deContext | dmCell | dmCellView | oaLib |  
str = None)  
Returns: dbPref
```

Summary:

Sets a preference value in the specified scope.

Category:

Foundation

Licenses:

None

Parameters

- name – Preference name. Note, it is not required for the preference to exist.
- value – Set the preference to this value.
- scope – Optionally set the value at this scope

Returns

Preference object which was modified by the command

setProgress

Syntax

```
setProgress(  
position: int,  
/,  
*,  
label: str = None,  
of: dbProgress)  
Returns: None
```

Summary:

Updates the current position in a dbProgress and redraws its progress bar, if shown.

Category:

Foundation

Licenses:

None

Parameters

- position – New position.
- label – Updates current suboperation label (Default is: “)
- of – Progress to update.

Returns

None

setViaDef**Syntax**

```
setViaDef(  
    def_: oaViaDef,  
    /,  
    *,  
    of: oaVia)  
Returns: None
```

Summary:

Sets the definition of via.

Category:

Design Editing

Licenses:

Basic or Frontend

Parameters

- def_ – A new definition to be set.
- of – The via whose definition will be changed.

Returns

None

showAbstractGeneratorOptions

Syntax

```
showAbstractGeneratorOptions(  
*,  
    cellView: dmCellView = None,  
    design: oaDesign = None,  
    window: giWindow | int = None,  
    parent: giDialog = None,  
    optionSet: str = None)  
Returns: giDialog
```

Summary:

Shows parameters sets

Category:

Layout Editing

Licenses:

Basic

Parameters

- cellView – The source cellView.
- design – The source design.
- window – Window or window id with source design context.
- parent – Parent dialog.
- optionSet – Option Set name. (Default is: 'default')

Returns

Dialog containing abstract generator options editor

showAdvancedPrintOptions

Syntax

```
showAdvancedPrintOptions(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Dispalys advanced print options

Category:

Printing and Images

Licenses:

None

Parameters

parent – The window that this dialog is associated.

Returns

The dialog

showCellViewFromCellView

Syntax

```
showCellViewFromCellView(  
*,  
    parent: giWindow | int = None,  
    src: dmCellView | dmCellViewName = None,  
    dest: dmCellView | dmCellViewName = None,  
    optionSet: str = None,  
    defaultEditor: str = None)  
Returns: giDialog
```

Summary:

Creates a cellview from the given cellview.

Category:

Data Translation

Licenses:

None

Parameters

- parent – Parent window.
- src – The source cellview.
- dest – The destination cellview.

- optionSet – The Optional Set.
- defaultEditor – The default editor. (Default is: 'dbGenerateCellviewEditor')

Returns

Dialog Window

showCellViewFromPinList

Syntax

```
showCellViewFromPinList(  
*,  
    parent: giWindow | int = None,  
    dest: dmCellView | dmCellViewName = None,  
    defaultEditor: str = None)  
Returns: giDialog
```

Summary:

Creates a cellview from the pin list entered by user.

Category:

Data Translation

Licenses:

None

Parameters

- parent – Parent window.
- dest – The destination cellview.
- defaultEditor – The default editor. (Default is: 'dbGenerateCellviewEditor')

Returns

Dialog Window

showExportDef

Syntax

```
showExportDef()  
Returns: giDialog
```

Arguments:

Summary:

Shows Export DEF dialog

Category:

Data Translation

Licenses:

None

Returns

Write DEF Dialog

showExportImage

Syntax

```
showExportImage (  
    *,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Dispalys Export Image Dialog

Category:

Printing and Images

Licenses:

None

Parameters

parent – The window that this dialog is associated.

Returns

The dialog

showExportLef

Syntax

```
showExportLef()  
Returns: giDialog
```

Arguments:

Summary:

Shows Export LEF dialog

Category:

Data Translation

Licenses:

None

Returns

Write LEF Dialog

showExportMCellFile

Syntax

```
showExportMCellFile(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows Export MCell Dialog

Category:

Data Management

Licenses:

Basic

Parameters

parent – parent widget

Returns

Export MCell Dialog

showExportOasis

Syntax

```
showExportOasis(  
*,  
    parent: giWindow | int = None,  
    **kwargs)  
Returns: giDialog
```

Summary:

Shows Export oasis dialog

Category:

Data Translation

Licenses:

None

Parameters

parent – Parent window on which this dialog will be centered.

Returns

OASIS Out Dialog

showExportStream

Syntax

```
showExportStream(  
    **kwargs)  
Returns: giDialog
```

Arguments:

Summary:

Shows Export Stream dialog

Category:

Data Translation

Licenses:

None

Returns

Stream Out Dialog

showFindReplace

Syntax

```
showFindReplace(  
*,  
    parent: giWindow | int = None,  
    cellView: dmCellView = None,  
    instPath: str = None,  
    viewSearchList: str = None,  
    viewStopList: str = None,  
    startLevel: int = None,  
    stopLevel: int = None,  
    depth: int = None)  
Returns: None
```

Summary:

Show a find_replace main dialog.

Category:

Design Editing

Licenses:

None

Parameters

- parent – The window to use for browsing and highlighting search results.
- cellView – The top cell view to use.
- instPath – The occurrence path used to limit hierarchical searches. (Default is: "")
- viewSearchList – The view search list used for hierarchical descent. (Default is: "")
- viewStopList – The view stop list used for hierarchical descent. (Default is: "")
- startLevel – The start level of a hierarchical search. (Default is: 0)
- stopLevel – The stop level of a hierarchical search. (Default is: 32)
- depth – The hierarchical descent depth limit. (Default is: 32)

Returns

None

showGenerateAbstract

Syntax

```
showGenerateAbstract()  
Returns: giDialog
```

Arguments:

Summary:

Shows Generate Abstract dialog.

Category:

Layout Editor

Licenses:

Basic

Returns

Dialog Window

showGenerateLibertyFile

Syntax

```
showGenerateLibertyFile()  
Returns: None
```

Arguments:

Summary:

Shows Genarte Liberty file dialog. This command is defunct!

Category:

Data Translation

Licenses:

None

Returns

None

showGenerateSchematic

Syntax

```
showGenerateSchematic()  
Returns: giDialog
```

Arguments:

Summary:

Shows Schematic Generation dialog

Category:

Data Translation

Licenses:

None

Returns

Schematic Generation Dialog

showGenericPrint

Syntax

```
showGenericPrint(  
*,  
parent: giDialog | giWindow | int = None)  
Returns: giDialog
```

Summary:

Displays Generic Print dialog

Category:

Printing and Images

Licenses:

None

Parameters

parent – The window or dialog that this dialog is associated.

Returns

The dialog

showImportDef

Syntax

```
showImportDef()  
Returns: giDialog
```

Arguments:

Summary:

Shows Import DEF dialog

Category:

Data Translation

Licenses:

None

Returns

Read DEF Dialog

showImportLef

Syntax

```
showImportLef()  
Returns: giDialog
```

Arguments:

Summary:

Shows Import LEF dialog

Category:

Data Translation

Licenses:

None

Returns

Read LEF Dialog

showImportMCellFile

Syntax

```
showImportMCellFile(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows Import MCell Dialog

Category:

Data Management

Licenses:

Basic

Parameters

parent – parent widget

Returns

Import MCell Dialog

showImportOasis

Syntax

```
showImportOasis(  
*,  
    parent: giWindow | int = None,  
    **kwargs)  
Returns: giDialog
```

Summary:

Shows Import OASIS dialog

Category:

Data Translation

Licenses:

None

Parameters

parent – Parent window on which this dialog will be centered.

Returns

Oasis in Dialog

showImportSimulationDataDialog

Syntax

```
showImportSimulationDataDialog(  
*,  
    window: int,  
    context: deContext)  
Returns: None
```

Summary:

Invoke import simualtion data dialog

Category:

Database

Licenses:

Frontend

Parameters

- window – The window to invoke the dialog
- context – The context design

Returns

None

showImportStream

Syntax

```
showImportStream()  
Returns: giDialog
```

Arguments:

Summary:

Shows Import STREAM dialog

Category:

Data Translation

Licenses:

None

Returns

Stream in Dialog

showImportText

Syntax

```
showImportText()  
Returns: giDialog
```

Arguments:

Summary:

Shows Import Text dialog

Category:

Data Translation

Licenses:

None

Returns

Import Text in Dialog

showImportVerilog

Syntax

```
showImportVerilog()  
Returns: giDialog
```

Arguments:

Summary:

Shows Import Verilog dialog

Category:

Data Translation

Licenses:

None

Returns

Verilog in Dialog

showLoadPreferences

Syntax

```
showLoadPreferences(  
*,  
    scope: dbScope | deContext | str = None)  
Returns: None
```

Summary:

Shows the Import Preferences dialog. If the dialog is already opened, it will be raised and the existing object returned.

Category:

Foundation

Licenses:

None

Parameters

scope – The scope from which to retrieve the preference value.

Returns

None

showManageTechnology

Syntax

```
showManageTechnology(  
*,  
    lib: oaLib = None,  
    libName: str = None)  
Returns: giDialog
```

Summary:

show manage technology

Category:

Data Management

Licenses:

None

Parameters

- lib – OA library from/to which technology file will be exported/imported.
- libName – Name of OA library, from/to which technology data will be exported/imported.

Returns

Dialog to manage technology

showPrint

Syntax

```
showPrint(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open Print Dialog

Category:

Printing and Images

Licenses:

None

Parameters

parent – The window that this dialog is associated.

Returns

The dialog

showPrintQueueStatus

Syntax

```
showPrintQueueStatus()  
Returns: giWindow
```

Arguments:

Summary:

Dispaly Print queue status window

Category:

Printing and Images

Licenses:

None

Returns

The 'Print Queue Status' window

showPurgePreferences

Syntax

```
showPurgePreferences(  
*,  
    scope: dbScope | deContext | str = None)  
Returns: None
```

Summary:

Shows the Purge Preferences dialog. If the dialog is already opened, it will be raised and the existing object returned.

Category:

Foundation

Licenses:

None

Parameters

scope – The scope from which to retrieve the preference value.

Returns

None

showRefreshMCells

Syntax

```
showRefreshMCells(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows Refresh MCell Dialog

Category:

Data Management

Licenses:

Basic

Parameters

parent – parent widget

Returns

Refresh MCell Dialog

showSavePreferences

Syntax

```
showSavePreferences(  
*,  
    scope: dbScope | deContext | str = None)  
Returns: None
```

Summary:

Shows the Export Preferences dialog. If the dialog is already opened, it will be raised and the existing object returned.

Category:

Foundation

Licenses:

None

Parameters

scope – The scope from which to retrieve the preference value.

Returns

None

showUnitDisplayScale

Syntax

```
showUnitDisplayScale(  
*,  
    parent: giDialog | giWindow | int)  
Returns: giDialog
```

Summary:

Show unit display scale dialog

Category:

Foundation

Licenses:

None

Parameters

parent – Parent Window(id).

Returns

Created Unit Display Scale

snapPoints

Syntax

```
snapPoints(  
points: list[point],  
/  
*,  
design: oaDesign = None,  
offset: point,  
spacing: point,  
round: db.snapPoints.Round = None)  
Returns: point
```

Summary:

Snap points based on the given offset point and spacing.

Category:

Foundation

Licenses:

None

Parameters

- points – The list of points that form snap points.
- design – The design to get DBUPerUU.
- offset – The distance between the origin of the manufacturing grid and the origin of the snap grid.
- spacing – The distance between snap coordinates.
- round – Rounding method ({ down | rounding | up } (Default is: 'rounding'))

Returns

The snapped points

sort

Syntax

```
sort(  
  collection: dbIterator,  
  /,  
  *,  
  attr: str,  
  reverse: bool = None)  
Returns: dbCollection | dbIterator
```

Summary:

Creates a new dbCollection which contains items from another collection in sorted order.

Category:

Foundation

Licenses:

None

Parameters

- collection – A list of objects which will be converted.
- attr – Object attribute to sort by.
- reverse – Sort in reverse order. (Default is: False)

Returns

New collection in sorted order.

source

Syntax

```
source(  
  filePath: str)  
Returns: None
```

Summary:

Source a TCL or Synenc-encoded TCL command file

Category:

Foundation

Licenses:

None

Parameters

filePath – TCL file

Returns

None

sourceEnc

Syntax

```
sourceEnc(  
    filename: str  
Returns: None
```

Summary:

sources a file encrypted in the Laker format

Category:

DRD

Licenses:

None

Parameters

filename – File to check if encrypted

Returns

None

startStatistics

Syntax

```
startStatistics(  
    *,  
    openhier: bool | None = None,  
    designopen: bool | None = None,  
    oainit: bool | None = None,  
    rqinit: bool | None = None,  
    render: bool | None = None,  
    pcellcaching: bool | None = None,
```

```
file: str | None = None)
Returns: None
```

Summary:

Start statistics gathering.

Category:

Foundation

Licenses:

None

Parameters

- openhier – If true, time the total open hierarchy which includes design open and init (Default is: True)
- designopen – If true, time the opening of oa designs (Default is: True)
- oainit – If true, time the OA init dynamic tables (Default is: True)
- rqinit – If true, time the RQ initialization (Default is: True)
- render – If true, time the first draw of the render after hierarchical open (Default is: True)
- pcellcaching – If true, time Pcell caching (Default is: False)
- file – Output to the given file.

Returns

None

stopStatistics

Syntax

```
stopStatistics()
Returns: None
```

Arguments:

Summary:

Stop statistics gathering.

Category:

Foundation

Licenses:

None

Returns

None

transform

Syntax

```
transform(  
points: list[point],  
/,  
*,  
occurrence: oaHierPath | str,  
design: oaDesign,  
inverse: bool = None)  
Returns: point
```

Summary:

Transforms the specified points with the transform from specified instances.

Category:

Database

Licenses:

None

Parameters

- points – The list of points that form transformed input.
- occurrence – Instance hierarchical path to transform.
- design – Design in which to create the transformed input.
- inverse – Specify whether transform top points to design in hierarchy. (Default is: False)

Returns

Transformed User Points

triggerParamCallback

Syntax

```
triggerParamCallback(  
    param: dbParam)  
Returns: str
```

Summary:

Triggers the change callback for a parameter

Category:

Foundation

Licenses:

None

Parameters

param – The input parameter

Returns

The evaluated value of the callback string

unregisterGenerator

Syntax

```
unregisterGenerator(  
    *,  
    for_: str,  
    from_: str | None = None,  
    of: str | None = None)  
Returns: None
```

Summary:

Unregister cellview from cellview generator.

Category:

Data Translation

Licenses:

None

Parameters

- `for_` – The destination viewtype name for the generator.
- `of` – The source viewtype name for the generator.

Returns

None

unregisterInterfaceReader

Syntax

```
unregisterInterfaceReader(  
    *,  
    viewType: str)  
Returns: None
```

Summary:

Unregister an interface reader.

Category:

Data Translation

Licenses:

None

Parameters

`viewType` – The viewtype name for the reader to deregister.

Returns

None

updateCDFParams

Syntax

```
updateCDFParams(  
    cell: dmCell)  
Returns: None
```

Summary:

Synchronizes CDF parameter information of given cell with the parameters of its schematic and text views.

Category:

Data Translation

Licenses:

None

Parameters

cell – Cell for which CDF parameter information should be updated.

Returns

None

vdrcMgrAddNet

Syntax

```
vdrcMgrAddNet (  
*,  
    window: giWindow | int,  
    context: heContext,  
    name: str = None,  
    names: str = None,  
    all: bool = None,  
    voltageRange: str = None)  
Returns: None
```

Summary:

Add Net to VDRC Manager

Category:

Database

Licenses:

None

Parameters

- window – VDRC Manager Window
- context – Schematic Hierarchy Context
- name – Net Name to Add to VDRC Window
- names – Net Name Patterns to Add to VDRC Window

- all – Add all schematic nets to VDRC Window
- voltageRange – Add nets with Simulation Voltage Range to VDRC Window

Returns

None

vdrcMgrAddNetPair

Syntax

```
vdrcMgrAddNetPair(  
*,  
    window: giWindow | int,  
    context: heContext,  
    name: str,  
    udv: bool)  
Returns: None
```

Summary:

Add Net Pair to VDRC Manager

Category:

Database

Licenses:

None

Parameters

- window – VDRC Manager Window
- context – Schematic Hierarchy Context
- name – Net Pair Name to Add to VDRC Window
- udv – Is the Net Pair UDV Type, true is UDV, false is Vsync

Returns

None

vdrcMgrAnnotateNet

Syntax

```
vdrcMgrAnnotateNet(  
*,
```

```
    window: giWindow | int,  
    context: heContext,  
    names: str)  
Returns: None
```

Summary:

Annotate Simulation Value to Schematic

Category:

Database

Licenses:

None

Parameters

- window – VDRC Manager Window
- context – Schematic Hierarchy Context
- names – Net Names to Annotate to Schematic Design

Returns

None

vdrcMgrAnnotateNetPair

Syntax

```
vdrcMgrAnnotateNetPair(  
*,  
    window: giWindow | int,  
    context: heContext,  
    names: str,  
    types: str,  
    userValue: float = None)  
Returns: None
```

Summary:

Annotate Simulation Value or User Defined Value to Schematic Design

Category:

Database

Licenses:

None

Parameters

- window – VDRC Manager Window
- context – Schematic Hierarchy Context
- names – Net Pair Names to Annotate to Schematic Design
- types – Net Pair Types to Annotate to Schematic Design
- userValue – User defined value to Annotate to Schematic Design

Returns

None

vdrcMgrCheckHierarchyMismatch

Syntax

```
vdrcMgrCheckHierarchyMismatch(  
*,  
    window: giWindow | int)  
Returns: None
```

Summary:

Update Hierarchy Mismatch Reports to VDRC Manager

Category:

Database

Licenses:

None

Parameters

window – VDRC Manager Window

Returns

None

vdrcMgrCheckSymbolPlacement

Syntax

```
vdrcMgrCheckSymbolPlacement(  
*,  
    window: giWindow | int,
```

```
context: deContext)
Returns: None
```

Summary:

Check Layout Symbol Placement

Category:

Database

Licenses:

None

Parameters

- window – VDRC Manager Window
- context – Design Context

Returns

None

vdrcMgrClearAllData

Syntax

```
vdrcMgrClearAllData (
*,
window: giWindow | int,
context: deContext = None)
Returns: None
```

Summary:

Clear All Data in the VDRC Manager

Category:

Database

Licenses:

None

Parameters

- window – VDRC Manager Window
- context – Layout design Context

Returns

None

vdrcMgrDeleteNet

Syntax

```
vdrcMgrDeleteNet(  
*,  
    window: giWindow | int,  
    name: str)  
Returns: None
```

Summary:

Delete Net in VDRC Manager

Category:

Database

Licenses:

None

Parameters

- window – VDRC Manager Window
- name – Net Name to Delete from VDRC Window

Returns

None

vdrcMgrDeleteNetPair

Syntax

```
vdrcMgrDeleteNetPair(  
*,  
    window: giWindow | int,  
    name: str,  
    udv: bool)  
Returns: None
```

Summary:

Remove Net Pair in VDRC Manager

Category:

Database

Licenses:

None

Parameters

- window – VDRC Manager Window
- name – Net Pair Name to Delete from VDRC Window
- udv – Is the Net Pair UDV Type, true is UDV, false is Vsync

Returns

None

vdrcMgrImportLayout

Syntax

```
vdrcMgrImportLayout(  
*,  
    window: giWindow | int,  
    context: deContext,  
    hierarchy: bool = None)  
Returns: None
```

Summary:

Import Layout Data to VDRC Manager

Category:

Database

Licenses:

None

Parameters

- window – VDRC Manager Window
- context – Design Context
- hierarchy – Import all hierarchies of layout designs

Returns

None

vdrcMgrImportSchematic

Syntax

```
vdrcMgrImportSchematic(  
*,  
    window: giWindow | int,  
    context: deContext | heContext,  
    hierarchy: bool = None)  
Returns: None
```

Summary:

Import Schematic Data to VDRC Manager

Category:

Database

Licenses:

None

Parameters

- window – VDRC Manager Window
- context – Design or Hierarchy Context of schematic design
- hierarchy – Import all hierarchies of schematic designs

Returns

None

vdrcMgrRefresh

Syntax

```
vdrcMgrRefresh(  
*,  
    window: giWindow | int = None)  
Returns: None
```

Summary:

Import Schematic Data to VDRC Manager

Category:

Database

Licenses:

None

Parameters

window – VDRC Manager Window to refresh. Refresh all the VDRC Manager Windows if no window specified.

Returns

None

viaDefExists

Syntax

```
viaDefExists(  
    name: str,  
    /,  
    *,  
    of: oaTech | oaVia,  
    local: bool = None)  
Returns: None
```

Summary:

Checks if a via definition with specified name exists in the specified technology.

Category:

Design Editing

Licenses:

None

Parameters

- name – The name of definition to be checked.
- of – The via or technology which will be checked.
- local – Search only in local technology or not. (Default is: True)

Returns

None

viewExists

Syntax

```
viewExists(  
    name: str,  
    /,  
    *,  
    of: dmCell | oaInst)  
Returns: None
```

Summary:

Checks if a view with specified name exists for the specified cell.

Category:

Design Editing

Licenses:

None

Parameters

- name – The name of view to be checked.
- of – The cell or instance which will be checked.

Returns

None

vwait

Syntax

```
vwait(  
    name: str)  
Returns: None
```

Summary:

Block script execution until variable changes.

Category:

Foundation

Licenses:

None

Parameters

name – Variable name which should be monitored for changes.

Returns

None

snps.custom.cmd.idb

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform database operations.

For details, see:

- [query](#)

query

Syntax

```
query(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Starts the 'Query' command in the specified window.

Category:

Foundation

Licenses:

None

Parameters

context – Context of activation.

Returns

None

snps.custom.cmd.de

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform layout, schematic, and symbol editing, and design navigation.

For details, see:

- [abortCommand](#)
- [activateTools](#)
- [addCapability](#)
- [addObject](#)
- [addPoint](#)
- [addTemplateView](#)
- [addTool](#)
- [applyOLPPreset](#)
- [audit](#)
- [cancelCommand](#)
- [cancelTransaction](#)
- [clearHighlights](#)
- [clearUnusedDesigns](#)
- [clone](#)
- [close](#)
- [CommandInterface](#)
- [commandOption](#)
- [completeShape](#)
- [copy](#)
- [copyBookmark](#)
- [createActivationEngine](#)

- [copyClipboardToLib](#)
- [createApplication](#)
- [createArgument](#)
- [createArgumentGroup](#)
- [createAssociation](#)
- [createBookmark](#)
- [createCanvas](#)
- [createColorExpression](#)
- [createCommand](#)
- [createContext](#)
- [createCrossProbeTool](#)
- [createErrorViewerDatabase](#)
- [createErrorViewerMarker](#)
- [createFavoriteDevice](#)
- [createFavoriteDeviceCategory](#)
- [createGradient](#)
- [createHierObject](#)
- [createHighlightSelector](#)
- [createHighlightSet](#)
- [createInterfaceElement](#)
- [createLPPInput](#)
- [createLPPSet](#)
- [createNetHighlightSet](#)
- [createObjectFilter](#)
- [createOLPPreset](#)
- [createOnCanvasWidget](#)
- [createOverlay](#)

- [createProbe](#)
- [createSelectionEngine](#)
- [createShapeEngine](#)
- [createTemplate](#)
- [createTool](#)
- [crossCheckViews](#)
- [cut](#)
- [cycleActiveFigure](#)
- [cycleTapObject](#)
- [deactivateTools](#)
- [defineOrigin](#)
- [deleteBookmarks](#)
- [deprecateCommand](#)
- [descend](#)
- [descendInPlace](#)
- [descendIntoGroup](#)
- [deselect](#)
- [deselectAll](#)
- [deselectAllLogic](#)
- [deselectLogic](#)
- [enableColorAnnotator](#)
- [enablePartitioningDisplay](#)
- [enableRotationAndScaling](#)
- [endDrag](#)
- [endLog](#)
- [endTransaction](#)
- [errorViewerLoadResult](#)

- [exportBookmarks](#)
- [exportProbeCellSettings](#)
- [extractInterfaceElements](#)
- [findMessage](#)
- [finishEvalTextRendering](#)
- [fit](#)
- [getActiveCommand](#)
- [getActiveContext](#)
- [getActiveFigure](#)
- [getActiveLPP](#)
- [getApplications](#)
- [getAssociations](#)
- [getBackgroundImporters](#)
- [getBBox](#)
- [getBinsForGradient](#)
- [getBookmarks](#)
- [getBreakpoints](#)
- [getCapabilities](#)
- [getClipboardLibs](#)
- [getClips](#)
- [getColorAnnotators](#)
- [getCommands](#)
- [getContexts](#)
- [getCrossProbeTools](#)
- [getCursor](#)
- [getCursorSnapPoint](#)
- [getDBLimits](#)

- [getDesignTemplates](#)
- [getEditors](#)
- [getErrorViewerDatabases](#)
- [getFavoriteDeviceCategories](#)
- [getFavoriteDevices](#)
- [getFigures](#)
- [getHighlighted](#)
- [getHighlightSets](#)
- [getInterfaceElements](#)
- [getLayers](#)
- [getLookaheadMessages](#)
- [getLPPs](#)
- [getLPPSets](#)
- [getObjectFilters](#)
- [getOLPPresets](#)
- [getOpenFiles](#)
- [getOutdatedDesigns](#)
- [getOutdatedDMData](#)
- [getOverlays](#)
- [getPartitioner](#)
- [getProbeBitNames](#)
- [getProbeFigures](#)
- [getProbeNets](#)
- [getProbes](#)
- [getRecentFiles](#)
- [getSelected](#)
- [getSelectedLogic](#)

- [getSelectedVertex](#)
- [getTemplates](#)
- [getTools](#)
- [getTransactions](#)
- [getViewport](#)
- [gotoViewport](#)
- [help](#)
- [highlight](#)
- [importBookmarks](#)
- [importProbeCellSettings](#)
- [isEditable](#)
- [isInUndo](#)
- [isModified](#)
- [linkContext](#)
- [loadFavoriteDevices](#)
- [loadInterfaceElements](#)
- [loadOLPPresets](#)
- [loadProbes](#)
- [navigateTo](#)
- [next](#)
- [open](#)
- [openCanvasPropertyEditor](#)
- [openClipboardLib](#)
- [openEditor](#)
- [openFile](#)
- [pan](#)
- [paste](#)

- [redoSelection](#)
- [redoTransaction](#)
- [redraw](#)
- [registerHelp](#)
- [removeBreakpoints](#)
- [removeObject](#)
- [removePoint](#)
- [removeTool](#)
- [renumberInstances](#)
- [repeatCommand](#)
- [replay](#)
- [replayCommand](#)
- [reportOpenTransactions](#)
- [restoreBookmark](#)
- [return_](#)
- [revert](#)
- [run](#)
- [runBackgroundImporter](#)
- [save](#)
- [saveAs](#)
- [saveFavoriteDevices](#)
- [saveInterfaceElements](#)
- [saveOLPPresets](#)
- [saveProbes](#)
- [select](#)
- [selectAll](#)
- [selectLogic](#)

- [sendComplete](#)
- [sendMessage](#)
- [setActiveLPP](#)
- [setBreakpoint](#)
- [setCursor](#)
- [setEditMode](#)
- [setErrorViewerCategory](#)
- [setErrorViewerDetailInfo](#)
- [setErrorViewerFieldName](#)
- [setErrorViewerTip](#)
- [setViewport](#)
- [shiftCursor](#)
- [showAbout](#)
- [showBookmarkManager](#)
- [showCloseDesigns](#)
- [showConnectionsNavigator](#)
- [showConnectionsNavigatorOptions](#)
- [showCreateBookmark](#)
- [showCrossCheckOptions](#)
- [showDescendDialog](#)
- [showDesignsPrompt](#)
- [showGeneralOptions](#)
- [showMakeDesignsReadOnly](#)
- [showOpenDesign](#)
- [showParamFilter](#)
- [showProbeBitSelectionDialog](#)
- [showProperties](#)

- [showRefreshDesigns](#)
- [showRenumberInstances](#)
- [showReturnToLevelDialog](#)
- [showSaveDesignAs](#)
- [sleep](#)
- [snapToObject](#)
- [startDrag](#)
- [startLog](#)
- [startRendererStatistics](#)
- [startTransaction](#)
- [stop](#)
- [stopBackgroundImporter](#)
- [stopRendererStatistics](#)
- [suppressMessage](#)
- [transformPoint](#)
- [undoSelection](#)
- [undoTransaction](#)
- [unhighlight](#)
- [uniquify](#)
- [zoom](#)

abortCommand

Syntax

```
abortCommand(  
    name: str = None,  
    /,  
    *,  
    window: giWindow | int = None,  
    context: deContext = None)  
Returns: None
```

Summary:

Aborts the current user interactive command.

Category:

Foundation

Licenses:

None

Parameters

- name – The name of the command to cancel.
- window – Window in which to send the event.
- context – Context in which to send the event.

Returns

None

activateTools

Syntax

```
activateTools(  
tools: dbCollection,  
/,  
*,  
context: deContext)  
Returns: None
```

Summary:

Activates one or more tools within the specified context. If already activated command will not alter its state.

Category:

Foundation

Licenses:

None

Parameters

- tools – Collection of deTool objects to activate.
- context – Context in which to activate tools.

Returns

None

addCapability

Syntax

```
addCapability(  
  object: deCapability | str,  
  /,  
  *,  
  to: deLinkedContext,  
  enabled: bool = None)  
Returns: deCapability
```

Summary:

Add a deCapability to a deLinkedContext, thus enabling this capability in its primary design context.

Category:

Foundation

Licenses:

None

Parameters

- object – Capability to add.
- to – Linked context to modify.
- enabled – Optional initial enabled attribute. (Default is: True)

Returns

Newly added capability.

addObject

Syntax

```
addObject(  
  object: deFigure,  
  /,  
  *,  
  window: giWindow | int = None,
```

```
context: deContext = None)
Returns: None
```

Summary:

Add object event's equivalent command.

Category:

Foundation

Licenses:

None

Parameters

- object – Object to select.
- window – Window in which to send the event.
- context – Context in which to send the event.

Returns

None

addPoint

Syntax

```
addPoint(
point: point,
/,
*,
window: giWindow | int = None,
context: deContext = None,
relative: bool = None,
snap: bool = None)
Returns: None
```

Summary:

Add point event's equivalent command.

Category:

Foundation

Licenses:

None

Parameters

- point – Point to add, in user units
- window – Window in which to send the event.
- context – Context in which to send the event.
- relative – Specifies that this point is relative to the last entered point. (Default is: False)
- snap – Specifies that this point shall be snapped to angle mode and gravity. (Default is: True)

Returns

None

addTemplateView

Syntax

```
addTemplateView(  
viewName: str,  
/,  
*,  
template: deTemplate,  
view: dmCellView = None,  
data: dbCollection | list[deFigure] | list[oaFig] = None,  
sourceNetlist: dbCollection | list[oaFig] = None,  
staticClone: bool = None)  
Returns: None
```

Summary:

Command to add a new view to an existing template cell.

Category:

Templates Category

Licenses:

Frontend

Parameters

- viewName – Name of new view in the template cell.
- template – Template in which to add new view.
- view – Cellview to copy into the template.
- data – Data to copy into the new template view.

- `sourceNetlist` – Defines circuit structure of source.
- `staticClone` – If adding a `maskLayout` view, mark the new view for use as a static clone (Default is: `False`)

Returns

None

addTool

Syntax

```
addTool(  
    tool: deTool,  
    /,  
    *,  
    editor: deEditor | str = None,  
    workspace: giWorkspace = None)  
Returns: None
```

Summary:

Adds a tool to an association and thus makes the tool valid for designs opened with the given association

Category:

Foundation

Licenses:

None

Parameters

- `tool` – Tool to add to the given association.
- `editor` – Editor for which this tool is available.
- `workspace` – Workspace

Returns

None

applyOLPPreset

Syntax

```
applyOLPPreset(  
    title_or_index: str,  
    /,  
    *,  
    operation: de.applyOLPPreset.Operation = None,  
    to: deContext | oaDesign)  
Returns: None
```

Summary:

Applies the given deOLPPreset to the given design.

Category:

Layout Editing

Licenses:

None

Parameters

- title_or_index – OLP preset title or index
- operation – OLP presets operation: replace, or, not. ({ not | or | replace } (Default is: 'replace')
- to – Design or context which to apply settings from the given deOLPPreset

Returns

None

audit

Syntax

```
audit(  
    *,  
    filePath: str = None,  
    auditCommands: list[str] = None)  
Returns: str
```

Summary:

Returns a formatted report of all usages of deprecated interface elements in the given file or in all files in the startup paths of Custom Infrastructure.

Category:

Foundation

Licenses:

None

Parameters

- `filePath` – Script file to analyze
- `auditCommands` – Warnings are generated for each call to a command in the audit commands list.

Returns

Deprecated usages report

cancelCommand

Syntax

```
cancelCommand(  
*,  
    window: giWindow | int = None,  
    context: deContext = None)  
Returns: None
```

Summary:

Remove all points from a rubber shape

Category:

Foundation

Licenses:

None

Parameters

- `window` – Window in which to send the event.
- `context` – Context in which to send the event.

Returns

None

cancelTransaction

Syntax

```
cancelTransaction(  
    transaction: deTransaction)  
Returns: None
```

Summary:

Cancel Transaction

Category:

Foundation

Licenses:

None

Parameters

transaction – Transaction to cancel

Returns

None

clearHighlights

Syntax

```
clearHighlights(  
    *,  
    sets: list[deHighlightSet] = None,  
    design: oaDesign = None,  
    context: deContext = None,  
    clearPersistent: bool = None)  
Returns: None
```

Summary:

Clear all objects from highlight set or all highlight sets which contain objects of design.

Category:

Design Editing

Licenses:

None

Parameters

- sets – Highlight sets to clear
- design – Design for which all highlights will be cleared
- context – Context whose highlight sets will be cleared
- clearPersistent – Clear persistent highlight sets (Default is: False)

Returns

None

clearUnusedDesigns

Syntax

```
clearUnusedDesigns()  
Returns: None
```

Arguments:

Summary:

Removes unused designs from memory.

Category:

Foundation

Licenses:

None

Returns

None

clone

Syntax

```
clone(  
context: deContext,  
/,  
*,  
headless: bool = None)  
Returns: deContext
```

Summary:

Clones a Design Editor context and optionally clones the editor (if headless is false).

Category:

Foundation

Licenses:

None

Parameters

- context – Context object to clone.
- headless – Specifies that the new context will be headless (no window). (Default is: False)

Returns

Newly created context.

close

Syntax

```
close(  
  contexts: dbCollection)  
Returns: None
```

Summary:

Closes all open files in the context, discarding changes and destroys the context object.

Category:

Foundation

Licenses:

None

Parameters

contexts – Collection of context objects whose open design(s) will be closed.

Returns

None

CommandInterface

Class CommandInterface

Bases: `object`

Interactive command interface base class

commandOption

Syntax

```
commandOption(  
name: str,  
/,  
*,  
window: giWindow | int = None,  
context: deContext = None,  
point: point = None)  
Returns: None
```

Summary:

Command option event's equivalent command

Category:

Foundation

Licenses:

None

Parameters

- name – commandOption event type
- window – Window in which to send the event.
- context – Context in which to send the event.
- point – Point to send with the event

Returns

None

completeShape

Syntax

```
completeShape(  
  point: point = None,  
  /,  
  *,  
  window: giWindow | int = None,  
  context: deContext = None)  
Returns: None
```

Summary:

Completes shape.

Category:

Foundation

Licenses:

None

Parameters

- point – Point to send with event
- window – Window in which to send the event.
- context – Context in which to send the event.

Returns

None

copy

Syntax

```
copy(  
  objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
    list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
    | list[oaVia],  
  /,  
  *,  
  anchor: point = None,  
  selectResult: bool = None)  
Returns: None
```

Summary:

Copy objects to clipboard.

Category:

Design Editing

Licenses:

None

Parameters

- `objects` – List of objects to copy.
- `anchor` – Anchor point for paste.
- `selectResult` – Whether add the result to the selection set. (Default is: True)

Returns

None

copyBookmark**Syntax**

```
copyBookmark(  
    bookmark: deBookmark,  
    /,  
    *,  
    name: str,  
    force: bool = None)  
Returns: deBookmark
```

Summary:

Creates a copy of a bookmark with a new name.

Category:

Navigation

Licenses:

None

Parameters

- bookmark – The bookmark to copy
- name – The name of the new bookmark
- force – Allow overwriting of an existing bookmark (Default is: True)

Returns

The new bookmark copy

createActivationEngine

Syntax

```
createActivationEngine(  
*,  
    filterProc: list[variant] = None,  
    objectPickProc: list[variant] = None,  
    addPointProc: list[variant] = None,  
    startDragProc: list[variant] = None,  
    endDragProc: list[variant] = None,  
    snapMotionProc: list[variant] = None,  
    activeObjectChangeProc: list[variant] = None,  
    lpps: list[deTechLPP] = None,  
    objTypes: list[deObjectFilter] = None,  
    forceObjTypes: list[deObjectFilter] = None,  
    excludeObjTypes: list[deObjectFilter] = None,  
    depth: int = None,  
    mode: de.createActivationEngine.Mode = None,  
    viewDesign: bool = None,  
    includeOverlays: bool = None)  
Returns: deActivationEngine
```

Summary:

Creates an activation engine which enables active object highlighting and responds to addPoint events by calling a specified objectPickProc if the addPoint occurred over an object.

Category:

Foundation

Licenses:

None

Parameters

- `filterProc` – Procedure called for each object under the cursor. Return false to prevent activation.
- `objectPickProc` – Procedure called when `addPoint` events occur over an active object.
- `addPointProc` – Procedure called when clicking in the canvas.
- `startDragProc` – Procedure called when initiating a drag.
- `endDragProc` – Procedure called when completing a drag.
- `snapMotionProc` – Procedure called when moving the cursor.
- `activeObjectChangeProc` – Procedure called when active object changed.
- `lpps` – The list of the lpps to activate objects on.
- `objTypes` – The list of the object types can be activated.
- `forceObjTypes` – The list of the object types which should override the types from the OLP filter.
- `excludeObjTypes` – The list of the object types can not be activated.
- `depth` – Activation depth. (Default is: 0)
- `mode` – Activation mode. (`{ align | normal | tap }`) (Default is: 'normal')
- `viewDesign` – View design. (Default is: False)
- `includeOverlays` – Include overlays. (Default is: False)

Returns

A `deActivationEngine` object.

copyClipboardToLib

Syntax

```
copyClipboardToLib(  
  deClipboard: deClipboard,  
  /,  
  *,  
  to: oaLib)  
Returns: deClipboard
```

Summary:

Copies a clipboard to a clipboard library.

Category:

Clipboard

Licenses:

None

Parameters

- deClipboard – Clipboard to copy
- to – Target library

Returns

New clipboard object

createApplication

Syntax

```
createApplication(  
name: str,  
/,  
*,  
title: str,  
openProc: list[variant],  
canCreate: bool = None,  
canOpenMultiple: bool = None)  
Returns: deApplication
```

Summary:

Creates a new application which can be used to open the specified file name

Category:

Foundation

Licenses:

None

Parameters

- name – Unique string identifier for new application.
- title – String that will be shown in various interfaces.
- openProc – Procedure to evaluate using syntax given in description.

- `canCreate` – Indicates that this application can create new cellViews (Default is: `False`)
- `canOpenMultiple` – Indicates that this application can open multiple files in a single invocation (Default is: `False`)

Returns

Newly created application object.

createArgument

Syntax

```
createArgument(  
name: str = None,  
/,  
*,  
types: list[str] = None,  
description: str,  
isList: bool = None,  
optional: bool = None,  
hidden: bool = None,  
default: variant = None)  
Returns: deArgument
```

Summary:

Creates a new argument definition for use with `de::createCommand`.

Category:

Foundation

Licenses:

None

Parameters

- `name` – Name of the argument. (Default is: `'<name>'`)
- `types` – Argument type names.
- `description` – Help string for the argument.
- `isList` – Specifies that the argument accepts a list. (Default is: `False`)
- `optional` – Indicates whether the argument is optional or not. (Default is: `False`)
- `hidden` – Indicates whether the argument is hidden or not. (Default is: `False`)
- `default` – A default value for optional argument.

Returns

Newly created argument.

createArgumentGroup

Syntax

```
createArgumentGroup(  
arguments: list[deArgument | str],  
/,  
*,  
optional: bool = None)  
Returns: deArgumentGroup
```

Summary:

Creates an argument group.

Category:

Foundation

Licenses:

None

Parameters

- arguments – List of deArguments or argument names.
- optional – Specifies that the argument group is optional to the command. (Default is: False)

Returns

Newly created argument group object.

createAssociation

Syntax

```
createAssociation(  
name: str = None,  
/,  
*,  
fileName: str,  
application: deApplication | str = None,  
default: bool = None,  
defaultViewName: str = None)  
Returns: deAssociation
```

Summary:

Creates an association between a DM file name and a deApplication object.

Category:

Foundation

Licenses:

None

Parameters

- `fileName` – DM File name to associate.
- `application` – Application to invoke for this file name
- `default` – Sets this association as the default for the specified file name. (Default is: `False`)
- `defaultViewName` – Default view name to use for newly created cellViews. (Default is: `"`)

Returns

Newly created association object.

createBookmark

Syntax

```
createBookmark(  
    _name_: str,  
    /,  
    *,  
    window: giWindow | int,  
    comment: str = None,  
    force: bool = None)  
Returns: deBookmark
```

Summary:

Creates a new edit-context bookmark.

Category:

Navigation

Licenses:

None

Parameters

- `_name_` – The name of the new bookmark
- `window` – The window to save in a bookmark
- `comment` – A user-specified comment (Default is: “)
- `force` – Allow overwriting of an existing bookmark (Default is: True)

Returns

The new bookmark

createCanvas

Syntax

```
createCanvas(  
    _none_: str = None,  
    /,  
    *,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    tooltip: str = None,  
    context: deContext = None,  
    viewport: box = None,  
    cursor: point = None,  
    snapCursor: point = None,  
    display: drDisplay = None,  
    autoFit: bool = None,  
    width: int = None,  
    height: int = None,  
    **kwargs)  
Returns: deCanvas
```

Summary:

Creates a new deCanvas widget and adds it to the specified parent.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `context` – Context to depict.
- `viewport` – Initial viewport
- `cursor` – Initial cursor position
- `snapCursor` – Initial snap cursor position
- `display` – Display resource display to use for rendering
- `autoFit` – Automatically fit the viewport to the extents of the edit design whenever the canvas size, or design size, changes (Default is: False)
- `width` – Canvas width, in pixels, or 0, to expand to fill available space (Default is: 0)
- `height` – Canvas height, in pixels, or 0, to expand to fill available space (Default is: 0)

Returns

Newly created deCanvas object

createColorExpression

Syntax

```
createColorExpression(  
*,  
    expression: str,  
    color: str,  
    propagate: de.createColorExpression.Propagate = None,  
    shape: de.createColorExpression.Shape = None)  
Returns: deColorExpression
```

Summary:

Creates color expression object

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- **expression** – Boolean expression identifying the objects that should be colored with given color
- **color** – Density color name, which should be used for coloring the objects matching the boolean expression
- **propagate** – Color propagation type up the hierarchy ({ none | oneColor | worstColor } (Default is: 'oneColor'))
- **shape** – Shape type to be used for colorizing instances ({ circle | rectangle } (Default is: 'rectangle'))

Returns

Created color expression object

createCommand

Syntax

```
createCommand(  
name: deCommandInterface | str,  
/  
*,  
description: str,  
category: str,  
type: de.createCommand.Type = None,  
scope: de.createCommand.Scope = None,  
arguments: list[deArgument] = None,  
customSortingComparator: str = None,  
customActiveObjectToolbarProc: variant = None,  
exclusive: list[deArgumentGroup] = None,  
title: str = None,  
label: str = None,  
returnTypes: list[str] = None,  
returnDesc: str = None,  
requiresWriteMode: bool = None,  
license: list[str] = None,  
licenseDesc: list[str] = None,  
**kwargs)  
Returns: deCommand
```

Summary:

Creates and registers a user-defined Tcl interactive or procedure

Category:

Foundation

Licenses:

None

Parameters

- name – Name of the command.
- description – Short description of the command.
- category – String category name.
- type – Type of command to define. ({ interactive | procedural })
- scope – Scope of interactive command. ({ context | design } (Default is: 'design'))
- arguments – List of arguments.
- customSortingComparator – Sorting comparator function name. (Default is: "")
- customActiveObjectToolbarProc – Active Object information display proc.
- exclusive – List of exclusive argument groups.
- title – Command title, used for its COD title. (Default is: "")
- label – Command title, used for its COT label. (Default is: "")
- returnTypes – Object returned type.
- returnDesc – Description of command result. (Default is: "")
- requiresWriteMode – Indicated whether write access to the design is required or not. (Default is: False)
- license – Required license feature names.
- licenseDesc – Set of license feature names to show in command help declaration.

Returns

Newly created interactive command. Note that this command returns "" when registering procedures.

createContext

Syntax

```
createContext(  
  object: deBookmark | dmCellView | dmCellViewName | dmDMFile |  
    oaDesign,  
  /,  
  *,  
  readOnly: bool = None,  
  instPath: str = None)  
Returns: deContext
```

Summary:

Creates a new deContext object for the specified file, cellView's primary file or bookmark object.

Category:

Foundation

Licenses:

None

Parameters

- object – Cellview (primary file), DM file or bookmark object to open.
- readOnly – Opens the design in read-only mode. (Default is: False)
- instPath – Optional instance path to edit. (Default is: '/')

Returns

Newly created context.

createCrossProbeTool

Syntax

```
createCrossProbeTool(  
  name: str,  
  /,  
  *,  
  contextFilterProc: list[variant],  
  crossProbeProc: list[variant])  
Returns: deCrossProbeTool
```

Summary:

Creates a new cross probe tool object and returns it. If a cross probe tool already exists with the specified name it will be replaced with the new object.

Category:

Design Editing

Licenses:

None

Parameters

- name – English name of the new cross probe tool object.
- contextFilterProc – Tcl/Python procedure which returns 1 if this tool is valid for a context and 0 otherwise.
- crossProbeProc – Tcl/Python procedure which creates and returns associated probe objects.

Returns

New cross probe tool object

createErrorViewerDatabase

Syntax

```
createErrorViewerDatabase(  
name: str,  
/  
*,  
design: oaDesign,  
title: str = None,  
closeable: bool = None)  
Returns: deErrorViewerDatabase
```

Summary:

Create deErrorViewerDatabase object

Category:

Error Viewer

Licenses:

None

Parameters

- name – Unique name/identifier
- design – The base design of the database
- title – The title show in the tab widget of Error Viewer
- closeable – Indicate whether the database can be closed via the close button in the Error Viewer (Default is: False)

Returns

Newly created deErrorViewerDatabase object

createErrorViewerMarker

Syntax

```
createErrorViewerMarker(  
of: deErrorViewerDatabase,  
/  
*,  
points: list[point],  
closed: bool = None,  
severity: de.createErrorViewerMarker.Severity = None,  
user1: str = None,  
user2: str = None,  
user3: str = None,  
user4: str = None,  
tool: str = None,  
shortMsg: str = None,  
msg: str = None)  
Returns: oaObject
```

Summary:

Create marker to deErrorViewerDatabase

Category:

Error Viewer

Licenses:

None

Parameters

- of – Specify the deErrorViewerDatabase object
- points – Specify the geometry of the marker

- **closed** – Specify whether the geometry is closed or not (Default is: True)
- **severity** – Specify the severity value of marker ({ acknowledgedWarning | annotation | criticalError | error | fatalError | info | signedOffCriticalError | signedOffError | warning } (Default is: 'error'))
- **user1** – Specify the value for field “user1”
- **user2** – Specify the value for field “user2”
- **user3** – Specify the value for field “user3”
- **user4** – Specify the value for field “user4”
- **tool** – Specify the value for field “tool”
- **shortMsg** – Specify the value for field “shortMsg”
- **msg** – Specify the value for field “msg”

Returns

Marker Object

createFavoriteDevice

Syntax

```
createFavoriteDevice(  
*,  
    master: dmCellView,  
    params: list[map_] = None,  
    alias: str = None,  
    viewType: de.createFavoriteDevice.ViewType = None)  
Returns: deDevice
```

Summary:

Creates a new favorite device

Category:

Design Editing

Licenses:

None

Parameters

- master – Master cellView
- params – Optional list of parameter overrides in the format {{name value} {name value}}
- alias – An optional alias for the favorite device
- viewType – View type ({ maskLayout | schematic })

Returns

Newly created favorite device.

createFavoriteDeviceCategory

Syntax

```
createFavoriteDeviceCategory(  
    _name_: str,  
    /,  
    *,  
    devices: dbCollection = None)  
Returns: deDeviceCategory
```

Summary:

Creates a new favorite device category

Category:

Design Editing

Licenses:

None

Parameters

- _name_ – Category name
- devices – Devices to add to this category

Returns

Newly created favorite device category.

createGradient

Syntax

```
createGradient(  
*,  
    expression: str,  
    minValue: str,  
    maxValue: str,  
    count: int,  
    startColor: str,  
    endColor: str,  
    propagate: de.createGradient.Propagate = None)  
Returns: deGradient
```

Summary:

Creates gradient object

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- expression – Expression the evaluated values of which are gradiented by color
- minValue – Minimum value of the value range to be gradiented
- maxValue – Maximum value of the value range to be gradiented
- count – The number of colors the values in [min,max] range should be gradiented
- startColor – Density color name, which should be used as first color gradienting
- endColor – Density color name, which can be used as last color for the gradienting
- propagate – Color propagation type up the hierarchy ({ none | oneColor | worstColor }
(Default is: 'worstColor')

Returns

Created gradient object

createHierObject

Syntax

```
createHierObject(  
  object: dbCore | oaObject,  
  /,  
  *,  
  hierPath: deContext | oaHierPath)  
Returns: deHierObject
```

Summary:

Creates a new deHierObject

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- object – Referenced object
- hierPath – Hierarchical path from top design to object

Returns

New deHierObject

createHighlightSelector

Syntax

```
createHighlightSelector(  
  _none_: str = None,  
  /,  
  *,  
  parent: giAssistant | giContainer | giDialog | giWindow = None,  
  enabled: bool = None,  
  shown: bool = None,  
  prefName: dbPrefName = None,  
  valueChangeProc: list[variant] = None,  
  label: str = None,  
  value: str = None,  
  showNames: bool = None,  
  showAuto: bool = None)  
Returns: deHighlightSelector
```

Summary:

Creates a new deHighlightSelector widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: "")
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: "")
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `label` – Widget label, shown in the left column. (Default is: "")
- `value` – Initial value. (Default is: "")
- `showNames` – Display highlight names. (Default is: False)
- `showAuto` – Enable automatic colorization. (Default is: False)

Returns

New object.

createHighlightSet

Syntax

```
createHighlightSet(  
    name: str,  
    /,  
    *,  
    design: oaDesign,  
    packet: str = None,  
    packetFill: bool = None,
```



```
persistent: bool = None,  
public: bool = None,  
shadow: bool = None,  
visible: bool = None,  
drawInvisible: bool = None)  
Returns: de.HighlightSet
```

Summary:

Create new highlight set within the specified design.

Category:

Design Editing

Licenses:

None

Parameters

- name – Name of new created set
- design – Design from which objects will be highlighted
- packet – Packet defines the drawing style
- packetFill – Packet Fill defines the fill drawing style (Default is: True)
- persistent – If true, this highlight set will be saved to the design and restored in future sessions (Default is: False)
- public – Show this set in selectors of interactive commands (Default is: False)
- shadow – Enables “Shadow Mode” for this set (Default is: False)
- visible – Determines if the highlight set is visible (Default is: True)
- drawInvisible – Highlight invisible figure if true (Default is: True)

Returns

Newly created highlight set

createInterfaceElement

Syntax

```
createInterfaceElement(  
*,  
on: oaInstTerm | oaNet,  
direction: de.createInterfaceElement.Direction,  
params: str = None,
```

```
context: heContext,  
netlister: nlAMSNetlister)  
Returns: nlInterfaceElement
```

Summary:

Creates interface element on net or instance terminal in the given hierarchy context.

Category:

Netlist Tools

Licenses:

None

Parameters

- on – Net or instance terminal on which the IE is created.
- direction – Direction of created IE. ({ a2d | d2a | e2n | e2r | inout | n2e | r2e })
- params – Parameter values string with format “<param_name_1> <param_value_1> <param_name_2> <param_value_2> ...” (Default is: “)
- context – Hierarchy context
- netlister – AMS netlister according to which the partition is made.

Returns

Interface element object

createLPPInput

Syntax

```
createLPPInput(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
prefName: dbPrefName = None,  
valueChangeProc: list[variant] = None,  
label: str = None,  
value: lpp = None,  
readOnly: bool = None,  
valid: bool = None,  
required: bool = None,  
design: oaDesign = None,
```

```
tech: oaTech = None,  
lpps: list[deTechLPP] = None)  
Returns: deLPPInput
```

Summary:

Creates a new deLPPInput widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `label` – Widget label, shown in the left column. (Default is: “)
- `value` – Initial value.
- `readOnly` – Initial read-only state. (Default is: False)
- `valid` – Initial valid state. (Default is: True)
- `required` – Specifies this is a required input. (Default is: False)
- `design` – Design from which to retrieve LPPs
- `tech` – Tech from which to retrieve LPPs
- `lpps` – collection of deLPPs

Returns

New object.

createLPPSet

Syntax

```
createLPPSet(  
    title: str,  
    /,  
    *,  
    lpps: list[lpp | list[deTechLPP]] = None,  
    defaultLPP: deTechLPP | lpp = None,  
    shown: bool = None)  
Returns: deLPPSet
```

Summary:

Creates a new deLPPSet object. If a set with the same title already exists, it is silently replaced with this new object. It is not possible to replace built-in LPP sets with this command.

Category:

Layout Editing

Licenses:

None

Parameters

- title – Title of the new LPP Set
- lpps – TCL list of layer purpose pairs or collection of deLPP objects
- defaultLPP – Default LPP to set active when this set is selected
- shown – If true, this set is shown in the OLP (Default is: True)

Returns

New deLPPSet object

createNetHighlightSet

Syntax

```
createNetHighlightSet(  
    name: str,  
    /,  
    *,  
    net: oaBitNet,  
    context: deContext,  
    packet: str = None,
```

```
    shadow: bool = None)
Returns: deSet
```

Summary:

Create new net highlight set within the specified properties

Category:

Design Editing

Licenses:

None

Parameters

- name – Name of new created set
- net – Net to highlight
- context – Context from which objects will be highlighted
- packet – Packet name or packet alias which defines the drawing style for this set (Default is: 'highlight1')
- shadow – Enables "Shadow mode" for this set (Default is: False)

Returns

Newly created highlight set

createObjectFilter

Syntax

```
createObjectFilter(
name: str,
/,
*,
title: str,
filterProc: str,
design: oaDesign)
Returns: None
```

Summary:

Adds a custom object filter.

Category:

Layout Editing

Licenses:

None

Parameters

- name – Object Filter name
- title – Object filter title
- filterProc – Tcl procedure name to filter objects
- design – Design where the Object Filter must be add

Returns

None

createOLPPreset

Syntax

```
createOLPPreset(  
title: str,  
/,  
*,  
id: int = None,  
from_: deContext | oaDesign = None,  
of: deContext | oaDesign = None)  
Returns: deOLPPreset
```

Summary:

Creates a new deOLPPreset from the current design state.

Category:

Layout Editing

Licenses:

None

Parameters

- title – OLP preset title
- id – OLP preset id (Default is: 0)
- of – Design or context from which to obtain settings for the new deOLPPreset

Returns

New deOLPPreset object

createOnCanvasWidget

Syntax

```
createOnCanvasWidget(  
    name: str,  
    /,  
    *,  
    shown: bool = None,  
    window: giWindow | int,  
    viewportPoint: point = None,  
    closeProc: list[variant] = None)  
Returns: giDialog
```

Summary:

Creates a new giDialog placed on a parent window canvas.

Category:

Design Editor

Licenses:

None

Parameters

- name – Widget name
- shown – Initial shown state. (Default is: True)
- window – Parent window
- viewportPoint – Coordinate of widget position on a viewport.
- closeProc – Callback procedure to call when dialog closes. Called with the dialog object

Returns

New object.

createOverlay

Syntax

```
createOverlay(  
name: str,  
/,  
*,  
title: str = None,  
base: oaDesign,  
design: dmCellView | oaDesign = None,  
packet: drPacket | drPacketAlias = None,  
shown: bool = None,  
drawPlane: de.createOverlay.DrawPlane = None,  
ignoreDrawFilter: bool = None,  
selectable: bool = None,  
selectLogic: bool = None,  
context: deContext = None,  
customActiveObjectToolbarProc: str = None)  
Returns: deOverlay
```

Summary:

Creates an overlay, which is a design that is drawn on top of another design.

Category:

Foundation

Licenses:

None

Parameters

- name – Unique identifier for a given base design
- title – Optional text description of the overlay design
- base – Design over which to draw the overlay
- design – Overlay design. If not specified, a temporary cellview is created
- packet – Packet to override normal drawing attributes
- shown – Enables drawing this overlay (Default is: True)
- drawPlane – Draw plane to use for this overlay ({ annotation | edit | static } (Default is: 'annotation')
- ignoreDrawFilter – Draws every shape in the overlay design (Default is: False)
- selectable – Permit selecting objects in this overlay (Default is: False)

- selectLogic – Selecting figures will select their logic in parent context (Default is: False)
- context – Limit drawing this overlay to a specific context
- customActiveObjectToolbarProc – Active Object information display proc. (Default is: ")

Returns

Newly created overlay.

createProbe

Syntax

```
createProbe(  
    name: str,  
    /,  
    *,  
    context: deContext | heContext,  
    type: de.createProbe.Type,  
    tool: str = None,  
    packet: str = None,  
    description: str = None,  
    longDescription: str = None,  
    purpose: de.createProbe.Purpose = None,  
    terminalByPosition: bool = None,  
    caseSensitive: bool = None,  
    crossProbe: deCrossProbeTool | str = None,  
    allowObjectNotFoundMessage: bool = None,  
    traceProc: list[variant] = None)  
Returns: deProbe
```

Summary:

Creates a new probe object for the specified hierarchical context. The unnamed argument specifies the probed object name. The probe may be removed with the db::destroy command.

Category:

Design Editing

Licenses:

None

Parameters

- name – Object reference within the target design specified by the context.
- context – Hierarchical context including the top design and lineage.

- **type** – Type of probe to create. ({ cellview | instTerm | instance | net | path | terminal })
- **tool** – Tool name.
- **packet** – Packet alias name of display resources.
- **description** – Probe description.
- **longDescription** – String describing a probe in detail.
- **purpose** – Probe purpose. ({ cross | error | info | other | warning })
- **terminalByPosition** – Indicates that the defined instance terminal uses terminal position. (Default is: False)
- **caseSensitive** – Case sensitive creation of probes. (Default is: True)
- **crossProbe** – If specified, creates a probe in the specified context and cross probes via the specified cross probe tool.
- **allowObjectNotFoundMessage** – Enables printing information message when an object is not found. (Default is: True)
- **traceProc** – Tcl procedure to enable tracing through devices in net probes highlighting. This procedure is called for each oaInstTerm on the probed net, and it should return a list of additional oaBitNets to trace. Signature: <list of oaBitNets> traceProc <heContext> <oaInstTerm>. Alternatively, insert 'snpsCustom' to trace through devices by using Probe Trace Through options in the General Options dialog box.

Returns

Newly created probe object

createSelectionEngine

Syntax

```
createSelectionEngine (  
*,  
    filterProc: list[variant] = None,  
    startDragSelectedProc: list[variant] = None,  
    endDragSelectedProc: list[variant] = None,  
    dragMotionProc: list[variant] = None,  
    dragSnapMotionProc: list[variant] = None,  
    selectionChangeProc: list[variant] = None,  
    clickSelectedProc: list[variant] = None,  
    topNet: list[str] = None,  
    select: de.createSelectionEngine.Select = None)  
Returns: deSelectionEngine
```

Summary:

Creates a selection engine which enables active object highlighting and responds to semantic events by selecting shapes.

Category:

Foundation

Licenses:

None

Parameters

- `filterProc` – Procedure called for each selectable object. Return false to prevent selection.
- `startDragSelectedProc` – Called when initiating a drag over selected objects.
- `endDragSelectedProc` – Called when completing a drag with selected objects.
- `dragMotionProc` – Called for each motion when dragging selected objects.
- `dragSnapMotionProc` – Called for each snap motion when dragging selected objects.
- `selectionChangeProc` – Called when selection changes.
- `clickSelectedProc` – Called when clicking on a selected object.
- `topNet` – Select only objects on the given net
- `select` – Selection mode. ({ deselect | replace | select })

Returns

A `deSelectionEngine` object.

createShapeEngine

Syntax

```
createShapeEngine(  
*,  
  shapeType: de.createShapeEngine.ShapeType,  
  angleMode: de.createShapeEngine.AngleMode = None,  
  completeProc: list[variant],  
  pointsChangedProc: list[variant] = None,  
  stretchProc: list[variant] = None,  
  numPoints: int = None,  
  draw: bool = None)  
Returns: deShapeEngine
```

Summary:

Creates a shape engine capable of providing point, line, rectangle and polygon input from a Tcl-defined interactive command.

Category:

Foundation

Licenses:

None

Parameters

- `shapeType` – Type of point input. ({ line | point | polygon | rectangle })
- `angleMode` – Optional angle mode to restrict edges of multi-point objects. ({ anyAngle | diagonal | orthogonal | xFirst | yFirst } (Default is: 'anyAngle'))
- `completeProc` – The name of the procedure to call when input is complete.
- `pointsChangedProc` – Optional procedure to call when points are added or removed.
- `stretchProc` – Optional procedure to call every time the runtime shape is stretched, by cursor motion, before adding a new point.
- `numPoints` – Optional number of points requested. Specifying 0 indicates that the input is terminated manually by the user. (Default is: 0)
- `draw` – Draw points as a white line. (Default is: True)

Returns

A deShapeEngine object.

createTemplate

Syntax

```
createTemplate(  
  name: str,  
  /,  
  *,  
  libName: dmLibName,  
  netlist: dbCollection | list[deFigure] | list[oaFig] = None,  
  from_: dmCellView = None,  
  of: dmCellView = None,  
  description: str = None)  
Returns: deTemplate
```

Summary:

Creates a new template with the given name in the given OA library.

Category:

Templates Category

Licenses:

Frontend

Parameters

- name – Name of new template cell.
- libName – Name of library to contain new template.
- netlist – Collection of oaFig | deFigure | lxInst | dbNetlistInst objects containing instances that will define the template netlist structure.
- of – Cellview to use for the template netlist structure.
- description – User supplied description of the template.

Returns

The newly created template.

createTool

Syntax

```
createTool(  
    name: str,  
    /,  
    *,  
    title: str,  
    initProc: list[variant] = None,  
    doneProc: list[variant] = None,  
    activateProc: list[variant] = None,  
    deactivateProc: list[variant] = None,  
    decorateProc: list[variant] = None)  
Returns: deTool
```

Summary:

Creates a new tool object. If the tool already exists, it will be destroyed. Note that internally defined tools may not be destroyed.

Category:

Foundation

Licenses:

None

Parameters

- name – Unique string identifier for new tool
- title – String that will be shown in various interfaces such as the Tools menu
- initProc – Tcl procedure, which takes 1 argument, the context in which the tool is being initialized. Procedure is responsible for initializing resources, checking out licenses, etc.
- doneProc – Tcl procedure, which takes 1 argument, the context in which the tool is being shutdown due to the context closing or cellview being closed. Releases resources, licenses, etc.
- activateProc – Tcl procedure which is called after -initProc and when re-activated after previously being deactivated.
- deactivateProc – Tcl procedure which is called when the tool is no longer valid for the given context (ie, descend into unsupported editor).
- decorateProc – Tcl procedure which is called after the editor gets a new window type for decoration. This method is called after activateProc and may be called multiple times as context's window type changes.

Returns

Newly created tool

crossCheckViews

Syntax

```
crossCheckViews(  
name: list[str] = None,  
/,  
*,  
design: deContext | dmCellView | dmCellViewName,  
checkOnly: bool = None,  
caseInsensitive: bool = None,  
severity: de.crossCheckViews.Severity = None,  
checkPortOrder: bool = None,  
checkTerminalType: bool = None,  
ignoreEmptyPortOrder: bool = None,
```

```
verbose: bool = None)  
Returns: list[deViolation]
```

Summary:

Perform pin/terminal consistency check.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- name – List of views or cellviews in 'l/c/v' format to compare with the control design.
- design – Design which is compared with all given cellviews.
- checkOnly – Indicates whether marker objects should be created. (Default is: False)
- caseInsensitive – Indicates whether the terminal name matching should be insensitive.
- severity – Severity level for violations ({ error | warning })
- checkPortOrder – Indicates whether port orders should be checked. (Default is: True)
- checkTerminalType – Indicates whether terminal types should be checked. (Default is: True)
- ignoreEmptyPortOrder – Indicates whether port order check is ignored if main checked view has no portOrder property. (Default is: True)
- verbose – Indicates whether message should be printed when no violation was found. (Default is: True)

Returns

Collection of violations created

cut

Syntax

```
cut(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia],  
/,  
*,
```

```
    anchor: point = None)
Returns: None
```

Summary:

Copies supplied objects into clipboard and then deletes the supplied objects

Category:

Design Editing

Licenses:

Basic or Frontend

Parameters

- objects – List of objects to cut.
- anchor – Anchor point for paste.

Returns

None

cycleActiveFigure

Syntax

```
cycleActiveFigure(
window: deCanvas | giWindow | int = None,
/,
*,
point: point = None,
direction: de.cycleActiveFigure.Direction = None)
Returns: deFigure
```

Summary:

Cycles the active figure.

Category:

Design Editing

Licenses:

None

Parameters

- window – Window or canvas
- point – Center of region to look for active objects in.
- direction – Cycling direction.) ({ next | prev } (Default is: 'next')

Returns

Next figure in Cycle through figures

cycleTapObject

Syntax

```
cycleTapObject(  
*,  
    window: giWindow | int = None,  
    context: deContext = None,  
    direction: de.cycleTapObject.Direction = None)  
Returns: deFigure
```

Summary:

Cycles thru the objects in the tap set.

Category:

Design Editing

Licenses:

None

Parameters

- window – Window in which to cycle in.
- context – Context in which to cycle in.
- direction – Direction to cycle. ({ next | prev } (Default is: 'next')

Returns

Tap object

deactivateTools

Syntax

```
deactivateTools(  
    tools: dbCollection)  
Returns: None
```

Summary:

Deactivates one or more tools within the specified context. If already deactivated command will not alter its state.

Category:

Foundation

Licenses:

None

Parameters

tools – Collection of deToolInst objects to deactivate in the specified context. To retrieve these objects, retrieve the 'tools' attribute on the deEditorInst object. The deTool argument value is deprecated.

Returns

None

defineOrigin

Syntax

```
defineOrigin(  
    point: point,  
    /,  
    *,  
    design: oaDesign)  
Returns: None
```

Summary:

Moves all objects within the specified design by the inverse of the vector given by the specified coordinate

Category:

Design Editing

Licenses:

Basic or Frontend

Parameters

- point – Point which will be 0,0 after translating all objects
- design – Design in which to move objects

Returns

None

deleteBookmarks

Syntax

```
deleteBookmarks(  
    bookmarks: list[deBookmark])  
Returns: None
```

Summary:

Deletes the specified bookmarks.

Category:

Navigation

Licenses:

None

Parameters

bookmarks – The collection of bookmarks to delete

Returns

None

deprecateCommand

Syntax

```
deprecateCommand(  
    *,  
    namespace: str = None,  
    command: deCommand | str = None,  
    argument: str = None,  
    argumentValueType: str = None,
```

```
replacement: str = None,  
defunct: bool = None)  
Returns: None
```

Summary:

Marks a command, argument, argument value type or namespace as deprecated or defunct.

Category:

Foundation

Licenses:

None

Parameters

- namespace – Namespace to deprecate
- command – Command or procedure name to deprecate
- argument – Argument to deprecate
- argumentValueType – Argument value type to deprecate
- replacement – Name of replacement command, argument, value type, or namespace to see regarding deprecating or defuncting of this item. (Default is: “")
- defunct – Mark object defunct (Default is: False)

Returns

None

descend

Syntax

```
descend(  
name: str,  
/,  
*,  
context: deContext,  
master: dmCellView | dmCellViewName | dmDMFile = None,  
readOnly: bool = None)  
Returns: deContext
```

Summary:

Descends into a new design, leaving the former open within the specified context.

Category:

Navigation

Licenses:

None

Parameters

- name – Instance name.
- context – Context object to modify
- master – File or cellview into which to descend.
- readOnly – Open design in read-only mode. (Default is: False)

Returns

Modified context object

descendInPlace

Syntax

```
descendInPlace(  
    lineage: oaHierPath | oaInst | str,  
    /,  
    *,  
    context: deContext,  
    readOnly: bool = None)  
Returns: deContext
```

Summary:

Descends into a new design, leaving the former open.

Category:

Navigation

Licenses:

None

Parameters

- lineage – Instance, instance name or lineage specification
- context – Context object to modify
- readOnly – Open design in read-only mode (Default is: False)

Returns

Modified context object

descendIntoGroup

Syntax

```
descendIntoGroup(  
  object: oaFigGroup | str,  
  /,  
  *,  
  context: deContext)  
Returns: deContext
```

Summary:

Descends into a fig group.

Category:

Navigation

Licenses:

None

Parameters

- object – Figure group or name of a figure group in the edit design to edit
- context – Context object to modify

Returns

Modified context object

deselect

Syntax

```
deselect(  
  objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
    list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
    | list[oaVia])  
Returns: None
```

Summary:

Deselects (removes from the given set) given objects.

Category:

Design Editing

Licenses:

None

Parameters

objects – Objects to deselect

Returns

None

deselectAll

Syntax

```
deselectAll(  
  context: deContext)  
Returns: None
```

Summary:

Deselects all objects in given select set and active select set.

Category:

Design Editing

Licenses:

None

Parameters

context – Design in which to deselect all objects

Returns

None

deselectAllLogic

Syntax

```
deselectAllLogic(  
  *,  
  context: dbCollection)  
Returns: None
```

Summary:

Deselect all logic in the specified context and all linked contexts.

Category:

Design Editing

Licenses:

None

Parameters

context – Context within which to deselect logic.

Returns

None

deselectLogic

Syntax

```
deselectLogic(  
objects: dbCollection | list[oaFig] | list[oaFigGroup] |  
         list[oaGroup] | list[oaInst] | list[oaInstTerm] | list[oaNet] |  
         list[oaPin] | list[oaTerm],  
/,  
*,  
context: deContext)  
Returns: dbCollection
```

Summary:

Deselects logic within the specified design context.

Category:

Design Editing

Licenses:

None

Parameters

- objects – Logic to deselect
- context – Context within which to deselect logic.

Returns

Deselected objects which were previously selected

enableColorAnnotator

Syntax

```
enableColorAnnotator(  
context: deContext,  
/  
*,  
applicationName: str,  
dbHandle: str,  
dbTableNames: list[str],  
objectType: de.enableColorAnnotator.ObjectType,  
objectNameColumn: str = None,  
gradientBy: dbCollection = None,  
colorizeBy: dbCollection = None,  
occurrenceColumn: str = None)  
Returns: deColorAnnotator
```

Summary:

Enables color annotator in given design context for given application data

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- context – design context
- applicationName – Color annotator application name
- dbHandle – SQLite database handle
- dbTableNames – List of tables to be looked up for objects to be colorized
- objectType – Object type for which color annotation is done ({ instance | net })
- objectNameColumn – Name of the column containing the object name or occurrence/ <objectName> of the objects on which color annotation is done (Default is: 'name')
- gradientBy – Collection of gradient objects

- **colorizeBy** – Collection of color expression objects
- **occurrenceColumn** – The column name (or expression) that contains the occurrence path of the objects on which color annotation is done. Should be specified if the column specified in **-objectNameColumn** field doesn't contain the occurrence path

Returns

Newly created color annotator object.

enablePartitioningDisplay

Syntax

```
enablePartitioningDisplay(  
*,  
    context: deContext,  
    netlister: nlAMSNetlister,  
    enable: bool = None,  
    simulatorReportsPath: str = None,  
    onlyRefresh: bool = None)  
Returns: None
```

Summary:

Enables or disable the partition visualization in the given design context.

Category:

Netlist Tools

Licenses:

Frontend

Parameters

- **context** – Design context
- **netlister** – AMS netlister according to which the partition is made.
- **enable** – Enables/disables partition visualization. (Default is: True)
- **simulatorReportsPath** – Path to simulation reports for partition recognition. Should point to folder containing subfolders 'results' and 'netlist', or immediately to 'results' ('../netlist' should also exist). (Default is: "")
- **onlyRefresh** – If true, the partition domains information is not updated when enabling the visualization. (Default is: False)

Returns

None

enableRotationAndScaling

Syntax

```
enableRotationAndScaling(  
  object: dmCellViewName | list[dmCellView] | list[dmDMFile])  
Returns: int
```

Summary:

Given a collection of dmCellView objects, define as a superMaster using oaPcellDef snpsRotateAndScale.

Category:

Foundation

Licenses:

Backend

Parameters

object – One or more cellview (primary file) or DM file.

Returns

int return value.

endDrag

Syntax

```
endDrag(  
  point: point,  
  /,  
  *,  
  window: giWindow | int = None,  
  context: deContext = None)  
Returns: None
```

Summary:

Completes dragging

Category:

Foundation

Licenses:

None

Parameters

- point – Point at which to start the drag event
- window – Window in which to send the event.
- context – Context in which to send the event.

Returns

None

endLog

Syntax

```
endLog(  
    file: str  
Returns: None
```

Summary:

Closes a secondary log file previously opened with de::startLog.

Category:

Foundation

Licenses:

None

Parameters

file – Full filesystem path to the secondary log file

Returns

None

endTransaction

Syntax

```
endTransaction(  
    transaction: de.Transaction,  
    /,  
    *,  
    dirties: bool = None)  
Returns: None
```

Summary:

End Transaction

Category:

Foundation

Licenses:

None

Parameters

- transaction – Transaction to end
- dirties – This transaction should be considered to be design dirtying (Default is: True)

Returns

None

errorViewerLoadResult

Syntax

```
errorViewerLoadResult(  
    *,  
    window: int,  
    createToEditDesign: bool = None,  
    speedMode: bool = None,  
    type: de.errorViewerLoadResult.Type = None,  
    title: str = None,  
    toolName: str = None,  
    ruleFile: str = None,  
    waiveFile: str = None,  
    resultFile: str,  
    gpdLoadType: de.errorViewerLoadResult.GpdLoadType = None)  
Returns: None
```

Summary:

Load Result to Error Viewer

Category:

Error Viewer

Licenses:

None

Parameters

- window – Load result to error viewer associated with the given window index or the current edit design if not specify this option.
- createToEditDesign – True to create marker to edit design, false to overlay design (Default is: False)
- speedMode – Use speed mode to load result, this option is only valid for calibre type result (Default is: False)
- type – Specify the result file type ({ calibre | gpd }) (Default is: 'calibre')
- title – Specify the title of the new tab, this only valid when createToEditDesign is false
- toolName – Specify the tool name
- ruleFile – Specify the rule file
- waiveFile – Specify the waive file
- resultFile – Specify the result file
- gpdLoadType – Specify the gpd loading type ({ All | Open | Short }) (Default is: 'All')

Returns

None

exportBookmarks

Syntax

```
exportBookmarks(  
    bookmarks: list[deBookmark],  
    /,  
    *,  
    filePath: str,  
    force: bool = None)  
Returns: None
```

Summary:

Exports the specified bookmarks to a file.

Category:

Navigation

Licenses:

None

Parameters

- `bookmarks` – The collection of bookmarks to export.
- `filePath` – The path to the destination bookmark file.
- `force` – If false, the command will throw an error if the specified file exists. If true, the file will be silently overwritten. (Default is: True)

Returns

None

exportProbeCellSettings**Syntax**

```
exportProbeCellSettings(  
*,  
    value: str | None = None,  
    file: str)  
Returns: None
```

Summary:

Export Probe Trace Through Cell settings to specified file.

Category:

Design Editing

Licenses:

None

Parameters

- `value` – Probe Trace Through Cell Settings to save
- `file` – File Name to export the Probe Trace Through Cell Settings

Returns

None

extractInterfaceElements

Syntax

```
extractInterfaceElements(  
    *,  
    context: heContext,  
    netlist: nlAMSNetlist,  
    path: str)  
Returns: dbCollection
```

Summary:

Extracts interface elements from the simulation report.

Category:

Netlist Tools

Licenses:

None

Parameters

- context – Hierarchy context in which the interface elements are created
- netlist – AMS netlist according to which the partition is made.
- path – Path to simulation reports for interface elements extraction. Should point to folder containing subfolders 'results' and 'netlist', or immediately to 'results' ('../netlist' should also exist).

Returns

Collection of interface element objects successfully extracted from reports.

findMessage

Syntax

```
findMessage(  
    messageID: str,  
    /,  
    *,  
    args: list[str] = None)  
Returns: str
```


Summary:

Retrieves an externalized message string by its messageId.

Category:

Foundation

Licenses:

None

Parameters

- `messageID` – The message ID of the message to be returned.
- `args` – The message arguments

Returns

The message string found and formatted

finishEvalTextRendering**Syntax**

```
finishEvalTextRendering(  
*,  
    window: deCanvas | giWindow | int = None,  
    incremental: bool = None)  
Returns: int
```

Summary:

Invoke and wait for the completion of delayed text evaluations in a window canvas

Category:

Navigation

Licenses:

None

Parameters

- `window` – Window or canvas
- `incremental` – When true, the command will only operate on the next batch of pending labels within a `deEvalTextRenderTimeout` timeout. (Default is: False)

Returns

The number of delayed labels remaining unprocessed

fit

Syntax

```
fit(  
*,  
    window: deCanvas | giWindow | int = None,  
    fitGrid: bool = None,  
    fitView: bool = None,  
    fitEdit: bool = None,  
    set: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
    list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
    | list[oaVia] = None,  
    scale: float = None)  
Returns: None
```

Summary:

Fits the viewport either to view cellView or edit cellView or grid or any set of objects

Category:

Navigation

Licenses:

None

Parameters

- window – Window or canvas
- fitGrid – Fit to grid (Default is: False)
- fitView – Fit to view design (Default is: False)
- fitEdit – Fit to edit design (Default is: True)
- set – Collection of objects to fit to
- scale – Maximum fraction of each axis that the data extents may cover (Default is: 0.9)

Returns

None

getActiveCommand

Syntax

```
getActiveCommand(  
    *,  
    window: giWindow | int = None,  
    context: deContext = None)  
Returns: deCommand
```

Summary:

Returns the name of the active command.

Category:

Foundation

Licenses:

None

Parameters

- window – Window in which to query the command.
- context – Context in which to query the command.

Returns

The active command. "" if none.

getActiveContext

Syntax

```
getActiveContext()  
Returns: deContext
```

Arguments:

Summary:

Returns the active context which corresponds to the last focused editor window.

Category:

Foundation

Licenses:

None

Returns

Active context

getActiveFigure

Syntax

```
getActiveFigure(  
window: deCanvas | giWindow | int = None,  
/,  
*,  
point: point = None,  
index: int = None,  
intent: de.getActiveFigure.Intent = None)  
Returns: deFigure
```

Summary:

Returns current active deFigure.

Category:

Design Editing

Licenses:

None

Parameters

- window – Window or canvas
- point – If specified, calculates a new active set and returns the first figure or the one specified by -index.
- index – Returns the nth item in the active set. (Default is: 0)
- intent – Specifies selection mode ({ deselect | none | select } (Default is: 'none')

Returns

Active Object

getActiveLPP

Syntax

```
getActiveLPP(  
*,  
/
```

```
design: deContext | oaDesign)
Returns: deLPP
```

Summary:

Gets the active LPP.

Category:

Layout Editing

Licenses:

None

Parameters

design – Design for which this applies. If a context is specified, the edit design is used,

Returns

LPP to make active

getApplications

Syntax

```
getApplications(
name: str = None)
Returns: dbCollection
```

Summary:

Returns a collection of registered applications, optionally filtered by name.

Category:

Foundation

Licenses:

None

Parameters

name – Optional glob-style pattern to match against each application's name. (Default is: '*')

Returns

Defined applications

getAssociations

Syntax

```
getAssociations()  
Returns: dbCollection
```

Arguments:

Summary:

Returns a collection recently defined associations

Category:

Foundation

Licenses:

None

Returns

Defined associations

getBackgroundImporters

Syntax

```
getBackgroundImporters(  
objects: list[dmCell] | list[dmCellView] | list[oaLib])  
Returns: dbCollection
```

Summary:

Gets background importers for digital designs.

Category:

Foundation

Licenses:

Backend

Parameters

objects – A collection of selected oaLib or dmCell or dmCellView.

Returns

Background importers

getBBox

Syntax

```
getBBox(  
    figures: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
            list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
            | list[oaVia])  
Returns: box
```

Summary:

Returns a merged bBox from a collection of figures.

Category:

Design Editing

Licenses:

None

Parameters

figures – Collection of figures.

Returns

Merged bBox

getBinsForGradient

Syntax

```
getBinsForGradient(  
    *,  
    gradient: deGradient,  
    dbHandle: str,  
    dbTableNames: list[str])  
Returns: dbCollection
```

Summary:

Gets bins for given gradient and database tables

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- gradient – Gradient object
- dbHandle – Database handle
- dbTableNames – List of tables to be looked up for objects to be colorized

Returns

Collection of bin objects

getBookmarks

Syntax

```
getBookmarks(  
    pattern: str = None,  
    /,  
    *,  
    regexp: bool = None)  
Returns: list[deBookmark]
```

Summary:

Returns bookmarks with optional filtering by bookmark name.

Category:

Navigation

Licenses:

None

Parameters

- pattern – A pattern for bookmark names (Default is: '*')
- regexp – When true, the pattern is a regular expression. Otherwise, the pattern uses glob-style wildcards. (Default is: False)

Returns

The bookmarks

getBreakpoints

Syntax

```
getBreakpoints()  
Returns: str
```

Arguments:

Summary:

Returns a Tcl list of break point line numbers from the log file which is being replayed.

Category:

Foundation

Licenses:

None

Returns

Tcl list of breakpoint line numbers

getCapabilities

Syntax

```
getCapabilities(  
name: str = None)  
Returns: dbCollection
```

Summary:

Returns a collection of all known capabilities.

Category:

Foundation

Licenses:

None

Parameters

name – Glob-style pattern to match against capability name. (Default is: '*')

Returns

All known capabilities.

getClipboardLibs

Syntax

```
getClipboardLibs(  
  none: str = None)  
Returns: list[oaLib]
```

Summary:

Returns a collection of clipboard libraries.

Category:

Clipboard

Licenses:

None

Parameters

none – Glob-style pattern to match against clipboard library name (Default is: '*')

Returns

Collection of clipboard libraries

getClips

Syntax

```
getClips(  
  none: str = None,  
  /,  
  *,  
  lib: oaLib = None,  
  viewType: str = None)  
Returns: dbCollection
```

Summary:

Returns a collection of clipboards.

Category:

Clipboard

Licenses:

None

Parameters

- none – Glob-style pattern to match against clipboard name (Default is: “*”)
- lib – Clipboard library from which to return clipboards
- viewType – Optionally filter returned clipboards by view type (Default is: “”)

Returns

Collection of clipboards

getColorAnnotators

Syntax

```
getColorAnnotators(  
*,  
    context: deContext = None)  
Returns: dbCollection
```

Summary:

Gets enabled color annotator objects

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

context – design context

Returns

Enabled color annotator objects

getCommands

Syntax

```
getCommands(  
pattern: str = None,  
/,  
*,  
    includeDeprecated: bool = None)  
Returns: list[deCommand]
```

Summary:

Returns a collection of registered commands.

Category:

Foundation

Licenses:

None

Parameters

- `pattern` – Glob-style pattern to match the command name against. (Default is: `'*'`)
- `includeDeprecated` – Whether to include deprecated commands in the result.

Returns

Collection of commands that satisfy the given filters.

getContexts

Syntax

```
getContexts (  
    *,  
    window: giWindow | int = None,  
    file: dmDMFile = None)  
Returns: dbCollection
```

Summary:

Returns all open data contexts.

Category:

Foundation

Licenses:

None

Parameters

- `window` – Only return the context associated with the given window or window id.
- `file` – Only return contexts which have the specified file open in their descend stack.

Returns

All open data contexts.

getCrossProbeTools

Syntax

```
getCrossProbeTools(  
    name: str = None,  
    /,  
    *,  
    context: deContext | heContext = None)  
Returns: list[deCrossProbeTool]
```

Summary:

Returns a collection of registered cross probe tool objects optionally filtered by name or by those available for a given context.

Category:

Design Editing

Licenses:

None

Parameters

- name – If specified, only return cross probe tools whose name match this glob-style pattern
- context – If specified, only return cross probe tools available for this context

Returns

Collection of cross probe tools

getCursor

Syntax

```
getCursor(  
    window: deCanvas | giWindow | int = None)  
Returns: point
```

Summary:

Returns cursor current position.

Category:

Design Editing

Licenses:

None

Parameters

window – Window or canvas

Returns

The current cursor location

getCursorSnapPoint

Syntax

```
getCursorSnapPoint(  
    window: deCanvas | giWindow | int = None)  
Returns: point
```

Summary:

Returns cursor snap point for the design.

Category:

Design Editing

Licenses:

None

Parameters

window – Window or canvas

Returns

Current window snap point

getDBLimits

Syntax

```
getDBLimits(  
    *,  
    expression: str,  
    dbHandle: str,  
    dbTableNames: list[str])  
Returns: float
```

Summary:

Gets database limits for given expression

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- expression – Expression to be evaluated in given database tables
- dbHandle – SQLite database handle
- dbTableNames – List of tables on which given expression should be evaluated

Returns

List of minimum and max values

getDesignTemplates

Syntax

```
getDesignTemplates(  
*,  
    cellView: dmCellView | dmCellViewName)  
Returns: list[dmCellView]
```

Summary:

Get master schematic templates in the given design

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

cellView – The top cellview.

Returns

A collection of CellViews

getEditors

Syntax

```
getEditors(  
    name: str = None)  
Returns: dbCollection
```

Summary:

Returns all registered editors, optionally restricted to those whose name matches a given glob-style pattern.

Category:

Foundation

Licenses:

None

Parameters

name – Pattern to match editor names against. (Default is: “)

Returns

All registered editors

getErrorViewerDatabases

Syntax

```
getErrorViewerDatabases(  
    pattern: str = None)  
Returns: dbCollection
```

Summary:

Get all deErrorViewerDatabases

Category:

Error Viewer

Licenses:

None

Parameters

pattern – Pattern to match object name.

Returns

deErrorViewerDatabase objects matching search criteria.

getFavoriteDeviceCategories

Syntax

```
getFavoriteDeviceCategories()  
Returns: dbCollection
```

Arguments:

Summary:

Gets a collection of device categories

Category:

Design Editing

Licenses:

None

Returns

Collection of device categories

getFavoriteDevices

Syntax

```
getFavoriteDevices()  
Returns: dbCore
```

Arguments:

Summary:

Gets a collection of devices

Category:

Design Editing

Licenses:

None

Returns

Collection of devices.

getFigures

Syntax

```
getFigures(  
objects: list[point | list[deFigure] | list[deSelectionSet] |  
list[oaFig] | list[oaFigGroup] | list[oaInst] | list[oaMarker] |  
list[oaShape] | list[oaVia]] = None,  
/,  
*,  
type: de.getFigures.Type = None,  
design: oaDesign = None,  
touch: bool = None,  
depth: int = None,  
partial: bool = None,  
lpps: list[deTechLPP] = None,  
objectTypes: list[deObjectFilter] = None,  
excludeObjectTypes: list[deObjectFilter] = None,  
hierPath: oaHierPath = None)  
Returns: list[deFigure]
```

Summary:

Returns the figures in a given region.

Category:

Design Editing

Licenses:

None

Parameters

- objects – Figures or query region points.
- type – Indicates whether points represent ({ line | point | polygon | rectangle })
- design – The design.
- touch – Returns figures which touch the region in addition to those enclosed by it (Default is: False)
- depth – Number of hierarchical levels to traverse. A negative number indicates all levels (Default is: 0)
- partial – Return partial figures (Default is: False)

- lpps – Only return shapes on the specified LPPs.
- objectTypes – Only return objects matching specified filters.
- excludeObjectTypes – Exclude objects matching specified filters.
- hierPath – Root hierarchical path from where to find figures

Returns

Objects matching criteria

getHighlighted

Syntax

```
getHighlighted(  
*,  
    set: deHighlightSet = None)  
Returns: list[deFigure]
```

Summary:

Returns iterator over highlight set elements.

Category:

Design Editing

Licenses:

None

Parameters

set – Set to get highlights from

Returns

Collection of objects from the set

getHighlightSets

Syntax

```
getHighlightSets(  
    name: str = None,  
    /,  
    *,  
    design: oaDesign = None,  
    context: deContext = None)  
Returns: list[deHighlightSet]
```

Summary:

Returns highlight set with given name.

Category:

Design Editing

Licenses:

None

Parameters

- name – Glob-style pattern to match highlight set name against (Default is: “")
- design – Design from which to retrieve highlight set
- context – Context from which to return all highlight sets registered between the top and edit designs

Returns

Matching highlight set

getInterfaceElements

Syntax

```
getInterfaceElements(  
*,  
    context: heContext,  
    netlister: nlAMSNetlister,  
    hierarchy: bool = None)  
Returns: dbCollection
```

Summary:

Retrieves all interface elements created for the given design context for partition visualization according to the given AMS netlister.

Category:

Netlist Tools

Licenses:

None

Parameters

- context – Hierarchy context
- netlister – AMS netlister according to which the partition is made.
- hierarchy – If is true, not only the IEs of the current design, but all the IEs of the entire hierarchy are collected. (Default is: False)

Returns

Collection of interface element objects.

getLayers

Syntax

```
getLayers(  
    name: str = None,  
    /,  
    *,  
    from_: deContext | oaDesign = None,  
    of: deContext | oaDesign = None)  
Returns: list[deLayer]
```

Summary:

Returns an ordered collection of deLayer objects for a given design, along with the object validity, selectability and visibility settings. One deLayer object is returned for each layer referenced by an LPP in the specified container.

Category:

Layout Editing

Licenses:

None

Parameters

- name – Optional glob-style pattern matched against the layer name (Default is: “*”)
- of – Design or context from which the LPPs are retrieved

Returns

Ordered collection of deLayers matching the pattern and the filter.

getLookaheadMessages

Syntax

```
getLookaheadMessages()  
Returns: str
```

Arguments:

Summary:

Returns the list of look-ahead strings currently in the look-ahead buffer.

Category:

Foundation

Licenses:

None

Returns

Tcl list of lookahead messages

getLPPs

Syntax

```
getLPPs(  
name: lpp = None,  
/,  
*,  
from_: deContext | oaDesign | oaTech = None,  
of: deContext | oaDesign | oaTech = None,  
set: deLPPSet | str = None)  
Returns: list[deTechLPP]
```

Summary:

Returns an ordered collection of deLPP objects for a given design, along with the object validity, selectability and visibility settings.

Category:

Layout Editing

Licenses:

None

Parameters

- name – Layer Purpose pair (Default is: '* *')
- of – Design, context or technology from which the LPPs are retrieved
- set – deLPPSet object or set title

Returns

Collection of lpp matching the pattern and the filter

getLPPSets

Syntax

```
getLPPSets(  
    pattern: str = None  
)  
Returns: dbCollection
```

Summary:

Return a collection of LPP sets, optionally filtered by title. Built-in LPP sets are not returned.

Category:

Layout Editing

Licenses:

None

Parameters

pattern – Glob-style pattern to match against deLPPSet titles (Default is: '*')

Returns

Collection of matching deLPPSet objects.

getObjectFilters

Syntax

```
getObjectFilters(  
    name: str = None,  
    /,  
    *,  
    from_: deContext | oaDesign | list[deLayer] = None,
```

```
of: deContext | oaDesign | list[deLayer] = None)
Returns: list[deObjectFilter]
```

Summary:

Returns a collection of all OLP object filters from an open design, optionally filtered by group name or by -filter.

Category:

Layout Editing

Licenses:

None

Parameters

- name – Object filter name (Default is: '.')
- of – Design, context or layer from where the ObjectFilters must be get

Returns

Collection of object filters matching the pattern and the filter.

getOLPPresets

Syntax

```
getOLPPresets(  
pattern: str = None)  
Returns: dbCollection
```

Summary:

Returns a collection of all OLP presets defined in a session.

Category:

Layout Editing

Licenses:

None

Parameters

pattern – Glob-style pattern to match preset title (Default is: '*')

Returns

Collection of matching deOLPPreset objects.

getOpenFiles

Syntax

```
getOpenFiles(  
*,  
    context: deContext = None,  
    modified: bool = None)  
Returns: list[dmDMFile]
```

Summary:

Returns all currently open files, optionally restricted by modified state or being open in given context.

Category:

Foundation

Licenses:

None

Parameters

- context – Only return files open in specified context.
- modified – Only return files which are either modified or unmodified by providing this flag. If unspecified, all files will be returned.

Returns

All currently open files (in any context)

getOutdatedDesigns

Syntax

```
getOutdatedDesigns()  
Returns: list[oaDesign]
```

Arguments:

Summary:

Returns a collection of designs that have changed on disk since they were initially opened.

Category:

Foundation

Licenses:

None

Returns

Designs that are newer on disk than in memory.

getOutdatedDMData

Syntax

```
getOutdatedDMData()  
Returns: list[oaDMData]
```

Arguments:

Summary:

Returns a collection of DMData that has been changed on disk since was initially opened.

Category:

Foundation

Licenses:

None

Returns

DMData that is newer on disk than in memory.

getOverlays

Syntax

```
getOverlays(  
    name: str = None,  
    /,  
    *,  
    base: oaDesign = None)  
Returns: dbCollection
```

Summary:

Returns all registered overlays.

Category:

Foundation

Licenses:

None

Parameters

- name – Glob-style pattern to match against overlay name (Default is: '*')
- base – If specified, returned collection will return deOverlays registered for this design.

Returns

All registered overlays

getPartitioner

Syntax

```
getPartitioner(  
*,  
    context: deContext,  
    netlist: nlAMSNetlist = None)  
Returns: nlPartitioner
```

Summary:

Retrieves partitioner created for the given design context according to the given AMS netlist

Category:

Netlist Tools

Licenses:

None

Parameters

- context – design context
- netlist – AMS netlist according to which the partition is made

Returns

Partitioner object

getProbeBitNames

Syntax

```
getProbeBitNames(  
*,  
    probe: deProbe,  
    context: deContext,  
    includeSymbolPins: bool = None,  
    maxDepth: int = None)  
Returns: str
```

Summary:

Finds all equivalent bit names for the probe.

Category:

Design Editing

Licenses:

None

Parameters

- probe – Probe for which the bit names are retrieved
- context – Design context for which the bit names are retrieved
- includeSymbolPins – Get symbol pin bit nets. (Default is: True)
- maxDepth – Get the maximum level to search below the given context (Default is: -1)

Returns

List of bit names involved to the probe, include equivalent bit nets

getProbeFigures

Syntax

```
getProbeFigures(  
*,  
    probe: deProbe,  
    context: deContext)  
Returns: list[deFigure]
```

Summary:

Finds figures (both 0x and 1x) highlighted for the probe in the specified design context.

Category:

Design Editing

Licenses:

None

Parameters

- probe – Probe for which the figures are retrieved
- context – Design context for which the figures are retrieved

Returns

Collection of figures involved to this probe highlighting

getProbeNets

Syntax

```
getProbeNets (  
    *,  
    probe: deProbe,  
    context: deContext)  
Returns: list[oaNet]
```

Summary:

Finds bit net(s) highlighted for the net probe in the specified context.

Category:

Design Editing

Licenses:

None

Parameters

- probe – Probe for which the nets are retrieved
- context – Design context for which the nets are retrieved

Returns

Collection of bit nets involved to the probe highlighting

getProbes

Syntax

```
getProbes(  
*,  
    context: deContext | heContext,  
    object: deFigure | oaFig = None,  
    allHierarchy: bool = None)  
Returns: list[deProbe]
```

Summary:

Finds probes which correspond to a specified object. If an object is not supplied, returns all probes defined relative to the specified heContext including lineage.

Category:

Design Editing

Licenses:

None

Parameters

- context – Context for retrieving probes.
- object – If specified, returns probes whose highlight set includes the given object
- allHierarchy – If true, the probes are found in the entire hierarchy starting from top. (Default is: False)

Returns

Collection of deProbe objects defined relative to specified design

getRecentFiles

Syntax

```
getRecentFiles()  
Returns: list[dmDMFile]
```

Arguments:

Summary:

Returns a collection of recently opened files. The collection is ordered such that the first element is the most recently opened file and subsequent elements are increasingly less recently opened.

Category:

Foundation

Licenses:

None

Returns

Recently opened files in decreasingly-recent order

getSelected

Syntax

```
getSelected(  
*,  
    design: oaDesign,  
    partial: bool = None)  
Returns: list[deFigure]
```

Summary:

Returns iterator over selection set elements.

Category:

Design Editing

Licenses:

None

Parameters

- design – Design from which to return selected objects
- partial – Include partial regions in result (Default is: True)

Returns

Collection of objects from the set

getSelectedLogic

Syntax

```
getSelectedLogic(  
*,  
    context: deContext,
```

```
hier: bool = None  
Returns: dbCollection
```

Summary:

Returns a collection of logic selected in the specified design context.

Category:

Design Editing

Licenses:

None

Parameters

- context – Design context from which to retrieve selected logic.
- hier – If false, return only logic in the edit design. (Default is: False)

Returns

Selected logic

getSelectedVertex

Syntax

```
getSelectedVertex(  
deFigure: list[deFigure])  
Returns: None
```

Summary:

Return the cordinate(s) of selected vertexes.

Category:

Design Editing

Licenses:

None

Parameters

deFigure – Return point of vertex selected

Returns

None

getTemplates

Syntax

```
getTemplates(  
  glob: str = None,  
  /,  
  *,  
  libName: dmLibName = None)  
Returns: dbCollection
```

Summary:

Returns a collection of deTemplate objects for each template found in the given library or in all libraries.

Category:

Templates Category

Licenses:

None

Parameters

- glob – Glob-style name filter. (Default is: '*')
- libName – Name of library to search. (Default is: '')

Returns

Collection of deTemplate objects

getTools

Syntax

```
getTools(  
  name: str = None)  
Returns: dbCollection
```

Summary:

Returns all registered tools.

Category:

Foundation

Licenses:

None

Parameters

name – Pattern to match tool names against. (Default is: “)

Returns

All registered tools

getTransactions

Syntax

```
getTransactions(  
    pattern: str = None,  
    /,  
    *,  
    design: oaDesign = None,  
    config: heConfig = None,  
    regexp: bool = None)  
Returns: list[deTransaction]
```

Summary:

Get Transactions

Category:

Foundation

Licenses:

None

Parameters

- pattern – Pattern to filter the transaction names (Default is: ‘*’)
- design – oaDesign option
- config – Config object
- regexp – True, if the pattern is a regular expression. False, if it uses glob-style wildcards. (Default is: False)

Returns

Transaction objects found

getViewport

Syntax

```
getViewport(  
*,  
    window: deCanvas | giWindow | int = None,  
    ofEdit: bool = None)  
Returns: box
```

Summary:

Returns the coordinates of current viewport

Category:

Navigation

Licenses:

None

Parameters

- window – Window or canvas
- ofEdit – When true, the command returns the viewport of the active edit design. Otherwise it returns that of the active view design (Default is: True)

Returns

The current viewport

gotoViewport

Syntax

```
gotoViewport(  
*,  
    window: deCanvas | giWindow | int = None,  
    dir: int = None)  
Returns: box
```

Summary:

Navigates the viewport history

Category:

Navigation

Licenses:

None

Parameters

- window – Window or canvas
- dir – Negative numbers go backward in history and positive numbers forward (Default is: -1)

Returns

The new viewport

help

Syntax

```
help(  
*,  
    helpID: str = None,  
    collectionID: str = None,  
    command: str = None,  
    window: giWindow | int = None,  
    assistant: giAssistant | str = None,  
    dialog: giDialog | str = None)  
Returns: None
```

Summary:

Displays a help topic.

Category:

Foundation

Licenses:

None

Parameters

- helpID – Help ID of topic.
- collectionID – Collection ID of topic.
- command – Command for which to show help.
- window – Window(id) for which to show help.

- assistant – Assistant for which to show help.
- dialog – Dialog for which to show help.

Returns

None

highlight

Syntax

```
highlight(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
         | list[oaVia],  
/,  
*,  
set: deHighlightSet = None)  
Returns: None
```

Summary:

Adds a collection of objects to the highlight set

Category:

Design Editing

Licenses:

None

Parameters

- objects – Objects to highlight
- set – Highlight set into which to add objects

Returns

None

importBookmarks

Syntax

```
importBookmarks(  
*,  
filePath: str,
```

```
force: bool = None)
Returns: list[deBookmark]
```

Summary:

Imports bookmarks from a specified file.

Category:

Navigation

Licenses:

None

Parameters

- filePath – The path to an external bookmark file
- force – If false, the command will throw an error if the bookmark with the same name already exists. If true, the bookmark will be silently overwritten. (Default is: True)

Returns

The imported bookmarks

importProbeCellSettings

Syntax

```
importProbeCellSettings(  
*,  
file: str)  
Returns: None
```

Summary:

Import Probe Trace Through Cell settings from specified file.

Category:

Design Editing

Licenses:

None

Parameters

file – File Name to import the Probe Trace Through Cell Settings

Returns

None

isEditable

Syntax

```
isEditable(  
    object: deContext | dmCellView | dmCellViewName | dmDMFile |  
           oaDesign)  
Returns: bool
```

Summary:

Retrieves the open mode of the edit design in the specified context or an arbitrary design opened within any context.

Category:

Foundation

Licenses:

None

Parameters

object – Design from which to retrieve the edit mode. If a context is specified, the edit design is implied. If a dmCellView is specified, the primary file is implied.

Returns

Edit mode for specified design (or edit design of specified context)

isInUndo

Syntax

```
isInUndo(  
    *,  
    design: oaDesign)  
Returns: bool
```

Summary:

Checks if the design is in undo or not

Category:

Foundation

Licenses:

None

Parameters

design – oaDesign option

Returns

Is in Undo

isModified

Syntax

```
isModified(  
  object: deContext | dmCellView | dmCellViewName | dmDMFile |  
    oaDesign)  
Returns: bool
```

Summary:

Returns true if the specified file is open and modified within any context. If a context is provided, the edit design will be queried.

Category:

Foundation

Licenses:

None

Parameters

object – File or context from which to query the modification state. If a context is specified, the edit design is implied. If a dmCellView is specified, the primary file is implied.

Returns

Whether the file is modified

linkContext

Syntax

```
linkContext(  
  context: deContext,  
  /,  
  *,  
  ,
```



```
to: deContext,  
name: str,  
title: str,  
mapToProc: list[variant] = None,  
mapFromProc: list[variant] = None)  
Returns: deLinkedContext
```

Summary:

Links a context to another context, specifying a name and title for the link.

Category:

Foundation

Licenses:

None

Parameters

- context – Context to link to another context.
- to – Context to link with.
- name – Link name, e.g., lxSchematic or cpHelixResults.
- title – Title to use in the UI, e.g., SDL Schematic or Helix Results.
- mapToProc – Mapping Tcl procedure from layout object to linked context compatible object.
- mapFromProc – Mapping Tcl procedure from linked context compatible object to layout object.

Returns

New object.

loadFavoriteDevices

Syntax

```
loadFavoriteDevices(  
*,  
filePath: str)  
Returns: None
```

Summary:

Loads favorite devices from a file on disk

Category:

Design Editing

Licenses:

None

Parameters

filePath – Path of file to load

Returns

None

loadInterfaceElements

Syntax

```
loadInterfaceElements(  
*,  
    filePath: str)  
Returns: dbCollection
```

Summary:

Loads interface elements saved in the specified file.

Category:

Netlist Tools

Licenses:

None

Parameters

filePath – Filename to load

Returns

Collection of interface element objects successfully loaded from file.

loadOLPPresets

Syntax

```
loadOLPPresets(  
*,
```

```
filePath: str)
Returns: None
```

Summary:

Loads presets from the given xml file

Category:

Layout Editing

Licenses:

None

Parameters

filePath – file path

Returns

None

loadProbes

Syntax

```
loadProbes (
*,
filePath: str)
Returns: list[deProbe]
```

Summary:

Loads probes defined in the specified file. The context stored in the probe file must be opened in a DE editor window. Any probe defined in the file which is not valid will be ignored.

Category:

Design Editing

Licenses:

None

Parameters

filePath – Filename to load.

Returns

Collection of deProbe objects created from the file

navigateTo

Syntax

```
navigateTo(  
  lineage: heContext | oaHierPath | str,  
  /,  
  *,  
  context: deContext,  
  readOnly: bool = None)  
Returns: deContext
```

Summary:

Navigates to the specified occurrence path via a sequence of return and descend operations.

Category:

Navigation

Licenses:

None

Parameters

- lineage – Lineage specification.
- context – Context in which to descend/return.
- readOnly – Specifies the open mode for the final design in the lineage. (Default is: True)

Returns

Modified context object

next

Syntax

```
next(  
  count: int | None = None)  
Returns: None
```

Summary:

Executes the specified number of commands of the log file.

Category:

Foundation

Licenses:

None

Parameters

count – Number of commands to execute. (Default is: 1)

Returns

None

open

Syntax

```
open (  
object: dmCellViewName | list[deBookmark] | list[dmCellView] |  
list[dmDMFile],  
/  
*,  
readOnly: bool = None,  
instPath: str = None,  
depth: int = None,  
application: deApplication | str = None,  
headless: bool = None)  
Returns: deContext
```

Summary:

Spawns the registered application for the specified file (or primary file of a cellView).

Category:

Foundation

Licenses:

None

Parameters

- object – One or more cellview (primary file), DM file or bookmark objects to open.
- readOnly – Opens the design in read-only mode. (Default is: False)
- instPath – Optional instance path to edit. (Default is: '/')
- depth – Open a maximum of -depth levels from this design.

- application – Application to open
- headless – No window will be created for the new context. (Default is: False)

Returns

Newly created context object

openCanvasPropertyEditor

Syntax

```
openCanvasPropertyEditor(  
*,  
    figure: deFigure,  
    context: deContext)  
Returns: None
```

Summary:

Open On Canvas Property Editor

Category:

Foundation

Licenses:

None

Parameters

- figure – The figure
- context – Designe context which contains an figure

Returns

None

openClipboardLib

Syntax

```
openClipboardLib(  
none: str,  
    /,  
*,  
    path: str)  
Returns: oaLib
```

Summary:

Opens a clipboard library

Category:

Clipboard

Licenses:

None

Parameters

- none – Library name
- path – Library path

Returns

Clipboard library

openEditor**Syntax**

```
openEditor(  
editor: deEditor = None,  
/,  
*,  
context: deContext,  
attrs: dbObject = None,  
applyWorkspace: bool = None)  
Returns: deEditorInst
```

Summary:

Launches an editor for the specified context.

Category:

Foundation

Licenses:

None

Parameters

- editor – Editor to invoke.
- context – Context to open in an editor.

- `attrs` – Initial attributes for the editor.
- `applyWorkspace` – Apply last-used workspace

Returns

Newly created editor instance

openFile

Syntax

```
openFile(  
  object: dmCellView | dmCellViewName | dmDMFile | oaDesign,  
  /,  
  *,  
  context: deContext)  
Returns: None
```

Summary:

Opens a design file for write, in an existing design context.

Category:

Foundation

Licenses:

None

Parameters

- `object` – Design file to open.
- `context` – Context in which to open the file.

Returns

None

pan

Syntax

```
pan(  
  *,  
  window: deCanvas | giWindow | int = None,  
  to: point = None,  
  direction: de.pan.Direction = None,  
  units: float = None,
```



```
multiplier: float = None)
Returns: None
```

Summary:

Moves the viewport to different position on the canvas without changing the scale factor

Category:

Navigation

Licenses:

None

Parameters

- window – Window or canvas
- to – Coordinate to pan
- direction – Direction to pan ({ E | N | NE | NW | S | SE | SW | W })
- units – Shifts the viewport the specified number of units
- multiplier – Multiplier of viewport width or height by which to pan (Default is: 0.5)

Returns

None

paste

Syntax

```
paste(
    oaDesign: oaDesign,
    /,
    *,
    point: point,
    orient: de.paste.Orient = None,
    connectivity: bool = None,
    highlight: bool = None,
    columns: int = None,
    cols: int = None,
    dxColumn: float = None,
    dxCol: float = None,
    dyColumn: float = None,
    dyCol: float = None,
    rows: int = None,
    dxRow: float = None,
    dyRow: float = None,
```

```
clip: deClipboard = None)  
Returns: None
```

Summary:

Places copies of objects from the clipboard buffer into the target design.

Category:

Design Editing

Licenses:

Basic or Frontend

Parameters

- oaDesign – Design where objects will be pasted.
- point – Point in design at which the clipboard contents will be pasted
- orient – Rotation and mirroring for objects within the clipboard ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 })
- connectivity – Option to preserve connectivity on paste. (Default is: True)
- highlight – Option to preserve highlight on paste (Default is: False)
- columns – Number of copies along the X axis to create. (Default is: 1)
- cols – Number of copies along the X axis to create. (Default is: 1)
- dxColumn – Distance from column to column on X direction for arrayed copy in uu.
- dxCol – Distance from column to column on X direction for arrayed copy in uu.
- dyColumn – Distance from column to column on Y direction for non-aligned arrayed copy in uu.
- dyCol – Distance from column to column on Y direction for non-aligned arrayed copy in uu.
- rows – Number of copies along the Y axis to create. (Default is: 1)
- dxRow – Distance from row to row on X direction for non-aligned arrayed copy in uu
- dyRow – Distance from row to row on Y direction for arrayed copy in uu
- clip – Clip to paste

Returns

None

redoSelection

Syntax

```
redoSelection(  
*,  
    design: oaDesign)  
Returns: None
```

Summary:

Undo last selection operation.

Category:

Design Editing

Licenses:

None

Parameters

design – Design in which to redo last selection

Returns

None

redoTransaction

Syntax

```
redoTransaction(  
*,  
    steps: int = None,  
    transaction: deTransaction = None,  
    design: oaDesign = None,  
    config: heConfig = None)  
Returns: None
```

Summary:

Redo Transaction

Category:

Foundation

Licenses:

None

Parameters

- steps – Number of operations to be redone (Default is: 1)
- transaction – Transaction to move forward the transaction history
- design – Design object
- config – Config object

Returns

None

redraw

Syntax

```
redraw(  
*,  
    window: deCanvas | giWindow | int = None,  
    **kwargs)  
Returns: None
```

Summary:

Redraw the active widget

Category:

Navigation

Licenses:

None

Parameters

window – Window or canvas

Returns

None

registerHelp

Syntax

```
registerHelp(  
*,  
    helpID: str = None,  
    collectionID: str = None,
```

```
target: list[str],
type: de.registerHelp.Type,
command: deCommand | str = None,
windowType: giWindowType | str = None,
assistant: giAssistant | str = None,
dialog: giDialog | str = None)
Returns: None
```

Summary:

Registers a new topic.

Category:

Foundation

Licenses:

None

Parameters

- helpID – Help ID of topic.
- collectionID – Collection ID of topic.
- target – Target of topic, related to type.
- type – Type of topic. ({ pdf | proc | url })
- command – Command for which to create a topic.
- windowType – Window type for which to create a topic.
- assistant – Assistant for which to create a topic.
- dialog – Dialog for which to create a topic.

Returns

None

removeBreakpoints

Syntax

```
removeBreakpoints (
lines: list[int] = None)
Returns: None
```

Summary:

Removes break points from the log file which is being replayed.

Category:

Foundation

Licenses:

None

Parameters

lines – Tcl list of break point line numbers to remove.

Returns

None

removeObject

Syntax

```
removeObject(  
  object: deFigure,  
  /,  
  *,  
  window: giWindow | int = None,  
  context: deContext = None)  
Returns: None
```

Summary:

Remove object event's equivalent command.

Category:

Foundation

Licenses:

None

Parameters

- object – Object to deselect.
- window – Window in which to send the event.
- context – Context in which to send the event.

Returns

None

removePoint

Syntax

```
removePoint(  
point: point,  
/,  
*,  
window: giWindow | int = None,  
context: deContext = None)  
Returns: None
```

Summary:

Removes the last added point

Category:

Foundation

Licenses:

None

Parameters

- point – Point to remove, in user units
- window – Window in which to send the event.
- context – Context in which to send the event.

Returns

None

removeTool

Syntax

```
removeTool(  
tool: deTool,  
/,  
*,  
editor: deEditor | str = None)  
Returns: None
```

Summary:

Removes a tool from its association with an editor and thus makes the tool unavailable for designs opened with the given association.

Category:

Foundation

Licenses:

None

Parameters

- tool – Tool to remove from the given association.
- editor – Editor for which tool is no longer available.

Returns

None

renumberInstances

Syntax

```
renumberInstances(  
instances: list[deFigure] | list[oaFig] | list[oaInst],  
/,  
*,  
ordering: de.renumberInstances.Ordering = None,  
startFrom: int = None,  
customPrefix: str = None,  
prefixToUse: de.renumberInstances.PrefixToUse = None)  
Returns: list[oaInst]
```

Summary:

Renumbers the instances in the specified collection.

Category:

Design Editing

Licenses:

Basic or Frontend

Parameters

- instances – Collection of instances to modify from the same design
- ordering – Specifies if the renumbering will be row or column based. ({ column | row } (Default is: 'row'))
- startFrom – The start number to use for indexing. (Default is: 0)

- **customPrefix** – The additional prefix string to append while composing prefixes. (Default is: “)
- **prefixToUse** – The number prefix to use while indexing instances ({ current | customOnly | default }) (Default is: ‘current’)

Returns

Collection of modified instances

repeatCommand

Syntax

```
repeatCommand(  
*,  
    window: giWindow | int = None,  
    context: deContext = None)  
Returns: None
```

Summary:

Restarts a previous command

Category:

Foundation

Licenses:

None

Parameters

- **window** – Window in which to start the command.
- **context** – Context in which to start the command.

Returns

None

replay

Syntax

```
replay(  
text: str | None = None,  
/,  
*,  
filepath: str | None = None,
```

```
body: str | None = None,  
interactive: bool | None = None)  
Returns: None
```

Summary:

Reruns the commands and other operations in the log file or as specified by the command arguments.

Category:

Foundation

Licenses:

None

Parameters

- text – File or log body to replay.
- filepath – File to replay.
- body – Replay log body.
- interactive – Enters replay in interactive debug mode. (Default is: False)

Returns

None

replayCommand

Syntax

```
replayCommand(  
name: deCommand | str = None,  
/,  
*,  
window: giWindow | int = None,  
context: deContext = None)  
Returns: None
```

Summary:

Replay an interactive edit on the current select set

Category:

Foundation

Licenses:

None

Parameters

- name – Command to replay
- window – Window in which to replay the command.
- context – Context in which to replay the command.

Returns

None

reportOpenTransactions

Syntax

```
reportOpenTransactions(  
*,  
    design: oaDesign = None,  
    type: de.reportOpenTransactions.Type = None,  
    close: bool = None)  
Returns: None
```

Summary:

Report Open Transactions.

Category:

Foundation

Licenses:

None

Parameters

- design – Design for which to report open transactions.
- type – Transaction types to report. ({ all | design | runtime } (Default is: 'all'))
- close – If true, close any open transactions. (Default is: False)

Returns

None

restoreBookmark

Syntax

```
restoreBookmark(  
bookmark: deBookmark,  
/,  
*,  
window: giWindow | int)  
Returns: None
```

Summary:

This command restores any edit-context bookmark.

Category:

Navigation

Licenses:

None

Parameters

- bookmark – The bookmark to restore.
- window – The window in which to restore the bookmark.

Returns

None

return_

Syntax

```
return_(  
context: deContext,  
/,  
*,  
levels: int = None,  
**kwargs)  
Returns: deContext
```

Summary:

Closes the edit file, returning to the parent file. If the edit file is the view file, the view file will also change.

Category:

Navigation

Licenses:

None

Parameters

- context – Context object in which to return.
- levels – Number of levels to close. Use a negative number to indicate all levels. (Default is: 1)

Returns

Modified context object

revert

Syntax

```
revert(  
collection: dbCollection | dmCellViewName | list[dmCellView] |  
list[dmDMFile] | list[oaDesign],  
/,  
*,  
force: bool = None)  
Returns: None
```

Summary:

Discards all unsaved changes in the specified files, restoring it to the on-disk version.

Category:

Foundation

Licenses:

None

Parameters

- collection – Collection of contexts whose edit design to revert
- force – Reverts the design even if there are unsaved edits and its open in a graphical editor (Default is: False)

Returns

None

run

Syntax

```
run()  
Returns: None
```

Arguments:

Summary:

Executes commands of the log file until a breakpoint or the end of file is reached.

Category:

Foundation

Licenses:

None

Returns

None

runBackgroundImporter

Syntax

```
runBackgroundImporter(  
objects: list[dmCell] | list[dmCellView] | list[oaLib])  
Returns: None
```

Summary:

Runs background importer for digital designs.

Category:

Foundation

Licenses:

Backend

Parameters

objects – A collection of selected oaLib or dmCell or dmCellView.

Returns

None

save

Syntax

```
save(  
  collection: dbCollection | dmCellViewName | oaDesign |  
    list[dmCellView] | list[dmDMFile])  
Returns: None
```

Summary:

Saves the specified files which must be opened in a context.

Category:

Foundation

Licenses:

None

Parameters

collection – Collection of contexts whose edit file will be saved or the file itself.

Returns

None

saveAs

Syntax

```
saveAs(  
  object: deContext | dmCellView | dmCellViewName | dmDMFile |  
    oaDesign,  
  /,  
  *,  
  to: dmCellView | dmCellViewName,  
  force: bool = None)  
Returns: None
```

Summary:

Saves the specified file into a new or existing cellView.

Category:

Foundation

Licenses:

None

Parameters

- `object` – Context whose edit file will be saved or the file itself.
- `to` – Target cellview into which the file will be saved.
- `force` – When saving to a different cellview which already exists, abort unless `-force` is true.

Returns

None

saveFavoriteDevices

Syntax

```
saveFavoriteDevices(  
    _devices_: dbCollection = None,  
    /,  
    *,  
    filePath: str,  
    force: bool = None)  
Returns: None
```

Summary:

Writes favorite devices to a file on disk, optionally overwriting an existing file

Category:

Design Editing

Licenses:

None

Parameters

- `_devices_` – Collection of devices to save. If omitted, all favorite devices are saved
- `filePath` – Path of file to write
- `force` – If true, overwrite existing files (Default is: False)

Returns

None

saveInterfaceElements

Syntax

```
saveInterfaceElements(  
    elements: dbCollection,  
    /,  
    *,  
    filePath: str,  
    force: bool = None)  
Returns: None
```

Summary:

Saves interface elements to the specified file.

Category:

Netlist Tools

Licenses:

None

Parameters

- elements – Interface elements to save
- filePath – Filename to save
- force – Overwrite existing file (Default is: False)

Returns

None

saveOLPPresets

Syntax

```
saveOLPPresets(  
    preset: dbCollection,  
    /,  
    *,  
    filePath: str,  
    overwrite: bool = None)  
Returns: None
```

Summary:

Saves deOLPPreset state from the current design state to tagged xml file.

Category:

Layout Editing

Licenses:

None

Parameters

- preset – Collection of matching deOLPPreset objects.
- filePath – file path
- overwrite – overwrites the file (Default is: False)

Returns

None

saveProbes

Syntax

```
saveProbes(  
    probeList: list[deProbe],  
    /,  
    *,  
    filePath: str,  
    force: bool = None)  
Returns: None
```

Summary:

Saves specified probes defined for a design to a specified file. If the filename exists, the command will throw an error unless force is true. The file will be written even if no probes exist in the given probes collection. All probes in the collection must come from the same root context

Category:

Design Editing

Licenses:

None

Parameters

- `probeList` – List of probe objects to save.
- `filePath` – File path for probes to save.
- `force` – Override file if already exists. (Default is: False)

Returns

None

select

Syntax

```
select(  
  objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
           list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
           | list[oaVia],  
  /,  
  *,  
  replace: bool = None)  
Returns: None
```

Summary:

Selects specified object, sub-object or collection.

Category:

Design Editing

Licenses:

None

Parameters

- `objects` – Collection of selected figures
- `replace` – Deselect all objects not included in the specified figure set (Default is: False)

Returns

None

selectAll

Syntax

```
selectAll(  
    context: deContext)  
Returns: list[deFigure]
```

Summary:

Selects (adds to active select set) all objects in the design associated with the specified select set.

Category:

Design Editing

Licenses:

None

Parameters

context – Design in which to select all objects

Returns

Collection of selected figures

selectLogic

Syntax

```
selectLogic(  
    objects: dbCollection | list[oaFig] | list[oaFigGroup] |  
        list[oaGroup] | list[oaInst] | list[oaInstTerm] | list[oaNet] |  
        list[oaPin] | list[oaTerm],  
    /,  
    *,  
    context: deContext)  
Returns: dbCollection
```

Summary:

Select logic within the specified design context.

Category:

Design Editing

Licenses:

None

Parameters

- objects – Logic to select
- context – Context within which to select logic.

Returns

Collection of selected hierarchical objects

sendComplete

Syntax

```
sendComplete(  
  command: deCommand,  
  /,  
  *,  
  eventData: list[deFigure] | list[oaFig] | list[oaFigGroup] |  
             list[oaInst] | list[oaShape] | list[oaVia])  
Returns: None
```

Summary:

Sends an OnPostCommandComplete event to Custom Infrastructure. This command is used by Tcl Interactive Commands to signal the completion of an interactive edit.

Category:

Foundation

Licenses:

None

Parameters

- command – The Interactive Command object which complete a cycle.
- eventData – Event data to send with the event.

Returns

None

sendMessage

Syntax

```
sendMessage(  
    message: str,  
    /,  
    *,  
    severity: de.sendMessage.Severity = None,  
    logFiles: list[str] = None)  
Returns: None
```

Summary:

Sends an informational, warning or error message to the console or shell as well as to the log file.

Category:

Foundation

Licenses:

None

Parameters

- `message` – The message string.
- `severity` – The message severity. ({ `error` | `information` | `logComment` | `warning` }) (Default is: 'information')
- `logFiles` – Tcl list of secondary log files to which the message will also be sent.

Returns

None

setActiveLPP

Syntax

```
setActiveLPP(  
    lpp: deLayer | deTechLPP)  
Returns: None
```

Summary:

Sets the active LPP.

Category:

Layout Editing

Licenses:

None

Parameters

lpp – LPP to make active.

Returns

None

setBreakpoint

Syntax

```
setBreakpoint(  
line: int | None = None)  
Returns: None
```

Summary:

Sets a break point in the log file which is being replayed.

Category:

Foundation

Licenses:

None

Parameters

line – Line number of a break point to be set.

Returns

None

setCursor

Syntax

```
setCursor(  
window: deCanvas | giWindow | int = None,  
/,  
*,
```

```
point: point,  
**kwargs)  
Returns: point
```

Summary:

Returns cursor current position.

Category:

Design Editing

Licenses:

None

Parameters

- window – Window or canvas
- point – Point to move cursor to.

Returns

The new cursor location

setEditMode

Syntax

```
setEditMode(  
collection: dbCollection | dmCellViewName | list[dmCellView] |  
list[dmDMFile] | list[oaDesign],  
/,  
*,  
editable: bool,  
save: bool = None)  
Returns: None
```

Summary:

Changes the open mode of the specified files.

Category:

Foundation

Licenses:

None

Parameters

- collection – Collection of contexts containing edit designs to make editable.
- editable – If true, design will be set to read/write mode. If false, design will be set to read-only mode.
- save – Save changes prior to changing to read-only.

Returns

None

setErrorViewerCategory

Syntax

```
setErrorViewerCategory(  
of: deErrorViewerDatabase,  
/,  
*,  
fields: de.setErrorViewerCategory.Fields,  
hide: de.setErrorViewerCategory.Hide = None)  
Returns: None
```

Summary:

Set the category of deErrorViewerDatabase

Category:

Error Viewer

Licenses:

None

Parameters

- of – The deErrorViewerDatabase object
- fields – List of fields that forms the category ({ associatedObjects | design | message | parentMarkers | severity | shortMsg | tool | user1 | user2 | user3 | user4 })
- hide – Specify a list of fields that will not be selected by default in the category menu ({ associatedObjects | design | message | parentMarkers | severity | shortMsg | tool | user1 | user2 | user3 | user4 })

Returns

None

setErrorViewerDetailInfo

Syntax

```
setErrorViewerDetailInfo(  
  of: deErrorViewerDatabase,  
  /,  
  *,  
  fields: de.setErrorViewerDetailInfo.Fields)  
Returns: None
```

Summary:

Customize the detail information for deErrorViewerDatabase

Category:

Error Viewer

Licenses:

None

Parameters

- of – The deErrorViewerDatabase object
- fields – Specify the fields to be shown in the detail information widget in error viewer
({ associatedObjects | design | message | parentMarkers | severity | shortMsg | tool |
user1 | user2 | user3 | user4 })

Returns

None

setErrorViewerFieldName

Syntax

```
setErrorViewerFieldName(  
  of: deErrorViewerDatabase,  
  /,  
  *,  
  field: de.setErrorViewerFieldName.Field,  
  name: str)  
Returns: None
```

Summary:

Customize the field name for each field.

Category:

Error Viewer

Licenses:

None

Parameters

- of – Specify the deErrorViewerDatabase object
- field – Specify the field ({ associatedObjects | design | message | parentMarkers | severity | shortMsg | tool | user1 | user2 | user3 | user4 })
- name – Specify the name for the field

Returns

None

setErrorViewerTip

Syntax

```
setErrorViewerTip(  
of: deErrorViewerDatabase,  
/,  
*,  
fields: de.setErrorViewerTip.Fields)  
Returns: None
```

Summary:

Customize the tip for deErrorViewerDatabase

Category:

Error Viewer

Licenses:

None

Parameters

- of – Specify the deErrorViewerDatabase object
- fields – Specify the fields to be shown in the tip ({ design | message | severity | shortMsg | tool | user1 | user2 | user3 | user4 })

Returns

None

setViewport

Syntax

```
setViewport(  
*,  
    window: deCanvas | giWindow | int = None,  
    box: box)  
Returns: box
```

Summary:

Sets the viewport to the rectangle chosen by user

Category:

Navigation

Licenses:

None

Parameters

- window – Window or canvas
- box – Bounding box for zooming

Returns

The new viewport

shiftCursor

Syntax

```
shiftCursor(  
window: deCanvas | giWindow | int = None,  
/,  
*,  
    point: point = None,  
    delta: point = None,  
    nsteps: int = None,  
    left: int = None,  
    right: int = None,  
    down: int = None,  
    up: int = None,
```

```
**kwargs)  
Returns: point
```

Summary:

Shifts the cursor with given parameters.

Category:

Design Editing

Licenses:

None

Parameters

- window – Window or canvas
- point – Point from which the cursor should be moved in user units.
- delta – Move vector (A pair of offsets in both directions.).
- nsteps – Number of steps in which to subdivide the move vector. (Default is: 1)
- left – Number of grid units to shift left. (Default is: 0)
- right – Number of grid units to shift right. (Default is: 0)
- down – Number of grid units to shift down. (Default is: 0)
- up – Number of grid units to shift up. (Default is: 0)

Returns

Final cursor position in user units

showAbout

Syntax

```
showAbout()  
Returns: None
```

Arguments:

Summary:

Displays the About Custom Infrastructure splash dialog.

Category:

Foundation

Licenses:

None

Returns

None

showBookmarkManager

Syntax

```
showBookmarkManager()  
Returns: None
```

Arguments:

Summary:

Command will show Bookmark Manager.

Category:

Navigation

Licenses:

None

Returns

None

showCloseDesigns

Syntax

```
showCloseDesigns(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows a dialog listing all open designs.

Category:

Foundation

Licenses:

None

Parameters

parent – Parent window on which this dialog will be centered. May be any window, not just design windows.

Returns

Newly created dialog.

showConnectionsNavigator

Syntax

```
showConnectionsNavigator(  
    window: giWindow | int = None,  
    /,  
    *,  
    parent: giWindow | int = None)  
Returns: None
```

Summary:

This command invokes inherited connections dialog.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- window – The window to use for showing inherited connections.
- parent – The window to use for showing inherited connections.

Returns

None

showConnectionsNavigatorOptions

Syntax

```
showConnectionsNavigatorOptions(  
    window: giWindow | int = None,
```

```
//  
*,  
parent: giWindow | int = None)  
Returns: None
```

Summary:

This command invokes options dialog for connections navigator window.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- window – The connections navigator window which options to show.
- parent – The connections navigator window which options to show.

Returns

None

showCreateBookmark

Syntax

```
showCreateBookmark(  
*,  
parent: giWindow | int = None)  
Returns: None
```

Summary:

Opens the “Create Bookmark” dialog.

Category:

Navigation

Licenses:

None

Parameters

parent – The window for which to create a bookmark

Returns

None

showCrossCheckOptions

Syntax

```
showCrossCheckOptions(  
    window: giWindow | int = None,  
    /,  
    *,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Brings up the cross view check options dialog.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

parent – Optional window to associate with dialog specifically to supply scope for the preference values

Returns

Dialog window with cross view check options

showDescendDialog

Syntax

```
showDescendDialog(  
    *,  
    context: deContext,  
    inst: oaInst,  
    position: point = None,  
    readOnly: bool = None,  
    newWindow: bool = None)  
Returns: None
```

Summary:

Shows the Descend Into dialog which prompts for an alternate switched master into which to descend.

Category:

Navigation

Licenses:

None

Parameters

- context – Context object for which to open the dialog.
- inst – Instance into which to descend.
- position – Specifies a coordinate over the instance which will be used to calculate the row/column of an array and populate the form.
- readOnly – Opens the design in read-only mode. (Default is: False)
- newWindow – Opens the design in a new window. (Default is: False)

Returns

None

showDesignsPrompt

Syntax

```
showDesignsPrompt(  
    name: str,  
    /,  
    *,  
    title: str,  
    prompt: str,  
    buttons: list[str],  
    default: str = None,  
    cancelButton: str = None,  
    files: list[dmDMFile] = None,  
    execProc: str,  
    wait: bool = None)  
Returns: giDialog
```

Summary:

This command shows the 'Close Designs' dialog.

Category:

Foundation

Licenses:

None

Parameters

- name – Name of the new dialog object.
- title – Dialog title.
- prompt – Dialog prompt.
- buttons – Button titles.
- default – Default button. Must match a string given in buttons.
- cancelButton – Default button. Must match a string given in buttons. (Default is: 'Cancel')
- files – Files to list. These must be open in a design editor.
- execProc – Callback procedure called when a button is pressed.
- wait – If true, the dialog will be shown modal and this command will not terminate until after executing a button callback. (Default is: False)

Returns

Newly created dialog

showGeneralOptions

Syntax

```
showGeneralOptions(  
*,  
    parent: giWindow | int = None)  
Returns: None
```

Summary:

Shows the General Options dialog. If the dialog is already opened, it will be raised and the existing object returned.

Category:

Foundation

Licenses:

None

Parameters

parent – Parent window.

Returns

None

showMakeDesignsReadOnly

Syntax

```
showMakeDesignsReadOnly(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows a dialog listing all open editable designs.

Category:

Foundation

Licenses:

None

Parameters

parent – Parent window on which this dialog will be centered. May be any window, not just design windows.

Returns

Newly created dialog.

showOpenDesign

Syntax

```
showOpenDesign(  
*,  
    window: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows the Open Design dialog. If the dialog is already opened, it will be raised and focused.

Category:

Foundation

Licenses:

None

Parameters

window – Window for which to show this dialog

Returns

Newly created dialog.

showParamFilter

Syntax

```
showParamFilter(  
*,  
    parent: giWindow | int)  
Returns: giDialog
```

Summary:

Show parameter filter dialog

Category:

Foundation

Licenses:

None

Parameters

parent – Parent Window(id).

Returns

Created parameter filter dialog

showProbeBitSelectionDialog

Syntax

```
showProbeBitSelectionDialog(  
    name: str,  
    /,  
    *,  
    type: de.showProbeBitSelectionDialog.Type,  
    window: giWindow | int)  
Returns: str
```

Summary:

Displays the bit selection dialog for probing objects with multiple bits.

Category:

Design Editing

Licenses:

None

Parameters

- name – Name of the bussed object.
- type – Type of probe ({ instTerm | instance | net | term })
- window – The parent window for the dialog.

Returns

User-selected probe name.

showProperties

Syntax

```
showProperties(  
    *,  
    context: deContext = None,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Show Property Editor

Category:

Foundation

Licenses:

None

Parameters

- context – Context containing selection to show
- parent – Parent design editor window.

Returns

Properties dialog object

showRefreshDesigns

Syntax

```
showRefreshDesigns(  
*,  
    parent: giWindow | int = None,  
    quiet: bool = None,  
    designs: list[dmDMFile] | list[oaDesign] = None)  
Returns: giDialog
```

Summary:

Shows a dialog listing all open, read-only and out-of-date designs which have changed on disk since being opened.

Category:

Foundation

Licenses:

None

Parameters

- parent – Parent editor window for the Refresh Designs dialog.
- quiet – If true, disables all information messages shown when designs are reverted or when no designs should be reverted. (Default is: False)
- designs – Specifies the set of designs or files to prompt for

Returns

Shown dialog or "" if the dialog is not shown due to all designs being refreshed.

showRenumberInstances

Syntax

```
showRenumberInstances(  
    *,  
    window: giWindow | int = None)  
Returns: giDialog
```

Summary:

Opens a dialog for renumbering instances of the active design.

Category:

Design Editing

Licenses:

Basic or Frontend

Parameters

window – Window or window id to which the dialog will be associated.

Returns

Renumber instances dialog

showReturnToLevelDialog

Syntax

```
showReturnToLevelDialog(  
    *,  
    context: deContext)  
Returns: giDialog
```

Summary:

Shows a Return to Level dialog.

Category:

Foundation

Licenses:

None

Parameters

context – Parent context of the dialog.

Returns

Newly created dialog.

showSaveDesignAs

Syntax

```
showSaveDesignAs (  
    *,  
    context: deContext)  
Returns: giDialog
```

Summary:

Shows the ‘Save Design As’ dialog. If the dialog is already opened, it will be raised and focused.

Category:

Foundation

Licenses:

None

Parameters

context – Context whose edit file will be saved.

Returns

Newly created dialog.

sleep

Syntax

```
sleep(  
    _: int)  
Returns: None
```

Summary:

Number of seconds to sleep.

Category:

Foundation

Licenses:

None

Parameters

_ – Number of seconds to sleep.

Returns

None

snapToObject

Syntax

```
snapToObject(  
    window: deCanvas | giWindow | int = None,  
    /,  
    *,  
    object: deFigure | oaFig,  
    point: point)  
Returns: point
```

Summary:

Returns a point snapped to the given object.

Category:

Design Editing

Licenses:

None

Parameters

- window – Window or canvas
- object – Object to snap to.
- point – Center of region to look for active objects in.

Returns

Point snapped to given object

startDrag

Syntax

```
startDrag(  
point: point,  
/,  
*,  
window: giWindow | int = None,  
context: deContext = None,  
intent: de.startDrag.Intent = None)  
Returns: None
```

Summary:

Starts dragging

Category:

Foundation

Licenses:

None

Parameters

- point – Point at which to start the drag event.
- window – Window in which to send the event.
- context – Context in which to send the event.
- intent – Specifies the intended response for this event. ({ default | deselect | select }
(Default is: 'default'))

Returns

None

startLog

Syntax

```
startLog(  
file: str,  
/,
```

```
*,  
banner: str | None = None)  
Returns: None
```

Summary:

Initializes a secondary log file for use with `de::sendMessage`.

Category:

Foundation

Licenses:

None

Parameters

- `file` – Full filesystem path to the secondary log file
- `banner` – Optional banner with which to initialize the secondary log file (Default is: “)

Returns

None

startRendererStatistics

Syntax

```
startRendererStatistics(  
*,  
file: str | None = None,  
method: bool | None = None,  
design: bool | None = None)  
Returns: None
```

Summary:

Start renderer statistics gathering.

Category:

Navigation

Licenses:

None

Parameters

- file – Output to the given file.
- method – If true, report the drawing method (Default is: True)
- design – If true, report the design info (Default is: True)

Returns

None

startTransaction

Syntax

```
startTransaction(  
name: str = None,  
/,  
*,  
design: oaDesign = None,  
config: heConfig = None)  
Returns: deTransaction
```

Summary:

Start Transaction

Category:

Foundation

Licenses:

None

Parameters

- name – Name of the transaction
- design – oaDesign option
- config – Config object

Returns

Transaction object

stop

Syntax

```
stop()  
Returns: None
```

Arguments:

Summary:

Stops execution of the log file commands.

Category:

Foundation

Licenses:

None

Returns

None

stopBackgroundImporter

Syntax

```
stopBackgroundImporter(  
objects: list[dmCell] | list[dmCellView] | list[oaLib])  
Returns: None
```

Summary:

Stops background importer.

Category:

Foundation

Licenses:

Backend

Parameters

objects – A collection of selected oaLib or dmCell or dmCellView.

Returns

None

stopRendererStatistics

Syntax

```
stopRendererStatistics()  
Returns: None
```

Arguments:

Summary:

Stop renderer statistics gathering.

Category:

Navigation

Licenses:

None

Returns

None

suppressMessage

Syntax

```
suppressMessage(  
    id: str,  
    /,  
    *,  
    suppress: bool | None = None)  
Returns: None
```

Summary:

Suppress message with given ID

Category:

Foundation

Licenses:

None

Parameters

- id – Suppressing message ID or wildcard.
- suppress – Message suppression flag (Default is: True)

Returns

None

transformPoint

Syntax

```
transformPoint(  
point: point,  
/,  
*,  
to: de.transformPoint.To,  
window: giWindow | int = None)  
Returns: point
```

Summary:

Converts coordinates between edit design and view design.

Category:

Foundation

Licenses:

None

Parameters

- point – Point to convert.
- to – The type of coordinate to convert to. ({ edit | view })
- window – Window identifier.

Returns

Converted point

undoSelection

Syntax

```
undoSelection(  
*,
```



```
design: oaDesign)
Returns: None
```

Summary:

Undo last selection operation.

Category:

Design Editing

Licenses:

None

Parameters

design – Design in which to undo last selection

Returns

None

undoTransaction

Syntax

```
undoTransaction(
*,
  steps: int = None,
  transaction: deTransaction = None,
  design: oaDesign = None,
  config: heConfig = None)
Returns: None
```

Summary:

Undo Transaction

Category:

Foundation

Licenses:

None

Parameters

- steps – Number of operations to be undone
- transaction – Transaction to move forward the transaction history

- design – Design object
- config – Config object

Returns

None

unhighlight

Syntax

```
unhighlight(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
         | list[oaVia],  
/,  
*,  
set: deHighlightSet = None)  
Returns: None
```

Summary:

Unhighlight (remove from the given set) given objects.

Category:

Design Editing

Licenses:

None

Parameters

- objects – Objects to unhighlight
- set – Highlight set from which to remove objects

Returns

None

uniquify

Syntax

```
uniquify(  
instance: oaInst,  
/,  
*,
```

```
to: dmCellView | oaLib)
Returns: oaInst
```

Summary:

Duplicates an instance master and modifies the instance to point to the new master

Category:

Design Editing

Licenses:

Basic or Frontend

Parameters

- instance – Instance to uniquify.
- to – Optionally specify a specific master to use or a library within which a unique master will be automatically created.

Returns

Newly created instance object

zoom

Syntax

```
zoom (
*,
window: deCanvas | giWindow | int = None,
factor: float,
center: point = None)
Returns: None
```

Summary:

Zooms in and out by given factor

Category:

Navigation

Licenses:

None

Parameters

- window – Window or canvas
- factor – Zoom factor
- center – Coordinate about which to zoom

Returns

None

snps.custom.cmd.ide

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform layout, schematic, and symbol editing, and design navigation. Interactive commands are listed in this chapter.

For details, see:

- [createProbe](#)
- [defineOrigin](#)
- [descend](#)
- [descendIntoGroup](#)
- [editCanvasText](#)
- [enterBox](#)
- [enterLine](#)
- [enterPoint](#)
- [enterPolygon](#)
- [highlight](#)
- [pan](#)
- [pick](#)
- [selectByName](#)
- [selectByRegion](#)

- [setViewport](#)
- [zoom](#)

createProbe

Syntax

```
createProbe(  
  context: deContext | int = None)  
Returns: None
```

Summary:

This command allows the user to interactively specify a new probe object with either canvas or text entry input.

Category:

Design Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

defineOrigin

Syntax

```
defineOrigin(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Starts the Define Origin command in the specified window.

Category:

Design Editing

Licenses:

Basic or Frontend

Parameters

context – Context of activation.

Returns

None

descend

Syntax

```
descend(  
context: deContext | int = None,  
/,  
*,  
newWindow: bool = None,  
promptView: bool = None,  
name: str = None,  
type: ide.descend.Type = None,  
readOnly: ide.descend.ReadOnly = None,  
inPlace: bool = None,  
promptLineage: bool = None,  
shadow: bool = None)  
Returns: None
```

Summary:

Starts the 'Descend' command in the specified window.

Category:

Navigation

Licenses:

None

Parameters

- context – Context of activation.
- newWindow – Defines the default value for the 'New Window' option. Specifying this value will modify the preference. (Default is: False)
- promptView – Defines the default value for "Prompt for View" option. Specifying this value will modify the preference value.

- **name** – Specifies the instance or figGroup name for immediate descent. (Default is: "")
- **type** – Specifies if target descend type is instance, figure group or both. ({ both | group | instance }) (Default is: 'instance')
- **readOnly** – Specifies the descend open mode. Specifying this value will modify the deDescend preference value. ({ 0 | 1 | auto | false | true }) (Default is: 'auto')
- **inPlace** – Defines the default value for the 'In Place' option. Specifying this value will modify the preference. (Default is: False)
- **promptLineage** – Defines the default value for 'Prompt for Lineage' option. Specifying this value will modify the preference value. (Default is: False)
- **shadow** – Specifies if selected lineage is shadowed. (Default is: False)

Returns

None

descendIntoGroup

Syntax

```
descendIntoGroup(  
    context: deContext | int = None  
Returns: None
```

Summary:

Starts the 'Descend Into Group' command in the specified window.

Category:

Navigation

Licenses:

None

Parameters

context – Context of activation.

Returns

None

editCanvasText

Syntax

```
editCanvasText(  
context: deContext | int = None,  
/,  
*,  
object: deFigure | oaFig)  
Returns: None
```

Summary:

Starts an on canvas editing session for the given text object.

Category:

Design Editing

Licenses:

Frontend

Parameters

- context – Context of activation.
- object – Text object to edit on canvas.

Returns

None

enterBox

Syntax

```
enterBox(  
context: deContext | int = None,  
/,  
*,  
prompt: str = None,  
draw: bool = None,  
returnViewCoordinates: bool = None,  
commandTitle: str = None)  
Returns: point
```

Summary:

Enter Box command.

Category:

Design Editing

Licenses:

None

Parameters

- context – Context of activation.
- prompt – Command prompt. (Default is: 'Enter Box')
- draw – Draw the box in the canvas (Default is: True)
- returnViewCoordinates – Returns points relative to edit design (false) or view design (true) (Default is: False)
- commandTitle – Command title. (Default is: 'Enter Box')

Returns

None

enterLine**Syntax**

```
enterLine(  
context: deContext | int = None,  
/,  
*,  
prompt: str = None,  
draw: bool = None,  
angle: str = None,  
returnViewCoordinates: bool = None,  
commandTitle: str = None,  
numPoints: int = None)  
Returns: point
```

Summary:

Enter Line command.

Category:

Design Editing

Licenses:

None

Parameters

- context – Context of activation.
- prompt – Command prompt. (Default is: 'Enter Line')
- draw – Draw the line after first point entered (Default is: True)
- angle – Angle mode used to enter points (Default is: 'orthogonal')
- returnViewCoordinates – Returns points relative to edit design (false) or view design (true) (Default is: False)
- commandTitle – Command title. (Default is: 'Enter Line')
- numPoints – Completes execution after reaching the specified number of points

Returns

None

enterPoint

Syntax

```
enterPoint(  
context: deContext | int = None,  
/,  
*,  
prompt: str = None,  
returnViewCoordinates: bool = None,  
commandTitle: str = None)  
Returns: point
```

Summary:

Enter Point command.

Category:

Design Editing

Licenses:

None

Parameters

- context – Context of activation.
- prompt – Command prompt. (Default is: 'Enter Point')

- `returnViewCoordinates` – Return point relative to edit design (false) or view design (true) (Default is: False)
- `commandTitle` – Command title. (Default is: 'Enter Point')

Returns

None

enterPolygon

Syntax

```
enterPolygon(  
context: deContext | int = None,  
/,  
*,  
prompt: str = None,  
draw: bool = None,  
angle: ide.enterPolygon.Angle = None,  
returnViewCoordinates: bool = None,  
commandTitle: str = None)  
Returns: point
```

Summary:

Enter Polygon command.

Category:

Design Editing

Licenses:

None

Parameters

- `context` – Context of activation.
- `prompt` – Command prompt. (Default is: 'Enter Polygon')
- `draw` – Draw the polygon after first point entered (Default is: True)
- `angle` – Angle mode used to enter points ({ any | diagonal | orthogonal }) (Default is: 'orthogonal')
- `returnViewCoordinates` – Returns points relative to edit design (false) or view design (true) (Default is: False)
- `commandTitle` – Command title. (Default is: 'Enter Polygon')

Returns

None

highlight

Syntax

```
highlight(  
context: deContext | int = None,  
/,  
*,  
mode: ide.highlight.Mode = None)  
Returns: None
```

Summary:

Interactive highlight command.

Category:

Design Editing

Licenses:

None

Parameters

- context – Context of activation.
- mode – Highlight mode ({ name | shapes })

Returns

None

pan

Syntax

```
pan(  
context: deContext | int = None,  
/,  
*,  
point: point = None)  
Returns: None
```

Summary:

Interactive version of de::pan command.

Category:

Navigation

Licenses:

None

Parameters

- context – Context of activation.
- point – Specifies the drag start point and enters “one shot mode” where the COT is not updated.

Returns

None

pick

Syntax

```
pick(  
context: deContext | int = None,  
/,  
*,  
prompt: str = None,  
oneShot: bool = None,  
types: list[str] = None,  
pickProc: list[attrObject],  
unpickProc: list[attrObject] = None,  
cancelProc: list[attrObject] = None,  
filterProc: list[attrObject] = None,  
abortProc: list[attrObject] = None)  
Returns: None
```

Summary:

Perform an interactive pick for design objects

Category:

Design Editing

Licenses:

None

Parameters

- context – Context of activation.
- prompt – Prompt text for design editor status bar
- oneShot – Terminate command after single pick (Default is: False)
- types – List of selectable logical objects: net,instance, terminal and instTerm
- pickProc – Procedure triggered when an object is picked
- unpickProc – Optional procedure triggered when an object is Ctrl-picked
- cancelProc – Optional procedure triggered when the command is cancelled
- filterProc – Optional pre selection filter procedure
- abortProc – Optional procedure triggered when the command is aborted

Returns

None

selectByName

Syntax

```
selectByName(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive select object by name command.

Category:

Design Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

selectByRegion

Syntax

```
selectByRegion(  
context: deContext | int = None,  
/,  
*,  
region: ide.selectByRegion.Region = None,  
select: bool = None,  
criteria: ide.selectByRegion.Criteria = None,  
point: point = None)  
Returns: None
```

Summary:

Interactive select objects by region command.

Category:

Design Editing

Licenses:

None

Parameters

- context – Context of activation.
- region – Specifies a region type to use ({ line | point | polygon | rectangle })
- select – If true, select if false, deselect
- criteria – Specifies the select criteria ({ enclose | touch })
- point – Specifies the first input coordinate

Returns

None

setViewport

Syntax

```
setViewport(  
context: deContext | int = None,  
/,  
*,  
point: point = None,  
direction: ide.setViewport.Direction = None,
```

```
recursive: bool = None,  
log: ide.setViewport.Log = None)  
Returns: None
```

Summary:

Interactive version of de::setViewport command.

Category:

Navigation

Licenses:

None

Parameters

- context – Context of activation.
- point – Specifies the drag start point and enters “one shot mode” where the COT is not updated.
- direction – Direction to zoom ({ bidir | in | out } (Default is: 'in'))
- recursive – Optionally override the deSetViewRecursive option. Specifying this option will modify the preference value.
- log – Logging mode ({ auto | explicit } (Default is: 'auto'))

Returns

None

zoom

Syntax

```
zoom(  
context: deContext | int = None,  
/,  
*,  
point: point = None,  
direction: ide.zoom.Direction = None,  
log: ide.zoom.Log = None)  
Returns: None
```

Summary:

Interactive version of de::zoom command.

Category:

Navigation

Licenses:

None

Parameters

- context – Context of activation.
- point – Specifies the drag start point and enters “one shot mode” where the COT is not updated.
- direction – Direction to zoom ({ bidir | in | out } (Default is: ‘in’)
- log – Logging mode ({ auto | explicit } (Default is: ‘auto’)

Returns

None

snps.custom.cmd.di

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform operations in the Design Insights tool.

This section covers the following topics:

- [closeResultsCellView](#)
- [createResultsCellView](#)
- [getCapacitors](#)
- [getCapAttr](#)
- [getCorners](#)
- [getElmoreDelayResult](#)
- [getElmoreDelayResultAttr](#)
- [getFanoutResult](#)
- [getFanoutResultAttr](#)

- [getInstAttr](#)
- [getInsts](#)
- [getLayers](#)
- [getModels](#)
- [getNetAttr](#)
- [getNets](#)
- [getNodeAttr](#)
- [getResAttr](#)
- [getResistanceContributionAttr](#)
- [getResistanceResult](#)
- [getResistanceResultAttr](#)
- [getResistors](#)
- [getTermAttr](#)
- [getTerms](#)
- [getViewTypes](#)
- [listAttrs](#)
- [loadResultsCellView](#)
- [runElmoreDelayAnalysis](#)
- [runFanoutAnalysis](#)
- [runResistanceAnalysis](#)

closeResultsCellView

Syntax

```
closeResultsCellView()  
Returns: None
```

Arguments:

Summary:

Close the currently open cell view.

Category:

Physical Design Insights

Licenses:

None

Returns

None

createResultsCellView

Syntax

```
createResultsCellView(  
*,  
    lcv: str,  
    modelLibs: str = None,  
    modelSyntax: str = None,  
    overwrite: bool = None,  
    paths: str,  
    selectNet: str = None,  
    skipNet: str = None,  
    type: di.createResultsCellView.Type,  
    useITFLayerName: bool = None,  
    vbias: float = None,  
    vbiasMappingFile: str = None)  
Returns: xtJob
```

Summary:

Create cell view by rc_preprocessor with '-path'.

Category:

Physical Design Insights

Licenses:

None

Parameters

- lcv – Target lcv name to create
- modelLibs – Specified the list of the lists to specify the corner, model libs and model sections.
- modelSyntax – Specifies the model syntax type. 'hspice' 'eldo' and 'spectre' supported.

- `overwrite` – Specifies if overwrite cell view or not
- `paths` – Specifies the file paths `rc_preprocessor` read
- `selectNet` – Specifies the select net pattern
- `skipNet` – Specifies the skip net pattern
- `type` – Specifies the format the `rc_preprocessor` read. `gpd` and `dspf` formats supported. (`{ dspf | gpd }`)
- `useITFLayerName` – Specify if the layer use ITF layer or not
- `vbias` – Specifies the vbias number
- `vbiasMappingFile` – Specified the list of the lists to specify the corner, model libs and model sections.

Returns

Return the submitted job object

getCapacitors

Syntax

```
getCapacitors(  
*,  
    netID: int | None = None,  
    netName: str | None = None)  
Returns: str
```

Summary:

Report the capacitors' ids from the specified net id/name.

Category:

Physical Design Insights

Licenses:

None

Parameters

- `netID` – Specifies the net id of the capacitors.
- `netName` – Specifies the net name of the capacitors.

Returns

Return list of the capacitors' ids.

getCapAttr

Syntax

```
getCapAttr(  
*,  
    attribute: str | None = None,  
    corners: str | None = None,  
    id: int)  
Returns: str
```

Summary:

Get capacitor data by specified capacitor id and attribute name.

Category:

Physical Design Insights

Licenses:

None

Parameters

- attribute – Specifies the target attribute name.
- corners – Specify the list of the corner name to determine the order of the multi-corner attribute value.
- id – Specifies the target id.

Returns

Return list of the attribute data.

getCorners

Syntax

```
getCorners()  
Returns: str
```

Arguments:

Summary:

Get corners information

Category:

Physical Design Insights

Licenses:

None

Returns

corners information

getElmoreDelayResult

Syntax

```
getElmoreDelayResult(  
*,  
    net: str = None,  
    type: di.getElmoreDelayResult.Type)  
Returns: str
```

Summary:

Get the Elmore Delay Result for current loaded cell view.

Category:

Physical Design Insights

Licenses:

None

Parameters

- net – Specifies the target net name.
- type – Specifies the type of the report. ({ all | instpin_to_port | others | point_to_point | pull_down | pull_up })

Returns

Return the ids of the summary.

getElmoreDelayResultAttr

Syntax

```
getElmoreDelayResultAttr(  
*,  
    attribute: str | None = None,  
    corners: str | None = None,  
    id: int)  
Returns: str
```

Summary:

Get the Elmore Delay Result for current loaded cell view.

Category:

Physical Design Insights

Licenses:

None

Parameters

- `attribute` – Specifies the target attribute name.
- `corners` – Specify the list of the corner name to determine the order of the multi-corner attribute value.
- `id` – Specifies the target id.

Returns

Return list of the attribute data.

getFanoutResult

Syntax

```
getFanoutResult(  
*,  
    net: str = None,  
    type: di.getFanoutResult.Type)  
Returns: str
```

Summary:

Get the Fanout Result for current loaded cell view.

Category:

Physical Design Insights

Licenses:

None

Parameters

- `net` – Specifies the target net name.
- `type` – Specifies the type of the report. ({ `all` | `instpin_to_port` | `others` | `point_to_point` | `pull_down` | `pull_up` })

Returns

Return the ids of the summary.

getFanoutResultAttr

Syntax

```
getFanoutResultAttr(  
*,  
    attribute: str | None = None,  
    corners: str | None = None,  
    id: int)  
Returns: str
```

Summary:

Get the Fanout Result Attribute data from current fanout result.

Category:

Physical Design Insights

Licenses:

None

Parameters

- attribute – Specifies the target attribute name.
- corners – Specify the list of the corner name to determine the order of the multi-corner attribute value.
- id – Specifies the target id.

Returns

Return list of the attribute data.

getInstAttr

Syntax

```
getInstAttr(  
*,  
    attribute: str | None = None,  
    corners: str | None = None,  
    id: int)  
Returns: str
```


Summary:

Get instance data by specified instance id and attribute name.

Category:

Physical Design Insights

Licenses:

None

Parameters

- `attribute` – Specifies the target attribute name.
- `corners` – Specify the list of the corner name to determine the order of the multi-corner attribute value.
- `id` – Specifies the target id.

Returns

Return list of the attribute data.

getInsts

Syntax

```
getInsts(  
*,  
    name: str | None = None,  
    netId: int | None = None,  
    netName: str | None = None,  
    termID: int | None = None,  
    termName: str | None = None)  
Returns: str
```

Summary:

Report the list of the instances' ids from the netlist.

Category:

Physical Design Insights

Licenses:

None

Parameters

- name – Specifies the name of the instance.
- netId – Specifies net name of the instance.
- netName – Specifies net name of the instance.
- termID – Specifies term id of the net.
- termName – Specifies term name of the instance.

Returns

Return list of the instances' ids.

getLayers

Syntax

```
getLayers(  
*,  
    layerName: str | None = None)  
Returns: str
```

Summary:

Get layers information

Category:

Physical Design Insights

Licenses:

None

Parameters

layerName – specified layer name

Returns

layers information

getModels

Syntax

```
getModels(  
*,
```

```
modelName: str | None = None)  
Returns: str
```

Summary:

Get models information

Category:

Physical Design Insights

Licenses:

None

Parameters

modelName – specified model name

Returns

models information

getNetAttr

Syntax

```
getNetAttr(  
*,  
attribute: str | None = None,  
corners: str | None = None,  
id: int)  
Returns: str
```

Summary:

Get net data by specified net id and attribute name.

Category:

Physical Design Insights

Licenses:

None

Parameters

- attribute – Specifies the target attribute name.
- corners – Specify the list of the corner name to determine the order of the multi-corner attribute value.
- id – Specifies the target id.

Returns

Return list of the attribute data.

getNets

Syntax

```
getNets(  
*,  
    name: str | None = None,  
    termID: int | None = None,  
    termName: str | None = None)  
Returns: str
```

Summary:

Report the list of the nets' ids from the netlist.

Category:

Physical Design Insights

Licenses:

None

Parameters

- name – Specifies the name of the net
- termID – Specifies term id of the net.
- termName – Specifies term name of the net.

Returns

Return list of the nets' ids.

getNodeAttr

Syntax

```
getNodeAttr(  
*,  
    attribute: str | None = None,  
    corners: str | None = None,  
    id: int)  
Returns: str
```

Summary:

Get node data by specified node id and attribute name.

Category:

Physical Design Insights

Licenses:

None

Parameters

- attribute – Specifies the target attribute name.
- corners – Specify the list of the corner name to determine the order of the multi-corner attribute value.
- id – Specifies the target id.

Returns

Return list of the attribute data.

getResAttr

Syntax

```
getResAttr(  
*,  
    attribute: str | None = None,  
    corners: str | None = None,  
    id: int)  
Returns: str
```

Summary:

Get resistor data by specified resistor id and attribute name.

Category:

Physical Design Insights

Licenses:

None

Parameters

- `attribute` – Specifies the target attribute name.
- `corners` – Specify the list of the corner name to determine the order of the multi-corner attribute value.
- `id` – Specifies the target id.

Returns

Return list of the attribute data.

getResistanceContributionAttr

Syntax

```
getResistanceContributionAttr(  
*,  
    attribute: str | None = None,  
    corners: str | None = None,  
    id: int)  
Returns: str
```

Summary:

Get the Effective Resistance Contribution Attribute data from opened effective resistance result.

Category:

Physical Design Insights

Licenses:

None

Parameters

- attribute – Specifies the target attribute name.
- corners – Specify the list of the corner name to determine the order of the multi-corner attribute value.
- id – Specifies the target id.

Returns

Return list of the attribute data.

getResistanceResult

Syntax

```
getResistanceResult(  
*,  
    net: str = None,  
    type: di.getResistanceResult.Type)  
Returns: str
```

Summary:

Get the Effective Resistance Result for current loaded cell view.

Category:

Physical Design Insights

Licenses:

None

Parameters

- net – Specifies the target net name.
- type – Specifies the type of the report. ({ all | instpin_to_port | others | point_to_point | pull_down | pull_up })

Returns

Return the ids of the summary.

getResistanceResultAttr

Syntax

```
getResistanceResultAttr(  
*,  
    attribute: str | None = None,  
    corners: str | None = None,  
    id: int)  
Returns: str
```

Summary:

Get the Effective Resistance Result for current loaded cell view.

Category:

Physical Design Insights

Licenses:

None

Parameters

- attribute – Specifies the target attribute name.
- corners – Specify the list of the corner name to determine the order of the multi-corner attribute value.
- id – Specifies the target id.

Returns

Return list of the attribute data.

getResistors

Syntax

```
getResistors(  
*,  
    netID: int | None = None,  
    netName: str | None = None)  
Returns: str
```

Summary:

Report the resistors' ids from the specified net id/name.

Category:

Physical Design Insights

Licenses:

None

Parameters

- netID – Specifies the net id of the resistors.
- netName – Specifies the net name of the resistors.

Returns

Return list of the resistors' ids.

getTermAttr**Syntax**

```
getTermAttr(  
*,  
    attribute: str | None = None,  
    corners: str | None = None,  
    id: int)  
Returns: str
```

Summary:

Get term data by specified term id and attribute name.

Category:

Physical Design Insights

Licenses:

None

Parameters

- attribute – Specifies the target attribute name.
- corners – Specify the list of the corner name to determine the order of the multi-corner attribute value.
- id – Specifies the target id.

Returns

Return list of the attribute data.

getTerms

Syntax

```
getTerms (  
*,  
    instID: int | None = None,  
    instName: str | None = None,  
    name: str | None = None,  
    netID: int | None = None,  
    netName: str | None = None)  
Returns: str
```

Summary:

Report the list of the terms' ids from the netlist.

Category:

Physical Design Insights

Licenses:

None

Parameters

- instID – Specifies instance id of the terms.
- instName – Specifies instance name of the terms.
- name – Specifies the name of the terms.
- netID – Specifies net id of the terms.
- netName – Specifies net name of the terms.

Returns

Return list of the terms' ids.

getViewTypes

Syntax

```
getViewTypes (  
*,  
    type: di.getViewTypes.Type)  
Returns: str
```

Summary:

Get view type names.

Category:

Design Insights

Licenses:

None

Parameters

type – Type ({ logical | physical })

Returns

View type

listAttrs**Syntax**

```
listAttrs(  
*,  
    type: di.listAttrs.Type)  
Returns: str
```

Summary:

List all attributes for specified type of the di::get*Attr commands

Category:

Physical Design Insights

Licenses:

None

Parameters

type – List all attributes for specified type of the di::get*Attr commands. ({ cap | elmoreDelayResult | fanoutResult | inst | net | node | res | resistanceContributionResult | resistanceResult | term })

Returns

The list of the attribute name

loadResultsCellView

Syntax

```
loadResultsCellView(  
*,  
    lcv: dmCellView | dmCellViewName)  
Returns: None
```

Summary:

Open a cell view.

Category:

Physical Design Insights

Licenses:

None

Parameters

lcv – Specify a cell view to be opened

Returns

None

runElmoreDelayAnalysis

Syntax

```
runElmoreDelayAnalysis(  
*,  
    corners: str = None,  
    drivers: str = None,  
    includeDeviceTermCapacitance: bool = None,  
    loads: str = None,  
    mode: di.runElmoreDelayAnalysis.Mode,  
    nets: str = None)  
Returns: xtJob
```

Summary:

Get the Elmore delay result for currently loaded cell view.

Category:

Physical Design Insights

Licenses:

None

Parameters

- corners – Specify the list of the corner name.
- drivers – Specifies the list of the driver's term id.
- includeDeviceTermCapacitance – Specify if the analysis have to include term cap or not.
- loads – Specifies the list of the load's term id.
- mode – Specifies mode of the command. ({ automatic | p2p })
- nets – Specifies list of the net id.

Returns

Return the submitted job object

runFanoutAnalysis

Syntax

```
runFanoutAnalysis(  
*,  
    corners: str = None,  
    drivers: str = None,  
    loads: str = None,  
    mode: di.runFanoutAnalysis.Mode,  
    nets: str = None)  
Returns: xtJob
```

Summary:

Get the fanout result for currently loaded cell view.

Category:

Physical Design Insights

Licenses:

None

Parameters

- corners – Specify the list of the corner name.
- drivers – Specifies the list of the driver's term id.

- loads – Specifies the list of the load's term id.
- mode – Specifies mode of the command. ({ automatic | p2p })
- nets – Specifies list of the net id.

Returns

Return the submitted job object

runResistanceAnalysis

Syntax

```
runResistanceAnalysis(  
*,  
    corners: str = None,  
    drivers: str = None,  
    loads: str = None,  
    mode: di.runResistanceAnalysis.Mode,  
    nets: str = None)  
Returns: xtJob
```

Summary:

Get the effective resistance result for current loaded cell view.

Category:

Physical Design Insights

Licenses:

None

Parameters

- corners – Specify the list of the corner name.
- drivers – Specifies the list of the driver's term id.
- loads – Specifies the list of the load's term id.
- mode – Specifies mode of the command. ({ automatic | instP2p | p2p })
- nets – Specifies list of the net id.

Returns

Return the submitted job object

snps.custom.cmd.dm

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform database operations.

This section covers the following topics:

- [addICCRefLibs](#)
- [addToCellCategory](#)
- [addToLibDefs](#)
- [associateVersionControl](#)
- [checkIn](#)
- [checkLock](#)
- [checkOut](#)
- [checkStatus](#)
- [compress](#)
- [convertDMSystem](#)
- [copyCellCategory](#)
- [copyCells](#)
- [copyCellsUnique](#)
- [copyCellViews](#)
- [copyLib](#)
- [copyProps](#)
- [createCell](#)
- [createCellCategory](#)
- [createCellInput](#)
- [createCellView](#)
- [createCellViewInput](#)

- [createLib](#)
- [createLibInput](#)
- [createMilkywayLib](#)
- [createMultiCellViewInput](#)
- [deleteFromCellCategory](#)
- [findCell](#)
- [findCellView](#)
- [findDMData](#)
- [findDMFile](#)
- [get3DICLayerMap](#)
- [get3DICRefLibs](#)
- [getCategoriesFromCategory](#)
- [getCategoriesOfMember](#)
- [getCellCategories](#)
- [getCells](#)
- [getCellsFromCategory](#)
- [getCellViews](#)
- [getDMFileFollowers](#)
- [getDMFileLeader](#)
- [getDMFiles](#)
- [getFCLayerMap](#)
- [getFCRefLibs](#)
- [getFollowers](#)
- [getICC2LayerMap](#)
- [getICC2RefLibs](#)
- [getLeaders](#)
- [getLibDefPath](#)

- [getLibs](#)
- [getMilkywayRefLibs](#)
- [getOpenDesigns](#)
- [getSelected](#)
- [getStandardLibDefPath](#)
- [getTopLibDefPath](#)
- [getVersionControl](#)
- [getVersionControlSystems](#)
- [isDMFileLeader](#)
- [isHidden](#)
- [moveCells](#)
- [moveCellViews](#)
- [moveLib](#)
- [openLibraryManager](#)
- [openLibs](#)
- [refreshLibraryManager](#)
- [refreshLibs](#)
- [registerCopyProps](#)
- [registerVersionControl](#)
- [removeVersionControl](#)
- [renameCellCategory](#)
- [revert](#)
- [saveAsFCRefLib](#)
- [saveAsICC2RefLib](#)
- [set3DICLCVMap](#)
- [set3DICRefLibs](#)
- [setFCLCVMap](#)

- [setFCRefLibs](#)
- [setFollowers](#)
- [setICC2LCVMap](#)
- [setICC2RefLibs](#)
- [setICCTechSync](#)
- [setSelected](#)
- [showCopyCell](#)
- [showCopyCellView](#)
- [showCopyLibrary](#)
- [showCopyUnique](#)
- [showDeleteViewsByName](#)
- [showEditLibraryCompression](#)
- [showLibraryManager](#)
- [showMoveCell](#)
- [showMoveCellView](#)
- [showMoveLibrary](#)
- [showNewCell](#)
- [showNewCellView](#)
- [showNewLibrary](#)
- [showRenameCellCategory](#)
- [syncToVersion](#)
- [uncompress](#)
- [updateLib](#)

addICCRefLibs

Syntax

```
addICCRefLibs(  
  libs: oaCollection_oaLib_oaLib | list[oaLib])  
Returns: None
```

Summary:

Add reference libraries of the specified ICC Library

Category:

Data Management

Licenses:

Backend

Parameters

libs – ICC libraries

Returns

None

addToCellCategory

Syntax

```
addToCellCategory(  
    cellCat: dmCellCategory,  
    /,  
    *,  
    cell: list[dmCell] = None,  
    cellCategory: list[dmCellCategory] = None)  
Returns: dmCellCategory
```

Summary:

This method adds members, dmCells, or dmCellCategories to a cell category

Category:

Data Management

Licenses:

None

Parameters

- cellCat – The cell category to augment
- cell – A collection of Cells to add as members
- cellCategory – A collection of dmCellCategories to add as members

Returns

Cell category that was updated

addToLibDefs

Syntax

```
addToLibDefs(  
  name: str,  
  /,  
  *,  
  path: str,  
  assign: str = None,  
  addICCRRefLibs: bool = None)  
Returns: oaLib
```

Summary:

Adds a library reference to one's lib.defs file much like dm::createLib does with the new library and then opens the library

Category:

Data Management

Licenses:

Backend

Parameters

- name – Library name
- path – Relative or absolute path to the existing library
- assign – Space delimited name value pairs to be ASSIGNED to this library in the lib.defs file
- addICCRRefLibs – If the specified library is determined to be an ICC library, definitions for each of its reference libraries will also be added. (Default is: True)

Returns

Library that is opened

associateVersionControl

Syntax

```
associateVersionControl(  
    name: str,  
    /,  
    *,  
    lib: oaLib = None,  
    libName: str = None)  
Returns: None
```

Summary:

Specifies the name of the VC system that manages a given open library. It is an error if the VC system name is not already registered with DM or if this library is already registered with a different VC system

Category:

Data Management

Licenses:

None

Parameters

- name – The name of version control system
- lib – An open library to be managed by this version control system
- libName – Name of the library to use

Returns

None

checkIn

Syntax

```
checkIn(  
    container: list[dmCellCategory] | list[dmCellView] | list[dmDMFile])  
Returns: None
```

Summary:

Triggers checkInProc if container is managed. Error is thrown in check in is not successful. If container is not managed, then returns an error

Category:

Data Management

Licenses:

None

Parameters

container – DM collection to check in

Returns

None

checkLock

Syntax

```
checkLock(  
  objects: dmCellName | dmCellViewName | dmLibName | list[dmCell] |  
           list[dmCellView] | list[dmDMFile] | list[oaLib],  
  /,  
  *,  
  recursive: bool = None,  
  release: bool = None)  
Returns: enDMLockStatus
```

Summary:

Checks whether the given files or containers are locked, or tries to release abandoned locks.

Category:

Data Management

Licenses:

None

Parameters

- objects – The container(s) or file(s) to check
- recursive – Checks recursively, if true (Default is: False)
- release – Release abandoned locks, if true (Default is: False)

Returns

The lock status

checkOut

Syntax

```
checkOut (  
  container: list[dmCellCategory] | list[dmCellView] | list[dmDMFile])  
Returns: None
```

Summary:

Triggers checkOutProc if container is managed. Error is thrown in check out is not successful. If container is not managed, then returns an error

Category:

Data Management

Licenses:

None

Parameters

container – DM collection to check out

Returns

None

checkStatus

Syntax

```
checkStatus (  
  container: dmCellCategory | dmCellView | dmDMFile)  
Returns: enVersionControlStatus
```

Summary:

Checks status for the given object, by triggering the registered checkStatusProc. Top category file cannot be queried with this command.

Category:

Data Management

Licenses:

None

Parameters

container – CellView, DMFile, or CellCategory to query

Returns

Returns the version control status (checkedOut, checkedIn) for the specified object, or unmanaged if the DM object is not managed with a version control system.

compress

Syntax

```
compress(  
library: dmLibName | oaLib,  
/,  
*,  
compressionLevel: dm.compress.CompressionLevel = None,  
check: bool = None)  
Returns: None
```

Summary:

Compresses given library using oazip -compress

Category:

Data Management

Licenses:

None

Parameters

- library – The library or library name, to compress
- compressionLevel – The compression level. Supported values are 1-9 ({ 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 })
- check – Checks compression state of the library

Returns

None

convertDMSystem

Syntax

```
convertDMSystem(  
lib: oaLib | str,  
/,  
*,  
dmSystem: str)  
Returns: oaLib
```

Summary:

Converts a library from its current DM system to the specified DM system

Category:

Data Management

Licenses:

None

Parameters

- lib – The library to convert
- dmSystem – The name of the new DM system

Returns

Newly converted library

copyCellCategory

Syntax

```
copyCellCategory(  
sourceCat: dmCellCategory,  
/,  
*,  
name: str,  
parent: dmCellCategory = None)  
Returns: dmCellCategory
```

Summary:

Copies the supplied cell category to a new cell category in the same library with the supplied name

Category:

Data Management

Licenses:

None

Parameters

- sourceCat – The cell category to copy
- name – The name of the new cell category
- parent – The category that this will be a subcategory of

Returns

New cell category

copyCells

Syntax

```
copyCells(  
sourceCells: list[dmCell],  
/  
*,  
libDestName: str,  
cellDestName: str = None,  
update: bool = None,  
skipPcells: bool = None,  
updateSearch: dm.copyCells.UpdateSearch = None,  
hier: bool = None,  
includeViews: list[str] = None,  
excludeViews: list[str] = None,  
includeViewTypes: list[str] = None,  
excludeViewTypes: list[str] = None,  
skipBackupFiles: bool = None,  
excludeLibs: list[str] = None,  
renamePattern: str = None,  
regexp: bool = None,  
category: list[str] = None,  
maintainCategories: bool = None,  
destinationConflicts: dm.copyCells.DestinationConflicts = None,  
sourceConflicts: dm.copyCells.SourceConflicts = None,  
cellPattern: str = None)  
Returns: list[dmCell]
```

Summary:

Copies the specified cells to the specified location

Category:

Data Management

Licenses:

None

Parameters

- sourceCells – The Cell to copy
- libDestName – Destination library name
- cellDestName – Destination cell name required for copying single cellView (Default is: “”)
- update – Issue the pre/post copy callbacks for each individual DMFile in the copied cells. This allows references in those files to be updated (Default is: True)
- skipPcells – Allows to skip PCell references to be copied and updated. (Default is: False)
- updateSearch – Specify where to search for references to change ({ changed | destinationLib | none | writableLibs })
- hier – For cellViews being copied that have oaDesigns, descend the hierarchy under that cellView copying the masters of each instance even when they belong to cells that aren't in the source list. (Default is: False)
- includeViews – This specifies the set of cellViews inside each copied Cell that should be copied
- excludeViews – This specifies the set of cellViews inside each copied Cell that should not be copied. This option cannot be used when -includeViews is used
- includeViewTypes – Specifies the set of ViewType names of Views inside each copied cell that should also be copied. List members are ViewTypes that support regular expression or wildcard matching. This option cannot be used when -excludeViewTypes is used
- excludeViewTypes – This specifies the set of ViewType names of Views inside each copied cell that should not be copied. List members are ViewTypes that support regular expression or wildcard matching. This option cannot be used when -includeViewTypes is used
- skipBackupFiles – If true, the backup files in views are skipped during copy operation. (Default is: False)
- excludeLibs – Specifies libraries whose cellViews should not be copied

- **renamePattern** – Pattern for renaming a cell where conflicts exist. Can contain %C %L and %N (Default is: '%C_%L')
- **regex** – if true the pattern is interpreted as a regular expression, otherwise the pattern uses wildcards (Default is: True)
- **category** – Adds all cells copied to to the named category. Creates the category if necessary
- **maintainCategories** – Adds copied cell into allcategories that the source cell is a member of (Default is: True)
- **destinationConflicts** – Action to take when a copied cell already exists in the destination ({ overwrite | rename | skip } (Default is: 'rename')
- **sourceConflicts** – Action to take when copied cells from multiple source libraries have the same name ({ overwrite | rename | select | skip } (Default is: 'rename')
- **cellPattern** – Force rename cell names before -renamePattern apply. Can contain %C %L and %N (Default is: '')

Returns

Collection of new cells

copyCellsUnique

Syntax

```
copyCellsUnique(  
*,  
    topCellView: dmCellView = None,  
    topCell: dmCell = None,  
    leafCellView: dmCellView = None,  
    leafCell: dmCell = None,  
    libDestName: str,  
    renamePattern: str = None,  
    destinationConflicts: dm.copyCellsUnique.DestinationConflicts = None,  
    skipBackupFiles: bool = None)  
Returns: list[dmCell]
```

Summary:

Copies the hierarchy of specified cellViews to the specified location

Category:

Data Management

Licenses:

None

Parameters

- topCellView – The top cellView of the hierarchy to copy
- topCell – The top cell of the hierarchy to copy
- leafCellView – The leaf cellView of hierarchy to copy
- leafCell – The leaf cell of hierarchy to copy
- libDestName – Destination library name
- renamePattern – Pattern for renaming all cells. Can contain %C %L and %N (Default is: '%C_%L')
- destinationConflicts – Action to take when a copied cell already exists in the destination ({ overwrite | skip } (Default is: 'skip'))
- skipBackupFiles – If true, the backup files in views are skipped during copy operation (Default is: False)

Returns

Collection of new cells

copyCellViews

Syntax

```
copyCellViews(  
    sourceCellViews: list[dmCellView],  
    /,  
    *,  
    libDestName: str,  
    cellDestName: str,  
    viewDestName: str = None,  
    update: bool = None,  
    skipPcells: bool = None,  
    hier: bool = None,  
    destinationConflicts: dm.copyCellViews.DestinationConflicts = None,  
    sourceConflicts: dm.copyCellViews.SourceConflicts = None,  
    includeViews: list[str] = None,  
    excludeViews: list[str] = None,  
    excludeLibs: list[str] = None,  
    renamePattern: str = None,  
    includeViewTypes: list[str] = None,  
    excludeViewTypes: list[str] = None,  
    skipBackupFiles: bool = None,
```

```
    regexp: bool = None,  
    cellPattern: str = None)  
Returns: list[dmCellView]
```

Summary:

Copy the specified cellView(s) to specified location

Category:

Data Management

Licenses:

None

Parameters

- sourceCellViews – CellView to copy
- libDestName – Destination library name
- cellDestName – Destination cell name
- viewDestName – Destination view name, required for copying single cellViews (Default is: “)
- update – Issue the pre/post copy callbacks for each individual DMFile so their internal references can be updated (Default is: True)
- skipPcells – Allows to skip PCell references to be copied and updated. (Default is: False)
- hier – Copies instance master Cells (Default is: False)
- destinationConflicts – Action to take when a copied cell already exists in the destination ({ overwrite | rename | skip } (Default is: ‘rename’)
- sourceConflicts – Action to take when copied cells from multiple source libraries have the same name ({ overwrite | rename | skip } (Default is: ‘rename’)
- includeViews – When -hier is used specifies the set of cellViews associated with the instance master Cells should be copied
- excludeViews – When -hier is used specifies the set of cellViews associated with the instance master cellView that should not be copied
- excludeLibs – Specifies libraries whose cellViews should not be copied
- renamePattern – Pattern for renaming a cell where conflicts exist. Can contain %C %L and %N (Default is: ‘%C_%L’)

- `includeViewTypes` – When `-hier` is used specifies the set of cellViews associated with the instance master cells that should also be copied. List members are ViewType names that support regular expression or wildcard matching. This option cannot be used when `-excludeViewTypes` is used
- `excludeViewTypes` – When `-hier` is used specifies the set of cellViews associated with the instance master cells that should not be copied. List members are ViewType names that support regular expression or wildcard matching. This option cannot be used when `-includeViewTypes` is used
- `skipBackupFiles` – If true, the backup files in views are skipped during copy operation. (Default is: False)
- `regex` – if true the pattern is interpreted as a regular expression, otherwise the pattern uses wildcards (Default is: True)
- `cellPattern` – Force rename cell names before `-renamePattern` apply. Can contain %C %L and %N (Default is: “)

Returns

Collection of new cell views

copyLib

Syntax

```
copyLib(  
sourceLib: oaLib | str,  
/,  
*,  
name: str,  
path: str,  
update: bool = None,  
skipPcells: bool = None,  
updateSearch: dm.copyLib.UpdateSearch = None,  
hier: bool = None,  
sourceConflicts: dm.copyLib.SourceConflicts = None,  
includeViews: list[str] = None,  
excludeViews: list[str] = None,  
excludeLibs: list[str] = None,  
includeViewTypes: list[str] = None,  
excludeViewTypes: list[str] = None,  
skipBackupFiles: bool = None,  
regex: bool = None,  
renamePattern: str = None,  
assign: str = None,  
cellPattern: str = None)  
Returns: oaLib
```

Summary:

Copies the specified Library to the new location

Category:

Data Management

Licenses:

None

Parameters

- `sourceLib` – The Library to copy
- `name` – The name of the new library
- `path` – The location of the new library
- `update` – Issue the pre/post callbacks for each dmfile (Default is: True)
- `skipPcells` – Allows to skip PCell references to be copied and updated. (Default is: False)
- `updateSearch` – Specify where to search for references to change ({ changed | destinationLib | none | writableLibs })
- `hier` – Descend the design hierarchy for each cellView that contains an oaDesign and copy instance master Cells even when they reside outside of the source specified (Default is: False)
- `sourceConflicts` – Action to take when copied cells from multiple source libraries have the same name ({ overwrite | rename | skip }) (Default is: 'rename')
- `includeViews` – Specifies the set of cellViews inside each Cell that should be copied
- `excludeViews` – Specifies the set of cellviews associated with each cell that should not be copied
- `excludeLibs` – Specifies libraries whose cellViews should not be copied
- `includeViewTypes` – The set of ViewType names of views inside each cell that should be copied. List members are ViewTypes and can use regular expressions or wildcards. This cannot be used if `excludeViewTypes` is used
- `excludeViewTypes` – The set of ViewType names of views associated with each copied cell that should not be copied. List members are ViewTypes and can use regular expressions or wildcards. This cannot be used if `includeViewTypes` is used
- `skipBackupFiles` – If true, the backup files in views are skipped during copy operation. (Default is: False)

- **regexp** – if true the pattern is interpreted as a regular expression, otherwise the pattern uses wildcards (Default is: True)
- **renamePattern** – Pattern for renaming a cell where conflicts exist. Can contain %C %L and %N (Default is: '%C_%L')
- **assign** – Library attributes
- **cellPattern** – Force rename cell names before -renamePattern apply. Can contain %C %L and %N (Default is: "")

Returns

New library

copyProps

Syntax

```
copyProps(  
*,  
    src: dmCell | oaLib,  
    dest: dmCell | oaLib)  
Returns: None
```

Summary:

Copies properties from one dmContainer to the other

Category:

Data Management

Licenses:

None

Parameters

- **src** – The source dmContainer, can be dmCell or oaLib
- **dest** – Destination dmContainer, can be dmCell or oaLib

Returns

None

createCell

Syntax

```
createCell(  
name: str,  
/,  
*,  
lib: oaLib = None,  
libName: dmLibName = None)  
Returns: dmCell
```

Summary:

Creates a cell - DM and OA versions

Category:

Data Management

Licenses:

None

Parameters

- name – Cell name
- lib – The Library in which Cell should be created
- libName – The Library name in which Cell should be created

Returns

New cell

createCellCategory

Syntax

```
createCellCategory(  
name: str,  
/,  
*,  
lib: oaLib = None,  
libName: str = None,  
cell: list[dmCell] = None,  
cellCategory: list[dmCellCategory] = None,  
parent: dmCellCategory = None)  
Returns: dmCellCategory
```

Summary:

Creates a cell category in the specified library

Category:

Data Management

Licenses:

None

Parameters

- name – The name of the cell category
- lib – The library in which to create the cell category
- libName – The library in which to create the cell category
- cell – A collection of dmCells to add as members
- cellCategory – A collection of dmCellCategories to add as members
- parent – The category that this will be a subcategory of

Returns

New cell category

createCellInput

Syntax

```
createCellInput(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
prefName: dbPrefName = None,  
toolTip: str = None,  
valueChangeProc: list[variant] = None,  
label: str = None,  
value: dmCell | str = None,  
readOnly: bool = None,  
valid: bool = None,  
required: bool = None,  
requireExisting: bool = None,  
showReadOnlyLibraries: bool = None,  
filterProc: list[variant] = None)  
Returns: dmCellInput
```

Summary:

Creates a new dmCellInput widget and adds it to the specified parent

Category:

Data Management

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `label` – Widget label, shown in the left column. (Default is: ‘Cell’)
- `value` – Initial value.
- `readOnly` – Initial read-only state. (Default is: False)
- `valid` – Initial valid state. (Default is: True)
- `required` – Specifies this is a required input. (Default is: False)
- `requireExisting` – Specifies that the cell must exist. (Default is: True)
- `showReadOnlyLibraries` – Show read only libraries or not (Default is: True)
- `filterProc` – Procedure to execute for each cellView object when populating the list of completions. Returns 1 if the cellView should be available or 0 otherwise.

Returns

New object.

createCellView

Syntax

```
createCellView(  
name: str,  
/,  
*,  
cell: dmCell | dmCellName,  
viewType: str)  
Returns: dmCellView
```

Summary:

Creates a cellView - DM and OA versions

Category:

Data Management

Licenses:

None

Parameters

- name – CellView name
- cell – The Cell in which the CellView should be created
- viewType – ViewType for cellView

Returns

New cell view

createCellViewInput

Syntax

```
createCellViewInput(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
prefName: dbPrefName = None,  
toolTip: str = None,  
valueChangeProc: list[variant] = None,  
label: str = None,  
value: dmCellView | str = None,
```

```
readOnly: bool = None,  
valid: bool = None,  
required: bool = None,  
requireExisting: bool = None,  
showReadOnlyLibraries: bool = None,  
filterProc: list[variant] = None)  
Returns: dmCellViewInput
```

Summary:

Creates a new dmCellViewInput widget and adds it to the specified parent

Category:

Data Management

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `label` – Widget label, shown in the left column. (Default is: ‘View’)
- `value` – Initial value.
- `readOnly` – Initial read-only state. (Default is: False)
- `valid` – Initial valid state. (Default is: True)
- `required` – Specifies this is a required input. (Default is: False)
- `requireExisting` – Specifies that the cellView must exist. (Default is: True)

- showReadOnlyLibraries – Show read only libraries or not (Default is: True)
- filterProc – Procedure to execute for each cellView object when populating the list of completions. Returns 1 if the cellView should be available or 0 otherwise.

Returns

New object.

createLib

Syntax

```
createLib(  
name: str,  
/,  
*,  
path: str,  
dmSystem: str = None,  
assign: str = None,  
attrs: list[map_] = None,  
compressionLevel: dm.createLib.CompressionLevel = None)  
Returns: oaLib
```

Summary:

Creates a library and updates the lib.defs

Category:

Data Management

Licenses:

None

Parameters

- name – Library name
- path – Location of the library
- dmSystem – DM system for the library (Default is: 'oaDMFileSys')
- assign – Name value pairs to be ASSIGNED to this library in the lib.defs file
- attrs – A list of persistent attributes to be given to the new library
- compressionLevel – Defines compression level of created library. Supported values are 1-9. Only oaDMFileSys and oaDMTurbo libraries are supported ({ 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 })

Returns

New library

createLibInput

Syntax

```
createLibInput(  
    _none_: str = None,  
    /,  
    *,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    prefName: dbPrefName = None,  
    toolTip: str = None,  
    valueChangeProc: list[variant] = None,  
    label: str = None,  
    value: str = None,  
    readOnly: bool = None,  
    valid: bool = None,  
    required: bool = None,  
    requireExisting: bool = None,  
    showReadOnlyLibraries: bool = None,  
    filterProc: list[variant] = None)  
Returns: dmLibInput
```

Summary:

Creates a new dmLibInput widget and adds it to the specified parent

Category:

Data Management

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)

- **prefName** – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- **toolTip** – Optional widget toolTip. (Default is: “)
- **valueChangeProc** – Procedure to execute when the value is changed. Called with the widget object
- **label** – Widget label, shown in the left column. (Default is: ‘Library’)
- **value** – Initial value.
- **readOnly** – Initial read-only state. (Default is: False)
- **valid** – Initial valid state. (Default is: True)
- **required** – Specifies this is a required input. (Default is: False)
- **requireExisting** – Specifies that the library must exist. (Default is: True)
- **showReadOnlyLibraries** – Show read only libraries or not (Default is: True)
- **filterProc** – Procedure to execute for each cellView object when populating the list of completions. Returns 1 if the cellView should be available or 0 otherwise.

Returns

New object.

createMilkywayLib

Syntax

```
createMilkywayLib(  
name: str,  
/,  
*,  
milkywayTechFile: str,  
refLibs: list[str] = None,  
busNameStyle: str = None,  
useExtendedLayers: bool = None)  
Returns: None
```

Summary:

Creates a Milkyway database using the specified Milkyway technology file

Category:

Data Translation

Licenses:

Backend

Parameters

- name – File path of the milkyway library to be created
- milkywayTechFile – Path, including the file name of technology file to use
- refLibs – List of paths (absolute or relative to main library) to reference libraries
- busNameStyle – Bus naming style for ICC
- useExtendedLayers – Enable extended layer support in the new library (Default is: False)

Returns

None

createMultiCellViewInput

Syntax

```
createMultiCellViewInput(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
prefName: dbPrefName = None,  
toolTip: str = None,  
filterProc: list[variant] = None,  
valueChangeProc: list[variant] = None,  
value: dmCellView = None,  
readOnly: bool = None,  
valid: bool = None,  
required: bool = None,  
requireExisting: bool = None)  
Returns: dmMultiCellViewInput
```

Summary:

Creates a new dmMultiCellViewInput widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `filterProc` – Procedure to execute for each `cellView` object when populating the list of completions. Returns 1 if the `cellView` should be available or 0 otherwise.
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `value` – Initial value.
- `readOnly` – Initial read-only state. (Default is: False)
- `valid` – Initial valid state. (Default is: True)
- `required` – Specifies if this is a required input. (Default is: False)
- `requireExisting` – Specifies that the library must exist. (Default is: True)

Returns

New `dmMultiCellViewInput` widget

deleteFromCellCategory

Syntax

```
deleteFromCellCategory(  
    cellCat:  dmCellCategory,  
    /,  
    *,  
    cell:  list[dmCell] = None,  
    cellCategory:  list[dmCellCategory] = None)  
Returns: dmCellCategory
```

Summary:

This method removes members, dmCells, or dmCellCategories from a cell category

Category:

Data Management

Licenses:

None

Parameters

- cellCat – The cell category to augment
- cell – A collection of Cells to remove from category
- cellCategory – A collection of dmCellCategories to remove from category

Returns

Cell category that was updated

findCell

Syntax

```
findCell(  
name: str,  
/,  
*,  
lib: oaLib = None,  
libName: dmLibName = None)  
Returns: dmCell
```

Summary:

Creates a DM cell - no OA cell will be created

Category:

Data Management

Licenses:

None

Parameters

- name – The name of the Cell to find
- lib – The Library in which Cell exists
- libName – The Library name in which Cell exists

Returns

Cell object of the specified name

findCellView

Syntax

```
findCellView(  
name: str,  
/,  
*,  
cell: dmCell = None,  
cellName: str = None,  
lib: oaLib = None,  
libName: str = None)  
Returns: dmCellView
```

Summary:

Returns a dmCellView of the specified name within the specified cell/lib

Category:

Data Management

Licenses:

None

Parameters

- name – The name of the CellView to find
- cell – The Cell in which the CellView exists
- cellName – The Cell name in which the CellView exists
- lib – The Library within which the CellView from the specified cellName exists
- libName – The Library name within which the CellView from the specified cellName exists

Returns

CellView object of the specified name

findDMData

Syntax

```
findDMData(  
    dmContainer: dmCell | dmCellName | dmCellView | dmCellViewName |  
    dmLibName | oaLib)  
Returns: oaDMData
```

Summary:

Attempts to find open DM data for given library, cell or cellView. If successful, oaDMData object is returned, otherwise, empty string is returned.

Category:

Data Management

Licenses:

None

Parameters

dmContainer – DM Data container:

Returns

DM Data object of the specified container

findDMFile

Syntax

```
findDMFile(  
    name: str,  
    /,  
    *,  
    dmContainer: dmCell | dmCellName | dmCellView | dmCellViewName |  
    dmLibName | oaLib)  
Returns: dmDMFile
```

Summary:

Creates a DM dm_file - no OA OADMFile will be created

Category:

Data Management

Licenses:

None

Parameters

- name – DM file name
- dmContainer – DM File container:

Returns

DM File object of the specified name

get3DICLayerMap

Syntax

```
get3DICLayerMap(  
    library: oaLib)  
Returns: str
```

Summary:

Get 3DIC Compiler layer mappings

Category:

Data Management

Licenses:

None

Parameters

library – Library

Returns

List of layer name pairs

get3DICRefLibs

Syntax

```
get3DICRefLibs(  
    *,
```

```
lib: oaLib)  
Returns: str
```

Summary:

Get NDM refLibs from snpsNDM library

Category:

Data Management

Licenses:

Backend

Parameters

lib – Library

Returns

Reflib paths

getCategoriesFromCategory

Syntax

```
getCategoriesFromCategory(  
pattern: str = None,  
/,  
*,  
category: dmCellCategory)  
Returns: list[dmCellCategory]
```

Summary:

Returns a collection of cell categories that are members of the specified cell category

Category:

Data Management

Licenses:

None

Parameters

- pattern – A glob-style wildcard pattern to match cell names (Default is: ‘*’)
- category – The cell category to query

Returns

Collection of cell categories in the cell category

getCategoriesOfMember

Syntax

```
getCategoriesOfMember(  
    pattern: str = None,  
    /,  
    *,  
    cellCategory: dmCellCategory = None,  
    cell: dmCell = None)  
Returns: list[dmCellCategory]
```

Summary:

This function returns a collection of cell categories whose names match the supplied pattern and for which the supplied cell or cell category is a member

Category:

Data Management

Licenses:

None

Parameters

- pattern – A glob-style wildcard pattern to match cell names (Default is: '*')
- cellCategory – The cell category member to query
- cell – The cell to query

Returns

A collection of cell categories

getCellCategories

Syntax

```
getCellCategories(  
    pattern: str = None,  
    /,  
    *,  
    lib: oaLib = None,  
    libName: str = None,
```

```
topLevel: bool = None)
Returns: list[dmCellCategory]
```

Summary:

Returns a collection of cell categories in the specified library

Category:

Data Management

Licenses:

None

Parameters

- pattern – A glob-style wildcard pattern to match cell names (Default is: '*')
- lib – The libraries from which to retrieve cell categories
- libName – The libraries from which to retrieve cell categories
- topLevel – Only include top level categories (Default is: False)

Returns

A collection of cell categories

getCells

Syntax

```
getCells(
pattern: str = None,
/,
*,
lib: list[oaLib] = None,
libName: dmLibName = None)
Returns: list[dmCell]
```

Summary:

gets cells from lib, matching the specified pattern

Category:

Data Management

Licenses:

None

Parameters

- pattern – A glob-style wildcard pattern to match cell names (Default is: ‘*’)
- lib – The libraries from which to retrieve cells
- libName – The libraries from which to retrieve cells

Returns

A collection of Cells

getCellsFromCategory

Syntax

```
getCellsFromCategory(  
    pattern: str = None,  
    /,  
    *,  
    category: dmCellCategory)  
Returns: list[dmCell]
```

Summary:

Returns a collection of dmCells that are members of the specified cell category

Category:

Data Management

Licenses:

None

Parameters

- pattern – A glob-style wildcard pattern to match cell names (Default is: ‘*’)
- category – The cell category to query

Returns

A collection of cells in the cell category

getCellViews

Syntax

```
getCellViews(  
    pattern: str = None,  
    /,
```

```
*,  
cellName: str = None,  
cell: list[dmCell] = None,  
libName: str = None,  
lib: list[oaLib] = None)  
Returns: list[dmCellView]
```

Summary:

get cellViews, matching the specified pattern

Category:

Data Management

Licenses:

None

Parameters

- pattern – Glob-style wildcard pattern to match cellview names (Default is: '*')
- cellName – The cell name from which to retrieve CellViews
- cell – The list of cells from which to retrieve cellviews
- libName – The library name from which to retrieve CellViews from the specified cellName
- lib – The collection of libraries within which to retrieve CellViews from the specified cellName

Returns

A collection of CellViews

getDMFileFollowers

Syntax

```
getDMFileFollowers(  
pattern: str = None,  
/  
*,  
leader: dmDMFile)  
Returns: list[dmDMFile]
```

Summary:

Returns a collection of DMFiles that are considered followers of this DMFile

Category:

Data Management

Licenses:

None

Parameters

- pattern – A glob-style wildcard pattern to match follower names against (Default is: ‘*’)
- leader – The DM file to query

Returns

A collection of followers of a DMFile

getDMFileLeader

Syntax

```
getDMFileLeader(  
    dm_file: dmDMFile)  
Returns: dmDMFile
```

Summary:

Returns the leader of this DMFile

Category:

Data Management

Licenses:

None

Parameters

dm_file – The DM file to query

Returns

The leader of the DMFile

getDMFiles

Syntax

```
getDMFiles(  
    pattern: str = None,
```

```
//  
*,  
dmContainer: dmCell | dmCellView | oaLib | str)  
Returns: list[dmDMFile]
```

Summary:

gets DMFiles, matching the specified pattern

Category:

Data Management

Licenses:

None

Parameters

- pattern – A glob-style wildcard pattern to match library names (Default is: '*')
- dmContainer – The DM container from which to retrieve dm files

Returns

A collection of DM Files

getFCLayerMap

Syntax

```
getFCLayerMap(  
library: oaLib)  
Returns: str
```

Summary:

Get Fusion Compiler layer mappings

Category:

Data Management

Licenses:

None

Parameters

library – Library

Returns

List of layer name pairs

getFCRefLibs

Syntax

```
getFCRefLibs (  
*,  
    lib: oaLib)  
Returns: str
```

Summary:

Get NDM refLibs from snpsNDM library

Category:

Data Management

Licenses:

Backend

Parameters

lib – Library

Returns

Reflib paths

getFollowers

Syntax

```
getFollowers (  
pattern: str | None = None,  
    /,  
    *,  
    leader: str)  
Returns: str
```

Summary:

Return a list of Followers for the supplied Leader

Category:

Data Management

Licenses:

None

Parameters

- pattern – A glob-style wildcard pattern to match follower names against (Default is: '*')
- leader – The DMFile name (leader name) to query

Returns

A list of strings containing the names of followers

getICC2LayerMap

Syntax

```
getICC2LayerMap(  
    library: oaLib)  
Returns: str
```

Summary:

Get ICC2 layer mappings

Category:

Data Management

Licenses:

None

Parameters

library – Library

Returns

List of layer name pairs

getICC2RefLibs

Syntax

```
getICC2RefLibs(  
    *,  
    lib: oaLib)  
Returns: str
```


Summary:

Get NDM refLibs from snpsNDM library

Category:

Data Management

Licenses:

Backend

Parameters

lib – Library

Returns

Reflib paths

getLeaders

Syntax

```
getLeaders(  
    pattern: str | None = None)  
Returns: str
```

Summary:

Return a strings list, defined as Leaders in the Lib

Category:

Data Management

Licenses:

None

Parameters

pattern – A glob-style wildcard pattern for leader names (Default is: '*')

Returns

List of matching DMFile names defined as Leaders in the library

getLibDefPath

Syntax

```
getLibDefPath(  
    *,  
    name: str | None = None)  
Returns: str
```

Summary:

Get the default path to the library definition file

Category:

Data Management

Licenses:

None

Parameters

name – name of library definitions file (Default is: 'lib.defs')

Returns

Full default path, including the filename, for the library definition file

getLibs

Syntax

```
getLibs(  
    pattern: str = None)  
Returns: list[oaLib]
```

Summary:

gets opened libraries, matching the specified pattern

Category:

Data Management

Licenses:

None

Parameters

pattern – A glob-style wildcard pattern to match library names (Default is: '*')

Returns

A collection of libraries

getMilkywayRefLibs

Syntax

```
getMilkywayRefLibs (  
    filepath: str,  
    /,  
    *,  
    hier: bool | None = None)  
Returns: str
```

Summary:

Returns a Tcl list of Milkyway reference library names from a library.

Category:

Data Translation

Licenses:

None

Parameters

- filepath – Filepath of the Milkyway library from which to get the specified reference libraries (ref. lib. list)
- hier – If true, does a deep extraction of all reference libraries (Default is: False)

Returns

List of filepaths to the reference libraries specified in this Milkyway library

getOpenDesigns

Syntax

```
getOpenDesigns ()  
Returns: list[oaDesign]
```

Arguments:

Summary:

get open designs

Category:

Data Management

Licenses:

None

Returns

A collection of open designs

getSelected

Syntax

```
getSelected(  
    *,  
    window: giWindow = None)  
Returns: list[dmCell] | list[dmCellCategory] | list[dmCellView] |  
    list[dmDMFile] | list[oaLib]
```

Summary:

Returns the list of items currently selected in the specified Library Manager window

Category:

Data Management

Licenses:

None

Parameters

window – A Library Manager window (Default is: ")

Returns

A collection of Selected Items

getStandardLibDefPath

Syntax

```
getStandardLibDefPath()  
Returns: str
```

Arguments:

Summary:

Get the path to the current session's standard library definition file path

Category:

Data Management

Licenses:

None

Returns

Full path, including the filename, for the current session's standard library definition file

getTopLibDefPath

Syntax

```
getTopLibDefPath()  
Returns: str
```

Arguments:

Summary:

Get the path to the current session's top library definition file

Category:

Data Management

Licenses:

None

Returns

Full path, including the filename, for the current session's top library definition file

getVersionControl

Syntax

```
getVersionControl(  
lib: oaLib)  
Returns: str
```

Summary:

Returns the name of the version control system that manages a given open library. If this library does not have a registered version control system then it returns an empty string

Category:

Data Management

Licenses:

None

Parameters

lib – An open library managed by this version control system

Returns

The name of the version control system

getVersionControlSystems

Syntax

```
getVersionControlSystems(  
    name: str = None)  
Returns: list[dmVersionControlSystem]
```

Summary:

Returns a collection of dmVersionControlSystem objects

Category:

Data Management

Licenses:

None

Parameters

name – The name of the VC system (Default is: “)

Returns

A collection of Version controls

isDMFileLeader

Syntax

```
isDMFileLeader(  
    dm_file: dmDMFile)  
Returns: bool
```

Summary:

Checks if this DMFile is a leader, if yes return true

Category:

Data Management

Licenses:

None

Parameters

dm_file – The DM file to query

Returns

TRUE if the DMFile is a leader; FALSE otherwise

isHidden

Syntax

```
isHidden(  
    object: dmCell | oaLib | str,  
    /,  
    *,  
    lib: oaLib = None,  
    libName: str = None)  
Returns: bool
```

Summary:

Checks whether the cell is hidden, which means it is included in Hidden category hierarchy. Or checks if the library has INVISIBLE attribute assigned in lib.defs file.

Category:

Data Management

Licenses:

None

Parameters

- object – Cell or Library, to check
- lib – The Library
- libName – The Library name

Returns

Returns true if cell is included in Hidden category or in its subcategories. Returns true if library is defined INVISIBLE in lib.defs, false otherwise.

moveCells

Syntax

```
moveCells(  
sourceCells: list[dmCell],  
/,  
*,  
libDestName: str,  
cellDestName: str = None,  
update: bool = None,  
skipPcells: bool = None,  
updateSearch: dm.moveCells.UpdateSearch = None,  
category: list[str] = None,  
maintainCategories: bool = None,  
destinationConflicts: dm.moveCells.DestinationConflicts = None,  
cellPattern: str = None)  
Returns: None
```

Summary:

Moves an existing cells to a new location

Category:

Data Management

Licenses:

None

Parameters

- sourceCells – Cells to move
- libDestName – Destination library name
- cellDestName – Destination cell name, required when moving single cell (Default is: “)

- **update** – Issue the pre/post callbacks for each dmfile (Default is: True)
- **skipPcells** – Allows to skip PCell references to be copied and updated. (Default is: False)
- **updateSearch** – Specify where to search for references to change ({ changed | destinationLib | none | writableLibs })
- **category** – Adds all cells moved to to the named category. Creates the category if necessary
- **maintainCategories** – Adds copied cell into allcategories that the source cell is a member of (Default is: True)
- **destinationConflicts** – Action to take when a copied cell already exists in the destination ({ overwrite | rename | skip } (Default is: 'rename'))
- **cellPattern** – Force rename cell names before -renamePattern apply. Can contain %C %L and %N (Default is: "")

Returns

None

moveCellViews

Syntax

```
moveCellViews(  
    sourceCellViews: list[dmCellView],  
    /,  
    *,  
    libDestName: str,  
    cellDestName: str,  
    viewDestName: str = None,  
    update: bool = None,  
    skipPcells: bool = None,  
    cellPattern: str = None)  
Returns: oaLib
```

Summary:

Moves an existing cellviews to a new location

Category:

Data Management

Licenses:

None

Parameters

- sourceCellViews – CellViews to move
- libDestName – Destination library name
- cellDestName – Destination cell name
- viewDestName – Destination view name required when moving one cellview (Default is: “)
- update – Issue the pre/post callbacks for each dmfile (Default is: True)
- skipPcells – Allows to skip PCell references to be copied and updated. (Default is: False)
- cellPattern – Force rename cell names before -renamePattern apply. Can contain %C %L and %N (Default is: “)

Returns

The library being moved

moveLib

Syntax

```
moveLib(  
sourceLib: oaLib | str,  
/,  
*,  
name: str,  
path: str,  
update: bool = None,  
skipPcells: bool = None,  
assign: str = None,  
cellPattern: str = None)  
Returns: oaLib
```

Summary:

Moves the specified Library to the new location

Category:

Data Management

Licenses:

None

Parameters

- sourceLib – Library to move
- name – Destination library name
- path – Destination library path
- update – Issue the pre/post callbacks for each dmfile (Default is: True)
- skipPcells – Allows to skip PCell references to be copied and updated. (Default is: False)
- assign – Library attributes
- cellPattern – Force rename cell names before -renamePattern apply. Can contain %C %L and %N (Default is: “)

Returns

The library being moved

openLibraryManager

Syntax

```
openLibraryManager()  
Returns: giWindow
```

Arguments:

Summary:

Open Library Manager

Category:

Data Management

Licenses:

None

Returns

The window identifier

openLibs

Syntax

```
openLibs (  
*,  
  name:  str  |  None  =  None,  
  path:  str  |  None  =  None)  
Returns: None
```

Summary:

Opens libraries specified in lib.defs file

Category:

Data Management

Licenses:

None

Parameters

- name – name of library definitions file (Default is: 'lib.defs')
- path – path to lib.defs file (Default is: '')

Returns

None

refreshLibraryManager

Syntax

```
refreshLibraryManager(  
window:  giWindow  =  None)  
Returns: None
```

Summary:

Refreshes the specified Library Manager window or all Library Manager windows if argument not specified

Category:

Data Management

Licenses:

None

Parameters

window – A Library Manager window (Default is: “)

Returns

None

refreshLibs

Syntax

```
refreshLibs (  
path: str | None = None)  
Returns: None
```

Summary:

Refresh Library Definition File Libs

Category:

Data Management

Licenses:

None

Parameters

path – full path libdef file to open (Default is: “)

Returns

None

registerCopyProps

Syntax

```
registerCopyProps (  
names: list[str],  
/,  
*,  
tclProc: str,  
overwrite: bool = None)  
Returns: None
```

Summary:

Registers tcl proc that will be call by the dm::copyProps command

Category:

Data Management

Licenses:

None

Parameters

- names – a list of property names
- tclProc – The Tcl proc to use to copy the supplied list of property. Proc signature should be: `proc myProc { dboaProplter <oaLib>|<dmCell> } {}`
- overwrite – If false, appends the supplied Tcl proc to the list of previously supplied Tcl proc for properties of the same name. If true, the previous list of Tcl procedures is cleared (Default is: False)

Returns

None

registerVersionControl

Syntax

```
registerVersionControl(  
name: str,  
/  
*,  
primary: bool = None,  
checkInProc: dbExecObject | str = None,  
checkOutProc: dbExecObject | str = None,  
testCheckOutProc: dbExecObject | str = None,  
deleteProc: dbExecObject | str = None,  
deleteProcWithOp: dbExecObject | str = None,  
deleteByVersionControl: bool = None,  
createProc: dbExecObject | str = None,  
libraryOpenProc: dbExecObject | str = None,  
libraryCreateProc: dbExecObject | str = None,  
libDefsUpdateProc: dbExecObject | str = None,  
copyProc: dbExecObject | str = None,  
testCopyProc: dbExecObject | str = None,  
testCategoryProc: dbExecObject | str = None,  
dataExposedProc: dbExecObject | str = None,  
checkStatusProc: dbExecObject | str = None,  
revertProc: dbExecObject | str = None,  
syncToVersionProc: dbExecObject | str = None,  
showDMFiles: bool = None,  
**kwargs)  
Returns: dmVersionControlSystem
```

Summary:

Provides DM with the name of a VC system integration and specifies if this VC system can be used in creating new libraries

Category:

Data Management

Licenses:

None

Parameters

- name – The name of version control system
- primary – Specify if this is the primary version control system for this session
- checkInProc – The name of the function to be called when library elements are to be checked in
- checkOutProc – The name of the function to be called when library elements need to be editable
- testCheckOutProc – The name of the function to be called to check if library elements can be checked out
- deleteProc – The name of the function to call before deleting library data
- deleteProcWithOp – Alternative delete proc that gets an operation argument
- deleteByVersionControl – State if the integration will do the deletion when the deleteProc is called
- createProc – The name of the function to call before creating library data
- libraryOpenProc – The name of the function to be called when a library managed by this version control system is opened
- libraryCreateProc – The name of the function to be called when a library is created
- libDefsUpdateProc – The name of the function to be called with a path to a lib.defs file when a library is created
- copyProc – The name of the function to be called when library elements will be copied or moved
- testCopyProc – The name of the function to be called to check if library elements can be copied or moved

- `testCategoryProc` – The name of the function to be called to check if categories can be checked out
- `dataExposedProc` – The name of the function to be called when a view or category is exposed in Library Manager
- `checkStatusProc` – The name of the function to be called to check the status
- `revertProc` – The name of the function to be called to revert a check out of library elements
- `syncToVersionProc` – The name of the function to be called to sync a library element to a specific version
- `showDMFiles` – Specify if show dmFiles is supported by VC system

Returns

Version Control System created

removeVersionControl

Syntax

```
removeVersionControl(  
*,  
    lib: oaLib = None,  
    libName: str = None)  
Returns: None
```

Summary:

Specifies that this library should no longer be associated with its version control system. Operations on this library will no longer trigger VC callbacks.

Category:

Data Management

Licenses:

None

Parameters

- `lib` – An open library managed by the version control system
- `libName` – Name of the library to use

Returns

None

renameCellCategory

Syntax

```
renameCellCategory(  
    cellCat:  dmCellCategory,  
    /,  
    *,  
    name:  str)  
Returns: dmCellCategory
```

Summary:

Rename a cell category

Category:

Data Management

Licenses:

None

Parameters

- cellCat – The cell category to rename
- name – The new name of the cell category

Returns

Cell category that was updated

revert

Syntax

```
revert(  
    container:  list[dmCellCategory] | list[dmCellView] | list[dmDMFile])  
Returns: None
```

Summary:

Reverts (cancels checkout) dmCellView, dmFile or dmCellCategory.

Category:

Data Management

Licenses:

None

Parameters

container – DM collection to revert

Returns

None

saveAsFCRefLib

Syntax

```
saveAsFCRefLib(  
*,  
    lib: oaLib,  
    path: str,  
    **kwargs)  
Returns: None
```

Summary:

Save snpsNDM library as NDM LibCell library

Category:

Data Management

Licenses:

Backend

Parameters

- lib – Library
- path – Path

Returns

None

saveAsICC2RefLib

Syntax

```
saveAsICC2RefLib(  
*,  
    lib: oaLib,  
    path: str,  
    **kwargs)  
Returns: None
```

Summary:

Save snpsNDM library as NDM LibCell library

Category:

Data Management

Licenses:

Backend

Parameters

- lib – Library
- path – Path

Returns

None

set3DICLCVMap

Syntax

```
set3DICLCVMap (  
*,  
    library: oaLib,  
    file: str)  
Returns: None
```

Summary:

Set LCV mapping for a snpsNDM library

Category:

Data Management

Licenses:

Backend

Parameters

- library – Library
- file – LCV mapping file

Returns

None

set3DICRefLibs

Syntax

```
set3DICRefLibs (  
*,  
  lib: oaLib,  
  paths: list[str] = None,  
  addLib: list[str] = None,  
  addDir: str = None,  
  removeLib: list[str] = None,  
  removeAll: bool = None)  
Returns: None
```

Summary:

Set NDM refLibs in a snpsNDM library

Category:

Data Management

Licenses:

Backend

Parameters

- lib – Library
- paths – Set the given libraries
- addLib – Add the given libraries to the existing reference libraries list
- addDir – Add all libraries from given directory to the existing reference libraries list
- removeLib – Remove the given libraries from the existing reference libraries list
- removeAll – Remove all reference libraries

Returns

None

setFCLCVMap

Syntax

```
setFCLCVMap (  
*,  
  library: oaLib,
```

```
file: str)
Returns: None
```

Summary:

Set LCV mapping for a snpsNDM library

Category:

Data Management

Licenses:

Backend

Parameters

- library – Library
- file – LCV mapping file

Returns

None

setFCRefLibs

Syntax

```
setFCRefLibs (
*,
lib: oaLib,
paths: list[str] = None,
addLib: list[str] = None,
addDir: str = None,
removeLib: list[str] = None,
removeAll: bool = None)
Returns: None
```

Summary:

Set NDM refLibs in a snpsNDM library

Category:

Data Management

Licenses:

Backend

Parameters

- lib – Library
- paths – Set the given libraries
- addLib – Add the given libraries to the existing reference libraries list
- addDir – Add all libraries from given directory to the existing reference libraries list
- removeLib – Remove the given libraries from the existing reference libraries list
- removeAll – Remove all reference libraries

Returns

None

setFollowers

Syntax

```
setFollowers(  
*,  
    leader: str,  
    followers: list[str],  
    append: bool = None)  
Returns: None
```

Summary:

Define a set of Followers for the supplied Leader

Category:

Data Management

Licenses:

None

Parameters

- leader – The DMFile name for which followers are being defined
- followers – A list of DMFile names that are followers of the leader
- append – The list of cells from which to retrieve cellviews (Default is: False)

Returns

None

setICC2LCVMap

Syntax

```
setICC2LCVMap(  
*,  
    library: oaLib,  
    file: str)  
Returns: None
```

Summary:

Set LCV mapping for a snpsNDM library

Category:

Data Management

Licenses:

Backend

Parameters

- library – Library
- file – LCV mapping file

Returns

None

setICC2RefLibs

Syntax

```
setICC2RefLibs(  
*,  
    lib: oaLib,  
    paths: list[str] = None,  
    addLib: list[str] = None,  
    addDir: str = None,  
    removeLib: list[str] = None,  
    removeAll: bool = None)  
Returns: None
```

Summary:

Set NDM refLibs in a snpsNDM library

Category:

Data Management

Licenses:

Backend

Parameters

- lib – Library
- paths – Set the given libraries
- addLib – Add the given libraries to the existing reference libraries list
- addDir – Add all libraries from given directory to the existing reference libraries list
- removeLib – Remove the given libraries from the existing reference libraries list
- removeAll – Remove all reference libraries

Returns

None

setICCTechSync

Syntax

```
setICCTechSync(  
lib: oaLib,  
/,  
*,  
layerMap: str = None,  
viaMap: str = None)  
Returns: None
```

Summary:

Reconfigures the specified ICC library to either synchronize technology data or to use separate and independent technology data.

Category:

Data Translation

Licenses:

Backend

Parameters

- `lib` – Library to modify.
- `layerMap` – Specifies a layer mapping file. Only valid when synchronization is disabled. (Default is: No layer mapping file)
- `viaMap` – Specifies a via mapping file. Only valid when synchronization is disabled. (Default is: No via mapping file)

Returns

None

setSelected

Syntax

```
setSelected(  
objects: list[dmCell] | list[dmCellCategory] | list[dmCellView] |  
         list[dmDMFile] | list[oaLib] = None,  
/,  
*,  
window: giWindow)  
Returns: None
```

Summary:

Sets the selected library objects in the specified Library Manager window

Category:

Data Management

Licenses:

None

Parameters

- `objects` – List of objects of same kind
- `window` – A Library Manager window

Returns

None

showCopyCell

Syntax

```
showCopyCell(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Show Copy Cell window

Category:

Data Management

Licenses:

None

Parameters

parent – parent widget

Returns

The dialog

showCopyCellView

Syntax

```
showCopyCellView(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Show Copy CellView window

Category:

Data Management

Licenses:

None

Parameters

parent – parent widget

Returns

The dialog

showCopyLibrary

Syntax

```
showCopyLibrary(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Show Copy Library window

Category:

Data Management

Licenses:

None

Parameters

parent – parent widget

Returns

The dialog

showCopyUnique

Syntax

```
showCopyUnique(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Show Copy Unique window

Category:

Data Management

Licenses:

None

Parameters

parent – parent widget

Returns

The dialog

showDeleteViewsByName

Syntax

```
showDeleteViewsByName(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Show Delete Views By Name dialog

Category:

Data Management

Licenses:

None

Parameters

parent – parent widget

Returns

The dialog

showEditLibraryCompression

Syntax

```
showEditLibraryCompression(  
*,
```

```
parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Show Edit Library Compression Dialog

Category:

Data Management

Licenses:

None

Parameters

parent – parent widget

Returns

The dialog

showLibraryManager

Syntax

```
showLibraryManager()  
Returns: None
```

Arguments:

Summary:

Open Library Manager

Category:

Data Management

Licenses:

None

Returns

None

showMoveCell

Syntax

```
showMoveCell(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Show Move Cell window

Category:

Data Management

Licenses:

None

Parameters

parent – parent widget

Returns

The dialog

showMoveCellView

Syntax

```
showMoveCellView(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Show Move CellView window

Category:

Data Management

Licenses:

None

Parameters

parent – parent widget

Returns

The dialog

showMoveLibrary

Syntax

```
showMoveLibrary(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Show Move Library window

Category:

Data Management

Licenses:

None

Parameters

parent – parent widget

Returns

The dialog

showNewCell

Syntax

```
showNewCell(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Show New Cell window

Category:

Data Management

Licenses:

None

Parameters

parent – parent widget

Returns

The dialog

showNewCellView

Syntax

```
showNewCellView(  
*,  
    parent: giDialog | giWindow | int = None)  
Returns: giDialog
```

Summary:

Show New CellView window

Category:

Data Management

Licenses:

None

Parameters

parent – parent widget

Returns

The dialog

showNewLibrary

Syntax

```
showNewLibrary(  
*,  
    parent: giDialog | giWindow | int = None,
```



```
libName: str = None,  
dmType: dm.showNewLibrary.DmType = None)  
Returns: giDialog
```

Summary:

Show New Library window

Category:

Data Management

Licenses:

None

Parameters

- parent – parent widget
- libName – Library name to be initialized
- dmType – DM type specifier ({ any | filesys | icc | icc2 })

Returns

The dialog

showRenameCellCategory

Syntax

```
showRenameCellCategory(  
*,  
parent: giWindow | int = None,  
cellCategory: dmCellCategory)  
Returns: giDialog
```

Summary:

Show Rename Cell Category dialog

Category:

Data Management

Licenses:

None

Parameters

- parent – parent widget
- cellCategory – cellCategory

Returns

The dialog

syncToVersion

Syntax

```
syncToVersion(  
    container: list[dmCellCategory] | list[dmCellView] | list[dmDMFile],  
    /,  
    *,  
    to: str = None)  
Returns: None
```

Summary:

Syncs current (latest) version to specified (to) version

Category:

Data Management

Licenses:

None

Parameters

- container – DM collection to sync
- to – Version of container to sync to

Returns

None

uncompress

Syntax

```
uncompress(  
    library: dmLibName | oaLib)  
Returns: None
```

Summary:

Uncompresses given library using oazip -decompress

Category:

Data Management

Licenses:

None

Parameters

library – The library or library name, to uncompress

Returns

None

updateLib

Syntax

```
updateLib(  
lib: oaLib)  
Returns: None
```

Summary:

Update NDM library file

Category:

Data Management

Licenses:

None

Parameters

lib – NDM Library

Returns

None

snps.custom.cmd.dr

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you administer display resources.

This section covers the following topics:

- [createColor](#)
- [createColorSelector](#)
- [createDisplay](#)
- [createLineStyle](#)
- [createLineStyleSelector](#)
- [createPacket](#)
- [createPacketAlias](#)
- [createStipple](#)
- [createTextPacket](#)
- [deleteColor](#)
- [deleteLineStyle](#)
- [deleteStipple](#)
- [getColors](#)
- [getDisplays](#)
- [getLineStyles](#)
- [getPacketAliases](#)
- [getPackets](#)
- [getStipples](#)
- [getTextPackets](#)
- [loadDisplayResources](#)
- [mergeDisplayResources](#)

- [renameColor](#)
 - [renameLineStyle](#)
 - [renameStipple](#)
 - [saveDisplayResources](#)
 - [showDisplayResourceEditor](#)
-

createColor

Syntax

```
createColor(  
name: str,  
/,  
*,  
display: drDisplay,  
red: int,  
green: int,  
blue: int,  
blink: bool = None)  
Returns: None
```

Summary:

Create color

Category:

Display Resources

Licenses:

None

Parameters

- name – name of color to create
- display – display object
- red – red value of the color
- green – green value of the color
- blue – blue value of the color
- blink – blinkness of color (Default is: False)

Returns

None

createColorSelector

Syntax

```
createColorSelector(  
    _none_: str = None,  
    /,  
    *,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    prefName: dbPrefName = None,  
    tooltip: str = None,  
    valueChangeProc: list[variant] = None,  
    label: str = None,  
    value: str = None,  
    showNames: bool = None,  
    display: str = None,  
    iconWidth: int = None,  
    iconHeight: int = None)  
Returns: deColorSelector
```

Summary:

Creates a new drColorSelector widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)

- `toolTip` – Optional widget `toolTip`. (Default is: “")
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `label` – Widget label, shown in the left column. (Default is: “")
- `value` – Initial value. (Default is: “")
- `showNames` – Display color names. (Default is: False)
- `display` – Display name. (Default is: ‘display’)
- `iconWidth` – Icon Width. (Default is: 16)
- `iconHeight` – Icon Height. (Default is: 16)

Returns

New object.

createDisplay

Syntax

```
createDisplay(  
name: str,  
/,  
*,  
maxColors: int = None,  
maxStipples: int = None,  
maxLineStyle: int = None,  
force: bool = None)  
Returns: drDisplay
```

Summary:

Create display

Category:

Display Resources

Licenses:

None

Parameters

- `name` – name of display to create
- `maxColors` – fixed size of color table (Default is: 0)

- `maxStipples` – fixed size of stipple table (Default is: 0)
- `maxLineStyle` – fixed size of linestyle table (Default is: 0)
- `force` – If set to true an existing display with specified name will be overwritten (Default is: False)

Returns

The new display object

createLineStyle

Syntax

```
createLineStyle(  
name: str,  
/,  
*,  
display: drDisplay,  
width: int,  
pattern: list[int])  
Returns: None
```

Summary:

Create LineStyle

Category:

Display Resources

Licenses:

None

Parameters

- `name` – name of linestyle to create
- `display` – display object
- `width` – thickness of linestyle
- `pattern` – pattern of linestyle

Returns

None

createLineStyleSelector

Syntax

```
createLineStyleSelector(  
    _none_: str = None,  
    /,  
    *,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    prefName: dbPrefName = None,  
    toolTip: str = None,  
    valueChangeProc: list[variant] = None,  
    label: str = None,  
    value: str = None,  
    showNames: bool = None,  
    display: str = None,  
    iconWidth: int = None)  
Returns: deLineStyleSelector
```

Summary:

Creates a new drLineStyleSelector widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `label` – Widget label, shown in the left column. (Default is: “)

- value – Initial value. (Default is: "")
- showNames – Display line style names. (Default is: False)
- display – Display name. (Default is: 'display')
- iconWidth – Icon Width. (Default is: 50)

Returns

New object.

createPacket

Syntax

```
createPacket(  
name: str,  
/,  
*,  
display: drDisplay,  
stipple: str,  
lineStyle: str = None,  
fill: str,  
outline: str,  
fillStyle: str = None)  
Returns: drPacket
```

Summary:

Create packet

Category:

Display Resources

Licenses:

None

Parameters

- name – name of the packet to create
- display – display object
- stipple – the name of stipple
- lineStyle – the name of linestyle (Default is: '_NA_')
- fill – the name of fill color

- `outline` – the name of outline color
- `fillStyle` – the name of fill style (Default is: 'outlineStipple')

Returns

The new display packet

createPacketAlias

Syntax

```
createPacketAlias(  
name: str,  
/,  
*,  
packet: str,  
display: drDisplay)  
Returns: drPacketAlias
```

Summary:

Create packet_alias

Category:

Display Resources

Licenses:

None

Parameters

- `name` – The name of packet_alis to create
- `packet` – The name of packet to which created packetAlias will point
- `display` – The display object in which to create packetAlias

Returns

The new display packet alias

createStipple

Syntax

```
createStipple(  
name: str,  
/,  
*,
```

```
display: drDisplay,  
pattern: matrix)  
Returns: None
```

Summary:

Create stipple

Category:

Display Resources

Licenses:

None

Parameters

- name – name of stipple to create
- display – display object
- pattern – pattern of stipple

Returns

None

createTextPacket

Syntax

```
createTextPacket(  
name: str,  
/,  
*,  
display: drDisplay,  
color: str,  
backgroundColor: str = None,  
fontName: str = None,  
height: str = None,  
bold: str = None,  
italic: str = None,  
underline: str = None)  
Returns: drTextPacket
```

Summary:

Create text packet

Category:

Display Resources

Licenses:

None

Parameters

- name – Name of text packet
- display – display object
- color – Foreground color
- backgroundColor – Background color (Default is: 'inherited')
- fontName – Name of font (Default is: 'inherited')
- height – Height of text in points (Default is: 'inherited')
- bold – Bold (Default is: 'inherited')
- italic – Italic (Default is: 'inherited')
- underline – Underline (Default is: 'inherited')

Returns

The new display textPacket

deleteColor

Syntax

```
deleteColor(  
name: str,  
/,  
*,  
display: drDisplay)  
Returns: None
```

Summary:

Delete Color

Category:

Display Resources

Licenses:

None

Parameters

- name – name of object to delete
- display – display object

Returns

None

deleteLineStyle

Syntax

```
deleteLineStyle(  
name: str,  
/,  
*,  
display: drDisplay)  
Returns: None
```

Summary:

Delete LineStyle

Category:

Display Resources

Licenses:

None

Parameters

- name – name of object to delete
- display – display object

Returns

None

deleteStipple

Syntax

```
deleteStipple(  
name: str,  
/,  
*,
```

```
display: drDisplay)
Returns: None
```

Summary:

Delete Stipple

Category:

Display Resources

Licenses:

None

Parameters

- name – name of object to delete
- display – display object

Returns

None

getColors

Syntax

```
getColors(
name: str = None,
/,
*,
display: drDisplay = None)
Returns: list[drColor]
```

Summary:

Get Colors

Category:

Display Resources

Licenses:

None

Parameters

- name – name of color to get (Default is: '*')
- display – display object

Returns

Collection of display colors

getDisplay

Syntax

```
getDisplay(  
name: str = None)  
Returns: list[drDisplay]
```

Summary:

Get Displays

Category:

Display Resources

Licenses:

None

Parameters

name – name of display to get (Default is: “)

Returns

Collection of all existing display objects

getLineStyle

Syntax

```
getLineStyle(  
name: str = None,  
/,  
*,  
display: drDisplay = None)  
Returns: list[drLineStyle]
```

Summary:

Get LineStyles

Category:

Display Resources

Licenses:

None

Parameters

- name – name of linestyle to get (Default is: '*')
- display – display object

Returns

Collection of display linestyles

getPacketAliases

Syntax

```
getPacketAliases(  
name: str = None,  
/,  
*,  
display: drDisplay = None,  
packet: drPacket = None)  
Returns: list[drPacketAlias]
```

Summary:

Get PacketAliases

Category:

Display Resources

Licenses:

None

Parameters

- name – name of packet_alias to get (Default is: '*')
- display – display object
- packet – packet object

Returns

Collection of display packet aliases

getPackets

Syntax

```
getPackets(  
name: str = None,  
/,  
*,  
display: drDisplay = None)  
Returns: list[drPacket]
```

Summary:

Get Packets

Category:

Display Resources

Licenses:

None

Parameters

- name – name of packet to get (Default is: '*')
- display – display object

Returns

Collection of display packets

getStipples

Syntax

```
getStipples(  
name: str = None,  
/,  
*,  
display: drDisplay = None)  
Returns: list[drStipple]
```

Summary:

Get Stipples

Category:

Display Resources

Licenses:

None

Parameters

- name – name of stipple to get (Default is: '*')
- display – display object

Returns

Collection of display stipples

getTextPackets

Syntax

```
getTextPackets(  
name: str = None,  
/,  
*,  
display: drDisplay = None)  
Returns: dbCollection
```

Summary:

Get text packets

Category:

Display Resources

Licenses:

None

Parameters

- name – name of text packet to get (Default is: '*')
- display – display object

Returns

Collection of display text packets

loadDisplayResources

Syntax

```
loadDisplayResources(  
*,  
    filePath: str = None,  
    reload: bool = None,  
    reloadDefault: bool = None,  
    format: dr.loadDisplayResources.Format = None)  
Returns: None
```

Summary:

Load display resources

Category:

Display Resources

Licenses:

None

Parameters

- filePath – Full or relative path of display resource file to open (Default is: “)
- reload – load a file or reload startup data (Default is: False)
- reloadDefault – reloads the default display.tcl file (Default is: False)
- format – Format of DR file to load ({ auto | drf | dsp | tcl })

Returns

None

mergeDisplayResources

Syntax

```
mergeDisplayResources(  
*,  
    srcFilePaths: list[str],  
    srcFormats: list[str] = None,  
    destFilePath: str,  
    load: bool = None,  
    destFormat: dr.mergeDisplayResources.DestFormat = None,  
    force: bool = None)  
Returns: None
```

Summary:

Merge display resources

Category:

Display Resources

Licenses:

None

Parameters

- `srcFilePaths` – One or more full or partial paths of DR files to merge
- `srcFormats` – Format(s) of source files
- `destFilePath` – Full or relative path of DR file to save
- `load` – Load merged DR data into memory (Default is: False)
- `destFormat` – Format of DR file to load ({ auto | drf | tcl })
- `force` – Overwrite existing file if true (Default is: False)

Returns

None

renameColor

Syntax

```
renameColor(  
name: str,  
/,  
*,  
to: str,  
display: drDisplay)  
Returns: None
```

Summary:

Rename color

Category:

Display Resources

Licenses:

None

Parameters

- name – name of color to rename
- to – new name of renamed color
- display – the own display object

Returns

None

renameLineStyle

Syntax

```
renameLineStyle(  
    name: str,  
    /,  
    *,  
    to: str,  
    display: drDisplay)  
Returns: None
```

Summary:

Rename LineStyle

Category:

Display Resources

Licenses:

None

Parameters

- name – name of linestyle to be renamed
- to – new name of renamed linestyle
- display – the own display object

Returns

None

renameStipple

Syntax

```
renameStipple(  
    name: str,  
    /,  
    *,  
    to: str,  
    display: drDisplay)  
Returns: None
```

Summary:

Rename stipple

Category:

Display Resources

Licenses:

None

Parameters

- name – name of stipple to rename
- to – new name of renamed stipple
- display – the own display object

Returns

None

saveDisplayResources

Syntax

```
saveDisplayResources(  
    *,  
    filePath: str,  
    modifiedOnly: bool | None = None,  
    tcl: bool | None = None,  
    force: bool | None = None)  
Returns: None
```

Summary:

Save display resources

Category:

Display Resources

Licenses:

None

Parameters

- `filePath` – Full or relative path of display resource file to save
- `modifiedOnly` – Save all DR data or only modifications since the last reload (Default is: False)
- `tcl` – Format of DR file to be saved (Default is: True)
- `force` – Overwrite existing file if true (Default is: False)

Returns

None

showDisplayResourceEditor

Syntax

```
showDisplayResourceEditor(  
*,  
    parent: giWindow | int = None,  
    lpp: deTechLPP | lpp = None,  
    packet: drPacket | str = None)  
Returns: giWindow
```

Summary:

Shows display resource editor

Category:

Display Resources

Licenses:

None

Parameters

- `parent` – The window that this dialog is associated.
- `lpp` – LPP to automatically select.
- `packet` – PACKET to automatically select.

Returns

The created or existing Display Resource Editor window

snps.custom.cmd.dspf

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you use DSPF files.

This section covers the following topics:

- [getLines](#)
- [patch](#)

getLines

Syntax

```
getLines(  
    dspfFilePath: str,  
    /,  
    *,  
    instPrefixes: list[str],  
    depth: int = None,  
    debug: dspf.getLines.Debug = None,  
    pdkPattern: str = None)  
Returns: str
```

Summary:

Returns lines related to specified instance prefixes in the DSPF file.

Category:

DSPF

Licenses:

ASO DSPF

Parameters

- dspfFilePath – Input DSPF filepath.
- instPrefixes – List of DSPF instance prefixes.

- depth – Extent of neighborhood.
- debug – To specify debug action. ({ printContextMap | printPrefixMap })
- pdkPattern – Additional regex pattern for matching DSPF prefixes.

Returns

Lines related to passed DSPF instance prefixes.

patch

Syntax

```
patch(  
    dspfFilePath: str,  
    /,  
    *,  
    patch: list[str],  
    outputFile: str,  
    label: str = None)  
Returns: None
```

Summary:

Apply specified patch to the passed DSPF file.

Category:

DSPF

Licenses:

ASO DSPF

Parameters

- dspfFilePath – Input DSPF filepath.
- patch – Patch contents (output of dspf::computePatch).
- outputFile – Output DSPF filepath.
- label – Label to print in output DSPF file.

Returns

None

snps.custom.cmd.gi

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform graphical infrastructure operations supporting the Custom Compiler window framework and for creating custom user interfaces.

This section covers the following topics:

- [addActions](#)
- [addAssistant](#)
- [addMenu](#)
- [addToolBar](#)
- [addWindow](#)
- [applyWorkspace](#)
- [closeFrames](#)
- [closeWindows](#)
- [collapse](#)
- [configureTable](#)
- [createAction](#)
- [createActionCategory](#)
- [createAppGrid](#)
- [createAppIcon](#)
- [createAssistant](#)
- [createBinding](#)
- [createBindingSet](#)
- [createBooleanInput](#)
- [createCarousel](#)
- [createCarouselItem](#)
- [createCheckableGroup](#)

- [createColumn](#)
- [createDataType](#)
- [createDbTable](#)
- [createDialog](#)
- [createDrag](#)
- [createDynamicList](#)
- [createFileInput](#)
- [createFrame](#)
- [createGroup](#)
- [createInlineGroup](#)
- [createLabel](#)
- [createListInput](#)
- [createMenu](#)
- [createMutexInput](#)
- [createNumberInput](#)
- [createPointListInput](#)
- [createPushButton](#)
- [createRow](#)
- [createScrollArea](#)
- [createSplitterGroup](#)
- [createTabGroup](#)
- [createTable](#)
- [createTextInput](#)
- [createTheme](#)
- [createToolbar](#)
- [createWebView](#)
- [createWindow](#)

- [createWindowType](#)
- [createWorkspace](#)
- [createX11Window](#)
- [drop](#)
- [execDialog](#)
- [executeAction](#)
- [expand](#)
- [exportTable](#)
- [findChild](#)
- [getActionCategories](#)
- [getActions](#)
- [getActiveDialog](#)
- [getActiveTab](#)
- [getActiveWindow](#)
- [getApplcons](#)
- [getAssistants](#)
- [getBindings](#)
- [getBindingSets](#)
- [getCells](#)
- [getChildren](#)
- [getColumns](#)
- [getContextMenu](#)
- [getDataTypes](#)
- [getDialogs](#)
- [getField](#)
- [getFrames](#)
- [getMenus](#)

- [getRows](#)
- [getSectionSizes](#)
- [getThemes](#)
- [getToolbars](#)
- [getWindows](#)
- [getWindowTypes](#)
- [getWorkspaces](#)
- [getXidUsage](#)
- [hideContextMenu](#)
- [layout](#)
- [loadBindings](#)
- [loadWorkspaces](#)
- [moveColumns](#)
- [moveRows](#)
- [openTerminal](#)
- [pickWindow](#)
- [pressButton](#)
- [prompt](#)
- [registerWebViewScheme](#)
- [remove](#)
- [reportWidgetState](#)
- [saveBindings](#)
- [saveWorkspaces](#)
- [scrollTo](#)
- [sendGUIEvent](#)
- [setActiveDialog](#)
- [setActiveTab](#)

- [setActiveWindow](#)
- [setContextMenu](#)
- [setCurrentIndex](#)
- [setDropAction](#)
- [setField](#)
- [setFocus](#)
- [setItemSelection](#)
- [setSectionSizes](#)
- [showBusyCursor](#)
- [showConsole](#)
- [showContextMenu](#)
- [showCustomizeWindow](#)
- [showHideAppIcons](#)
- [showKeyboardShortcuts](#)
- [showSaveHistoryAs](#)
- [sortItems](#)
- [unregisterWebViewScheme](#)

addAction

Syntax

```
addAction(  
actions: list[giAction | str],  
/,  
*,  
to: giActionCategory | giMenu | giStatusBar | giToolBar,  
after: giAction = None,  
before: giAction = None)  
Returns: giAction
```

Summary:

Add a list of actions to the given container.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- actions – list of action names to add
- to – Container in which actions will be added.
- after – Action after which actions will be added
- before – Action before which actions will be added

Returns

The action added to the specified container

addAssistant

Syntax

```
addAssistant(  
  assistant: giAssistant,  
  /,  
  *,  
  to: giWindow | giWindowType | giWorkspace,  
  dock: gi.addAssistant.Dock = None,  
  floating: bool = None,  
  after: giAssistant = None,  
  before: giAssistant = None,  
  tabbed: bool = None)  
Returns: giAssistant
```

Summary:

Adds the specified assistant to a container. An assistant object representing the assistant within the container will be returned. Destroying this object will remove the assistant from the container.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- assistant – Assistant to add.
- to – Window, window type to which to add the assistant.
- dock – In which dock area to place assistant. ({ bottom | left | right | top })
- floating – Whether the assistant is floating. (Default is: False)
- after – Assistant after which will be added.
- before – Assistant before which will be added.
- tabbed – Whether the assistant is tabbed. (Default is: True)

Returns

The assistant added to the specified window or window type

addMenu

Syntax

```
addMenu(  
menu: giMenu,  
/,  
*,  
to: giWindow | giWindowType | giWorkspace,  
after: giMenu = None,  
before: giMenu = None)  
Returns: None
```

Summary:

Add given menu to the given container.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- menu – Menu which will be added.
- to – Container to which to add the menu

- after – Menu after which will be added.
- before – Menu before which will be added.

Returns

None

addToolbar

Syntax

```
addToolbar(  
toolbar: giToolbar,  
/,  
*,  
to: giWindow | giWindowType | giWorkspace,  
dock: gi.addToolbar.Dock = None,  
after: giToolbar = None,  
before: giToolbar = None,  
newline: bool = None,  
show: bool = None)  
Returns: giToolbar
```

Summary:

Add given toolbar to the given container.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- toolbar – Toolbar which will be added.
- to – Window in which toolbar will be added.
- dock – Toolbar dock area. ({ bottom | left | right | top } (Default is: 'top'))
- after – Toolbar after which will be added.
- before – Toolbar before which will be added.
- newline – Show toolbar in new line (Default is: False)
- show – Flag show(true) or hide(false) the toolbar. (Default is: True)

Returns

The toolbar added to the window

addWindow

Syntax

```
addWindow(  
    window: giWindow | giWindowId,  
    /,  
    *,  
    to: giFrame | int = None,  
    side: gi.addWindow.Side = None,  
    layout: gi.addWindow.Layout = None,  
    before: giWindow | giWindowId = None,  
    after: giWindow | giWindowId = None)  
Returns: None
```

Summary:

Adds the specified window to a frame and removes it from its current frame

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- window – Window to add.
- to – Frame into which to add the window.
- side – Frame side to place the window ({ bottom | left | right | top })
- layout – Expected final layout of the window ({ left-right | left-right-bottom | left-top-bottom | top-bottom | top-bottom-right | top-left-right })
- before – Window within the same frame before which to place this window.
- after – Window within the same frame after which to place this window.

Returns

None

applyWorkspace

Syntax

```
applyWorkspace(  
  name: giWorkspace | str,  
  /,  
  *,  
  to: giWindow | giWindowId)  
Returns: None
```

Summary:

Applies a workspace to the specified window and any others linked in the workspace object.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Workspace object or workspace name to apply.
- to – Window to modify.

Returns

None

closeFrames

Syntax

```
closeFrames(  
  frames: list[giFrame])  
Returns: None
```

Summary:

Closes the specified frames and all contained windows.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

frames – Collection of frames to close.

Returns

None

closeWindows

Syntax

```
closeWindows(  
    windows: giDialog | list[giWindow])  
Returns: None
```

Summary:

Closes the specified windows.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

windows – Collection of windows or dialog object to close.

Returns

None

collapse

Syntax

```
collapse(  
    name: str,  
    /,  
    *,  
    in: giAssistant | giBanner | giContainer | giDialog |  
    giStatusBar | giToolbar | giWindow,  
    index: str)  
Returns: None
```

Summary:

Collapse item in dialog, assistant, window or toolbar.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Tree name within container to collapse given item
- in_ – Widget container.
- index – Index of item to collapse

Returns

None

configureTable

Syntax

```
configureTable(  
operation:  gi.configureTable.Operation,  
/,  
*,  
parent:  list[giAssistant | giDialog | giWindow],  
data:  str,  
tableName:  list[str])  
Returns: None
```

Summary:

Command to setup table configurations

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- operation – Specify the operation to log. ({ disableFilter | enableFilter | export | freezeColumn | groupByColumn | hideColumn | moveColumn | resizeColumnToContents | setColumnFilter | showColumn | sortColumn | unfreezeColumn | ungroup })
- parent – Parent widget to which table belongs.
- data – Extra data for some operations.
- tableName – Table name.

Returns

None

createAction

Syntax

```
createAction(  
name: str,  
/,  
*,  
title: str = None,  
command: script = None,  
menu: giMenu = None,  
widgetProc: list[variant] = None,  
label: str = None,  
icon: str = None,  
largeIcon: str = None,  
prompt: str = None,  
toolTip: str = None,  
autoDisable: bool = None,  
history: bool = None,  
enabled: bool = None,  
log: gi.createAction.Log = None,  
checkProc: list[variant] = None,  
defunct: bool = None,  
deprecated: bool = None,  
replacement: str = None)  
Returns: giAction
```

Summary:

Creates an action or redefines an existing Tcl action.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Unique identifier of new action.
- title – Text that appears in menus.
- command – Command to evaluate when actions are executed.
- menu – Menu this action references (for creating menu buttons and cascades).
- widgetProc – Tcl procedure which takes the action reference as an argument and creates and returns an Opal widget.
- label – Toolbar suffix text.
- icon – File path to 16x16 graphical image.
- largelcon – File path to 24x24 graphical image.
- prompt – Status bar prompt string.
- toolTip – Mouse-over bubble text.
- autoDisable – If true, disable this action in read-only editor windows. (Default is: False)
- history – If true, this action may be displayed in action history widgets. (Default is: False)
- enabled – Initial enabled state. (Default is: True)
- log – Action logging method. ({ action | command | none } (Default is: 'command'))
- checkProc – Create a checkable action and call this procedure with the giAction when toggled.
- defunct – True if the action is defunct (Default is: False)
- deprecated – True if the action is deprecated (Default is: False)
- replacement – Name of replacement action to see regarding deprecating or defunct of this action.

Returns

The new action

createActionCategory

Syntax

```
createActionCategory(  
*,  
    title: str,  
    windowType: giWindowType,  
    actions: list[giAction | str])  
Returns: giActionCategory
```

Summary:

Creates a new action category.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- title – Unique title of action category.
- windowType – Window type for which this category is valid.
- actions – Tcl list of giAction objects or names of giAction objects to include in the category

Returns

The new category

createAppGrid

Syntax

```
createAppGrid(  
_none_: str = None,  
/,  
*,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    iconSize: list[int] = None,  
    icons: dbCollection)  
Returns: giAppGrid
```

Summary:

Creates a new giAppGrid widget.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `iconSize` – Icon size for grid items
- `icons` – Specifies collection of icons to add on grid

Returns

New object.

createAppIcon**Syntax**

```
createAppIcon(  
_none_: str = None,  
/,  
*,  
enabled: bool = None,  
shown: bool = None,  
toolTip: str = None,  
label: str = None,  
category: str = None,  
product: str = None,  
icon: str = None,  
execProc: list[variant] = None,  
contextMenuProc: list[variant] = None)  
Returns: giAppIcon
```

Summary:

Creates a new giAppIcon and shows in all giAppGrids.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `label` – Label to show. (Default is: “)
- `category` – Category on which giAppIcon will be added. (Default is: “)
- `product` – Product icons to show (Default is: “)
- `icon` – Icon filename, searched within the icon search path, which will shown on button (Default is: “)
- `execProc` – Procedure to execute when clicked
- `contextMenuProc` – Procedure to execute when right-clicking on this widget to request a context menu

Returns

New object.

createAssistant

Syntax

```
createAssistant(  
name: str,  
/,  
*,  
title: str,  
createProc: list[variant],  
preShowProc: list[variant] = None,  
preHideProc: list[variant] = None,  
orientation: gi.createAssistant.Orientation = None,  
windowTypes: list[gi.WindowType | str] = None)  
Returns: giAssistant
```

Summary:

Creates an assistant definition.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Assistant name.
- title – Assistant title, i.e. “Object/Layer Panel”
- createProc – Creation procedure, called with the giAssistant object being shown.
- preShowProc – Callback called immediately before showing the assistant each time. Called with the giAssistant object being shown.
- preHideProc – Callback called immediately before hiding the assistant each time. Called with the giAssistant object being shown
- orientation – Restricts assistant orientation. ({ both | horizontal | vertical } (Default is: ‘both’)
- windowTypes – Tcl list of window types to which this assistant may be added.

Returns

Newly created assistant object

createBinding

Syntax

```
createBinding(  
*,  
event: str,  
windowType: giWindowType | giWindowTypeName = None,  
assistant: giAssistant | giAssistantName = None,  
activeOperation: str = None,  
activeObject: str = None,  
activeTool: str = None,  
activeWorkspace: str = None,  
widgetType: str = None,  
keyword: str = None,  
set: str | list[giBindingSet] = None,  
action: giAction | giActionName = None,
```

```
userCreated: bool = None,  
autoRepeat: bool = None)  
Returns: list[giBinding]
```

Summary:

Creates a new binding within given binding sets (or all existing sets if one is not specified).

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- event – GUI event to which action is bound.
- windowType – Optionally restrict this binding to the specified window type.
- assistant – Optionally restrict this binding to the specified assistant.
- activeOperation – Regular expression to match against the top command on the command stack.
- activeObject – Regular expression to match against the type of active object which is window dependent.
- activeTool – Regular expression to match against “tools” active within the current window. Although this can mean different things to different windows, it’s currently only used for DE tools in editor windows.
- activeWorkspace – Regular expression to match against “workspace” active within the current window.
- widgetType – Widget type name to which this binding applies. (Default is: “”)
- keyword – Binding keyword. (Default is: “”)
- set – One or more binding sets in which to define this binding. Defaults to the all defined binding sets.
- action – Action or action name to invoke for the specified event.
- userCreated – If true, binding will be automatically saved and loaded through xml file (Default is: False)
- autoRepeat – If true, binding can be automatically repeated (Default is: False)

Returns

Newly created bindings.

createBindingSet

Syntax

```
createBindingSet(  
    name: str,  
    /,  
    *,  
    from: giBindingSet | str = None,  
    of: giBindingSet | str = None)  
Returns: None
```

Summary:

Creates and returns a new binding set.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – The name of the binding set to create.
- of – Optionally copy an existing binding set into the new set.

Returns

None

createBooleanInput

Syntax

```
createBooleanInput(  
    _none_: str = None,  
    /,  
    *,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    prefName: dbPrefName = None,  
    tooltip: str = None,
```

```
valueChangeProc: list[variant] = None,  
label: str = None,  
value: bool = None)  
Returns: giBooleanInput
```

Summary:

Creates a new giBooleanInput widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `label` – Widget label, shown next to the check box. (Default is: “)
- `value` – Initial value. (Default is: False)

Returns

New object.

createCarousel

Syntax

```
createCarousel(  
_none_: str = None,  
/,  
*,
```

```
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
label: str = None,  
iconSize: list[int] = None)  
Returns: giCarousel
```

Summary:

Creates a new giCarousel widget.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `label` – Label to show. (Default is: “)
- `iconSize` – Icon size for carousel items

Returns

New object.

createCarouselItem

Syntax

```
createCarouselItem(  
_none_: str = None,  
/,  
*,  
enabled: bool = None,  
shown: bool = None,  
toolTip: str = None,  
label: str = None,  
icon: str = None,  
parent: giCarousel = None,  
execProc: list[variant] = None,
```



```
contextMenuProc: list[variant] = None)
Returns: giCarouselItem
```

Summary:

Creates a new giCarouselItem within a giCarousel widget.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `label` – Label to show. (Default is: “)
- `icon` – Icon filename, searched within the icon search path, which will shown on button (Default is: “)
- `parent` – Parent carousel
- `execProc` – Procedure to execute when clicked
- `contextMenuProc` – Procedure to execute when right-clicking on this widget to request a context menu

Returns

New object.

createCheckableGroup

Syntax

```
createCheckableGroup(
  _none_: str = None,
  /,
  *,
  parent: giAssistant | giContainer | giDialog | giWindow = None,
  enabled: bool = None,
  shown: bool = None,
  prefName: dbPrefName = None,
```

```
toolTip: str = None,  
label: str = None,  
value: bool = None,  
collapsible: bool = None,  
valueChangeProc: list[variant] = None)  
Returns: giCheckableGroup
```

Summary:

Creates a new giCheckableGroup container and adds it to the specified parent.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `label` – Group box label. (Default is: “)
- `value` – Initial value (Default is: False)
- `collapsible` – Render a collapsible group box instead of a checkable one. (Default is: False)
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object

Returns

New object.

createColumn

Syntax

```
createColumn(  
*,  
    parent: giTable,  
    label: str = None,  
    dataType: giDataType | str = None,  
    stretch: bool = None,  
    readOnly: bool = None,  
    shown: bool = None,  
    before: giColumn = None,  
    after: giColumn = None,  
    tooltip: str = None,  
    decorateProc: list[variant] = None)  
Returns: giColumn
```

Summary:

Adds a column to the specified table

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- parent – Table within which to add the new column
- label – Column header. Note that column headers are hidden for the table until at least one column has a non-empty label (Default is: "")
- dataType – Cell-specific data type. (Default is: 'string')
- stretch – If true, this column automatically resizes when the container is resized. Note that columns do not automatically resize when the table has user-resizable columns (Default is: False)
- readOnly – Forces all contained cells to be considered read-only (Default is: False)
- shown – Initial shown state for this column (Default is: True)
- before – Column position
- after – Column position

- tooltip – Tooltip for column header (Default is: “)
- decorateProc – Procedure to modify style attributes of displayed giCells, called with giCells as they are revealed.

Returns

Newly created column.

createDataType

Syntax

```
createDataType (  
name: str,  
/  
*,  
toDisplayProc: list[variant] = None,  
toValueProc: list[variant] = None,  
updateValueProc: list[variant] = None,  
editProc: list[variant] = None,  
compareProc: list[variant] = None)  
Returns: giDataType
```

Summary:

Creates a new data type object.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Data type name consisting of characters from the set [A-Z][a-z].
- toDisplayProc – Optional display procedure which takes a value object and returns a display string. If not specified, the value is returned as a string
- toValueProc – Optional reverse-display procedure which takes user-entered data and converts it back to a value. When updateValueProc is specified, this procedure is only called when the current value is blank (“”)
- updateValueProc – Optional reverse-display update procedure, similar to toValueProc except the current value is also passed in addition to the user-entered data

- `editProc` – Optional custom procedure which returns an edit widget for a value. If unspecified, a text input widget is assumed
- `compareProc` – Optional procedure which compares two values. Returns a negative number if the first is less than the second, 0 if they are equal, and a positive number if the first is larger than the second. If unspecified, an alphanumeric string comparison will be used.

Returns

Newly created data type object.

createDbTable

Syntax

```
createDbTable(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
label: str = None,  
dbPath: str = None,  
dbHandle: str = None,  
dbTableName: str = None,  
query: str = None,  
filterExpressions: str = None,  
filterExpressionsOp: str = None,  
sort: str = None,  
allowMoveColumns: bool = None,  
allowSortColumns: bool = None,  
alternatingRowColors: bool = None,  
showColumnFilter: bool = None,  
allowFreezeColumns: bool = None,  
allowGroupByColumns: bool = None,  
allowExportToCSV: bool = None,  
selectionModel: gi.createDbTable.SelectionModel = None,  
columnResizeMode: gi.createDbTable.ColumnResizeMode = None,  
saveColumnConfig: bool = None,  
selectionChangeProc: list[variant] = None,  
sortChangeProc: list[variant] = None,  
doubleClickProc: list[variant] = None,  
filterChangeProc: list[variant] = None,  
toolTip: str = None,  
contextMenuProc: list[variant] = None,  
**kwargs)  
Returns: giDbTable
```

Summary:

Creates a new giTable widget viewing a database

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `label` – Widget label, shown in the left column. If a name is not specified, the widget is “unlabeled” and extends into the label column (Default is: “)
- `dbPath` – Database file to populate table
- `dbHandle` – Handle for the database to populate table
- `dbTableName` – Database table name
- `query` – Override `dbTableName`, filter and sort with an explicit SQL query. Also disables `-allowSortColumns`
- `filterExpressions` – SQL filter expressions (Default is: “)
- `filterExpressionsOp` – Operator for joining SQL filter expressions (Default is: ‘AND’)
- `sort` – Table sorting specification (Default is: “)
- `allowMoveColumns` – If true, users may move columns by dragging the column headers (Default is: False)
- `allowSortColumns` – If true, clicking column headers will sort rows (Default is: False)
- `alternatingRowColors` – If true, rows will be shown with alternating background colors. Use this for large tables (Default is: False)
- `showColumnFilter` – Specifies whether column filtering is enabled. (Default is: False)
- `allowFreezeColumns` – If true, right clicking on the column header will show a context menu allowing the user to toggle the freeze/unfreeze state of each column. (Default is: True)

- `allowGroupByColumns` – Specifies whether grouping is enabled for the table. (Default is: `False`)
- `allowExportToCSV` – Specifies whether export table in `.csv` file is enabled. (Default is: `False`)
- `selectionModel` – Default selection model (`{ multipleCells | multipleRows | singleCell | singleRow }`) (Default is: `'singleRow'`)
- `columnResizeMode` – Specifies column resize mode of the table. (`{ fixed | interactive | resizeToContents | stretch }`) (Default is: `'interactive'`)
- `saveColumnConfig` – Remember column width, order, shown and frozen states (Default is: `False`)
- `selectionChangeProc` – Procedure to execute when the selection is changed. Called with the table object. Retrieve selection from attribute
- `sortChangeProc` – Procedure to execute when the sort is changed by the user clicking the header. Called with the table object. Retrieve sort spec from attribute
- `doubleClickProc` – Procedure to execute when a double click is done over a cell. Called with the cell over which a double click was detected.
- `filterChangeProc` – Procedure to execute when the filter spec is changed.
- `toolTip` – Optional widget `toolTip`. (Default is: `"`)
- `contextMenuProc` – Procedure to execute when right-clicking on this widget to request a context menu

Returns

Newly created table.

createDialog

Syntax

```
createDialog(  
_none_: str,  
/,  
*,  
enabled: bool = None,  
shown: bool = None,  
parent: giDialog | giWindow | int = None,  
title: str,  
prefScope: dbScope = None,  
styleSheet: str = None,  
showApply: bool = None,  
showHelp: bool = None,  

```

```
stayOnTop: bool = None,  
execProc: list[variant] = None,  
closeProc: list[variant] = None,  
validProc: list[variant] = None,  
defaultsProc: list[variant] = None,  
defaultsMenuProc: list[variant] = None,  
preLoadPrefsProc: list[variant] = None,  
postSavePrefsProc: list[variant] = None,  
topicId: str = None,  
**kwargs)  
Returns: giDialog
```

Summary:

Creates a new giDialog object

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Dialog name
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `parent` – Parent window or dialog
- `title` – Dialog title.
- `prefScope` – Preference scope for contained widgets with preference defaults
- `styleSheet` – Dialog style sheet (Default is: "")
- `showApply` – Show an Apply button in the dialog. Only applies if `-execProc` is also specified (Default is: True)
- `showHelp` – If set, a Help button will be shown in the button bar (Default is: True)
- `stayOnTop` – If true, stay on top of all dialogs and windows (Default is: False)
- `execProc` – If set, OK/Apply buttons may be added which trigger this method. Called with the dialog object
- `closeProc` – Optional procedure to execute when the dialog is closed (either via Cancel or after OK/Apply). Called with the dialog object

- `validProc` – Optional procedure to execute when the valid state of a child, required widget changes. Called with the dialog object only
- `defaultsProc` – Optional procedure to call when the Defaults button is pressed. Called with the dialog object only
- `defaultsMenuProc` – Optional procedure to call when an action of Defaults button's menu is pressed. Called with the dialog object and the action name
- `preLoadPrefsProc` – Optional procedure to call before loading preferences from specified file. Called with the dialog object and file path
- `postSavePrefsProc` – Optional procedure to call after saving preferences in specified file. Called with the dialog object and file path.
- `topicId` – Topic Id.

Returns

New object.

createDrag

Syntax

```
createDrag(  
*,  
    source: giAssistant | giWidget | giWindow,  
    data: dbCollection | dbIterator)  
Returns: giDrag
```

Summary:

Creates a new `giDrag` object for drag and drop

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `source` – Source widget, assistant or window.
- `data` – Objects to drag.

Returns

New object.

createDynamicList

Syntax

```
createDynamicList(  
    _none_: str = None,  
    /,  
    *,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    prefName: dbPrefName = None,  
    toolTip: str = None,  
    valueChangeProc: list[variant] = None,  
    label: str = None,  
    value: list[str] = None,  
    selection: list[int] = None,  
    editWidget: giWidget = None,  
    selectionChangeProc: list[dbCore | giDialog | str] = None,  
    readOnly: bool = None,  
    valid: bool = None,  
    required: bool = None,  
    unique: bool = None,  
    contextMenuProc: list[variant] = None)  
Returns: giDynamicList
```

Summary:

Creates a new giDynamicList widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- `toolTip` – Optional widget toolTip. (Default is: “)

- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `label` – Widget label, shown in the left column. (Default is: “)
- `value` – Initial value.
- `selection` – Initial list selection
- `editWidget` – Widget used to edit items in the list. If unset, a text input widget is assumed
- `selectionChangeProc` – Procedure to execute when the selection is changed. Called with the widget object. Retrieve selection from attribute
- `readOnly` – Initial read-only state. (Default is: False)
- `valid` – Initial valid state. (Default is: True)
- `required` – Specifies this is a required input. (Default is: False)
- `unique` – Specifies whether list should contain unique values. (Default is: False)
- `contextMenuProc` – Procedure to execute when right-clicking on this widget to request a context menu

Returns

New object.

createFileInput

Syntax

```
createFileInput(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
prefName: dbPrefName = None,  
toolTip: str = None,  
valueChangeProc: list[variant] = None,  
label: str = None,  
value: str = None,  
fileType: gi.createFileInput.FileType = None,  
fileMasks: list[str] = None,  
mode: gi.createFileInput.Mode = None,  
prompt: str = None,  
relativePath: str = None,  
defaultBrowsePath: str = None,
```

```
resolveEnvVars: bool = None,  
readOnly: bool = None,  
valid: bool = None,  
required: bool = None,  
ignoreValidation: bool = None)  
Returns: giFileInput
```

Summary:

Creates a new giFileInput widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `label` – Widget label, shown in the left column. (Default is: “)
- `value` – Initial value. (Default is: “)
- `fileType` – Specifies the requested input ({ directory | either | file } (Default is: ‘file’)
- `fileMasks` – List of file masks in the formatLabel (pattern) used for the browse dialog.
- `mode` – Initial file browsing mode ({ open | save | select } (Default is: ‘select’)
- `prompt` – Initial browse dialog prompt string. (Default is: ‘Select <label>’)
- `relativePath` – Specifies an alternate path from which to provide completions and validation. (Default is: “)

- `defaultBrowsePath` – Default path for the browse dialog in case one is not provided in the text input field. (Default is: "")
- `resolveEnvVars` – Resolve environment variables. (Default is: True)
- `readOnly` – Initial read-only state. (Default is: False)
- `valid` – Initial valid state. (Default is: True)
- `required` – Specifies this is a required input. (Default is: False)
- `ignoreValidation` – Specifies this to ignore validation. (Default is: False)

Returns

New object.

createFrame

Syntax

```
createFrame(  
*,  
    window: giWindow | giWindowId = None)  
Returns: giFrame
```

Summary:

Creates a new frame with the specified window or if one is not provided, a frame showing the Home Page.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

`window` – Window to show within the new frame.

Returns

Newly created frame object

createGroup

Syntax

```
createGroup(  
    _none_: str = None,  
    /,  
    *,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    toolTip: str = None,  
    label: str = None,  
    decorated: bool = None)  
Returns: giGroup
```

Summary:

Creates a new giGroup widget and adds it to the specified parent.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “”)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `toolTip` – Optional widget toolTip. (Default is: “”)
- `label` – Group box header. If set to the empty string, a frame is shown instead. (Default is: “”)
- `decorated` – Initial decorated state. (Default is: True)

Returns

New object.

createInlineGroup

Syntax

```
createInlineGroup(  
    _none_: str = None,  
    /,  
    *,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    toolTip: str = None,  
    label: str = None)  
Returns: giInlineGroup
```

Summary:

Creates a new giInlineGroup container and adds it to the specified parent.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “”)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `toolTip` – Optional widget toolTip. (Default is: “”)
- `label` – Widget label, shown in the left column. (Default is: “”)

Returns

New object.

createLabel

Syntax

```
createLabel(  
    _none_: str = None,  
    /,
```

```
*,
parent: giAssistant | giContainer | giDialog | giWindow = None,
enabled: bool = None,
shown: bool = None,
toolTip: str = None,
label: str = None,
icon: str = None)
Returns: giLabel
```

Summary:

Creates a new giLabel widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `label` – Label to show. (Default is: “)
- `icon` – Icon filename, searched within the icon search path, which overrides `-label` (Default is: “)

Returns

New object.

createListInput

Syntax

```
createListInput(
_none_: str = None,
/,
*,
parent: giAssistant | giContainer | giDialog | giWindow = None,
enabled: bool = None,
shown: bool = None,
```



```
prefName: dbPrefName = None,  
toolTip: str = None,  
valueChangeProc: list[variant] = None,  
label: str = None,  
value: list[str] = None,  
readOnly: bool = None,  
valid: bool = None,  
required: bool = None,  
doubleClickProc: list[variant] = None,  
header: str = None,  
filterLabel: str = None,  
items: list[str] = None,  
showFilter: bool = None,  
allowSort: bool = None,  
selectionModel: gi.createListInput.SelectionModel = None,  
viewType: gi.createListInput.ViewType = None,  
showToggleButtons: bool = None,  
contextMenuProc: list[variant] = None,  
dragProc: list[variant] = None)  
Returns: giListInput
```

Summary:

Creates a new giListInput widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `label` – Widget label, shown in the left column. (Default is: “)

- `value` – Initial value.
- `readOnly` – Initial read-only state. (Default is: `False`)
- `valid` – Initial valid state. (Default is: `True`)
- `required` – Specifies this is a required input. (Default is: `False`)
- `doubleClickProc` – Procedure to execute when double clicking on a row. Called with the `giListInput` and the double-clicked item
- `header` – Optional header title to show above contained elements. If clicked, sorts (Default is: `""`)
- `filterLabel` – Label shown to the left of the filter input. May be unset to remove the label (Default is: `'Filter:'`)
- `items` – Items shown within the list
- `showFilter` – Shows a filter field above the list (Default is: `False`)
- `allowSort` – If true, items are sorted. (Default is: `True`)
- `selectionModel` – Selection model (single or multiple) (`{ multiple | single }`) (Default is: `'single'`)
- `viewType` – View type(list, checkbox or dualList) (`{ checkbox | dualList | list }`) (Default is: `'list'`)
- `showToggleButton` – Show toggle buttons for checkable lists (Default is: `True`)
- `contextMenuProc` – Procedure to execute when right-clicking on this widget to request a context menu
- `dragProc` – Procedure to invoke when a drag starts. Called with the widget object.

Returns

New object.

createMenu

Syntax

```
createMenu(  
    name: str,  
    /,  
    *,  
    title: str,  
    preShow: str = None,  
    dynamic: bool = None,  
    save: bool = None,
```

```
shown: bool = None)
Returns: giMenu
```

Summary:

Creates a new menu.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Unique name of menu.
- title – Menu title.
- preShow – Procedure to invoke before displaying menu. (Default is: “)
- dynamic – If true, indicates that the contents of this menu change automatically. Specifically excludes it from graphical customization. (Default is: False)
- save – If true, the menu may be recreated by a Workspace object. (Default is: False)
- shown – If true, indicates that the menu should be shown (Default is: True)

Returns

The new menu

createMutexInput

Syntax

```
createMutexInput(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
prefName: dbPrefName = None,  
tooltip: str = None,  
valueChangeProc: list[variant] = None,  
label: str = None,  
enum: list[str],  
viewType: gi.createMutexInput.ViewType = None,  
value: str = None,  
radioRows: int = None,
```

```
    comboWidth: int = None,  
    dataType: giDataType | str = None)  
Returns: giMutexInput
```

Summary:

Creates a new giMutexInput widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: "")
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: "")
- `toolTip` – Optional widget toolTip. (Default is: "")
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `label` – Widget label, shown in the left column. (Default is: "")
- `enum` – Possible values.
- `viewType` – View type ({ auto | combo | radio } (Default is: 'auto'))
- `value` – Initial value.
- `radioRows` – Initial number of rows to show when multiple radio buttons are used. (Default is: 1)
- `comboWidth` – Initial minimum number of character width to show in the combo box view. (Default is: 10)
- `dataType` – Data type for performing display to value conversions (Default is: 'string')

Returns

New object.

createNumberInput

Syntax

```
createNumberInput(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
prefName: dbPrefName = None,  
toolTip: str = None,  
valueChangeProc: list[variant] = None,  
label: str = None,  
value: str = None,  
width: int = None,  
valueType: gi.createNumberInput.ValueType = None,  
viewType: gi.createNumberInput.ViewType = None,  
sliderLabels: gi.createNumberInput.SliderLabels = None,  
sliderOrientation: gi.createNumberInput.SliderOrientation = None,  
minValue: str = None,  
maxValue: str = None,  
stepValue: str = None,  
suffix: str = None,  
readOnly: bool = None,  
valid: bool = None,  
required: bool = None)  
Returns: giNumberInput
```

Summary:

Creates a new giNumberInput widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: "")
- `parent` – Dialog, group or other container into which this widget should be placed.

- enabled – Initial enabled state. (Default is: True)
- shown – Initial shown state. (Default is: True)
- prefName – Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: "")
- toolTip – Optional widget toolTip. (Default is: "")
- valueChangeProc – Procedure to execute when the value is changed. Called with the widget object
- label – Widget label, shown in the left column. (Default is: "")
- value – Initial value. (Default is: "")
- width – Maximum width, in characters, to show at a time in the widget. (Default is: 0)
- valueType – Value type (integer or floating point). ({ engineering | float | int } (Default is: 'int'))
- viewType – Number input view type (text or slider). ({ slider | text } (Default is: 'text'))
- sliderLabels – Labels to show to the left/right of the slider ({ minCurrent | minMax | none } (Default is: 'minMax'))
- sliderOrientation – Slider orientation ({ horizontal | vertical } (Default is: 'horizontal'))
- minValue – Initial minimum entered value (Default is: "")
- maxValue – Initial maximum entered value (Default is: "")
- stepValue – Initial step value (Default is: '1')
- suffix – Suffix label to show to the right of the input (ie, "ms") (Default is: "")
- readOnly – Initial read-only state. (Default is: False)
- valid – Initial valid state.
- required – Specifies this is a required input. (Default is: False)

Returns

New object.

createPointListInput

Syntax

```
createPointListInput(  
    _none_: str = None,  
    /,  
    *,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    prefName: dbPrefName = None,  
    toolTip: str = None,  
    valueChangeProc: list[variant] = None,  
    label: str = None,  
    value: str = None,  
    readOnly: bool = None,  
    required: bool = None,  
    isRectilinearPolygon: bool = None,  
    showEditorButton: bool = None)  
Returns: giPointListInput
```

Summary:

Creates a new giPointListInput widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object

- `label` – Widget label, shown in the left column. (Default is: "")
- `value` – Initial value. (Default is: "")
- `readOnly` – Initial read-only state. (Default is: False)
- `required` – Specifies this is a required input. (Default is: False)
- `isRectilinearPolygon` – Specifies whether polygon should be checked to be rectilinear. (Default is: False)
- `showEditorButton` – Specifies whether edit button should be shown. (Default is: True)

Returns

New object.

createPushButton

Syntax

```
createPushButton(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
toolTip: str = None,  
label: str = None,  
icon: str = None,  
execProc: list[variant] = None)  
Returns: giPushButton
```

Summary:

Creates a new `giPushButton` widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: "")
- `parent` – Dialog, group or other container into which this widget should be placed.

- **enabled** – Initial enabled state. (Default is: True)
- **shown** – Initial shown state. (Default is: True)
- **toolTip** – Optional widget toolTip. (Default is: “)
- **label** – Button text
- **icon** – Icon filename, searched within the icon search path
- **execProc** – Procedure to execute when the button is pressed. Called with the button widget.

Returns

New object.

createRow

Syntax

```
createRow(  
*,  
    parent: giRow | giTable,  
    label: str = None,  
    expanded: bool = None,  
    shown: bool = None,  
    before: giRow = None,  
    after: giRow = None,  
    height: int = None)  
Returns: giRow
```

Summary:

Adds a row to the specified table

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- **parent** – The new row can be added to a table or as a child of an existing row to create a tree widget
- **label** – Row header. Note that row headers are hidden for the table until at least one row has a non-empty label (Default is: “)

- expanded – Indicates expansion state for a tree structure (Default is: False)
- shown – Initial shown state (Default is: True)
- before – Row position relative to parent
- after – Row position relative to parent
- height – Height of the row in a table. (Default is: -1)

Returns

Newly created row.

createScrollArea

Syntax

```
createScrollArea(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
rowsToShow: int = None,  
showHorizontalScrollBar: gi.createScrollArea.ShowHorizontalScrollBar =  
None,  
showVerticalScrollBar: gi.createScrollArea.ShowVerticalScrollBar =  
None)  
Returns: giScrollArea
```

Summary:

Creates giScrollArea widget and adds it to specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- _none_ – Widget name. (Default is: “)
- parent – Dialog, group or other container into which this widget should be placed.
- rowsToShow – Maximal row count which giScrollArea must show without adding scrollbars. Defaults to 0 meaning scroll area will free resize in vertical direction. (Default is: 0)

- `showHorizontalScrollBar` – Show horizontal scrollbar ({ auto | false | true } (Default is: 'auto'))
- `showVerticalScrollBar` – Show vertical scrollbar ({ auto | false | true } (Default is: 'auto'))

Returns

New object.

createSplitterGroup

Syntax

```
createSplitterGroup(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
toolTip: str = None,  
orientation: gi.createSplitterGroup.Orientation = None)  
Returns: giSplitterGroup
```

Summary:

Creates a new `giSplitterGroup` widget and adds it to the specified parent.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: "")
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `toolTip` – Optional widget `toolTip`. (Default is: "")
- `orientation` – Layout orientation, horizontal or vertical ({ horizontal | vertical } (Default is: 'horizontal'))

Returns

New object.

createTabGroup

Syntax

```
createTabGroup(  
    _none_: str = None,  
    /,  
    *,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    tooltip: str = None,  
    tabRaiseProc: list[dbCore | giDialog | str] = None)  
Returns: None
```

Summary:

Creates a new giTabGroup widget and adds it to the specified parent.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `tooltip` – Optional widget tooltip. (Default is: “)
- `tabRaiseProc` – Tcl expression to evaluate when a tab is raised

Returns

None

createTable

Syntax

```
createTable(  
    _none_: str = None,  
    /,  
    *,  
    parent: giAssistant | giContainer | giDialog | giWindow = None,  
    enabled: bool = None,  
    shown: bool = None,  
    label: str = None,  
    readOnly: bool = None,  
    allowAddRows: bool = None,  
    allowMoveRows: bool = None,  
    allowMoveColumns: bool = None,  
    allowResizeColumns: bool = None,  
    allowSortColumns: bool = None,  
    allowHideColumns: bool = None,  
    allowFreezeColumns: bool = None,  
    alternatingRowColors: bool = None,  
    selectionModel: gi.createTable.SelectionModel = None,  
    showAllContent: bool = None,  
    saveColumnConfig: bool = None,  
    editTrigger: gi.createTable.EditTrigger = None,  
    selectionChangeProc: list[variant] = None,  
    valueChangeProc: list[variant] = None,  
    addRowProc: list[variant] = None,  
    doubleClickProc: list[variant] = None,  
    tooltip: str = None,  
    contextMenuProc: list[variant] = None,  
    dragProc: list[variant] = None)  
Returns: giTable
```

Summary:

Creates a new giTable widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)

- shown – Initial shown state. (Default is: True)
- label – Widget label, shown in the left column. If a name is not specified, the widget is “unlabeled” and extends into the label column (Default is: “)
- readOnly – Initial read-only state (Default is: False)
- allowAddRows – If true, Click to Add will appear in the last row of the table to allow the user to add additional rows (Default is: False)
- allowMoveRows – If true, users may move rows by dragging the row headers (Default is: False)
- allowMoveColumns – If true, users may move columns by dragging the column headers (Default is: False)
- allowResizeColumns – If true, users may resize columns by dragging the separator between column headers. Note that this disables automatic column sizing (Default is: False)
- allowSortColumns – If true, clicking column headers will sort rows (Default is: False)
- allowHideColumns – If true, right clicking on the column header will show a context menu allowing the user to toggle the show/hide state of each column. (Default is: False)
- allowFreezeColumns – If true, right clicking on the column header will show a context menu allowing the user to toggle the freeze/unfreeze state of each column. (Default is: True)
- alternatingRowColors – If true, rows will be shown with alternating background colors. Use this for large tables (Default is: False)
- selectionModel – Default selection model ({ multipleCells | multipleRows | singleCell | singleRow }) (Default is: ‘singleRow’)
- showAllContent – Show all content of table (Default is: False)
- saveColumnConfig – Remember column width, order, shown and frozen states (Default is: False)
- editTrigger – editTrigger mode. ({ doubleClick | select }) (Default is: ‘select’)
- selectionChangeProc – Procedure to execute when the selection is changed. Called with the table object. Retrieve selection from attribute
- valueChangeProc – Procedure to execute when the value is changed. Called with the cell which was modified
- addRowProc – Procedure to call when a row is added via “Click to add”. Called with the new giRow object

- `doubleClickProc` – Procedure to execute when a double click is done over a cell. Called with the cell over which a double click was detected.
- `toolTip` – Optional widget `toolTip`. (Default is: “)
- `contextMenuProc` – Procedure to execute when right-clicking on this widget to request a context menu
- `dragProc` – Procedure to invoke when a drag starts. Called with the widget object.

Returns

Newly created table.

createTextInput

Syntax

```
createTextInput(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
prefName: dbPrefName = None,  
toolTip: str = None,  
valueChangeProc: list[variant] = None,  
completions: list[str] = None,  
label: str = None,  
value: str = None,  
height: int = None,  
width: int = None,  
echo: bool = None,  
wordWrap: bool = None,  
readOnly: bool = None,  
valid: bool = None,  
required: bool = None)  
Returns: giTextInput
```

Summary:

Creates a new `giTextInput` widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “")
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `prefName` – Optional preference name within top-level container’s scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: “")
- `toolTip` – Optional widget toolTip. (Default is: “")
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `completions` – Initial list of completions.
- `label` – Widget label, shown in the left column. (Default is: “")
- `value` – Initial value. (Default is: “")
- `height` – Maximum height, in characters, to show at a time in the widget. Defaults to 1, meaning a single-line text input will be used. A value of 0 implies a vertically expanding text field (Default is: 1)
- `width` – Maximum width, in characters, to show at a time in the widget. Defaults to 0, meaning the widget will expand to fill available space horizontally (Default is: 0)
- `echo` – Specifies if asterisks will be displayed instead of entered characters. (Default is: False)
- `wordWrap` – Enables or disables word wrap in multi-line text inputs. Has no impact when height is equal to 1. (Default is: True)
- `readOnly` – Initial read-only state. (Default is: False)
- `valid` – Initial valid state. (Default is: True)
- `required` – Specifies this is a required input. (Default is: False)

Returns

New object.

createTheme

Syntax

```
createTheme(  
name: str,  
/,  
*,  
copy: giTheme = None)  
Returns: giTheme
```

Summary:

Create a new theme, optionally based on an existing one returned by `gi::getThemes`.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Theme name.
- copy – Copy colors from an existing theme.

Returns

New giTheme

createToolbar

Syntax

```
createToolbar(  
name: str,  
/,  
*,  
title: str = None,  
orientation: gi.createToolbar.Orientation = None,  
dynamic: bool = None,  
save: bool = None)  
Returns: giToolbar
```

Summary:

Creates a new toolbar.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Unique name of toolbar.
- title – Toolbar title. (Default is: "")
- orientation – Restricts toolbar orientation ({ both | horizontal | vertical } (Default is: 'both'))
- dynamic – If true, indicates that the contents of this toolbar change automatically. Specifically excludes it from graphical customization. (Default is: False)
- save – If true, the toolbar may be recreated by a Workspace object. (Default is: False)

Returns

The new toolbar

createWebView

Syntax

```
createWebView(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
label: str = None,  
enabled: bool = None,  
shown: bool = None,  
source: str,  
showNavigation: bool = None,  
showLocation: bool = None,  
newWindowProc: list[variant] = None)  
Returns: giWebView
```

Summary:

Creates a new giWebView widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `label` – Widget label, shown in the left column. (Default is: “)
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `source` – HTML File or Url.
- `showNavigation` – Specifies if navigation is shown. (Default is: False)
- `showLocation` – Specifies if location is shown. (Default is: False)
- `newWindowProc` – Procedure to execute when creating new window.

Returns

New object.

createWindow

Syntax

```
createWindow(  
*,  
    windowType: giWindowType = None,  
    title: str = None,  
    prompt: str = None,  
    showTabs: bool = None,  
    parent: giWindow = None)  
Returns: giWindow
```

Summary:

Creates a window of a window type if the specified window type was created in Tcl.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `windowType` – Window type of which to create window. If not specified, will be created a window of type specified by `giNewWindowType` preference.
- `title` – Optional window title if each window is titled differently than its window type. (Default is: “)
- `prompt` – Prompt to show in the status bar. (Default is: “)
- `showTabs` – Indicates if window may be placed in a frame with more than one window. (Default is: True)
- `parent` – Optional parent window.

Returns

Newly created window

createWindowType

Syntax

```
createWindowType (  
name: str,  
/,  
*,  
title: str,  
createProc: list[variant],  
preCloseProc: list[variant] = None,  
icon: str = None,  
appName: str = None,  
log: gi.createWindowType.Log = None)  
Returns: giWindowType
```

Summary:

Creates a window type.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `name` – Window type name.
- `title` – Window type title, i.e., “Schematic Editor”

- `createProc` – Creation procedure, called with the `giWindow` object being shown.
- `preCloseProc` – Procedure to call immediately before closing the window. Called with window argument and forced flag.
- `icon` – Optional icon found within the icon search path (Default is: “)
- `appName` – Optional application name to show within windows of this type (Default is: “)
- `log` – Logging mode (`{ auto | explicit }`) (Default is: ‘auto’)

Returns

Newly created window type

createWorkspace

Syntax

```
createWorkspace(  
    name: str,  
    /,  
    *,  
    title: str,  
    window: giWindow | int = None,  
    windows: list[giWindow | int] = None,  
    windowType: giWindowType | str = None,  
    default: bool = None,  
    includeGeometry: bool = None,  
    initProc: list[variant] = None,  
    doneProc: list[variant] = None,  
    **kwargs)  
Returns: giWorkspace
```

Summary:

Creates a new workspace object from one or more open windows.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `name` – Name, unique identifier.
- `title` – Title to show in the menu.

- `window` – Window layout to capture in the workspace.
- `windows` – One or more window objects or window ids whose configuration will be stored in the workspace.
- `windowType` – Window type for which to create workspace.
- `default` – Make the workspace default. (Default is: False)
- `includeGeometry` – Store window geometries in the workspace. (Default is: False)
- `initProc` – The `initProc` is called after the workspace layout has been applied.
- `doneProc` – The `doneProc` is called before removing the workspace layout (and applying some other workspace).

Returns

Newly created workspace object.

createX11Window

Syntax

```
createX11Window(  
*,  
    parent:  giAssistant | giContainer | giDialog | giWindow = None)  
Returns: giX11Window
```

Summary:

Creates a new `giX11Window` widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

`parent` – Dialog, group or other container into which this widget should be placed.

Returns

New `giX11Window` object.

drop

Syntax

```
drop(  
    indexes: list[str],  
    /,  
    *,  
    in_: giAssistant | giDialog | giToolbar | giWindow,  
    target: str,  
    position: point,  
    enDropAction: gi.drop.EnDropAction = None,  
    source: str = None,  
    sourceContainer: giAssistant | giDialog | giToolbar | giWindow =  
        None)  
Returns: None
```

Summary:

Drag and drop operation.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- indexes – Indexes of items to drag and drop
- in_ – Dialog, toolbar, assistant or window which contains the widget.
- target – Target widget name within container to drop items
- position – Point where the drop occurs
- enDropAction – Proposed drop action ({ copy | ignore | link | mask | move } (Default is: 'ignore')
- source – Source widget name
- sourceContainer – Dialog, toolbar, assistant or window which contains source widget

Returns

None

execDialog

Syntax

```
execDialog(  
  _none_: giDialog)  
Returns: bool
```

Summary:

Waits until the dialog is closed and returns false if cancelled, true if closed by any other button.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

none – Dialog on which to wait.

Returns

True if dialog accepted, false if cancelled

executeAction

Syntax

```
executeAction(  
  action: giAction | giActionName,  
  /,  
  *,  
  in: giAssistant | giCentralWidget | giDialog | giWindow |  
  giWindowId)  
Returns: None
```

Summary:

Executes a giAction object as if its menu or toolbar entry/button were pressed.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `action` – Action to execute.
- `in_` – Assistant, window, canvas or dialog in which to execute the action.

Returns

None

expand

Syntax

```
expand(  
    name: str,  
    /,  
    *,  
    in_: giAssistant | giBanner | giContainer | giDialog |  
        giStatusBar | giToolbar | giWindow,  
    index: str)  
Returns: None
```

Summary:

Expand item in dialog, assistant, window or toolbar.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `name` – Tree name within container to expand given item
- `in_` – Widget container.
- `index` – Index of item to expand

Returns

None

exportTable

Syntax

```
exportTable(  
  _none_: str,  
  /,  
  *,  
  parent: giAssistant | giDialog | giWindow,  
  filePath: str)  
Returns: None
```

Summary:

Exports table data

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Table name
- `parent` – Parent window or dialog
- `filePath` – File path to export table data.

Returns

None

findChild

Syntax

```
findChild(  
  none: str,  
  /,  
  *,  
  in_: giAssistant | giContainer | giDialog | giWindow)  
Returns: dbCore | str
```

Summary:

Find and returns a child widget object from a container An absolute path relative to the container may be specified by using a / for the first character. Relative paths may also be

be specified. The command returns an error if more than one match is found within the container.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- none – Widget name to find.
- in_ – Container object to search.

Returns

Object or attribute value with specified name or "" if it could not be found

getActionCategories

Syntax

```
getActionCategories(  
    expression: str = None,  
    /,  
    *,  
    windowType: giWindowType = None)  
Returns: list[giActionCategory]
```

Summary:

Returns a collection of action categories, optionally restricted to a given windowType.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- expression – A wildcard-style pattern to match action categories name against. (Default is: '*')
- windowType – Optionally restrict the returned collection to those categories for the specified window type.

Returns

Collection of matching action categories

getActions

Syntax

```
getActions(  
    expression: str = None,  
    /,  
    *,  
    from_: giStatusBar | giWindow | list[giMenu] | list[giToolbar] =  
        None,  
    of: giStatusBar | giWindow | list[giMenu] | list[giToolbar] =  
        None,  
    includeDeprecated: bool = None)  
Returns: list[giAction]
```

Summary:

Returns a collection of globally defined actions or references within a container.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- expression – A wildcard-style pattern to match actions name against. (Default is: '*')
- of – Containers from which to return actions
- includeDeprecated – Whether to include deprecated actions in the result.

Returns

A collection of actions

getActiveDialog

Syntax

```
getActiveDialog()  
Returns: giDialog
```

Arguments:

Summary:

Most recently opened dialog; "" if there isn't one.

Category:

Graphical Infrastructure

Licenses:

None

Returns

Most recently focused dialog object

getActiveTab

Syntax

```
getActiveTab(  
    name: str,  
    /,  
    *,  
    in_: giAssistant | giBanner | giContainer | giDialog |  
        giStatusBar | giToolbar | giWindow)  
Returns: str
```

Summary:

Gets the given tab widget active tab.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Tab widget name to get the active tab
- in_ – Widget container.

Returns

Active tab name

getActiveWindow

Syntax

```
getActiveWindow()  
Returns: None
```

Arguments:

Summary:

Returns the active window. Note that this is not necessarily an editor window.

Category:

Graphical Infrastructure

Licenses:

None

Returns

None

getAppIcons

Syntax

```
getAppIcons(  
    pattern: str = None)  
Returns: giAppIcon
```

Summary:

Returns all giAppIcon objects

Category:

Graphical Infrastructure

Licenses:

None

Parameters

pattern – Glob-style pattern matched against the icon name. (Default is: '*')

Returns

A collection of matching icons

getAssistants

Syntax

```
getAssistants(  
    assistant: str = None,  
    /,  
    *,  
    from_: giWindow | giWindowType | giWorkspace = None,  
    of: giWindow | giWindowType | giWorkspace = None)  
Returns: list[giAssistant]
```

Summary:

Returns a collection of assistants either defined globally or referenced by a container.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- assistant – A pattern matching assistant name.
- of – Container from which to return collection.

Returns

A collection of dockable assistants

getBindings

Syntax

```
getBindings(  
    *,  
    set: giBindingSet | str = None)  
Returns: None
```

Summary:

Returns a collection of all bindings within a binding set, optionally filtered.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

set – The binding set or the name of the binding set from which to retrieve bindings.

Returns

None

getBindingSets

Syntax

```
getBindingSets(  
    name: str | None = None  
Returns: None
```

Summary:

Returns a collection of existing binding sets, optionally filtered.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

name – Optional glob-style pattern to match against binding set names and filter the returned list.

Returns

None

getCells

Syntax

```
getCells(  
    *,  
    parent: giTable = None,  
    row: giRow = None,  
    column: giColumn = None)  
Returns: dbCollection
```


Summary:

Retrieves a collection of cell objects from a table, row and/or column object

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- parent – Table from which to retrieve cells
- row – Retrieve cells from the specified row
- column – Retrieve cells from the specified column

Returns

A collection of matching cells

getChildren

Syntax

```
getChildren(  
pattern: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow)  
Returns: dbCollection
```

Summary:

Returns collection of children widgets, found hierarchically.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- pattern – Glob-style pattern to match against widget name (Default is: '*')
- parent – Container object to search.

Returns

Collection of children widgets

getColumns

Syntax

```
getColumns (  
    *,  
    parent: giDbTable | giTable)  
Returns: dbCollection
```

Summary:

Returns all columns in the specified table

Category:

Graphical Infrastructure

Licenses:

None

Parameters

parent – Parent table

Returns

A collection of matching columns

getContextMenu

Syntax

```
getContextMenu (  
    name: str = None,  
    //,  
    *,  
    windowType: giAssistant | giWindowType)  
Returns: None
```

Summary:

Returns the context menu associated with a given context name.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Context name with which this menu is associated. For editor windows, specify the object type name. (Default is: “)
- windowType – Window type or assistant for which menu will be registered.

Returns

None

getDataTypes

Syntax

```
getDataTypes(  
    pattern: str = None)  
Returns: dbCollection
```

Summary:

Returns a collection of defined data types

Category:

Graphical Infrastructure

Licenses:

None

Parameters

pattern – Optional glob-style pattern which restricts the returned collection to those data types matching this pattern

Returns

A collection of matching data types

getDialogs

Syntax

```
getDialogs(  
    name: str = None,  
    /,
```

```
*,  
parent: giDialog | giWindow = None)  
Returns: list[giDialog]
```

Summary:

Returns a collection of open dialogs, optionally restricted by their name

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Glob-style pattern to match the dialog name against.
- parent – Dialog's parent which is specified window or dialog.

Returns

Collection of dialogs

getField

Syntax

```
getField(  
name: str,  
/,  
*,  
in_: giAssistant | giBanner | giContainer | giDialog |  
giStatusBar | giToolbar | giWindow,  
index: str = None)  
Returns: str
```

Summary:

Presses button in dialog, assistant, window or toolbar.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Widget name to get
- in_ – Widget container.
- index – Index of item to get value

Returns

Field value

getFrames

Syntax

```
getFrames(  
    index: int = None  
Returns: list[giFrame]
```

Summary:

Returns the collection of open frames

Category:

Graphical Infrastructure

Licenses:

None

Parameters

index – A pattern matching frame id.

Returns

Collection of frames

getMenus

Syntax

```
getMenus(  
    expression: str = None,  
    /,  
    *,  
    from_: dbCollection | list[giWindow] | list[giWindowType] = None,  
    of: dbCollection | list[giWindow] | list[giWindowType] = None)  
Returns: list[giMenu]
```

Summary:

Returns a collection of globally defined menu definitions or references within a container (window type or window).

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `expression` – A wildcard-style pattern to match menus name against. (Default is: `'*'`)
- `of` – Container of menus.

Returns

A collection of menus

getRows

Syntax

```
getRows (  
    *,  
    parent: giRow | giTable)  
Returns: dbCollection
```

Summary:

Returns all rows in the specified table or those which are children of a row within a tree

Category:

Graphical Infrastructure

Licenses:

None

Parameters

`parent` – Parent table or hierarchical row

Returns

A collection of matching rows

getSectionSizes

Syntax

```
getSectionSizes(  
    name: str,  
    /,  
    *,  
    in_: giAssistant | giBanner | giContainer | giDialog |  
        giStatusBar | giToolbar | giWindow)  
Returns: int
```

Summary:

Gets a splitter or column stretch information.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Widget name within container to get section sizes
- in_ – Widget container.

Returns

Section sizes

getThemes

Syntax

```
getThemes(  
    name: str = None)  
Returns: dbCollection
```

Summary:

Returns a collection of existing giTheme objects.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

name – A glob-style pattern matching the theme name.

Returns

Collection of giThemes

getToolbars

Syntax

```
getToolbars(  
    expression: str = None,  
    /,  
    *,  
    from_: dbCollection | list[giWindow] | list[giWindowType] = None,  
    of: dbCollection | list[giWindow] | list[giWindowType] = None)  
Returns: list[giToolbar]
```

Summary:

Returns a collection of globally defined toolbar definitions or references within a container (window type or window).

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- expression – A wildcard-style pattern to match toolbars name against. (Default is: “*”)
- of – Container of toolbars.

Returns

A collection of toolbars

getWindows

Syntax

```
getWindows(  
    index: giWindowId = None,  
    /,  
    *,  
    type: list[giWindowType] = None,  
    frame: giFrame | int = None)  
Returns: list[giWindow]
```

Summary:

Returns a collection of windows optionally restricted by their definition.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- index – Window id.
- type – Return only windows of specified window types.
- frame – Return only windows contained within the specified frame.

Returns

A collection of windows

getWindowTypes

Syntax

```
getWindowTypes(  
    name: str = None)  
Returns: list[giWindowType]
```

Summary:

Returns a collection of defined window types.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

name – A pattern matching window type name.

Returns

A collection of window types

getWorkspaces

Syntax

```
getWorkspaces(  
    expression: str = None,  
    /,  
    *,  
    windowType: giWindowType = None)  
Returns: dbCollection
```

Summary:

Returns a collection of defined workspaces.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- expression – A glob-style pattern matching workspace name. (Default is: '*')
- windowType – Window type filter.

Returns

Collection of workspaces

getXidUsage

Syntax

```
getXidUsage(  
    type: gi.getXidUsage.Type)  
Returns: None
```

Summary:

Retrieves xid usage statistics.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

type – resource type ({ GCs | all | dump | glyphsets | pictures | pixmaps | windows })

Returns

None

hideContextMenu**Syntax**

```
hideContextMenu()  
Returns: None
```

Arguments:

Summary:

Hides any open context menu in any window.

Category:

Graphical Infrastructure

Licenses:

None

Returns

None

layout**Syntax**

```
layout(  
    _none_: giWidget,  
    /,
```

```
*,  
align: giWidget = None,  
justify: gi.layout.Justify = None,  
equalWidth: bool = None,  
before: giWidget = None,  
after: giWidget = None,  
leftOf: giWidget = None,  
rightOf: giWidget = None)  
Returns: None
```

Summary:

Redefines a widget's position and layout attributes within its container. Widgets may be aligned with other widgets in the same container.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget or container to layout.
- `align` – Optionally horizontally-align this widget with another in the same container. This effectively creates a column.
- `justify` – Justify the widget against the position of its column. ({ center | left | right } (Default is: 'left'))
- `equalWidth` – Specifies this widget should be the same size as the widget to which it is relative. Requires `-leftOf` or `-rightOf`. (Default is: False)
- `before` – Place this widget in a new row before another widget.
- `after` – Place this widget in a new row before after widget.
- `leftOf` – Place this widget in the same row as another widget and to its left.
- `rightOf` – Place this widget in the same row as another widget and to its right.

Returns

None

loadBindings

Syntax

```
loadBindings(  
*,  
  filePath: str,  
  set: giBindingSet | str = None)  
Returns: None
```

Summary:

Loads bindings from an XML file into the specified set.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- filePath – Filename to write.
- set – Binding set into which bindings will be created.

Returns

None

loadWorkspaces

Syntax

```
loadWorkspaces(  
*,  
  filePath: str)  
Returns: dbCollection
```

Summary:

Loads a collection of workspaces from a file on disk.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

filePath – File path to save.

Returns

Collection of workspaces which were loaded.

moveColumns

Syntax

```
moveColumns (  
columns: dbCollection,  
/,  
*,  
before: giColumn = None,  
after: giColumn = None)  
Returns: None
```

Summary:

Move one or more columns within a table

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- columns – Columns to move
- before – Specifies another column in the table before which all columns will be placed
- after – Specifies another column in the table after which all columns will be placed

Returns

None

moveRows

Syntax

```
moveRows (  
rows: dbCollection,  
/,  
*,
```

```
before: giRow = None,  
after: giRow = None,  
relative: gi.moveRows.Relative = None)  
Returns: None
```

Summary:

Moves one or more rows within a table

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- rows – Rows to move
- before – Specifies another row in the table before which all rows will be placed
- after – Specifies another row in the table after which all rows will be placed
- relative – Specifies a relative offset (up, down, top or bottom) to move rows within the table ({ bottom | down | top | up })

Returns

None

openTerminal

Syntax

```
openTerminal(  
*,  
path: str = None)  
Returns: xtJob
```

Summary:

Launches terminal command in given path, or in current working directory if the path is not given. The terminal command is given by the preference giTerminalCommand, which defaults to “xterm”

Category:

Foundation

Licenses:

None

Parameters

path – The path where to open the terminal (Default is: “)

Returns

The created terminal job

pickWindow

Syntax

```
pickWindow(  
    *,  
    title: str = None,  
    prompt: str = None,  
    windows: list[giWindow] = None)  
Returns: giWindow
```

Summary:

Prompts the user to select a window.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- title – Dialog title.
- prompt – Optional prompt to show in the dialog above the window list
- windows – Specifies the list of windows to show.

Returns

Returns selected window.

pressButton

Syntax

```
pressButton(  
    name: str,  
    /,  
    *,  
    in_: giAssistant | giBanner | giContainer | giDialog |  
        giStatusBar | giToolbar | giWindow)  
Returns: None
```

Summary:

Presses button in dialog, assistant, window or toolbar.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Button name within container to press
- in_ – Widget container.

Returns

None

prompt

Syntax

```
prompt(  
    bodyText: str,  
    /,  
    *,  
    title: str,  
    buttons: list[str],  
    default: str = None,  
    cancel: str = None,  
    icon: gi.prompt.Icon = None,  
    leftButtons: list[str] = None,  
    name: str = None,  
    parent: giAssistant | giDialog | giWindow = None)  
Returns: None
```

Summary:

Prompts the user with a modal dialog which returns the pressed button when clicked.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `bodyText` – Text shown in body of dialog
- `title` – Text shown in title bar of dialog
- `buttons` – List of button names
- `default` – Name of button in `-buttons` which is equivalent to hitting return. By default the leftmost button in `-buttons` will be default button.
- `cancel` – Name of button in `-buttons` which is equivalent to hitting escape. By default the rightmost button in `-buttons` will be cancel button.
- `icon` – Icon severity. (`{ error | information | question | warning }`) (Default is: 'information')
- `leftButtons` – List of left-aligned buttons
- `name` – Name for this dialog. By default, the name is `giPrompt`, but a unique suffix may be appended if more than one instance of this dialog exists at a time. (Default is: "")
- `parent` – Specifies the prompt dialog's parent dialog, window or assistant. This causes the prompt to appear centered over its parent.

Returns

None

registerWebViewScheme

Syntax

```
registerWebViewScheme(  
    name: str,  
    /,  
    *,  
    loadProc: list[variant])  
Returns: None
```

Summary:

Creates and registers a new scheme for WebView.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Scheme name
- loadProc – Procedure to invoke when a URL is clicked

Returns

None

remove**Syntax**

```
remove (  
  item: giAction | giAssistant | giMenu | giToolbar)  
Returns: None
```

Summary:

Removes an action, menu, toolbar or assistant reference from its container previously added with one of the gi::add* commands.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

item – Item which will be removed.

Returns

None

reportWidgetState

Syntax

```
reportWidgetState(  
    widget: giAssistant | giDialog | giMenu | giStatusBar |  
           giToolBar | giWindow,  
    /,  
    *,  
    filePath: str = None)  
Returns: str
```

Summary:

Returns an string representation of the current state of the indicated widget and its children.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- widget – Widget of which to report state.
- filePath – File path where to save report file. (Default is: "")

Returns

XML string representing widget state

saveBindings

Syntax

```
saveBindings(  
    bindings: giBindingSet | list[giBinding] = None,  
    /,  
    *,  
    filePath: str)  
Returns: None
```

Summary:

Saves a collection of bindings to an XML file. By default, all bindings in the active binding set are assumed. If the file already exists, it will be overwritten.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- bindings – Bindings to save.
- filePath – Filename to write.

Returns

None

saveWorkspaces

Syntax

```
saveWorkspaces(  
    workspaces: dbCollection,  
    /,  
    *,  
    filePath: str,  
    force: bool = None)  
Returns: None
```

Summary:

Saves a collection of workspaces to a file on disk.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- workspaces – Collection of workspaces.
- filePath – File path to save.
- force – Overwrite existing files. (Default is: False)

Returns

None

scrollTo

Syntax

```
scrollTo(  
_target_: giCell | giColumn | giRow)  
Returns: None
```

Summary:

Cell, row or column to scroll to within the table

Category:

Graphical Infrastructure

Licenses:

None

Parameters

`_target_` – Target cell, row or column

Returns

None

sendGUIEvent

Syntax

```
sendGUIEvent(  
event: str,  
/,  
*,  
to: deCanvas | giAssistant | giWindow)  
Returns: None
```

Summary:

Activate the specified window (or assistant's window) and send a GUI event as if generated by user input.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- event – GUI event to send.
- to – Window, assistant or canvas to receive the event.

Returns

None

setActiveDialog

Syntax

```
setActiveDialog(  
    dialog: giDialog)  
Returns: None
```

Summary:

Activates given dialog;

Category:

Graphical Infrastructure

Licenses:

None

Parameters

dialog – Dialog to set as the active dialog

Returns

None

setActiveTab

Syntax

```
setActiveTab(  
    name: str,  
    /,  
    *,  
    in_: giAssistant | giBanner | giContainer | giDialog |  
        giStatusBar | giToolbar | giWindow,  
    tabName: str)  
Returns: None
```

Summary:

Sets active widget in the given tab widget.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Tab widget name within container to set active tab
- in_ – Widget container.
- tabName – Tab name of tab widget to set active

Returns

None

setActiveWindow**Syntax**

```
setActiveWindow(  
window: giWindow | giWindowId,  
/  
*,  
raise_: bool = None,  
force: bool = None)  
Returns: None
```

Summary:

Sets the active window and optionally raises it.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `window` – `Window(id)` to set active.
- `raise_` – Raise the window as well. (Default is: `False`)
- `force` – Forces external window manager to activate this window. (Default is: `False`)

Returns

None

setContextMenu

Syntax

```
setContextMenu(  
name: str = None,  
/,  
*,  
windowType: giAssistant | giWindowType,  
menu: giMenu | str)  
Returns: None
```

Summary:

Associates a menu with a context state name.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `name` – Context name with which to associate the given menu. (Default is: `""`)
- `windowType` – Window type or assistant for which menu will be registered.
- `menu` – Menu to register. May be empty to unregister the menu definition.

Returns

None

setCurrentIndex

Syntax

```
setCurrentIndex(  
name: str,  
/,  
*,  
in_: giAssistant | giBanner | giContainer | giDialog |  
giStatusBar | giToolbar | giWindow,  
index: str)  
Returns: None
```

Summary:

Sets the currentIndex in a table/tree widget.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Name of a table/tree widget in the specified dialog, toolbar, assistant or window.
- in_ – Widget container.
- index – Item index in the table widget to make current.

Returns

None

setDropAction

Syntax

```
setDropAction(  
_target_: giWidget,  
/,  
*,  
type: str,  
dropProc: list[variant],  
dragEnterProc: list[variant] = None,  
dragLeaveProc: list[variant] = None)  
Returns: giDrag
```

Summary:

Specifies a TCL procedure to invoke when a drop occurs over -target which contains data type -type.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_target_` – Target widget.
- `type` – Supported data type.
- `dropProc` – Procedure to invoke when drop occurs. Called with the `giDrag` object.
- `dragEnterProc` – Procedure to invoke when the cursor enters the target widget. Called with the `giDrag` object. Return true to accept the drag, false to reject.
- `dragLeaveProc` – Procedure to invoke when the cursor leaves the target widget. Called with the `giDrag` object.

Returns

New object.

setField

Syntax

```
setField(  
name: str,  
/,  
*,  
in_: giAssistant | giBanner | giContainer | giDialog |  
giStatusBar | giToolbar | giWindow,  
index: str = None,  
value: str)  
Returns: None
```

Summary:

Presses button in dialog, assistant, window or toolbar.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Field name to set
- in_ – Widget container.
- index – Index of item to set value
- value – Value to set the widget to

Returns

None

setFocus

Syntax

```
setFocus(  
_target_: giWidget)  
Returns: None
```

Summary:

Sets focus on passed giWidget

Category:

Graphical Infrastructure

Licenses:

None

Parameters

target – Target widget.

Returns

None

setItemSelection

Syntax

```
setItemSelection(  
name: str,
```

```
/,
*,
in_: giAssistant | giBanner | giContainer | giDialog |
giStatusBar | giToolbar | giWindow,
index: list[str])
Returns: None
```

Summary:

Set Selection in dialog, assistant, window or toolbar.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Widget name within container to set selection given items.
- in_ – Widget container.
- index – Index of item to set selection.

Returns

None

setSectionSizes

Syntax

```
setSectionSizes(
name: str,
/,
*,
in_: giAssistant | giBanner | giContainer | giDialog |
giStatusBar | giToolbar | giWindow,
values: list[int])
Returns: None
```

Summary:

Replays a splitter or column stretch operation.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Widget name within container to set section sizes
- in_ – Widget container.
- values – List of section sizes

Returns

None

showBusyCursor

Syntax

```
showBusyCursor()  
Returns: None
```

Arguments:

Summary:

Changes the cursor graphic to a busy cursor.

Category:

Graphical Infrastructure

Licenses:

None

Returns

None

showConsole

Syntax

```
showConsole()  
Returns: None
```

Arguments:

Summary:

Shows and activates console window.

Category:

Foundation

Licenses:

None

Returns

None

showContextMenu

Syntax

```
showContextMenu(  
_none_: giAssistant | giWidget | giWindow)  
Returns: giMenu
```

Summary:

Shows the Context Menu for the specified window or widget

Category:

Graphical Infrastructure

Licenses:

None

Parameters

none – Window, assistant or widget whose context menu to show.

Returns

A menu which was shown

showCustomizeWindow

Syntax

```
showCustomizeWindow(  
window: giWindow | giWindowId = None)  
Returns: None
```

Summary:

Shows the Customize Window dialog for the specified window.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

window – Window to customize.

Returns

None

showHideAppIcons

Syntax

```
showHideAppIcons(  
*,  
parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Displays Show/Hide Icons dialog

Category:

Graphical Infrastructure

Licenses:

None

Parameters

parent – The window that this dialog is associated.

Returns

The dialog

showKeyboardShortcuts

Syntax

```
showKeyboardShortcuts (  
    *,  
    parent: giWindow | giWindowId = None)  
Returns: giDialog
```

Summary:

Shows the Keyboard Shortcuts dialog.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

parent – Window to modify.

Returns

Keyboard Shortcuts dialog.

showSaveHistoryAs

Syntax

```
showSaveHistoryAs ()  
Returns: None
```

Arguments:

Summary:

The dialog.

Category:

Graphical Infrastructure

Licenses:

None

Returns

None

sortItems

Syntax

```
sortItems(  
name: str,  
/,  
*,  
in_: giAssistant | giBanner | giContainer | giDialog |  
giStatusBar | giToolBar | giWindow,  
column: str,  
order: gi.sortItems.Order)  
Returns: None
```

Summary:

Sort Table or Tree column items in dialog, assistant, window or toolbar.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- name – Widget name which column should be sorted.
- in_ – Widget container.
- column – Column which items should be sorted.
- order – Sort Order. ({ ascending | descending })

Returns

None

unregisterWebViewScheme

Syntax

```
unregisterWebViewScheme (  
    name: str)  
Returns: None
```

Summary:

Unregisters a scheme for WebView.

Category:

Graphical Infrastructure

Licenses:

None

Parameters

name – Scheme name

Returns

None

snps.custom.cmd.he

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform editing of hierarchy configuration (config) views.

This section covers the following topics:

- [addRule](#)
- [addViewGroup](#)
- [clone](#)
- [closeConfig](#)
- [closePropDB](#)
- [computeId](#)
- [createConfig](#)

- [createConfigTemplate](#)
- [createContext](#)
- [createLogicalSelectionEngine](#)
- [createNetInput](#)
- [createOverlayConfig](#)
- [createOverlayView](#)
- [createRule](#)
- [createViewGroup](#)
- [deleteRule](#)
- [deleteViewGroup](#)
- [expandViewList](#)
- [findPropDB](#)
- [findSwitchedMaster](#)
- [foreach](#)
- [getConfigTemplates](#)
- [getConfigView](#)
- [getExternalFileName](#)
- [getInstHeaders](#)
- [getInsts](#)
- [getProps](#)
- [getPropValue](#)
- [getRules](#)
- [getSelected](#)
- [getViewGroups](#)
- [isExternalFile](#)
- [isLeaf](#)
- [listPropDBs](#)

- [openConfig](#)
- [openPropDB](#)
- [popContext](#)
- [pushContext](#)
- [queryViewSelection](#)
- [saveAsPropDB](#)
- [saveConfig](#)
- [savePropDB](#)
- [setPropValue](#)
- [setRuleFromDE](#)
- [showExportToFile](#)
- [showHierarchyEditor](#)
- [showObject](#)
- [undoRedoHistory](#)

addRule

Syntax

```
addRule(  
    rule: heRule,  
    /,  
    *,  
    config: heConfig)  
Returns: None
```

Summary:

Add the rule to the config

Category:

Hierarchy Editing

Licenses:

None

Parameters

- rule – The rule to be added
- config – The config object modified

Returns

None

addViewGroup

Syntax

```
addViewGroup(  
viewGroup: heViewNameGroup,  
/,  
*,  
config: heConfig)  
Returns: None
```

Summary:

Add a new view group

Category:

Hierarchy Editing

Licenses:

None

Parameters

- viewGroup – View name group
- config – Config object to modify.

Returns

None

clone

Syntax

```
clone(  
context: heContext)  
Returns: heContext
```

Summary:

Clones an existing hierarchy context.

Category:

Hierarchy Editing

Licenses:

None

Parameters

context – Context object to clone.

Returns

Newly created context.

closeConfig

Syntax

```
closeConfig(  
*,  
    config: heConfig)  
Returns: None
```

Summary:

Close a config

Category:

Hierarchy Editing

Licenses:

None

Parameters

config – The config object modified

Returns

None

closePropDB

Syntax

```
closePropDB(  
    hePropDB: hePropDB)  
Returns: None
```

Summary:

Closes the specified hePropDB.

Category:

Hierarchy Editing

Licenses:

None

Parameters

hePropDB – The database to close.

Returns

None

computeId

Syntax

```
computeId(  
    *,  
    context: heContext = None,  
    instHeader: heInstHeader = None)  
Returns: str
```

Summary:

Calculates a unique string for given heContext/heInstanceHeader

Category:

Hierarchy Editing

Licenses:

None

Parameters

- context – The queried heContext. You can only set <-context> or <-instHeader>, not both.
- instHeader – The queried heInstHeader. You can only set <-context> or <-instHeader>, not both.

Returns

The unique identifier of the passed in object

createConfig

Syntax

```
createConfig(  
*,  
  topCellView: dmCellView,  
  viewSearchList: list[str] = None,  
  viewStopList: list[str] = None,  
  bindProp: str = None,  
  cellView: dmCellView = None,  
  libSearchList: list[str] = None,  
  libOpenList: list[str] = None,  
  cellOpenList: list[dmCell | str] = None,  
  overlayViewName: str = None,  
  templateName: str = None)  
Returns: heConfig
```

Summary:

Create a new config

Category:

Hierarchy Editing

Licenses:

None

Parameters

- topCellView – Top Cell View
- viewSearchList – View Search List
- viewStopList – View Stop List
- bindProp – Property Binding

- cellView – Cell View
- libSearchList – lib search list
- libOpenList – lib open list
- cellOpenList – cell open list
- overlayViewName – Default Overlay View Name
- templateName – Config Template Name

Returns

New config object

createConfigTemplate

Syntax

```
createConfigTemplate(  
    name: str,  
    /,  
    *,  
    filePath: str)  
Returns: heConfigTemplate
```

Summary:

Create a new config template

Category:

Hierarchy Editing

Licenses:

None

Parameters

- name – Name
- filePath – File Path

Returns

The created heConfigTemplate

createContext

Syntax

```
createContext(  
*,  
    cellView: dmCellView | dmCellViewName,  
    viewSearchList: list[str] = None,  
    viewStopList: list[str] = None)  
Returns: heContext
```

Summary:

Create a new context

Category:

Hierarchy Editing

Licenses:

None

Parameters

- cellView – Top Cell View
- viewSearchList – View Search List
- viewStopList – Stop List

Returns

New context object

createLogicalSelectionEngine

Syntax

```
createLogicalSelectionEngine(  
*,  
    filterProc: list[variant] = None,  
    selectionChangeProc: list[variant] = None,  
    types: list[str] = None,  
    hier: bool = None)  
Returns: heLogicalSelectionEngine
```

Summary:

Creates a logical selection engine for use in an interactive command.

Category:

Foundation

Licenses:

None

Parameters

- `filterProc` – Procedure called for each selectable object. Return false to prevent activation.
- `selectionChangeProc` – Procedure called when the logical selection changes, even externally.
- `types` – List of logical types to select. ({ `Inst` | `InstTerm` | `Net` | `Term` | `Pin` | `FigGroup` })
- `hier` – If true, select logical objects at any level of hierarchy. If false, select only 0x logical objects. (Default is: False)

Returns

The newly created logical selection engine object.

createNetInput

Syntax

```
createNetInput(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
toolTip: str = None,  
valueChangeProc: list[variant] = None,  
label: str = None,  
hier: bool = None,  
requireExisting: bool = None,  
logicalTypes: he.createNetInput.LogicalTypes = None,  
context: deContext,  
allowSelectionSync: bool = None,  
textChangeProc: list[variant] = None,  
allowPattern: bool = None,  
readOnly: bool = None,  
valid: bool = None,  
required: bool = None)  
Returns: heNetInput
```

Summary:

Creates a new heNetInput widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “”)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `toolTip` – Optional widget toolTip. (Default is: “”)
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the widget object
- `label` – Widget label, shown in the left column. (Default is: “”)
- `hier` – Support selection of hierarchical nets above and below the current design (Default is: False)
- `requireExisting` – Automatically invalidate entered names which do not exist. (Default is: True)
- `logicalTypes` – Ordered list of object types to select and synchronize with hierarchical selection. ({ inst | instTerm | net | netGroup | pin | term })
- `context` – Hierarchical context in which to select nets
- `allowSelectionSync` – Automatically synchronize objects with logical selection (Default is: True)
- `textChangeProc` – Procedure to execute when the widget text is changed. Called with the widget object
- `allowPattern` – If true, the widget will follow `giFilterPatternStyle` preference. (Default is: False)
- `readOnly` – Initial read-only state. (Default is: False)

- valid – Initial valid state. (Default is: True)
- required – Specifies this is a required input. (Default is: False)

Returns

New heNetInput object.

createOverlayConfig

Syntax

```
createOverlayConfig(  
src: dmCellView | dmCellViewName,  
/  
*,  
name: str = None,  
dest: dmCellView | dmCellViewName = None,  
template: str,  
overwrite: bool = None)  
Returns: dmCellView
```

Summary:

Creates an overlay config.

Category:

Hierarchy Editing

Licenses:

Frontend

Parameters

- src – The cellview to use as source for config's top.
- name – Overlay config's name to create
- dest – The destination overlay config to create
- template – Template to determine viewSearchList and viewStopList.
- overwrite – Overwrite the target if exists. (Default is: False)

Returns

New Overlay Config.

createOverlayView

Syntax

```
createOverlayView(  
name: str = None,  
/,  
*,  
src: dmCellView | dmCellViewName = None,  
overwrite: bool = None,  
context: heContext)  
Returns: dmCellView
```

Summary:

Create an Overlay View

Category:

Hierarchy Editing

Licenses:

None

Parameters

- name – The overlay view name. If not specified the occurrence path of the context is used. (Default is: “)
- src – The cellview to copy to create an overlay view. If not specified the context’s cellview is copied.
- overwrite – Overwrite the existing Design. (Default is: False)
- context – The overlay config to create the overlay veiw in.

Returns

New Overlay Cell View.

createRule

Syntax

```
createRule(  
*,  
viewSearchList: list[str] = None,  
selectView: str = None,  
selectCell: str = None,  
selectLib: str = None,  
open: bool = None,
```

```
stop: bool = None,  
cell: dmCell = None,  
cellView: dmCellView = None,  
instName: str = None,  
occurrence: str = None)  
Returns: heRule
```

Summary:

Create a new rule

Category:

Hierarchy Editing

Licenses:

None

Parameters

- viewSearchList – View Search List. You can only set one of the following arguments at a time: <-viewSearchList>, <-selectView>, <-selectCell>, <-selectLib>, <-open>, or <-stop>.
- selectView – Select View Name. You can only set one of the following arguments at a time: <-viewSearchList>, <-selectView>, <-selectCell>, <-selectLib>, <-open>, or <-stop>.
- selectCell – Select Cell Name. You can only set one of the following arguments at a time: <-viewSearchList>, <-selectView>, <-selectCell>, <-selectLib>, <-open>, or <-stop>.
- selectLib – Select Lib Name. You can only set one of the following arguments at a time: <-viewSearchList>, <-selectView>, <-selectCell>, <-selectLib>, <-open>, or <-stop>.
- open – Bind to Open Flag. You can only set one of the following arguments at a time: <-viewSearchList>, <-selectView>, <-selectCell>, <-selectLib>, <-open>, or <-stop>.
- stop – Stop Flag. You can only set one of the following arguments at a time: <-viewSearchList>, <-selectView>, <-selectCell>, <-selectLib>, <-open>, or <-stop>.
- cell – Cell used to specify cell level. You can only set one of the following arguments/combination of arguments at a time: <-occurrence>, <-cell>, or <-cellView> and <-instName>.
- cellView – Cell View used to specify instance level, must be used together with <-instName>. You can only set one of the following arguments/combination of arguments at a time: <-occurrence>, <-cell>, or <-cellView> and <-instName>.

- `instName` – Instance Name, must be used together with `<-cellView>`. You can only set one of the following arguments/combination of arguments at a time: `<-occurrence>`, `<-cell>`, or `<-cellView>` and `<-instName>`. (Default is: `""`)
- `occurrence` – Occurrence Path. You can only set one of the following arguments/combination of arguments at a time: `<-occurrence>`, `<-cell>`, or `<-cellView>` and `<-instName>`. (Default is: `""`)

Returns

The created rule

createViewGroup

Syntax

```
createViewGroup(  
  name:  str,  
  /,  
  *,  
  views: list[str])  
Returns: heViewNameGroup
```

Summary:

Create a new view group

Category:

Hierarchy Editing

Licenses:

None

Parameters

- `name` – view group name
- `views` – associated views

Returns

The newly created view name group object

deleteRule

Syntax

```
deleteRule(  
  rule:  heRule,
```

```
//  
*,  
config: heConfig)  
Returns: None
```

Summary:

Delete the rule from the config

Category:

Hierarchy Editing

Licenses:

None

Parameters

- rule – The rule to be deleted
- config – The config object to be modified

Returns

None

deleteViewGroup

Syntax

```
deleteViewGroup(  
viewGroup: heViewNameGroup,  
//  
*,  
config: heConfig)  
Returns: None
```

Summary:

Create a delete view group

Category:

Hierarchy Editing

Licenses:

None

Parameters

- viewGroup – The view name group to be added to group
- config – heConfig to remove the named view name group from

Returns

None

expandViewList

Syntax

```
expandViewList(  
*,  
    config: heConfig,  
    views: list[str])  
Returns: str
```

Summary:

Expand view list

Category:

Hierarchy Editing

Licenses:

None

Parameters

- config – The heConfig used to evaluate the view name groups
- views – The list of view names to expand

Returns

The fully expanded list of views

findPropDB

Syntax

```
findPropDB(  
name: str,  
/,  
*,  
    topCellView: dmCellView | dmCellViewName)  
Returns: None
```

Summary:

Finds the specified hePropDB in memory.

Category:

Hierarchy Editing

Licenses:

None

Parameters

- name – The name of database to search for.
- topCellView – The cellview to search.

Returns

None

findSwitchedMaster**Syntax**

```
findSwitchedMaster(  
*,  
    context: heContext = None,  
    inst: oaInst = None,  
    instHeader: heInstHeader = None)  
Returns: heContext
```

Summary:

Finds the switched master of an instance

Category:

Hierarchy Editing

Licenses:

None

Parameters

- context – Parent Context, must be used together with <-inst>. You can only set one of the following arguments/combination of arguments at a time: <-instHeader>, or <-context> and <-inst>.
- inst – Instance, must be used together with <-context>. You can only set one of the following arguments/combination of arguments at a time: <-instHeader>, or <-context> and <-inst>.
- instHeader – Instance Header. You can only set one of the following arguments/combination of arguments at a time: <-instHeader>, or <-context> and <-inst>.

Returns

The found context object

foreach

Syntax

```
foreach(  
  variable: str,  
  context: dmCellView | heContext,  
  script: script,  
  skipIgnoredInstances: bool = None)  
Returns: None
```

Summary:

foreach loop for hierarchy

Category:

Hierarchy Editing

Licenses:

None

Parameters

- variable – Context loop variable name.
- context – CellView or context to iterate on.
- script – TCL script to execute.

Returns

None

getConfigTemplates

Syntax

```
getConfigTemplates(  
    pattern: str)  
Returns: heConfigTemplate
```

Summary:

Get config templates

Category:

Hierarchy Editing

Licenses:

None

Parameters

pattern – A pattern to use for matching template names.

Returns

Matching configuration templates

getConfigView

Syntax

```
getConfigView(  
    *,  
    window: giWindow | int)  
Returns: heConfig
```

Summary:

Get the associated config view object

Category:

Hierarchy Editing

Licenses:

None

Parameters

window – Hierarchy Editor window

Returns

The config object

getExternalFileName

Syntax

```
getExternalFileName(  
    context: heContext,  
    /*,  
    *,  
    nameType: bool = None)  
Returns: None
```

Summary:

Returns the external file name of the current cell view of the hierarchy context. If the cell view is not linked to an external file, returns empty string.

Category:

Hierarchy Editing

Licenses:

None

Parameters

- context – The heContext to get the current cell view
- nameType – The option to get the file name with absolute path or relative path. If it is set to true, and it will return relative path. (Default is: True)

Returns

None

getInstHeaders

Syntax

```
getInstHeaders(  
    /*,  
    *,  
    context: heContext)  
Returns: list[heInstHeader]
```

Summary:

Return the collection of inst headers

Category:

Hierarchy Editing

Licenses:

None

Parameters

context – The queried context to determine child context

Returns

A collection of child <heInstHeader> objects

getInsts

Syntax

```
getInsts(  
*,  
    instHeader: heInstHeader,  
    singleBit: bool = None)  
Returns: list[oaInst]
```

Summary:

Return the instances bound to the given heInstHeader

Category:

Hierarchy Editing

Licenses:

None

Parameters

- instHeader – The queried instance header
- singleBit – Flag to indicate if should iterate over all singleBit instances. (Default is: False)

Returns

Collection of instances associated with the passed master

getProps

Syntax

```
getProps(  
  name: str = None,  
  /,  
  *,  
  db: hePropDB,  
  context: heContext = None,  
  inst: oaInst | str = None,  
  scope: he.getProps.Scope = None)  
Returns: dbCollection
```

Summary:

Returns the collection of properties for a given instance/context/scope.

Category:

Hierarchy Editing

Licenses:

None

Parameters

- name – Filters returned properties by name.
- db – The hierarchical property database.
- context – The occurrence.
- inst – The instance to check within the specified occurrence.
- scope – The scope to inspect. Defaults to all scopes. ({ cell | instance | occurrence })

Returns

The collection of heProps that matches the specified query

getPropValue

Syntax

```
getPropValue(  
  name: str,  
  /,  
  *,  
  db: hePropDB,  
  context: heContext,
```

```
inst: oaInst | str = None)
Returns: str
```

Summary:

Returns the effective value of the named property.

Category:

Hierarchy Editing

Licenses:

None

Parameters

- name – The name of the queried property.
- db – The hePropDB to search.
- context – The occurrence.
- inst – If specified the instance to check inside the specified -context

Returns

The collection of hePropsthat matches the specified query.

getRules

Syntax

```
getRules(
config: heConfig,
/,
*,
viewSearchList: list[str] = None,
selectView: str = None,
selectCell: str = None,
selectLib: str = None,
open: bool = None,
stop: bool = None,
cell: dmCell = None,
cellView: dmCellView = None,
instName: str = None,
occurrence: str = None)
Returns: list[heRule]
```

Summary:

Create a new rule

Category:

Hierarchy Editing

Licenses:

None

Parameters

- **config** – Config
- **viewSearchList** – View Search List. You can only set one of the following arguments at a time: `<-viewSearchList>`, `<-selectView>`, `<-selectCell>`, `<-selectLib>`, `<-open>`, or `<-stop>`.
- **selectView** – Select View Name. You can only set one of the following arguments at a time: `<-viewSearchList>`, `<-selectView>`, `<-selectCell>`, `<-selectLib>`, `<-open>`, or `<-stop>`.
- **selectCell** – Select Cell Name. You can only set one of the following arguments at a time: `<-viewSearchList>`, `<-selectView>`, `<-selectCell>`, `<-selectLib>`, `<-open>`, or `<-stop>`.
- **selectLib** – Select Lib Name. You can only set one of the following arguments at a time: `<-viewSearchList>`, `<-selectView>`, `<-selectCell>`, `<-selectLib>`, `<-open>`, or `<-stop>`.
- **open** – Bind to Open Flag. You can only set one of the following arguments at a time: `<-viewSearchList>`, `<-selectView>`, `<-selectCell>`, `<-selectLib>`, `<-open>`, or `<-stop>`.
- **stop** – Stop Flag. You can only set one of the following arguments at a time: `<-viewSearchList>`, `<-selectView>`, `<-selectCell>`, `<-selectLib>`, `<-open>`, or `<-stop>`.
- **cell** – Cell used to specify cell level. You can only set one of the following arguments/combination of arguments at a time: `<-occurrence>`, `<-cell>`, or `<-cellView>` and `<-instName>`.
- **cellView** – Cell View used to specify instance level, must be used together with `<-instName>`. You can only set one of the following arguments/combination of arguments at a time: `<-occurrence>`, `<-cell>`, or `<-cellView>` and `<-instName>`.
- **instName** – Instance Name, must be used together with `<-cellView>`. You can only set one of the following arguments/combination of arguments at a time: `<-occurrence>`, `<-cell>`, or `<-cellView>` and `<-instName>`. (Default is: “”)
- **occurrence** – Occurrence Path. You can only set one of the following arguments/combination of arguments at a time: `<-occurrence>`, `<-cell>`, or `<-cellView>` and `<-instName>`. (Default is: “”)

Returns

The found rules

getSelected

Syntax

```
getSelected(  
    *,  
    window: giWindow | int)  
Returns: list[heContext]
```

Summary:

Returns the list of context objects currently selected in the specified Hierarchy Editor window

Category:

Hierarchy Editing

Licenses:

None

Parameters

window – Hierarchy Editor window

Returns

A collection <heContext> of selected item(s)

getViewGroups

Syntax

```
getViewGroups(  
    filter: str = None,  
    /,  
    *,  
    config: heConfig)  
Returns: list[heViewNameGroup]
```

Summary:

Create a get view groups

Category:

Hierarchy Editing

Licenses:

None

Parameters

- filter – A glob style filter for the returned collection (Default is: “")
- config – The configuration queried.

Returns

Collection of <heViewNameGroup> objects matching the filter

isExternalFile**Syntax**

```
isExternalFile(  
    context: heContext)  
Returns: bool
```

Summary:

Returns true if the current cell view of the hierarchy context is linked to an external file.

Category:

Hierarchy Editing

Licenses:

None

Parameters

context – The heContext to get the current cell view

Returns

If the object/context is a leaf

isLeaf**Syntax**

```
isLeaf(  
    inst: oaInst = None,
```

```
//  
*,  
context: heContext)  
Returns: bool
```

Summary:

Determines if an instance or heContext is a leaf

Category:

Hierarchy Editing

Licenses:

None

Parameters

- inst – The instance checked
- context – The heContext queried to determine if the passed in object or heContext is a leaf

Returns

If the object/context is a leaf

listPropDBs

Syntax

```
listPropDBs(  
name: str = None,  
//  
*,  
topCellView: dmCellView | dmCellViewName)  
Returns: dbCollection
```

Summary:

Lists the names of hePropDb's stored in a specified oaCellView.

Category:

Hierarchy Editing

Licenses:

None

Parameters

- **name** – If not specified all names are returned. If specified the list is filtered by the given glob style expression (Default is: “)
- **topCellView** – The top cellview to search for the named hePropDB.

Returns

The list of HDB names stored in the specified cellview

openConfig

Syntax

```
openConfig(  
*,  
    cellView: dmCellView = None,  
    pathName: str = None,  
    readOnly: bool = None)  
Returns: heConfig
```

Summary:

Open a config either an oaCellView or a file on disk

Category:

Hierarchy Editing

Licenses:

None

Parameters

- **cellView** – The dmCellView to open as a config. You can only set <-cellView> or <-pathName>, not both.
- **pathName** – The path of the file. You can only set <-cellView> or <-pathName>, not both.
- **readOnly** – The access mode used to open. If true, the config will be opened read-only. (Default is: False)

Returns

The opened config object

openPropDB

Syntax

```
openPropDB(  
    name: str = None,  
    /,  
    *,  
    topCellView: dmCellView | dmCellViewName = None,  
    filePath: str = None,  
    readOnly: bool = None,  
    scratch: bool = None)  
Returns: hePropDB
```

Summary:

Opens and/or creates an hePropDB on disk.

Category:

Hierarchy Editing

Licenses:

None

Parameters

- name – Hierarchical Property Database. (Default is: “)
- topCellView – The top cellview to search for the named hePropDB.
- filePath – The path to an HDB file to open or create
- readOnly – If true the returned property database will not be editable. (Default is: False)
- scratch – If true the database is created in-memory without modifying the file system. (Default is: False)

Returns

Newly create property DB

popContext

Syntax

```
popContext(  
    context: heContext)  
Returns: None
```


Summary:

“Goes” to the parent of given context

Category:

Hierarchy Editing

Licenses:

None

Parameters

context – The heContext to push

Returns

None

pushContext

Syntax

```
pushContext(  
    context: heContext,  
    /,  
    *,  
    inst: oaInst,  
    master: dmCellView | dmCellViewName | oaDesign = None)  
Returns: None
```

Summary:

“Pushes” into the given context

Category:

Hierarchy Editing

Licenses:

None

Parameters

- context – The heContext to push
- inst – The instance to descend into
- master – The new master of heContext after push

Returns

None

queryViewSelection

Syntax

```
queryViewSelection(  
    *,  
    context: heContext,  
    inst: oaInst)  
Returns: str
```

Summary:

Query view selection

Category:

Hierarchy Editing

Licenses:

None

Parameters

- context – The queried heContext
- inst – instance queried

Returns

Describes how final switched master for given object is got

saveAsPropDB

Syntax

```
saveAsPropDB(  
    hePropDB: hePropDB,  
    /,  
    *,  
    filePath: str = None,  
    topCellView: dmCellView | dmCellViewName = None,  
    overwrite: bool = None,  
    name: str = None)  
Returns: None
```

Summary:

Saves the specified hePropDB.

Category:

Hierarchy Editing

Licenses:

None

Parameters

- hePropDB – The database to save.
- filePath – The path to write the given database.
- topCellView – The cellview to save the database to.
- overwrite – If false the command will not overwrite existing database to. (Default is: False)
- name – The new name to save the database to.

Returns

None

saveConfig**Syntax**

```
saveConfig(  
    config: heConfig,  
    /,  
    *,  
    cellView: dmCellView | dmCellViewName = None,  
    pathName: str = None)  
Returns: heConfig
```

Summary:

Saves the specified heConfig to the associated cellView, an external file, or to a new cellView.

Category:

Hierarchy Editing

Licenses:

None

Parameters

- config – The heConfig to be saved
- cellView – New cellView or existing config cellview. You can only set <-cellView> or <-pathName>, not both. (Default is: “)
- pathName – New config file. You can only set <-cellView> or <-pathName>, not both. (Default is: “)

Returns

The saved <heConfig> object

savePropDB

Syntax

```
savePropDB(  
  hePropDB: hePropDB)  
Returns: None
```

Summary:

Saves the specified hePropDB.

Category:

Hierarchy Editing

Licenses:

None

Parameters

hePropDB – The database to save.

Returns

None

setPropValue

Syntax

```
setPropValue(  
  name: str,  
  /,  
  *,  
  value: str,
```

```
db: hePropDB,  
context: heContext,  
inst: oaInst | str = None,  
scope: he.setPropValue.Scope = None,  
propType: he.setPropValue.PropType = None)  
Returns: heProp
```

Summary:

Sets the specified property to the given value.

Category:

Hierarchy Editing

Licenses:

None

Parameters

- name – The name of the property
- value – The value of the property
- db – The HDB to modify
- context – The queried heContext
- inst – Instance Name
- scope – Specifies the scope to modify ({ cell | instance | occurrence } (Default is: 'occurrence'))
- propType – Specifies the type to use for creating the property ({ bool | float | int | string })

Returns

The created/modified property

setRuleFromDE

Syntax

```
setRuleFromDE(  
*,  
viewName: str,  
instName: str = None,  
occurrence: str = None,  
in_: giWindow)  
Returns: None
```

Summary:

Sets a view selection rule to the instance with specified name in specified window.

Category:

Hierarchy Editing

Licenses:

None

Parameters

- viewName – The view name to select
- instName – The name of the instance to add the rule to
- occurrence – The occurrence path to add the rule to
- in_ – The window with the design containing specified instance

Returns

None

showExportToFile

Syntax

```
showExportToFile(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Invokes the heExportToFile dialog

Category:

Hierarchy Editing

Licenses:

None

Parameters

parent – Active editor window

Returns

Show Export To File

showHierarchyEditor

Syntax

```
showHierarchyEditor(  
*,  
    cellView: dmCellView = None,  
    config: heConfig = None,  
    readOnly: bool = None)  
Returns: giWindow
```

Summary:

Creates a Top Level Hierarchy Editor

Category:

Hierarchy Editing

Licenses:

None

Parameters

- cellView – Config CellView opened.
- config – Config object opened.
- readOnly – Opens config in read-only mode. (Default is: False)

Returns

Newly created hierarchy editor window

showObject

Syntax

```
showObject(  
    object: deHierObject,  
    /,  
    *,  
    assistant: giAssistant)  
Returns: None
```

Summary:

Shows the specified object in the Design Navigator.

Category:

Design Editing

Licenses:

None

Parameters

- object – Object to show
- assistant – Design Navigator assistant object.

Returns

None

undoRedoHistory

Syntax

```
undoRedoHistory(  
*,  
    windowId: int,  
    redo: bool | None = None,  
    steps: int | None = None)  
Returns: None
```

Summary:

Performs undo/redo for Hierarchy Editor

Category:

Hierarchy Editing

Licenses:

None

Parameters

- windowId – The window id of Hierarchy Editor
- redo – If true performs redo, else - undo by default. (Default is: False)
- steps – Number of undo/redo history items (Default is: 1)

Returns

None

snps.custom.cmd.il

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform PDK procedures.

This section covers the following topics:

- [evalNamedArgFuncCall](#)
- [sprintf](#)
- [toTcl](#)

evalNamedArgFuncCall

Syntax

```
evalNamedArgFuncCall(  
    function: str,  
    /,  
    *,  
    arguments: list[str])  
Returns: None
```

Summary:

Evaluate named arguments function call.

Category:

Pdk Commands

Licenses:

None

Parameters

- function – Function Name
- arguments – Arguments List

Returns

None

sprintf

Syntax

```
sprintf(  
  argument: list[str])  
Returns: str
```

Summary:

Prints the given string

Category:

Pdk Commands

Licenses:

None

Parameters

argument – The string that need to be formatted.

Returns

Formatted string

toTcl

Syntax

```
toTcl(  
  script: str)  
Returns: None
```

Summary:

Translates Lisp expressions to Tcl.

Category:

Foundation

Licenses:

None

Parameters

script – The Lisp expression to translate.

Returns

None

snps.custom.cmd.le

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform layout editing.

This section covers the following topics:

- [addDummy](#)
- [addToFigGroup](#)
- [align](#)
- [alignAngle](#)
- [autoFix](#)
- [autoVia](#)
- [boolean](#)
- [breakSync](#)
- [changePinRelationship](#)
- [checkCapacitance](#)
- [checkDRCViolations](#)
- [checkDRCViolationsByICV](#)
- [checkEMResConstraints](#)
- [checkOpens](#)
- [checkPinConnectivity](#)
- [checkShapeEM](#)
- [checkShield](#)

- [checkShorts](#)
- [checkTrackSnapping](#)
- [checkVias](#)
- [checkVoltageAnnotations](#)
- [chop](#)
- [clearFlightlines](#)
- [clearPinConnections](#)
- [clearVoltagePhysicalAnnotations](#)
- [clearVoltages](#)
- [cloneMaster](#)
- [closeICVLManager](#)
- [compactBus](#)
- [connectICCNet](#)
- [convertToAliasLayer](#)
- [convertToArray](#)
- [convertToPath](#)
- [convertToPolygon](#)
- [copy](#)
- [copyHighlighted](#)
- [copyLook](#)
- [copyTrackRegionDef](#)
- [correctOffgrid](#)
- [createAttributeLabel](#)
- [createBlockage](#)
- [createBoundary](#)
- [createCellViewFromSelection](#)
- [createCoordinateMark](#)

- [createCorridorArea](#)
- [createDRDLineEdit](#)
- [createDRDTreeWidget](#)
- [createEllipse](#)
- [createFlightlines](#)
- [createGlobalTrackRegion](#)
- [createGuardRing](#)
- [createICCInst](#)
- [createICCNet](#)
- [createInst](#)
- [createLabel](#)
- [createMPP](#)
- [createMPPDef](#)
- [createMPPEnclosedPathDef](#)
- [createMPPEnclosedShapeDef](#)
- [createMPPMasterPathDef](#)
- [createMPPOffsetPathDef](#)
- [createMPPRingDef](#)
- [createMPPSubRectDef](#)
- [createNetGroup](#)
- [createOffTrackInfo](#)
- [createPath](#)
- [createPathSeg](#)
- [createPin](#)
- [createPinsFromText](#)
- [createPolygon](#)
- [createRectangle](#)

- [createRouteCorridor](#)
- [createRow](#)
- [createSiteDef](#)
- [createStripedWire](#)
- [createStripeSegment](#)
- [createSyncFigGroup](#)
- [createSyncGroup](#)
- [createTerm](#)
- [createTrack](#)
- [createTrackGroupDef](#)
- [createTrackPattern](#)
- [createTrackPatternDef](#)
- [createTrackRegion](#)
- [createTrackRegionDef](#)
- [createVia](#)
- [createVoltagePhysicalAnnotations](#)
- [cutCorner](#)
- [cycleDensity](#)
- [delete](#)
- [deleteCutMetal](#)
- [deleteDensityResult](#)
- [deleteGeneratedPins](#)
- [designSummary](#)
- [displayFlightlines](#)
- [displayPinConnections](#)
- [distribute](#)
- [evalViaParams](#)

- [exportResistanceQuery](#)
- [externalizeColor](#)
- [extractLayoutContext](#)
- [fill](#)
- [fixTrackSnapping](#)
- [flatten](#)
- [generateAbstract](#)
- [generateAntennaRunset](#)
- [generateCutMetal](#)
- [generateGuardRing](#)
- [generateLabel](#)
- [generateLayout](#)
- [generateLayoutContext](#)
- [generatePGMarkers](#)
- [generatePins](#)
- [generateShapes](#)
- [generateSoftAbstract](#)
- [getAbuttedInsts](#)
- [getAbuttedMPPs](#)
- [getAreaAndPerimeter](#)
- [getBoundaryTrackRegionDefs](#)
- [getCommonPathPoints](#)
- [getCoordinateMarks](#)
- [getFlightlines](#)
- [getGateDistance](#)
- [getGateRectangles](#)
- [getGlobalTrackRegions](#)

- [getIncompleteNets](#)
- [getIntersections](#)
- [getMPPDefs](#)
- [getMPPs](#)
- [getNearestTracks](#)
- [getNetGroups](#)
- [getNetIslands](#)
- [getPseudoParallelNets](#)
- [getRouteCorridors](#)
- [getRuleInfoByICV](#)
- [getStripedWires](#)
- [getSyncGroups](#)
- [getTrackGroupDefs](#)
- [getTrackPatternDefs](#)
- [getTrackPatterns](#)
- [getTrackRegionDefs](#)
- [getTrackRegions](#)
- [getTracks](#)
- [getTracksInRegion](#)
- [getUnboundGlobalTrackRegions](#)
- [getUnboundTrackRegions](#)
- [getUnboundTracks](#)
- [getUnnettedFigs](#)
- [highlightConnected](#)
- [importCommandBindings](#)
- [importResistanceQuery](#)
- [importVoltages](#)

- [isInSync](#)
- [snps.custom.cmd.le](#)
- [merge](#)
- [move](#)
- [padLabel](#)
- [pasteLook](#)
- [propagateConnectivity](#)
- [refreshDensityResult](#)
- [regenerateMPP](#)
- [remaster](#)
- [removeFromFigGroup](#)
- [removeOverlappingCuts](#)
- [removeUnboundInstanceTerms](#)
- [repeatCopy](#)
- [replaceVia](#)
- [reportCapacitance](#)
- [reportCELayers](#)
- [reportConnectivityPoints](#)
- [reportEM](#)
- [reportNets](#)
- [reportRC](#)
- [reportResistance](#)
- [reportShield](#)
- [resetVoltageOverrides](#)
- [reshape](#)
- [resize](#)
- [rotate](#)

- [scalarizeArrayInst](#)
- [setNetAnchor](#)
- [showAlignAssistOptions](#)
- [showCommandBindings](#)
- [showCreateTrack](#)
- [showCreateTrackGroupDefinition](#)
- [showCreateTrackRegionDefinition](#)
- [showDefineTerminals](#)
- [showDesignOptions](#)
- [showDRDOptions](#)
- [showDRDRuleSelection](#)
- [showEditTrack](#)
- [showEditTrackGroupDefinition](#)
- [showEditTrackRegion](#)
- [showImportLayout](#)
- [showLayoutGeneratorOptions](#)
- [showOLPOptions](#)
- [showTrackRegionOptions](#)
- [split](#)
- [splitArray](#)
- [splitIntoBoxes](#)
- [stretch](#)
- [syncTo](#)
- [toggleDRDTreeWidget](#)
- [trimWire](#)
- [validateColor](#)
- [yank](#)

- [snps.custom.cmd.le.pd](#)
- [snps.custom.cmd.le.pkg](#)

addDummy

Syntax

```
addDummy(  
box: box,  
/,  
*,  
design: oaDesign,  
dummyRule: bool = None,  
mode: le.addDummy.Mode = None,  
PMOSGateNet: str = None,  
PMOSSDNet: str = None,  
NMOSGateNet: str = None,  
NMOSSDNet: str = None,  
prefix: str = None,  
fingerCount: int = None,  
libName: str = None,  
cellName: str = None,  
viewName: str = None,  
interAbuttedChains: bool = None)  
Returns: oaInst
```

Summary:

Add Dummy Device

Category:

Layout Editing

Licenses:

Basic

Parameters

- box – The coordinates of bounding box of Add Dummy.
- design – Design in which to create the Add Dummy.
- dummyRule – Specify whether to add dummy by lxDummyRule or not. (Default is: False)
- mode – The mode of add dummy. ({ auto | fill | insert } (Default is: 'auto'))
- PMOSGateNet – The gate net name of the pmos dummy device.

- PMOSSDNet – The S/D net name of the pmos dummy device.
- NMOSGateNet – The gate net name of the nmos dummy device.
- NMOSSDNet – The S/D net name of the nmos dummy device.
- prefix – The prefix name of dummy device. (Default is: 'lxDummy_')
- fingerCount – The finger count of dummy device. (Default is: 1)
- libName – The library name of the dummy device.
- cellName – The cell name of the dummy device.
- viewName – The view name of the dummy device.
- interAbuttedChains – Specify if only filled the inter abutted chains. (Default is: False)

Returns

Newly created dummy device

addToFigGroup

Syntax

```
addToFigGroup(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
         | list[oaVia],  
/,  
*,  
to: deFigure | oaFigGroup)  
Returns: oaFigGroup
```

Summary:

Add objects to the figure group

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – Collection of objects to add to the given oaFigGroup
- to – oaFigGroup to add the given objects to.

Returns

The OA figure group object updated by the command.

align

Syntax

```
align(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
        list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
        | list[oaVia],  
/,  
*,  
src: deFigure | oaFig,  
side: le.align.Side = None,  
distance: float = None,  
direction: le.align.Direction = None,  
individually: bool = None,  
byPitch: bool = None)  
Returns: list[deFigure] | list[oaFig]
```

Summary:

Align specified objects

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – List of target objects to align.
- src – Source object for alignment, can be also a sub-object.
- side – Alignment side. The value of this option will be used only if at least one full object is specified. ({ bottom | horizontalMiddle | left | right | top | verticalMiddle } (Default is: 'bottom'))
- distance – Distance between aligning objects. (Default is: 0)
- direction – Direction according to which the distance between the aligning edges should be applied. ({ bottom | left | right | top } (Default is: 'right'))

- individually – Control alignment of objects as pseudo group or individually. (Default is: True)
- byPitch – Control alignment of objects by pitch or by spacing. (Default is: False)

Returns

Collection of aligned objects.

alignAngle

Syntax

```
alignAngle(  
*,  
    sourceInst: deFigure | oaFig,  
    referenceInst: deFigure | oaFig,  
    sourcePinShape: deFigure | oaFig,  
    referencePinShape: deFigure | oaFig,  
    spacing: float = None,  
    orthogonalSpacing: float = None,  
    wholeChain: bool = None)  
Returns: None
```

Summary:

Align Angle

Category:

Layout Editing

Licenses:

Apex

Parameters

- sourceInst – Source instance for angle alignment
- referenceInst – Reference instance for angle alignment
- sourcePinShape – Source pin shape for angle alignment
- referencePinShape – Reference pin shape for angle alignment
- spacing – Spacing between aligning objects. (Default is: 0)
- orthogonalSpacing – Orthogonal spacing between aligning objects. (Default is: 0)
- wholeChain – Control alignment of objects align with whole chain or individually. (Default is: True)

Returns

None

autoFix

Syntax

```
autoFix(  
    box: box = None,  
    /,  
    *,  
    design: oaDesign,  
    insertJogs: le.autoFix.InsertJogs = None,  
    lockInstances: bool = None,  
    lockVias: bool = None,  
    lockInvisibleShapes: bool = None)  
Returns: list[deFigure]
```

Summary:

Correct process and design constraint errors.

Category:

Layout Editing

Licenses:

Backend

Parameters

- box – The coordinates of bounding box of region to run AutoFix
- design – Design in which to run AutoFix
- insertJogs – Specifies if AutoFix should insert jogs ({ aggressively | conservatively | never } (Default is: 'conservatively'))
- lockInstances – Specifies if AutoFix should move instances or not (Default is: False)
- lockVias – Specifies if AutoFix should move vias or not (Default is: False)
- lockInvisibleShapes – Specifies if AutoFix should modify invisible shapes or not (Default is: False)

Returns

Collection of deFigures modified by autoFix.

autoVia

Syntax

```
autoVia(  
*,  
    design: oaDesign,  
    box: box = None,  
    point: point = None,  
    depthOn0xView: bool = None,  
    nets: str = None,  
    sameNetOnly: bool = None,  
    sameNetOnlyNoNetAnyNet: bool = None,  
    sameNetOnlyNoNetNoNet: bool = None,  
    allowExceed: bool = None,  
    fitToOverlappedArea: bool = None,  
    createMetalShape: bool = None,  
    allOverlappedArea: bool = None,  
    allowStackedVia: bool = None,  
    checkEnvAware: bool = None,  
    autoBridgeVia: bool = None,  
    minCutNum: int = None,  
    colRowDir: le.autoVia.ColRowDir = None,  
    viaSetting: str = None,  
    layerDisable: str = None)  
Returns: oaVia
```

Summary:

Auto via

Category:

Layout Editing

Licenses:

Basic

Parameters

- design – Design in which to create the auto via.
- box – Option to specify area for auto via creation.
- point – Option to specify point for auto via creation.
- depthOn0xView – Option to specify the depth is on 0x or current stop view level, default is 0x. (Default is: True)
- nets – Option to specify the name of nets on searching the candidates. (Default is: 'empty')

- sameNetOnly – Option to restrict auto via between two equivalent net. (Default is: True)
- sameNetOnlyNoNetAnyNet – Option to create auto via only on same net shapes where no net matches any net. (Default is: True)
- sameNetOnlyNoNetNoNet – Option to create auto via only on same net shapes where no net matches no net. (Default is: True)
- allowExceed – Option to allow metal shape exceed overlapped area. (Default is: True)
- fitToOverlappedArea – Option to fit metal shape on overlapped area. (Default is: False)
- createMetalShape – Option to create extra metal shapes. (Default is: False)
- allOverlappedArea – Option to auto via on all polygon area for point mode. (Default is: False)
- allowStackedVia – Option to allow create stacked via. (Default is: True)
- checkEnvAware – Option to check environment aware. (Default is: True)
- autoBridgeVia – Option to enable auto bridge via. (Default is: False)
- minCutNum – Option to specify number of minimum cuts. (Default is: 1)
- colRowDir – Option to specify direction of column and row. ({ Auto | H | V } (Default is: 'Auto'))
- viaSetting – Option to specify via setting. Format e.g. {m1 viadef1 cutclass1} or {m1 viadef1 auto} or {m1 auto cutclass1} or {{m1 viadef1 cutclass1} {m2 viadef2 cutclass2}}
- layerDisable – Option to specify disable layers. Format e.g. {m6 m7 m8}

Returns

Created auto via

boolean

Syntax

```
boolean(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
         | list[oaVia],  
/,  
*,  
operation: le.boolean.Operation,  
points: list[point] = None,  
result: le.boolean.Result = None)  
Returns: list[oaFig]
```

Summary:

Boolean

Category:

Layout Editing

Licenses:

Basic

Parameters

- **objects** – List of selected objects.
- **operation** – Operation which is performed. ({ and | not | or | xor })
- **points** – Point list to form an area for Boolean operation.
- **result** – Result type of the operation. ({ newWithActiveLPP | replace | replaceWithActiveLPP })

Returns

Collection of Boolean objects.

breakSync**Syntax**

```
breakSync(  
  objects: list[deFigure] | list[oaFigGroup])  
Returns: list[oaFigGroup]
```

Summary:

Breaks the synchronous relation between the provided oaFigGroup objects.

Category:

Layout Editing

Licenses:

Backend

Parameters

objects – Collection of oaFigGroup/deFigure objects for which the synchronous relation is to be broken.

Returns

Collection of oaFigGroup objects obtained by breaking the provided oaFigGroup objects.

changePinRelationship

Syntax

```
changePinRelationship(  
    pins: list[oaPin],  
    /,  
    *,  
    type: le.changePinRelationship.Type)  
Returns: None
```

Summary:

Change pin relationship

Category:

Layout Editing

Licenses:

Basic

Parameters

- pins – Collection of oaPin objects on which operation should be performed.
- type – New value for relationship for provided list of pins. ({ mustJoin | strong | weak })

Returns

None

checkCapacitance

Syntax

```
checkCapacitance(  
    scope: oaDesign | list[oaNet])  
Returns: str
```

Summary:

Check capacitance information

Category:

Layout Editing

Licenses:

Backend

Parameters

scope – net to be reported.

Returns

check capacitance constraints .

checkDRCViolations

Syntax

```
checkDRCViolations(  
  objects: oaGroup | list[deFigure] | list[deSelectionSet] |  
    list[oaFig] | list[oaFigGroup] | list[oaInst] | list[oaMarker] |  
    list[oaNet] | list[oaShape] | list[oaVia] = None,  
  /,  
  *,  
  design: oaDesign = None,  
  box: box = None)  
Returns: list[oaFig]
```

Summary:

Check DRC violations

Category:

Layout Editing

Licenses:

Backend

Parameters

- objects – List of objects to check DRC.
- design – Design in which to check drc
- box – The coordinates of bounding box of region to check drc

Returns

Collection of Markers.

checkDRCViolationsByICV

Syntax

```
checkDRCViolationsByICV(  
*,  
    design: oaDesign,  
    box: box,  
    runset: str,  
    oriRunset: str,  
    layerMap: str = None,  
    arguments: str = None,  
    format: str,  
    file: str = None,  
    runDir: str = None,  
    checkRules: str = None,  
    skipRules: str = None,  
    controlVars: str = None,  
    showErrorViewer: bool = None,  
    background: bool = None,  
    checkLayersOption: int = None)  
Returns: None
```

Summary:

Invoke Window Checking by ICV Live

Category:

Layout Editing

Licenses:

Backend

Parameters

- design – Design in which to check drc
- box – The coordinates of bounding box of region to check drc
- runset – The runset for drc checking
- oriRunset – The original runset for drc checking
- layerMap – The layer mapping for drc checking
- arguments – The arguments for drc checking
- format – The input format (OpenAccess|Stream) for drc checking
- file – The input file for drc checking

- runDir – The directory to run drc checking
- checkRules – The rules to check for drc checking
- skipRules – The rules to skip for drc checking
- controlVars – The control variables of drc checking
- showErrorViewer – Invoke error viewer after drc checking
- background – Invoke drc checking in the background
- checkLayersOption – Specify check layers option

Returns

None

checkEMResConstraints

Syntax

```
checkEMResConstraints(  
objects: oaDesign | list[oaGroup] | list[oaNet],  
/,  
*,  
EM: bool = None,  
Res: bool = None,  
commonPath: bool = None,  
commonPathMode: int = None,  
**kwargs)  
Returns: str
```

Summary:

check electromigration and resistance constraints

Category:

Layout Editing

Licenses:

Backend

Parameters

- objects – nets, net groups or design to be checked.
- EM – check EM constraint violations for wires and vias (Default is: True)
- Res – check resistance constraint violations (Default is: True)

- **commonPath** – check common path resistance constraint violations (Default is: True)
- **commonPathMode** – mode of common path resistance calculations: search its whole connection network or shortest path, and report the first or all violating common paths (Default is: 0)

Returns

check electromigration and resistance constraints .

checkOpens

Syntax

```
checkOpens (  
*,  
    design: oaDesign = None,  
    nets: oaNet | list[oaNet] = None,  
    depth: int = None,  
    hierarchyMode: le.checkOpens.HierarchyMode = None,  
    followFlightlines: bool = None)  
Returns: list[oaMarker]
```

Summary:

Reports connectivity open nets by creating markers.

Category:

Layout Editing

Licenses:

Elite

Parameters

- **design** – design object.
- **nets** – Nets for which to check opens.
- **depth** – Depth
- **hierarchyMode** – Hierarchy mode ({ full | top | view })
- **followFlightlines** – Reports opens based on flightlines

Returns

Collection of Markers

checkPinConnectivity

Syntax

```
checkPinConnectivity(  
objects: oaDesign | list[deFigure] | list[deSelectionSet] |  
list[oaFig] | list[oaFigGroup] | list[oaInst] | list[oaMarker] |  
list[oaShape] | list[oaVia])  
Returns: list[oaFig]
```

Summary:

Check pin connectivity

Category:

Layout Editing

Licenses:

Basic

Parameters

objects – Collection of objects to be checked for pin accessibility.

Returns

Collection of Markers.

checkShapeEM

Syntax

```
checkShapeEM(  
scope: oaNet = None,  
/,  
*,  
layerNum: int = None,  
layerName: str = None,  
type: str,  
current: float = None,  
width: float = None,  
length: float = None,  
color: str = None,  
columnNum: int = None,  
rowNum: int = None,  
cutWidth: float = None,  
cutLength: float = None,  
layer1Width: float = None,  
layer1Length: float = None,  
layer2Width: float = None,
```



```
layer2Length: float = None,  
currentDir: str = None)  
Returns: str
```

Summary:

check shape em information

Category:

Layout Editing

Licenses:

Backend

Parameters

- scope – net to be reported.
- layerNum – Specify layer number.
- layerName – Specify layer name.
- type – Specify EM type in Avg/Peak/RMS/DC.
- current – Specify current in mA.
- width – Specify width in um.
- length – Specify length in um.
- color – Specify color in A/B.
- columnNum – Specify cut column number.
- rowNum – Specify cut row number.
- cutWidth – Specify cut width in um.
- cutLength – Specify cut length in um.
- layer1Width – Specify layer1 width in um.
- layer1Length – Specify layer1 length in um.
- layer2Width – Specify layer2 width in um.
- layer2Length – Specify layer2 length in um.
- currentDir – Specify cut current direction in UP/DOWN.

Returns

The shape EM.

checkShield

Syntax

```
checkShield(  
  objects: oaDesign | list[oaGroup] | list[oaNet])  
Returns: str
```

Summary:

Check net shield constraint

Category:

Layout Editing

Licenses:

Basic

Parameters

objects – nets, net groups or design to be reported.

Returns

Check net shield constraints.

checkShorts

Syntax

```
checkShorts(  
  *,  
  design: oaDesign,  
  hierarchyMode: le.checkShorts.HierarchyMode = None,  
  depth: int = None,  
  skipCellsWithNoConnectivity: bool = None)  
Returns: None
```

Summary:

Checks shorts with full hierarchy

Category:

Layout Editing

Licenses:

Basic

Parameters

- design – design object
- hierarchyMode – short mode ({ full | top | view })
- depth – depth
- skipCellsWithNoConnectivity – skipCellsWithNoConnectivity

Returns

None

checkTrackSnapping

Syntax

```
checkTrackSnapping(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaInst] | list[oaShape] | list[oaVia] = None,  
/,  
*,  
region: box = None,  
context: deContext = None,  
design: oaDesign = None,  
createMarker: bool = None,  
checkSignalType: bool = None,  
checkColor: bool = None,  
**kwargs)  
Returns: list[leOffTrackInfo]
```

Summary:

Check Track Sanpping Status of Objects

Category:

Layout Editing

Licenses:

Backend

Parameters

- objects – List of objects to check track snapping.
- region – Region to check track snapping

- context – Context to check track snapping
- design – Top design to check track snapping
- createMarker – Specifies whether to create error markers (Default is: False)
- checkSignalType – Specifies whether to check signal type mismatch (Default is: True)
- checkColor – Specifies whether to check color mismatch (Default is: True)

Returns

Collection of `leOffTrackInfo` for off-track objects

checkVias

Syntax

```
checkVias(  
*,  
    design: oaDesign,  
    mode: le.checkVias.Mode = None,  
    depth: int = None,  
    layers: list[oaPhysicalLayer | str] = None,  
    region: box = None,  
    nets: list[oaBitNet | str] = None,  
    routeTypes: list[str] = None,  
    checkSize: bool = None,  
    numCuts: int = None,  
    allowExceed: bool = None,  
    viaStackingRule: bool = None,  
    miscTypes: list[str] = None)  
Returns: list[oaFig]
```

Summary:

Checks for missing / undersized vias.

Category:

Layout Editing

Licenses:

Basic

Parameters

- design – Design to check.
- mode – Mode for via checker ({ constraints | misc | reliability } (Default is: 'reliability'))
- depth – Depth of hierarchy to check.

- layers – Range of layers to check.
- region – Region to check.
- nets – Nets to check.
- routeTypes – Route types to check.
- checkSize – Flag to turn on/off via size check.
- numCuts – Specify the minimum number of cuts.
- allowExceed – Flag to turn on/off dropping via without full metal coverage.
- viaStackingRule – Set to enable via stacking rule checking (Default is: False)
- miscTypes – Misc. checkings to enable. Available checking: “overlapAbut”.

Returns

Collection of Markers

checkVoltageAnnotations

Syntax

```
checkVoltageAnnotations(  
context: deContext = None,  
/,  
*,  
scope: le.checkVoltageAnnotations.Scope = None,  
nets: str | list[oaNet] = None,  
depth: le.checkVoltageAnnotations.Depth = None,  
nets_with_voltage_value: bool = None,  
only_voltage_lpp: bool = None,  
check_udv: bool = None,  
check_vsync: bool = None,  
check_vflag: bool = None,  
missing_net_annotation: bool = None,  
missing_label: bool = None,  
multiple_labels: bool = None,  
incorrect_label_position: bool = None,  
incorrect_label_value: bool = None,  
incorrect_label_lpp: bool = None,  
missing_voltage_inst: bool = None,  
extra_voltage_inst: bool = None,  
mismatched_voltage_inst_connection: bool = None,  
missing_voltage_inst_connection: bool = None,  
mismatched_voltage_inst_params: bool = None,  
mismatched_voltage_inst_master: bool = None,  
misplaced_voltage_inst: bool = None,  
net_has_voltage_inst_conflicts: bool = None,
```

```
connected_shape_layer_conflicts: bool = None)  
Returns: list[oaFig]
```

Summary:

Check voltage annotations for all nets.

Category:

Layout Editing

Licenses:

Backend

Parameters

- context – Context object of the create physical voltage annotations operation.
- scope – Check text label annotation scope. ({ Design | Nets | Viewport }) (Default is: 'Design')
- nets – oaNet or net name list for Nets scope.
- depth – Check text label annotation depth. ({ 0x | Full | View }) (Default is: '0x')
- nets_with_voltage_value – Check nets with voltage value only (Default is: False)
- only_voltage_lpp – Check voltage lpp only (Default is: False)
- check_udv – Check udv instance only (Default is: True)
- check_vsync – Check vsync instance only (Default is: True)
- check_vflag – Check vflag instance only (Default is: True)
- missing_net_annotation – Specifies if report missing net annotation or not (Default is: True)
- missing_label – Specifies if report missing label or not. (Default is: True)
- multiple_labels – Specifies if report multiple labels or not (Default is: True)
- incorrect_label_position – Specifies if report incorrect label position or not (Default is: True)
- incorrect_label_value – Specifies if report incorrect label value or not (Default is: True)
- incorrect_label_lpp – Specifies if report incorrect label lpp or not (Default is: True)
- missing_voltage_inst – Specifies if report missing voltage instance or not (Default is: True)
- extra_voltage_inst – Specifies if report extra voltage instance or not (Default is: True)

- `mismatched_voltage_inst_connection` – Specifies if report mismatched voltage instance connection or not (Default is: True)
- `missing_voltage_inst_connection` – Specifies if report missing voltage instance connection or not (Default is: True)
- `mismatched_voltage_inst_params` – Specifies if report mismatched voltage instance params or not (Default is: True)
- `mismatched_voltage_inst_master` – Specifies if report mismatched voltage instance master or not (Default is: True)
- `misplaced_voltage_inst` – Specifies if report misplaced voltage instance or not (Default is: True)
- `net_has_voltage_inst_conflicts` – Specifies if report net has voltage instance conflicts or not (Default is: True)
- `connected_shape_layer_conflicts` – Specifies if report connected shape layer conflicts or not (Default is: True)

Returns

Collection of Makrers

chop

Syntax

```
chop(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
         | list[oaVia],  
/,  
*,  
design: oaDesign = None,  
points: list[point],  
regionType: le.chop.RegionType = None,  
levels: int = None,  
anchor: point = None,  
snapToGrid: le.chop.SnapToGrid = None,  
copy: bool = None,  
keep: bool = None,  
keepPath: bool = None,  
chopLayers: list[lpp | str] = None,  
selectResult: bool = None)  
Returns: None
```

Summary:

Removes data cut by the given region or splits objects into pieces.

Category:

Layout Editing

Licenses:

Basic

Parameters

- **objects** – List of objects to chop.
- **design** – Design to chop objects in
- **points** – A list of points defining the contour of shape that specifies the chop region.
- **regionType** – Type of region passed to the -points argument. ({ polygon | polyline | rectangle }) (Default is: 'rectangle')
- **levels** – Number of hierarchical levels beyond which to ignore instances which overlap the chop region. (Default is: 0)
- **anchor** – Anchor point for paste.
- **snapToGrid** – Specify if yanked results will be snapped to manufacturing grid. ({ abort | off | on }) (Default is: 'on')
- **copy** – Whether the chopped objects will be copied to clipboard. (Default is: True)
- **keep** – Whether chopped objects will be kept in the design. (Default is: False)
- **keepPath** – Whether to chop Paths and PathSegs based on the intersection point of the centerline and keep Paths and PathSegs in their object type. (Default is: False)
- **chopLayers** – List containing lpp for chop layers or { default | all }
- **selectResult** – Whether add the result to the selection set. (Default is: False)

Returns

None

clearFlightlines

Syntax

```
clearFlightlines(  
  objects: list[oaGroup] | list[oaNet])  
Returns: None
```


Summary:

Clears flightlines.

Category:

Layout Editing

Licenses:

Basic

Parameters

objects – Collection of flightline, group or net objects to remove flightline display

Returns

None

clearPinConnections

Syntax

```
clearPinConnections(  
  objects: list[oaPin])  
Returns: None
```

Summary:

Clears pin connection flightlines.

Category:

Layout Editing

Licenses:

Basic

Parameters

objects – Collection of pin objects to remove pin connection flightline display

Returns

None

clearVoltagePhysicalAnnotations

Syntax

```
clearVoltagePhysicalAnnotations(  
    context: deContext = None)  
Returns: None
```

Summary:

Clear voltage physical annotation information in layout.

Category:

Layout Editing

Licenses:

Backend

Parameters

context – context to be cleared

Returns

None

clearVoltages

Syntax

```
clearVoltages(  
    context: deContext = None)  
Returns: None
```

Summary:

Clear voltage information on nets in layout.

Category:

Layout Editing

Licenses:

Backend

Parameters

context – context to be cleared

Returns

None

cloneMaster

Syntax

```
cloneMaster(  
    object: deFigure | oaInst,  
    /,  
    *,  
    cellName: str,  
    overwrite: bool = None)  
Returns: list[oaInst]
```

Summary:

Clone master

Category:

Layout Editing

Licenses:

Basic

Parameters

- object – Instance to clone master
- cellName – Cell name for the new cloned master
- overwrite – Specifies if need to overwrite in case (Default is: False)

Returns

The remastered instance.

closeICVManager

Syntax

```
closeICVManager()  
Returns: bool
```

Arguments:

Summary:

Force to close the icvl_manager

Category:

Layout Editing

Licenses:

Backend

Returns

icvl_manager be closed or not.

compactBus

Syntax

```
compactBus(  
objects: dbCollection | list[deFigure] | list[deSelectionSet] |  
list[oaFig] | list[oaFigGroup] | list[oaInst] | list[oaMarker] |  
list[oaShape] | list[oaVia],  
/,  
*,  
mode: le.compactBus.Mode,  
leadingBit: deFigure | oaFig = None,  
spacing: float = None,  
pitch: float = None,  
refPoints: list[point] = None,  
fanInOutAlignment: le.compactBus.FanInOutAlignment = None,  
fanInOutJogSpacingMode: le.compactBus.FanInOutJogSpacingMode = None,  
fanInOutJogSpacingValue: float = None,  
ladderAlignment: le.compactBus.LadderAlignment = None,  
ladderAlignmentOffset: float = None,  
ladderJogCount: int = None)  
Returns: None
```

Summary:

Compress and Expand bus bits.

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – path, pathsegs to be Compress/Expand.
- mode – Specifies operation mode to Compress/Expand bus. ({ compact | fanInOut | ladder })
- leadingBit – Specifies leading bit for compact mode.
- spacing – Specifies spacing value between bits of bus in compact, fanInOut and ladder mode (Default is: 0)
- pitch – Specifies pitch value between bits of bus in compact mode. (Default is: 0)
- refPoints – Three points orthogonal polyline point list to apply in fanInOut and ladder mode.
- fanInOutAlignment – Specifies align mode which use in fanInOut mode. ({ center | centerWire | side }) (Default is: 'center')
- fanInOutJogSpacingMode – Specifies jog spacing mode which use in fanInOut mode. ({ asSource | asTarget | specify }) (Default is: 'asTarget')
- fanInOutJogSpacingValue – Specifies jog spacing value when fanInOut spacing mode is "specify". (Default is: 0)
- ladderAlignment – Specifies align mode which use in ladder mode. ({ centerline | edge | offset }) (Default is: 'centerline')
- ladderAlignmentOffset – Specifies offset between reference line and wire when ladder alignment mode is "offset". (Default is: 0)
- ladderJogCount – Specifies amount of maximum jogs in ladder mode. (Default is: 0)

Returns

None

connectICNet

Syntax

```
connectICNet(  
Net: oaNet = None,  
/,  
*,  
to: list[oaInstTerm])  
Returns: None
```

Summary:

Creates a new net in ICC design

Category:

ICC Link

Licenses:

None

Parameters

- Net – Net object to connect, if not given specified objects will be disconnected
- to – Collection of instance terminals to assign to the given net

Returns

None

convertToAliasLayer**Syntax**

```
convertToAliasLayer(  
*,  
    design: oaDesign = None,  
    libName: str = None)  
Returns: bool
```

Summary:

Convert the shapes/standard vias with source LPP to alias LPP for given design or all designs of the given library.

Category:

Layout Editing

Licenses:

Basic

Parameters

- design – design object
- libName – library name

Returns

return true if conversion is successfully completed.

convertToArray

Syntax

```
convertToArray(  
objects: list[deFigure] | list[deSelectionSet] | list[oaInst],  
/,  
*,  
mode: le.convertToArray.Mode,  
style: le.convertToArray.Style = None,  
matchAllProperties: bool = None)  
Returns: list[oaInst]
```

Summary:

Convert to array

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – Instances to be converted to array
- mode – Specify a mode, either singleArray or multipleArrays. This indicates whether converting to a single array or multiple ones. ({ multipleArrays | singleArray })
- style – Specify a style, either rowFirst (default), columnFirst, or auto. This works only when multipleArrays mode is selected. ({ auto | columnFirst | rowFirst } (Default is: 'rowFirst'))
- matchAllProperties – Indicate whether all selected instances should have the same properties. (Default is: True)

Returns

Collection of instances after converting to array.

convertToPath

Syntax

```
convertToPath(  
objects: list[deFigure] | list[oaFig],  
//,  
*,  
ignore: bool = None,  
mode: le.convertToPath.Mode = None,  
result: le.convertToPath.Result = None)  
Returns: list[oaShape]
```

Summary:

Converts specified polygons or rectangles to paths

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – The list of objects that should be converted.
- ignore – Ignore if inconvertible to user's specified type but convertible to another type. (Default is: True)
- mode – Determine which mode to convert. ({ auto | manual } (Default is: 'auto'))
- result – Determine the object type to which the input shape is converted. ({ path | pathseg } (Default is: 'path'))

Returns

Collection of converted paths

convertToPolygon

Syntax

```
convertToPolygon(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia],  
//,  
*,
```



```
numPoints: int = None)
Returns: list[oaShape]
```

Summary:

Converts specified objects to polygons

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – The list of objects that should be converted.
- numPoints – Number of vertices to convert an ellipse into a polygon.

Returns

Collection of converted polygon shapes

copy

Syntax

```
copy(
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]
| list[oaVia],
/,
*,
anchor: point,
dx: float = None,
dy: float = None,
rows: int = None,
dxRow: float = None,
dyRow: float = None,
cols: int = None,
dxCol: float = None,
dyCol: float = None,
rotate: le.copy.Rotate = None,
dest: oaDesign = None,
group: bool = None,
keepSync: bool = None,
keepInSyncGroup: bool = None,
keepNet: bool = None,
keepColor: bool = None,
keepHighlight: bool = None,
```

```
keepMember: bool = None,  
byPitch: bool = None,  
lpp: str = None)  
Returns: list[deFigure] | list[oaFig]
```

Summary:

Copy

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – List of objects to copy.
- anchor – Anchor point.
- dx – X projection of copy vector. (Default is: 0)
- dy – Y projection of copy vector. (Default is: 0)
- rows – Number of rows. (Default is: 1)
- dxRow – X projection of distance between rows. (Default is: 0)
- dyRow – Y projection of distance between rows. (Default is: 0)
- cols – Number of columns. (Default is: 1)
- dxCol – X projection of distance between columns. (Default is: 0)
- dyCol – Y projection of distance between columns. (Default is: 0)
- rotate – Rotation to apply on objects. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 } (Default is: 'R0'))
- dest – Design to copy objects into.
- group – Specifies if the objects are added to a figure group (Default is: False)
- keepSync – Keeps the objects to copy and their resulting copy synchronized. (Default is: False)
- keepInSyncGroup – Keeps the resulting groups synchronized if the groups to copy are synchronous groups. (Default is: False)
- keepNet – Keeps net names in the resulting copy. (Default is: False)

- keepColor – Keeps the color of objects. (Default is: False)
- keepHighlight – Keeps the highlight of objects. (Default is: False)
- keepMember – Keeps the membership of objects. (Default is: False)
- byPitch – Places the result by pitch or spacing. (Default is: True)
- lpp – Specifies the LPP of the resulting copy. (Default is: “”)

Returns

Collection of copied objects.

copyHighlighted

Syntax

```
copyHighlighted(  
name: str = None,  
/,  
*,  
source: oaDesign,  
destination: dmCellView | dmCellViewName | oaDesign,  
overwrite: bool = None,  
copyNet: bool = None,  
copyColor: bool = None,  
packet: str = None)  
Returns: None
```

Summary:

Copy Highlighted to Cellview.

Category:

Design Editing

Licenses:

None

Parameters

- name – Name of highlight set
- source – Source design to search for highlight sets containing figures
- destination – Destination design or cellview to create or append
- overwrite – Overwrites or appends to existing cellview (Default is: True)
- copyNet – Copy connectivity (flattened) (Default is: False)

- copyColor – Copy MPT color (Default is: False)
- packet – Packet defines the drawing style

Returns

None

copyLook

Syntax

```
copyLook(  
*,  
  objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
           list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
           | list[oaVia])  
Returns: list[oaFig]
```

Summary:

Copy look from selection

Category:

Layout Editing

Licenses:

Basic

Parameters

objects – List of objects to be copied.

Returns

Collection of copied objects.

copyTrackRegionDef

Syntax

```
copyTrackRegionDef(  
name: str = None,  
/,  
*,  
  to: oaDesign | oaTech,  
  reference: leTrackRegionDef,  
  overwrite: le.copyTrackRegionDef.Overwrite = None)  
Returns: leTrackRegionDef
```

Summary:

Creates an leTrackRegionDef in technology or design database by referencing to a given leTrackRegionDef object. Note that the command also copies leTrackGroupDef and leTrack objects, those are referenced by the given leTrackRegionDef, to the target database when they can not be found in the target database.

Category:

Layout Editing

Licenses:

Basic

Parameters

- name – Name of the trackRegionDef: If not specified the reference object name is taken. (Default is: "")
- to – Design or Tech in which to copy the trackRegionDef
- reference – Reference trackRegionDef object from which to copy
- overwrite – Determines whether or not to overwrite object if an object with the same name but different settings already exists in target database ({ full | none | rename | skip } (Default is: 'none'))

Returns

The newly copied leTrackRegionDef object

correctOffgrid

Syntax

```
correctOffgrid(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaShape] | list[oaVia],  
/,  
*,  
grid: le.correctOffgrid.Grid = None)  
Returns: None
```

Summary:

Correct Off-grid objects

Category:

Layout Editing

Licenses:

Backend

Parameters

- objects – List of objects to correct.
- grid – Grid to correct to ({ manufacturing | snapping })

Returns

None

createAttributeLabel

Syntax

```
createAttributeLabel(  
string: str,  
/,  
*,  
parent: deFigure | oaFig,  
valueOnly: bool = None,  
lpp: lpp,  
origin: point,  
just: le.createAttributeLabel.Just = None,  
orient: le.createAttributeLabel.Orient = None,  
font: le.createAttributeLabel.Font = None,  
height: float = None,  
drafting: bool = None,  
overbar: bool = None)  
Returns: oaShape
```

Summary:

Create attribute label

Category:

Layout Editing

Licenses:

Basic

Parameters

- string – Text of the label
- parent – Specifies the oaFig or deFigure, whose parameter is to be displayed.

- `valueOnly` – Specifies whether to display the value only, or the name and value of an attribute. (Default is: `False`)
- `lpp` – The lpp on which to create the text.
- `origin` – Location of the origin of text.
- `just` – Justification of the text with respect to the origin. ({ `centerCenter` | `centerLeft` | `centerRight` | `lowerCenter` | `lowerLeft` | `lowerRight` | `upperCenter` | `upperLeft` | `upperRight` }) (Default is: `'centerCenter'`)
- `orient` – Orientation of the text. ({ `MX` | `MXR90` | `MY` | `MYR90` | `R0` | `R180` | `R270` | `R90` }) (Default is: `'R0'`)
- `font` – Font style with which the text is displayed. ({ `euroStyle` | `fixed` | `gothic` | `math` | `milSpec` | `roman` | `script` | `stick` | `swedish` }) (Default is: `'stick'`)
- `height` – Height of the text in user units. (Default is: `1`)
- `drafting` – Indicates whether the text should not be mirrored or rotated. (Default is: `True`)
- `overbar` – Indicates whether the text will be rendered with overbar. (Default is: `False`)

Returns

New label object (`oaAttrDisplay` or `oaPropDisplay`)

createBlockage

Syntax

```
createBlockage(  
    points: list[point],  
    /,  
    *,  
    design: oaDesign,  
    parent: oaInst = None,  
    type: le.createBlockage.Type,  
    layer: str = None,  
    hasWidth: bool = None,  
    width: float = None)  
Returns: oaBlockage
```

Summary:

Create blockage

Category:

Layout Editing

Licenses:

Basic

Parameters

- points – The list of points that form blockage.
- design – Design in which to create the blockage.
- parent – Parent oalnst object for blockage
- type – Blockage type ({ feedthru | fill | pin | placement | routing | screen | slot | wiring })
- layer – The layer on which to create the blockage (Default is: “”)
- hasWidth – Specifies if created blockage has effective width. (Default is: False)
- width – Effective width of blockage that acts as a halo around the boundary of the blockage object. (Default is: 0)

Returns

Newly created blockage object

createBoundary

Syntax

```
createBoundary(  
points: list[point],  
/  
*,  
design: oaDesign,  
type: le.createBoundary.Type,  
force: bool = None,  
boundaryLPP: lpp = None,  
snapPoints: bool = None,  
endShapes: bool = None)  
Returns: oaFig
```

Summary:

Create boundary

Category:

Layout Editing

Licenses:

Basic

Parameters

- points – The list of points that form boundary.
- design – Design in which to create the boundary.
- type – Boundary type ({ pr | snap | wideAlignment })
- force – Flag that specifies if old boundary should be replaced with new one (Default is: False)
- boundaryLPP – Specify the boundary LPP for wide alignment boundary.
- snapPoints – Specify if snap points according to the wide alignment constraint when create wide alignment boundary (Default is: True)
- endShapes – Specify the end shapes for wide alignment boundary. (Default is: False)

Returns

Newly created boundary object

createCellViewFromSelection

Syntax

```
createCellViewFromSelection(  
*,  
    viewName: str = None,  
    objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
    list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
    | list[oaVia],  
    libName: str = None,  
    cellName: str,  
    instName: str = None,  
    replace: bool = None,  
    origin: point,  
    status: le.createCellViewFromSelection.Status = None,  
    force: bool = None)  
Returns: oaDesign
```

Summary:

Create cell view from selection

Category:

Layout Editing

Licenses:

Basic

Parameters

- `viewName` – New view's name. (Default is: 'layout')
- `objects` – List of objects that should be go to the newly created cell view.
- `libName` – Library name were new cell view should be created. (Default is: 'Library of current cellView')
- `cellName` – Cell name for which new view shoul be created.
- `instName` – Instance name (in case if objects should be replaced by instance). (Default is: 'Prefix from master's meta data or 'I' concatenated with integer.')
- `replace` – Specifies if objects should be replaced by newly created instance. (Default is: False)
- `origin` – Location of the origin of cellView.
- `status` – Placement status for cellView. ({ fixed | locked | none | placed | unplaced } (Default is: 'placed'))
- `force` – Specifies if need to overwrite in case if cell view exists (Default is: True)

Returns

Newly created CellView object

createCoordinateMark

Syntax

```
createCoordinateMark(  
point: point,  
/  
*,  
design: oaDesign,  
note: str = None,  
persistent: bool = None)  
Returns: leCoordinateMark
```

Summary:

Creates a coordinate mark at coordinate specified.

Category:

Layout Editing

Licenses:

None

Parameters

- point – Point to be marked.
- design – Design in which to create the mark.
- note – Note of the mark. (Default is: “)
- persistent – Enable to keep coordinate mark in the design. (Default is: True)

Returns

Mark created.

createCorridorArea

Syntax

```
createCorridorArea(  
name: str = None,  
/  
*,  
routeCorridor: leRouteCorridor,  
rect: box = None,  
points: list[point] = None,  
pathWidth: float = None,  
startEndStyle: le.createCorridorArea.StartEndStyle = None,  
stopEndStyle: le.createCorridorArea.StopEndStyle = None,  
minLayer: str,  
maxLayer: str)  
Returns: leCorridorArea
```

Summary:

Creates a leCorridorArea object for use with a routing corridor.

Category:

Layout Editing

Licenses:

None

Parameters

- name – Optional name for the corridor area
- routeCorridor – Route corridor to which area will be added.
- rect – Rectangle defining corridor area.
- points – List of points defining either a boundary or path centerline of corridor area.

- `pathWidth` – if given, define path width to be used with points (Default is: 0)
- `startEndStyle` – End style for first point of path ({ `doubleExtend` | `extend` | `truncate` } (Default is: 'truncate')
- `stopEndStyle` – End style for last point of path ({ `doubleExtend` | `extend` | `truncate` } (Default is: 'truncate')
- `minLayer` – Layer name or number for lower most layer for use in area.
- `maxLayer` – Layer name or number for upper most layer for use in area.

Returns

Corridor Area Object

createDRDLineEdit

Syntax

```
createDRDLineEdit(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
prefName: dbPrefName = None,  
toolTip: str = None,  
valueChangeProc: list[variant] = None,  
label: str = None,  
value: str = None)  
Returns: drdLineEdit
```

Summary:

Creates a line edit that has clear text capability and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: "")
- `parent` – Dialog, group or other container into which this widget should be placed.

- **enabled** – Initial enabled state. (Default is: True)
- **shown** – Initial shown state. (Default is: True)
- **prefName** – Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default is: "")
- **toolTip** – Optional widget toolTip. (Default is: "")
- **valueChangeProc** – Procedure to execute when the value is changed. Called with the widget object
- **label** – Label of this line edit widget (Default is: "")
- **value** – initial value. (Default is: "")

Returns

New object.

createDRDTreeWidget

Syntax

```
createDRDTreeWidget(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
toolTip: str = None,  
contextMenuProc: list[variant] = None,  
selectionChangeProc: list[variant] = None,  
doubleClickChangeProc: list[variant] = None,  
valueChangeProc: list[variant] = None,  
flat: bool = None,  
hide: bool = None,  
hint: str = None,  
keywords: str = None)  
Returns: drdTreeWidget
```

Summary:

Creates a new drdTreeWidget widget and adds it to the specified parent

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `toolTip` – Optional widget toolTip. (Default is: “)
- `contextMenuProc` – Procedure to execute when right-clicking on this widget to request a context menu
- `selectionChangeProc` – Procedure to invoke when selection changed
- `doubleClickChangeProc` – Procedure to invoke when double click item value changed
- `valueChangeProc` – Procedure to invoke when value changed
- `flat` – the tree is flat or not (Default is: False)
- `hide` – the tree is invisible or not (Default is: False)
- `hint` – the hint string for tree header
- `keywords` – the keywords string for tree item (Default is: “* :”)

Returns

New object.

createEllipse

Syntax

```
createEllipse(  
box: box,  
/,  
*,  
design: oaDesign,  
lpp: lpp,  
net: oaNet | str = None)  
Returns: oaEllipse
```

Summary:

Create ellipse

Category:

Layout Editing

Licenses:

Basic

Parameters

- box – The coordinates of bounding box of ellipse.
- design – Design in which to create the ellipse.
- lpp – The lpp on which to create the ellipse.
- net – The net name or oaNet object to which ellipse should be assigned.

Returns

Newly created ellipse

createFlightlines

Syntax

```
createFlightlines(  
objects: list[oaGroup] | list[oaNet],  
/,  
*,  
packet: str = None,  
isVisible: bool = None)  
Returns: list[leFlightline]
```

Summary:

Create flightlines.

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – Collection of group or net objects to create flightline objects
- packet – Packet name to assign to flightline objects (Default is: 'y0Fit')
- isVisible – Visibility attribute of flightlineobjects (Default is: True)

Returns

Collection of flightline objects created

createGlobalTrackRegion

Syntax

```
createGlobalTrackRegion(  
    name: str = None,  
    /,  
    *,  
    design: oaDesign,  
    regionDef: leTrackRegionDef,  
    flipX: bool = None,  
    flipY: bool = None,  
    flipFirstX: bool = None,  
    flipFirstY: bool = None,  
    tileOffset: point = None,  
    orient: le.createGlobalTrackRegion.Orient = None)  
Returns: leGlobalTrackRegion
```

Summary:

Creates an leGlobalTrackRegion object in a design.

Category:

Layout Editing

Licenses:

Backend

Parameters

- name – Name of the global region object (Default is: "")
- design – Design in which to create the global region
- regionDef – leTrackRegionDef object used in the global region
- flipX – Determines if tiles in the global region are flipped about the X-axis (Default is: False)
- flipY – Determines if tiles in the global region are flipped about the Y-axis (Default is: False)
- flipFirstX – Determines if the first tile in the gobal region is flipped about the X-axis (Default is: False)

- flipFirstY – Determines if the first tile in the global region is flipped about the Y-axis (Default is: False)
- tileOffset – Offset of the first tile in the global region
- orient – The orientation of the created global track region ({ MX | MY | R0 | R180 } (Default is: 'R0'))

Returns

The newly created leGlobalTrackRegion object

createGuardRing

Syntax

```
createGuardRing(  
points: list[point],  
/  
*,  
design: oaDesign,  
master: dmCellView | dmCellViewName | oaDesign,  
params: list[map_] = None,  
evalCallbacks: bool = None,  
net: oaNet | str = None)  
Returns: oaInst
```

Summary:

Create Guard Ring

Category:

Layout Editing

Licenses:

Basic

Parameters

- points – The list of points that form guard ring.
- design – Design in which to create the guard ring.
- master – SuperMaster of guard ring.
- params – Parameters of editable pcell/mcell instance. Each item should have {parameter name, parameter value} format.

- `evalCallbacks` – Should callbacks be triggered or not. This option is for editable pcell guard ring only. (Default is: True)
- `net` – The net name or `oaNet` object to which guard ring should be assigned.

Returns

Newly created guard ring instance

createICCIInst

Syntax

```
createICCIInst(  
    name: str,  
    /,  
    *,  
    design: oaDesign,  
    master: oaDesign = None,  
    libName: str = None,  
    cellName: str = None,  
    viewName: str = None)  
Returns: oaInst
```

Summary:

Creates a new cell instance in ICC design

Category:

ICC Link

Licenses:

None

Parameters

- `name` – Instance name including optional module hierarchy
- `design` – The design where the instance should be created
- `master` – The master of the instance to be created
- `libName` – The name of the master cell library
- `cellName` – The name of the master cell
- `viewName` – The view name of the master cell (Default is: 'abstract')

Returns

Newly created object

createICNet

Syntax

```
createICNet(  
name: str,  
/,  
*,  
design: oaDesign,  
sigType: oaSigType = None)  
Returns: oaNet
```

Summary:

Creates a new net in ICC design

Category:

ICC Link

Licenses:

None

Parameters

- name – Tcl list of net names including optional module hierarchy
- design – The design where the net should be created
- sigType – Signal type for the new net

Returns

Newly created object

createInst

Syntax

```
createInst(  
name: str = None,  
/,  
*,  
master: oaDesign = None,  
libName: str = None,  
cellName: str = None,  
viewName: str = None,  
design: oaDesign,  
orient: le.createInst.Orient = None,  
origin: point,  
rows: int = None,
```

```
cols: int = None,  
dx: float = None,  
dy: float = None,  
status: le.createInst.Status = None,  
params: list[map_] = None,  
evalCallbacks: bool = None)  
Returns: oaInst
```

Summary:

Create instance

Category:

Layout Editing

Licenses:

Basic

Parameters

- name – Name of instance.
- master – Master of instance.
- libName – Name of master cell library. (Default is: 'library of current cellView')
- cellName – Name of master cell. Should be specified, if -master not specified.
- viewName – Name of master cellView. (Default is: 'layout')
- design – Design in which to create the instance.
- orient – Orientation of the instance. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 }) (Default is: 'R0')
- origin – Location of the origin of instance.
- rows – Number of rows. (Default is: 1)
- cols – Number of columns. (Default is: 1)
- dx – Delta between instances in X direction. (Default is: 'instance width')
- dy – Delta between instances in Y direction. (Default is: 'instance height')
- status – Placement status for instance. ({ fixed | locked | none | placed | unplaced }) (Default is: 'placed')
- params – Properties of instance (or parameters of pcell instance). Each item should have {property name, property value} format.
- evalCallbacks – Should callbacks be triggered or not. (Default is: True)

Returns

Newly created instance

createLabel

Syntax

```
createLabel(  
    string: str,  
    /,  
    *,  
    parent: deFigure | oaDesign | oaFig,  
    lpp: lpp,  
    origin: point,  
    just: le.createLabel.Just = None,  
    orient: le.createLabel.Orient = None,  
    font: le.createLabel.Font = None,  
    polygonFont: le.createLabel.PolygonFont = None,  
    spacing: float = None,  
    height: float = None,  
    drafting: bool = None,  
    overbar: bool = None)  
Returns: oaText
```

Summary:

Create label

Category:

Layout Editing

Licenses:

Basic

Parameters

- string – Text of the label
- parent – The design in which the label should be created, or the parent to which the label should be attached.
- lpp – The lpp on which to create the text.
- origin – Location of the origin of text.
- just – Justification of the text with respect to the origin. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight }) (Default is: 'centerCenter')

- orient – Orientation of the text. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 } (Default is: 'R0')
- font – Font style with which the text is displayed. ({ euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish } (Default is: 'stick')
- polygonFont – Font style of the polygon with which the text is displayed. ({ contemplate | merit | simple } (Default is: 'simple')
- spacing – Spacing of the text in user units. (Default is: 0)
- height – Height of the text in user units. (Default is: 1)
- drafting – Indicates whether the text should not be mirrored or rotated. (Default is: True)
- overbar – Indicates whether the text will be rendered with overbar. (Default is: False)

Returns

New label object (oaText)

createMPP

Syntax

```
createMPP(  
name: str = None,  
/,  
*,  
design: oaDesign,  
definition: str,  
points: list[point],  
lines: list[point] = None,  
properties: list[str] = None)  
Returns: leMPP
```

Summary:

This command creates a Multi Part Path.

Category:

Layout Editing

Licenses:

Basic

Parameters

- name – The name of Multi Part Path. (Default is: “)
- design – Design in which to create the mpp.
- definition – Definition for MPP.
- points – List of points to create a MPP object.
- lines – List of lines of the created MPP object, to create a MPP with lines.
- properties – List of properties for MPP object.

Returns

Newly created MPP object

createMPPDef

Syntax

```
createMPPDef(  
name: str,  
/,  
*,  
in_: oaTech = None,  
design: oaDesign = None,  
rings: list[leMPPRingDef | str | list[oaAppObject]],  
spacings: list[float] = None,  
spacing: list[int] = None,  
masterIndex: int = None,  
masterindex: int = None,  
exposedParams: list[str] = None)  
Returns: leMPPDef
```

Summary:

Create MPP definition

Category:

Data Management

Licenses:

None

Parameters

- name – The name for the MPP definition to create
- in_ – Technology in which to create the MPP definition

- design – Technology in which to create the MPP definition
- rings – The list of rings definition names or rings definitions
- spacings – The spacing list for each ring definition, corresponding to the ring list
- spacing – The spacing list for each ring definition, corresponding to the ring list
- masterIndex – The index indicating which ring is the master (Default is: 0)
- masterindex – The index indicating which ring is the master (Default is: 0)
- exposedParams – The list of exposed parameters for the MPP definition

Returns

Newly created MPP definition object

createMPPEnclosedPathDef

Syntax

```
createMPPEnclosedPathDef(  
name: str,  
/,  
*,  
in_: oaTech = None,  
design: oaDesign = None,  
layer: str,  
purpose: str = None,  
enclosure: float,  
endOfLineEnclosure: float = None,  
beginOffset: float = None,  
endOffset: float = None,  
connectivity: bool = None,  
conn: bool = None,  
chopable: bool = None,  
chop: bool = None,  
extension: le.createMPPEnclosedPathDef.Extension = None,  
exposedParams: list[str] = None)  
Returns: leMPPEnclosedPathDef
```

Summary:

Create MPP Enclosed Path definition

Category:

Data Management

Licenses:

None

Parameters

- name – The name for the MPP Enclosed Path definition to create
- in_ – Technology in which to create the MPP Enclosed Path definition
- design – Technology in which to create the MPP Enclosed Path definition
- layer – The layer on which to create each shape
- purpose – The purpose on which to create each shape (Default is: 'drawing')
- enclosure – The enclosure over the master path
- endOfLineEnclosure – The endOfLine enclosure over the master path. If not specified, enclosure value will be used.
- beginOffset – The begin offset from the master path
- endOffset – The end offset from the master path
- connectivity – Specifies whether the shape is connected to the ring net (Default is: False)
- conn – Specifies whether the shape is connected to the ring net (Default is: False)
- chopable – Specifies the whether the shape is chopable (Default is: False)
- chop – Specifies the whether the shape is chopable (Default is: False)
- extension – The begin/end extension style for each path shape ({ extend | round | truncate | variable } (Default is: 'truncate')
- exposedParams – The list of exposed parameters for the MPP Enclosed Path definition

Returns

Newly created MPP Enclosed Path definition object

createMPPEnclosedShapeDef

Syntax

```
createMPPEnclosedShapeDef(  
    name: str,  
    /,  
    *,  
    in_: oaTech = None,
```

```
design: oaDesign = None,  
layer: str,  
purpose: str = None,  
enclosure: float,  
connectivity: bool = None,  
conn: bool = None,  
exposedParams: list[str] = None)  
Returns: leMPPEnclosedShapeDef
```

Summary:

Create MPP Enclosed Shape definition

Category:

Data Management

Licenses:

None

Parameters

- name – The name for the MPP Enclosed Shape definition to create
- in_ – Technology in which to create the MPP Enclosed Shape definition
- design – Technology in which to create the MPP Enclosed Shape definition
- layer – The layer on which to create each shape
- purpose – The purpose on which to create each shape (Default is: 'drawing')
- enclosure – The enclosure over the master path
- connectivity – Specifies whether the shape is connected to the ring net (Default is: False)
- conn – Specifies whether the shape is connected to the ring net (Default is: False)
- exposedParams – The list of exposed parameters for the MPP Enclosed Shape definition

Returns

Newly created MPP Enclosed Shape definition object

createMPPMasterPathDef

Syntax

```
createMPPMasterPathDef(  
name: str,
```

```
/,
*,
in_: oaTech = None,
design: oaDesign = None,
layer: str,
purpose: str = None,
width: float,
connectivity: bool = None,
conn: bool = None,
chopable: bool = None,
chop: bool = None,
extension: le.createMPPMasterPathDef.Extension = None,
exposedParams: list[str] = None)
Returns: leMPPMasterPathDef
```

Summary:

Create MPP Master Path definition

Category:

Data Management

Licenses:

None

Parameters

- name – The name for the MPP Master Path definition to create
- in_ – Technology in which to create the MPP Master Path definition
- design – Technology in which to create the MPP Master Path definition
- layer – The layer on which to create each shape
- purpose – The purpose on which to create each shape (Default is: 'drawing')
- width – The width of each path shape
- connectivity – Specifies whether the shape is connected to the ring net (Default is: False)
- conn – Specifies whether the shape is connected to the ring net (Default is: False)
- chopable – Specifies the whether the shape is chopable (Default is: False)
- chop – Specifies the whether the shape is chopable (Default is: False)
- extension – The begin/end extension style for each path shape ({ extend | round | truncate | variable } (Default is: 'truncate')
- exposedParams – The list of exposed parameters for the MPP Master Path definition

Returns

Newly created MPP Master Path definition object

createMPPOffsetPathDef

Syntax

```
createMPPOffsetPathDef(  
    name: str,  
    /,  
    *,  
    in_: oaTech = None,  
    design: oaDesign = None,  
    layer: str,  
    purpose: str = None,  
    width: float,  
    offset: float = None,  
    beginOffset: float = None,  
    endOffset: float = None,  
    connectivity: bool = None,  
    conn: bool = None,  
    chopable: bool = None,  
    chop: bool = None,  
    extension: le.createMPPOffsetPathDef.Extension = None,  
    exposedParams: list[str] = None)  
Returns: leMPPOffsetPathDef
```

Summary:

Create MPP Offset Path definition

Category:

Data Management

Licenses:

None

Parameters

- name – The name for the MPP Offset Path definition to create
- in_ – Technology in which to create the MPP Offset Path definition
- design – Technology in which to create the MPP Offset Path definition
- layer – The layer on which to create each shape
- purpose – The purpose on which to create each shape (Default is: 'drawing')

- width – The width of each path shape
- offset – The offset from the master path (Default is: 0)
- beginOffset – The begin offset from the master path
- endOffset – The end offset from the master path
- connectivity – Specifies whether the shape is connected to the ring net (Default is: False)
- conn – Specifies whether the shape is connected to the ring net (Default is: False)
- chopable – Specifies the whether the shape is chopable (Default is: False)
- chop – Specifies the whether the shape is chopable (Default is: False)
- extension – The begin/end extension style for each path shape ({ extend | round | truncate | variable } (Default is: 'truncate'))
- exposedParams – The list of exposed parameters for the MPP Offset Path definition

Returns

Newly created MPP Offset Path definition object

createMPPRingDef

Syntax

```
createMPPRingDef(  
name: str,  
/,  
*,  
in_: oaTech = None,  
design: oaDesign = None,  
netName: str = None,  
netname: str = None,  
masterPath: leMPPMasterPathDef | str,  
masterpath: str = None,  
enclosedPaths: list[leMPPEnclosedPathDef | str | list[oaAppObject]]  
= None,  
enclosedpaths: list[str] = None,  
offsetPaths: list[leMPPOffsetPathDef | str | list[oaAppObject]] =  
None,  
offsetpaths: list[str] = None,  
subRects: list[leMPPSubRectDef | str | list[oaAppObject]] = None,  
subrects: list[str] = None,  
enclosedShapes: list[leMPPEnclosedShapeDef | str |  
list[oaAppObject]] = None,  
enclosedshapes: list[str] = None,
```

```
exposedParams: list[str] = None)  
Returns: leMPPRingDef
```

Summary:

Create MPP Ring definition

Category:

Data Management

Licenses:

None

Parameters

- name – The name for the MPP Ring definition to create
- in_ – Technology in which to create the MPP Ring definition
- design – Technology in which to create the MPP Ring definition
- netName – The default net name for the MPP Ring
- netname – The default net name for the MPP Ring
- masterPath – The master path definition name or master path definition
- masterpath – The master path definition name or master path definition
- enclosedPaths – The list of enclosed path definition names or enclosed path definitions
- enclosedpaths – The list of enclosed path definition names or enclosed path definitions
- offsetPaths – The list of offset path definition names or offset path definitions
- offsetpaths – The list of offset path definition names or offset path definitions
- subRects – The list of sub rectangle definition names or sub rectangle definitions
- subrects – The list of sub rectangle definition names or sub rectangle definitions
- enclosedShapes – The list of enclosed shapes definition names or enclosed shapes definitions
- enclosedshapes – The list of enclosed shapes definition names or enclosed shapes definitions
- exposedParams – The list of exposed parameters for the MPP Ring definition

Returns

Newly created MPP Ring definition object

createMPPSubRectDef

Syntax

```
createMPPSubRectDef(  
name: str,  
/  
*,  
in_: oaTech = None,  
design: oaDesign = None,  
layer: str,  
purpose: str = None,  
width: float,  
height: float,  
offset: float = None,  
beginOffset: float = None,  
endOffset: float = None,  
connectivity: bool = None,  
conn: bool = None,  
chopable: bool = None,  
chop: bool = None,  
minEnclosure: float = None,  
minenclosure: float = None,  
endOfLine: float = None,  
eol: float = None,  
spacing: float = None,  
numRows: int = None,  
numrows: int = None,  
spacingType: le.createMPPSubRectDef.SpacingType = None,  
spacetype: le.createMPPSubRectDef.Spacetype = None,  
exposedParams: list[str] = None)  
Returns: leMPPSubRectDef
```

Summary:

Create MPP Sub Rectangle definition

Category:

Data Management

Licenses:

None

Parameters

- name – The name for the MPP Sub Rectangle definition to create
- in_ – Technology in which to create the MPP Sub Rectangle definition
- design – Technology in which to create the MPP Sub Rectangle definition

- **layer** – The layer on which to create each shape
- **purpose** – The purpose on which to create each shape (Default is: 'drawing')
- **width** – width of each sub rectangle
- **height** – height of each sub rectangle
- **offset** – The offset from the master path (Default is: 0)
- **beginOffset** – The begin offset from the master path
- **endOffset** – The end offset from the master path
- **connectivity** – Specifies whether the shape is connected to the ring net (Default is: False)
- **conn** – Specifies whether the shape is connected to the ring net (Default is: False)
- **chopable** – Specifies the whether the shape is chopable (Default is: False)
- **chop** – Specifies the whether the shape is chopable (Default is: False)
- **minEnclosure** – The minimum enclosure over the sub rectangles. If not specified, min enclosure of LPP will be used.
- **minenclosure** – The minimum enclosure over the sub rectangles. If not specified, min enclosure of LPP will be used.
- **endOfLine** – The end of line extension past the sub rectangles. If not specified, min enclosure of LPP will be used.
- **eol** – The end of line extension past the sub rectangles. If not specified, min enclosure of LPP will be used.
- **spacing** – The spacing between each sub rectangle. If not specified, min spacing of LPP will be used
- **numRows** – The number of rows of sub rectangles (Default is: 1)
- **numrows** – The number of rows of sub rectangles (Default is: 1)
- **spacingType** – The spacing type between sub rectangles ({ fit | fitBegin | min | minBegin }) (Default is: 'fit')
- **spacetype** – The spacing type between sub rectangles ({ fit | fitBegin | min | minBegin })
- **exposedParams** – The list of exposed parameters for the MPP Sub Rectangle definition

Returns

Newly created MPP Sub Rectangle definition object

createNetGroup

Syntax

```
createNetGroup(  
  objects: list[oaNet],  
  //  
  *,  
  name: str)  
Returns: oaGroup
```

Summary:

Create net group.

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – Collection of nets to include in created group
- name – Name of group to create

Returns

Created group

createOffTrackInfo

Syntax

```
createOffTrackInfo(  
  object: deFigure | oaFig)  
Returns: leOffTrackInfo
```

Summary:

Create off track info object

Category:

Layout Editing

Licenses:

Basic

Parameters

object – Object to create offTrackInfo.

Returns

Off-track information object.

createPath

Syntax

```
createPath(  
points: list[point],  
/  
*,  
design: oaDesign,  
width: float,  
lpp: lpp,  
style: le.createPath.Style = None,  
beginExtendVal: float = None,  
endExtendVal: float = None,  
net: oaNet | str = None,  
checkIntersection: bool = None)  
Returns: oaPath
```

Summary:

Create path

Category:

Layout Editing

Licenses:

Basic

Parameters

- points – The list of points that form path.
- design – Design in which to create the path.
- width – Path width
- lpp – The lpp on which to create the path.
- style – The begin/end style for path ({ extend | round | truncate | variable })
- beginExtendVal – Beginning extension for variable style
- endExtendVal – End extension for variable style

- `net` – The net name or `oaNet` object to which path should be assigned.
- `checkIntersection` – Check self-intersection of a path's centerline or not when creating paths. (Default is: `True`)

Returns

Newly created path

createPathSeg

Syntax

```
createPathSeg(  
points: list[point],  
/,  
*,  
design: oaDesign,  
width: float,  
lpp: lpp,  
beginStyle: le.createPathSeg.BeginStyle = None,  
beginExt: float = None,  
beginLDiag: float = None,  
beginRDiag: float = None,  
beginRHalfWidth: float = None,  
endStyle: le.createPathSeg.EndStyle = None,  
endExt: float = None,  
endLDiag: float = None,  
endRDiag: float = None,  
endRHalfWidth: float = None,  
net: oaNet | str = None)  
Returns: oaPathSeg
```

Summary:

Create path segment

Category:

Layout Editing

Licenses:

Basic

Parameters

- `points` – The list of points that form `path_segment`.
- `design` – Design in which to create the `path_segment`.
- `width` – Path width

- lpp – The lpp on which to create the path_segment.
- beginStyle – The begin style for path segment ({ chamfer | custom | extend | truncate | variable })
- beginExt – Beginning extension
- beginLDiag – Begin extension's left diagonal extension
- beginRDiag – Begin extension's right diagonal extension
- beginRHalfWidth – Begin extension's right half width
- endStyle – The end style for path segment ({ chamfer | custom | extend | truncate | variable })
- endExt – End extension
- endLDiag – End extension's left diagonal extension
- endRDiag – End extension's right diagonal extension
- endRHalfWidth – End extension's right half width
- net – The net name or oaNet object to which path_segment should be assigned.

Returns

Newly created path segment

createPin

Syntax

```
createPin(  
name: str = None,  
/,  
*,  
design: oaDesign = None,  
term: oaTerm | str,  
shapes: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia],  
accessDir: int = None,  
status: le.createPin.Status = None)  
Returns: oaPin
```

Summary:

Create pin

Category:

Layout Editing

Licenses:

Basic

Parameters

- name – Name of pin.
- design – Design in which to create the pin.
- term – Terminal, or terminal name, of pin.
- shapes – Shapes to be attached to pin.
- accessDir – Access direction for pin. Any combination of none:0, top:1, bottom:2, left:4, right:8. (Default is: 15)
- status – Placement status for pin. ({ fixed | locked | none | placed | unplaced }) (Default is: 'placed')

Returns

Newly created pin

createPinsFromText

Syntax

```
createPinsFromText (  
*,  
    design: list[dmCellView] | list[oaDesign] | list[oaDesign])  
Returns: None
```

Summary:

create pins from overlapped texts

Category:

Layout Editing

Licenses:

Basic

Parameters

design – design or cellView object

Returns

None

createPolygon

Syntax

```
createPolygon(  
points: list[point],  
/,  
*,  
net: oaNet | str = None,  
design: oaDesign,  
lpp: lpp)  
Returns: oaPolygon
```

Summary:

Create polygon

Category:

Layout Editing

Licenses:

Basic

Parameters

- points – The list of points that form polygon.
- net – The net name or oaNet object to which polygon should be assigned.
- design – Design in which to create the polygon.
- lpp – The lpp on which to create the polygon.

Returns

Newly created polygon

createRectangle

Syntax

```
createRectangle(  
box: box,  
/,  
*,
```

```
design: oaDesign,  
lpp: lpp,  
net: oaNet | str = None,  
color: str = None,  
lockColor: bool = None)  
Returns: oaRect
```

Summary:

Create rectangle

Category:

Layout Editing

Licenses:

Basic

Parameters

- box – The coordinates of bounding box of rectangle.
- design – Design in which to create the rectangle.
- lpp – The lpp on which to create the rectangle.
- net – The net name or oaNet object to which rectangle should be assigned.
- color – The color of the rectangle. (Default is: 'noColor')
- lockColor – Specify if lock color for created rectangle. (Default is: False)

Returns

Newly created rectangle

createRouteCorridor

Syntax

```
createRouteCorridor(  
name: str,  
/,  
*,  
design: oaDesign,  
nets: list[oaNet] = None)  
Returns: leRouteCorridor
```

Summary:

Creates a route corridor in the design.

Category:

Layout Editing

Licenses:

None

Parameters

- name – Name of routing corridor
- design – The design in which to create the routing corridor.
- nets – Collection of nets to assign to the routing corridor.

Returns

Newly created routing corridor object

createRow

Syntax

```
createRow(  
points: list[point],  
/,  
*,  
design: oaDesign,  
siteDef: oaScalarSiteDef | str,  
name: str = None,  
direction: bool = None,  
rowOrigin: le.createRow.RowOrigin = None,  
siteSpacing: float = None,  
rowSpacing: float = None,  
coreToleftSpacing: float = None,  
coreTobottomSpacing: float = None,  
coreTorightSpacing: float = None,  
coreTotopSpacing: float = None,  
rowPairing: bool = None,  
startFromFirstRow: bool = None,  
flipFirstRow: bool = None)  
Returns: oaRow
```

Summary:

Create row

Category:

Layout Editing

Licenses:

Basic

Parameters

- points – The list of points that form row.
- design – Design in which to create the row.
- siteDef – Site definition or its name, for new object.
- name – The prefix of row name (Default is: 'R')
- direction – Row grow direction, Horizontal if true or Vertical if false (Default is: True)
- rowOrigin – The row origin ({ leftbottom | lefttop | rightbottom | righttop } (Default is: 'leftbottom'))
- siteSpacing – The spacing between the sites. (Default is: 0)
- rowSpacing – The spacing between the rows. (Default is: 0)
- coreToLeftSpacing – The left spacing between row region and boundary. (Default is: 0)
- coreToBottomSpacing – The bottom spacing between row region and boundary. (Default is: 0)
- coreToRightSpacing – The right spacing between row region and boundary. (Default is: 0)
- coreToTopSpacing – The top spacing between row region and boundary. (Default is: 0)
- rowPairing – The pairing flag of rows (Default is: False)
- startFromFirstRow – The start first flag of pairing rows (Default is: False)
- flipFirstRow – The flip first flag of pairing rows (Default is: False)

Returns

Collection of new rows.

createSiteDef

Syntax

```
createSiteDef(  
    name: str,  
    /,  
    *,  
    design: oaDesign,
```

```
type: le.createSiteDef.Type = None,  
width: float,  
height: float,  
syminx: bool = None,  
syminy: bool = None,  
syminrot: bool = None)  
Returns: oaSiteDef
```

Summary:

Create Site Definition.

Category:

Layout Editing

Licenses:

Basic

Parameters

- name – The name of the siteDef to create
- design – Design in which to create the site_def.
- type – The type of siteDef to create ({ core | pad })
- width – The width of the siteDef to create
- height – The height of the siteDef to create
- syminx – The X symmetry of the siteDef to create (Default is: False)
- syminy – The Y symmetry of the siteDef to create (Default is: False)
- syminrot – The rotation symmetry of the siteDef to create (Default is: False)

Returns

Newly created site def object

createStripedWire

Syntax

```
createStripedWire(  
segments: list[leStripeSegment],  
/  
*,  
design: oaDesign = None,  
net: oaNet | str = None,  
beginStyle: le.createStripedWire.BeginStyle = None,
```

```
endStyle: le.createStripedWire.EndStyle = None,  
beginExtendVal: float = None,  
endExtendVal: float = None,  
beginStrap: bool = None,  
endStrap: bool = None,  
slot: bool = None,  
stagger: bool = None)  
Returns: leStripedWire
```

Summary:

Create striped wire

Category:

Layout Editing

Licenses:

Backend

Parameters

- segments – List of stripe segments.
- design – Design in which to create the striped wire.
- net – Net name or oaNet.
- beginStyle – Begin style of striped wire. ({ extend | truncate | variable })
- endStyle – End style of striped wire. ({ extend | truncate | variable })
- beginExtendVal – Begin extension of striped wire.
- endExtendVal – End extension of striped wire.
- beginStrap – Begin strap of striped wire.
- endStrap – End strap of striped wire.
- slot – slot of striped wire.
- stagger – stagger of striped wire.

Returns

Striped wire created

createStripeSegment

Syntax

```
createStripeSegment(  
points: list[point],  
/  
*,  
width: float = None,  
spacing: float = None,  
stripes: int = None,  
lpp: lpp,  
via: list[oaViaDef | str] = None,  
offset: float = None,  
slotLength: float = None,  
slotSpacing: float = None)  
Returns: leStripeSegment
```

Summary:

Create a tcl stripe segment object

Category:

Layout Editing

Licenses:

Backend

Parameters

- points – Point list of stripe segment.
- width – Stripe segment width.
- spacing – Stripe segment spacing.
- stripes – Stripe segment number.
- lpp – LPP of stripe segment.
- via – Via definition name or oaViaDef.
- offset – Stripe segment offset.
- slotLength – Stripe segment slot length.
- slotSpacing – Stripe segment slot spacing.

Returns

Stripe segment created

createSyncFigGroup

Syntax

```
createSyncFigGroup(  
    name: str = None,  
    /,  
    *,  
    in_: oaGroup | str,  
    objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
            list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
            | list[oaVia],  
    source: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
            list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
            | list[oaVia] = None,  
    objectsOrder: list[deFigure] | list[deSelectionSet] | list[oaFig]  
                  | list[oaFigGroup] | list[oaInst] | list[oaMarker] |  
                  list[oaShape] | list[oaVia] = None,  
    sourceOrder: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
                  list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
                  | list[oaVia] = None)  
Returns: oaFig
```

Summary:

Creates an oaFigGroup from the given layout objects and adds them to the specified oaGroup.

Category:

Layout Editing

Licenses:

Backend

Parameters

- name – Name of the oaFigGroup.
- in_ – Synchronous group that oaFigGroup is to be added to.
- objects – Collection of oaFig/deFigure objects or a single oaFig/deFigure to create the oaFigGroup with.
- source – Collection of oaFig/deFigure objects or a single oaFig/deFigure from one of the peer oaFigGroups to associate correspondence.
- objectsOrder – Order of oaFig/deFigure objects to sync the oaFigGroup with.
- sourceOrder – Order of oaFig/deFigure objects from one of the peer oaFigGroups to associate correspondence.

Returns

oaFig created by the command.

createSyncGroup

Syntax

```
createSyncGroup(  
name: str = None,  
/,  
*,  
design: oaDesign)  
Returns: oaGroup
```

Summary:

Creates an oaGroup in the given design.

Category:

Layout Editing

Licenses:

Backend

Parameters

- name – Name of the synchronous oaGroup.
- design – Layout design where oaGroup is to be created.

Returns

The oaGroup object created by the command.

createTerm

Syntax

```
createTerm(  
name: str = None,  
/,  
*,  
net: oaNet | str,  
design: oaDesign = None,  
type: le.createTerm.Type = None,  
assignmentName: str = None,  
defaultNet: oaNet | str = None)  
Returns: oaTerm
```

Summary:

Create term

Category:

Layout Editing

Licenses:

Basic

Parameters

- name – The name for terminal being created.
- net – The net name or oaNet object to which terminal should be created.
- design – The design, where net should be found (if net name is provided).
- type – The type of term being created. ({ input | inputOutput | jumper | output | switch | tristate | unused } (Default is: 'inputOutput'))
- assignmentName – The name of connect definition for terminal.
- defaultNet – Default net or it's name of connect definition.

Returns

Newly created terminal object

createTrack

Syntax

```
createTrack(  
name: str,  
/  
*,  
displayName: str = None,  
in_: oaDesign | oaTech,  
trackType: le.createTrack.TrackType,  
offset: float,  
width: float = None,  
color: str = None,  
visible: bool = None,  
active: bool = None,  
signalType: le.createTrack.SignalType = None,  
gridOffsetLow: float = None,  
gridOffsetHigh: float = None,  
gridPointsLow: list[float] = None,  
gridPointsHigh: list[float] = None,  
)
```

```
gridVisibleLow: bool = None,  
gridVisibleHigh: bool = None,  
**kwargs)  
Returns: leTrack
```

Summary:

Creates an in-memory leTrack object.

Category:

Layout Editing

Licenses:

Basic

Parameters

- name – Name of the leTrack object
- displayName – Display Name of the leTrack object (Default is: "")
- in_ – Design or tech in which to create the track
- trackType – Track type ({ default | nonReserved | reserved })
- offset – Offset of track from region tile lower left
- width – Width of the track (Default is: 0)
- color – Color of the track (Default is: 'noColor')
- visible – Default visible state of track (Default is: False)
- active – Default active state of track (Default is: True)
- signalType – Restricts nets that can snap to track based on net sigType. ({ all | analog | clock | ground | power | reset | scan | signal | tieHi | tieLo | tieoff }) (Default is: 'all')
- gridOffsetLow – Offset for grid starting points (Default is: 0)
- gridOffsetHigh – Offset for grid ending points (Default is: 0)
- gridPointsLow – Starting points of grid
- gridPointsHigh – Ending points of grid
- gridVisibleLow – Grid visible low of track (Default is: True)
- gridVisibleHigh – Grid visible high of track (Default is: True)

Returns

The newly created leTrack object

createTrackGroupDef

Syntax

```
createTrackGroupDef(  
    name: str,  
    /,  
    *,  
    displayName: str = None,  
    in_: oaDesign | oaTech,  
    tracks: dbCollection,  
    layer: str,  
    direction: le.createTrackGroupDef.Direction,  
    pitch: float | point,  
    allowCrossBoundary: bool = None,  
    renderPurpose: str = None,  
    LEGShapeSnapPurposes: str = None,  
    **kwargs)  
Returns: leTrackGroupDef
```

Summary:

Creates an leTrackGroupDef object in a technology or design database.

Category:

Layout Editing

Licenses:

Basic

Parameters

- name – trackGroupDef name
- displayName – trackGroupDef display name (Default is: "")
- in_ – Tech or design in which to create the def
- tracks – Collection of leTrack objects
- layer – Layer of the trackGroupDef (all tracks in the groupDef will be on this layer)
- direction – Direction of the tracks in the def ({ horizontal | vertical })
- pitch – Pitch of group (both X and Y directions)

- **allowCrossBoundary** – Determines if tracks from group can cross region boundary (Default is: True)
- **renderPurpose** – Purpose to be used to render tracks. Default value is "" in which case 'drawing' purpose will be used (Default is: "")
- **LEGShapeSnapPurposes** – Only shapes with the purposes defined can snap to track LEGs. Default value is "" meaning all purposes are allowed (Default is: "")

Returns

The newly created leTrackGroupDef object

createTrackPattern

Syntax

```
createTrackPattern(  
name: str,  
/,  
*,  
design: deContext | oaDesign,  
from_: leTrackPattern = None,  
of: leTrackPattern = None,  
trackPatternDef: leTrackPatternDef = None,  
color: str = None,  
box: box = None,  
blockPitch: float = None,  
offset: float = None,  
visible: bool = None,  
active: bool = None,  
gridOffsetLow: float = None,  
gridOffsetHigh: float = None,  
gridPointsLow: list[float] = None,  
gridPointsHigh: list[float] = None)  
Returns: leTrackPattern
```

Summary:

Creates an leTrackPattern in a design database.

Category:

Layout Editing

Licenses:

Basic

Parameters

- name – TrackPattern name
- design – Design in which to create the pattern
- of – Reference for unspecified options
- trackPatternDef – Definition for this pattern
- color – Color of pattern
- box – Optional pattern extend
- blockPitch – Block pitch in user units
- offset – Distance from design origin to first track
- visible – Default visibility of tracks in this pattern (Default is: True)
- active – Default activity for tracks in this pattern (Default is: False)
- gridOffsetLow – Offset for grid starting points (Default is: 0)
- gridOffsetHigh – Offset for grid ending points
- gridPointsLow – Starting points of grid
- gridPointsHigh – Ending points of grid

Returns

The newly created leTrackPattern

createTrackPatternDef

Syntax

```
createTrackPatternDef(  
name: str,  
/,  
*,  
from_: leTrackPatternDef = None,  
of: leTrackPatternDef = None,  
orientation: le.createTrackPatternDef.Orientation = None,  
layer: str = None,  
purpose: str = None,  
in_: deContext | oaDesign | oaTech,  
patternType: le.createTrackPatternDef.PatternType = None,  
trackPitch: float,  
offset: float = None,  
numTracks: int = None,  
blockPitch: float = None,  
)
```

```
width: float = None,  
visible: bool = None,  
active: bool = None,  
color: str = None,  
autoGenerateTrack: bool = None)  
Returns: leTrackPatternDef
```

Summary:

Creates an leTrackPatternDef in a technology or design database.

Category:

Layout Editing

Licenses:

Basic

Parameters

- name – TrackPatternDef name
- of – Reference for unspecified options
- orientation – Pattern Def orientation: Horizontal or Vertical ({ horizontal | vertical })
- layer – The layer which snaps to this pattern.
- purpose – Purpose on the given layer which snaps to this pattern (Default is: 'oaAny')
- in_ – Design or Tech in which to create the Def
- patternType – Pattern Type ({ default | nonReserved | reserved } (Default is: 'default'))
- trackPitch – Track pitch.
- offset – Offset of first track from design origin (Default is: 0)
- numTracks – Number of tracks in a block (Default is: 0)
- blockPitch – Block pitch in user units (Default is: 0)
- width – Width parameter for non-reserved and reserved patterns (Default is: 0)
- visible – Default visibility of tracks in this pattern (Default is: True)
- active – Default activity for tracks in this pattern (Default is: False)
- color – Color of pattern definition (Default is: 'noColor')
- autoGenerateTrack – The auto generate track flag of the track pattern definition (Default is: True)

Returns

Newly created leTrackPatternDef

createTrackRegion

Syntax

```
createTrackRegion(  
    name: str = None,  
    /,  
    *,  
    design: oaDesign,  
    regionDef: leTrackRegionDef,  
    points: list[point],  
    flipX: bool = None,  
    flipY: bool = None,  
    flipFirstX: bool = None,  
    flipFirstY: bool = None,  
    offsetTarget: le.createTrackRegion.OffsetTarget = None,  
    tileOffset: point = None,  
    visible: bool = None,  
    orient: le.createTrackRegion.Orient = None,  
    **kwargs)  
Returns: leTrackRegion
```

Summary:

Creates an leTrackRegion in a design database.

Category:

Layout Editing

Licenses:

Backend

Parameters

- name – Name of the region object (Default is: “)
- design – Design in which to create the region
- regionDef – leTrackRegionDef object used in the region
- points – List of points
- flipX – Determines if tiles in the region are flipped about the X-axis (Default is: False)
- flipY – Determines if tiles in the region are flipped about the Y-axis (Default is: False)

- flipFirstX – Determines if the first tile in the region is flipped about the X-axis (Default is: False)
- flipFirstY – Determines if the first tile in the region is flipped about the Y-axis (Default is: False)
- offsetTarget – Determines if the offset of tracks is relative to the design axis or region boundary ({ design | region } (Default is: 'design'))
- tileOffset – Offset of the first tile in the region
- visible – Determines if semantic oaAreaBoundary can be drawn (independent of deObjectFilter) (Default is: True)
- orient – The orientation of the created track region ({ MX | MY | R0 | R180 } (Default is: 'R0'))

Returns

The newly created leTrackRegion object

createTrackRegionDef

Syntax

```
createTrackRegionDef(  
    name: str,  
    /,  
    *,  
    in_: oaDesign | oaTech,  
    groupDefs: list[oaAppObject],  
    flipX: bool = None,  
    flipXGroups: list[oaAppObject] = None,  
    flipY: bool = None,  
    flipYGroups: list[oaAppObject] = None,  
    flipTilesX: bool = None,  
    flipTilesY: bool = None,  
    flipTilesXFirst: bool = None,  
    flipTilesYFirst: bool = None,  
    minCountX: int = None,  
    minCountY: int = None,  
    maxCountX: int = None,  
    maxCountY: int = None,  
    snapGrid: point = None,  
    snapOffset: point = None,  
    regionDefType: le.createTrackRegionDef.RegionDefType = None,  
    autoGenerateGlobalRegion: bool = None,  
    enclosedGroups: list[oaAppObject] = None,  
    groupEnclosure: point = None,  
    defaultTracksVisible: int = None,  
    defaultTracksActive: int = None,  
)
```

```
flipGroupsFirst: list[oaAppObject] = None,  
groupFlipTarget: le.createTrackRegionDef.GroupFlipTarget = None,  
**kwargs)  
Returns: leTrackRegionDef
```

Summary:

Creates an leTrackRegionDef in a technology or design database.

Category:

Layout Editing

Licenses:

Basic

Parameters

- name – trackRegionDef name
- in_ – Design or Tech in which to create the Def
- groupDefs – Collection of leTrackGroupDef objects
- flipX – Determines if groups in the def are flipped about the X-axis (Default is: False)
- flipXGroups – Collection of leTrackGroupDef to flip in X
- flipY – Determines if groups in the def are flipped about the Y-axis (Default is: False)
- flipYGroups – Collection of leTrackGroupDef to flip in Y
- flipTilesX – Determines default value of flipX when creating leTrackRegion / leGlobalTrackRegion. (Default is: False)
- flipTilesY – Determines default value of flipY when creating leTrackRegion / leGlobalTrackRegion. (Default is: False)
- flipTilesXFirst – Determines default value of flipFirstX when creating leTrackRegion / leGlobalTrackRegion. (Default is: False)
- flipTilesYFirst – Determines default value of flipFirstY when creating leTrackRegion / leGlobalTrackRegion. (Default is: False)
- minCountX – Minimum number of least common multiple (LCM) times groups repeated in the X direction (Default is: 1)
- minCountY – Minimum number of least common multiple (LCM) times groups repeated in the Y direction (Default is: 1)
- maxCountX – Maximum number of least common multiple (LCM) times groups repeated in the X direction (Default is: -1)

- `maxCountY` – Maximum number of least common multiple (LCM) times groups repeated in the Y direction (Default is: -1)
- `snapGrid` – Snap grid to use during interactive creation and editing of `leTrackRegion`
- `snapOffset` – Snap offset to use during interactive creation and editing of `leTrackRegion`
- `regionDefType` – Determines the region def type (`{ placement | routing }`) (Default is: 'routing')
- `autoGenerateGlobalRegion` – Auto create a global region on first design open (Default is: False)
- `enclosedGroups` – Collection of `leTrackGroupDef` that abide by `groupEnclosure` value
- `groupEnclosure` – Vector value to enclose group definitions inside the region boundary. X value is used for horizontal groups, and Y value for vertical groups.
- `defaultTracksVisible` – Integer value that allows user to override active attribute for all `leTracks` (Default is: -1)
- `defaultTracksActive` – Integer value that allows user to override visible attribute for all `leTracks` (Default is: -1)
- `flipGroupsFirst` – Collection of `leTrackGroupDef` to flip first. Valid only when `flipX/flipY` is true or `flipXGroups/flipYGroups` is specified.
- `groupFlipTarget` – Determines if flip group inside region or tile. (`{ region | tile }`) (Default is: 'tile')

Returns

The newly created `leTrackRegionDef` object

createVia

Syntax

```
createVia(  
*,  
  design: oaDesign,  
  definition: oaViaDef | str,  
  origin: point,  
  params: list[map_] = None,  
  orient: le.createVia.Orient = None)  
Returns: oaVia
```

Summary:

Create via

Category:

Layout Editing

Licenses:

Basic

Parameters

- design – Design in which to create the via.
- definition – Via definition or its name, for new object.
- origin – Location of the origin of via.
- params – List of properties for via object. Each item should have {name value} format. These properties will be assigned to via object.
- orient – Orientation of the via. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 } (Default is: 'R0'))

Returns

Newly created via

createVoltagePhysicalAnnotations

Syntax

```
createVoltagePhysicalAnnotations(  
context: deContext = None,  
/,  
*,  
method: le.createVoltagePhysicalAnnotations.Method = None,  
scope: le.createVoltagePhysicalAnnotations.Scope = None,  
nets: str | list[oaNet] = None,  
depth: le.createVoltagePhysicalAnnotations.Depth = None,  
height: float = None,  
font: le.createVoltagePhysicalAnnotations.Font = None,  
orient: le.createVoltagePhysicalAnnotations.Orient = None,  
notation: le.createVoltagePhysicalAnnotations.Notation = None,  
annotateProc: str = None,  
clearProc: str = None,  
overwrite: bool = None,  
**kwargs)  
Returns: None
```

Summary:

Create physical voltage annotations for all nets.

Category:

Layout Editing

Licenses:

Backend

Parameters

- context – Context object of the create physical voltage annotations operation.
- method – Physical annotation method. ({ custom | label } (Default is: 'label'))
- scope – (Label method only) Text label annotation scope. ({ Design | Nets | Viewport } (Default is: 'Design'))
- nets – (Label method only) oaNet or net name list for Nets scope.
- depth – (Label method only) Text label annotation depth. ({ 0x | Full | View } (Default is: '0x'))
- height – (Label method only) Text height in user unit. (Default is: 0.2)
- font – (Label method only) Text font. ({ euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish } (Default is: 'stick'))
- orient – (Label method only) Text orientation. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 } (Default is: 'R0'))
- notation – (Label method only) Number notation. ({ engineering | literal | scientific } (Default is: 'literal'))
- annotateProc – (Custom method only) Annotation procedure name for the custom method.
- clearProc – (Custom method only) Name of the Tcl procedure for removing the annotations.
- overwrite – Overwrite the existing annotations. (Default is: True)

Returns

None

cutCorner

Syntax

```
cutCorner(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
         | list[oaVia],  
/,  
*,  
cutLength: float,  
cutSides: int,  
remainAsPath: bool = None)  
Returns: list[oaFig]
```

Summary:

cut corner from selection

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – List of objects to be cut.
- cutLength – The distance from the corner to the point that will be cut.
- cutSides – The number of sides that each corner will be cut.
- remainAsPath – Option to control whether remain as path after cut path. (Default is: False)

Returns

Collection of objects.

cycleDensity

Syntax

```
cycleDensity(  
window: deCanvas | giWindow | int = None)  
Returns: None
```

Summary:

Cycles the density tip.

Category:

Layout Editing

Licenses:

Backend

Parameters

window – Window or canvas

Returns

None

delete

Syntax

```
delete(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
         | list[oaVia],  
/,  
*,  
partialObject: le.delete.PartialObject = None,  
**kwargs)  
Returns: None
```

Summary:

Delete specified objects

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – List of objects to delete. Objects are represented by a selectable figure.
- partialObject – Partial object ({ full | ignore | partial }) (Default is: 'ignore')

Returns

None

deleteCutMetal

Syntax

```
deleteCutMetal(  
    layer: str,  
    /,  
    *,  
    design: oaDesign,  
    area: le.deleteCutMetal.Area = None,  
    points: list[point] = None,  
    cutPurpose: str = None,  
    onlyShapeBelowCut: bool = None)  
Returns: None
```

Summary:

Delete Cut Metal

Category:

Layout Editing

Licenses:

Basic

Parameters

- layer – The target layer which user need to handle.
- design – Design in which to active delete cut metal command.
- area – The area to use delete cut metal command ({ design | polygon | rectangle } (Default is: 'design'))
- points – Point list for the rectangle/polygon area, based on which area type is specified.
- cutPurpose – Specify the purpose for cut-metal shapes in ICC codesign flow.
- onlyShapeBelowCut – Specify to only delete shape below cut. (Default is: False)

Returns

None

deleteDensityResult

Syntax

```
deleteDensityResult(  
    window: deCanvas | giWindow | int = None,  
    /,  
    *,  
    layer: str = None,  
    group: str = None,  
    all: bool = None)  
Returns: None
```

Summary:

Delete the density result.

Category:

Layout Editing

Licenses:

Backend

Parameters

- window – Window or canvas
- layer – Layer name in layer mode to delete density.
- group – Group name in config file mode to delete density.
- all – Specify whether to delete all results. Ignore layer and group option if specified. (Default is: False)

Returns

None

deleteGeneratedPins

Syntax

```
deleteGeneratedPins(  
    *,  
    design: oaDesign)  
Returns: None
```

Summary:

Delete the pins which are generated by generate pins command

Category:

Layout Editing

Licenses:

Basic

Parameters

design – Design in which to delete generated pins.

Returns

None

designSummary

Syntax

```
designSummary()  
Returns: str
```

Arguments:

Summary:

Generate a summary of the design.

Category:

Layout Editing

Licenses:

None

Returns

Design Summary.

displayFlightlines

Syntax

```
displayFlightlines(  
    objects: list[oaGroup] | list[oaNet])  
Returns: None
```

Summary:

Displays flightlines.

Category:

Layout Editing

Licenses:

Basic

Parameters

objects – Collection of flightline, group or net objects to display flightlines

Returns

None

displayPinConnections

Syntax

```
displayPinConnections(  
  objects: list[oaPin])  
Returns: None
```

Summary:

Displays pin connection flightlines.

Category:

Layout Editing

Licenses:

Basic

Parameters

objects – Collection of pins to display flightlines

Returns

None

distribute

Syntax

```
distribute(  
  objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
    list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
    | list[oaVia],
```



```
/,
*,
src: deFigure | oaFig,
alignSide: le.distribute.AlignSide = None,
direction: le.distribute.Direction = None,
distance: float = None,
byPitch: bool = None,
sort: le.distribute.Sort = None,
sortDirection: le.distribute.SortDirection = None,
distributeFirstObject: bool = None)
Returns: list[deFigure] | list[oaFig]
```

Summary:

Distribtue specified objects

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – List of target objects to distribute.
- src – Source object for distribute, can be also a sub-object.
- alignSide – Alignment side. The value of this option will be used only if at least one full object is specified. ({ bottom | horizontalMiddle | left | right | top | verticalMiddle } (Default is: 'bottom'))
- direction – Direction according to which the distance between the aligning edges should be applied. ({ bottom | left | right | top } (Default is: 'right'))
- distance – Distance between distributing objects. (Default is: 0)
- byPitch – Control distribution of objects by pitch or by spacing. (Default is: False)
- sort – Direction according to which the distance between the aligning edges should be applied. ({ cell | instance | net } (Default is: 'instance'))
- sortDirection – Direction according to sorting distribute objects. ({ fromBottom | fromLeft | fromRight | fromTop | none } (Default is: 'none'))
- distributeFirstObject – Control distribute the first object or not. (Default is: False)

Returns

Collection of aligned objects.

evalViaParams

Syntax

```
evalViaParams(  
*,  
    techLib: oaLib = None,  
    techLibName: dmLibName = None,  
    design: oaDesign = None,  
    layer1: lpp,  
    layer2: lpp,  
    layer1Box: box = None,  
    layer2Box: box = None,  
    layer1Width: float = None,  
    layer2Width: float = None,  
    layer1Dir: le.evalViaParams.Layer1Dir = None,  
    layer2Dir: le.evalViaParams.Layer2Dir = None,  
    overlapBox: box,  
    cutSize: str = None,  
    cutClass: str = None)  
Returns: oaViaParam
```

Summary:

Calculate via params from given arguments

Category:

Layout Editing

Licenses:

Basic

Parameters

- techLib – OA library where to query rules from its tech database
- techLibName – Name of OA library where to query rules from its tech database
- design – OA design where to query rules from
- layer1 – Bottom layer of the via
- layer2 – Upper layer of the via
- layer1Box – Box of layer1. Valid only when both layer1Box and layer2Box are specified.
- layer2Box – Box of layer2. Valid only when both layer1Box and layer2Box are specified.

- layer1Width – Width of layer1. Valid only when both layer1Box and layer2Box are specified. Queried from layer box if no specified.
- layer2Width – Width of layer2. Valid only when both layer1Box and layer2Box are specified. Queried from layer box if no specified.
- layer1Dir – Direction of layer1. Valid only when both layer1Box and layer2Box are specified. Queried from layer box if no specified. ({ downToUp | leftToRight | rightToLeft | upToDown })
- layer2Dir – Direction of layer2. Valid only when both layer1Box and layer2Box are specified. Queried from layer box if no specified. ({ downToUp | leftToRight | rightToLeft | upToDown })
- overlapBox – Overlap box of layer1 and layer2
- cutSize – Specify cut size. Could be single value or pair value. For example, {0.5} or {0.2 0.3}
- cutClass – Specify cutClass name.

Returns

the evaluated oaViaParam

exportResistanceQuery

Syntax

```
exportResistanceQuery(  
    scope: oaDesign,  
    /,  
    *,  
    filePath: str)  
Returns: str
```

Summary:

export resistance query data to a file

Category:

Layout Editing

Licenses:

Backend

Parameters

- scope – design to be reported.
- filePath – Specify the file path to export query data to

Returns

success to export query data or not.

externalizeColor

Syntax

```
externalizeColor(  
*,  
    design: oaDesign = None,  
    libName: str = None,  
    anchorsOnly: bool = None,  
    purpose: str = None)  
Returns: None
```

Summary:

Externalizes the color of the given design or all designs of the given library.

Category:

Layout Editing

Licenses:

Basic

Parameters

- design – design object
- libName – library name
- anchorsOnly – copy only anchor objects (Default is: True)
- purpose – design or cellView object (Default is: 'drawing')

Returns

None

extractLayoutContext

Syntax

```
extractLayoutContext(  
    context: deContext,  
    /,  
    *,  
    path: str,  
    presetLCV: dmCellView | dmCellViewName = None)  
Returns: None
```

Summary:

extract specified objects to yaml in layout context

Category:

Layout Editing

Licenses:

Custom Compiler AI

Parameters

- context – Context of activation.
- path – Folder path to store the extracted info.
- presetLCV – LCV that store trackPcell preset.

Returns

None

fill

Syntax

```
fill(  
    *,  
    points: list[point] = None,  
    objects: list[deFigure] | list[deSelectionSet] | list[oaFig] =  
    None,  
    design: oaDesign = None,  
    cellName: str = None,  
    master: oaDesign = None,  
    fillIn: list[deFigure] | list[deSelectionSet] | list[oaFig] =  
    None,  
    libName: str = None,  
    viewName: str = None,
```

```
distributed: bool = None,  
spacing: str = None,  
enclosure: str = None,  
orient: le.fill.Orient = None,  
just: le.fill.Just = None,  
array: bool = None,  
newCellView: bool = None,  
newLibName: str = None,  
newCellName: str = None,  
newViewName: str = None,  
excludeLPPs: str = None,  
excludeHalo: str = None,  
excludeDepth: int = None,  
tiling: bool = None)  
Returns: list[oaFig]
```

Summary:

Fill figures

Category:

Layout Editing

Licenses:

Basic

Parameters

- points – Point list of the fill region.
- objects – Collection of oaFig for extracting fill regions.
- design – Design where figures are filled with. (Default is: 'editing design')
- cellName – Cell name of the master cellview to fill in. This must be specified if -fillIn is not specified.
- master – Master of the cellview to fill in.
- fillIn – Collection of oaFig to fill in.
- libName – Library name of the master cellview to fill in. (Default is: 'library of -design')
- viewName – View name of the master cellview to fill in. (Default is: 'layout')
- distributed – Specifies whether to distribute the available space when -tiling false is specified. (Default is: False)
- spacing – The spacing value between two consecutive filling objects. (Default is: 0)
- enclosure – The enclosure size to the area edge. (Default is: 0)

- **orient** – Orientation of the filler figure. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 }) (Default is: 'R0')
- **just** – Justification of the filler figure with respect to the origin. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight }) (Default is: 'centerCenter')
- **array** – Specifies whether to fill area by array instance (arrayInst) objects. (Default is: True)
- **newCellView** – Specifies whether to create a new cell view for filling objects. (Default is: False)
- **newLibName** – Specifies the library name of the new cell view which collects the filling instances. (Default is: 'library of -design')
- **newCellName** – Specifies the cell name of the new cell view which collects the filling instances. (Default is: 'cell of -design')
- **newViewName** – Specifies the view name of the new cell view which collects the filling instances. (Default is: 'layout_fill')
- **excludeLPPs** – Specifies the list of excluded LPPs to avoid filling objects on the the regions.
- **excludeHalo** – Specifies the distance between filling objects and excluded regions. (Default is: 0)
- **excludeDepth** – Specifies the depth of excluded LPP shapes. (Default is: 0)
- **tiling** – Specifies whether to use the tiling method to fill the specified area. (Default is: False)

Returns

Collection of filled figures.

fixTrackSnapping

Syntax

```
fixTrackSnapping(  
objects: leOffTrackInfo | list[deFigure] | list[deSelectionSet] |  
         list[leOffTrackInfo] | list[oaFig] | list[oaInst] | list[oaShape]  
         | list[oaVia],  
/  
**kwargs)  
Returns: list[leOffTrackInfo]
```

Summary:

Fix Track Sanpping of Objects

Category:

Layout Editing

Licenses:

Backend

Parameters

objects – List of objects to check track snapping.

Returns

Collection of leOffTrackInfo for unfixable objects

flatten

Syntax

```
flatten(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
        list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
        | list[oaVia],  
/,  
*,  
levels: int = None,  
connectivity: bool = None,  
pcells: bool = None,  
vias: bool = None,  
pins: bool = None,  
pinText: bool = None,  
mpps: bool = None,  
stripedWires: bool = None,  
figGroups: bool = None,  
hierarchyName: bool = None,  
returnResult: bool = None,  
scalarizeArrayInst: bool = None)  
Returns: None
```

Summary:

Flattens instances and vias.

Category:

Layout Editing

Licenses:

Basic

Parameters

- **objects** – List of objects to flatten.
- **levels** – Number of hierarchical levels to flatten. (Default is: 0)
- **connectivity** – Option to preserve original connectivity or not. (Default is: True)
- **pcells** – Option to preserve or flatten pcells. (Default is: True)
- **vias** – Option to preserve or flatten vias. (Default is: True)
- **pins** – Preserve instance pins and net terminals. (Default is: True)
- **pinText** – Option to preserve pin text. (Default is: True)
- **mpps** – Option to preserve or flatten MPPs. (Default is: True)
- **stripedWires** – Option to preserve or flatten striped wires. (Default is: True)
- **figGroups** – Option to preserve or flatten figure groups. (Default is: True)
- **hierarchyName** – Option to take hierarchy name as prefix. (Default is: True)
- **returnResult** – Option to return a collection of flattened objects. (Default is: True)
- **scalarizeArrayInst** – Option to transform the array into a group of individual instances. (Default is: False)

Returns

None

generateAbstract

Syntax

```
generateAbstract(  
*,  
    src: oaDesign = None,  
    srcLibName: str = None,  
    srcCellName: str = None,  
    srcViewName: str = None,  
    srcCategoryName: str = None,  
    ignoreCell: str = None,  
    destLibName: str = None,  
    destCellName: str = None,  
    destViewName: str = None,  
    options: dbGenOptionSet = None,
```

```
force: bool = None,  
batch: bool = None,  
genCellNamePrefix: str = None,  
group: str | xtJobGroup = None,  
exitProc: dbExecObject | str | xtProcedure = None,  
tcl: str = None)  
Returns: dmCellView | list[dmCellView]
```

Summary:

Creates abstract from given design.

Category:

Layout Editing

Licenses:

Basic

Parameters

- src – The source design.
- srcLibName – The library name of the source design.
- srcCellName – The cell name(s) of the source design.
- srcViewName – The view name of the source design.
- srcCategoryName – The category name of the source design.
- ignoreCell – The cell names to be ignored.
- destLibName – The library name of the destination design.
- destCellName – The cell name(s) of the destination design.
- destViewName – The view name of the destination design.
- options – The options set.
- force – Indicates if the user wishes to overwrite an existing design. The default value is false.
- batch – Specify to run abstract generation in batch mode. The default value is false.
- genCellNamePrefix – Specify cell name prefix.
- group – The job group under which this job is added. (for batch mode).
- exitProc – Procedure to call once the job has exited. (for batch mode).
- tcl – Specify scripts to run in batch mode.

Returns

New cell view(s)

generateAntennaRunset

Syntax

```
generateAntennaRunset(  
*,  
    src: oaDesign = None,  
    srcLibName: str = None,  
    srcCellName: str = None,  
    srcViewName: str = None,  
    options: dbGenOptionSet = None,  
    allLayers: bool = None)  
Returns: None
```

Summary:

Creates runset from given design.

Category:

Layout Editing

Licenses:

Basic

Parameters

- src – The source design.
- srcLibName – The library name of the source design.
- srcCellName – The cell name of the source design.
- srcViewName – The view name of the source design.
- options – The options set.
- allLayers – Use all related layers to generate runset

Returns

None

generateCutMetal

Syntax

```
generateCutMetal(  
lpp: lpp,  
/,  
*,  
design: oaDesign,  
area: le.generateCutMetal.Area = None,  
points: list[point] = None,  
dir: le.generateCutMetal.Dir = None,  
onTop: bool = None,  
cutPurpose: str = None,  
space: float = None,  
eraseWidth: float = None,  
eraseLength: float = None,  
eraseSpace: float = None,  
generateOnInstBoundary: bool = None,  
boundaryLpp: lpp = None,  
onlyShapeBelowCut: bool = None,  
increment: bool = None,  
autoMode: bool = None,  
cutLPP: lpp = None,  
findSingleMetal: bool = None,  
enableHalfWidthMetal: bool = None,  
blockagePurpose: str = None,  
spacingLessThanCutWidth: bool = None,  
connectedMetalToTop: bool = None,  
extendBlockage: bool = None,  
checkShort: bool = None,  
generateOnPolyEndedSide: bool = None,  
checkExistingCutMetal: bool = None,  
fillMode: bool = None,  
fillMinSpacing: float = None)  
Returns: list[oaFig]
```

Summary:

Generate Cut Metal

Category:

Layout Editing

Licenses:

Basic

Parameters

- lpp – The target LPP which user need to handle.
- design – Design in which to active generate cut metal command.
- area – The area to use generate cut metal command ({ design | point | polygon | rectangle } (Default is: 'design'))
- points – Point list for the rectangle/polygon area, based on which area type is specified.
- dir – The track direction of target layer. Default is preferred routing direction ({ H | V })
- onTop – Specify if generate shapes on top level. (Default is: True)
- cutPurpose – Specify the purpose for cut-metal shapes.
- space – The min spacing for violation checking.
- eraseWidth – The width of erasing layer.
- eraseLength – The length of erasing layer.
- eraseSpace – The min spacing of erasing layer.
- generateOnInstBoundary – Specify if generate cut shapes on instance boundary. (Default is: True)
- boundaryLpp – Specify the boundary LPP.
- onlyShapeBelowCut – Specify if only generate shapes below cut metal in ICC codesign flow. (Default is: False)
- increment – Specify if generate cut metal in incremental mode. (Default is: False)
- autoMode – Specify if generate cut metal in auto mode. (Default is: False)
- cutLPP – Specify the cut LPP that below-cut shapes should be inserted.
- findSingleMetal – Specify to find single metal for below-cut shapes insertion. (Default is: False)
- enableHalfWidthMetal – Enable half width generation when generate shape below cut in ICC design. (Default is: False)
- blockagePurpose – Specify blockage purpose name when enable half width generation. The shape below cut will be half-width when blockages shapes exist.
- spacingLessThanCutWidth – Specify to generate shape below cut when metal spacing is less than cut width. (Default is: True)
- connectedMetalToTop – Specify whether to generate shape below cut in instance for connected metal. (Default is: False)

- `extendBlockage` – Specify to extend blockage when generate shape below cut. Metal layer with blockage purpose will be collected for inserting cut metal. (Default is: `False`)
- `checkShort` – Specify to check 0x net on the metal shape. Won't generate cut metal when 0x nets are the same. (Default is: `False`)
- `generateOnPolyEndedSide` – When generate cut metal on instance boundary, cut metal will be placed on poly-ended side (Default is: `False`)
- `checkExistingCutMetal` – When generate cut metal, don't need to insert again if cut metal exists nearby. (Default is: `False`)
- `fillMode` – Specify to use fill cell mode when generate cut metal. (Default is: `False`)
- `fillMinSpacing` – Specify the minimum spacing to do fill mode. (Default is: `False`)

Returns

Collection of generated objects.

generateGuardRing

Syntax

```
generateGuardRing(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaShape] | list[oaVia],  
/,  
*,  
autoConfig: bool = None,  
perInstance: bool = None,  
master: dmCellView | dmCellViewName | oaDesign = None,  
style: le.generateGuardRing.Style = None,  
autoStyle: le.generateGuardRing.AutoStyle = None,  
spacing: str = None,  
layerSpacing: list[str] = None,  
individualWrap: bool = None,  
mergeDistance: float = None,  
notch: float = None,  
removeJog: float = None,  
autoNotch: float = None,  
params: list[map_] = None,  
rings: le.generateGuardRing.Rings = None,  
evalCallbacks: bool = None,  
net: list[oaNet | str] = None,  
rows: int = None,  
bodyWidth: float = None,  
metWidthEqBody: le.generateGuardRing.MetWidthEqBody = None,  
cornerCut: le.generateGuardRing.CornerCut = None,  
sameMetWidth: le.generateGuardRing.SameMetWidth = None)  
Returns: list[oaInst]
```

Summary:

Generate Guard Ring

Category:

Layout Editing

Licenses:

Basic

Parameters

- **objects** – List of objects to be enclosed by generated guard ring.
- **autoConfig** – Specify if enable auto-configuration mode. (Default is: False)
- **perInstance** – Generate guard rings for each selected instance. Only for auto-configuration mode. (Default is: False)
- **master** – SuperMaster of guard ring. Not for auto-configuration mode.
- **style** – Style to form guard ring. Not for auto-configuration mode. ({ conformal | rectangle } (Default is: 'rectangle')
- **autoStyle** – Style to form guard ring. Only for auto-configuration mode. ({ auto | conformal | rectangle } (Default is: 'auto')
- **spacing** – Minimum spacing between enclosed objects' boundary and boundary of generated guard ring. Not for auto-configuration mode. (Default is: '0')
- **layerSpacing** – Specify the minimum spacing between enclosed objects' layer boundary and layer boundaries of generated guard ring. Each item should have {guard ring LPP, object LPP, spacing value} format. For example: {{{M1 drawing} {DIFF drawing} 0.5}}. Not for auto-configuration mode.
- **individualWrap** – Generate guard rings for individual selected objects. Not for auto-configuration mode. (Default is: False)
- **mergeDistance** – The distance value to control how to group the individual objects. Not for auto-configuration mode. (Default is: 0)
- **notch** – Minimum notch value. Not for auto-configuration mode. (Default is: 0)
- **removeJog** – Minimum jog value to remove. (Default is: 0)
- **autoNotch** – Specify minimum notch value instead of use the pre-defined notch value in the auto-configuration file. Only for auto-configuration mode.
- **params** – Parameters of pcell instance. Each item should have {parameter name, parameter value} format. Not for auto-configuration mode.

- **rings** – Number of guard rings to generate up to maximum definition count in auto-configuration file. Only for auto-configuration mode. ({ 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | max }) (Default is: 'max')
- **evalCallbacks** – Should callbacks be triggered or not. (Default is: True)
- **net** – The net names or oaNet objects to guard rings which should be assigned.
- **rows** – Cut rows of MCell guard ring. Only for auto-configuration mode.
- **bodyWidth** – Body width of MCell guard ring. Only for auto-configuration mode.
- **metWidthEqBody** – Policy to enable option Metal Width Equals Body Width. Only for auto-configuration mode. ({ auto | disable | enable }) (Default is: 'auto')
- **cornerCut** – Policy to enable option Generate Corner Cut. Only for auto-configuration mode. ({ auto | disable | enable }) (Default is: 'auto')
- **sameMetWidth** – Policy to enable option Same Metal Width. Only for auto-configuration mode. ({ auto | disable | enable }) (Default is: 'auto')

Returns

Collection of guard ring instances.

generateLabel

Syntax

```
generateLabel(  
*,  
  design: oaDesign,  
  fromNet: bool = None,  
  fromTerm: bool = None,  
  fromInstance: bool = None,  
  area: le.generateLabel.Area = None,  
  lppMethod: le.generateLabel.LppMethod = None,  
  mapFile: str = None,  
  textType: le.generateLabel.TextType = None,  
  attach: bool = None,  
  box: box = None,  
  points: list[point] = None,  
  point: point = None,  
  objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
  list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
  | list[oaVia] = None,  
  topMetalOnly: bool = None,  
  useVirtualConnection: bool = None,  
  generateDeviceNamePerGate: bool = None,  
  textLpp: lpp = None,  
  instTextLpp: lpp = None,  
)
```



```
just: le.generateLabel.Just = None,  
orient: le.generateLabel.Orient = None,  
font: le.generateLabel.Font = None,  
height: float = None,  
drafting: bool = None,  
overbar: bool = None,  
style: le.generateLabel.Style = None,  
location: le.generateLabel.Location = None,  
inward: float = None)  
Returns: list[oaShape]
```

Summary:

Generate Labels

Category:

Layout Editing

Licenses:

Basic

Parameters

- design – Design in which to generate labels.
- fromNet – Specifies whether to generate labels from net. (Default is: False)
- fromTerm – Specifies whether to generate labels from term. (Default is: False)
- fromInstance – Specifies whether to generate labels from instance. (Default is: False)
- area – The area to generate labels. ({ design | point | polyline | region | selection | viewPort } (Default is: 'design'))
- lppMethod – Specifies an LPP method. ({ inherit | map | specify | techLib } (Default is: 'inherit'))
- mapFile – Specifies a map file.
- textType – Specifies a text type. ({ attribute | text } (Default is: 'attribute'))
- attach – Specifies whether attach label to object. (Default is: True)
- box – The coordinates of bounding box in region mode.
- points – Point list in polyline mode.
- point – Location in point mode.
- objects – The list of objects in selection mode.
- topMetalOnly – Specifies whether to use top metal for island only. (Default is: False)

- `useVirtualConnection` – Specifies whether to use virtual connection. (Default is: `False`)
- `generateDeviceNamePerGate` – Specifies whether to generate device name per gate. (Default is: `False`)
- `textLpp` – Specifies a text LPP for labels generated on shapes.
- `instTextLpp` – Specifies a text LPP for labels generated on instances.
- `just` – Justification of the text with respect to the origin. (`{ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight }`) (Default is: `'centerCenter'`)
- `orient` – Orientation of the text. (`{ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 }`) (Default is: `'R0'`)
- `font` – Font style with which the text is displayed. (`{ euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish }`) (Default is: `'stick'`)
- `height` – Height of the text in user units. (Default is: `1`)
- `drafting` – Indicates whether the text should not be mirrored or rotated. (Default is: `True`)
- `overbar` – Indicates whether the text will be rendered with overbar. (Default is: `False`)
- `style` – Specifies whether to generate label with automatically determined label attributes. (`{ auto | manual }`) (Default is: `'manual'`)
- `location` – Specifies location which is used to locate the label origin on the object. (`{ bottom | center | left | left_right | right | top | top_bottom }`) (Default is: `'center'`)
- `inward` – Specifies the offset in user units to move the label origin inward when generating label with user-defined location. (Default is: `0`)

Returns

Collection of label object.

generateLayout

Syntax

```
generateLayout(  
*,  
  dest: dmCellView | dmCellViewName,  
  interface: dbInterfaceSet,  
  options: dbGenOptionSet = None)  
Returns: None
```

Summary:

Generates layout from given schematic

Category:

Layout Editor

Licenses:

Basic

Parameters

- `dest` – The destination cellview.
- `interface` – The interface.
- `options` – The options value.

Returns

None

generateLayoutContext

Syntax

```
generateLayoutContext(  
    context: deContext,  
    /,  
    *,  
    path: str,  
    techMapping: str = None,  
    overwrite: bool = None,  
    skipTrackPcell: bool = None)  
Returns: None
```

Summary:

generate specified objects from yaml to layout context

Category:

Layout Editing

Licenses:

Custom Compiler AI

Parameters

- context – Context of activation.
- path – Folder path to store the extracted info.
- techMapping – Technology mapping file.
- overwrite – Overwrite duplicate object.
- skipTrackPcell – skip extracting trackPcellRegion.

Returns

None

generatePGMarkers

Syntax

```
generatePGMarkers(  
*,  
    nets: list[oaNet] = None,  
    design: oaDesign = None,  
    layers: str = None,  
    routeTypeOnly: bool = None)  
Returns: None
```

Summary:

Generate PG markers

Category:

Layout Editing

Licenses:

Basic

Parameters

- nets – Collection of net objects.
- design – Design to search in.
- layers – defines on which layers PG markers are generated.
- routeTypeOnly – defines whether shapes should be filtered by route type for OA design or not.

Returns

None

generatePins

Syntax

```
generatePins (
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]
| list[oaVia] = None,
/,
*,
design: oaDesign,
area: le.generatePins.Area = None,
points: list[point] = None,
fromText: bool = None,
lppMethod: le.generatePins.LppMethod = None,
mapFile: str = None,
extractDepth: int = None,
textDepth: int = None,
useVirtualConnection: bool = None,
extractThroughVia: bool = None,
pinSize: str = None,
fromNet: bool = None,
includePinFromInstance: bool = None,
shapeDepth: le.generatePins.ShapeDepth = None,
assignToShape: bool = None,
purposeList: str = None,
excludeLayers: str = None,
topLayerNum: int = None,
metalOnly: bool = None,
forLogicTermsOnly: bool = None,
autoPlacePin: bool = None,
nets: str | list[oaNet] = None,
createSquarePin: bool = None,
ignoreNets: str | list[oaNet] = None,
skipUserCreatedPins: bool = None,
enableMultithread: bool = None,
createPinLabel: bool = None,
style: le.generatePins.Style = None,
textType: le.generatePins.TextType = None,
attach: bool = None,
followInstPinLabel: bool = None,
just: le.generatePins.Just = None,
orient: le.generatePins.Orient = None,
pinLabelLppMethod: le.generatePins.PinLabelLppMethod = None,
pinLabelMapFile: str = None,
pinLabelLpp: lpp = None,
font: le.generatePins.Font = None,
height: float = None,
```

```
drafting: bool = None,  
overbar: bool = None)  
Returns: list[oaShape]
```

Summary:

Generate Pins

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – List of selected objects.
- design – Design in which to generate pins.
- area – The area to generate pins. ({ design | polygon | rectangle | selection | viewport } (Default is: 'design'))
- points – Point list for the rectangle/polygon area, based on which area type is specified.
- fromText – Specify if generate pins from text or not. (Default is: True)
- lppMethod – Lpp method for generate pins from text. ({ map | sameLpp | techLib } (Default is: 'sameLpp'))
- mapFile – Map file for generate pins from text.
- extractDepth – The extract shape depth to generate pins from text. (Default is: 0)
- textDepth – The text depth to generate pins from text. (Default is: 0)
- useVirtualConnection – Specify if use virtual connection in generate pins. (Default is: True)
- extractThroughVia – Specify if extract shapes through via in generate pins. (Default is: False)
- pinSize – Specify the pin size for new shapes in generate pins.
- fromNet – Specify if generate pins from net or not. (Default is: False)
- includePinFromInstance – Specify if include pin from instance when generate pins from net. (Default is: False)
- shapeDepth – Specify shape depth when generate pins from net. ({ includeInstPin | instPinOnly | top })

- `assignToShape` – Specify if assign pin information to 0x shapes directly. (Default is: True)
- `purposeList` – Specify the purpose list for new shapes in generate pins. (Default is: 'pin drawing')
- `excludeLayers` – Specify the exclude layers in generate pins.
- `topLayerNum` – Specify the top layer number for extract pin shapes. < 1 means search all layers. (Default is: -1)
- `metalOnly` – Specify only metal layers are used when specifying top layer number to generate pin. (Default is: True)
- `forLogicTermsOnly` – Specify if generate pins for logic terms only. (Default is: True)
- `autoPlacePin` – Specify if enable auto place pin. (Default is: False)
- `nets` – Net names or `oaNets` for generate pin from net.
- `createSquarePin` – Specify if enable create square pin. (Default is: False)
- `ignoreNets` – Net names or `oaNets` for generate pin from net to ignore net.
- `skipUserCreatedPins` – Specify to skip nets with user created pins. (Default is: False)
- `enableMultithread` – Specify to enable multithread to trace connection. (Default is: False)
- `createPinLabel` – Specify to create pin label for generate pins from net. (Default is: False)
- `style` – Specifies whether to create pin label with automatically determined label attributes. ({ auto | manual }) (Default is: 'auto')
- `textType` – Specifies pin label's text type. ({ attribute | text }) (Default is: 'attribute')
- `attach` – Specifies whether attach label to object. (Default is: True)
- `followInstPinLabel` – Specify to create pin label by following instance pins' label. (Default is: False)
- `just` – Justification of the pin label with respect to the origin. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight }) (Default is: 'centerCenter')
- `orient` – Orientation of the pin label. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 }) (Default is: 'R0')
- `pinLabelLppMethod` – Specifies LPP method for pin label. ({ map | sameLpp | specify | techLib }) (Default is: 'sameLpp')

- pinLabelMapFile – Map file for generating pin labels.
- pinLabelLpp – Specifies a text LPP for pin labels.
- font – Font style with which the pin label is displayed. ({ euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish }) (Default is: 'stick')
- height – Height of the pin label in user units. (Default is: 1)
- drafting – Indicates whether the pin label should not be mirrored or rotated. (Default is: True)
- overbar – Indicates whether the pin label will be rendered with overbar. (Default is: False)

Returns

Collection of pin figures.

generateShapes

Syntax

```
generateShapes(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia] = None,  
/,  
*,  
design: oaDesign = None,  
lpp1: lpp,  
lpp2: lpp = None,  
lpp2s: list[lpp] = None,  
lpp: lpp,  
operation: le.generateShapes.Operation,  
levels: int = None,  
numPoints: int = None,  
region: list[point] = None,  
pointsCollection: bool = None)  
Returns: dbCollection | list[oaShape]
```

Summary:

Generate Shapes

Category:

Layout Editing

Licenses:

Basic

Parameters

- **objects** – Collection of objects, based on which the shapes are generated.
- **design** – Design, based on which the shapes are generated.
- **lpp1** – First layer & purpose pair. All objects from the input shape collection on this lpp will be the first sub-set to the shape generation operation.
- **lpp2** – Second layer & purpose pair. All objects from the input shape collection on this lpp will be the second sub-set to the shape generation operation.
- **lpp2s** – Second layer & purpose pair list. All objects from the input shape collection on this lpp will be the second sub-set to the shape generation operation.
- **lpp** – Layer & purpose pair for the output shapes
- **operation** – Operation which is performed. ({ and | andnot | or | xor })
- **levels** – Number of hierarchical levels beyond which to ignore shapes contained by instances (Default is: 0)
- **numPoints** – Number of vertices to convert an ellipse into a polygon.
- **region** – Point list of generate shape region.
- **pointsCollection** – Return the points collection instead of generate shapes (Default is: False)

Returns

Collection of new shapes

generateSoftAbstract

Syntax

```
generateSoftAbstract(  
*,  
    src: oaDesign = None,  
    srcLibName: str = None,  
    srcCellName: str = None,  
    srcViewName: str = None,  
    destLibName: str = None,  
    destCellName: str = None,  
    destViewName: str = None,  
    options: dbGenOptionSet = None,  
    force: bool = None)  
Returns: dmCellView
```

Summary:

Creates soft abstract from given design.

Category:

Layout Editing

Licenses:

Basic

Parameters

- `src` – The source design.
- `srcLibName` – The library name of the source design.
- `srcCellName` – The cell name(s) of the source design.
- `srcViewName` – The view name of the source design.
- `destLibName` – The library name of the destination design.
- `destCellName` – The cell name(s) of the destination design.
- `destViewName` – The view name of the destination design.
- `options` – The options set.
- `force` – Indicates if the user wishes to overwrite an existing design. The default value is `false`.

Returns

New cell view

getAbuttedInsts

Syntax

```
getAbuttedInsts(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
         | list[oaVia],  
/,  
*,  
direction: le.getAbuttedInsts.Direction = None,  
neighborOnly: bool = None,  
sameSide: bool = None)  
Returns: list[oaInst]
```

Summary:

Returns a collection of abutted instances

Category:

Layout Editing

Licenses:

None

Parameters

- **objects** – Instances to search for abutment partners
- **direction** – Direction (left / right / all) to search for abutment partners ({ all | left | right } (Default is: 'all'))
- **neighborOnly** – Return only immediate neighbors (Default is: False)
- **sameSide** – Trace all instances on same side (Default is: True)

Returns

Collection of abutted instances.

getAbuttedMPPs

Syntax

```
getAbuttedMPPs (  
*,  
    design: oaDesign)  
Returns: dbCollection
```

Summary:

Returns a collection of abutted mpps

Category:

Layout Editing

Licenses:

None

Parameters

design – Design to get abutted mpps.

Returns

List of abutted mpps in given design.

getAreaAndPerimeter

Syntax

```
getAreaAndPerimeter(  
    instance: oaInst,  
    /,  
    *,  
    perFinger: bool = None,  
    includeGateEdges: bool = None,  
    depth: int = None)  
Returns: None
```

Summary:

Computes area and perimeter of source and drain regions of passed oaInst device.

Category:

Layout Editing

Licenses:

Basic

Parameters

- instance – The oaInst handle
- perFinger – Specifies if area values are to be scaled per finger (Default is: False)
- includeGateEdges – Specifies if gate edges should be included in perimeter computation (Default is: False)
- depth – Option to specify number of hierarchical levels to traverse. A negative number indicates all levels

Returns

None

getBoundaryTrackRegionDefs

Syntax

```
getBoundaryTrackRegionDefs(  
    pattern: str = None,
```

```
//  
*,  
tech: oaTech)  
Returns: list[lbBoundaryTrackRegionDef]
```

Summary:

Collection of lbBoundaryTrackRegionDef objects.

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Regexp pattern for name matching. (Default is: '*')
- tech – Technology to search.

Returns

Collection of lbBoundaryTrackRegionDef objects.

getCommonPathPoints

Syntax

```
getCommonPathPoints(  
*,  
source: deFigure,  
targets: list[deFigure],  
distance: float)  
Returns: point
```

Summary:

List of violation points

Category:

Layout Editing

Licenses:

Basic

Parameters

- source – source object
- targets – target objects
- distance – distance from source

Returns

Point for first branch relative to targets

getCoordinateMarks

Syntax

```
getCoordinateMarks(  
*,  
    design: oaDesign,  
    type: le.getCoordinateMarks.Type = None)  
Returns: list[leCoordinateMark]
```

Summary:

Obtains the coordinate marks for a specified design.

Category:

Layout Editing

Licenses:

None

Parameters

- design – Design in which to look for the coordinate marks.
- type – Coordinate mark type ({ all | nonPersistent | persistent } (Default is: 'persistent'))

Returns

Collection of coordinate marks

getFlightlines

Syntax

```
getFlightlines(  
*,  
    of: list[oaGroup] | list[oaNet])  
Returns: list[leFlightline]
```

Summary:

Get flightlines.

Category:

Layout Editing

Licenses:

Basic

Parameters

of – Collection of net or group objects to retrieve flightline objects

Returns

Collection of flightline objects

getGateDistance

Syntax

```
getGateDistance(  
    object: oaDesign | oaInst,  
    /,  
    *,  
    rectangle: box = None,  
    lpps: list[str],  
    direction: le.getGateDistance.Direction,  
    type: le.getGateDistance.Type,  
    index: int = None,  
    halo: float = None,  
    extend: float = None,  
    extend_direction: le.getGateDistance.Extend_direction = None,  
    operation: le.getGateDistance.Operation = None,  
    split: bool = None,  
    corner: le.getGateDistance.Corner = None,  
    depth: int = None,  
    debug: bool = None)  
Returns: float
```

Summary:

Computes average distance from 'gate' edge to the edges from target lpps.

Category:

Layout Editing

Licenses:

Basic

Parameters

- object – The oaInst | oaDesign handle
- rectangle – Option to specify Gate rectangle
- lpps – Option to specify target LPPs
- direction – Option to specify direction of computation ({ bottom | left | left_bottom | left_top | right | right_bottom | right_top | top })
- type – Option to specify distance type ({ any | external | internal })
- index – Option to specify distance index (Default is: 1)
- halo – Option to specify neighborhood range
- extend – Option to specify extension value for gate edge
- extend_direction – Option to specify direction of computation ({ bottom | left | right | top })
- operation – Option to specify boolean operation across layers ({ and | not | or })
- split – Option to specify if detailed split results are needed (Default is: False)
- corner – Option to specify corner type for computation ({ concave | convex })
- depth – Option to specify number of hierarchical levels to traverse. A negative number indicates all levels
- debug – Option to enable internal debugging (Default is: False)

Returns

Computed average distance

getGateRectangles

Syntax

```
getGateRectangles(  
    instance: oaInst,  
    /,  
    *,  
    depth: int = None)  
Returns: str
```


Summary:

Returns transistor instance Gate geometry as list of rectangles.

Category:

Layout Editing

Licenses:

Basic

Parameters

- instance – The oalnst handle
- depth – Option to specify number of hierarchical levels to traverse. A negative number indicates all levels

Returns

Computed Gate rectangles

getGlobalTrackRegions**Syntax**

```
getGlobalTrackRegions(  
    pattern: str = None,  
    /,  
    *,  
    design: oaDesign)  
Returns: list[leGlobalTrackRegion]
```

Summary:

Collection of leGlobalTrackRegion objects.

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Regexp pattern for name matching. (Default is: '*')
- design – Design to search.

Returns

Collection of leGlobalTrackRegion objects.

getIncompleteNets

Syntax

```
getIncompleteNets(  
    object: deContext | oaDesign  
Returns: list[oaNet]
```

Summary:

Get incomplete nets.

Category:

Layout Editing

Licenses:

Basic

Parameters

object – Design or context to retrieve incomplete nets

Returns

Collection of net objects

getIntersections

Syntax

```
getIntersections(  
    object: oaDesign | oaRef,  
    /,  
    *,  
    point: point,  
    direction: le.getIntersections.Direction,  
    length: float,  
    lpp1: lpp,  
    operation: le.getIntersections.Operation = None,  
    lpp2: lpp = None,  
    halo: float = None,  
    depth: int = None,  
    stepSize: int = None)  
Returns: str
```

Summary:

Gets intersections of asked LPPs on specified query line.

Category:

Layout Editing

Licenses:

Basic

Parameters

- `object` – The `oaRef` | `oaDesign` handle
- `point` – Option to specify starting point for query line
- `direction` – Option to specify direction of query line ({ `bottom` | `left` | `right` | `top` })
- `length` – Option to specify length of query line
- `lpp1` – Option to specify first LPP for query
- `operation` – Option to specify boolean operation across layers ({ `and` | `not` | `not_touch` | `or` | `touch` })
- `lpp2` – Option to specify second LPP for query
- `halo` – Option to specify neighborhood range
- `depth` – Option to specify number of hierarchical levels to traverse. A negative number indicates all levels
- `stepSize` – Option to specify step size in number of manufacturing grids for converting non orthogonal edges to orthogonal edges

Returns

Computed intersection values

getMPPDefs

Syntax

```
getMPPDefs(  
    pattern: str = None,  
    /,  
    *,  
    in_: oaDesign | oaTech,  
    local: bool = None)
```

```
Returns: list[leMPPDef] | leMPPEnclosedPathDef |  
leMPPEnclosedShapeDef | leMPPMasterPathDef | leMPPOffsetPathDef |  
leMPPRingDef | leMPPSubRectDef
```

Summary:

Returns a collection of MPP Definitions

Category:

Data Management

Licenses:

None

Parameters

- pattern – Globe-style pattern to match the object name. (Default is: '.')
- in_ – Technology or design from which to collect for the MPP objects
- local – Only consider the given technology (Default is: False)

Returns

Collection of MPP Definition objects that match the search criteria.

getMPPs

Syntax

```
getMPPs(  
pattern: str = None,  
/,  
*,  
design: oaDesign)  
Returns: list[deFigure]
```

Summary:

Returns a collection of MPPs

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Globe-style pattern to match the object name. (Default is: '.')
- design – Design in which to look for the MPP object.

Returns

Collection of MPP objects that match the search criteria.

getNearestTracks

Syntax

```
getNearestTracks(  
point: point,  
/  
*,  
design: oaDesign,  
layer: str,  
numTracks: int = None,  
color: str = None,  
width: float = None,  
direction: le.getNearestTracks.Direction = None,  
sigType: le.getNearestTracks.SigType = None,  
trackType: le.getNearestTracks.TrackType = None,  
snapMode: le.getNearestTracks.SnapMode = None)  
Returns: dbCollection
```

Summary:

Return effective tracks nearest to a point.

Category:

Layout Editing

Licenses:

None

Parameters

- point – point argument specifies the point at which should be nearest the tracks.
- design – Design in which should be find the tracks
- layer – layer argument specified the exact layer.
- numTracks – numTracks argument specifies the number of tracks. (Default is: 1)
- color – color argument specifies the exact color.

- width – width argument specifies the exact width.
- direction – direction argument specifies the exact direction. ({ horizontal | vertical })
- sigType – sigType argument specifies the exact signalType. ({ all | analog | clock | ground | power | reset | scan | signal | tieHi | tieLo | tieoff })
- trackType – trackType argument specifies the exact trackType. ({ all | default | nonReserved | reserved }) (Default is: 'all')
- snapMode – snapMode argument specifies the snapping mode ({ down | nearest | up }) (Default is: 'nearest')

Returns

Collection of track info.

getNetGroups

Syntax

```
getNetGroups(  
  object: deContext | oaDesign  
Returns: list[oaGroup]
```

Summary:

Get net groups.

Category:

Layout Editing

Licenses:

Basic

Parameters

object – Design or context to retrieve net groups

Returns

Collection of group objects

getNetIslands

Syntax

```
getNetIslands(  
  *,
```

```
of: oaNet)  
Returns: None
```

Summary:

Get islands collection of oa::oaNet.

Category:

Layout Editing

Licenses:

Basic

Parameters

of – Net where need to get islands.

Returns

None

getPseudoParallelNets

Syntax

```
getPseudoParallelNets(  
*,  
  design: oaDesign)  
Returns: list[netsCollection]
```

Summary:

Get Pseudo Parallel nets collection.

Category:

Layout Editing

Licenses:

Basic

Parameters

design – Design where need to get Pseudo Parallel Nets.

Returns

Collection of Pseudo Parallel Nets

getRouteCorridors

Syntax

```
getRouteCorridors(  
    pattern: str = None,  
    /,  
    *,  
    of: oaDesign)  
Returns: list[leRouteCorridor]
```

Summary:

Collection of leRouteCorridors matching the pattern and the filter.

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Pattern for objects name matching. (Default is: '*')
- of – Design to fetch routing corridors from.

Returns

Collection of routing corridors in the design.

getRuleInfoByICV

Syntax

```
getRuleInfoByICV(  
    *,  
    type: str = None,  
    runset: str = None,  
    oriRunset: str = None,  
    layerMap: str = None,  
    runDir: str = None,  
    arguments: str = None,  
    name: str = None,  
    queryDelimiter: str = None,  
    design: oaDesign,  
    controlVars: str = None,  
    background: bool = None)  
Returns: None
```


Summary:

Get Rule Information by ICV Live

Category:

Layout Editing

Licenses:

Backend

Parameters

- type – What type of information to get
- runset – Runset file to acquire information
- oriRunset – Original runset file to acquire information
- layerMap – Layer mapping file to acquire information
- runDir – Run directory
- arguments – The arguments to acquire rule information
- name – The rule name to acquire rule description
- queryDelimiter – The rule name to acquire rule delimiter
- design – Design in which to get all layer information
- controlVars – The control variables to configure runset
- background – Parse icv runset in the background

Returns

None

getStripedWires

Syntax

```
getStripedWires(  
    pattern: str = None,  
    //,  
    *,  
    design: oaDesign)  
Returns: list[oaFigGroup]
```

Summary:

Returns a collection of striped wires

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Globe-style pattern to match the object name. (Default is: `'.*'`)
- design – Design in which to look for the stripedWire object.

Returns

Collection of stripedWire objects that match the search criteria.

getSyncGroups**Syntax**

```
getSyncGroups(  
    pattern: str = None,  
    /,  
    *,  
    design: oaDesign)  
Returns: list[oaGroup]
```

Summary:

Returns a collection of synchronous oaGroup objects for the design.

Category:

Layout Editing

Licenses:

Backend

Parameters

- pattern – Glob style pattern to match synchronous group names. (Default is: `'*'`)
- design – Design queried for synchronous oaGroup objects.

Returns

Collection of synchronous oaGroup objects found in the design.

getTrackGroupDefs

Syntax

```
getTrackGroupDefs(  
    pattern: str = None,  
    /,  
    *,  
    of: oaDesign | oaTech,  
    **kwargs)  
Returns: list[leTrackGroupDef]
```

Summary:

Collection of leTrackGroupDef objects.

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Regexp pattern for name matching. (Default is: '*')
- of – Container to search.

Returns

Collection of leTrackGroupDef objects.

getTrackPatternDefs

Syntax

```
getTrackPatternDefs(  
    pattern: str = None,  
    /,  
    *,  
    from_: oaDesign | oaTech = None,  
    of: oaDesign | oaTech = None)  
Returns: list[leTrackPatternDef]
```

Summary:

Collection of leTrackPatternDefs matching the pattern and the filter.

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Globe style pattern for objects name matching. (Default is: “*”)
- of – Container to search.

Returns

Collection of leTrackPatternDefs matching the pattern and the filter.

getTrackPatterns

Syntax

```
getTrackPatterns(  
pattern: str = None,  
/,  
*,  
from_: oaDesign = None,  
of: oaDesign = None)  
Returns: list[leTrackPattern]
```

Summary:

Collection of leTrackPatterns matching the pattern and the filter.

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Globe style pattern for objects name matching. (Default is: “*”)
- of – Design to search.

Returns

Collection of leTrackPatterns matching the pattern and the filter.

getTrackRegionDefs

Syntax

```
getTrackRegionDefs(  
    pattern: str = None,  
    /,  
    *,  
    of: oaDesign | oaTech,  
    **kwargs)  
Returns: list[leTrackRegionDef]
```

Summary:

Collection of leTrackRegionDef objects.

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Regexp pattern for name matching. (Default is: '*')
- of – Container to search.

Returns

Collection of leTrackRegionDef objects.

getTrackRegions

Syntax

```
getTrackRegions(  
    pattern: str = None,  
    /,  
    *,  
    design: oaDesign,  
    **kwargs)  
Returns: list[leTrackRegion]
```

Summary:

Collection of leTrackRegion objects.

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Regexp pattern for name matching. (Default is: '*')
- design – Design to search.

Returns

Collection of leTrackRegion objects.

getTracks**Syntax**

```
getTracks(  
pattern: str = None,  
/,  
*,  
from_: oaDesign | oaTech = None,  
of: oaDesign | oaTech = None,  
trackGroupDef: leTrackGroupDef = None,  
trackRegion: leGlobalTrackRegion | oaAreaBoundary = None,  
**kwargs)  
Returns: list[leTrack]
```

Summary:

Collection of leTrack objects.

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Regexp pattern for name matching. (Default is: '*')
- of – Container of the leTrack object to search.
- trackGroupDef – TrackGroup definition where leTrack is used
- trackRegion – leTrackRegion or leGlobalTrackRegion from which to return leTracks

Returns

Collection of leTrack objects.

getTracksInRegion

Syntax

```
getTracksInRegion(  
box: list[point],  
/,  
*,  
design: oaDesign,  
layer: str,  
color: str = None,  
width: float = None,  
direction: le.getTracksInRegion.Direction = None,  
sigType: le.getTracksInRegion.SigType = None,  
trackType: le.getTracksInRegion.TrackType = None)  
Returns: dbCollection
```

Summary:

Return effective tracks in a region.

Category:

Layout Editing

Licenses:

None

Parameters

- box – List of points
- design – Design in which should be find the tracks
- layer – layer argument specified the exact layer.
- color – color argument specifies the exact color.

- width – width argument specifies the exact width.
- direction – direction argument specifies the exact direction. ({ horizontal | vertical })
- sigType – sigType argument specifies the exact signalType. ({ all | analog | clock | ground | power | reset | scan | signal | tieHi | tieLo | tieoff })
- trackType – trackType argument specifies the exact trackType. ({ all | default | nonReserved | reserved }) (Default is: 'all')

Returns

Collection of track info.

getUnboundGlobalTrackRegions

Syntax

```
getUnboundGlobalTrackRegions(  
    pattern: str = None,  
    /,  
    *,  
    design: oaDesign)  
Returns: list[leGlobalTrackRegion]
```

Summary:

Collection of unbound leGlobalTrackRegion objects.

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Regexp pattern for name matching. (Default is: '*')
- design – Design to search.

Returns

Collection of unbound leGlobalTrackRegion objects.

getUnboundTrackRegions

Syntax

```
getUnboundTrackRegions(  
    pattern: str = None,  
    /,  
    *,  
    design: oaDesign)  
Returns: list[leTrackRegion]
```

Summary:

Collection of unbound leTrackRegion objects.

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Regexp pattern for name matching. (Default is: '*')
- design – Design to search.

Returns

Collection of unbound leTrackRegion objects.

getUnboundTracks

Syntax

```
getUnboundTracks(  
    pattern: str = None,  
    /,  
    *,  
    design: oaDesign)  
Returns: list[leTrack]
```

Summary:

Collection of unbound leTrack objects.

Category:

Layout Editing

Licenses:

None

Parameters

- pattern – Regexp pattern for name matching. (Default is: '*')
- design – Design to search.

Returns

Collection of unbound leTrack objects.

getUnnettedFigs

Syntax

```
getUnnettedFigs(  
*,  
    design: oaDesign)  
Returns: list[oaFig]
```

Summary:

Get unnetted figures of design.

Category:

Layout Editing

Licenses:

Basic

Parameters

design – Design to search in.

Returns

Collection of unnetted pin figures.

highlightConnected

Syntax

```
highlightConnected(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
    list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
    | list[oaVia],  
/,
```

```
*,
context: deContext,
optionSet: str = None,
name: str = None,
packet: str = None,
depth: le.highlightConnected.Depth = None,
highlightParents: bool = None,
propagatingShapesOnly: bool = None,
region: box = None,
initialColorPropagation: bool = None,
stopAt: le.highlightConnected.StopAt = None,
point: point = None)
Returns: deHighlightSet
```

Summary:

Adds a collection of connected objects to the highlight set

Category:

Design Editing

Licenses:

None

Parameters

- **objects** – Figures to initiate highlight connected
- **context** – Context to initiate highlight
- **optionSet** – Name of the dbGenOptionSet (setType highlightConnectedGenerator) to use (Default is: 'default')
- **name** – Name of highlight set to create or append to (Default is: "")
- **packet** – Packet to use when creating deHighlightSet (Default is: 'inherited')
- **depth** – Depth to trace figures ({ 0x | full | view } (Default is: 'view')
- **highlightParents** – Trace parent designs (Default is: True)
- **propagatingShapesOnly** – Limit tracing to shapes that are propagating (Default is: True)
- **region** – Limit tracing to region
- **initialColorPropagation** – Controls if color propagation performed prior to any erasing constraints applied (Default is: True)

- stopAt – Highlight connected stopping directive ({ branch | changeType | fullIsland | neighbor } (Default is: 'fullIsland'))
- point – Specifies coordinates of input point

Returns

Highlight set which is created by le::highlightConnected command

importCommandBindings

Syntax

```
importCommandBindings(  
*,  
    filePath: str)  
Returns: None
```

Summary:

Import interactive command key bindings.

Category:

Layout Editing

Licenses:

Basic

Parameters

filePath – Specify the import file.

Returns

None

importResistanceQuery

Syntax

```
importResistanceQuery(  
scope: oaDesign,  
/,  
*,  
    filePath: str,  
    append: bool = None)  
Returns: str
```

Summary:

import resistance query data from a file

Category:

Layout Editing

Licenses:

Backend

Parameters

- scope – design to be reported.
- filePath – Specify the file path to import query data from
- append – Sepecify the mode to import query data. If true, append the imported data to exiting data; if false, replace existing data with imported data (Default is: False)

Returns

success to import query data or not.

importVoltages**Syntax**

```
importVoltages(  
context: deContext = None,  
/,  
*,  
config: dmCellView | dmCellViewName,  
overwrite: bool = None)  
Returns: None
```

Summary:

Import voltages.

Category:

Layout Editing

Licenses:

Backend

Parameters

- context – Context object of the import voltages operation.
- config – The configuration view for the import voltages operation.
- overwrite – Overwrite the existing voltages. (Default is: True)

Returns

None

isInSync

Syntax

```
isInSync(  
    object: deFigure | oaFigGroup | oaGroup  
)  
Returns: bool
```

Summary:

Checks if the provided oaGroup is synchronized.

Category:

Layout Editing

Licenses:

Backend

Parameters

object – An oaGroup/oaFigGroup/deFigure object to check for synchronization.

Returns

If the provided oaGroup is synchronized.

snps.custom.cmd.le

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform layout editing.

This section covers the following topics:

merge

Syntax

```
merge(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
        list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
        | list[oaVia],  
/,  
*,  
numPoints: int = None,  
preserveRouteType: bool = None,  
preservePaths: bool = None,  
preserveNets: bool = None,  
preserveProperties: bool = None)  
Returns: list[oaFig]
```

Summary:

Merge

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – List of objects to merge.
- numPoints – Number of vertices to convert an ellipse into a polygon.
- preserveRouteType – Preserve route type if route types of source objects are same (Default is: True)
- preservePaths – Preserve path/pathSeg if result object type is polygon (Default is: False)
- preserveNets – Option to preserve or ignore nets (Default is: True)
- preserveProperties – Option to preserve or ignore properties (Default is: True)

Returns

Collection of merged shapes.

move

Syntax

```
move (
  objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |
    list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]
    | list[oaVia],
  /,
  *,
  dx: float = None,
  dy: float = None,
  angleMode: le.move.AngleMode = None,
  anchor: point = None,
  rotate: le.move.Rotate = None,
  dest: oaDesign = None,
  removeUnanchoredNet: bool = None)
Returns: list[deFigure] | list[oaFig]
```

Summary:

Move

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – List of objects to move.
- dx – X projection of move vector. (Default is: 0)
- dy – Y projection of move vector. (Default is: 0)
- angleMode – Specify angle mode to move objects. ({ anyAngle | diagonal | orthogonal } (Default is: 'anyAngle'))
- anchor – Anchor point.
- rotate – Rotation to apply on objects. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 } (Default is: 'R0'))
- dest – Design to move objects into.
- removeUnanchoredNet – Option to remove unanchored net (Default is: False)

Returns

Collection of moved objects.

padLabel

Syntax

```
padLabel(  
*,  
    design: oaDesign,  
    padLpp: list[lpp],  
    textLpp: list[lpp],  
    textHeight: float = None,  
    textXOffset: str = None,  
    textYOffset: str = None,  
    padOperator1: le.padLabel.PadOperator1 = None,  
    padOperator2: le.padLabel.PadOperator2 = None,  
    padSize1: float = None,  
    padSize2: float = None,  
    insertFile: list[str] = None,  
    outputFile: list[str] = None,  
    levels: int = None)  
Returns: None
```

Summary:

Generate pad label

Category:

Layout Editing

Licenses:

Basic

Parameters

- design – Design in which to create the padLabel.
- padLpp – LPP of pad.
- textLpp – LPP of text.
- textHeight – Text height.
- textXOffset – Text x offset.
- textYOffset – Text y offset.

- padOperator1 – Pad area search operator. ({ equal | greater | greaterEqual | less | lessEqual })
- padOperator2 – Pad area search operator. ({ equal | greater | greaterEqual | less | lessEqual })
- padSize1 – Pad area size.
- padSize2 – Pad area size.
- insertFile – Insert file path.
- outputFile – Output file path.
- levels – Number of hierarchical levels for pad label. (Default is: 0)

Returns

None

pasteLook

Syntax

```
pasteLook(  
*,  
  objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
           list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
           | list[oaVia])  
Returns: list[oaFig]
```

Summary:

Paste look from selection

Category:

Layout Editing

Licenses:

Basic

Parameters

objects – List of objects to be pasted.

Returns

Collection of pasted objects.

propagateConnectivity

Syntax

```
propagateConnectivity(  
*,  
    design: oaDesign = None,  
    objects: list[oaInstTerm] | list[oaNet] | list[oaTerm] = None,  
    hierarchyMode: le.propagateConnectivity.HierarchyMode = None,  
    depth: int = None,  
    mode: le.propagateConnectivity.Mode = None)  
Returns: None
```

Summary:

Propagate Connectivity

Category:

Layout Editing

Licenses:

Basic

Parameters

- design – Design object.
- objects – Collection of objects.
- hierarchyMode – Depth mode ({ full | top | view })
- depth – depth
- mode – Connectivity propagation mode ({ netlist | shape })

Returns

None

refreshDensityResult

Syntax

```
refreshDensityResult(  
window: deCanvas | giWindow | int = None,  
/,  
*,  
    layer: str = None,  
    group: str = None,
```

```
all: bool = None)
Returns: None
```

Summary:

Refresh the density result.

Category:

Layout Editing

Licenses:

Backend

Parameters

- window – Window or canvas
- layer – Layer name in layer mode to refresh density.
- group – Group name in config file mode to refresh density.
- all – Specify whether to refresh all results. Ignore layer and group option if specified.
(Default is: False)

Returns

None

regenerateMPP

Syntax

```
regenerateMPP(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia])  
Returns: list[oaFigGroup]
```

Summary:

Regenerate MPP

Category:

Layout Editing

Licenses:

Basic

Parameters

objects – List of mpps to regenerate.

Returns

Collection of regenerated mpps

remaster

Syntax

```
remaster(  
objects:  oaDesign  |  list[oaInst],  
/,  
*,  
colorCellMap:  str  =  None)  
Returns:  None
```

Summary:

Remaster instances

Category:

Layout Editing

Licenses:

Backend

Parameters

- objects – Design or instances to be remastered.
- colorCellMap – Color cell mapping files.

Returns

None

removeFromFigGroup

Syntax

```
removeFromFigGroup(  
objects:  list[deFigure]  |  list[deSelectionSet]  |  list[oaFig]  |  
list[oaFigGroup]  |  list[oaInst]  |  list[oaMarker]  |  list[oaShape]  
|  list[oaVia],  
/,  
*,
```

```
depth: le.removeFromFigGroup.Depth = None)
Returns: list[oaFigGroup]
```

Summary:

Remove objects from the figure group

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – Collection of objects to remove from the given figure group
- depth – Specifies whether or not given objects contained by a nested oaFigGroup will be removed from the current group only. ({ allNested | current } (Default is: 'current'))

Returns

Collection of OA figure group objects updated by the command.

removeOverlappingCuts

Syntax

```
removeOverlappingCuts (
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |
list[oaFigGroup] | list[oaInst] = None,
/,
*,
design: oaDesign,
area: le.removeOverlappingCuts.Area = None,
level: int = None)
Returns: None
```

Summary:

Remove overlapping cuts of guard rings

Category:

Layout Editing

Licenses:

Basic

Parameters

- **objects** – List of objects to consider for the overlapping cut removal of guard rings. This argument is must when area mode is set as selection.
- **design** – Design in which to create the remove overlapping cuts.
- **area** – The area mode for remove overlapping cuts command ({ design | selection } (Default is: 'design'))
- **level** – The hierarchical levels of guard rings should be considered (Default is: 0)

Returns

None

removeUnboundInstanceTerms

Syntax

```
removeUnboundInstanceTerms (  
*,  
    design:  oaDesign)  
Returns: None
```

Summary:

Remove unbound instance terms

Category:

Layout Editing

Licenses:

Basic

Parameters

design – Design to remove unbound instance terms.

Returns

None

repeatCopy

Syntax

```
repeatCopy (  
*,
```

```
    window: giWindow | int = None,  
    context: deContext = None)  
Returns: None
```

Summary:

Repeat last copy operation on selected objects

Category:

Layout Editing

Licenses:

Basic

Parameters

- window – Window in which to repeat copy.
- context – Context in which to repeat copy.

Returns

None

replaceVia

Syntax

```
replaceVia(  
    *,  
    design: oaDesign,  
    sourceCutNum: int = None,  
    targetCutNum: int = None)  
Returns: None
```

Summary:

Replace the vias with the source cut numbers to the vias with the target cut numbers.

Category:

Layout Editing

Licenses:

Basic

Parameters

- design – Design to replace via.
- sourceCutNum – Source cut number. (Default is: 1)
- targetCutNum – Target cut number. (Default is: 2)

Returns

None

reportCapacitance

Syntax

```
reportCapacitance(  
    scope: oaDesign | list[oaNet],  
    /,  
    *,  
    groupResult: str = None,  
    filePath: str = None,  
    verbose: bool = None,  
    blockingMode: bool = None,  
    exportToErrorViewer: bool = None,  
    **kwargs)  
Returns: str
```

Summary:

Report capacitance information

Category:

Layout Editing

Licenses:

Backend

Parameters

- scope – net to be reported.
- groupResult – Specify the way how to group capacitance result ({ Net | Layer | All }, default is Net)
- filePath – Specify csv file path.
- verbose – Specify whether to report polygon for point-to-point.

- `blockingMode` – Specify whether to run capacitance reporter in blocking mode.
- `exportToErrorViewer` – export result to error viewer (Default is: False)

Returns

The total capacitance and length of query net, detailed capacitance from the query net to other net .

reportCELayers

Syntax

```
reportCELayers(  
    design: oaDesign)  
Returns: None
```

Summary:

lists layers used by Connectivity an Highlight Connected

Category:

Layout Editing

Licenses:

Basic

Parameters

`design` – design object

Returns

None

reportConnectivityPoints

Syntax

```
reportConnectivityPoints(  
    *,  
    design: oaDesign,  
    file: str,  
    skipCutLayers: bool = None,  
    includeHierPath: bool = None)  
Returns: None
```

Summary:

Reports the points where need to be created pins.

Category:

Layout Editing

Licenses:

Elite

Parameters

- design – design object.
- file – Output file for reporting.
- skipCutLayers – Specifies whether we should report the cut layers object or not.
- includeHierPath – Specifies whether we should report the cut layers object or not.

Returns

None

reportEM**Syntax**

```
reportEM(  
scope: list[oaNet],  
/,  
*,  
compare: str = None,  
filePath: str = None,  
verbose: bool = None,  
showLength: bool = None,  
blockingMode: bool = None,  
exportToErrorViewer: bool = None,  
**kwargs)  
Returns: str
```

Summary:

Report electro migration information

Category:

Layout Editing

Licenses:

Backend

Parameters

- scope – nets to be reported.
- compare – Specify EM compare type (current or dimension). (Default is: 'current')
- filePath – Specify csv file path.
- verbose – Specify whether to report polygon for point-to-point.
- showLength – Specify whether to show EM length.
- blockingMode – Specify whether to run EM reporter in blocking mode.
- exportToErrorViewer – Specify whether to export result to error viewer (Default is: False)

Returns

The total electro migration.

reportNets

Syntax

```
reportNets(  
objects: oaDesign | list[oaGroup] | list[oaNet],  
/,  
*,  
ratio: float = None,  
length: float = None,  
filePath: str = None,  
**kwargs)  
Returns: str
```

Summary:

Report nets information

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – nets, net groups or design to be reported.
- ratio – Specify the routed / manhattan length ratio. Nets with ratio over the value will be reported (Default is: 0)
- length – Specify a length threshold. Nets with routed length over the value will be reported (Default is: 0)
- filePath – Specify the output file of CSV type.

Returns

The report.

reportRC

Syntax

```
reportRC(  
    scope: oaDesign | oaNet | list[oaNet],  
    /,  
    *,  
    reportUsing: str = None,  
    source: str = None,  
    destination: str = None,  
    sourceShorted: bool = None,  
    destinationShorted: bool = None,  
    query: str = None,  
    queryAll: bool = None,  
    depth: str = None,  
    region: str = None,  
    compare: bool = None,  
    operatingPoint: le.reportRC.OperatingPoint = None,  
    spacingMode: le.reportRC.SpacingMode = None,  
    densityMode: le.reportRC.DensityMode = None,  
    density: float = None,  
    filePath: str = None,  
    blockingMode: bool = None,  
    exportToErrorViewer: bool = None,  
    reportElmoreDelay: bool = None,  
    reportLayerDistribution: bool = None,  
    **kwargs)  
Returns: str
```

Summary:

Report resistance and capacitance information

Category:

Layout Editing

Licenses:

Backend

Parameters

- scope – net or design to be reported.
- reportUsing – Specify the type of resistance reporter. Default: Selected Path.
- source – Specify source point and layer. It is ignored when -reportUsing is not 'Selected Path'
- destination – Specify destination point and layer. It is ignored when -reportUsing is not 'Selected Path'
- sourceShorted – Specify whether the source shorted. It is ignored when -reportUsing is not 'Selected Path'
- destinationShorted – Specify whether the destination shorted. It is ignored when -reportUsing is not 'Selected Path'
- query – Specify query set name. It is ignored when -reportUsing is not 'Selected Path'
- queryAll – Specify whether to report all query set. It is ignored when -reportUsing is not 'Selected Path'
- depth – Specify the depth of resistance reporter. Default is using preference 'leElectricalReporterViewDepth'
- region – Specify the region {{llx lly} {urx ury}} of resistance reporter. If it is not specified, the resgion is full design
- compare – Specify whether to report the status (pass/fail) when reporting type is 'All Constraints'
- operatingPoint – Specify the operating point. ({ maximum | minimum })
- spacingMode – Specify the spacing mode. ({ layout | minimum })
- densityMode – Specify the density mode. ({ layout | specified })
- density – Specify the density value.
- filePath – Specify csv file path.
- blockingMode – Specify whether to run resistance reporter in blocking mode.

- `exportToErrorViewer` – Specify whether to export result to error viewer (Default is: False)
- `reportElmoreDelay` – Specify whether to report elmore delay (Default is: False)
- `reportLayerDistribution` – Specify whether to report layer distributio (Default is: False)

Returns

The total resistance and capacitance.

reportResistance

Syntax

```
reportResistance(  
scope: oaDesign | oaNet | list[oaNet],  
/  
*,  
reportUsing: str = None,  
source: str = None,  
destination: str = None,  
sourceShorted: bool = None,  
destinationShorted: bool = None,  
query: str = None,  
queryAll: bool = None,  
depth: str = None,  
region: str = None,  
compare: bool = None,  
operatingPoint: le.reportResistance.OperatingPoint = None,  
spacingMode: le.reportResistance.SpacingMode = None,  
densityMode: le.reportResistance.DensityMode = None,  
density: float = None,  
filePath: str = None,  
blockingMode: bool = None,  
exportToErrorViewer: bool = None,  
reportElmoreDelay: bool = None,  
reportLayerDistribution: bool = None,  
**kwargs)  
Returns: str
```

Summary:

Report resistance information

Category:

Layout Editing

Licenses:

Backend

Parameters

- scope – net or design to be reported.
- reportUsing – Specify the type of resistance reporter. Default: Selected Path.
- source – Specify source point and layer. It is ignored when -reportUsing is not 'Selected Path'
- destination – Specify destination point and layer. It is ignored when -reportUsing is not 'Selected Path'
- sourceShorted – Specify whether the source shorted. It is ignored when -reportUsing is not 'Selected Path'
- destinationShorted – Specify whether the destination shorted. It is ignored when -reportUsing is not 'Selected Path'
- query – Specify query set name. It is ignored when -reportUsing is not 'Selected Path'
- queryAll – Specify whether to report all query set. It is ignored when -reportUsing is not 'Selected Path'
- depth – Specify the depth of resistance reporter. Default is using preference 'leElectricalReporterViewDepth'
- region – Specify the region {{llx lly} {urx ury}} of resistance reporter. If it is not specified, the resgion is full design
- compare – Specify whether to report the status (pass/fail) when reporting type is 'All Constraints'
- operatingPoint – Specify the operating point. ({ maximum | minimum })
- spacingMode – Specify the spacing mode. ({ layout | minimum })
- densityMode – Specify the density mode. ({ layout | specified })
- density – Specify the density value.
- filePath – Specify csv file path.
- blockingMode – Specify whether to run resistance reporter in blocking mode.
- exportToErrorViewer – Specify whether to export result to error viewer (Default is: False)
- reportElmoreDelay – Specify whether to report elmore delay (Default is: False)
- reportLayerDistribution – Specify whether to report layer distributio (Default is: False)

Returns

The total resistance.

reportShield

Syntax

```
reportShield(  
objects: oaDesign | list[oaGroup] | list[oaNet],  
/  
*,  
filePath: str = None,  
**kwargs)  
Returns: str
```

Summary:

Report net shield information

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – nets, net groups or design to be reported.
- filePath – Specify the output file of CSV type.

Returns

Report net shield information.

resetVoltageOverrides

Syntax

```
resetVoltageOverrides(  
context: deContext = None,  
/  
*,  
nets: list[oaBitNet | str] = None)  
Returns: None
```

Summary:

Removes override voltage values and return to the original source values.

Category:

Layout Editing

Licenses:

Backend

Parameters

- context – Context object of the reset operation.
- nets – Collection of nets whose voltage values will be reset. Operate on all nets if not specified.

Returns

None

reshape

Syntax

```
reshape(  
object: deFigure | oaFig,  
/,  
*,  
type: le.reshape.Type,  
points: list[point],  
clockwise: bool = None,  
op: le.reshape.Op = None,  
numPoints: int = None)  
Returns: list[oaFig]
```

Summary:

Reshape

Category:

Layout Editing

Licenses:

Basic

Parameters

- object – Object to reshape.
- type – Shape type of point list. ({ polygon | polyline | rectangle })
- points – Point list for object, based on which a new shape is formed.
- clockwise – Portion of the path (relative to direction from first to second points from point list) to be replaced if reshape object is path or path-seg otherwise the direction in which the point list is traversed. (Default is: True)
- op – Reshape mode for polygon and rectangle shape type ({ ANDNOT | OR } (Default is: 'OR'))
- numPoints – Number of vertices to convert an ellipse into a polygon.

Returns

Collection of reshaped objects.

resize

Syntax

```
resize(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
        list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
        | list[oaVia],  
/  
*,  
sizeOperation: le.resize.SizeOperation = None,  
size: float = None,  
left: float = None,  
right: float = None,  
top: float = None,  
bottom: float = None,  
result: le.resize.Result = None)  
Returns: list[oaFig]
```

Summary:

Resize

Category:

Layout Editing

Licenses:

Basic

Parameters

- **objects** – List of objects to resize.
- **sizeOperation** – Size operation which is performed. ({ anisotropic | isotropic })
- **size** – Resize uniform value for shape objects.
- **left** – Resize various value for shape objects.
- **right** – Resize various value for shape objects.
- **top** – Resize various value for shape objects.
- **bottom** – Resize various value for shape objects.
- **result** – Result operation which is performed. ({ newWithActiveLPP | replace | replaceWithActiveLPP })

Returns

Collection of resized objects.

rotate

Syntax

```
rotate(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
        list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
        | list[oaVia],  
/  
*,  
angle: float = None,  
flip: le.rotate.Flip = None,  
origin: point = None,  
numPoints: int = None,  
separate: bool = None,  
just: le.rotate.Just = None)  
Returns: list[oaFig]
```

Summary:

Rotate

Category:

Layout Editing

Licenses:

Basic

Parameters

- `objects` – List of objects to be rotated.
- `angle` – Angle of rotation.
- `flip` – Mirror type of rotation. ({ `horizontal` | `vertical` })
- `origin` – Location of the origin of rotation.
- `numPoints` – Number of vertices to convert an ellipse into a polygon.
- `separate` – Flag to turn on/off rotating objects separately (Default is: `False`)
- `just` – Justification of the rotation with respect to the origin. ({ `centerCenter` | `centerLeft` | `centerRight` | `lowerCenter` | `lowerLeft` | `lowerRight` | `origin` | `upperCenter` | `upperLeft` | `upperRight` }) (Default is: `'origin'`)

Returns

Rotated objects.

scalarizeArrayInst

Syntax

```
scalarizeArrayInst(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia])  
Returns: list[oaInst]
```

Summary:

Convert array instances to collection of scalar instances.

Category:

Layout Editing

Licenses:

Basic

Parameters

`objects` – Collection of array instances to scalarize.

Returns

Collection of instances after scalarization.

setNetAnchor

Syntax

```
setNetAnchor(  
collection: list[deFigure] | list[oaShape] | list[oaVia],  
/,  
*,  
value: bool)  
Returns: None
```

Summary:

Sets anchors to pin figures.

Category:

Layout Editing

Licenses:

Basic

Parameters

- collection – The collection of oaPinFig or deFigures for the command to work on
- value – Value to set connectivity anchor

Returns

None

showAlignAssistOptions

Syntax

```
showAlignAssistOptions()  
Returns: None
```

Arguments:

Summary:

Shows the Align Assist Options dialog. If the dialog is already opened for the given window, it will be raised and the existing object returned.

Category:

Canvas

Licenses:

None

Returns

None

showCommandBindings

Syntax

```
showCommandBindings(  
*,  
    parent: giWindow | int = None)  
Returns: None
```

Summary:

Display the interactive command key bindings.

Category:

Layout Editing

Licenses:

Basic

Parameters

parent – Window(id) of layout editor window to show command binding

Returns

None

showCreateTrack

Syntax

```
showCreateTrack(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows Create Track Dialog

Category:

Layout Editing

Licenses:

Backend

Parameters

parent – Parent window.

Returns

The created or existing Create Track Dialog

showCreateTrackGroupDefinition

Syntax

```
showCreateTrackGroupDefinition(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows Create Track Group Definition Dialog

Category:

Layout Editing

Licenses:

None

Parameters

parent – Parent window.

Returns

The created or existing Create Track Group Definition Dialog

showCreateTrackRegionDefinition

Syntax

```
showCreateTrackRegionDefinition(  
*,
```



```
parent: giWindow | int = None)
Returns: giDialog
```

Summary:

Shows Create Track Region Definition Dialog

Category:

Layout Editing

Licenses:

Backend

Parameters

parent – Parent window.

Returns

The created or existing Create Track Region Definition Dialog

showDefineTerminals

Syntax

```
showDefineTerminals (
*,
parent: giWindow | int = None)
Returns: giDialog
```

Summary:

Show Define terminal command

Category:

Layout Editing

Licenses:

Basic

Parameters

parent – Window for which to show this dialog

Returns

The Define Terminals dialog

showDesignOptions

Syntax

```
showDesignOptions(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows the Layout Design Options dialog. If the dialog is already opened for the given window, it will be raised and the existing object returned.

Category:

Layout Editing

Licenses:

None

Parameters

parent – Parent window.

Returns

Newly opened or already opened dialog

showDRDOptions

Syntax

```
showDRDOptions(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows DRD Options

Category:

Layout Editing

Licenses:

Backend

Parameters

parent – Parent window.

Returns

Dialog containing drd options

showDRDRuleSelection

Syntax

```
showDRDRuleSelection(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows DRD Rule Selection

Category:

Layout Editing

Licenses:

Backend

Parameters

parent – Parent window.

Returns

Dialog containing drd options

showEditTrack

Syntax

```
showEditTrack(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows Edit Track Dialog

Category:

Layout Editing

Licenses:

None

Parameters

parent – Parent window.

Returns

The created or existing Edit Track Dialog

showEditTrackGroupDefinition

Syntax

```
showEditTrackGroupDefinition(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows Edit Track Group Defintion Dialog

Category:

Layout Editing

Licenses:

None

Parameters

parent – Parent window.

Returns

The created or existing Edit Track Group Defintion Dialog

showEditTrackRegion

Syntax

```
showEditTrackRegion(  
*,
```

```
parent: giWindow | int = None)
Returns: giDialog
```

Summary:

Shows Edit Track Region Dialog

Category:

Layout Editing

Licenses:

None

Parameters

parent – Parent window.

Returns

The created or existing Edit Track Region Dialog

showImportLayout

Syntax

```
showImportLayout (
*,
parent: giWindow | int = None)
Returns: giDialog
```

Summary:

Open the LE Import Layout dialog.

Category:

Layout Editing

Licenses:

Basic

Parameters

parent – Window(id) of layout editor window to import layout in

Returns

The Dialog

showLayoutGeneratorOptions

Syntax

```
showLayoutGeneratorOptions(  
*,  
    parent: giDialog,  
    optionSet: str,  
    src: dmCellView,  
    dst: dmCellView)  
Returns: None
```

Summary:

Show generates layout option dialog

Category:

Layout Editor

Licenses:

Basic

Parameters

- parent – Parent dialog.
- optionSet – Option Set name.
- src – The source cellView.
- dst – The target cellView.

Returns

None

showOLPOptions

Syntax

```
showOLPOptions(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows Object/Layer Options Dialog

Category:

Layout Editing

Licenses:

None

Parameters

parent – Parent window.

Returns

The created or existing Object/Layer Options Dialog

showTrackRegionOptions

Syntax

```
showTrackRegionOptions (  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Shows Track Region Options Dialog

Category:

Layout Editing

Licenses:

None

Parameters

parent – Parent window.

Returns

The created or existing Track Region Options Dialog

split

Syntax

```
split(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
        list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
        | list[oaVia],  
/,  
*,  
type: le.split.Type,  
points: list[point],  
dx: float = None,  
dy: float = None,  
lockAngles: bool = None)  
Returns: list[oaFig]
```

Summary:

Split

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – Collection of objects to be split.
- type – Shape type for the point list. ({ polygon | polyline | rectangle })
- points – Point list for the object, based on which the split shape is determined.
- dx – Stretch in the horizontal direction. (Default is: 0)
- dy – Stretch in the vertical direction. (Default is: 0)
- lockAngles – Flag to turn on/off angle locking. (Default is: True)

Returns

Collection of split objects.

splitArray

Syntax

```
splitArray(  
  objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
           list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
           | list[oaVia],  
  /,  
  *,  
  points: list[point],  
  toggleArray: bool = None)  
Returns: list[oaInst]
```

Summary:

Split array

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – Collection of array instances to be split.
- points – Point list for the line, based on which the split segment is determined.
- toggleArray – Flag to toggle array or not. (Default is: False)

Returns

Collection of split instances or array instances.

splitIntoBoxes

Syntax

```
splitIntoBoxes(  
  object: oaPath | oaPathSeg | oaPolygon | oaRect,  
  /,  
  *,  
  type: le.splitIntoBoxes.Type = None)  
Returns: str
```

Summary:

Splits the given orthogonal shape object into a list of non-overlapping boxes.

Category:

Layout Editing

Licenses:

Basic

Parameters

- object – The object to be split
- type – Option to specify X | Y type of split ({ maxX | maxY })

Returns

List of boxes

stretch

Syntax

```
stretch(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
        list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
        | list[oaVia],  
/,  
*,  
dx: float = None,  
dy: float = None,  
lockAngles: bool = None,  
permitJogs: bool = None,  
keepConnected: le.stretch.KeepConnected = None,  
keepAbutment: bool = None,  
removeUnanchoredNet: bool = None)  
Returns: list[oaFig]
```

Summary:

This command receives list of partially or fully selected objects and stretches them based on modification vector.

Category:

Layout Editing

Licenses:

Basic

Parameters

- **objects** – The collection of full or partial objects that should be stretched.
- **dx** – X projection of stretch vector. (Default is: 0)
- **dy** – Y projection of stretch vector. (Default is: 0)
- **lockAngles** – Flag to specify if need to keep the same angles of stretchable objects. (Default is: True)
- **permitJogs** – Flat go turn on/off jog insertion at the end of path/pathSeg objects. (Default is: False)
- **keepConnected** – The connection mode for stretching ({ backbone | fixedVias | individual }) (Default is: 'individual')
- **keepAbutment** – Keep Abutment Flag (Default is: False)
- **removeUnanchoredNet** – Option to remove unanchored net (Default is: False)

Returns

Collection of stretched objects.

syncTo

Syntax

```
syncTo (  
  object: deFigure | oaFigGroup)  
Returns: oaGroup
```

Summary:

For a given oaFigGroup object, this command synchronizes it's peers within the same synchronous group to it.

Category:

Layout Editing

Licenses:

Backend

Parameters

object – An oaFigGroup or deFigure object to be synced.

Returns

The oaGroup object owning the input oaFigGroup and its peers.

toggleDRDTreeWidget

Syntax

```
toggleDRDTreeWidget(  
    _target_: giWidget,  
    /,  
    *,  
    checked: bool = None)  
Returns: None
```

Summary:

Toggle check state of a drdTreeWidget by selection

Category:

Graphical Infrastructure

Licenses:

None

Parameters

- `_target_` – Target widget to toggle selected items
- `checked` – Toggle selected items to checked or unchecked

Returns

None

trimWire

Syntax

```
trimWire(  
    objects: dbCollection | list[deFigure] | list[deSelectionSet] |  
             list[oaFig] | list[oaFigGroup] | list[oaInst] | list[oaMarker] |  
             list[oaNet] | list[oaShape] | list[oaVia],  
    /,  
    *,  
    method: le.trimWire.Method,  
    from_: le.trimWire.From_ = None,  
    of: le.trimWire.Of = None,  
    edge: le.trimWire.Edge = None,
```

```
preserveEndStyle: bool = None,  
byNet: bool = None,  
toConnected: bool = None,  
toViewLevel: bool = None,  
skipAndFindClosestAlignedEdge: bool = None)  
Returns: None
```

Summary:

Trim paths, branches or both

Category:

Layout Editing

Licenses:

Basic

Parameters

- objects – paths, pathsegs, vias or nets to be trimmed.
- method – Trimming method of Trim Wire Command. ({ both | branch | path })
- of – Trimming direction of path method. ({ auto | bottom | left | right | top }) (Default is: 'auto')
- edge – Ending edge of a via neighbor for path method. ({ path | via }) (Default is: 'path')
- preserveEndStyle – Preserve the original end style of path/pathseg objects after trimming. (Default is: True)
- byNet – Use the neighbors of the same net for trimming path/pathseg. (Default is: False)
- toConnected – Trim to connected object. (Default is: False)
- toViewLevel – Trim to view level. (Default is: True)
- skipAndFindClosestAlignedEdge – Skip and find next closest aligned edge. (Default is: False)

Returns

None

validateColor

Syntax

```
validateColor(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaNet] | list[oaShape] |  
         list[oaVia] = None,  
/,  
*,  
fix: bool = None,  
scope: le.validateColor.Scope = None,  
no_color: bool = None,  
track: bool = None,  
constraint: bool = None,  
conflict: bool = None,  
not_locked: bool = None,  
waive: bool = None,  
levels: int = None,  
**kwargs)  
Returns: None
```

Summary:

validateColor

Category:

Layout Editing

Licenses:

Backend

Parameters

- objects – List of objects to validate color.
- fix – Specifies whether to fix the violation or not (Default is: False)
- scope – Scope to validate color. ({ design | object | viewport } (Default is: 'object')
- no_color – Specifies whether to report no/same color violation or not. (Default is: True)
- track – Specifies whether to report track mismatch violation or not. (Default is: True)
- constraint – Specifies whether to report constraint mismatch violation or not. (Default is: True)
- conflict – Specifies whether to report color conflict violation or not. (Default is: True)
- not_locked – Specifies whether to report not locked violation or not. (Default is: False)

- **waive** – Specifies whether to waive violation of fully covered shapes. (Default is: False)
- **levels** – Number of hierarchical levels to validate. (Default is: 0)

Returns

None

yank

Syntax

```
yank(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
         | list[oaVia] = None,  
/,  
*,  
design: oaDesign = None,  
points: list[point],  
regionType: le.yank.RegionType = None,  
levels: int = None,  
anchor: point = None,  
snapToGrid: le.yank.SnapToGrid = None,  
filterByOLP: bool = None,  
flatten: bool = None,  
selectResult: bool = None)  
Returns: None
```

Summary:

Copies data by the given region into the clipboard.

Category:

Layout Editing

Licenses:

None

Parameters

- **objects** – List of objects to yank.
- **design** – Design in which to active yank command.
- **points** – A list of points defining the contour of shape that specifies the yank region.
- **regionType** – Type of region passed to the -points argument. ({ polygon | rectangle }
(Default is: 'rectangle')

- **levels** – Number of hierarchical levels beyond which to ignore instances which overlap the yank region. (Default is: 0)
- **anchor** – Anchor point for paste.
- **snapToGrid** – Specify if yanked results will be snapped to manufacturing grid. ({ abort | off | on } (Default is: 'on')
- **filterByOLP** – Specifies to filter yank results by OLP settings. (Default is: False)
- **flatten** – Specifies to flatten all input objects. This option only works when -filterByOLP is true. (Default is: False)
- **selectResult** – Whether add the result to the selection set. (Default is: True)

Returns

None

snps.custom.cmd.le.pd

This chapter lists the commands in the `pd` module for layout.

This section covers the following topics:

- [drawMap](#)
- [clearMap](#)
- [createMap](#)
- [deleteMap](#)
- [drawLegend](#)

drawMap

Syntax

```
drawMap(  
    map:  pdMeasurementMap,  
    /,  
    *,  
    sqlQuery:  str,  
    minValue:  float,  
    maxValue:  float,  
    drawText:  bool = None,  
    suffix:    str  = None,
```



```
precision: int = None)  
Returns: None
```

Summary:

Draws Rectangles on the measurement map.

Category:

Power Device

Licenses:

None

Parameters

- map – Measurement Map object
- sqlQuery – SQL Query for drawing measurement map
- minValue – Minimum Value for Show Measurement Map
- maxValue – Maximum Value for Show Measurement Map
- drawText – Draw Text on Show Measurement Map
- suffix – Text Suffix on Show Measurement Map
- precision – Value Precision on Show Measurement Map Map

Returns

None

clearMap

Syntax

```
clearMap(  
map: pdMeasurementMap)  
Returns: None
```

Summary:

Clear Show Measurement Map on specified design.

Category:

Power Device

Licenses:

None

Parameters

map – Measurement Map object

Returns

None

createMap

Syntax

```
createMap(  
*,  
    design: oaDesign,  
    sqlDBFileName: str)  
Returns: pdMeasurementMap
```

Summary:

Create Measurement Map on the specified design.

Category:

Power Device

Licenses:

None

Parameters

- design – Design in which to create measurement map
- sqlDBFileName – SQL DB File Name

Returns

Power Device Measurement Map Object

deleteMap

Syntax

```
deleteMap(  
map: pdMeasurementMap)  
Returns: None
```

Summary:

Delete Measurement Map.

Category:

Power Device

Licenses:

None

Parameters

map – Measurement Map object

Returns

None

drawLegend

Syntax

```
drawLegend(  
map:  pdMeasurementMap,  
/  
*,  
minValue:  float,  
maxValue:  float,  
suffix:  str = None,  
precision:  int = None)  
Returns:  None
```

Summary:

Draws Legend Rectangles on the measurement map.

Category:

Power Device

Licenses:

None

Parameters

- map – Measurement Map object
- minValue – Minimum Value for Show Measurement Map
- maxValue – Maximum Value for Show Measurement Map
- suffix – Text Suffix on Show Measurement Map Legend
- precision – Value Precision on Show Measurement Map Legend

Returns

None

snps.custom.cmd.le.pkg

This chapter lists the commands in the `pkg` module for layout, allowing you for power device creation and editing.

This section covers the following topics:

- [highlightPlaneBoundary](#)
- [realizePlane](#)
- [replacePlaneBoundary](#)
- [reportDifferentialPairConsistency](#)
- [reportRedundantVias](#)
- [reportShieldingRate](#)
- [checkPlaneBoundary](#)
- [createDegassingHole](#)
- [createPlaneBoundary](#)
- [deleteDegassingHole](#)
- [deleteFloatingShapes](#)
- [deleteRedundantVias](#)
- [generatePlaneBoundary](#)
- [generateRedundantVias](#)
- [generateSealRing](#)
- [generateZigzag](#)
- [getPlaneBoundaries](#)

highlightPlaneBoundary

Syntax

```
highlightPlaneBoundary(  
*,  
    design: oaDesign,  
    net: oaNet | str = None,  
    nets: str = None,  
    excludeNets: str = None,  
    lpp: lpp,  
    plane: bool = None,  
    sameNet: bool = None,  
    diffNet: bool = None,  
    openNet: bool = None)  
Returns: list[oaFig]
```

Summary:

highlight plane boundary

Category:

Layout Editing

Licenses:

Apex

Parameters

- design – Design in which to create the highlight plane.
- net – The net name or oaNet object to which highlight plane should be assigned.
- nets – Specify plane nets. (Default is: “)
- excludeNets – Specify plane exclude nets. (Default is: “)
- lpp – The lpp on which to create the highlight plane.
- plane – Highlight plane. (Default is: True)
- sameNet – Highlight same nets. (Default is: True)
- diffNet – Highlight different nets. (Default is: False)
- openNet – Highlight open net figures. (Default is: False)

Returns

Collection of highlight plane boundary.

realizePlane

Syntax

```
realizePlane(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaShape],  
/  
*,  
adoptTech: bool,  
wireWireSpacing: float = None,  
planeWireSpacing: float = None,  
planePlaneSpacing: float = None,  
keepBoundary: bool = None,  
planeWidthThreshold: float = None,  
patchConcave: bool = None,  
fixMinWidth: bool = None,  
removeFloating: bool = None,  
removeFloatingLength: float = None,  
hasD2D: bool = None,  
d2dLPP: lpp = None,  
planeWidthThresholdD2D: float = None,  
planeLineSpacingD2D: float = None)  
Returns: list[oaFig]
```

Summary:

Realize Plane

Category:

Layout Editing

Licenses:

Apex

Parameters

- objects – List of objects to realize.
- adoptTech – Look for related rules in packaging technology.
- wireWireSpacing – The spacing between two wires. (Default is: 10)
- planeWireSpacing – The spacing between plane to wire. (Default is: 10)
- planePlaneSpacing – The spacing between two planes. (Default is: 10)
- keepBoundary – Specifies whether to keep plane boundary after operation. (Default is: True)

- `planeWidthThreshold` – Shapes with a width wider than the value are treated as planes. Others are treated as wires. (Default is: 100)
- `patchConcave` – Automatically patch concave corners. (Default is: True)
- `fixMinWidth` – Automatically fix min width. (Default is: False)
- `removeFloating` – Automatically remove floating figures. (Default is: False)
- `removeFloatingLength` – Shapes with a length narrower than the value are treated as floating to remove. (Default is: 100)
- `hasD2D` – Whether to consider D2D-related rules. (Default is: False)
- `d2dLPP` – The lpp of D2D region.
- `planeWidthThresholdD2D` – Shapes with a width wider than the value are treated as planes in D2D region. Others are treated as wires.
- `planeLineSpacingD2D` – The spacing between plane and line in D2D region.

Returns

Collection of realized objects.

replacePlaneBoundary

Syntax

```
replacePlaneBoundary(  
    points: list[point],  
    /,  
    *,  
    design: oaDesign,  
    layers: str,  
    net: oaNet | str = None,  
    priority: int = None)  
Returns: None
```

Summary:

replace plane boundary

Category:

Layout Editing

Licenses:

Apex

Parameters

- points – The list of points that form plane.
- design – Design in which to create the plane.
- layers – The layers on which to replace the plane.
- net – The net name or oaNet object to which plane should be assigned.
- priority – Plane priority. (Default is: 15)

Returns

None

reportDifferentialPairConsistency

Syntax

```
reportDifferentialPairConsistency(  
diffpairs: oaDesign | str | list[oaGroup] | list[oaNet],  
/,  
*,  
filePath: str = None)  
Returns: None
```

Summary:

Report the Differential Pair Consistency

Category:

Layout Editing

Licenses:

Apex

Parameters

- diffpairs – constraint names, net names, constraint groups, nets, or current design to compute differential pair consistency
- filePath – output path for the report (Default is: “)

Returns

None

reportRedundantVias

Syntax

```
reportRedundantVias(  
*,  
    design: oaDesign,  
    net: oaNet | str = None)  
Returns: None
```

Summary:

Report Redundant Vias

Category:

Layout Editing

Licenses:

Apex

Parameters

- design – Design in which to report the redundantVias.
- net – The net name or oaNet object to which redundant vias are reported on.

Returns

None

reportShieldingRate

Syntax

```
reportShieldingRate(  
*,  
    design: oaDesign,  
    layerMappingFile: str = None,  
    arguments: str = None,  
    diebox: str = None,  
    d2d: str = None,  
    shieldSpacing: float = None,  
    shieldSpacingTolerance: float = None,  
    filePath: str = None)  
Returns: None
```

Summary:

Report Shielding Rate.

Category:

Layout Editing

Licenses:

Apex

Parameters

- design – Design to report shielding rate
- layerMappingFile – Specify the layer mapping file. (Default is: "")
- arguments – Specify the arguments passed to ICV. (Default is: '-oa_dm5')
- diebox – Specify the lpp of DIEBOX. (Default is: "")
- d2d – Specify the lpp of D2D. (Default is: "")
- shieldSpacing – Specify the spacing of shielding. (Default is: 10)
- shieldSpacingTolerance – Specify the tolerance of shielding spacing. (Default is: 0.001)
- filePath – Specify the output file. (Default is: "")

Returns

None

checkPlaneBoundary

Syntax

```
checkPlaneBoundary(  
*,  
    design: oaDesign,  
    mode: le.pkg.checkPlaneBoundary.Mode,  
    nets: str | list[oaNet] = None,  
    coordinateMark: bool = None,  
    floating: bool = None,  
    errorMarker: bool = None,  
    region: list[point] = None,  
    filePath: str = None)  
Returns: list[oaFig]
```

Summary:

check plane boundary

Category:

Layout Editing

Licenses:

Apex

Parameters

- design – Design in which to create the check plane.
- mode – Mode to report. ({ reportOpenAfterRealize | reportOpenBeforeRealize | reportPlaneGeneration })
- nets – Which nets or names of the nets to check LVS. If -nets is not specified, use the nets in the layer assignment defined in preference “rtPackagingPlaneBoundaryNetLayers”.
- coordinateMark – Create coordinate mark for open islands. (Default is: False)
- floating – Create coordinate mark on floating figures after plane realized. This option depends on -coordinateMark option (Default is: False)
- errorMarker – Create error marker for open bumps. (Default is: False)
- region – Point list region to create coordinate mark or error marker.
- filePath – Specify the output file.

Returns

Collection of open figures.

createDegassingHole

Syntax

```
createDegassingHole(  
*,  
  design: oaDesign,  
  layerMappingFile: str,  
  fobox: str,  
  keepRunResult: bool = None,  
  runset: str = None,  
  arguments: str = None,  
  diebox: str = None,  
  d2d: str = None,  
  ubm: str = None,  
  ipd: str = None,  
  bgab: str = None,  
  tiv: str = None)  
Returns: None
```

Summary:

Create degassing holes

Category:

Layout Editing

Licenses:

Apex

Parameters

- design – Design to create degassing hole
- layerMappingFile – Specify the layer mapping file.
- fobox – Specify the lpp of the fob.
- keepRunResult – Specify to keep ICV run result or not (Default is: False)
- runset – Specify the runset used for ICV. Auto generate runset if not specified. (Default is: “)
- arguments – Specify the arguments passed to ICV. (Default is: ‘-oa_dm5’)
- diebox – Specify the lpp of DIEBOX. (Default is: “)
- d2d – Specify the lpp of D2D. (Default is: “)
- ubm – Specify the lpp of UBM. (Default is: “)
- ipd – Specify the lpp of IPD. (Default is: “)
- bgab – Specify the lpp of BGAB region. (Default is: “)
- tiv – Specify the lpp of TIV. (Default is: “)

Returns

None

createPlaneBoundary

Syntax

```
createPlaneBoundary(  
    points: list[point],  
    /,  
    *,  
    design: oaDesign,  
    adoptTech: bool,  
    lpp: lpp,  
    net: oaNet | str = None,  
    wireWireSpacing: float = None,  
    planeWireSpacing: float = None,  
    planePlaneSpacing: float = None,  
)
```

```
planeWidthThreshold: float = None,  
excludeNet: str = None,  
patchConcave: bool = None,  
priority: int = None)  
Returns: list[oaFig]
```

Summary:

create plane boundary

Category:

Layout Editing

Licenses:

Apex

Parameters

- points – The list of points that form plane.
- design – Design in which to create the plane.
- adoptTech – Look for related rules in packaging technology.
- lpp – The lpp on which to create the plane.
- net – The net name or oaNet object to which plane should be assigned.
- wireWireSpacing – The spacing between two wires. (Default is: 10)
- planeWireSpacing – The spacing between plane to wire. (Default is: 10)
- planePlaneSpacing – The spacing between two planes. (Default is: 10)
- planeWidthThreshold – Shapes with a width wider than the value are treated as planes. Others are treated as wires. (Default is: 100)
- excludeNet – Exclude shapes of specified net. Separated by space. (Default is: “)
- patchConcave – Automatically patch concave corners. (Default is: True)
- priority – Plane priority. (Default is: 15)

Returns

Collection of created plane boundary.

deleteDegassingHole

Syntax

```
deleteDegassingHole(  
*,  
    design: oaDesign,  
    force: bool = None)  
Returns: None
```

Summary:

Delete degassing holes.

Category:

Layout Editing

Licenses:

Apex

Parameters

- design – Design in which to create the delete degassing holes.
- force – Force to overwrite design. (Default is: True)

Returns

None

deleteFloatingShapes

Syntax

```
deleteFloatingShapes(  
*,  
    design: oaDesign,  
    routerConfig: bool = None,  
    nets: str = None,  
    length: float = None)  
Returns: None
```

Summary:

delete floating shapes

Category:

Layout Editing

Licenses:

Apex

Parameters

- design – Design in which to create the deleteFloatingShapes.
- routerConfig – Specify nets from router configure file
- nets – Specify nets to destroy floating.
- length – Shapes with a length narrower than the value are treated as floating to delete. (Default is: 0)

Returns

None

deleteRedundantVias

Syntax

```
deleteRedundantVias(  
*,  
    box: box = None,  
    points: list[point] = None,  
    design: oaDesign)  
Returns: None
```

Summary:

Delete Redundant Vias

Category:

Layout Editing

Licenses:

Apex

Parameters

- box – Option to specify area for redundant via deletion.
- points – Option to specify area for redundant via deletion.
- design – Design in which to delete the redundant vias.

Returns

None

generatePlaneBoundary

Syntax

```
generatePlaneBoundary(  
*,  
  design: oaDesign,  
  config: list[str] = None,  
  initConstraintFile: str = None,  
  importConstraintFile: str = None,  
  exportConstraintFile: str = None,  
  net: oaNet | str = None,  
  oneIslandOnly: bool = None,  
  legalIslandOnly: bool = None,  
  combineCluster: bool = None,  
  pushRoute: bool = None,  
  combineRouter: bool = None,  
  excludeNets: str = None,  
  filterNets: str = None,  
  boneWithContour: bool = None,  
  primaryOnly: bool = None,  
  coverDiffNet: bool = None,  
  coverFOB: bool = None,  
  coverFOBSpacing: float = None,  
  escapeSpacing: str = None,  
  d2dEscapeSpacing: str = None,  
  d2dSizing: str = None,  
  surroundingComponentSizing: str = None,  
  surroundingDepth: float = None,  
  customizedClusterSpacing: float = None,  
  maxBoxClusterSpacing: float = None,  
  maxClusterCount: int = None,  
  maxBoxCount: int = None,  
  uTurn: str = None,  
  rmstMode: str = None,  
  escapeMode: str = None,  
  escapeDirection: str = None,  
  escapeNeighborSide: bool = None,  
  escapeFourDirection: bool = None,  
  escapeAllDifferentNet: bool = None,  
  mergeSameNet: bool = None,  
  tryNeighborSideCount: int = None,  
  zigzagMaxLength: float = None,  
  zigzagDepth: float = None,  
  minWidth: float = None,  
  minSpacing: float = None,  
  mergeShieldingPaths: bool = None,  
  mergeShapes: bool = None,  
  uniformGrow: list[bool] = None,  
  autoOrder: bool = None,  
  pointsCollection: bool = None,  
  patchConcave: bool = None,  

```



```
**kwargs)  
Returns: dbCollection | list[oaShape]
```

Summary:

generate plane boundary

Category:

Layout Editing

Licenses:

Apex

Parameters

- design – Design in which to create the generate plane.
- config – Configure file for generating. The format should be {{Layer Purpose} {Net Method LeftSizing RightSizing TopSizing BottomSizing}}Method: Border, Contour, Bone, All
- initConstraintFile – Create plane guides with net layer assignment automatically, and export the initial PG plane guides configuration file. Note that the net layer assignment preference rtPackagingPlaneBoundaryNetLayers should be set. (Default is: 'plane_constraint.csv')
- importConstraintFile – Create plane guides with the given plane constraint. (Default is: 'plane_constraint.csv')
- exportConstraintFile – Export plane constraint. (Default is: 'plane_constraint.csv')
- net – The net name or oaNet object to which generate plane should be assigned.
- oneIslandOnly – Only allow to create one island plane. (Default is: True)
- legalIslandOnly – Only allow to create legal island plane. (Default is: True)
- combineCluster – Combine Cluster to generate plane. (Default is: False)
- pushRoute – Post process push route. (Default is: False)
- combineRouter – Combine Router to generate plane. (Default is: False)
- excludeNets – Specify exclude nets (Default is: "")
- filterNets – Specify filter nets, don't create planes (Default is: "")
- boneWithContour – Using contour geometry to apply bone method
- primaryOnly – Only consider primary from router config

- coverDiffNet – Cover all different nets when generate plane boundary. (Default is: False)
- coverFOB – Cover FOB when generate plane boundary. (Default is: False)
- coverFOBSpacing – Cover FOB spacing when generate plane boundary. (Default is: 110)
- escapeSpacing – Escape spacing for different net. (Default is: “)
- d2dEscapeSpacing – D2D Escape spacing for different net. (Default is: “)
- d2dSizing – Sizing for same net in D2D region. (Default is: “)
- surroundingComponentSizing – Surrounding component sizing for same net. (Default is: “)
- surroundingDepth – Specify surrounding depth. (Default is: 0)
- customizedClusterSpacing – Spacing for cluster. (Default is: 0)
- maxBoxClusterSpacing – Spacing for maxBox cluster. (Default is: 0)
- maxClusterCount – Max count for cluster. (Default is: 0)
- maxBoxCount – Max count to create box. (Default is: 1)
- uTurn – Specify u-turn mode. (Default is: ‘Default’)
- rmstMode – Specify RMST method mode. (Default is: ‘Region’)
- escapeMode – Specify escape mode. (Default is: ‘Nearby’)
- escapeDirection – Specify escape direction. (Default is: “)
- escapeNeighborSide – Try escape to neighbor side. (Default is: True)
- escapeFourDirection – Try escape to four direction. (Default is: True)
- escapeAllDifferentNet – Try escape all different nets. (Default is: False)
- mergeSameNet – Merge Same net boundary. (Default is: True)
- tryNeighborSideCount – Count to try neighbor side. (Default is: 1)
- zigzagMaxLength – Specify zigzag max length. (Default is: 0)
- zigzagDepth – Specify zigzag depth. (Default is: 0)
- minWidth – Specify min width. (Default is: 0)
- minSpacing – Specify min spacing. (Default is: 0)
- mergeShieldingPaths – Merge shielding paths

- `mergeShapes` – Merge top level shapes
- `uniformGrow` – Uniform grow when sizing
- `autoOrder` – Automatic order. (Default is: False)
- `pointsCollection` – Return the points collection instead of generate plane boundary (Default is: False)
- `patchConcave` – Automatically patch concave corners. (Default is: True)

Returns

Collection of generate plane boundary.

generateRedundantVias

Syntax

```
generateRedundantVias(  
*,  
    box: box = None,  
    points: list[point] = None,  
    design: oaDesign,  
    net: oaNet | str,  
    spacingFactor: float = None)  
Returns: list[oaFig]
```

Summary:

Generate Redundant Vias

Category:

Layout Editing

Licenses:

Apex

Parameters

- `box` – Option to specify area for redundant via generation.
- `points` – Option to specify area for redundant via generation.
- `design` – Design in which to create the redundantVias.
- `net` – The net name or oaNet object to which redundant vias are generated on.
- `spacingFactor` – The spacing factor to generate redundant vias. (Default is: 1)

Returns

Collection of redundant vias.

generateSealRing

Syntax

```
generateSealRing(  
*,  
    design: oaDesign)  
Returns: list[oaShape]
```

Summary:

Generate seal rings.

Category:

Layout Editing

Licenses:

Apex

Parameters

design – Design in which to create the generate seal ring.

Returns

Collection of generated seal ring objects.

generateZigzag

Syntax

```
generateZigzag(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
    list[oaShape],  
/,  
*,  
    type: le.pkg.generateZigzag.Type = None,  
    targetEdge: int = None)  
Returns: list[oaFig]
```

Summary:

Generate Zigzag

Category:

Layout Editing

Licenses:

Apex

Parameters

- objects – List of objects to generate zigzag.
- type – Type of zigzag. ({ obtuseAngle | rightAngle } (Default is: 'rightAngle'))
- targetEdge – Target edge to generate zigzag. Any combination of center:0, top:1, bottom:2, left:4, right:8. (Default is: 15)

Returns

Collection of objects.

getPlaneBoundaries

Syntax

```
getPlaneBoundaries(  
*,  
    design: oaDesign)  
Returns: list[oaFig]
```

Summary:

get plane boundaries

Category:

Layout Editing

Licenses:

Apex

Parameters

design – Design in which to create the plane.

Returns

Collection of plane boundary.

snps.custom.cmd.ile

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform layout editing. Interactive commands are listed in this chapter.

For details, see:

- [addDummy](#)
- [addToFigGroup](#)
- [adjustCursor](#)
- [align](#)
- [alignAngle](#)
- [applyColor](#)
- [attachDetach](#)
- [autoChopGuardRing](#)
- [autoFix](#)
- [autoFlipHorizontal](#)
- [autoFlipVertical](#)
- [autoRotate90](#)
- [boolean](#)
- [bridgeInterconnect](#)
- [changePinRelationship](#)
- [checkCongestion](#)
- [checkDensity](#)
- [checkVias](#)
- [checkVoltageAnnotations](#)
- [chop](#)
- [clearDensityDisplay](#)

- [clearVoltagePhysicalAnnotations](#)
- [clearVoltages](#)
- [cloneMaster](#)
- [collapseDensityTree](#)
- [color](#)
- [compactBus](#)
- [continueBus](#)
- [convertToArray](#)
- [convertToPath](#)
- [convertToPolygon](#)
- [copy](#)
- [copyLook](#)
- [copyToClipboard](#)
- [correctOffGrid](#)
- [createAttributeLabel](#)
- [createBlockage](#)
- [createBoundary](#)
- [createBus](#)
- [createCellViewFromSelection](#)
- [createCoordinateMark](#)
- [createEllipse](#)
- [createFigGroup](#)
- [createGlobalTrackRegion](#)
- [createGuardRing](#)
- [createInst](#)
- [createInterconnect](#)
- [createLabel](#)

- [createMPP](#)
- [createPin](#)
- [createPinsFromSelection](#)
- [createPolygon](#)
- [createRectangle](#)
- [createRouteCorridor](#)
- [createRow](#)
- [createRuler](#)
- [createTopology](#)
- [createTrackRegion](#)
- [createTrackRegionPCell](#)
- [createTransmissionLine](#)
- [createVia](#)
- [createVoltageConfig](#)
- [createVoltagePhysicalAnnotations](#)
- [cut](#)
- [cutCorner](#)
- [delete](#)
- [deleteCutMetal](#)
- [deleteFigGroup](#)
- [distributeAlign](#)
- [eraseLayer](#)
- [expandDensityTree](#)
- [fill](#)
- [flatten](#)
- [generateCutMetal](#)
- [generateGuardRing](#)

- [generateLabel](#)
- [generateLayoutPins](#)
- [generateMPP](#)
- [generatePins](#)
- [generatePlacementTrackRegion](#)
- [generateRoutingTrackRegion](#)
- [generateShapes](#)
- [heal](#)
- [hierarchicalCopy](#)
- [highlightConnected](#)
- [highlightPath](#)
- [importVoltages](#)
- [inputCoord](#)
- [lineSelect](#)
- [measureAngle](#)
- [measureDistance](#)
- [measurePath](#)
- [merge](#)
- [move](#)
- [padLabel](#)
- [partialToFull](#)
- [paste](#)
- [pasteLook](#)
- [reassignNet](#)
- [removeFromFigGroup](#)
- [removeOverlappingCuts](#)
- [replaceVia](#)

- [reportCapacitance](#)
- [reportEM](#)
- [reportParasitic](#)
- [reportResistance](#)
- [reshape](#)
- [reshapeInterconnect](#)
- [resize](#)
- [rotate](#)
- [scalarizeArrayInst](#)
- [selectByLPP](#)
- [selectObjects](#)
- [selectRouteObjects](#)
- [showDensityDisplaySetting](#)
- [showEMCalculator](#)
- [split](#)
- [splitArray](#)
- [stackInterconnect](#)
- [stretch](#)
- [stretchArray](#)
- [stretchWidth](#)
- [stripeWire](#)
- [tap](#)
- [trimWire](#)
- [validateColor](#)
- [yank](#)
- [snps.custom.cmd.ile.pkg](#)

addDummy

Syntax

```
addDummy(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive add_dummy command

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

addToFigGroup

Syntax

```
addToFigGroup(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command to add objects to an existing OA figure group.

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

adjustCursor

Syntax

```
adjustCursor(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive adjust cursor command

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

align

Syntax

```
align(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive align command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

alignAngle

Syntax

```
alignAngle(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive align angle command

Category:

Layout Editing

Licenses:

Apex

Parameters

context – Context of activation.

Returns

None

applyColor

Syntax

```
applyColor(  
context: deContext | int = None,  
/,  
*,  
action: ile.applyColor.Action)  
Returns: None
```

Summary:

Interactive apply_color command

Category:

Layout Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- action – apply color action ({ color1 | color2 | color3 | color4 | colorNone | colorSame | lock | shift | unlock })

Returns

None

attachDetach

Syntax

```
attachDetach(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive attach/detach command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

autoChopGuardRing

Syntax

```
autoChopGuardRing(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive Auto Chop Guard Ring Command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

autoFix

Syntax

```
autoFix(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive autoFix command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

autoFlipHorizontal

Syntax

```
autoFlipHorizontal(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive auto flip horizontal command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

autoFlipVertical

Syntax

```
autoFlipVertical(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive auto flip vertical command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

autoRotate90

Syntax

```
autoRotate90(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive auto rotate 90 command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

boolean

Syntax

```
boolean(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive boolean command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

bridgeInterconnect

Syntax

```
bridgeInterconnect(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Brings up Bridge Interconnect toolbar and starts the operation of bridging interconnect objects.

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

changePinRelationship

Syntax

```
changePinRelationship(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive change pin relationship command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

checkCongestion

Syntax

```
checkCongestion(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive check congestion command

Category:

Layout Editing

Licenses:

Router

Parameters

context – Context of activation.

Returns

None

checkDensity

Syntax

```
checkDensity(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive check density command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

checkVias

Syntax

```
checkVias(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive check vias command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

checkVoltageAnnotations

Syntax

```
checkVoltageAnnotations(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive check voltage annotation command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

chop

Syntax

```
chop(  
context: deContext | int = None,  
/,  
*,  
log: ile.chop.Log = None)  
Returns: None
```

Summary:

Interactive le chop command.

Category:

Layout Editing

Licenses:

Basic

Parameters

- context – Context of activation.
- log – Logging mode ({ auto | explicit } (Default is: 'auto'))

Returns

None

clearDensityDisplay

Syntax

```
clearDensityDisplay(  
context: deContext | int = None)  
Returns: None
```

Summary:

Clears the density display.

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

clearVoltagePhysicalAnnotations

Syntax

```
clearVoltagePhysicalAnnotations(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive clear physical annotation command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

clearVoltages

Syntax

```
clearVoltages(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive clear annotation command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

cloneMaster

Syntax

```
cloneMaster(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive clone master command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

collapseDensityTree

Syntax

```
collapseDensityTree(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Collapse the density tree.

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

color

Syntax

```
color(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive color command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

compactBus

Syntax

```
compactBus(  
context: deContext | int = None)  
Returns: None
```

Summary:

Brings up Compress and Expand Bus toolbar and starts the operation to bus bits.

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

continueBus

Syntax

```
continueBus(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive continue bus command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

convertToArray

Syntax

```
convertToArray(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive convert to array command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

convertToPath

Syntax

```
convertToPath(  
    context: deContext | int = None  
Returns: None
```

Summary:

Interactive convert polygon to path/pathSeg command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

convertToPolygon

Syntax

```
convertToPolygon(  
    context: deContext | int = None  
Returns: None
```

Summary:

Interactive convert to polygon command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

copy

Syntax

```
copy(  
context: deContext | int = None,  
/,  
*,  
object: deFigure | oaFig = None,  
point: point = None)  
Returns: None
```

Summary:

Interactive copy command

Category:

Layout Editing

Licenses:

Basic

Parameters

- context – Context of activation.
- object – Object for exclusive copy
- point – Specifies the drag start point and the COT is not displayed. Should be specified at the same time with -object.

Returns

None

copyLook

Syntax

```
copyLook(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive copy look command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

copyToClipboard

Syntax

```
copyToClipboard(  
context: deContext | int = None,  
/,  
*,  
log: ile.copyToClipboard.Log = None)  
Returns: None
```

Summary:

Interactive layout copy selected objects to clipboard.

Category:

Layout Editing

Licenses:

None

Parameters

- context – Context of activation.
- log – Logging mode ({ auto | explicit } (Default is: 'auto'))

Returns

None

correctOffGrid

Syntax

```
correctOffGrid(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive Correct Off-grid command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createAttributeLabel

Syntax

```
createAttributeLabel(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create attribute label command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createBlockage

Syntax

```
createBlockage(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create blockage command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createBoundary

Syntax

```
createBoundary(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create boundary command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createBus

Syntax

```
createBus(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create bus command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createCellViewFromSelection

Syntax

```
createCellViewFromSelection(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create Cell View from Selection command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createCoordinateMark

Syntax

```
createCoordinateMark(  
context: deContext | int = None)  
Returns: None
```

Summary:

Starts the interactive Create Coordinate Mark command.

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

createEllipse

Syntax

```
createEllipse(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create ellipse command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createFigGroup

Syntax

```
createFigGroup(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive figure group creation command with selection and origin specification support

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createGlobalTrackRegion

Syntax

```
createGlobalTrackRegion(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create global track region command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

createGuardRing

Syntax

```
createGuardRing(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create guard ring command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createInst

Syntax

```
createInst(  
context: deContext | int = None,  
/  
*,  
device: deDevice = None,  
hidden: bool = None,  
**kwargs)  
Returns: None
```

Summary:

Interactive create inst command

Category:

Layout Editing

Licenses:

Basic

Parameters

- context – Context of activation.
- device – If specified, populates the Create Instance dialog with the master and parameters from the specified favorite device object.
- hidden – If true, starts the command in a hidden state without affecting the COT or showing the COD. (Default is: False)

Returns

None

createInterconnect

Syntax

```
createInterconnect(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create interconnect command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createLabel

Syntax

```
createLabel(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create label command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createMPP

Syntax

```
createMPP(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create MPP command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createPin

Syntax

```
createPin(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create pin command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createPinsFromSelection

Syntax

```
createPinsFromSelection(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create pins from selection command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createPolygon

Syntax

```
createPolygon(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create polygon command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createRectangle

Syntax

```
createRectangle(  
context: deContext | int = None,  
/,  
*,  
log: ile.createRectangle.Log = None)  
Returns: None
```

Summary:

Interactive create rectangle command

Category:

Layout Editing

Licenses:

Basic

Parameters

- context – Context of activation.
- log – Logging mode ({ auto | explicit } (Default is: 'auto'))

Returns

None

createRouteCorridor

Syntax

```
createRouteCorridor(  
context: deContext | int = None)  
Returns: None
```


Summary:

Interactive create routing corridor command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createRow

Syntax

```
createRow(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create row command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createRuler

Syntax

```
createRuler(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Starts the interactive Create Ruler command in the specified layout window.

Category:

Design Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

createTopology

Syntax

```
createTopology(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create topology command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

createTrackRegion

Syntax

```
createTrackRegion(  
    context: deContext | int = None  
Returns: None
```

Summary:

Interactive create track region command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

createTrackRegionPCell

Syntax

```
createTrackRegionPCell(  
    context: deContext | int = None,  
    /,  
    *,  
    log: ile.createTrackRegionPCell.Log = None)  
Returns: None
```

Summary:

Interactive create track region pcell command

Category:

Layout Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- log – Logging mode ({ auto | explicit } (Default is: 'auto'))

Returns

None

createTransmissionLine

Syntax

```
createTransmissionLine(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create transmission line command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createVia

Syntax

```
createVia(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create via command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

createVoltageConfig**Syntax**

```
createVoltageConfig(  
    context: deContext | int = None,  
    /,  
    **kwargs)  
Returns: None
```

Summary:

Interactive create new config view command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

createVoltagePhysicalAnnotations

Syntax

```
createVoltagePhysicalAnnotations(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive physical annotate voltages command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

cut

Syntax

```
cut(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive layout cut selected objects to clipboard.

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

cutCorner

Syntax

```
cutCorner(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive cut_corner shape object command

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

delete

Syntax

```
delete(  
context: deContext | int = None,  
/,  
*,  
log: ile.delete.Log = None)  
Returns: None
```

Summary:

Interactive delete command

Category:

Layout Editing

Licenses:

Basic

Parameters

- context – Context of activation.
- log – Logging mode ({ auto | explicit } (Default is: 'auto'))

Returns

None

deleteCutMetal

Syntax

```
deleteCutMetal(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive delete cut metal command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

deleteFigGroup

Syntax

```
deleteFigGroup(  
context: deContext | int = None)  
Returns: None
```


Summary:

Interactive delete/ungroup fig group command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

distributeAlign**Syntax**

```
distributeAlign(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive distribute align command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

eraseLayer

Syntax

```
eraseLayer(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive erase layer command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

expandDensityTree

Syntax

```
expandDensityTree(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Expand the density tree.

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

fill

Syntax

```
fill(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive fill instances command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

flatten

Syntax

```
flatten(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive le flatten command.

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

generateCutMetal

Syntax

```
generateCutMetal(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive generate cut metal command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

generateGuardRing

Syntax

```
generateGuardRing(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive auto guard ring creation command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

generateLabel

Syntax

```
generateLabel(  
    context: deContext | int = None  
Returns: None
```

Summary:

Interactive generate label command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

generateLayoutPins

Syntax

```
generateLayoutPins(  
    *,  
    parent: giWindow | int = None  
Returns: giDialog
```

Summary:

Generate Layout Pins

Category:

Layout Editing

Licenses:

None

Parameters

parent – parent widget

Returns

The dialog

generateMPP

Syntax

```
generateMPP(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive semi-automatic MPP generation command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

generatePins

Syntax

```
generatePins(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive generate pins command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

generatePlacementTrackRegion

Syntax

```
generatePlacementTrackRegion(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive generate placement track region

Category:

Layout Editing

Licenses:

Track Region Generation

Parameters

context – Context of activation.

Returns

None

generateRoutingTrackRegion

Syntax

```
generateRoutingTrackRegion(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive generate routing track region

Category:

Layout Editing

Licenses:

Track Region Generation

Parameters

context – Context of activation.

Returns

None

generateShapes

Syntax

```
generateShapes(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive generate shapes command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

heal

Syntax

```
heal(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive Heal Command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

hierarchicalCopy

Syntax

```
hierarchicalCopy(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive hierarchical copy command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

highlightConnected

Syntax

```
highlightConnected(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive highlight connected command.

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

highlightPath

Syntax

```
highlightPath(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive shortest path command.

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

importVoltages

Syntax

```
importVoltages(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive annotate voltages command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

inputCoord

Syntax

```
inputCoord(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive coordinate input command

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

lineSelect

Syntax

```
lineSelect(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive in-command line selection

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

measureAngle

Syntax

```
measureAngle(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactively measure angle

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

measureDistance

Syntax

```
measureDistance(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive measure distance command

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

measurePath

Syntax

```
measurePath(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactively measure path

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

merge

Syntax

```
merge(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive merge command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

move

Syntax

```
move (  
  context: deContext | int = None,  
  /,  
  *,  
  object: deFigure | oaFig = None,  
  point: point = None)  
Returns: None
```

Summary:

Interactive move command

Category:

Layout Editing

Licenses:

Basic

Parameters

- context – Context of activation.
- object – Object for exclusive move
- point – Specifies the drag start point and the COT is not displayed. Should be specified at the same time with -object.

Returns

None

padLabel

Syntax

```
padLabel (  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive pad label command

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

partialToFull

Syntax

```
partialToFull(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive partial to full command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

paste

Syntax

```
paste(  
  context: deContext | int = None,  
  /,  
  *,  
  clip: deClipboard = None)  
Returns: None
```

Summary:

Interactive paste command

Category:

Layout Editing

Licenses:

Basic

Parameters

- context – Context of activation.
- clip – Clipboard to paste

Returns

None

pasteLook

Syntax

```
pasteLook(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive copy look command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

reassignNet

Syntax

```
reassignNet(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive reassign net command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

removeFromFigGroup

Syntax

```
removeFromFigGroup(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command to remove objects from an existing OA figure group.

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

removeOverlappingCuts

Syntax

```
removeOverlappingCuts(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive Remove Overlapping Cuts Command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

replaceVia

Syntax

```
replaceVia(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive replace via command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

reportCapacitance

Syntax

```
reportCapacitance(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive parasitic reporter command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

reportEM

Syntax

```
reportEM(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive EM reporter command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

reportParasitic

Syntax

```
reportParasitic(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive parasitic reporter command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

reportResistance

Syntax

```
reportResistance(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive parasitic reporter command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

reshape

Syntax

```
reshape(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive reshape command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

reshapeInterconnect

Syntax

```
reshapeInterconnect(  
    context: deContext | int = None)  
Returns: None
```

Summary:

The command brings up Reshape Interconnect toolbar and starts interconnect reshaping.

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

resize

Syntax

```
resize(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive resize command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

rotate

Syntax

```
rotate(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive rotate command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

scalarizeArrayInst

Syntax

```
scalarizeArrayInst(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive le scalarize command.

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

selectByLPP

Syntax

```
selectByLPP(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive select objects by lpp command.

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

selectObjects

Syntax

```
selectObjects(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive select_objects shape object command

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

selectRouteObjects

Syntax

```
selectRouteObjects(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive select route objects.

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

showDensityDisplaySetting

Syntax

```
showDensityDisplaySetting(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive show density display setting command.

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

showEMCalculator

Syntax

```
showEMCalculator(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive EM calculator command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

split

Syntax

```
split(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive split command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

splitArray

Syntax

```
splitArray(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive split array command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

stackInterconnect

Syntax

```
stackInterconnect(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive stack interconnect command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

stretch

Syntax

```
stretch(  
    context: deContext | int = None,  
    /,  
    *,  
    object: deFigure | oaFig = None,  
    point: point = None)  
Returns: None
```

Summary:

The command brings up Stretch toolbar and starts interactive stretching.

Category:

Layout Editing

Licenses:

Basic

Parameters

- context – Context of activation.
- object – Object for exclusive stretch
- point – Specifies the drag start point and the COT is not displayed. Should be specified at the same time with -object.

Returns

None

stretchArray

Syntax

```
stretchArray(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive stretch array instance command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

stretchWidth

Syntax

```
stretchWidth(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command to Stretch wire width

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

stripeWire

Syntax

```
stripeWire(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive stripe wire command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

tap

Syntax

```
tap(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive tap shape object command

Category:

Layout Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

trimWire

Syntax

```
trimWire(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive trim wire command

Category:

Layout Editing

Licenses:

Basic

Parameters

context – Context of activation.

Returns

None

validateColor

Syntax

```
validateColor(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive color validate command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

yank

Syntax

```
yank(  
context: deContext | int = None,  
/,  
*,  
log: ile.yank.Log = None)  
Returns: None
```

Summary:

Interactive le yank command.

Category:

Layout Editing

Licenses:

None

Parameters

- context – Context of activation.
- log – Logging mode ({ auto | explicit } (Default is: 'auto'))

Returns

None

snps.custom.cmd.ile.pkg

This chapter lists the commands in the `pd` module for layout. Interactive commands are listed in this chapter.

This section covers the following topics:

- [createDegassingHole](#)
- [createPlaneBoundary](#)
- [definePackagingLibrary](#)
- [generateRedundantVias](#)
- [generateZigzag](#)
- [realizePlane](#)
- [replacePlaneBoundary](#)

createDegassingHole

Syntax

```
createDegassingHole(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create degassing hole command.

Category:

Layout Editing

Licenses:

Apex

Parameters

context – Context of activation.

Returns

None

createPlaneBoundary

Syntax

```
createPlaneBoundary(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create plane boundary command.

Category:

Layout Editing

Licenses:

Apex

Parameters

context – Context of activation.

Returns

None

definePackagingLibrary

Syntax

```
definePackagingLibrary(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive define packaging library command

Category:

Layout Editing

Licenses:

Apex

Parameters

context – Context of activation.

Returns

None

generateRedundantVias

Syntax

```
generateRedundantVias(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive generate redundant vias command

Category:

Layout Editing

Licenses:

Apex

Parameters

context – Context of activation.

Returns

None

generateZigzag

Syntax

```
generateZigzag(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive generate zigzag command.

Category:

Layout Editing

Licenses:

Apex

Parameters

context – Context of activation.

Returns

None

realizePlane

Syntax

```
realizePlane(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive realize plane command.

Category:

Layout Editing

Licenses:

Apex

Parameters

context – Context of activation.

Returns

None

replacePlaneBoundary

Syntax

```
replacePlaneBoundary(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive replace plane boundary command.

Category:

Layout Editing

Licenses:

Apex

Parameters

context – Context of activation.

Returns

None

snps.custom.cmd.lp

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform placement operations.

For details, see:

- [align](#)
- [applyTemplates](#)
- [autoPinPlace](#)
- [autoPlace](#)
- [checkPlacementRule](#)
- [cngenlib](#)
- [copyResult](#)

- [createGraph](#)
- [doDeviceGrouping](#)
- [doStdCellGrouping](#)
- [exportPinPlacement](#)
- [extractRelativeString](#)
- [extractTopology](#)
- [fixPlacementRule](#)
- [fixRelativeConstraint](#)
- [generateConstraint](#)
- [group](#)
- [initCellInfo](#)
- [insertBoundary](#)
- [insertColumn](#)
- [insertDecapFiller](#)
- [insertDiode](#)
- [insertTap](#)
- [optimizePins](#)
- [placePins](#)
- [showOptions](#)

align

Syntax

```
align(  
*,  
    design: oaDesign,  
    sources: list[str],  
    target: str,  
    sourceEdge: str,  
    targetEdge: str,  
    direction: str = None,  
    moveInHierarchy: bool = None,  
    considerMustJoinPins: bool = None)  
Returns: None
```

Summary:

align instances, figGroups and terminals

Category:

Layout Placer

Licenses:

Elite

Parameters

- design – Active design
- sources – List of instance names, figGroup names or terminals to align to the target
- target – Target instance, figGroup or terminal (reference)
- sourceEdge – Source edge (right | left | center | origin | top | bottom)
- targetEdge – Target edge (right | left | center | origin | top | bottom)
- direction – Align direction (vertical | horizontal)
- moveInHierarchy – Specify to move instances/figGroups inside the hierarchy or only on top-level
- considerMustJoinPins – Consider must join pins when terminal is specified (Default is: False)

Returns

None

applyTemplates

Syntax

```
applyTemplates(  
cell: deContext | dmCellView | oaDesign | str,  
/,  
*,  
templateLibrary: oaLib | str,  
onlyIncludeRealizedInstances: bool = None)  
Returns: dbCollection
```

Summary:

Applies the templates found in the specified template library to the schematic, creating constraints in the layout.

Category:

Layout Placer

Licenses:

Backend

Parameters

- cell – Layout cell to be constrained.
- templateLibrary – Template library from which templates will be applied.
- onlyIncludeRealizedInstances – To only include realized instances

Returns

Collection of cmConstraint objects

autoPinPlace

Syntax

```
autoPinPlace(  
*,  
mode: lp.autoPinPlace.Mode,  
file: str = None,  
context: deContext | oaDesign = None,  
window: str = None,  
distribute: bool = None,  
**kwargs)  
Returns: None
```

Summary:

auto pin placer

Category:

Layout Placer

Licenses:

Backend

Parameters

- mode – Mode of pin placer ({ constraintFile | optionsDialog | top | twoLevels })
- file – Input constraint file for pin placer
- context – Design context

- window – Window id of top level design
- distribute – Distribute pins on edge

Returns

None

autoPlace

Syntax

```
autoPlace(  
    cell: deContext | dmCellView | oaDesign | str,  
    /,  
    *,  
    placementLibrary: oaLib | str,  
    **kwargs)  
Returns: None
```

Summary:

Runs automatic placement.

Category:

Layout Placer

Licenses:

Backend

Parameters

- cell – Cell to place.
- placementLibrary – Library to hold placement results.

Returns

None

checkPlacementRule

Syntax

```
checkPlacementRule(  
    insts: str = None,  
    /,  
    *,
```

```
design: oaDesign)
Returns: list[oaMarker]
```

Summary:

Check placement rule violations among all instances

Category:

Layout Placer

Licenses:

Elite

Parameters

- insts – instance names to check placement rule violation
- design – Target design

Returns

Placement rule violation points

cngenlib

Syntax

```
cngenlib(  
*,  
techFilePath: str,  
techLib: str,  
libName: str,  
libPath: str)  
Returns: None
```

Summary:

Create New Placement Library.

Category:

Layout Placer

Licenses:

None

Parameters

- techFilePath – Santana tech file path which is used to generate library
- techLib – Reference Tech Library
- libName – Library to be generated using Santana Tech
- libPath – Path where library will be generated

Returns

None

copyResult

Syntax

```
copyResult(  
    cell: deContext | dmCellView | oaDesign | str,  
    /,  
    *,  
    design: deContext | dmCellView | oaDesign)  
Returns: None
```

Summary:

Copies placement results back to layout.

Category:

Layout Placer

Licenses:

Backend

Parameters

- cell – Result to be copied.
- design – Design into which result will be copied.

Returns

None

createGraph

Syntax

```
createGraph(  
_none_: str = None,  
/,  
*,  
parent: giAssistant | giContainer | giDialog | giWindow = None,  
enabled: bool = None,  
shown: bool = None,  
gcpResultFile: str,  
minHeight: float = None,  
width: float = None,  
height: float = None,  
maxHeight: float = None,  
minWidth: float = None,  
maxWidth: float = None,  
selectedResults: str = None,  
deleteResults: str = None,  
label: str = None,  
selectionChangeProc: list[variant] = None,  
selectionDeleteProc: list[variant] = None,  
valueChangeProc: list[variant] = None,  
toolTip: str = None,  
contextMenuProc: list[variant] = None,  
dragProc: list[variant] = None)  
Returns: giGraph
```

Summary:

Creates a new giGraph widget and adds it to the specified parent

Category:

Layout Placer

Licenses:

None

Parameters

- `_none_` – Widget name. (Default is: “)
- `parent` – Dialog, group or other container into which this widget should be placed.
- `enabled` – Initial enabled state. (Default is: True)
- `shown` – Initial shown state. (Default is: True)
- `gcpResultFile` – File containing the result data for the GCP

- `minHeight` – min height, in integerminimum Height for the gcp generated results to display in pareto graph (Default is: 0)
- `width` – width, in double width of the only gcp result to be shown in pareto graph (Default is: 0)
- `height` – height, in double height of the only gcp result to be shown in pareto graph (Default is: 0)
- `maxHeight` – max height, in integermaximum Height for the gcp generated results to display in pareto graph (Default is: 0)
- `minWidth` – min width, in integerminimum width for the gcp generated results to display in pareto graph (Default is: 0)
- `maxWidth` – max width, in integermaximum width for the gcp generated results to display in pareto graph (Default is: 0)
- `selectedResults` – The selected gcp generated results (Default is: “)
- `deleteResults` – Gcp generated results to be deleted (Default is: “)
- `label` – Widget label, shown in the left column. If a name is not specified, the widget is “unlabeled” and extends into the label column (Default is: “)
- `selectionChangeProc` – Procedure to execute when the selection is changed. Called with the table object. Retrieve selection from attribute
- `selectionDeleteProc` – Procedure to execute when the selection is deleted. Called with the table object. Retrieve selection from attribute
- `valueChangeProc` – Procedure to execute when the value is changed. Called with the cell which was modified
- `toolTip` – Optional widget toolTip. (Default is: “)
- `contextMenuProc` – Procedure to execute when right-clicking on this widget to request a context menu
- `dragProc` – Procedure to invoke when a drag starts. Called with the widget object.

Returns

Newly created graph.

doDeviceGrouping

Syntax

```
doDeviceGrouping(  
context: deContext,
```

```
//  
*  
onlyIncludeRealizedInstances: bool = None)  
Returns: dbCollection
```

Summary:

do devices grouping and generating constraints to for placer.

Category:

Layout Placer

Licenses:

Backend

Parameters

- context – context pointer
- onlyIncludeRealizedInstances – To only include realized instances

Returns

Collection of cmConstraint objects

doStdCellGrouping

Syntax

```
doStdCellGrouping(  
context: deContext)  
Returns: dbCollection
```

Summary:

grouping standard cells in given standrad library for generating standard cell constraints.

Category:

Layout Placer

Licenses:

Backend

Parameters

context – context pointer

Returns

Collection of cmConstraint objects

exportPinPlacement

Syntax

```
exportPinPlacement(  
*,  
    level: str,  
    file: str,  
    window: str | None = None,  
    offset: bool | None = None,  
    coord: bool | None = None,  
    template: bool | None = None)  
Returns: None
```

Summary:

export pin placement

Category:

Layout Placer

Licenses:

Backend

Parameters

- level – Level of exported pin placement
- file – Target file path
- window – Window id of top level design
- offset – Whether to export pin offset
- coord – Whether to export pin coordinates
- template – Generate a template file in constraint file format

Returns

None

extractRelativeString

Syntax

```
extractRelativeString(  
    schDesign: oaDesign,  
    /,  
    *,  
    skipCell: list[str] = None)  
Returns: str
```

Summary:

relative string extraction

Category:

Layout Placer

Licenses:

Backend

Parameters

- schDesign – schematic design
- skipCell – Skip cells to extract (lib/cell)

Returns

Relative placement string.

extractTopology

Syntax

```
extractTopology(  
    cell: deContext | dmCellView | oaDesign,  
    /,  
    *,  
    xFactor: float = None,  
    yFactor: float = None,  
    mode: str = None,  
    halo: bool = None,  
    insts: list[str] = None)  
Returns: dbCollection
```

Summary:

Extract topology constraints from a pre-existing layout.

Category:

Layout Placer

Licenses:

Backend

Parameters

- cell – Cell from which layout topology will be extracted.
- xFactor – Horizontal scaling factor for spacing. (Default is: 1)
- yFactor – Vertical scaling factor for spacing. (Default is: 1)
- mode – Extract topology modes: relatives-only, relatives, clusters , slicing, instMatching, cmos, or resistorBank (Default is: 'relatives-only')
- halo – Whether to use halo to try to preserve alignment and spacing: true(1) or false(0) (Default is: True)
- insts – Instances to extract topology

Returns

Collection of cmConstraint objects

fixPlacementRule

Syntax

```
fixPlacementRule(  
    insts: str = None,  
    /,  
    *,  
    design: oaDesign)  
Returns: list[oaMarker]
```

Summary:

Fix placement rule violations among all instances

Category:

Layout Placer

Licenses:

Elite

Parameters

- insts – instance names to fix placement rule violation
- design – Target design

Returns

Non-fixable placement rule violation points

fixRelativeConstraint

Syntax

```
fixRelativeConstraint(  
    cell: oaDesign)  
Returns: None
```

Summary:

Fix relative constrains

Category:

Layout Placer

Licenses:

Backend

Parameters

cell – Cell from which layout topology will be extracted.

Returns

None

generateConstraint

Syntax

```
generateConstraint(  
    context: deContext,  
    /,  
    *,  
    applyTemplates: bool = None,  
    templateLibrary: str = None,  
    groupStdCells: bool = None,  
    stdCellLibraries: list[str] = None,  
    groupDevices: bool = None,  
    onlyIncludeRealizedInstances: bool = None,
```

```
ignoreDummy: bool = None,  
**kwargs)  
Returns: dbCollection
```

Summary:

Generating constraints.

Category:

Layout Placer

Licenses:

Backend

Parameters

- context – Context of activation.
- applyTemplates – Apply constraints in template library.
- templateLibrary – Template library from which templates will be applied.
- groupStdCells – Group by standard cell libraries.
- stdCellLibraries – Standard cell libraries.
- groupDevices – Group devices.
- onlyIncludeRealizedInstances – To only include realized instances
- ignoreDummy – ignore dummy instances

Returns

Collection of cmConstraint objects

group

Syntax

```
group(  
  groupName: str,  
  /,  
  *,  
  mode: str,  
  groupList: str | None = None,  
  spaceList: str | None = None)  
Returns: str
```

Summary:

group instances

Category:

Layout Placer

Licenses:

Backend

Parameters

- groupName – Group name
- mode – assign | clear | get | getSpace | setSpace
- groupList – Instance names
- spaceList – Space list (left, bottom, right, upper)

Returns

Instance names in group

initCellInfo

Syntax

```
initCellInfo(  
*,  
    file: str)  
Returns: None
```

Summary:

Initialize cell info

Category:

Layout Placer

Licenses:

Elite

Parameters

file – Config file for layout only cells

Returns

None

insertBoundary

Syntax

```
insertBoundary(  
    design: oaDesign,  
    /,  
    *,  
    mode: str,  
    prefix: str = None,  
    setFixed: bool = None,  
    rowPattern: str = None,  
    honorPRBoundary: bool = None,  
    **kwargs)  
Returns: list[oaInst]
```

Summary:

Insert boundary cells

Category:

Layout Placer

Licenses:

Elite

Parameters

- design – Design from which boundary cell will be inserted.
- mode – Place boundary cells to outside/inside of boundary (negative / positive) (default: positive)
- prefix – Prefix for names of inserted boundary cells (default: lp_boundary)
- setFixed – Set inserted boundary cells to be fixed (default: false)
- rowPattern – PN type of inserted horizontal boundary cells (starting from the bottom row)
- honorPRBoundary – Honor PR Boundary if its width / height is not multiple of 'site width' / 'row height'. (Default is: False)

Returns

Collection of created instances

insertColumn

Syntax

```
insertColumn(  
    design: oaDesign,  
    /,  
    *,  
    prefix: str = None,  
    setFixed: bool = None,  
    minDistance: float = None,  
    maxDistance: float,  
    pitch: float = None,  
    boundaryDistance: float = None,  
    xRange: list[float] = None,  
    minGap: float = None,  
    rowPattern: str = None,  
    **kwargs)  
Returns: list[oaInst]
```

Summary:

Insert column cells

Category:

Layout Placer

Licenses:

Elite

Parameters

- design – Design from which column cells will be inserted.
- prefix – Prefix for name of inserted column cells (default: lp_column)
- setFixed – Set inserted column cells to be fixed (default: false)
- minDistance – Minimum distance between two adjacent columns (default: 0.5 * maxDistance)
- maxDistance – Maximum distance between two adjacent columns
- pitch – Default center-to-center distance between two adjacent columns (default: 0.5 * (minDistance + maxDistance))
- boundaryDistance – Distance between boundary and left-most columns (default: 0.5 * maxDistance)
- xRange – Range of x coordination to insert column cells to.

- minGap – Minimum gap between each inserted column cell and its nearby instances (default: 0)
- rowPattern – PN type of inserted boundary column cells (starting from the bottom row).

Returns

Collection of created instances

insertDecapFiller

Syntax

```
insertDecapFiller(  
design: oaDesign,  
/  
*,  
decapPrefix: str = None,  
fillerPrefix: str = None,  
setFixed: bool = None,  
boundaryLpp: lpp = None,  
insertDecap: bool = None,  
insertFiller: bool = None,  
depth: int = None,  
**kwargs)  
Returns: list[oaInst]
```

Summary:

Insert decap and filler cells

Category:

Layout Placer

Licenses:

Elite

Parameters

- design – Design from which decap and filler will be inserted.
- decapPrefix – Prefix for names of inserted decap cells (default: lp_decap)
- fillerPrefix – Prefix for names of inserted filler cells (default: lp_filler)
- setFixed – Set inserted decap/filler cells to be fixed (default: false)
- boundaryLpp – LPP of boundary layer
- insertDecap – To insert decap cell (default is true)

- insertFiller – To insert filler cell (default is true)
- depth – The depth to traverse instances hierarchically (Default is: 0)

Returns

Collection of created instances

insertDiode

Syntax

```
insertDiode(  
*,  
  nets: list[list[oaNet]],  
  prefix: str = None,  
  setFixed: bool = None,  
  maxDistance: float,  
  checkExistence: bool = None,  
  minGap: float = None,  
  **kwargs)  
Returns: list[oaInst]
```

Summary:

Insert diode cells

Category:

Layout Placer

Licenses:

Elite

Parameters

- nets – List of nets on which to insert diode cells
- prefix – Prefix for names of inserted diode cells (default: lp_diode)
- setFixed – Set inserted diode cells to be fixed (default: false)
- maxDistance – Maximum distance between inserted diode cell and the primary input pin
- checkExistence – Check if there is any pre-placed diode cell on target nets (default: true)
- minGap – Minimum gap between each inserted diode cell and its nearby instances (default: 0)

Returns

Collection of created instances

insertTap

Syntax

```
insertTap(  
    design: oaDesign,  
    /,  
    *,  
    prefix: str = None,  
    setFixed: bool = None,  
    initialPlaceOnly: bool = None,  
    minDistance: float = None,  
    maxDistance: float,  
    pitch: float = None,  
    boundaryDistance: float = None,  
    skipAlternateRowCount: int = None,  
    skipBottomRowCount: int = None,  
    mode: str,  
    checkExistence: bool = None,  
    minGap: float = None,  
    type: str = None,  
    leftBoundary: bool = None,  
    rightBoundary: bool = None,  
    **kwargs)  
Returns: list[oaInst]
```

Summary:

Insert tap cells

Category:

Layout Placer

Licenses:

Elite

Parameters

- design – Design from which tap filler will be inserted.
- prefix – Prefix for names of inserted tap cells (default: lp_tap)
- setFixed – Set inserted tap cells to be fixed (default: false)
- initialPlaceOnly – Insert the tap cells with input parameters and ignore cell overlaps (default: false)

- `minDistance` – Minimum distance between two adjacent tap cells (default: $0.5 * \text{maxDistance}$)
- `maxDistance` – Maximum distance between two adjacent tap cells
- `pitch` – Default center-to-center distance between two adjacent tap cells (default: $0.5 * (\text{minDistance} + \text{maxDistance})$)
- `boundaryDistance` – Distance between boundary and lower left tap cell (default: $0.5 * \text{maxDistance}$)
- `skipAlternateRowCount` – Skip every alternate N rows after inserting a rows of tap cells (default: 1)
- `skipBottomRowCount` – Skip first N rows before inserting tap cells (default: 0)
- `mode` – Tap insertion modes (`columnAligned` / `checkerboard`)
- `checkExistence` – Check if there is any pre-placed tap cell (default: `true`)
- `minGap` – Minimum gap between each inserted tap cell and its nearby instances (default: 0)
- `type` – Insert tap type (`n` / `p`) (default: `p`)
- `leftBoundary` – Insert the left boundary tap cell (default: `false`)
- `rightBoundary` – Insert the right boundary tap cell (default: `false`)

Returns

Collection of created instances

optimizePins

Syntax

```
optimizePins(  
*,  
pins: list[oaPin],  
design: oaDesign,  
keepEdge: bool = None,  
keepOrder: bool = None,  
vertPinGap: int = None,  
horzPinGap: int = None,  
alignPath: lp.optimizePins.AlignPath = None)  
Returns: list[oaPin]
```

Summary:

optimize pins

Category:

Layout Placer

Licenses:

Backend

Parameters

- pins – Pin collection to be optimized
- design – The top-level design
- keepEdge – Whether to keep original edge of pins
- keepOrder – Whether to keep order of pins
- vertPinGap – Defines the vertical gap between pins
- horzPinGap – Defines the horizontal gap between pins
- alignPath – Defines align path method ({ keepOriginalLayer | none | usePathLayer }
(Default is: 'none')

Returns

Pin collection of optimized pins

placePins

Syntax

```
placePins(  
*,  
  nets: list[oaNet],  
  mode: str = None,  
  side: str = None,  
  layer: str,  
  purpose: str = None,  
  length: float,  
  x: float = None,  
  y: float = None,  
  avoidSignal: bool = None,  
  avoidPower: bool = None,  
  shapeType: str = None,  
  stagger: bool = None,  
  staggerParallelExt: float = None,  
  staggerPerpendicularExt: float = None,  
  staggerPerpendicularWidth: float = None,  
  verbose: int = None)  
Returns: list[oaPin]
```

Summary:

pin placer

Category:

Layout Placer

Licenses:

Backend

Parameters

- nets – List of nets on which to create pins
- mode – Mode of pin placement
- side – (Boundary mode) Specify side for pins to be placed on
- layer – Layer to use for created pins
- purpose – Purpose to use for created pins
- length – Exact length of pins in microns
- x – X location of desired pin in microns
- y – Y location of desired pin in microns
- avoidSignal – Avoid creating pins next to signal nets
- avoidPower – Avoid creating pins next to power nets
- shapeType – Type of created pin shape
- stagger – (Area mode) Stagger pins to spread out more
- staggerParallelExt – (Area mode) Set parallel extension of stagger pins
- staggerPerpendicularExt – (Area mode) Set perpendicular extension of stagger pins
- staggerPerpendicularWidth – (Area mode) Set perpendicular width of stagger pins
- verbose – Show debug messages from pin placer

Returns

Pin collection of created pins

showOptions

Syntax

```
showOptions(  
    *,  
    parent: giWindow | int = None,  
    **kwargs)  
Returns: giDialog
```

Summary:

Opens Placement Options dialog.

Category:

Layout Placer

Licenses:

Backend

Parameters

parent – Window or id of parent window from which to open Placement Options dialog.

Returns

The Dialog.

snps.custom.cmd.ilp

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform placement operations. Interactive commands are listed in this chapter.

This section covers the following topics:

- [autoPinPlace](#)
- [autoPlace](#)
- [copyResult](#)
- [extractTopology](#)

- [optimizePins](#)
- [placeToRows](#)

autoPinPlace

Syntax

```
autoPinPlace(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command to run automatic pin placement

Category:

Layout Placer

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

autoPlace

Syntax

```
autoPlace(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command to run automatic placement.

Category:

Layout Placer

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

copyResult

Syntax

```
copyResult(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command to copy placement results back to layout.

Category:

Layout Placer

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

extractTopology

Syntax

```
extractTopology(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command to extract the layout topology of the selected objects and create constraints.

Category:

Layout Placer

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

optimizePins

Syntax

```
optimizePins(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command to perform pin optimization

Category:

Layout Placer

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

placeToRows

Syntax

```
placeToRows(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command to perform place to rows

Category:

Layout Placer

Licenses:

Memory Placement Routing

Parameters

context – Context of activation.

Returns

None

snps.custom.cmd.lx

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform schematic driven layout operations.

For details, see:

- [addConditionalDummy](#)
- [addPenalty](#)
- [applyPresetParams](#)
- [changeInstanceMapping](#)
- [clearAnchors](#)
- [clearCrossHighlight](#)
- [clearHighlight](#)
- [clearNetVoltage](#)
- [clearRuler](#)
- [createSession](#)
- [establishCorrespondence](#)

- [establishCorrespondenceFromLVS](#)
- [expandSchematic](#)
- [exportSdlNetlist](#)
- [findCloneCandidates](#)
- [generatePlacementCatalog](#)
- [getAssociatedObject](#)
- [getAssociatedObjectByName](#)
- [getDeviceInfo](#)
- [getFoldingParam](#)
- [getFoldingThreshold](#)
- [getMismatchedConstraints](#)
- [getPlacement](#)
- [getSession](#)
- [getSessions](#)
- [highlightFlightline](#)
- [importSDLMappingFile](#)
- [loadAddShapeSettings](#)
- [loadCellTemplate](#)
- [snps.custom.cmd.lx](#)
- [preview](#)
- [realize](#)
- [realizeConstraint](#)
- [realizeTemplate](#)
- [rechain](#)
- [registerLayoutTemplate](#)
- [reloadSymbolicEditor](#)
- [removePeriphery](#)

- [reopenLogicalSource](#)
- [replacePlacement](#)
- [saveAddShapeSettings](#)
- [saveCellTemplate](#)
- [saveConstraint](#)
- [sedGetDesign](#)
- [setPlacement](#)
- [setupSDL](#)
- [showAddShapeSettingsDialog](#)
- [showAllNet](#)
- [showBatchCreateTemplate](#)
- [showCellTemplate](#)
- [showCheckerDialog](#)
- [showCheckOptions](#)
- [showCreateTemplate](#)
- [showDefineSource](#)
- [showDefineTarget](#)
- [showExpandSchematic](#)
- [showGateConnected](#)
- [showGateNet](#)
- [showGenerateLayout](#)
- [showImportLayout](#)
- [showODNet](#)
- [showOptions](#)
- [showPresetSetup](#)
- [showSDLCheckAndRepairWindow](#)
- [showSDLMapping](#)

- [showSymbolicDesignOptions](#)
- [showUpdateSchematic](#)
- [splitDevice](#)
- [unregisterLayoutTemplate](#)
- [updateConstraints](#)
- [updateLayout](#)
- [verify](#)
- [verifyNetVoltage](#)

addConditionalDummy

Syntax

```
addConditionalDummy(  
*,  
    window: int)  
Returns: None
```

Summary:

Add Conditional Dummy

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window to add penalty tiles

Returns

None

addPenalty

Syntax

```
addPenalty(  
*,
```

```
    window: int)
Returns: None
```

Summary:

Add Penalty Tiles

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window to add penalty tiles

Returns

None

applyPresetParams

Syntax

```
applyPresetParams (
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |
    list[oaFigGroup] | list[oaInst],
/,
*,
lcv: dmCellView | dmCellViewName)
Returns: None
```

Summary:

Apply preset parameter on layout instances

Category:

SDL

Licenses:

Backend

Parameters

- objects – List of instances to apply preset parameters.
- lcv – Option to specify the preset lcv that is used to apply on the layout instances

Returns

None

changeInstanceMapping

Syntax

```
changeInstanceMapping(  
*,  
    design: oaDesign,  
    instName: str,  
    lcv: dmCellView | dmCellViewName)  
Returns: None
```

Summary:

Change instance mapping

Category:

SDL

Licenses:

Backend

Parameters

- design – Target edit layout design
- instName – Logical instance name
- lcv – Target mapping cellview

Returns

None

clearAnchors

Syntax

```
clearAnchors(  
*,  
    design: oaDesign = None,  
    cellView: dmCellView | dmCellViewName = None,  
    keepByNames: str = None,  
    keepBySignalTypes: str = None)  
Returns: None
```

Summary:

Clear anchors of nets/vias.

Category:

SDL

Licenses:

Backend

Parameters

- design – Layout design will be cleared via/net shape anchors.
- cellView – Layout cellView will be cleared via/net shape anchors.
- keepByNames – Skip to remove anchors by net name.
- keepBySignalTypes – Skip to remove anchors by signal types.

Returns

None

clearCrossHighlight**Syntax**

```
clearCrossHighlight(  
*,  
    window: giWindow | int)  
Returns: None
```

Summary:

Clear cross highlight from symbolic editor

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Symbolic Editor Window(id) to clear highlight

Returns

None

clearHighlight

Syntax

```
clearHighlight(  
*,  
    window: int)  
Returns: None
```

Summary:

Clear highlight

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window for clearing highlight

Returns

None

clearNetVoltage

Syntax

```
clearNetVoltage(  
*,  
    design: oaDesign)  
Returns: None
```

Summary:

Clear the net voltage information in logical net.

Category:

SDL

Licenses:

Backend

Parameters

design – Layout design to be clear net voltage

Returns

None

clearRuler

Syntax

```
clearRuler(  
*,  
    window: int)  
Returns: None
```

Summary:

Clear ruler

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window for clearing ruler

Returns

None

createSession

Syntax

```
createSession(  
*,  
    target: deContext,  
    source: deContext)  
Returns: lxSession
```

Summary:

Create and initialize an SDL session.

Category:

SDL

Licenses:

None

Parameters

- target – The target layout design context
- source – The source design context bound by the physical configuration

Returns

SDL session

establishCorrespondence**Syntax**

```
establishCorrespondence (  
*,  
    design: oaDesign,  
    setIgnored: bool = None)  
Returns: None
```

Summary:

Establish correspondence.

Category:

SDL

Licenses:

Backend

Parameters

- design – Layout design would be analyzed and set correspondence
- setIgnored – Set ignored for the extra instances if all of logical instance are matched (Default is: False)

Returns

None

establishCorrespondenceFromLVS

Syntax

```
establishCorrespondenceFromLVS(  
*,  
    layout: dmCellView | dmCellViewName,  
    schematic: dmCellView | dmCellViewName = None,  
    path: str,  
    hierarchy: bool = None,  
    outputFile: str = None,  
    skipCellFile: str = None,  
    skipCells: list[str] = None,  
    skipLibraries: list[str] = None,  
    targets: dmCellView | dmCellViewName | list[dmCellView] = None,  
    setIgnored: bool = None,  
    fixConnectivity: bool = None,  
    resetConnection: bool = None,  
    scalarizeArrayInst: bool = None,  
    scalarizeProc: str = None,  
    splitDevice: bool = None,  
    splitProc: str = None,  
    lvsTool: lx.establishCorrespondenceFromLVS.LvsTool = None,  
    **kwargs)  
Returns: None
```

Summary:

Establishe Correspondence From the LVS result.

Category:

SDL

Licenses:

Elite

Parameters

- layout – Cell view of the layout
- schematic – Cell view of the schematic
- path – The run directory for LVS
- hierarchy – Set correspondence for the top designonly or layout hierarchy (Default is: False)

- `outputFile` – The output file for the renaming result. (Default is: "")
- `skipCellFile` – The file contains skip cell list which skips hierarchical verification.
- `skipCells` – The cells skip hierarchical verification.
- `skipLibraries` – The libraries skip hierarchical verification.
- `targets` – The target cell views. (Default is: "")
- `setIgnored` – Set ignored for the extra instances if all of logical instance are matched (Default is: False)
- `fixConnectivity` – Fix connectivity mismatch or not. (Default is: True)
- `resetConnection` – Reset connection or not. (Default is: False)
- `scalarizeArrayInst` – Scalarize array instances. Use `le::scalarizeArrayInst` if the scalarize procedure is not specified by `-scalarizeProc`. (Default is: False)
- `scalarizeProc` – The procedure to scalarize array instances. (Default is: 'le::scalarizeArrayInst')
- `splitDevice` – Split devices. Use `lx::splitDevice` if the split procedure is not specified by `-splitProc`. (Default is: False)
- `splitProc` – The procedure to split devices. (Default is: 'lx::splitDevice')
- `lvsTool` – The tool name used to run LVS. ({ Calibre | ICV }) (Default is: 'ICV')

Returns

None

expandSchematic

Syntax

```
expandSchematic(  
*,  
  source: dmCellView | oaLib | str,  
  target: oaLib | str,  
  copySchematic: bool = None,  
  excludeLibs: list[str] = None,  
  overwrite: bool = None,  
  symbolMapFile: str = None,  
  layoutMapFile: str = None,  
  referenceLayoutLib: str = None,  
  copyOnly: bool = None,  
  ignoreLibs: str = None,  
  ignoreAllExceptSource: bool = None,  
  overwriteLayout: bool = None,
```

```
sourceVSL: list[str] = None,  
logFile: str = None)  
Returns: None
```

Summary:

Expand Schematic

Category:

SDL

Licenses:

Backend

Parameters

- source – Source to be expanded, Schematic/Config cell-view or Library
- target – Destination Library for soft-macro cells
- copySchematic – Copy schematic from source to target (Default is: False)
- excludeLibs – Specifies libraries whose cellViews should not be copied
- overwrite – Action to take when a copied cell already exists in the destination (Default is: False)
- symbolMapFile – Specifies symbol mapping file
- layoutMapFile – Specifies layout mapping file
- referenceLayoutLib – Libraries from which layout cellviews are copied into target
- copyOnly – Copy schematic without creating soft macros (Default is: False)
- ignoreLibs – Libraries which will not be copied or expanded
- ignoreAllExceptSource – Ignore all libraries except source schematic library (Default is: False)
- overwriteLayout – Action to take when a reference layout already exists in the destination (Default is: False)
- sourceVSL – Specifies View Search List to use while elaborating the schematic
- logFile – Specifies log file

Returns

None

exportSdlNetlist

Syntax

```
exportSdlNetlist(  
    *,  
    src: dmCellView | dmCellViewName = None,  
    dst: dmCellView | dmCellViewName,  
    schematic: dmCellView | dmCellViewName = None,  
    config: dmCellView | dmCellViewName = None,  
    overwrite: bool = None,  
    **kwargs)  
Returns: None
```

Summary:

Export SDL Netlist

Category:

SDL

Licenses:

Elite

Parameters

- src – Cell view of source layout view
- dst – Cell view of destination netlist view
- schematic – Cell view of source schematic view
- config – Cell view of source config view
- overwrite – Overwrite existing destination netlist view (Default is: False)

Returns

None

findCloneCandidates

Syntax

```
findCloneCandidates(  
    objects: oaDesign | list[deFigure] | list[deSelectionSet] |  
    list[oaFig] | list[oaFigGroup] | list[oaInst] | list[oaMarker] |  
    list[oaShape] | list[oaVia],  
    /,  
    *,
```

```
design: oaDesign = None,  
**kwargs)  
Returns: dbCollection
```

Summary:

Returns clones for the given layout objects.

Category:

SDL

Licenses:

Backend

Parameters

- objects – Collection of layout objects representing the layout source clone
- design – Design to search for clone candidates. (Default is: 'Design that the input objects are from.')

Returns

Collection of collections of objects from a layout design representing the layout source clone.

generatePlacementCatalog

Syntax

```
generatePlacementCatalog(  
*,  
window: int,  
group: list[str],  
pattern: str,  
yaml: str = None,  
wMin: int = None,  
wMax: int = None,  
lMin: int = None,  
lMax: int = None,  
wbylMin: int = None,  
wbylMax: int = None,  
wbylIgnore: bool = None)  
Returns: None
```

Summary:

Create Placement Catalog for Design Centering

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- window – SED window(id) with group for which patterns are to be created
- group – The group of devices being optimized
- pattern – The pattern to place device in SED matching
- yaml – The YAML file name
- wMin – Minimum width W as a % of current width of device
- wMax – Maximum width W as a % of current width of device
- lMin – Minimum length L as a % of current length of device
- lMax – Maximum length L as a % of current length of device
- wbylMin – Minimum width to length ration W/L as a % of current ratio of device
- wbylMax – Maximum width to length ration W/L as a % of current ratio of device
- wbylIgnore – Ignore W/L checks during options generation

Returns

None

getAssociatedObject

Syntax

```
getAssociatedObject(  
  object: deContext | oaDesign | oaInst | oaNet | oaTerm)  
Returns: deContext | oaDesign | list[deFigure] | list[oaNet] |  
        list[oaTerm]
```

Summary:

Get the associated object

Category:

SDL

Licenses:

Backend

Parameters

object – Context, design, instance, net, or term to find associated objects for

Returns

Associated objects

getAssociatedObjectByName

Syntax

```
getAssociatedObjectByName(  
    name: str,  
    /,  
    *,  
    type: lx.getAssociatedObjectByName.Type,  
    context: deContext)  
Returns: list[deFigure] | list[oaNet] | list[oaTerm]
```

Summary:

Get the associated object by name

Category:

SDL

Licenses:

Backend

Parameters

- name – Object name to find associated objects for
- type – Object type ({ instance | net | terminal })
- context – Design window context

Returns

Associated objects

getDeviceInfo

Syntax

```
getDeviceInfo(  
*,  
    window: giWindow | int = None)  
Returns: None
```

Summary:

Provide to show SED device information

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window(id) of symbolic editor window to invoke show device information command

Returns

None

getFoldingParam

Syntax

```
getFoldingParam(  
*,  
    libName: str,  
    cellName: str)  
Returns: str
```

Summary:

Get the folding parameter name for a component.

Category:

SDL

Licenses:

Backend

Parameters

- libName – Library Name
- cellName – Cell Name

Returns

The folding parameter name

getFoldingThreshold

Syntax

```
getFoldingThreshold(  
*,  
    libName: str,  
    cellName: str)  
Returns: str
```

Summary:

Get the folding threshold value for a component.

Category:

SDL

Licenses:

Backend

Parameters

- libName – Library Name
- cellName – Cell Name

Returns

The folding threshold value

getMismatchedConstraints

Syntax

```
getMismatchedConstraints(  
    session: lxSession)  
Returns: dbCollection
```

Summary:

Return mismatched constraints in layout

Category:

SDL

Licenses:

Backend

Parameters

session – SDL session

Returns

Collection of cmConstraint objects

getPlacement

Syntax

```
getPlacement(  
*,  
    window: giWindow | int = None,  
    mode: lx.getPlacement.Mode = None,  
    single: bool = None)  
Returns: None
```

Summary:

Provides dump the placement in MDC

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- window – Window(id) of symbolic editor window to invoke get placement command
- mode – To show orientation of device ({ detail | normal })
- single – To show placement string by single line

Returns

None

getSession

Syntax

```
getSession(  
    window: giWindow | int = None)  
Returns: lxSession
```

Summary:

Get the SDL session.

Category:

SDL

Licenses:

Backend

Parameters

window – Window to get the SDL session from

Returns

SDL session

getSessions

Syntax

```
getSessions(  
    *,  
    window: giWindow | int = None)  
Returns: list[lxSession]
```

Summary:

Get SDL sessions

Category:

SDL

Licenses:

None

Parameters

window – Window to get the SDL session from

Returns

Sessions matching the search criteria

highlightFlightline

Syntax

```
highlightFlightline(  
*,  
    window: int,  
    net: str,  
    colorIndex: int)  
Returns: None
```

Summary:

Assign highlight on flightline

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- window – Window for assigning highlight
- net – Net name of flightlines to assign highlight
- colorIndex – Color index of highlight color

Returns

None

importSDLMappingFile

Syntax

```
importSDLMappingFile(  
*,  
    lib: oaLib | str,  
    filePath: list[str],
```

```
mode: lx.importSDLMappingFile.Mode = None)
Returns: None
```

Summary:

Import SDL Mapping File

Category:

SDL

Licenses:

Backend

Parameters

- lib – Destination library for sdl mapping file
- filePath – Specify sdl mapping file
- mode – Import Mode ({ append | replace } (Default is: 'replace'))

Returns

None

loadAddShapeSettings

Syntax

```
loadAddShapeSettings (
*,
window: giWindow | int = None,
set: str)
Returns: None
```

Summary:

Load Add Shape settings from external file.

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- window – Window Id to load Add Shapes.
- set – Load settings with specified name.

Returns

None

loadCellTemplate

Syntax

```
loadCellTemplate(  
*,  
    window: giWindow | int = None,  
    set: str)  
Returns: None
```

Summary:

Load the Cell Template from Section.

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- window – Window(id) from which to load Cell Template
- set – Set Name to load the Cell Template Settings

Returns

None

snps.custom.cmd.lx

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform schematic driven layout operations.

For details, see:

preview

Syntax

```
preview(  
*,  
    window: giWindow | int = None,  
    assistant: bool = None,  
    refresh: bool = None)  
Returns: None
```

Summary:

Preview the real layout of symbolic editor

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- window – Window(id) of symbolic editor window to invoke preview command
- assistant – Only update preview assistant if it is visible
- refresh – Refresh preview

Returns

None

realize

Syntax

```
realize(  
*,  
    window: giWindow | int = None)  
Returns: None
```

Summary:

Realize instances in layout design window

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window(id) of symbolic editor window to invoke realize command

Returns

None

realizeConstraint

Syntax

```
realizeConstraint(  
*,  
    context: deContext = None,  
    constraint: str = None,  
    point: point = None,  
    force: bool = None,  
    **kwargs)  
Returns: list[oaFig]
```

Summary:

realize constraint

Category:

Template Designer

Licenses:

Backend

Parameters

- context – Context of activation.
- constraint – Constraint names to be realized. (default: the only top constraint)
- point – Realize the constraint to the specific point. (default: {0 0})
- force – Force to realize the constraint even the placement group is already realized in target design. (Default is: True)

Returns

Collection of created target fig group

realizeTemplate

Syntax

```
realizeTemplate(  
*,  
    context: deContext,  
    libName: str,  
    cellName: str,  
    viewName: str,  
    point: point = None,  
    skipCells: list[str] = None)  
Returns: list[oaFig]
```

Summary:

Realize template view

Category:

SDL Template

Licenses:

Backend

Parameters

- context – Target context to realize template
- libName – Template library name
- cellName – Template cell name
- viewName – Template view name
- point – Realize template to specific point - default is (0, 0)
- skipCells – Cell names to be skipped

Returns

Collection of created top fig groups

rechain

Syntax

```
rechain(  
*,  
    window: int)  
Returns: None
```

Summary:

Re-chain devices

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window for rechainning

Returns

None

registerLayoutTemplate

Syntax

```
registerLayoutTemplate(  
name: str,  
    /,  
    *,  
    title: str,  
    optionSetType: str = None,  
    defaultOptionSetPref: str = None,  
    commandType: lx.registerLayoutTemplate.CommandType = None)  
Returns: None
```

Summary:

Register custom layout template.

Category:

SDL

Licenses:

Backend

Parameters

- name – Name of the custom layout template Tcl procedure.
- title – Title of the custom layout template
- optionSetType – Type option set name.
- defaultOptionSetPref – Name of the preference which holds the default option set name.
- commandType – Valid command type for the layout template ({ all | generateLayout | pickAndPlace })

Returns

None

reloadSymbolicEditor

Syntax

```
reloadSymbolicEditor(  
*,  
    window: int)  
Returns: None
```

Summary:

Reload devices from constraint

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window for reloading

Returns

None

removePeriphery

Syntax

```
removePeriphery(  
*,  
    window: giWindow | int)  
Returns: None
```

Summary:

Remove periphery

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window(id) of symbolic editor window to invoke remove periphery command

Returns

None

reopenLogicalSource

Syntax

```
reopenLogicalSource(  
*,  
    context: deContext = None,  
    **kwargs)  
Returns: deContext
```

Summary:

Reopen the logical source for an SDL session.

Category:

SDL

Licenses:

None

Parameters

context – Layout Design Window Context

Returns

Source context

replacePlacement

Syntax

```
replacePlacement(  
*,  
    window: int | None = None,  
    verbosity: int | None = None,  
    catalogFilename: str,  
    optVarName: str,  
    placementIndex: int)  
Returns: None
```

Summary:

Replaces placement for the Group

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- window – Window Id for SDL session
- verbosity – Level of debug messages (Default is: 0)
- catalogFilename – Name of the placement catalog file
- optVarName – Name of the optimization variable in placement catalog file
- placementIndex – Index of optVar placement to be used

Returns

None

saveAddShapeSettings

Syntax

```
saveAddShapeSettings(  
*,  
    window: giWindow | int = None,  
    set: str)  
Returns: None
```

Summary:

Save Add Shape settings in external file.

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- window – Window Id to save Add Shapes.
- set – Set name to save Add Shape settings.

Returns

None

saveCellTemplate

Syntax

```
saveCellTemplate(  
*,  
    window: giWindow | int = None,  
    set: str)  
Returns: None
```

Summary:

Load the Cell Template from Section.

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- window – Window(id) from which to save Cell Template
- set – Set Name to load the Cell Template Settings

Returns

None

saveConstraint

Syntax

```
saveConstraint(  
*,  
    window: int)  
Returns: None
```

Summary:

Save Constraint

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window for save constraint

Returns

None

sedGetDesign

Syntax

```
sedGetDesign(  
*,  
    window: giWindow | int,  
    type: lx.sedGetDesign.Type = None)  
Returns: oaDesign
```

Summary:

Returns the requested design id

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- window – Window(id) of symbolic editor window to invoke getDesign command
- type – Type of design used in SED session ({ preview | sdlLayout | symbolic } (Default is: 'sdlLayout')

Returns

OA Design Id

setPlacement

Syntax

```
setPlacement(  
*,  
    window: giWindow | int = None,  
    pattern: str,  
    column: int = None,  
    repeatX: int = None,  
    repeatY: int = None)  
Returns: None
```

Summary:

Provide to set placement in MDC

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- window – Window(id) of symbolic editor window to invoke set placement command
- pattern – To set input pattern

- column – To set column number
- repeatX – To set repeat in X
- repeatY – To set repeat in Y

Returns

None

setupSDL

Syntax

```
setupSDL(  
*,  
    layout: dmCellView | dmCellViewName = None,  
    config: dmCellView | dmCellViewName = None,  
    schematic: dmCellView | dmCellViewName = None)  
Returns: None
```

Summary:

A command to setup SDL.

Category:

SDL

Licenses:

Backend

Parameters

- layout – Cell view of the layout
- config – Cell view of the layout.config
- schematic – Cell view of the schematic

Returns

None

showAddShapeSettingsDialog

Syntax

```
showAddShapeSettingsDialog(  
*,
```

```
parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the Add Shape Settings dialog.

Category:

Symbolic Editing

Licenses:

Backend

Parameters

parent – Window Id to initialize Add Shapes.

Returns

The Dialog

showAllNet

Syntax

```
showAllNet (  
*,  
    window: int)  
Returns: None
```

Summary:

Show All Connections of Flightlines

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window for showing all flightlines

Returns

None

showBatchCreateTemplate

Syntax

```
showBatchCreateTemplate()  
Returns: None
```

Arguments:

Summary:

Shows the 'Batch Create Template' dialog

Category:

Template Designer

Licenses:

Backend

Returns

None

showCellTemplate

Syntax

```
showCellTemplate(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the Cell Template dialog.

Category:

Symbolic Editing

Licenses:

Backend

Parameters

parent – Window(id) from which to initialize Cell Template

Returns

The Dialog

showCheckerDialog

Syntax

```
showCheckerDialog(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the Check Error dialog.

Category:

Symbolic Editing

Licenses:

Backend

Parameters

parent – Window(id) from which to initialize Check Error Dialog

Returns

The Dialog

showCheckOptions

Syntax

```
showCheckOptions(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the ECO Setup Options dialog.

Category:

SDL

Licenses:

None

Parameters

parent – Window(id) of parent window from which to initialize ECO Setup preferences

Returns

The Dialog

showCreateTemplate

Syntax

```
showCreateTemplate()  
Returns: None
```

Arguments:

Summary:

Shows the 'Create Template' dialog. If the dialog is already opened, it will be raised and focused.

Category:

Template Designer

Licenses:

Backend

Returns

None

showDefineSource

Syntax

```
showDefineSource(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the SDL Define Logical Source dialog.

Category:

SDL

Licenses:

None

Parameters

parent – Window(id) of parent window to initialize SDL from

Returns

The Dialog

showDefineTarget

Syntax

```
showDefineTarget (  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the SDL Define Target dialog.

Category:

SDL

Licenses:

None

Parameters

parent – Window(id) of parent window to initialize SDL from

Returns

The Dialog

showExpandSchematic

Syntax

```
showExpandSchematic (  
*,
```

```
parent: giWindow | int = None)
Returns: giDialog
```

Summary:

Show Expand Schematic window

Category:

SDL

Licenses:

Backend

Parameters

parent – parent widget

Returns

The dialog

showGateConnected

Syntax

```
showGateConnected(
*,
window: int)
Returns: None
```

Summary:

Show All Flightlines Connected to Gate

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window for showing gate-to-all flightlines

Returns

None

showGateNet

Syntax

```
showGateNet (  
    *,  
    window: int)  
Returns: None
```

Summary:

Show Gate-to-Gate Flightlines

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window for showing gate-to-gate flightlines

Returns

None

showGenerateLayout

Syntax

```
showGenerateLayout (  
    *,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the SDL Generate Layout dialog.

Category:

SDL

Licenses:

Backend

Parameters

parent – Window(id) of layout editor window to generate layout in

Returns

The Dialog

showImportLayout

Syntax

```
showImportLayout (  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the LE Import Layout dialog.

Category:

Layout Editing

Licenses:

Basic

Parameters

parent – Window(id) of layout editor window to import layout in

Returns

The Dialog

showODNet

Syntax

```
showODNet (  
*,  
    window: int)  
Returns: None
```

Summary:

Show S/D-to-S/D Flightlines

Category:

Symbolic Editing

Licenses:

Backend

Parameters

window – Window for showing S/D-to-S/D flightlines

Returns

None

showOptions

Syntax

```
showOptions (  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the SDL Options dialog.

Category:

SDL

Licenses:

None

Parameters

parent – Window(id) of parent window from which to initialize SDL options

Returns

The Dialog

showPresetSetup

Syntax

```
showPresetSetup (  
**kwargs)  
Returns: giDialog
```

Arguments:

Summary:

Open Preset Setup Dialog

Category:

SDL

Licenses:

Backend

Returns

The Dialog

showSDLCheckAndRepairWindow

Syntax

```
showSDLCheckAndRepairWindow()  
Returns: None
```

Arguments:

Summary:

Show SDL check and repair window

Category:

SDL

Licenses:

Basic

Returns

None

showSDLMapping

Syntax

```
showSDLMapping(  
*,  
parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the dialog of SDL mapping.

Category:

SDL

Licenses:

Backend

Parameters

parent – Window(id) of parent window from which to initialize the dialog

Returns

The Dialog

showSymbolicDesignOptions

Syntax

```
showSymbolicDesignOptions(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the Symbolic Design Options dialog.

Category:

Symbolic Editing

Licenses:

Backend

Parameters

parent – Window(id) from which to initialize Symbolic Design Options

Returns

The Dialog

showUpdateSchematic

Syntax

```
showUpdateSchematic(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the SDL Update Schematic from Layout dialog.

Category:

SDL

Licenses:

Backend

Parameters

parent – Window(id) of layout editor window to update schematic from layout in

Returns

The Dialog

splitDevice

Syntax

```
splitDevice(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
    list[oaFigGroup] | list[oaInst],  
/,  
*,  
    externalConnection: lx.splitDevice.ExternalConnection = None,  
    pattern: str = None)  
Returns: list[oaInst]
```

Summary:

Split device

Category:

SDL

Licenses:

Backend

Parameters

- `objects` – List of instances to split.
- `externalConnection` – Option to specify how to create abutment external connection (`{ auto | disable | enable }`) (Default is: 'auto')
- `pattern` – Option to specify number of fingers for newly created splitted instance. (Default is: '1')

Returns

Collection of newly created instances after split.

unregisterLayoutTemplate

Syntax

```
unregisterLayoutTemplate(  
    name: str  
)  
Returns: None
```

Summary:

Unregister custom layout template.

Category:

SDL

Licenses:

Backend

Parameters

`name` – Name of the custom layout template Tcl procedure.

Returns

None

updateConstraints

Syntax

```
updateConstraints(  
    session: lxSession,  
    /*,  
    mode: lx.updateConstraints.Mode = None)  
Returns: dbCollection
```

Summary:

Transfer new and/or update existing logic constraints to layout

Category:

SDL

Licenses:

Backend

Parameters

- session – SDL session
- mode – Update Mode ({ all | exist | new } (Default is: 'new'))

Returns

Collection of cmConstraint objects

updateLayout

Syntax

```
updateLayout(  
    /*,  
    objects: list[deFigure] | list[oaFig] | list[oaFigGroup] |  
             list[oaInst] | list[oaNet] | list[oaShape] | list[oaVia] = None,  
    design: oaDesign = None,  
    **kwargs)  
Returns: None
```

Summary:

Auto fix violations in the layout.

Category:

SDL

Licenses:

Backend

Parameters

- objects – List of layout objects to be fixed
- design – Layout design to be fixed

Returns

None

verify

Syntax

```
verify(  
*,  
context: deContext = None,  
cellView: dmCellView | dmCellViewName = None,  
verifyMode: lx.verify.VerifyMode = None,  
createMarker: bool = None,  
hierarchy: bool = None,  
outputFile: str = None,  
skipCellFile: str = None,  
skipCells: list[str] = None,  
skipLibraries: list[str] = None,  
**kwargs)  
Returns: list[oaFig]
```

Summary:

Verify

Category:

SDL

Licenses:

Backend

Parameters

- context – Layout Design Window Context
- cellView – The physical cellview.
- verifyMode – Option to Specify which mode to verify ({ PLE | SDL | VDRC }) (Default is: 'SDL')

- createMarker – Whether to export violations as OA markers. (Default is: True)
- hierarchy – The flag whether to verify designs through the hierarchy. (Default is: False)
- outputFile – The output file. (Default is: “”)
- skipCellFile – The file contains skip cell list which skips hierarchical verification.
- skipCells – The cells skip hierarchical verification.
- skipLibraries – The libraries skip hierarchical verification.

Returns

Collection of Markers

verifyNetVoltage

Syntax

```
verifyNetVoltage(  
*,  
    design: oaDesign)  
Returns: None
```

Summary:

Verify the net voltage range between schematic net and layout net.

Category:

SDL

Licenses:

Backend

Parameters

design – Layout design to be verified

Returns

None

snps.custom.cmd.ilx

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These

submodules let you perform schematic driven layout operations. Interactive commands are listed in this chapter.

For details, see:

- [add](#)
- [addDummy](#)
- [addIncremental](#)
- [addPeripheralDummy](#)
- [alignGates](#)
- [alignIndividual](#)
- [alignRow](#)
- [applyPresetParams](#)
- [associateSpareDummy](#)
- [calculateGridSize](#)
- [chainSerialRes](#)
- [clone](#)
- [copy](#)
- [customizeSpacing](#)
- [delete](#)
- [deleteTrack](#)
- [distance](#)
- [establishCorrespondence](#)
- [flip](#)
- [flipRow](#)
- [foldGates](#)
- [generateGroupPlacements](#)
- [generateGuardRings](#)
- [generatePattern](#)
- [generatePlacement](#)

- [highlightFlightline](#)
- [insertColumn](#)
- [insertRow](#)
- [insertShift](#)
- [insertTrack](#)
- [manualCorrespondence](#)
- [merge](#)
- [mergeGates](#)
- [mirror](#)
- [modifyDummy](#)
- [modifyFlightline](#)
- [modifyPode](#)
- [modifyPreset](#)
- [move](#)
- [moveFlightline](#)
- [optimizeOD](#)
- [pickAndPlace](#)
- [reassignNet](#)
- [rechain](#)
- [removeColumn](#)
- [removeLayoutParam](#)
- [removeRow](#)
- [removeShift](#)
- [reportSummary](#)
- [resetCustomizedSpacing](#)
- [resetIndividualAlignment](#)
- [rotate](#)

- [sedRealize](#)
- [selectEmpty](#)
- [split](#)
- [splitDevice](#)
- [swapDevice](#)
- [swapInterdigitate](#)
- [swapSD](#)
- [swapTerm](#)
- [symbolicEditor](#)
- [transpose](#)
- [unalignGates](#)
- [unsetFingerGroup](#)
- [updateLayout](#)
- [viaTrackPlanner](#)

add

Syntax

```
add(  
context: deContext | int = None,  
/,  
*,  
window: int = None,  
**kwargs)  
Returns: None
```

Summary:

Interactive symbolic add device command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

addDummy

Syntax

```
addDummy(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic add dummy command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

addIncremental

Syntax

```
addIncremental(  
context: deContext | int = None,  
/,  
*,
```



```
    window: int = None)
Returns: None
```

Summary:

Interactive symbolic add incremental command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

addPeripheralDummy

Syntax

```
addPeripheralDummy(
context: deContext | int = None,
/,
*,
window: int = None)
Returns: None
```

Summary:

Interactive symbolic add peripheral dummies command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

alignGates

Syntax

```
alignGates(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic align gates command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

alignIndividual

Syntax

```
alignIndividual(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic align individual device command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

alignRow

Syntax

```
alignRow(  
context: deContext | int = None,  
/,  
*,  
window: int = None,  
alignment: ilx.alignRow.Alignment)  
Returns: None
```

Summary:

Interactive symbolic align row command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command
- alignment – alignment of align row command ({ bottom | center | top })

Returns

None

applyPresetParams

Syntax

```
applyPresetParams(  
    context: deContext | int = None  
Returns: None
```

Summary:

Brings up Apply Preset Params toolbar and starts apply the preset params on the selected layout instance.

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

associateSpareDummy

Syntax

```
associateSpareDummy(  
    context: deContext | int = None,  
    /,  
    *,  
    window: int = None)  
Returns: None
```

Summary:

Interactive symbolic associate spare dummy command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

calculateGridSize

Syntax

```
calculateGridSize(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic calculate grid size command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

chainSerialRes

Syntax

```
chainSerialRes(  
context: deContext | int = None,  
/,  
*,
```

```
    window: int = None)
Returns: None
```

Summary:

Interactive symbolic chain serial resistor command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

clone

Syntax

```
clone(
    context: deContext | int = None)
Returns: None
```

Summary:

Interactive clone command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

copy

Syntax

```
copy(  
context: deContext | int = None,  
/,  
*,  
window: int = None,  
**kwargs)  
Returns: None
```

Summary:

Interactive symbolic copy command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

customizeSpacing

Syntax

```
customizeSpacing(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic customize spacing command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

delete

Syntax

```
delete(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic delete command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

deleteTrack

Syntax

```
deleteTrack(  
  context: deContext | int = None,  
  /,  
  *,  
  window: int = None)  
Returns: None
```

Summary:

Interactive symbolic delete track command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

distance

Syntax

```
distance(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive Measure Distance Command in SED.

Category:

Symbolic Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

establishCorrespondence

Syntax

```
establishCorrespondence(  
    context: deContext | int = None,  
    /,  
    **kwargs)  
Returns: None
```

Summary:

Interactive establish correspondence command

Category:

SDL

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

flip

Syntax

```
flip(  
    context: deContext | int = None,  
    /,  
    *,  
    window: int = None)  
Returns: None
```

Summary:

Interactive symbolic flip command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

flipRow**Syntax**

```
flipRow(  
  context: deContext | int = None,  
  //,  
  *,  
  window: int = None)  
Returns: None
```

Summary:

Interactive symbolic flip row command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

foldGates

Syntax

```
foldGates(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic fold gates command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

generateGroupPlacements

Syntax

```
generateGroupPlacements(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic group placements generator command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

generateGuardRings

Syntax

```
generateGuardRings(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic generate guard rings command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

generatePattern

Syntax

```
generatePattern(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic generate pattern command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

generatePlacement

Syntax

```
generatePlacement(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic generate placement command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

highlightFlightline

Syntax

```
highlightFlightline(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic highlight flight lines command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

insertColumn

Syntax

```
insertColumn(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic insert column command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

insertRow

Syntax

```
insertRow(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic insert row command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

insertShift

Syntax

```
insertShift(  
context: deContext | int = None,  
/,  
*,  
window: int = None,  
direction: ilx.insertShift.Direction)  
Returns: None
```

Summary:

Interactive symbolic insert shift command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command
- direction – direction of insert shift command ({ down | right })

Returns

None

insertTrack

Syntax

```
insertTrack(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic insert track command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

manualCorrespondence

Syntax

```
manualCorrespondence(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive manual correspondence command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

merge

Syntax

```
merge(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic merge sd command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

mergeGates

Syntax

```
mergeGates(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic merge gates command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

mirror

Syntax

```
mirror(  
context: deContext | int = None,  
/,  
*,  
window: int = None,  
flip: ilx.mirror.Flip)  
Returns: None
```

Summary:

Interactive symbolic mirror command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command
- flip – Mirror type of rotation. ({ horizontal | rotate90 | vertical })

Returns

None

modifyDummy

Syntax

```
modifyDummy(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic modify dummy command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

modifyFlightline

Syntax

```
modifyFlightline(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic modify flight line command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

modifyPode

Syntax

```
modifyPode(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic modify pode net command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

modifyPreset

Syntax

```
modifyPreset(  
    context: deContext | int = None,  
    /,  
    *,  
    window: int = None)  
Returns: None
```

Summary:

Interactive symbolic modify preset command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

move

Syntax

```
move(  
    context: deContext | int = None,  
    /,  
    *,  
    window: int = None,  
    **kwargs)  
Returns: None
```

Summary:

Interactive symbolic move command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

moveFlightline

Syntax

```
moveFlightline(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic move flight line command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

optimizeOD

Syntax

```
optimizeOD(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic optimize OD sharing command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

pickAndPlace

Syntax

```
pickAndPlace(  
context: deContext | int = None,  
/,  
*,  
inst: list[str] = None,  
net: list[str] = None,  
term: list[str] = None,  
node: list[str] = None,  
group: list[str] = None)  
Returns: None
```

Summary:

Interactive pick and place command

Category:

Layout Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- inst – To specify the instance names that for placing
- net – To specify the net names that for placing
- term – To specify the term names that for placing
- node – To specify the node names that for placing
- group – To specify the group names that for placing

Returns

None

reassignNet

Syntax

```
reassignNet(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic reassign net command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

rechain

Syntax

```
rechain(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic rechain command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

removeColumn

Syntax

```
removeColumn(  
context: deContext | int = None,  
/,  
*,
```

```
    window: int = None)
Returns: None
```

Summary:

Interactive symbolic remove column command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

removeLayoutParam

Syntax

```
removeLayoutParam(
context: deContext | int = None,
/,
*,
window: int = None)
Returns: None
```

Summary:

Interactive symbolic remove layout parameters command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

removeRow

Syntax

```
removeRow(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic remove row command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

removeShift

Syntax

```
removeShift(  
context: deContext | int = None,  
/,  
*,  
window: int = None,  
direction: ilx.removeShift.Direction)  
Returns: None
```

Summary:

Interactive symbolic remove shift command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command
- direction – direction of remove shift command ({ left | up })

Returns

None

reportSummary

Syntax

```
reportSummary(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic report summary command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

resetCustomizedSpacing

Syntax

```
resetCustomizedSpacing(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic reset customized spacing command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

resetIndividualAlignment

Syntax

```
resetIndividualAlignment(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic reset individual alignment command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

rotate

Syntax

```
rotate(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic rotate command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

sedRealize

Syntax

```
sedRealize(  
context: deContext | int = None,  
/,  
*,  
session: lxSymbolicEditorSession = None)  
Returns: None
```

Summary:

Realize from Symbolic Editor

Category:

Layout Editing

Licenses:

Basic

Parameters

- context – Context of activation.
- session – Symbolic Editor session

Returns

None

selectEmpty

Syntax

```
selectEmpty(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic select empty command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

split

Syntax

```
split(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic split command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

splitDevice

Syntax

```
splitDevice(  
context: deContext | int = None)  
Returns: None
```

Summary:

Brings up Split Device toolbar and starts the operation of splitting instance objects.

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

swapDevice

Syntax

```
swapDevice(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic swap device command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

swapInterdigitate

Syntax

```
swapInterdigitate(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic swap interdigitate command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

swapSD

Syntax

```
swapSD(  
context: deContext | int = None,  
/,  
*,
```

```
    window: int = None)
Returns: None
```

Summary:

Interactive symbolic swap SD command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

swapTerm

Syntax

```
swapTerm(
context: deContext | int = None,
/,
*,
window: int = None)
Returns: None
```

Summary:

Interactive symbolic swap terms command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

symbolicEditor

Syntax

```
symbolicEditor(  
context: deContext | int = None,  
/,  
*,  
source: ilx.symbolicEditor.Source = None,  
mode: ilx.symbolicEditor.Mode = None)  
Returns: None
```

Summary:

Interactive Symbolic Editor command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- source – To specify source to get selected objects ({ layout | layoutDN | schematic })
- mode – To specify the mode of Symbolic Editor ({ chaining | matching })

Returns

None

transpose

Syntax

```
transpose(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic transpose command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

unalignGates**Syntax**

```
unalignGates(  
context: deContext | int = None,  
/,  
*,  
window: int = None)  
Returns: None
```

Summary:

Interactive symbolic unalign gates command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

unsetFingerGroup

Syntax

```
unsetFingerGroup(  
  context: deContext | int = None,  
  /,  
  *,  
  window: int = None)  
Returns: None
```

Summary:

Interactive symbolic unset finger group command

Category:

Symbolic Editing

Licenses:

Backend

Parameters

- context – Context of activation.
- window – Window(id) of symbolic editor window to invoke command

Returns

None

updateLayout

Syntax

```
updateLayout(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive update layout command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

viaTrackPlanner

Syntax

```
viaTrackPlanner(  
    context: deContext | int = None,  
    /,  
    *,  
    source: ilx.viaTrackPlanner.Source = None)  
Returns: None
```

Summary:

Interactive Via Track Planner

Category:

Layout Editor

Licenses:

Backend

Parameters

- context – Context of activation.
- source – To specify source to get selected objects ({ layout | layoutDN } (Default is: 'layout'))

Returns

None

snps.custom.cmd.ne

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform editing of netlist representations of designs..

For details, see:

- [getLogicals](#)

getLogicals

Syntax

```
getLogicals(  
    window: int = None,  
    /,  
    *,  
    position: point = None)  
Returns: list[neLogical]
```

Summary:

Returns the logicals found at given position of given window.

Category:

Netlist Editing

Licenses:

None

Parameters

- window – Window Id
- position – Column and row in neNetlist window of logical to fetch

Returns

Collection of logicals found at given position in given window

snps.custom.cmd.nl

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform netlisting tool operations.

For details, see:

- [addMapping](#)
- [addVariableReference](#)
- [addViewStopList](#)
- [append](#)
- [calculateMapping](#)
- [checkInstHeader](#)
- [checkVHDLPortConsistency](#)
- [checkVHDLUniqueEntityDefinitions](#)
- [clearBuffer](#)
- [clearModuleScope](#)
- [createAMSNetlister](#)
- [createHierMap](#)
- [createHSPICENameMap](#)
- [createNameMap](#)
- [createNetlister](#)
- [createOANameMap](#)
- [createParameterizedNameMap](#)
- [createPartitioner](#)
- [createTerm](#)
- [extractDigitalPowerDomains](#)
- [finalizeNameMap](#)

- [findAMSDomain](#)
- [findAMSDomainObjects](#)
- [findInstNamePrefix](#)
- [findModuleDir](#)
- [findModuleName](#)
- [findNetlistProcedure](#)
- [findParamMap](#)
- [findParamValue](#)
- [flattenDesign](#)
- [generateEvent](#)
- [getAlwaysWriteParamDefault](#)
- [getAMSNetlisters](#)
- [getBuffer](#)
- [getBufferLastPart](#)
- [getEffectiveNetlistInfo](#)
- [getExternalEntities](#)
- [getInstTerms](#)
- [getMappedContext](#)
- [getMappings](#)
- [getModuleDescription](#)
- [getModuleNameMapForAMS](#)
- [getNamesForMapping](#)
- [getNetlisters](#)
- [getNetMembers](#)
- [getNetType](#)
- [getParamNames](#)
- [getReferencedVariables](#)

- [getShortedNets](#)
- [getTerms](#)
- [getVirtualTerms](#)
- [getWireBitOrder](#)
- [getWorkerNetlisters](#)
- [indent](#)
- [instHeaderHandler](#)
- [isLeaf](#)
- [isModuleUpToDate](#)
- [isNetNameGlobal](#)
- [isStringParam](#)
- [isTextModule](#)
- [lookup](#)
- [map](#)
- [mapPath](#)
- [mapTerminalNames](#)
- [message](#)
- [moduleBodyHandler](#)
- [moduleHandler](#)
- [modulePreprocessHandler](#)
- [netlistBeginHandler](#)
- [netlistBodyHandler](#)
- [netlistEndHandler](#)
- [netlistMainHandler](#)
- [openNameMap](#)
- [preloadModuleNames](#)
- [printSpectreInst](#)

- [printSpiceInst](#)
- [printSpiceInstTerms](#)
- [refreshPartitioner](#)
- [revertOutdatedDesigns](#)
- [runNetlister](#)
- [saveModuleState](#)
- [saveNameMap](#)
- [setTermOrder](#)
- [setWireBitOrder](#)
- [uniqueify](#)

addMapping

Syntax

```
addMapping(  
*,  
  map: nlNameMap | nlOANameMap | nlParameterizedNameMap,  
  src: str,  
  dest: str = None,  
  module: heContext = None,  
  scope: str = None,  
  type: nl.addMapping.Type = None)  
Returns: None
```

Category:

Summary:

Adds an entry to the name map table

Licenses:

None

Parameters

- map – Name map object to be queried or modified
- src – Database namespace identifier to be mapped
- dest – Target namespace identifier to be reverse mapped
- module – Context specifying the module level tables

- scope – Scope of identifier
- type – Specifies special states in character maps ({ badChar | delete })

Returns

None

addVariableReference

Syntax

```
addVariableReference(  
    name: str,  
    /,  
    *,  
    netlister: nlNetlister,  
    master: heContext,  
    type: nl.addVariableReference.Type)  
Returns: str
```

Summary:

Adds an entry into the netlister's set of referenced variables.

Category:

Netlist Tools

Licenses:

None

Parameters

- name – The name of the reference.
- netlister – The netlister object to modify.
- master – The context object referencing the input variable.
- type – The type of variable reference. ({ all | design | inst | lineage | parent })

Returns

Entry into netlister's set of referenced variables

addViewStopList

Syntax

```
addViewStopList(  
*,  
  netlist:  nlNetlist,  
  viewStopList: list[str],  
  to:  nlAMSNetlist,  
  master: bool = None)  
Returns: None
```

Summary:

Associates the netlist with view stop list

Category:

Netlist Tools

Licenses:

None

Parameters

- netlist – The netlist to use for the given view stop list.
- viewStopList – The view stop list.
- to – The AMS netlist to add the given view stop list.
- master – If the added view stop list is for the master netlist. (Default is: False)

Returns

None

append

Syntax

```
append(  
string: str,  
/,  
*,  
  netlist:  nlNetlist,  
  comment: bool = None,  
  wrap: bool = None,  
  format: bool = None)  
Returns: None
```


Summary:

Appends given string to the buffer of the given netlister

Category:

Netlist Tools

Licenses:

None

Parameters

- `string` – The text to append to the buffer.
- `netlister` – The `nlNetlister` object
- `comment` – Is the appended string comment (Default is: `False`)
- `wrap` – Wrap long lines using the `lineWrapSuffix` when the text exceeds the maximum line length (Default is: `True`)
- `format` – Format appended text using current indentation, comment formatting and line wrapping. (Default is: `True`)

Returns

None

calculateMapping

Syntax

```
calculateMapping(  
*,  
  map: nlNameMap | nlOANameMap | nlParameterizedNameMap,  
  src: str,  
  fixedFirstChar: str = None,  
  module: heContext = None,  
  add: bool,  
  scope: str = None)  
Returns: str
```

Category:

Summary:

Calculates and optionally adds a source to destination association to a name map object

Licenses:

None

Parameters

- map – Name map object to be queried or modified
- src – Database identifier to be mapped.
- fixedFirstChar – First character
- module – Context specifying the module level
- add – If the name tables are to be updated
- scope – Scope of the identifier

Returns

Input identifier mapped into the proper namespace

checkInstHeader

Syntax

```
checkInstHeader(  
  instHeader: heInstHeader)  
Returns: None
```

Summary:

Verifies the compatibility of terminals and oaParams between placed and switched masters.

Category:

Netlist Tools

Licenses:

None

Parameters

instHeader – Instance header to process

Returns

None

checkVHDLPortConsistency

Syntax

```
checkVHDLPortConsistency(  
*,  
    contexts: list[heContext])  
Returns: None
```

Summary:

Checks the port names and directions between the designs of given contexts and model that is being bound, if such entity exists in VHDL models external directory.

Category:

Netlist Tools

Licenses:

None

Parameters

contexts – List of contexts to be checked.

Returns

None

checkVHDLUniqueEntityDefinitions

Syntax

```
checkVHDLUniqueEntityDefinitions(  
*,  
    leafList: list[heContext],  
    textList: list[heContext] = None)  
Returns: None
```

Summary:

Checks the entity names that differ only by case for 2 leaf cells or for text module and leaf cell, if entity of leaf cell exists in VHDL models external directory.

Category:

Netlist Tools

Licenses:

None

Parameters

- leafList – List of leaf contexts to be checked.
- textList – List of text contexts to be checked vs leaf contexts.

Returns

None

clearBuffer

Syntax

```
clearBuffer(  
*,  
  netlister: nlNetlister,  
  reset: bool = None)  
Returns: None
```

Summary:

Resets the netlister's buffer to the empty string

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The nlNetlister object
- reset – If true the current buffer position and indent level are set to zero (Default is: True)

Returns

None

clearModuleScope

Syntax

```
clearModuleScope(  
*,  
  map: nlNameMap | nlOANameMap | nlParameterizedNameMap,  
  module: heContext,
```

```
scope: str = None)
Returns: None
```

Category:

Summary:

Removes all table entries for the module and scope

Licenses:

None

Parameters

- map – Name map object to be queried or modified
- module – Context specifying the module level tables
- scope – Scope of identifier

Returns

None

createAMSNetlister

Syntax

```
createAMSNetlister(
name: str,
/,
*,
masterNetlister: str = None,
AMSBindings: list[str] = None,
unifiedNamespace: bool = None,
**kwargs)
Returns: nlAMSNetlister
```

Summary:

Create an AMSNetlister object

Category:

Netlist Tools

Licenses:

None

Parameters

- name – Netlister Name
- masterNetlister – Master Netlister Name (Default is: 'HSPICE')
- AMSBindings – The collection of nlNetlister name to view stop list bindings.
- unifiedNamespace – If set to true a common namespace is used for all name mapping. (Default is: True)

Returns

Newly created AMSNetlister object

createHierMap

Syntax

```
createHierMap(  
  map: nlNameMap | nlOANameMap | nlParameterizedNameMap  
Returns: nlNameMap
```

Category:

Summary:

Creates a hierarchical map object

Licenses:

None

Parameters

map – Name map object to be queried or modified

Returns

New nlHierMap object

createHSPICENAMEMap

Syntax

```
createHSPICENAMEMap(  
  *,  
  rootContext: heContext = None)  
Returns: nlNameMap
```

Category:

Summary:

Creates a nlNameMap to convert names to the HSPICE namespace

Licenses:

None

Parameters

rootContext – Root context for hierarchy path mapping

Returns

New nlHspiceMap object

createNameMap

Syntax

```
createNameMap(  
*,  
  openBrace: str = None,  
  closeBrace: str = None,  
  hierDelimiter: str = None,  
  rootContext: heContext = None,  
  leadDelimiter: bool = None,  
  scopeAliases: list[str] = None)  
Returns: nlNameMap
```

Category:

Summary:

Creates a new mapping

Licenses:

None

Parameters

- openBrace – Character beginning multi-bit index (Default is: “)
- closeBrace – Character ending multi-bit index (Default is: ”)
- hierDelimiter – Hierarchy delimiter (Default is: '/')
- rootContext – Root context for hierarchy path mapping

- `leadDelimiter` – If the destination path begins with a delimiter (Default is: `False`)
- `scopeAliases` – Scope check for identifier aliasing

Returns

New `nlNameMap` object

createNetlister

Syntax

```
createNetlister(  
    name: str,  
    /,  
    *,  
    type: nl.createNetlister.Type = None)  
Returns: nlNetlister
```

Summary:

Create a netlister object

Category:

Netlist Tools

Licenses:

None

Parameters

- `name` – Netlister Name
- `type` – The type of the parameters to return. (`{ hspice | user | verilog }`) (Default is: `'user'`)

Returns

Newly created netlister object

createOANameMap

Syntax

```
createOANameMap(  
    *,  
    openBrace: str = None,  
    closeBrace: str = None,  
    hierDelimiter: str = None,  
    rootContext: heContext = None,
```



```
leadDelimiter: bool = None,  
scopeAliases: list[str] = None,  
algo: bool = None,  
oaNameSpace: oaNameSpace)  
Returns: nlOANameMap
```

Category:

Summary:

Creates a new OA namespace mapping

Licenses:

None

Parameters

- openBrace – Character beginning multi-bit index (Default is: “)
- closeBrace – Character ending multi-bit index (Default is: “)
- hierDelimiter – Hierarchy delimiter (Default is: ‘/’)
- rootContext – Root context for hierarchy path mapping
- leadDelimiter – If the destination path begins with a delimiter (Default is: False)
- scopeAliases – Scope check for identifier aliasing
- algo – If mapping purely algorithmic, or can use stored table data (Default is: True)
- oaNameSpace – The destination namespace

Returns

New nlOANameMap object

createParameterizedNameMap

Syntax

```
createParameterizedNameMap(  
*,  
openBrace: str = None,  
closeBrace: str = None,  
hierDelimiter: str = None,  
rootContext: heContext = None,  
leadDelimiter: bool = None,  
scopeAliases: list[str] = None,  
escapePrefix: str = None,  
escapeSuffix: str = None,  
renamePrefix: str = None,
```

```
maxLength: int,  
startCharMap: nlNameMap = None,  
validCharMap: nlNameMap,  
keywords: list[str] = None,  
algo: bool = None,  
prependFirstChar: nl.createParameterizedNameMap.PrependFirstChar =  
None)  
Returns: nlNameMap
```

Category:

Summary:

Creates a parameterized name map

Licenses:

None

Parameters

- openBrace – Character beginning multi-bit index (Default is: “)
- closeBrace – Character ending multi-bit index (Default is: “)
- hierDelimiter – Hierarchy delimiter (Default is: ‘/’)
- rootContext – Root context for hierarchy path mapping
- leadDelimiter – If the destination path begins with a delimiter (Default is: False)
- scopeAliases – Scope check for identifier aliasing
- escapePrefix – String to prepend to invalid identifiers
- escapeSuffix – String to append to invalid identifiers
- renamePrefix – Prefix string for renaming invalid identifiers
- maxLength – The maximum identifier length
- startCharMap – Name map object to transform each start character to a valid destinationstring
- validCharMap – Name map object to transform each character to a valid destinationstring
- keywords – List of reserved identifiers
- algo – If mapping purely algorithmic, or can use stored table data (Default is: False)
- prependFirstChar – The algorithm used to determine if -fixedFirstChar should be prepended ({ always | auto })

Returns

New nlParameterizedNameMap object

createPartitioner

Syntax

```
createPartitioner(  
    *,  
    netlist: nlAMSNetlist,  
    context: heContext)  
Returns: nlPartitioner
```

Summary:

Creates partitioner for given hierarchy context based on given AMS netlist

Category:

Netlist Tools

Licenses:

None

Parameters

- netlist – AMS netlist which settings are used for partition.
- context – Hierarchy context for which the partitioner is destined

Returns

Partitioner object

createTerm

Syntax

```
createTerm(  
    noname: nlTerm | oaTerm | str,  
    /,  
    *,  
    context: heContext = None,  
    net: oaNet | str = None,  
    virtual: bool = None,  
    defaultNet: oaNet | str = None,  
    termType: nl.createTerm.TermType = None,  
    scalarize: bool = None)  
Returns: list[nlTerm]
```

Summary:

Create nlTerm object which represents terminal in the netlisting connectivity model.

Category:

Netlist Tools

Licenses:

None

Parameters

- noname – The terminal or terminal name.
- context – The context used to lookup terminals and nets specified by name.
- net – The net or net name from the terminal.
- virtual – If true, the created terminal(scalarize) should propagate as an inherited connection. (Default is: False)
- defaultNet – The default net used if '-virtual' is true.
- termType – Type of terminal to create. ({ input | inputOutput | jumper | output | switch | tristate | unused })
- scalarize – If true, the input terminal will be split into individual bit terms. (Default is: False)

Returns

Collection of created terminals

extractDigitalPowerDomains

Syntax

```
extractDigitalPowerDomains(  
*,  
    topCellView: heConfig | heContext,  
    createDomains: bool = None)  
Returns: nlICCSupplyInfo
```

Summary:

Scans a design hierarchy and extracts power supply information.

Category:

Netlist Tools

Licenses:

None

Parameters

- topCellView – The design to traverse
- createDomains – Indicates whether power domains should be created in the return data structure. (Default is: True)

Returns

The power domains found in the design hierarchy

finalizeNameMap

Syntax

```
finalizeNameMap(  
  map: nlNameMap | nlOANameMap | nlParameterizedNameMap,  
  /,  
  *,  
  module: heContext = None)  
Returns: None
```

Category:

Summary:

Removes identity mappings from the name mapobject

Licenses:

None

Parameters

- map – Name map object to be queried or modified
- module – Context specifying the module level tables

Returns

None

findAMSDomain

Syntax

```
findAMSDomain(  
*,  
  of: deFigure | oaInst | oaInstTerm | oaNet | oaShape | oaTerm  
      = None,  
  context: heContext,  
  partitioner: nlPartitioner)  
Returns: str
```

Summary:

Returns the domain type of specified object or context.

Category:

Netlist Tools

Licenses:

None

Parameters

- of – Object
- context – Hierarchy context
- partitioner – Partitioner to be used

Returns

Domain type

findAMSDomainObjects

Syntax

```
findAMSDomainObjects(  
name: nl.findAMSDomainObjects.Name,  
/,  
*,  
  context: heContext,  
  partitioner: nlPartitioner)  
Returns: list[oaBlockObject]
```

Summary:

Returns collection of OA objects of corresponding domain type in the current design of hierarchy context.

Category:

Netlist Tools

Licenses:

None

Parameters

- name – Domain type ({ AMS | analog | digital | mixed | unknown | verilogA })
- context – Hierarchy context
- partitioner – Partitioner to be used

Returns

Objects container

findInstNamePrefix

Syntax

```
findInstNamePrefix(  
*,  
    instHeader: heInstHeader,  
    tool: str,  
    info: str = None)  
Returns: str
```

Summary:

Determines the instance name prefix used for invoking netlist procedure.

Category:

Netlist Tools

Licenses:

None

Parameters

- instHeader – The queried heInstHeader
- tool – The tool name.
- info – Netlist info (Default is: 'simInfo')

Returns

Prefix used for all instnames bound to given heInstHeader

findModuleDir

Syntax

```
findModuleDir(  
    heContext: heContext,  
    /,  
    *,  
    map: nlNameMap | nlOANameMap | nlParameterizedNameMap)  
Returns: str
```

Category:

Summary:

Retrieves or calculates the directory name in the UNIX namespace for a context

Licenses:

None

Parameters

- heContext – Context object
- map – Name map object to be queried or modified

Returns

Unique directory name for the given context

findModuleName

Syntax

```
findModuleName(  
    heContext: heContext,  
    /,  
    *,  
    map: nlNameMap | nlOANameMap | nlParameterizedNameMap,  
    prefix: str = None,  
    suffix: str = None)  
Returns: str
```


Category:

Summary:

Retrieves the module name in the destination namespace for the given heContext

Licenses:

None

Parameters

- heContext – Context object
- map – Name map object to be queried or modified
- prefix – Prefix to add
- suffix – Suffix to add

Returns

Unique string for the given context

findNetlistProcedure

Syntax

```
findNetlistProcedure(  
*,  
    instHeader: heInstHeader,  
    tool: str)  
Returns: str
```

Summary:

Gets the netlist procedure's name.

Category:

Netlist Tools

Licenses:

None

Parameters

- instHeader – The instance header being processed.
- tool – The tool name to use for querying the Netlist info 'netlistProcedure' field.

Returns

Name of Tcl proc for netlisting instances of given helnstHeader

findParamMap

Syntax

```
findParamMap(  
*,  
    netlistInfo: str,  
    master: heContext)  
Returns: nlNameMap
```

Summary:

Retrieves the propMap for the given master in the specified netlist info scope.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlistInfo – Netlist Info identifier to use for finding the “propMapping” field.
- master – The component queried.

Returns

Name map object containing the mapping from database parameter name to simulator parameter name.

findParamValue

Syntax

```
findParamValue(  
param_name: str,  
/,  
*,  
    netlister: nlNetlister,  
    master: heContext,  
    inst: oaInst = None,  
    parent: heContext = None)  
Returns: str
```

Summary:

Find the parameter value for inst/master.

Category:

Netlist Tools

Licenses:

None

Parameters

- `param_name` – The name of the parameter to lookup.
- `netlister` – The netlister used to track parameter/variable references.
- `master` – The switched master.
- `inst` – The instance used to find parameter.
- `parent` – The parent context of the instance used to track parameters.

Returns

Evaluated value of the named parameter/property

flattenDesign

Syntax

```
flattenDesign(  
    cellView: dmCellView | dmCellViewName,  
    /,  
    *,  
    toolNames: str = None,  
    netlisterName: str = None,  
    src: deContext | dmCellView | dmCellViewName | heContext,  
    overwrite: bool = None,  
    viewSearchList: str = None,  
    viewStopList: str = None,  
    **kwargs)  
Returns: None
```

Summary:

Flattens a design into a new cellview.

Category:

Netlist Tools

Licenses:

None

Parameters

- `cellView` – Cellview to save the flattened design to.
- `toolNames` – The tool name(s) of netlister to find the parameters map for multiplication factor processing.
- `netlisterName` – The name of netlister to find the ignore settings
- `src` – Design to flatten.
- `overwrite` – Overwrite existing design if it exists. (Default is: False)
- `viewSearchList` – View Search List (Default is: 'schematic symbol')
- `viewStopList` – View Stop List (Default is: 'symbol')

Returns

None

generateEvent

Syntax

```
generateEvent(  
    type: nl.generateEvent.Type,  
    /,  
    *,  
    netlister: nlNetlister,  
    module: heContext = None,  
    instHeader: heInstHeader = None)  
Returns: None
```

Summary:

Calls the function registered with the netlister for the specified event.

Category:

Netlist Tools

Licenses:

None

Parameters

- type – The event to generate. ({ instHeader | instPost | instance | module | moduleBody | moduleFooter | moduleHeader | modulePreprocess | netlistBegin | netlistBody | netlistEnd | netlistMain | textModule })
- netlister – The netlister to use for generating events.
- module – The module currently being processed.
- instHeader – The instance header currently being processed.

Returns

None

getAlwaysWriteParamDefault

Syntax

```
getAlwaysWriteParamDefault(  
    name: str,  
    /,  
    *,  
    netlister: nlNetlister,  
    context: heContext)  
Returns: bool
```

Summary:

Determines whether the instance parameter with default value should be printed even with 'nl<name>WriteParamDefaults' preference set 'false'.

Category:

Netlist Tools

Licenses:

None

Parameters

- name – The name of the parameter.
- netlister – The current netlister.
- context – The master context of instance.

Returns

True if parameter should always be printed

getAMSNetlisters

Syntax

```
getAMSNetlisters(  
    pattern: str = None  
Returns: list[nlAMSNetlister]
```

Summary:

Returns the AMS netlister object.

Category:

Netlist Tools

Licenses:

None

Parameters

pattern – Pattern to match AMS netlister name. (Default is: '*')

Returns

Collection of AMS netlisters with names matching input pattern

getBuffer

Syntax

```
getBuffer(  
    *,  
    netlister: nlNetlister)  
Returns: str
```

Summary:

Gets the text in the netlister's output buffer

Category:

Netlist Tools

Licenses:

None

Parameters

netlister – The nlNetlister object

Returns

Contents of current netlister's output buffer

getBufferLastPart

Syntax

```
getBufferLastPart(  
    *,  
    netlister: nlNetlister,  
    start: int,  
    cut: bool = None)  
Returns: str
```

Summary:

Gets the last part of the buffer in the netlister starting from the specified position.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The nlNetlister object.
- start – The start position of the last part of the buffer.
- cut – If true, the last part of the buffer that was found is removed (Default is: False)

Returns

The last part of buffer in the netlister

getEffectiveNetlistInfo

Syntax

```
getEffectiveNetlistInfo(  
    name: nlNetlister,  
    /,  
    *,  
    context: heContext)  
Returns: str
```

Summary:

Gets the netlistInfo field to be used for the given device, for case of multiple info names declared. For only one name returns this sole name, like “db::getAttr netlistInfo ...”.

Category:

Netlist Tools

Licenses:

None

Parameters

- name – Netlister
- context – The context for which the netlistInfo is asked.

Returns

Value of netlistInfo name

getExternalEntities

Syntax

```
getExternalEntities(  
*,  
    directory: str)  
Returns: str
```

Summary:

Gets all entity and architecture pairs for all the files in the specified external directory

Category:

Data Translation

Licenses:

None

Parameters

directory – The name of the external directory

Returns

List of entity and architecture names existing in the specified external directory

getInstTerms

Syntax

```
getInstTerms (  
*,  
  netlister:  nlNetlister,  
  inst:  oaInst,  
  master:  heContext)  
Returns: nlInstTerm
```

Summary:

Retreives an iterator over nlInst term list.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The netlister object.
- inst – The instance inspected.
- master – The switched master of the given instance.

Returns

List of instance terminals

getMappedContext

Syntax

```
getMappedContext (  
*,  
  caseSensitive: bool = None,  
  map: nlHierMap,  
  path: str)  
Returns: heContext
```

Category:

Summary:

Gets the heContext associated with an instance name

Licenses:

None

Parameters

- `caseSensitive` – Specifies whether to respect (true) or ignore (false) case when searching for matches (Default is: True)
- `map` – The hierarchical name map to map the path with
- `path` – Path specifying the instance name to search for

Returns

Hierarchy context object whose top block corresponds to the instance path

getMappings

Syntax

```
getMappings (  
*,  
    map: nlNameMap | nlOANameMap | nlParameterizedNameMap,  
    module: heContext = None,  
    scope: str)  
Returns: None
```

Category:

Summary:

Retrieve all mappings from specified name scope

Licenses:

None

Parameters

- `map` – Name map object to be queried or modified
- `module` – Context specifying the module level tables
- `scope` – Scope of identifier

Returns

None

getModuleDescription

Syntax

```
getModuleDescription(  
*,  
    commentPrefix: str,  
    module: heContext)  
Returns: str
```

Summary:

Returns the description of the given module.

Category:

Netlist Tools

Licenses:

None

Parameters

- commentPrefix – Comment prefix
- module – The module

Returns

Generated module description

getModuleNameMapForAMS

Syntax

```
getModuleNameMapForAMS(  
*,  
    netlister: nlAMSNetlister | nlNetlister,  
    context: heContext)  
Returns: nlNameMap
```

Summary:

Returns name map of worker netlister used for the given module in AMS netlisting.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – Netlister object
- context – The context to retrieve the effective netlister for.

Returns

Name map for given module

getNamesForMapping

Syntax

```
getNamesForMapping(  
*,  
    netlister: nlAMSNetlister | nlNetlister,  
    context:   heContext,  
    objectType: nl.getNamesForMapping.ObjectType,  
    purpose:   nl.getNamesForMapping.Purpose)  
Returns: str
```

Summary:

Retrieves terminal/net names to be mapped.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The netlister object to query.
- context – The context to retrieve the names for.
- objectType – The object type (net or terminal) the name of which is retrieved. ({ net | terminal })
- purpose – The purpose for which the name mapping is needed. Can be 'shorted' for net; 'shorted' or 'globalForRemoveBang' for terminal. ({ globalForRemoveBang | shorted })

Returns

{<action_type> <first_name> [<second_name>] [<other_names>]}

Return type

List of names mapping data. The data has format

getNetlisters

Syntax

```
getNetlisters(  
    pattern: str = None  
Returns: list[nlNetlister]
```

Summary:

Returns the netlister object.

Category:

Netlist Tools

Licenses:

None

Parameters

pattern – Pattern to match netlister name. (Default is: '*')

Returns

Collection of netlisters with names matching input pattern

getNetMembers

Syntax

```
getNetMembers(  
    net: nlNet,  
    /,  
    *,  
    scalarize: bool = None,  
    **kwargs)  
Returns: nlNet
```

Summary:

Split an nlNet into component buses/bundles.

Category:

Netlist Tools

Licenses:

None

Parameters

- `net` – The input bundle.
- `scalarize` – Returns the consituent scalar nets. (Default is: False)

Returns

List of bundle or bus members

getNetType

Syntax

```
getNetType(  
    name: str,  
    /,  
    *,  
    netlister: nlNetlister,  
    context: heContext)  
Returns: str
```

Summary:

Returns netType property value for net, set directly or calculated based on lower hierarchy.

Category:

Netlist Tools

Licenses:

None

Parameters

- `name` – Net name.
- `netlister` – The netlister object.
- `context` – The current context.

Returns

Net type name to be printed

getParamNames

Syntax

```
getParamNames(  
    *,  
    netlist: nlNetlist,  
    master: heContext,  
    type: nl.getParamNames.Type = None)  
Returns: str
```

Summary:

Retreives a list of parameter names.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlist – The netlist used to find the list of applicable parameters.
- master – The module to use to determine the list of relevant parameter names.
- type – The type of the parameters to return. ({ all | design | inst | lineage | parent }
(Default is: 'all')

Returns

List of parameter names

getReferencedVariables

Syntax

```
getReferencedVariables(  
    name: str = None,  
    /,  
    *,  
    netlist: nlNetlist,  
    master: heContext,  
    type: nl.getReferencedVariables.Type = None)  
Returns: str
```

Summary:

Retrieves a list of the names of parameters referenced via parent/lineage PEL expressions for a given module.

Category:

Netlist Tools

Licenses:

None

Parameters

- **name** – A regular expression matched against the parameter/variable names. (Default is: `'*'`)
- **netlister** – The queried netlister object.
- **master** – The context containing the variable references.
- **type** – Type of tracked variable (`{ all | design | inst | lineage | parent }`) (Default is: `'all'`)

Returns

List of variable names referenced in netlisting of given heContext

getShortedNets

Syntax

```
getShortedNets (  
    *,  
    netlister: nlNetlister,  
    context: heContext,  
    **kwargs)  
Returns: str
```

Summary:

Get the list of shorted net names and their preferred equivalents due to patch cords, ignored/removed devices, and shorted instance pins.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The netlister object to query.
- context – The context to retrieve the shorted Nets for.

Returns

[list net1 preferred1 net2 preferred2 ...]

Return type

The list of shorted net names and their preferred equivalent net names in the following format

getTerms

Syntax

```
getTerms(  
    pattern: str = None,  
    /,  
    *,  
    netlister: nlNetlister,  
    context: heContext)  
Returns: nlTerm
```

Summary:

Retrieves an iterator over nl term list.

Category:

Netlist Tools

Licenses:

None

Parameters

- pattern – Pattern to match term name. (Default is: '*')
- netlister – The netlister object to query.
- context – The context to retrieve the list of nlTerms for.

Returns

List of nlTerms, in order matching that of nl::getInstTerms

getVirtualTerms

Syntax

```
getVirtualTerms(  
*,  
  netlister: nlNetlister,  
  context:  heContext)  
Returns: list[nlTerm]
```

This command is deprecated and scheduled for removal in a future release. Use instead 'nl::getTerms -filter "%isVirtual == 1"'. For migration assistance, see the Custom Infrastructure Tcl Reference Manual.

Summary:

Retrieves all virtual terminals of the given context.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The netlister object to query.
- context – The context to retrieve the list of nlTerms for.

Returns

Collection of found virtual terminals

getWireBitOrder

Syntax

```
getWireBitOrder(  
*,  
  wire: str,  
  netlister: nlNetlister,  
  module:  heContext)  
Returns: oaBitOrder
```

Summary:

Returns the bitOrder stored by calls to nl::setWireBitOrder

Category:

Netlist Tools

Licenses:

None

Parameters

- wire – The base name of the wire.
- netlister – The netlister to query.
- module – The context to query.

Returns

The bit order of the given wire name.

getWorkerNetlisters

Syntax

```
getWorkerNetlisters(  
*,  
    netlister: nlAMSNetlister,  
    view: str = None)  
Returns: list[nlNetlister]
```

Summary:

Returns the list of child netlisters for the given AMS netlister.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The netlister to use for the given view.
- view – The view name.

Returns

Collection of worker netlisters associated with given AMS netlister

indent

Syntax

```
indent(  
    script: str,  
    /*  
    *,  
    netlister: nlNetlister)  
Returns: None
```

Summary:

Changes the left indent margin of the netlister's output buffer by the number of spaces stored in the netlister's -indentDepth attribute.

Category:

Netlist Tools

Licenses:

None

Parameters

- script – The script to execute.
- netlister – The nlNetlister object

Returns

None

instHeaderHandler

Syntax

```
instHeaderHandler(  
    netlister: nlNetlister,  
    context: heContext,  
    header: heInstHeader)  
Returns: None
```

Summary:

Default implementation instance header event

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The netlister object.
- context – The module being netlisted.
- header – The instance header being processed.

Returns

None

isLeaf

Syntax

```
isLeaf(  
*,  
    netlister: nlNetlister,  
    master:   heContext,  
    domain:   nl.isLeaf.Domain = None)  
Returns: bool
```

Summary:

Returns true if the given context is leaf in the given hierarchy configuration.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The netlister object.
- master – The switched master of the given instance.
- domain – Optionally check to see if the master is bound to the given domain. This check is only applied in AMS netlisting. ({ analog | any | digital } (Default is: 'any'))

Returns

boolean

isModuleUpToDate

Syntax

```
isModuleUpToDate(  
    netlister: nlNetlister,  
    context: heContext)  
Returns: bool
```

Summary:

Default implementation for the isModuleUpToDate event.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The netlister object.
- context – The module object.

Returns

True if the module is up to date, otherwise false

isNetNameGlobal

Syntax

```
isNetNameGlobal(  
    net_name: str,  
    /,  
    *,  
    netlister: nlNetlister)  
Returns: bool
```

Summary:

Determines if the given net is global

Category:

Netlist Tools

Licenses:

None

Parameters

- `net_name` – The name of the net to check.
- `netlister` – The netlister object.

Returns

Is passed net name global anywhere in design hierarchy

isStringParam

Syntax

```
isStringParam(  
    param_name: str,  
    /,  
    *,  
    netlister: nlNetlister,  
    master: heContext)  
Returns: bool
```

Summary:

The netlister determine if the given parameter name is a string parameter.

Category:

Netlist Tools

Licenses:

None

Parameters

- `param_name` – The name of the parameter to lookup.
- `netlister` – The netlister used to track string parameters.
- `master` – The master used to determine if the named parameter is a string parameter.

Returns

If the given parameter is a string parameter or is used as input to a string parameter within the passed in master's design hierarchy.

isTextModule

Syntax

```
isTextModule(  
    context: heContext)  
Returns: bool
```

Summary:

Determines if the passed in module is a text module.

Category:

Netlist Tools

Licenses:

None

Parameters

context – The inspected module.

Returns

True if passed in module is a text module, false otherwise

lookup

Syntax

```
lookup(  
    *,  
    map: nlNameMap | nlOANameMap | nlParameterizedNameMap,  
    src: str = None,  
    dest: str = None,  
    scope: str = None,  
    module: heContext = None)  
Returns: None
```

Category:

Summary:

Query the name map object tables to determine if a mapping is stored for the passed-in identifier.

Licenses:

None

Parameters

- map – Name map object to be queried or modified
- src – Database identifier to process
- dest – Destination identifier to be reverse processed
- scope – Scope of identifier
- module – Context specifying the module level tables

Returns

None

map

Syntax

```
map(  
*,  
    map: nlNameMap | nlOANameMap | nlParameterizedNameMap,  
    src: str = None,  
    dest: str = None,  
    scope: str = None,  
    module: heContext = None)  
Returns: None
```

Category:

Summary:

Maps identifiers between namespaces

Licenses:

None

Parameters

- map – Name map object to be queried or modified
- src – Database identifier to be mapped
- dest – Destination identifier to be reverse mapped
- scope – Scope of identifier
- module – Context specifying the module level tables

Returns

None

mapPath

Syntax

```
mapPath(  
*,  
    src: str = None,  
    dest: str = None,  
    scope: str = None,  
    map: nlHierMap,  
    **kwargs)  
Returns: str
```

Category:

Summary:

Returns the mapped hierarchical path

Licenses:

None

Parameters

- src – Database identifier to process
- dest – Destination identifier to be reverse processed
- scope – Scope of identifier
- map – Hierarchical name map object to be queried

Returns

Mapped hierarchical path

mapTerminalNames

Syntax

```
mapTerminalNames(  
*,  
    netlister: nlAMSNetlister | nlNetlister)  
Returns: None
```

Summary:

Maps all terminal names in all modules.

Category:

Netlist Tools

Licenses:

None

Parameters

netlister – Netlister object

Returns

None

message**Syntax**

```
message(  
    message: str,  
    /,  
    *,  
    netlister: nlNetlister,  
    severity: nl.message.Severity = None)  
Returns: None
```

Summary:

Logs an error, warning and information messages.

Category:

Netlist Tools

Licenses:

None

Parameters

- message – The message to be reported.
- netlister – The netlister object used to log messages.
- severity – The severity of the message, error is default. ({ abort | error | info | warning }
(Default is: 'error'))

Returns

None

moduleBodyHandler

Syntax

```
moduleBodyHandler(  
    netlister: nlNetlister,  
    context: heContext)  
Returns: None
```

Summary:

Default implementation for the moduleBodyHandler event.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The netlister object.
- context – The module being netlisted.

Returns

None

moduleHandler

Syntax

```
moduleHandler(  
    netlister: nlNetlister,  
    context: heContext)  
Returns: None
```

Summary:

Default implementation for the moduleHandler event.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The netlister object.
- context – The module to netlist.

Returns

None

modulePreprocessHandler

Syntax

```
modulePreprocessHandler(  
    netlister: nlNetlister,  
    context: heContext)  
Returns: None
```

Summary:

Default implementation for the modulePreprocess event.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The netlister object.
- context – The module object.

Returns

None

netlistBeginHandler

Syntax

```
netlistBeginHandler(  
    netlister: nlNetlister)  
Returns: None
```

Summary:

The default handler for the netlistBegin event.

Category:

Netlist Tools

Licenses:

None

Parameters

netlister – The netlister object

Returns

None

netlistBodyHandler

Syntax

```
netlistBodyHandler(  
    netlister: nlNetlister)  
Returns: None
```

Summary:

The default handler for the netlistBody event.

Category:

Netlist Tools

Licenses:

None

Parameters

netlister – The netlister queried.

Returns

None

netlistEndHandler

Syntax

```
netlistEndHandler(  
    netlister: nlNetlister)  
Returns: None
```

Summary:

The default handler for the netlistEnd event.

Category:

Netlist Tools

Licenses:

None

Parameters

netlister – The netlister object.

Returns

None

netlistMainHandler

Syntax

```
netlistMainHandler(  
    netlister: nlNetlister)  
Returns: None
```

Summary:

The default handler for the NetlistMain event.

Category:

Netlist Tools

Licenses:

None

Parameters

netlister – The netlister used to generate events.

Returns

None

openNameMap

Syntax

```
openNameMap (  
    *,  
    filePath: str)  
Returns: nlNameMap
```

Category:

Summary:

Loads the name map files into memory

Licenses:

None

Parameters

filePath – Directory path where name map files are stored

Returns

New name map object with the contents of the directory loaded

preloadModuleNames

Syntax

```
preloadModuleNames (  
    netlister: nlNetlister)  
Returns: None
```

Summary:

Insures that text views are not name mapped.

Category:

Netlist Tools

Licenses:

None

Parameters

netlister – The netlister scanned for text views.

Returns

None

printSpectreInst

Syntax

```
printSpectreInst(  
*,  
    netlister: nlNetlister,  
    inst: oaInst,  
    master: heContext,  
    format: str = None,  
    subParam: str = None,  
    defaultParams: bool = None,  
    filterParamsByCDFUse: bool = None,  
    returnString: bool = None)  
Returns: str
```

Category:

Summary:

Composes the instance line for Spectre netlister.

Licenses:

None

Parameters

- netlister – The netlister object.
- inst – The netlisted instance.
- master – The master of the given instance.
- format – The format of instance line.
- subParam – Subparams list.
- defaultParams – Write default params. (Default is: True)
- filterParamsByCDFUse – If true, do not print the CDF parameters which 'use' attribute is false. If false, ignore the 'use' value.
- returnString – Compose return string instead of printing. (Default is: False)

Returns

Contents of the printed string

printSpiceInst

Syntax

```
printSpiceInst(  
*,  
  netlist: nlNetlist,  
  inst: oaInst,  
  master: heContext,  
  format: str = None,  
  subParam: str = None,  
  defaultParams: bool = None,  
  filterParamsByCDFUse: bool = None,  
  returnString: bool = None,  
  wrapQuotesForParamValue: bool = None)  
Returns: str
```

Category:

Summary:

Determines the Spice instance line.

Licenses:

None

Parameters

- netlist – The netlist object.
- inst – The netlisted instance.
- master – The master of the given instance.
- format – The format of instance line.
- subParam – Subparams list.
- defaultParams – Write default params. (Default is: True)
- filterParamsByCDFUse – If true, do not print the CDF parameters which 'use' attribute is false. If false, ignore the 'use' value.
- returnString – Compose return string instead of printing. (Default is: False)
- wrapQuotesForParamValue – Wrap parameter value with single quotes (") when printing the HSPICE netlist result. (Default is: True)

Returns

Contents of the printed string

printSpiceInstTerms

Syntax

```
printSpiceInstTerms(  
*,  
    netlist:  nlNetlist,  
    inst:     oaInst,  
    master:   heContext,  
    maxNumber: int = None)  
Returns: None
```

Category:

Summary:

Prints the instance terminals connections notation as a part of the SPICE-style instance line.

Licenses:

None

Parameters

- netlist – The netlist object.
- inst – The netlisted instance.
- master – The switched context of the given instance.
- maxNumber – Maximal number of printed connection. Value '-1' means no limitation. (Default is: -1)

Returns

None

refreshPartitioner

Syntax

```
refreshPartitioner(  
    name: nlPartitioner)  
Returns: None
```

Summary:

Recalculated partitioning within the partitioner object

Category:

Netlist Tools

Licenses:

None

Parameters

name – partitioner to be updated.

Returns

None

revertOutdatedDesigns

Syntax

```
revertOutdatedDesigns()  
Returns: bool
```

Arguments:

Summary:

Reverts read-only designs that are newer on disk than in memory.

Category:

Netlist Tools

Licenses:

None

Returns

Returns true if Ok is clicked. False if Cancel is clicked.

runNetlister

Syntax

```
runNetlister(  
    netlister: nlAMSNetlister | nlNetlister,  
    /,
```

```
*,  
cellView: dmCellView | dmCellViewName,  
filePath: str,  
viewSearchList: list[str] = None,  
viewStopList: list[str] = None,  
reference: str = None,  
writeNameMapping: bool = None)  
Returns: bool
```

Summary:

Runs netlisting on a cellview.

Category:

Netlist Tools

Licenses:

Foundation

Parameters

- netlister – Netlister Object
- cellView – Cellview to netlist
- filePath – Main output file name
- viewSearchList – View Search List
- viewStopList – View Stop List
- reference – Reference path for data to be used for incremental netlisting
- writeNameMapping – Save name mapping data to the target directory (Default is: True)

Returns

Completion status of netlisting job - true if complete

saveModuleState

Syntax

```
saveModuleState(  
netlister: nlNetlister,  
context: heContext)  
Returns: bool
```

Summary:

Default implementation for the saveModuleState event.

Category:

Netlist Tools

Licenses:

None

Parameters

- netlister – The netlister object.
- context – The module object.

Returns

True if the data saving passed successfully, otherwise false

saveNameMap

Syntax

```
saveNameMap (  
map:  nlNameMap | nlOANameMap | nlParameterizedNameMap,  
/,  
*,  
filePath: str)  
Returns: None
```

Category:

Summary:

Writes a name map object to disk

Licenses:

None

Parameters

- map – Name map object to be queried or modified
- filePath – File path of directory for saved name map files

Returns

None

setTermOrder

Syntax

```
setTermOrder(  
termNames: str | list[nlTerm],  
/,  
*,  
netlister: nlNetlister,  
of: heContext,  
appendVirtualTerminals: bool = None,  
onlyReorder: bool = None)  
Returns: None
```

Summary:

Assign term order for specified module.

Category:

Netlist Tools

Licenses:

None

Parameters

- termNames – The ordered terminals or terminal names.
- netlister – The netlister instance.
- of – The context for the specified terminal order.
- appendVirtualTerminals – Append the default virtual terminals to the specified terminal order. (Default is: False)
- onlyReorder – If true, the existing term order is re-ordered according to the terminal names. Used only with the string input of terminal names. (Default is: False)

Returns

None

setWireBitOrder

Syntax

```
setWireBitOrder(  
oaBitOrder: oaBitOrder,  
/,  
*,
```

```
wire: str,  
netlister: nlNetlister,  
module: heContext)  
Returns: None
```

Summary:

Sets the wire bit order in an internal table with the netlister

Category:

Netlist Tools

Licenses:

None

Parameters

- oaBitOrder – The new bit order of the given wire name.
- wire – The base name of the wire.
- netlister – The netlister to modify.
- module – The module where the wire name exists.

Returns

None

uniqueify

Syntax

```
uniqueify(  
identifier: str,  
/,  
*,  
index: int)  
Returns: str
```

Category:

Summary:

Calculates alternate names for identifiers

Licenses:

None

Parameters

- identifier – Identifier to modify
- index – Index to use to modify identifier

Returns

Unique string

snps.custom.cmd.po

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules provide a fast and easy geometry manipulation capability directly in Custom Compiler, without the need to write a runset or execute physical verification tools.

The Python po module allows you to do complex geometry operations using a daisy-chain method. For details, see:

- [and_](#)
- [angle](#)
- [append](#)
- [area](#)
- [chop](#)
- [coinEdge](#)
- [compress](#)
- [convexEdge](#)
- [createCollection](#)
- [createPointSet](#)
- [cut](#)
- [cutCorner](#)
- [density](#)
- [donut](#)
- [enclose](#)

- [encloseRect](#)
- [enclosure](#)
- [expandEdge](#)
- [extent](#)
- [extents](#)
- [external](#)
- [getAngle](#)
- [getArea](#)
- [getContours](#)
- [getDistance](#)
- [getHoles](#)
- [getIntersection](#)
- [getMaxBox](#)
- [getPerimeter](#)
- [getPoints](#)
- [getSpecifiedBoxes](#)
- [inside](#)
- [insideEdge](#)
- [interact](#)
- [internal](#)
- [isClockwise](#)
- [isHorizontal](#)
- [isVertical](#)
- [length](#)
- [makeClockwise](#)
- [manhattanContour](#)
- [not_](#)

- [offGrid](#)
- [or_](#)
- [outside](#)
- [outsideEdge](#)
- [pathLength](#)
- [perimeter](#)
- [polyToTrap](#)
- [rectangle](#)
- [rectangles](#)
- [rectEnclosure](#)
- [shapePoints](#)
- [shift](#)
- [size](#)
- [snapGrid](#)
- [straddle](#)
- [touch](#)
- [touchEdge](#)
- [transform](#)
- [vertex](#)
- [withEdge](#)
- [withNeighbor](#)
- [withWidth](#)
- [xor](#)

and_

Syntax

```
and_(  
input: dbCollection | poPointSet,  
/,
```

```
*,  
  with_: dbCollection | poPointSet)  
Returns: dbCollection | poPointSet
```

Summary:

Run the Boolean AND operation.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify the operand1 points container.
- with_ – Specify the operand2 points container.

Returns

The result points container

angle

Syntax

```
angle(  
  input: dbCollection,  
  /,  
  *,  
  value: str,  
  not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all edges from input-collection having a smaller angle magnitude with respect to the x-axis of the coordinate system, which conform to the constraint specified by “-value”.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify input-collection.
- `value` – Specify the small angle magnitude constraint interpreted in degrees. (e.g. `==90`)
- `not_` – “-not true” is the complement of the angle operation. (Default is: `False`)

Returns

The result points container

append

Syntax

```
append(  
input: list[oaShape | point | list[deFigure]],  
/  
*,  
to: dbCollection | poPointSet,  
dbu: int = None,  
dbuPerUU: int = None)  
Returns: None
```

Summary:

Append a polygon/polyline or polygons into a polygons/polylines container.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify the input objects.
- `to` – Specify the polygons/polylines container.
- `dbu` – Specify `dbuPerUU` for points input. (Default is: 1000)
- `dbuPerUU` – Specify `dbuPerUU` for points input. (Default is: 1000)

Returns

None

area

Syntax

```
area(  
  input: dbCollection,  
  /,  
  *,  
  value: str,  
  not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all merged-polygons in input-collection having areas that conform to the constraint specified by “-value”.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- value – Specify the constraint representing the area of merged polygons in user units squared. (e.g. >=2)
- not_ – “-not true” is the complement of the area operation. (Default is: False)

Returns

The result points container

chop

Syntax

```
chop(  
  input: dbCollection | poPointSet,  
  /,  
  *,  
  by: box)  
Returns: dbCollection | poPointSet
```

Summary:

Chop polylines by a box.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify the polyline points container.
- by – Specify the box to chop polylines

Returns

The result points container

coinEdge

Syntax

```
coinEdge(  
input: dbCollection,  
/,  
*,  
with_: dbCollection,  
dir: po.coinEdge.Dir = None,  
not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all edges or edge portions from input-collection coincident with edges of with-collection.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- with_ – Specify with-collection.
- dir – Specify the touch direction. ({ all | inside | outside } (Default is: 'all'))
- not_ – “-not true” is the complement of the coinEdge operation. (Default is: False)

Returns

The result points container

compress

Syntax

```
compress(  
    points: list[point],  
    /,  
    *,  
    closed: bool = None,  
    dbuPerUU: int = None)  
Returns: point
```

Summary:

Remove all collinear and coincident points.

Category:

Foundation

Licenses:

None

Parameters

- points – The list of points that form compressing points.
- closed – Specify the input points is closed or not. (Default is: True)
- dbuPerUU – Specify dbuPerUU for points input. (Default is: 10000)

Returns

The compressed points

convexEdge

Syntax

```
convexEdge(  
    input: dbCollection,  
    /,  
    *,  
    value: str = None,  
    angle1: str = None,  
    length1: str = None,
```



```
angle2: str = None,  
length2: str = None,  
withLength: str = None)  
Returns: dbCollection
```

Summary:

Select edges from input-collection based upon the number of convex endpoints the edges have.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- value – Specify the constraint representing the number of endpoints (0, 1, or 2), which must lie at a convex corner for an edge to be selected. (e.g. ==1)
- angle1 – Specify the angle constraint of one endpoint of edges. (e.g. ==90)
- length1 – Cooperate with “-angle1” and be specified in user units. Specify the length constraint of edges abutting the endpoint satisfying “-angle1”. (e.g. <=0.1)
- angle2 – Cooperate with “-angle1” and specify the angle constraint of the other endpoint of edges. (e.g. ==270)
- length2 – Cooperate with “-angle2” and be specified in user units. Specify the length constraint of edges abutting the endpoint satisfying “-angle2”. (e.g. <=0.1)
- withLength – Specify the length constraint in user units to limit the output edges. (e.g. ==0.05)

Returns

The result points container

createCollection

Syntax

```
createCollection(  
input: list[oaShape | point | list[deFigure] | list[oaShape]] =  
None,  
/,  
*,
```

```
dbu: int = None,  
dbuPerUU: int = None,  
closed: bool = None)  
Returns: dbCollection
```

Summary:

Create a container to contain multiple one-shot polygons or polylines.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify the input points, objects or collections.
- dbu – Specify dbuPerUU for points input. (Default is: 1000)
- dbuPerUU – Specify dbuPerUU for points input. (Default is: 1000)
- closed – Indicate the container containing multiple polygons points or multiple polylines points. (Default is: True)

Returns

The points collection

createPointSet

Syntax

```
createPointSet(  
input: list[oaShape | point] = None,  
/,  
*,  
dbu: int = None,  
dbuPerUU: int = None,  
closed: bool = None)  
Returns: poPointSet
```

Summary:

Create a points set to contain multiple one-shot polygons.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify the input oaShapes or points.
- dbu – Specify dbuPerUU for points input. (Default is: 1000)
- dbuPerUU – Specify dbuPerUU for points input. (Default is: 1000)
- closed – Indicate the poPointSet containing multiple polygons points or multiple polylines points. (Default is: True)

Returns

The points set

cut

Syntax

```
cut(  
input: dbCollection,  
/,  
*,  
with_: dbCollection,  
value: str = None,  
hasEvenOdd: po.cut.HasEvenOdd = None,  
not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all polygons from input-collection that share some (but not all) of their area with polygons of with-collection.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- with_ – Specify with-collection.

- `value` – Specify the selection criterion. (e.g. `>=2`)
- `hasEvenOdd` – Cooperate with “-value” to specify the extra selection criterion. (`{ even | none | odd }`) (Default is: ‘none’)
- `not_` – “-not true” is the complement of the cut operation. (Default is: False)

Returns

The result points container

cutCorner

Syntax

```
cutCorner(  
input: dbCollection | poPointSet,  
/,  
*,  
cutLength: float = None,  
numSides: int = None,  
selected: box = None,  
angleRange: str = None)  
Returns: dbCollection | poPointSet
```

Summary:

Cut the selected corners of polygons based on cut-length and number-of-sides.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify the operand1 points container.
- `cutLength` – Specify the cut Length for a corner.
- `numSides` – Specify the number of sides after cut a corner. (Default is: 1)
- `selected` – Specify the bounding box to determine the selected corners.
- `angleRange` – Specify the range of interior angles (<180 degree) to do cutCorner.

Returns

The result points container

density

Syntax

```
density(  
  input: dbCollection,  
  /,  
  *,  
  value: str,  
  insideOf: box | dbCollection = None,  
  design: oaDesign = None,  
  window: point = None,  
  step: point = None,  
  slideMode: po.density.SlideMode = None,  
  gradient: str = None,  
  gradientMode: po.density.GradientMode = None,  
  corner: bool = None,  
  outputAll: bool = None)  
Returns: dbCollection
```

Summary:

Output the data capture windows whose area ratio (density) meets the constraint specified by “-value”.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- value – Specify the density (area ratio) constraint. (e.g. >0.5)
- insideOf – If specify <box>, the box is the data capture extent. If specify <dbCollection>, the extents of polygons in <dbCollection> are the data capture extents.
- design – If specify <oaDesign>, the bounding box of <oaDesign> is the data capture extent.
- window – Specify the dimensions of the data capture window within the data capture extent.
- step – Specify the grid increment for which data capture windows are tiled within the data capture extent.

- **slideMode** – Cooperate with “-window” and “-step” to define how to handle data capture windows overlapping the right/top edges of data capture extents. ({ backup | ignore | truncate } (Default is: ‘backup’)
- **gradient** – Specify the gradient constraint representing the density value change from one window to adjacent nonoverlapping windows. (e.g. >0.3)
- **gradientMode** – Specify how to calculate the gradient of the data capture window. ({ absolute | relative } (Default is: ‘relative’)
- **corner** – If specify “-corner true”, the set of windows in the gradient calculation is extended to include the four windows touching current window on each corner. (Default is: False)
- **outputAll** – If specify “-outputAll true”, output all data capture windows. (Default is: False)

Returns

The result points container

donut

Syntax

```
donut(  
  input: dbCollection,  
  /,  
  *,  
  value: str = None,  
  not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all polygons from input-collection having interior cycles (holes) that are fully enclosed within a polygon.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify input-collection.
- `value` – Specify the specific number of interior cycles for a polygon. (e.g. ≥ 2)
- `not_` – “-not true” is the complement of the donut operation. (Default is: False)

Returns

The result points container

enclose

Syntax

```
enclose(  
input: dbCollection,  
/,  
*,  
with_: dbCollection,  
value: str = None,  
hasEvenOdd: po.enclose.HasEvenOdd = None,  
not_: bool = None)  
Returns: dbCollection
```

Summary:

Select polygons from input-collection containing any polygons of with-collection.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify input-collection.
- `with_` – Specify with-collection.
- `value` – Specify the selection criterion. (e.g. ≥ 2)
- `hasEvenOdd` – Cooperate with “-value” to specify the extra selection criterion. ({ even | none | odd }) (Default is: ‘none’)
- `not_` – “-not true” is the complement of the enclose operation. (Default is: False)

Returns

The result points container

encloseRect

Syntax

```
encloseRect(  
  input: dbCollection,  
  /,  
  *,  
  width: float,  
  length: float,  
  isOrthogonalOnly: bool = None,  
  not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all polygons from input-collection that can enclose an orthogonal rectangle of the specified width and length.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- width – Specify the width of the enclosed orthogonal rectangle in user units.
- length – Specify the length of the enclosed orthogonal rectangle in user units.
- isOrthogonalOnly – “-isOrthogonalOnly false” would select polygons containing orthogonal/diagonal rectangles. (Default is: True)
- not_ – “-not true” is the complement of the encloseRect operation. (Default is: False)

Returns

The result points container

enclosure

Syntax

```
enclosure(  
  input: dbCollection,  
  /,  
  *,  
  with_: dbCollection,  
  value: str,  
  metric: po.enclosure.Metric = None,  
  oppositeExt: float = None,  
  acute: po.enclosure.Acute = None,  
  parallel: po.enclosure.Parallel = None,  
  perpendicular: po.enclosure.Perpendicular = None,  
  obtuse: po.enclosure.Obtuse = None,  
  projectionFilter: po.enclosure.ProjectionFilter = None,  
  projLength: str = None,  
  angled: str = None,  
  cornerFilter: po.enclosure.CornerFilter = None,  
  abut: str = None,  
  singular: bool = None,  
  intersectingOnly: bool = None,  
  measureAll: bool = None,  
  outputType: po.enclosure.OutputType = None)  
Returns: dbCollection
```

Summary:

Measure the separation between the exterior-facing sides of edges in input-collection and the interior-facing sides of edges in with-collection.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- with_ – Specify with-collection.
- value – Specify the measuring constraint in user units. (e.g. <0.05)
- metric – Measure edge pairs based on the specific metric. ({ euclidean | opposite | square } (Default is: 'euclidean'))

- `oppositeExt` – Cooperate with “-metric opposite”. A non-negative number in user units specifies the amount to extend both ends of an edge. (Default is: 0)
- `acute` – Measure edge pairs based on the acute condition. ({ also | none | only } (Default is: ‘also’)
- `parallel` – Measure edge pairs based on the parallel condition. ({ also | none | only } (Default is: ‘also’)
- `perpendicular` – Measure edge pairs based on the perpendicular condition. ({ also | none | only } (Default is: ‘none’)
- `obtuse` – Measure edge pairs based on the obtuse condition. ({ also | none | only } (Default is: ‘none’)
- `projectionFilter` – Specify the condition of the mutual edge projection. ({ all | notProj | proj } (Default is: ‘all’)
- `projLength` – Cooperate with “-projectionFilter proj”. Specify the projection length constraint and the constraint value is a non-negative number in user units. (e.g. >0) (Default is: ‘>=0’)
- `angled` – Specify to measure the two edges only when the number of non-orthogonal edges in the pair meets the given constraint. (e.g. ==0)
- `cornerFilter` – Measure edge pairs based on corner-to-corner or corner-to-edge orientation. ({ all | corner | cornerToCorner | cornerToEdge | notCorner } (Default is: ‘all’)
- `abut` – Specify that separation between intersecting edges should also be checked under the specified constraint. (e.g. <90)
- `singular` – Handle polygon singularities that are point-to-edge or point-to-point polygon intersections. (Default is: False)
- `intersectingOnly` – Cooperate with “-abut” or “-singular”. Specify to measure only between intersecting edges and to ignore non-intersecting edges. (Default is: False)
- `measureAll` – By default, consider polygon containment criteria. If specify “-measureAll true”, ignore polygon containment criteria. (Default is: False)
- `outputType` – Specify the output type. ({ negEdgeA | negEdgeB | posEdgeA | posEdgeB | region } (Default is: ‘region’)

Returns

The result points container

expandEdge

Syntax

```
expandEdge (  
  input: dbCollection,  
  /,  
  *,  
  by: float,  
  dir: po.expandEdge.Dir = None,  
  extendBy: float = None,  
  factor: bool = None,  
  cornerFill: bool = None)  
Returns: dbCollection
```

Summary:

Expand edges of polygons from input-collection into rectangles in the specified direction and amounts.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- by – Specify the expanding value in user units.
- dir – Specify the expanding direction. ({ all | inside | outside } (Default is: 'all'))
- extendBy – Specify the amount to extend/retract the length of edges from input-collection prior to expanding them into rectangles. (Default is: 0)
- factor – If specify false, the values in -by and -extendBy are in user units. (Default is: False)
- cornerFill – If specify true, fill gaps between rectangles formed by the operation. (Default is: False)

Returns

The result points container

extent

Syntax

```
extent(  
  input: dbCollection,  
  /,  
  *,  
  outputBox: bool = None)  
Returns: dbCollection | oaBox
```

Summary:

Generate a bounding box that represents the minimum bounding box containing all polygons/polylines of input-collection.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- outputBox – By default, output points container. If specify “-outputBox true”, output oa::Box. (Default is: False)

Returns

The result points container or oa::Box

extents

Syntax

```
extents(  
  input: dbCollection)  
Returns: dbCollection
```

Summary:

For each merged polygon in input-collection, create the minimum bounding boxes. Merge the minimum bounding boxes to generate the operation results.

Category:

Foundation

Licenses:

None

Parameters

input – Specify input-collection.

Returns

The result points container

external

Syntax

```
external(  
input: dbCollection,  
/,  
*,  
with_: dbCollection = None,  
value: str,  
metric: po.external.Metric = None,  
oppositeExt: float = None,  
acute: po.external.Acute = None,  
parallel: po.external.Parallel = None,  
perpendicular: po.external.Perpendicular = None,  
obtuse: po.external.Obtuse = None,  
projectionFilter: po.external.ProjectionFilter = None,  
projLength: str = None,  
angled: str = None,  
cornerFilter: po.external.CornerFilter = None,  
abut: str = None,  
singular: bool = None,  
intersectingOnly: bool = None,  
measureAll: bool = None,  
outputType: po.external.OutputType = None)  
Returns: dbCollection
```

Summary:

Measure the separations between the exterior-facing sides of edges. po::external is principally used for spacing checks.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify input-collection.
- `with_` – Specify with-collection.
- `value` – Specify the measuring constraint in user units. (e.g. <0.05)
- `metric` – Measure edge pairs based on the specific metric. ({ `euclidean` | `opposite` | `square` }) (Default is: 'euclidean')
- `oppositeExt` – Cooperate with “-metric opposite”. A non-negative number in user units specifies the amount to extend both ends of an edge. (Default is: 0)
- `acute` – Measure edge pairs based on the acute condition. ({ `also` | `none` | `only` }) (Default is: 'also')
- `parallel` – Measure edge pairs based on the parallel condition. ({ `also` | `none` | `only` }) (Default is: 'also')
- `perpendicular` – Measure edge pairs based on the perpendicular condition. ({ `also` | `none` | `only` }) (Default is: 'none')
- `obtuse` – Measure edge pairs based on the obtuse condition. ({ `also` | `none` | `only` }) (Default is: 'none')
- `projectionFilter` – Specify the condition of the mutual edge projection. ({ `all` | `notProj` | `proj` }) (Default is: 'all')
- `projLength` – Cooperate with “-projectionFilter proj”. Specify the projection length constraint and the constraint value is a non-negative number in user units. (e.g. >0) (Default is: ' ≥ 0 ')
- `angled` – Specify to measure the two edges only when the number of non-orthogonal edges in the pair meets the given constraint. (e.g. $=0$)
- `cornerFilter` – Measure edge pairs based on corner-to-corner or corner-to-edge orientation. ({ `all` | `corner` | `cornerToCorner` | `cornerToEdge` | `notCorner` }) (Default is: 'all')
- `abut` – Specify that separation between intersecting edges should also be checked under the specified constraint. (e.g. <90)
- `singular` – Handle polygon singularities that are point-to-edge or point-to-point polygon intersections. (Default is: False)
- `intersectingOnly` – Cooperate with “-abut” or “-singular”. Specify to measure only between intersecting edges and to ignore non-intersecting edges. (Default is: False)

- **measureAll** – By default, consider polygon containment criteria. If specify “-measureAll true”, ignore polygon containment criteria. (Default is: False)
- **outputType** – Specify the output type. ({ negEdgeA | negEdgeB | posEdgeA | posEdgeB | region } (Default is: ‘region’)

Returns

The result points container

getAngle

Syntax

```
getAngle(  
input:  list[point],  
/,  
*,  
with_: list[point],  
obtuse: bool = None)  
Returns: float
```

Summary:

Get the smaller angle degrees of the two edges.

Category:

Foundation

Licenses:

None

Parameters

- **input** – The points represent one edge. This list only has two points.
- **with_** – The points represent the other edge. This list only has two points.
- **obtuse** – If specify “-obtuse true”, get the larger angle magnitude (>90 and <180) of the two edges. (Default is: False)

Returns

The smaller angle degrees of the two edges.

getArea

Syntax

```
getArea(  
  input: dbCollection | poPointSet  
Returns: float
```

Summary:

Get the total merged area of the input polygons.

Category:

Foundation

Licenses:

None

Parameters

input – Specify the input points container.

Returns

The total merged area of the input polygons.

getContours

Syntax

```
getContours(  
  input: dbCollection | poPointSet  
Returns: dbCollection | poPointSet
```

Summary:

Merge the input polygons and get the contours.

Category:

Foundation

Licenses:

None

Parameters

input – Specify the operand1 points container.

Returns

The result points container

getDistance

Syntax

```
getDistance(  
  points: list[point],  
  /,  
  *,  
  to: list[point])  
Returns: float
```

Summary:

Get the Euclidean distance of the two things.

Category:

Foundation

Licenses:

None

Parameters

- points – The list of points that form getting distance points.
- to – The other points to measure the distance.

Returns

The Euclidean distance between the two input points.

getHoles

Syntax

```
getHoles(  
  input: dbCollection | poPointSet)  
Returns: dbCollection | poPointSet
```

Summary:

Merge the input polygons and get the holes.

Category:

Foundation

Licenses:

None

Parameters

input – Specify the operand1 points container.

Returns

The result points container

getIntersection

Syntax

```
getIntersection(  
    points: list[point],  
    /,  
    *,  
    with_: list[point],  
    onSegment: bool = None)  
Returns: point
```

Summary:

Get the intersection of the two lines or two segments.

Category:

Foundation

Licenses:

None

Parameters

- points – The list of points that form line points.
- with_ – The other line/segment points.
- onSegment – Calculate the intersection of the two segments. (Default is: True)

Returns

The intersection of the two lines or two segments.

getMaxBox

Syntax

```
getMaxBox(  
    input: dbCollection,  
    /,  
    *,  
    anchorPoint: point = None)  
Returns: dbCollection
```

Summary:

Get the maximum area enclosed box.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- anchorPoint – Specify the anchor point in user units.

Returns

The result points container

getPerimeter

Syntax

```
getPerimeter(  
    input: dbCollection | poPointSet)  
Returns: float
```

Summary:

Get the total merged perimeter of the input polygons.

Category:

Foundation

Licenses:

None

Parameters

input – Specify the input points container.

Returns

The total merged perimeter of the input polygons.

getPoints

Syntax

```
getPoints(  
input:  poPointSet,  
/,  
*,  
reset: bool = None)  
Returns: point
```

Summary:

Get the points(in user units) of the points set iteratively.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify the points set.
- reset – Reset the iterator of points set. (Default is: False)

Returns

The points(in user units) in the points set

getSpecifiedBoxes

Syntax

```
getSpecifiedBoxes(  
input:  dbCollection,  
/,  
*,  
minWidth: float,  
minHeight: float,  
anchorPt: point,
```

```
maxWidth: float = None,  
maxHeight: float = None,  
opType: po.getSpecifiedBoxes.OpType = None)  
Returns: dbCollection
```

Summary:

Get the specified boxes which match the criteria based on a given anchor point.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- minWidth – Specify the minimum width of the specified box.
- minHeight – Specify the minimum height of the specified box.
- anchorPt – Specify the anchor point in user units.
- maxWidth – Specify the maximum width of the specified box. Only applied in opType = horizontal/vertical. (Default is: 0)
- maxHeight – Specify the maximum height of the specified box. Only applied in opType = horizontal/vertical. (Default is: 0)
- opType – Specify the operation type. ({ all | convexCorner | horizontal | vertical } (Default is: 'all'))

Returns

The result points container

inside

Syntax

```
inside(  
input: dbCollection,  
/,  
*,  
with_: dbCollection,  
not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all polygons from input-collection that share their entire areas with polygons of with-collection.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- with_ – Specify with-collection.
- not_ – “-not true” is the complement of the inside operation. (Default is: False)

Returns

The result points container

insideEdge

Syntax

```
insideEdge(  
input: dbCollection,  
/,  
*,  
with_: dbCollection,  
not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all edges or edge segments from input-collection that lie completely inside polygons of with-collection.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- with_ – Specify with-collection.
- not_ – “-not true” is the complement of the insideEdge operation. (Default is: False)

Returns

The result points container

interact

Syntax

```
interact(  
input: dbCollection,  
/,  
*,  
with_: dbCollection,  
value: str = None,  
hasEvenOdd: po.interact.HasEvenOdd = None,  
hasSingular: po.interact.HasSingular = None,  
not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all polygons from input-collection based on the type and number of intersections they have with polygons of with-collection.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- with_ – Specify with-collection.
- value – Specify the selection criterion. (e.g. >=3)
- hasEvenOdd – Cooperate with “-value” to specify the extra selection criterion. ({ even | none | odd }) (Default is: ‘none’)

- `hasSingular` – Indicate how to handle point-to-point and point-to-edge interactions. ({ `also` | `none` | `only` } (Default is: 'none'))
- `not_` – “-not true” is the complement of the interact operation. (Default is: False)

Returns

The result points container

internal

Syntax

```
internal(  
input: dbCollection,  
/,  
*,  
with_: dbCollection = None,  
value: str,  
metric: po.internal.Metric = None,  
oppositeExt: float = None,  
acute: po.internal.Acute = None,  
parallel: po.internal.Parallel = None,  
perpendicular: po.internal.Perpendicular = None,  
obtuse: po.internal.Obtuse = None,  
projectionFilter: po.internal.ProjectionFilter = None,  
projLength: str = None,  
angled: str = None,  
cornerFilter: po.internal.CornerFilter = None,  
abut: str = None,  
singular: bool = None,  
intersectingOnly: bool = None,  
measureAll: bool = None,  
outputType: po.internal.OutputType = None)  
Returns: dbCollection
```

Summary:

Measure the separations between the interior-facing sides of edges. `po::internal` is principally used for width and overlap checks.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify input-collection.
- `with_` – Specify with-collection.
- `value` – Specify the measuring constraint in user units. (e.g. <0.05)
- `metric` – Measure edge pairs based on the specific metric. ({ `euclidean` | `opposite` | `square` }) (Default is: `'euclidean'`)
- `oppositeExt` – Cooperate with “-metric opposite”. A non-negative number in user units specifies the amount to extend both ends of an edge. (Default is: 0)
- `acute` – Measure edge pairs based on the acute condition. ({ `also` | `none` | `only` }) (Default is: `'also'`)
- `parallel` – Measure edge pairs based on the parallel condition. ({ `also` | `none` | `only` }) (Default is: `'also'`)
- `perpendicular` – Measure edge pairs based on the perpendicular condition. ({ `also` | `none` | `only` }) (Default is: `'none'`)
- `obtuse` – Measure edge pairs based on the obtuse condition. ({ `also` | `none` | `only` }) (Default is: `'none'`)
- `projectionFilter` – Specify the condition of the mutual edge projection. ({ `all` | `notProj` | `proj` }) (Default is: `'all'`)
- `projLength` – Cooperate with “-projectionFilter proj”. Specify the projection length constraint and the constraint value is a non-negative number in user units. (e.g. >0) (Default is: `'>=0'`)
- `angled` – Specify to measure the two edges only when the number of non-orthogonal edges in the pair meets the given constraint. (e.g. $==0$)
- `cornerFilter` – Measure edge pairs based on corner-to-corner or corner-to-edge orientation. ({ `all` | `corner` | `cornerToCorner` | `cornerToEdge` | `notCorner` }) (Default is: `'all'`)
- `abut` – Specify that separation between intersecting edges should also be checked under the specified constraint. (e.g. <90)
- `singular` – Handle polygon singularities that are point-to-edge or point-to-point polygon intersections. (Default is: `False`)
- `intersectingOnly` – Cooperate with “-abut” or “-singular”. Specify to measure only between intersecting edges and to ignore non-intersecting edges. (Default is: `False`)

- **measureAll** – By default, consider polygon containment criteria. If specify “-measureAll true”, ignore polygon containment criteria. (Default is: False)
- **outputType** – Specify the output type. ({ negEdgeA | negEdgeB | posEdgeA | posEdgeB | region } (Default is: ‘region’)

Returns

The result points container

isClockwise

Syntax

```
isClockwise(  
points: list[point])  
Returns: bool
```

Summary:

Check the input points satisfying clockwise direction or not .

Category:

Foundation

Licenses:

None

Parameters

points – The list of points that form checking clockwise points.

Returns

Honor clockwise direction or not.

isHorizontal

Syntax

```
isHorizontal(  
points: list[point],  
/,  
*,  
dbuPerUU: int = None)  
Returns: bool
```

Summary:

Check the input points satisfying horizontal direction or not .

Category:

Foundation

Licenses:

None

Parameters

- points – The list of points that form checking horizontal points.
- dbuPerUU – Specify dbuPerUU for points input. (Default is: 1000)

Returns

Honor horizontal direction or not.

isVertical**Syntax**

```
isVertical(  
points: list[point],  
/,  
*,  
dbuPerUU: int = None)  
Returns: bool
```

Summary:

Check the input points satisfying vertical direction or not .

Category:

Foundation

Licenses:

None

Parameters

- points – The list of points that form checking vertical points.
- dbuPerUU – Specify dbuPerUU for points input. (Default is: 1000)

Returns

Honor vertical direction or not.

length

Syntax

```
length(  
  input: dbCollection,  
  /,  
  *,  
  value: str,  
  not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all edges from input-collection having length that conforms to the constraint specified by “-value”.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- value – Specify the length constraint in user units. (e.g. ≤ 0.2)
- not_ – “-not true” is the complement of the length operation. (Default is: False)

Returns

The result points container

makeClockwise

Syntax

```
makeClockwise(  
  points: list[point],  
  /,  
  *,  
  reverse: bool = None)  
Returns: point
```

Summary:

Guarantee the ordering of input points.

Category:

Foundation

Licenses:

None

Parameters

- `points` – The list of points that form making clockwise points.
- `reverse` – Guarantee the ordering of input points satisfying counter-clockwise. (Default is: `False`)

Returns

The points keep the specific ordering

manhattanContour

Syntax

```
manhattanContour(  
    input: dbCollection,  
    /,  
    *,  
    notchGap: float = None,  
    jogLen: float = None)  
Returns: dbCollection
```

Summary:

For orthogonal polygons, form a connected component and get its contour.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- notchGap – Specify the notchGap length in user units.
- jogLen – Specify the jog length in user units.

Returns

The result points container

not_

Syntax

```
not_(  
input: dbCollection | poPointSet,  
/,  
*,  
from_: dbCollection | poPointSet = None,  
of: dbCollection | poPointSet = None)  
Returns: dbCollection | poPointSet
```

Summary:

Run the Boolean NOT operation.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify the operand2 points container.
- of – Specify the operand1 points container.

Returns

The result points container

offGrid

Syntax

```
offGrid(  
input: dbCollection,
```

```
/,
*,
resolution: point,
offset: point = None,
edge: bool = None)
Returns: dbCollection
```

Summary:

Output off-grid polygons or edges based upon a user-specified grid in user units and parameter setting.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- resolution – Specify the grid resolution in user units. x-coordinate is applied in the x direction and y-coordinate is applied in the y-direction.
- offset – Specify an offset displacement to apply to the objects in input-collection before checking them against the grid resolution. (Default is: {0 0})
- edge – If its value is true, output the off-grid edges of objects in input-collection. (Default is: False)

Returns

The result points container

or_

Syntax

```
or_(
input: dbCollection | poPointSet,
/,
*,
with_: dbCollection | poPointSet = None)
Returns: dbCollection | poPointSet
```

Summary:

Run the Boolean OR operation.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify the operand1 points container.
- `with_` – Specify the operand2 points container.

Returns

The result points container

outside

Syntax

```
outside(  
input: dbCollection,  
/,  
*,  
with_: dbCollection,  
not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all polygons from input-collection with areas that lie completely outside all polygons of with-collection.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify input-collection.
- `with_` – Specify with-collection.
- `not_` – “-not true” is the complement of the outside operation. (Default is: False)

Returns

The result points container

outsideEdge

Syntax

```
outsideEdge(  
  input: dbCollection,  
  /,  
  *,  
  with_: dbCollection,  
  not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all edges or edge segments from input-collection that lie completely outside polygons of with-collection.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- with_ – Specify with-collection.
- not_ – “-not true” is the complement of the outsideEdge operation. (Default is: False)

Returns

The result points container

pathLength

Syntax

```
pathLength(  
  input: dbCollection,  
  /,  
  *,  
  value: str)  
Returns: dbCollection
```

Summary:

For each polyline in input-collection, select a maximum-length contiguous path of distinct edges satisfying the constraint specified by “-value”.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- value – Specify the constraint representing the total length of polylines in user units.
(e.g. <0.5)

Returns

The result points container

perimeter

Syntax

```
perimeter(  
  input: dbCollection,  
  /,  
  *,  
  value: str,  
  not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all merged-polygons in input collection with perimeters conforming to the constraint specified by “-value”.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- value – Specify the perimeter constraint for a merged-polygon. (e.g. ≥ 2)
- not_ – “-not true” is the complement of the perimeter operation. (Default is: False)

Returns

The result points container

polyToTrap

Syntax

```
polyToTrap(  
input: dbCollection | poPointSet,  
/,  
*,  
hSlicing: bool = None)  
Returns: dbCollection | poPointSet
```

Summary:

Split polygons into trapezoids by horizontal/vertical slicing.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify the operand1 points container.
- hSlicing – Slice polygons by horizontal direction or not. (Default is: True)

Returns

The result points container

rectangle

Syntax

```
rectangle(  
input: dbCollection,  
/,
```

```
*,
width: str = None,
length: str = None,
aspectRatio: str = None,
measure: po.rectangle.Measure = None,
not_: bool = None)
Returns: dbCollection
```

Summary:

Select rectangles from input-collection based on geometric properties.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- width – Specify the width criterion of selected rectangles in user units. (e.g. ≥ 0.05)
- length – Specify the length criterion of selected rectangles in user units. (e.g. ≥ 0.08)
- aspectRatio – Specify the aspect ratio constraint of the selected rectangles. (e.g. > 2)
- measure – Specify the measuring criterion. ({ extents | orthogonal } (Default is: 'orthogonal'))
- not_ – “-not true” is the complement of the rectangle operation. (Default is: False)

Returns

The result points container

rectangles

Syntax

```
rectangles (
*,
width: float,
length: float,
space: point,
insideOf: box | dbCollection,
offset: point = None,
maintainSpacing: bool = None,
dbuPerUU: int = None)
Returns: dbCollection
```

Summary:

Creates an array of rectangles of specified “–width”, “–length” and “–space” in user units.

Category:

Foundation

Licenses:

None

Parameters

- width – Specify the size of the created rectangles in x-coordinate direction in user units.
- length – Specify the size of the created rectangles in y-coordinate direction in user units.
- space – Specify the x/y spacing in user units between created rectangles.
- insideOf – Specify the extents to create rectangles.
- offset – Specify the x/y offset when keep spacing in y/x coordinate direction.
- maintainSpacing – If specify “-maintainSpacing true”, no other rectangle may fall within the spacing halo. (Default is: False)
- dbuPerUU – Specify dbuPerUU for points in output-collection. (Default is: 1000)

Returns

The result points container

rectEnclosure

Syntax

```
rectEnclosure(  
input: dbCollection,  
/,  
*,  
with_: dbCollection,  
goodRules: str,  
abut: str = None,  
singular: bool = None,  
outsideAlso: bool = None)  
Returns: dbCollection
```

Summary:

Provides efficient checking of multiple-condition enclosure rules for orthogonal rectangular polygons which edges parallel to database coordinate axes.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify input-collection.
- `with_` – Specify with-collection.
- `goodRules` – Specify the good rules based on the good rule format. (e.g. { {{0.15} {0.15} {0.15} } {{0} {0.5 opposite} {0} {0.5 opposite}} })
- `abut` – Specify that separation between intersecting edges should also be checked under the specified constraint. (e.g. <90)
- `singular` – Handle polygon singularities that are point-to-edge or point-to-point polygon intersections. (Default is: False)
- `outsideAlso` – If its value is true, outputs rectangles from input-collection that are not enclosed by rectangles of with-collection. (Default is: False)

Returns

The result points container

shapePoints

Syntax

```
shapePoints(  
  points: list[point],  
  /,  
  *,  
  jogLen: float,  
  closed: bool = None,  
  convexConcave: bool = None,  
  dbuPerUU: int = None)  
Returns: point
```

Summary:

Shape input points to remove jogs and output orthogonal points.

Category:

Foundation

Licenses:

None

Parameters

- points – The list of points that form shape points.
- jogLen – Specify the jog length in user units.
- closed – Specify the input points is closed or not. (Default is: True)
- convexConcave – Specify the two corners of jog edge satisfying 90 and 270 degrees. (Default is: True)
- dbuPerUU – Specify dbuPerUU for points input. (Default is: 1000)

Returns

The orthogonal points having no jogs

shift

Syntax

```
shift(  
input: dbCollection,  
/,  
*,  
by: point)  
Returns: dbCollection
```

Summary:

Shift all polygons of input-collection by the specified x-coordinate offset and y-coordinate offset.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- by – Specify the shifting offsets for x-coordinate and y-coordinate.

Returns

The result points container

size

Syntax

```
size(  
input: dbCollection | poPointSet,  
/,  
*,  
by: box | float | point,  
stepping: bool = None)  
Returns: dbCollection | poPointSet
```

Summary:

Do sizing uniformly or non-uniformly.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify the operand1 points container.
- by – Specify the values to do sizing uniformly or non-uniformly.
- stepping – If specify “-stepping true”, do growing/shrinking step by step. (Default is: True)

Returns

The result points container

snapGrid

Syntax

```
snapGrid(  
  input: dbCollection,  
  /,  
  *,  
  resolution: point,  
  overSize: bool = None,  
  underSize: bool = None)  
Returns: dbCollection
```

Summary:

Snap the input polygons on grid users specified.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- resolution – Specify the grid resolution in user units. x-coordinate is applied in the x direction and y-coordinate is applied in the y-direction.
- overSize – Specify the operation result would enclose the input polygons or not. “-overSize true” and “-underSize true” are exclusive. (Default is: True)
- underSize – Specify the operation result would be inside the input polygons or not. “-overSize true” and “-underSize true” are exclusive. (Default is: False)

Returns

The result points container

straddle

Syntax

```
straddle(  
  input: dbCollection,  
  /,  
  *,  
  with_: dbCollection,
```

```
not_: bool = None)
Returns: dbCollection
```

Summary:

Select all polygons from input-collection that straddle a polygon of with-collection.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- with_ – Specify with-collection.
- not_ – “-not true” is the complement of the straddle operation. (Default is: False)

Returns

The result points container

touch

Syntax

```
touch(
input: dbCollection,
/,
*,
with_: dbCollection,
value: str = None,
hasEvenOdd: po.touch.HasEvenOdd = None,
not_: bool = None)
Returns: dbCollection
```

Summary:

Select all polygons from input-collection that are completely outside all polygons of with-collection and share a coincident edge, or segment, with a polygon of with-collection.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify input-collection.
- `with_` – Specify with-collection.
- `value` – Specify the selection criterion. (e.g. `>=2`)
- `hasEvenOdd` – Cooperate with “-value” to specify the extra selection criterion. (`{ even | none | odd }` (Default is: ‘none’)
- `not_` – “-not true” is the complement of the touch operation. (Default is: False)

Returns

The result points container

touchEdge

Syntax

```
touchEdge(  
input: dbCollection,  
/,  
*,  
with_: dbCollection,  
dir: po.touchEdge.Dir = None,  
not_: bool = None)  
Returns: dbCollection
```

Summary:

Select all complete edges from input-collection that are coincident with any portion of edges in with-collection.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify input-collection.
- `with_` – Specify with-collection.
- `dir` – Specify the touch direction. (`{ all | inside | outside }` (Default is: ‘all’)
- `not_` – “-not true” is the complement of the touchEdge operation. (Default is: False)

Returns

The result points container

transform

Syntax

```
transform(  
points: list[point],  
/,  
*,  
selected: list[point] = None,  
offset: point = None,  
closed: bool = None,  
lockAngles: bool = None,  
keepLength: bool = None,  
keepHole: bool = None,  
lockEnds: bool = None,  
formLoop: bool = None,  
dbu: int = None,  
dbuPerUU: int = None,  
orient: po.transform.Orient = None)  
Returns: point
```

Summary:

Transform points based on the selected points, some flags, offset and orientation.

Category:

Foundation

Licenses:

None

Parameters

- points – The list of points that form transforming points.
- selected – Transform the selected points. If no selected points, transform all input points.
- offset – Specify the x-offset and y-offset in user units.
- closed – Specify the input points is closed or not. (Default is: True)
- lockAngles – Indicate a flag maintaining the corresponding angles of the selected points when transform points. (Default is: True)
- keepLength – Keep the length of adjacent selected points. (Default is: False)

- keepHole – Keep the number of the holes in the input closed points. (Default is: True)
- lockEnds – Lock the endpoints of the input polyline when the selected points exclude endpoints. (Default is: True)
- formLoop – Only for the polyline. Refine the start/end points when the input polyline is self intersecting. (Default is: False)
- dbu – Specify dbuPerUU for points input. (Default is: 1000)
- dbuPerUU – Specify dbuPerUU for points input. (Default is: 1000)
- orient – Orientation of the transforming points. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 } (Default is: 'R0'))

Returns

The transformed points. (If the result is self-intersecting, return empty list.)

vertex

Syntax

```
vertex(  
input: dbCollection,  
/,  
*,  
value: str)  
Returns: dbCollection
```

Summary:

Select all polygons from input-collection having vertex counts conforming to the value.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- value – Specify the specific number of vertices for a polygon. (e.g. <=4)

Returns

The result points container

withEdge

Syntax

```
withEdge(  
    input: dbCollection,  
    /,  
    *,  
    with_: dbCollection,  
    value: str = None,  
    not_: bool = None)  
Returns: dbCollection
```

Summary:

Select polygons from input-collection that have edges or edge segments on with-collection.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- with_ – Specify with-collection.
- value – Specify the specific number of edges or segments of with-collection. (e.g. >=2)
- not_ – “-not true” is the complement of the withEdge operation. (Default is: False)

Returns

The result points container

withNeighbor

Syntax

```
withNeighbor(  
    input: dbCollection,  
    /,  
    *,  
    with_: dbCollection = None,  
    value: str,  
    space: str,  
    metric: po.withNeighbor.Metric = None,
```

```
not_: bool = None)
Returns: dbCollection
```

Summary:

Select all orthogonal rectangles from input-collection if they have the specified number of orthogonal rectangles within the specified distance.

Category:

Foundation

Licenses:

None

Parameters

- input – Specify input-collection.
- with_ – Specify with-collection.
- value – Specify the selection criterion to represent the number of neighboring rectangles. (e.g. ==2)
- space – Specify the within distance in user units. (e.g. <=0.1)
- metric – Cooperate with “-space” to specify the measuring criterion. ({ center | euclidean | square } (Default is: ‘euclidean’)
- not_ – “-not true” is the complement of the withNeighbor operation. (Default is: False)

Returns

The result points container

withWidth

Syntax

```
withWidth(
input: dbCollection,
/,
*,
value: str = None,
not_: bool = None)
Returns: dbCollection
```

Summary:

Select all regions of polygons in input-collection that satisfy the width constraint to generate output polygons.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify input-collection.
- `value` – Specify the width constraint in user units. (e.g. <0.3)
- `not_` – “-not true” is the complement of the `withWidth` operation. (Default is: False)

Returns

The result points container

xor

Syntax

```
xor(  
input: dbCollection | poPointSet,  
/,  
*,  
with_: dbCollection | poPointSet)  
Returns: dbCollection | poPointSet
```

Summary:

Run the Boolean XOR operation.

Category:

Foundation

Licenses:

None

Parameters

- `input` – Specify the operand1 points container.
- `with_` – Specify the operand2 points container.

Returns

The result points container

snps.custom.cmd.ps

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you access OptoDesigner package.

For details, see:

- [getOptoDesigner](#)
- [startOptoDesigner](#)
- [terminateOptoDesigner](#)

getOptoDesigner

Syntax

```
getOptoDesigner()  
Returns: xtJob
```

Arguments:

Summary:

Get the OptoDesigner engine running job

Category:

Photonics

Licenses:

OptoCompilerElements

Returns

Job object of OptoDesigner

startOptoDesigner

Syntax

```
startOptoDesigner(  
*,  
    verbose: bool = None)  
Returns: xtJob
```

Summary:

Start connection to OptoDesigner engine

Category:

Photonics

Licenses:

OptoCompilerElements

Parameters

verbose – Start OptoDesigner engine in verbose mode (Default is: False)

Returns

Job object of OptoDesigner

terminateOptoDesigner

Syntax

```
terminateOptoDesigner()  
Returns: None
```

Arguments:

Summary:

Terminate connection to OptoDesigner engine

Category:

Photonics

Licenses:

OptoCompilerElements

Returns

None

snps.custom.cmd.rc

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These

submodules support the use of StarRC commands for parasitic resistance and capacitance analyses within the Custom Compiler environment.

For details, see:

- [getCells](#)
- [getCouplingCapacitors](#)
- [getElmoreDelay](#)
- [getGPDCorners](#)
- [getGPDLayers](#)
- [getGroundCapacitors](#)
- [getNets](#)
- [getPins](#)
- [getPointToPointResistance](#)
- [getPorts](#)
- [getResistors](#)
- [readParasitics](#)
- [removeAnnotatedParasitics](#)
- [reportGPDConfig](#)
- [reportGPDProperties](#)
- [resetGPDConfig](#)
- [setGPDConfig](#)

getCells

Syntax

```
getCells(  
*,  
    exact: bool = None,  
    quiet: bool = None,  
    ofObjects: dbCollection | str = None,  
    pattern: str = None)  
Returns: dbCollection
```

Summary:

This command will return a container of rcCell objects for the user specified query.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- `exact` – Disable the simple pattern matching for querying cells
- `quiet` – Suppresses warning and error messages if no objects match
- `ofObjects` – Creates a collection of cells from the specified objects
- `pattern` – Specifies the pattern for querying cells

Returns

Cells match with the specified query.

getCouplingCapacitors

Syntax

```
getCouplingCapacitors(  
*,  
  of: dbCollection | str = None,  
  from_: str = None,  
  to: str = None,  
  corners: str = None,  
  allCorners: bool = None)  
Returns: dbCollection
```

Summary:

This command will return a set of rcCouplingCapacitors for the given nets or nodes on the command line.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- `of` – Reports coupling capacitors associated with the specified nets
- `from_` – Constrains the selected capacitors to be part of a path starting from the specified node
- `to` – Constrains the selected capacitors to be part of a path ending with the specified node
- `corners` – Indicates the parasitic corner ordering for coupling capacitance values
- `allCorners` – Queries resistance values for all corners

Returns

Coupling capacitors for the given nets or nodes.

getElmoreDelay

Syntax

```
getElmoreDelay(  
*,  
  from_: str = None,  
  of: str = None,  
  to: str = None,  
  corners: str = None,  
  allCorners: bool = None)  
Returns: dbCollection
```

Summary:

This command will return Elmore delay for the given nodes on the command line.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- `of` – Constrains the selected resistors to be part of a path starting from the specified node
- `to` – Constrains the selected resistors to be part of a path ending with the specified node

- corners – Specifies the corners to query resistance values
- allCorners – Queries resistance values for all corners

Returns

Resistors for the given nets or nodes.

getGPDCorners

Syntax

```
getGPDCorners(  
*,  
  gpd: str)  
Returns: str
```

Summary:

This command will return a string of corners for the specified GPD.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

gpd – Specifies the path to GPD directory

Returns

Corners for the specified GPD.

getGPDLayers

Syntax

```
getGPDLayers(  
*,  
  gpd: str,  
  type: rc.getGPDLayers.Type = None)  
Returns: str
```

Summary:

This command will return a string of layers for the specified GPD.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- `gpd` – Specifies the path to GPD directory
- `type` – Specifies the type of layers to report ({ `all` | `routing` | `via` })

Returns

Layers for the specified GPD.

getGroundCapacitors

Syntax

```
getGroundCapacitors(  
*,  
  of: dbCollection | str = None,  
  from_: str = None,  
  to: str = None,  
  corners: str = None,  
  allCorners: bool = None)  
Returns: dbCollection
```

Summary:

This command will return a set of `rcGroundCapacitors` for the given nets or nodes on the command line.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- `of` – Reports ground capacitors associated with the specified nets
- `from_` – Constrains the selected capacitors to be part of a path starting from the specified node

- to – Constrains the selected capacitors to be part of a path ending with the specified node
- corners – Indicates the parasitic corner ordering for ground capacitance values
- allCorners – Queries capacitance values for all corners

Returns

Ground capacitors for the given nets or nodes.

getNets

Syntax

```
getNets(  
*,  
    exact: bool = None,  
    quiet: bool = None,  
    ofObjects: dbCollection | str = None,  
    pattern: str = None)  
Returns: dbCollection
```

Summary:

This command will return a container of rcNet objects for the user specified query.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- exact – Disable the simple pattern matching for querying nets
- quiet – Suppresses warning and error messages if no objects match
- ofObjects – Creates a collection of nets from the specified objects
- pattern – Specifies the pattern for querying nets

Returns

Nets match with the specified query.

getPins

Syntax

```
getPins(  
*,  
    exact: bool = None,  
    quiet: bool = None,  
    ofObjects: dbCollection | str = None,  
    pattern: str = None)  
Returns: dbCollection
```

Summary:

This command will return a container of rcPin objects for the user specified query.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- exact – Disable the simple pattern matching for querying pins
- quiet – Suppresses warning and error messages if no objects match
- ofObjects – Creates a collection of pins from the specified objects
- pattern – Specifies the pattern for querying pins

Returns

Pins match with the specified query.

getPointToPointResistance

Syntax

```
getPointToPointResistance(  
*,  
    from_: str = None,  
    of: str = None,  
    to: str = None,  
    corners: str = None,  
    allCorners: bool = None)  
Returns: dbCollection
```

Summary:

This command will return resistance for the given nodes on the command line.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- `of` – Constrains the selected resistors to be part of a path starting from the specified node
- `to` – Constrains the selected resistors to be part of a path ending with the specified node
- `corners` – Specifies the corners to query resistance values
- `allCorners` – Queries resistance values for all corners

Returns

Resistors for the given nets or nodes.

getPorts

Syntax

```
getPorts(  
*,  
    exact: bool = None,  
    quiet: bool = None,  
    ofObjects: dbCollection | str = None,  
    pattern: str = None)  
Returns: dbCollection
```

Summary:

This command will return a container of rcPort objects for the user specified query.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- `exact` – Disable the simple pattern matching for querying ports
- `quiet` – Suppresses warning and error messages if no objects match
- `ofObjects` – Creates a collection of ports from the specified objects
- `pattern` – Specifies the pattern for querying ports

Returns

Ports match with the specified query.

getResistors

Syntax

```
getResistors(  
*,  
  of: dbCollection | str = None,  
  from_: str = None,  
  to: str = None,  
  corners: str = None,  
  allCorners: bool = None)  
Returns: dbCollection
```

Summary:

This command will return a set of `rcResistor` objects for the given nets or nodes on the command line.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- `of` – Reports resistors associated with the specified nets
- `from_` – Constrains the selected resistors to be part of a path starting from the specified node
- `to` – Constrains the selected resistors to be part of a path ending with the specified node

- corners – Specifies the corners to query resistance values
- allCorners – Queries resistance values for all corners

Returns

Resistors for the given nets or nodes.

readParasitics

Syntax

```
readParasitics(  
*,  
  gpd: str | None = None,  
  keepCapacitiveCoupling: bool | None = None,  
  **kwargs)  
Returns: str
```

Summary:

This command will read parasitics data with the specified arguments.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- gpd – Read GPD from specified path
- keepCapacitiveCoupling – Indicates that the cross capacitors are to be kept in the RC networks data structure

Returns

Result of parasitics data.

removeAnnotatedParasitics

Syntax

```
removeAnnotatedParasitics()  
Returns: str
```

Arguments:

Summary:

This command will remove GPD parasitics data.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Returns

Result of removing GPD.

reportGPDConfig

Syntax

```
reportGPDConfig(  
*,  
  gpd: str,  
  type: rc.reportGPDConfig.Type = None,  
  includeStarRCOptions: bool = None)  
Returns: str
```

Summary:

This command will return a string of configurations for the specified GPD.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- gpd – Specifies the path to GPD directory
- type – Specifies the type of GPD configurations to report ({ active | original | user_defined })
- includeStarRCOptions – Includes StarRC GPD configuration options in the report

Returns

Configurations for the specified GPD.

reportGPDPProperties

Syntax

```
reportGPDPProperties(  
*,  
  gpd: str,  
  all: bool | None = None,  
  layers: bool | None = None,  
  corners: bool | None = None)  
Returns: str
```

Summary:

This command will return a string of properties for the specified GPD.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- gpd – Specifies the path to GPD directory
- all – Reports all types of information
- layers – Reports only the layers
- corners – Reports only the parasitic corners

Returns

Properties for the specified GPD.

resetGPDConfig

Syntax

```
resetGPDConfig(  
*,  
  gpd: str)  
Returns: str
```

Summary:

This command will reset configurations for the specified GPD.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

gpd – Specifies the path to GPD directory

Returns

None

setGPDConfig

Syntax

```
setGPDConfig(  
*,  
  gpd: str,  
  absoluteCouplingThreshold: float = None,  
  relativeCouplingThreshold: float = None,  
  couplingThresholdOperation: rc.setGPDConfig.CouplingThresholdOperation  
  = None,  
  netlistSelectNets: str = None,  
  netlistType: str = None)  
Returns: str
```

Summary:

This command will set configurations for the specified GPD.

Category:

Parasitics

Licenses:

Backend or Elite or Apex

Parameters

- gpd – Specifies the path to GPD directory
- absoluteCouplingThreshold – Specifies the absolute coupling threshold for filtering coupling capacitors. A capacitor having less than this capacitance value is split into two capacitors to ground.
- relativeCouplingThreshold – Includes StarRC GPD configuration options in the report

- `couplingThresholdOperation` – Specifies the type of GPD configurations to report ({ and | or })
- `netlistSelectNets` – Specifies a subset of nets to be read from the GPD database
- `netlistType` – Specifies the parasitics network form to consider when reading the GPD. The allowed settings are `no_couple`, `R`, `CG`, `CC`, `RCG`, and `RCC`

Returns

None

snps.custom.cmd.rt

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform routing and add custom route patterns for use in the Pattern Route Assistant.

For details, see:

- [connectCellRails](#)
- [createFreeFormTopology](#)
- [createGroupPattern](#)
- [createPowerStaples](#)
- [createPowerStripes](#)
- [createTrunkTopology](#)
- [createUniblock](#)
- [deleteTopology](#)
- [deleteTrunkAndShield](#)
- [deleteTrunkBox](#)
- [exportTopology](#)
- [forceConnect](#)
- [generateTrunk](#)
- [getTopologies](#)

- [getTopologyInfo](#)
- [getTopologyNets](#)
- [importTopology](#)
- [insertVias](#)
- [powerHookup](#)
- [reportSimilarity](#)
- [routeHierarchy](#)
- [snps.custom.cmd.rt.pkg](#)

connectCellRails

Syntax

```
connectCellRails(  
*,  
    design: oaDesign,  
    nets: list[str],  
    layer: str,  
    orient: rt.connectCellRails.Orient = None,  
    extendToGrid: bool = None,  
    autoConnect: bool = None,  
    insts: list[str] = None,  
    cells: list[str] = None,  
    trackRegionName: str = None,  
    regionPoints: list[point] = None,  
    excludeRegionPoints: list[point] = None,  
    lockColor: bool = None,  
    routeType: rt.connectCellRails.RouteType = None,  
    rules: str = None,  
    verbose: int = None)  
Returns: list[deFigure]
```

Summary:

Connect Cell Rails

Category:

GCR

Licenses:

Elite

Parameters

- design – The design
- nets – The list of net names. The power stripes or staples will alternate between the nets if more than 1 net is specified.
- layer – The layer on which to create the power stripe
- orient – The orientation of power stripes ({ horizontal | vertical })
- extendToGrid – Specifies true to extend the rail stripes to power grid, else to the PrBoundaries of all instances.
- autoConnect – Specifies true to insert vias to connect cell rails with other power stripes on adjacent metal layers
- insts – List of regular expressions to specify which instances should be connected by this command.
- cells – List of regular expressions to specify which cells should be connected by this command.
- trackRegionName – Name of the track region in which the stripes or staples should be created. The stripes or staples should follow the local tracks in the track region
- regionPoints – List of points of the rectilinear region in which the stripes or staples should be created.
- excludeRegionPoints – List of points of the rectilinear region in which the stripes or staples should not be created.
- lockColor – Specifies if lock color for created shapes.
- routeType – Specifies the route type for created shapes. ({ PGFollowPin | PGPinConnect | PGRing | PGStripe | customRoute | signalUser })
- rules – Specifies the set of rules to be honored (Default is: ")
- verbose – Verbose level

Returns

Collection of Newly Created Shapes

createFreeFormTopology

Syntax

```
createFreeFormTopology(  
    name: str = None,
```

```
/,
*,
net: oaNet | str,
design: oaDesign = None,
source: deFigure | oaShape | str,
target: deFigure | oaShape | str,
points: list[point] = None,
pattern: str = None,
connect_overlap: bool = None,
**kwargs)
Returns: str
```

Summary:

Create Free Form Topology

Category:

GCR

Licenses:

Backend

Parameters

- name – The name of topology
- net – The net name or oaNet object
- design – The design
- source – The name of source object for the free form topology.
- target – The name of target object for the free form topology.
- points – The list of points that form topology.
- pattern – The pattern string
- connect_overlap – Control whether to create connection to the overlapped pin or topology

Returns

Name of newly created topology

createGroupPattern

Syntax

```
createGroupPattern(
name: str = None,
```

```
/,
*,
net: oaNet | str,
design: oaDesign = None,
group: str,
pattern: str = None)
Returns: str
```

Summary:

Create group pattern

Category:

GCR

Licenses:

Backend

Parameters

- name – The name of topology
- net – The net name or oaNet object
- design – The design
- group – The name of group object for group pattern.
- pattern – The pattern string

Returns

Name of newly created pattern

createPowerStaples

Syntax

```
createPowerStaples(
*,
design: oaDesign,
nets: list[str],
create_net: bool = None,
layer: str,
pitch: float = None,
offset: float = None,
width: float = None,
length: float = None,
staplePitch: float = None,
orient: rt.createPowerStaples.Orient = None,
pullBackLow: float | str = None,
```

```
pullBackHigh: float | str = None,  
snapTrack: bool = None,  
useTrackSigType: bool = None,  
keepBoundary: bool = None,  
trackRegionName: str = None,  
regionPoints: list[point] = None,  
excludeRegionPoints: list[point] = None,  
lockColor: bool = None,  
routeType: rt.createPowerStaples.RouteType = None,  
rules: str = None,  
verbose: int = None,  
ported: bool | oaPin | list[oaPin] = None,  
pinPurposeNames: str = None,  
createPinShapes: bool = None)  
Returns: list[deFigure]
```

Summary:

Create Power Staples

Category:

GCR

Licenses:

Elite

Parameters

- design – The design
- nets – The list of net names. The power stripes or staples will alternate between the nets if more than 1 net is specified.
- create_net – Create a net if the specified net name doesn't exist.
- layer – The layer on which to create the power stripe
- pitch – The pitch at which the power stripe should be repeated
- offset – The offset of the first stripe from origin of the PRBoundary or the region.
- width – The width of the power stripe
- length – The staple length
- staplePitch – The staple pitch
- orient – The orientation of power stripes ({ horizontal | vertical })
- pullBackLow – The distance by which the stripe or staple should be pulled back from the beginning of region or PRBoundary (float value, 'halfDR', 'fullDR')

- pullBackHigh – The distance by which the stripe or staple should be pulled back from the end of region or PRBoundary (float value, 'halfDR', 'fullDR')
- snapTrack – Snap to track
- useTrackSigType – Use track sigType to find where the power stripe should be created.
- keepBoundary – Control whether the stripe with centerline coincident with region will be created
- trackRegionName – Name of the track region in which the stripes or staples should be created. The stripes or staples should follow the local tracks in the track region
- regionPoints – List of points of the rectilinear region in which the stripes or staples should be created.
- excludeRegionPoints – List of points of the rectilinear region in which the stripes or staples should not be created.
- lockColor – Specifies if lock color for created shapes.
- routeType – Specifies the route type for created shapes. ({ PGFollowPin | PGPinConnect | PGRing | PGStripe | customRoute | signalUser })
- rules – Specifies the set of rules to be honored (Default is: "")
- verbose – Verbose level
- ported – Create pin for the stripes or staples
- pinPurposeNames – A string of purpose names separated by spaces
- createPinShapes – Create new shapes when creating pin. The original stripes/staples are kept.

Returns

Collection of Newly Created Shapes

createPowerStripes

Syntax

```
createPowerStripes(  
*,  
    design: oaDesign,  
    nets: list[str],  
    create_net: bool = None,  
    layer: str,  
    pitch: float = None,
```

```
offset: float = None,  
width: float = None,  
useTrackSigType: bool = None,  
keepBoundary: bool = None,  
snapTrack: bool = None,  
orient: rt.createPowerStripes.Orient = None,  
pullBackLow: float | str = None,  
pullBackHigh: float | str = None,  
count: int = None,  
trackRegionName: str = None,  
regionPoints: list[point] = None,  
excludeRegionPoints: list[point] = None,  
lockColor: bool = None,  
routeType: rt.createPowerStripes.RouteType = None,  
rules: str = None,  
verbose: int = None,  
ported: bool | oaPin | list[oaPin] = None,  
pinPurposeNames: str = None,  
createPinShapes: bool = None)  
Returns: list[deFigure]
```

Summary:

Create Power Stripes

Category:

GCR

Licenses:

Elite

Parameters

- design – The design
- nets – The list of net names. The power stripes or staples will alternate between the nets if more than 1 net is specified.
- create_net – Create a net if the specified net name doesn't exist.
- layer – The layer on which to create the power stripe
- pitch – The pitch at which the power stripe should be repeated
- offset – The offset of the first stripe from origin of the PRBoundary or the region.
- width – The width of the power stripe
- useTrackSigType – Use track sigType to find where the power stripe should be created.

- **keepBoundary** – Control whether the stripe with centerline coincident with region will be created
- **snapTrack** – Snap to track
- **orient** – The orientation of power stripes ({ horizontal | vertical })
- **pullBackLow** – The distance by which the stripe or staple should be pulled back from the beginning of region or PRBoundary (float value, 'halfDR', 'fullDR')
- **pullBackHigh** – The distance by which the stripe or staple should be pulled back from the end of region or PRBoundary (float value, 'halfDR', 'fullDR')
- **count** – Number of stripes to be created in the region or PRBoundary.
- **trackRegionName** – Name of the track region in which the stripes or staples should be created. The stripes or staples should follow the local tracks in the track region
- **regionPoints** – List of points of the rectilinear region in which the stripes or staples should be created.
- **excludeRegionPoints** – List of points of the rectilinear region in which the stripes or staples should not be created.
- **lockColor** – Specifies if lock color for created shapes.
- **routeType** – Specifies the route type for created shapes. ({ PGFollowPin | PGPinConnect | PGRing | PGStripe | customRoute | signalUser })
- **rules** – Specifies the set of rules to be honored (Default is: "")
- **verbose** – Verbose level
- **ported** – Create pin for the stripes or staples
- **pinPurposeNames** – A string of purpose names separated by spaces
- **createPinShapes** – Create new shapes when creating pin. The original stripes/staples are kept.

Returns

Collection of Newly Created Shapes

createTrunkTopology

Syntax

```
createTrunkTopology(  
    name: str = None,  
    /,
```



```
*,  
object_names: list[str] = None,  
objects: list[deFigure] | list[oaFig] | list[oaShape] = None,  
net: oaNet | str,  
design: oaDesign = None,  
orient: rt.createTrunkTopology.Orient = None,  
pattern: str = None)  
Returns: str
```

Summary:

Create Trunk Topology

Category:

GCR

Licenses:

Backend

Parameters

- name – The name of topology
- object_names – The objects, which are used to create the topology.
- objects – List of objects.
- net – The net name or oaNet object
- design – The design
- orient – The orientation for trunk topology. ({ auto | horizontal | vertical })
- pattern – The pattern string

Returns

Name of newly created topology

createUniblock

Syntax

```
createUniblock(  
box: box,  
/,  
*,  
design: oaDesign,  
direction: rt.createUniblock.Direction,  
layer: oaLayer | str = None,
```

```
net: oaNet | str = None)
Returns: None
```

Summary:

Creates an uniblock box/area to block routing in a specified direction. The uniblock can be used for routing blockage but no spacing rule will be applied to it during routing

Category:

GCR

Licenses:

Backend

Parameters

- box – The coordinates of bounding box of uniblock.
- design – Design in which to create the uniblock.
- direction – The direction of the uniblock. ({ both | horizontal | vertical })
- layer – The layer on which to create the uniblock.
- net – The net name or oaNet object to which uniblock should be assigned.

Returns

None

deleteTopology

Syntax

```
deleteTopology(
name: str = None,
/,
*,
nets: list[oaNet] = None,
design: oaDesign = None,
PR: bool = None)
Returns: None
```

Summary:

Delete Topology

Category:

GCR

Licenses:

Backend

Parameters

- name – The name of topology
- nets – nets
- design – The design
- PR – Control whether to preserve existing PR constraints

Returns

None

deleteTrunkAndShield

Syntax

```
deleteTrunkAndShield(  
*,  
    design: oaDesign = None,  
    nets: str | list[oaNet],  
    layer: str = None)  
Returns: None
```

Summary:

Delete trunk and shield shapes

Category:

GCR

Licenses:

None

Parameters

- design – Specifies the design
- nets – Specifies the collection of nets
- layer – Specifies the layer

Returns

None

deleteTrunkBox

Syntax

```
deleteTrunkBox(  
*,  
    design: oaDesign)  
Returns: None
```

Summary:

Delete search boxes created from trunk generation command

Category:

GCR

Licenses:

None

Parameters

design – Specifies the design

Returns

None

exportTopology

Syntax

```
exportTopology(  
*,  
    nets: list[oaNet] = None,  
    design: oaDesign = None,  
    filePath: str = None,  
    overwrite: bool = None,  
    verbose: list[bool] = None,  
    exportName: bool = None)  
Returns: None
```

Summary:

Export Topology

Category:

GCR

Licenses:

Backend

Parameters

- nets – nets
- design – The design
- filePath – The name of file
- overwrite – Specifies whether to overwrite
- verbose – verbose
- exportName – Specifies whether to export topology name

Returns

None

forceConnect

Syntax

```
forceConnect (  
*,  
  hideShapes: bool | None = None,  
  relaxRules: bool | None = None)  
Returns: None
```

Summary:

Force opens to complete using hideShapes and ignoreRules

Category:

GCR

Licenses:

None

Parameters

- hideShapes – Hide shapes around source/target when routing (Default is: False)
- relaxRules – Relax and/or ignore rules when routing (Default is: False)

Returns

None

generateTrunk

Syntax

```
generateTrunk(  
*,  
  design: oaDesign = None,  
  nets: str | list[oaNet],  
  layer: str,  
  startSide: rt.generateTrunk.StartSide = None,  
  align: rt.generateTrunk.Align = None,  
  extensionSide: rt.generateTrunk.ExtensionSide = None,  
  freedom: float = None,  
  ratio: int = None,  
  num: int = None,  
  coord: float | str = None,  
  coordFreedom: float = None,  
  begin: float | str = None,  
  end: float | str = None,  
  shieldSide: rt.generateTrunk.ShieldSide = None,  
  shieldNetName: list[str] = None,  
  shieldWidth: float = None,  
  shieldGap: float = None,  
  ignoreTrackForShield: bool = None,  
  shieldRatio: int = None,  
  includeInsts: list[str] = None,  
  excludeInsts: list[str] = None,  
  trimLengthLow: float = None,  
  trimLengthHigh: float = None,  
  trimLengthDriver: float = None,  
  trimLengthReceiver: float = None,  
  shareTrack: bool = None,  
  shareShield: bool = None,  
  mergeShields: bool = None,  
  trackIndexes: list[int] = None,  
  groupMode: bool = None,  
  override: bool = None,  
  tag: str = None,  
  alignLayers: list[str] = None,  
  trunkOffset: float = None,  
  autoConnect: bool = None,  
  connectPositions: list[point] = None,  
  ported: bool = None,  
  drawBox: bool = None,  
  rules: str = None,  
  snapTrack: bool = None,  
  verbose: int = None,  
  width: float = None,  
  wrongWay: bool = None,  
  extendToExistingTrunk: str = None,  
  groupPinByRow: int = None,  
  checkPinConnectivity: bool = None,  
)
```

```
checkTrunkLength: float = None,  
sameTrackRegion: bool = None,  
allowSameNetOverlap: bool = None,  
multipleLayerWidths: list[str] = None,  
maxStackViaNum: int = None,  
**kwargs)  
Returns: list[deFigure]
```

Summary:

Trunk generation command

Category:

GCR

Licenses:

Elite

Parameters

- design – Specifies the design
- nets – Specifies the collection of nets
- layer – Specifies the layer
- startSide – Specifies where to start the search. Default: center ({ center | gravity | high | highpin | low | lowpin | majority })
- align – Specifies what kind of pins to be considered when calculate the location to start the search. Default: all ({ all | driver | instPin | receiver | topPin })
- extensionSide – Specifies the way to extend the trunk. Default: pinEdge ({ both | bottom | left | pinEdge | right | top })
- freedom – Specifies to enlarge the search box by the value. Default: 0.0
- ratio – Specifies the minimal percentage of required trunk coverage. Default: 100
- num – Specifies the number of trunks to be create. Default: 1
- coord – Specifies the coordinate to start the search
- coordFreedom – Specifies search range of the option -coord
- begin – Specifies the start endpoint of the trunk.
- end – Specifies the stop endpoint of the trunk.
- shieldSide – Specifies the parallel shielding side. Default: any ({ any | both | high | low })

- `shieldNetName` – Specifies the shield net name or 'power'
- `shieldWidth` – Specifies the shield width for shielding
- `shieldGap` – Specifies the default shield gap for shielding
- `ignoreTrackForShield` – Boolean option to ignore tracks for shielding. Default: false
- `shieldRatio` – Specifies the minimal percentage of required shielding coverage. Default: 100
- `includeInsts` – Specifies a list of instance names, and only pins from these instances will be considered
- `excludeInsts` – Specifies a list of instance names, and pins from these instances will be skipped
- `trimLengthLow` – Specifies the length the trunk to trim or grow at low end
- `trimLengthHigh` – Specifies the length the trunk to trim or grow at high end
- `trimLengthDriver` – Specifies the length the trunk to trim or grow at driver end
- `trimLengthReceiver` – Specifies the length the trunk to trim or grow at receiver end
- `shareTrack` – Boolean option to enable track sharing. Default: false
- `shareShield` – Boolean option to enable shield sharing. Default: true
- `mergeShields` – Boolean option to merge new shields and existing shields. Default: false
- `trackIndexes` – Specifies the track indexes
- `groupMode` – Specifies whether to generate trunks for multiple nets on consecutive tracks. Default: false
- `override` – Specifies whether to override existing trunk. Default: false
- `tag` – Specifies tag name added to the trunk.
- `alignLayers` – Specifies a list of layers to filter pin shapes
- `trunkOffset` – Specifies the offset to adjust the start search location. Default: 0.0
- `autoConnect` – Specifies whether to insert vias for the overlap areas between newly created trunk and pins or existing trunks. Default: false
- `connectPositions` – Specifies where to insert vias for the overlap areas between newly created trunk and pins or existing trunks.
- `ported` – Specifies whether to create pin object. Default: false

- `drawBox` – Boolean option to draw the search box in canvas. Default: `false`
- `rules` – Specifies the set of rules to be honored (Default is: `''`)
- `snapTrack` – Specifies whether to snap to track in the dirty mode. Default: `true`
- `verbose` – Specifies verbose level. Default: `1`
- `width` – Specifies the trunk width to create. Default: `0.0`
- `wrongWay` – Boolean option to generate wrong way trunk. Default: `false`
- `extendToExistingTrunk` – Specifies the layer to extend to connect
- `groupPinByRow` – Specifies the number of rows that are going to be taken as one unit for trunk generation.
- `checkPinConnectivity` – Boolean option to check if pins are the layer are already connected. Default: `false`
- `checkTrunkLength` – Specifies the minimum track length . Default: `0.0`
- `sameTrackRegion` – Boolean option to enable same track region preference. Default: `false`
- `allowSameNetOverlap` – Boolean option to allow generating trunks overlapping same-direction same-net trunks. Default: `true`
- `multipleLayerWidths` – Specifies a list of layer width to create stacked trunk shapes
- `maxStackViaNum` – Specifies the number of stack vias allowed to connect another overlapped shape.

Returns

Collection of newly created shapes

getTopologies

Syntax

```
getTopologies(  
*,  
    net: oaNet | str,  
    design: oaDesign = None)  
Returns: str
```

Summary:

Get Topologies

Category:

GCR

Licenses:

Backend

Parameters

- net – The net name or oaNet object
- design – The design

Returns

List of Topology Names

getTopologyInfo

Syntax

```
getTopologyInfo(  
    *,  
    net: oaNet | str,  
    design: oaDesign = None,  
    name: str)  
Returns: str
```

Summary:

Get Topology Info

Category:

GCR

Licenses:

Backend

Parameters

- net – The net name or oaNet object
- design – The design
- name – The name of topology

Returns

Topology Information

getTopologyNets

Syntax

```
getTopologyNets(  
    *,  
    design: oaDesign,  
    **kwargs)  
Returns: list[oaNet]
```

Summary:

Get Topology Nets

Category:

GCR

Licenses:

Backend

Parameters

design – The design

Returns

Collection of OA Nets

importTopology

Syntax

```
importTopology(  
    *,  
    nets: list[oaNet] = None,  
    design: oaDesign = None,  
    filePath: str,  
    overwrite: bool = None,  
    skipError: bool = None,  
    verbose: list[bool] = None)  
Returns: None
```

Summary:

Import Topology

Category:

GCR

Licenses:

Backend

Parameters

- nets – nets
- design – The design
- filePath – The name of file
- overwrite – Specifies whether to overwrite
- skipError – Control whether to skip data errors and continue importing other topologies. (Default: false)
- verbose – verbose

Returns

None

insertVias

Syntax

```
insertVias(  
*,  
    design: oaDesign,  
    nets: list[str],  
    minLayer: str = None,  
    maxLayer: str = None,  
    trackRegionName: str = None,  
    regionPoints: list[point] = None,  
    excludeRegionPoints: list[point] = None,  
    addPatch: bool = None,  
    allowExceed: bool = None,  
    routeType: rt.insertVias.RouteType = None,  
    rules: str = None,  
    verbose: int = None,  
    **kwargs)  
Returns: list[deFigure]
```

Summary:

Insert Vias

Category:

GCR

Licenses:

Elite

Parameters

- design – The design
- nets – The list of net names.
- minLayer – The min layer
- maxLayer – The max layer
- trackRegionName – Name of the track region in which the vias should be created
- regionPoints – List of points of the rectilinear region in which vias should be created.
- excludeRegionPoints – List of points of the rectilinear region in which vias should not be created.
- addPatch – Control whether to add wire patch when via covers protrude wire path or to fix DRCs for the added via.
- allowExceed – Allows the via metal shapes to extend beyond the overlapping paths. (Default is: true)
- routeType – Specifies the route type for created shapes. ({ PGFollowPin | PGPinConnect | PGRing | PGStripe | customRoute | signalUser })
- rules – Specifies the set of rules to be honored (Default is: “)
- verbose – Verbose level

Returns

Collection of Newly Created Shapes

powerHookup

Syntax

```
powerHookup(  
*,  
    design: oaDesign,  
    nets: list[str],  
    orient: rt.powerHookup.Orient = None,  
    allowed_layers: list[str] = None,  
    allowed_stripe_layers: list[str] = None,  
    stripe_search_distance: float = None,  
    use_1lp: bool = None,  
    trackRegionName: str = None,
```

```
regionPoints: list[point] = None,  
excludeRegionPoints: list[point] = None,  
rules: str = None,  
verbose: int = None,  
**kwargs)  
Returns: list[deFigure]
```

Summary:

Power Hook Up

Category:

GCR

Licenses:

Elite

Parameters

- design – The design
- nets – The list of net names. The power stripes or staples will alternate between the nets if more than 1 net is specified.
- orient – The orientation of power stripes ({ both | horizontal | vertical })
- allowed_layers – list of allowed routing layers to make connections
- allowed_stripe_layers – list of allowed stripe layers to make connections
- stripe_search_distance – stripe search distance
- use_illp – use illp engine
- trackRegionName – Name of the track region in which the stripes or staples should be created. The stripes or staples should follow the local tracks in the track region
- regionPoints – List of points of the rectilinear region in which the stripes or staples should be created.
- excludeRegionPoints – List of points of the rectilinear region in which the stripes or staples should not be created.
- rules – Specifies the set of rules to be honored (Default is: “")
- verbose – Verbose level

Returns

Collection of Newly Created Shapes

reportSimilarity

Syntax

```
reportSimilarity(  
*,  
    targetDesign: oaDesign,  
    sourceFilePath: str = None,  
    outputFilePath: str = None,  
    scorers: list[str] = None,  
    **kwargs)  
Returns: None
```

Summary:

Report routing similarity between this design and a source design

Category:

GCR

Licenses:

Custom Compiler AI

Parameters

- targetDesign – Target design to be compared with source
- sourceFilePath – Extracted data from source design. Use migrationSource property if not provided
- outputFilePath – Output report file path
- scorers – Decide the scorers to be ran

Returns

None

routeHierarchy

Syntax

```
routeHierarchy(  
*,  
    topDesign: oaDesign,  
    nets: list[oaNet],  
    order: rt.routeHierarchy.Order = None,  
    **kwargs)  
Returns: None
```

Summary:

Hierarchical route command

Category:

GCR

Licenses:

Elite

Parameters

- `topDesign` – Specifies top design for which hierarchical router is to be executed
- `nets` – Specifies the collection of nets
- `order` – Specifies the routing order. (default: `bottomUp`) (`{ bottomUp | specifiedNets }`)

Returns

None

snps.custom.cmd.rt.pkg

This chapter lists the commands in the `pkg` module for routing.

This section covers the following topics:

- [autoTaper](#)
- [pushRoute](#)
- [reportD2dRoutingBottleneck](#)
- [reportNetLength](#)
- [reportPlaneBoundaryViaConnections](#)
- [route](#)

autoTaper

Syntax

```
autoTaper(  
*,  
  shapes: list[deFigure] | list[deSelectionSet] | list[oaFig] =  
  None,
```



```
    nets: list[oaGroup] | list[oaNet] = None,  
    design: oaDesign = None)  
Returns: None
```

Summary:

Auto update tapers associated with selected objects

Category:

GCR

Licenses:

Apex

Parameters

- shapes – Collection of shapes to update connected tapers
- nets – Collection of nets or net groups to update tapers
- design – Update all tapers in this design

Returns

None

pushRoute

Syntax

```
pushRoute(  
    *,  
    shapes: list[deFigure] | list[deSelectionSet] | list[oaFig] =  
    None,  
    nets: list[oaGroup] | list[oaNet] = None,  
    pushVia: bool = None,  
    highlightModifiedShapes: bool = None,  
    shadow: bool = None,  
    packet: str = None,  
    clearHighlights: bool = None,  
    **kwargs)  
Returns: None
```

Summary:

Push routes near selected objects to fix DRC violations

Category:

GCR

Licenses:

Apex

Parameters

- shapes – Collection of shapes to push
- nets – Collection of nets or net groups to push
- pushVia – Allow pushing via (Default is: False)
- highlightModifiedShapes – Highlight modified shapes (Default is: False)
- shadow – Enable shadow for highlights (Default is: False)
- packet – Specify packet name for highlights (Default is: 'highlight1')
- clearHighlights – Clear Push Route created highlights

Returns

None

reportD2dRoutingBottleneck

Syntax

```
reportD2dRoutingBottleneck(  
*,  
  filePath: str | None = None,  
  exportMarkers: bool | None = None)  
Returns: None
```

Summary:

Report d2d routing bottleneck

Category:

GCR

Licenses:

Apex

Parameters

- filePath – Specify the output file
- exportMarkers – Export d2d routing bottleneck as OA markers (Default is: False)

Returns

None

reportNetLength

Syntax

```
reportNetLength(  
*,  
  nets: list[oaGroup] | list[oaNet] = None,  
  design: oaDesign = None,  
  skipSupply: bool = None,  
  shortestMinLength: float = None,  
  model: rt.pkg.reportNetLength.Model = None,  
  detourThreshold: float = None,  
  detourOnly: bool = None,  
  filePath: str = None,  
  exportMarkers: bool = None)  
Returns: None
```

Summary:

Report net length

Category:

GCR

Licenses:

Apex

Parameters

- nets – Report length of nets or net groups
- design – Report length of all nets in this design
- skipSupply – Skip report length for supply nets
- shortestMinLength – Specify shortest min length for estimated length
- model – Specify min length model for calculating detour ratio ({ hpwl | minDistanceLength })
- detourThreshold – Threshold of detour ratio for calculating final score
- detourOnly – Report only nets over detour ratio (Default is: False)
- filePath – Specify the output file
- exportMarkers – Export detour ratio violations as OA markers (Default is: False)

Returns

None

reportPlaneBoundaryViaConnections

Syntax

```
reportPlaneBoundaryViaConnections(  
*,  
    design: oaDesign = None,  
    filePath: str = None,  
    exportMarkers: bool = None)  
Returns: None
```

Summary:

Report plane boundary via connections for the current design

Category:

GCR

Licenses:

Apex

Parameters

- design – Design to be checked
- filePath – Specify the output file
- exportMarkers – Export violations as OA markers (Default is: False)

Returns

None

route

Syntax

```
route(  
*,  
    nets: list[oaGroup] | list[oaNet] = None,  
    design: oaDesign = None)  
Returns: None
```

Summary:

Auto route for packaging design

Category:

GCR

Licenses:

Apex

Parameters

- nets – Collection of nets or net groups to route
- design – Route all nets in this design

Returns

None

snps.custom.cmd.irt

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform routing and add custom route patterns for use in the Pattern Route Assistant. Interactive commands are listed in this chapter.

For details, see:

- [createRoute](#)

createRoute

Syntax

```
createRoute(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive create route command

Category:

Layout Editing

Licenses:

Backend

Parameters

context – Context of activation.

Returns

None

snps.custom.cmd.sdp

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you display memory resources.

For details, see:

- [MIU](#)

MIU

Syntax

```
MIU(  
*,  
    mode: str | None = None)  
Returns: None
```

Category:

Summary:

Check memory-in-use compared to previous call

Licenses:

None

Parameters

mode – Specify checking mode.

Returns

None

snps.custom.cmd.se

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform schematic editing.

For details, see:

- [activateSheet](#)
- [addSheets](#)
- [addToOverlayRendering](#)
- [align](#)
- [attachText](#)
- [check](#)
- [chopWires](#)
- [clearNetColorizations](#)
- [colorizeNets](#)
- [connectWires](#)
- [copy](#)
- [createArc](#)
- [createBlock](#)
- [createCheckOption](#)
- [createConnectDef](#)
- [createEllipse](#)
- [createInst](#)
- [createLine](#)
- [createMask](#)
- [createNetColorizationDefinition](#)
- [createPin](#)

- [createPolygon](#)
- [createRectangle](#)
- [createRule](#)
- [createSelectionShape](#)
- [createTable](#)
- [createText](#)
- [createViolation](#)
- [createWire](#)
- [createWireName](#)
- [createWireStubs](#)
- [delete](#)
- [disconnectWires](#)
- [enableDynamicNetLabels](#)
- [extractConnectivity](#)
- [findReferenceDifferences](#)
- [generateEstRCSchematic](#)
- [generateSchematic](#)
- [generateSymbol](#)
- [getActiveSheet](#)
- [getBillOfMaterials](#)
- [getColorizationDefinitions](#)
- [getDynamicNetLabels](#)
- [getNetComponents](#)
- [getPinWiringPoint](#)
- [getRules](#)
- [getShapeStyle](#)
- [getSheets](#)

- [getTables](#)
- [importConfig](#)
- [importSymbol](#)
- [isNetlistable](#)
- [move](#)
- [moveSheetToPosition](#)
- [populateDesignSummary](#)
- [propagatePortOrder](#)
- [registerLabelFunction](#)
- [registerParasiticTopology](#)
- [registerPinShape](#)
- [renameNet](#)
- [resizeSheets](#)
- [setConnectivityValid](#)
- [setShapeStyle](#)
- [setSheetColumns](#)
- [showCheckOptions](#)
- [showEditTable](#)
- [showLibraryCheck](#)
- [showSchDesignOptions](#)
- [showSymDesignOptions](#)
- [showSynchronizePortOrder](#)
- [stretch](#)
- [syncOverlayDesign](#)
- [unregisterLabelFunction](#)
- [updatePinLabels](#)

activateSheet

Syntax

```
activateSheet(  
  none: dbSheet | int,  
  /,  
  *,  
  window: giWindow | int = None)  
Returns: None
```

Summary:

Makes the selected sheet active.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- none – The sheet to activate.
- window – The window in which to activate the sheet

Returns

None

addSheets

Syntax

```
addSheets(  
  *,  
  design: oaDesign,  
  numStart: int = None,  
  numEnd: int = None,  
  border: dmCellView | dmCellViewName = None,  
  text: str = None)  
Returns: dbCollection
```

Summary:

Adds pages to a multisheet cellview.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- design – The multisheet design to update.
- numStart – The start number of the sheets to add. Should be specified in case if -numEnd argument is set.
- numEnd – The end number of the sheets to add.
- border – The border's cellview to use.
- text – The text description to set on a sheet. (Default is: ")

Returns

The list of new created multisheet pages.

addToOverlayRendering

Syntax

```
addToOverlayRendering(  
*,  
    packetName: str = None,  
    renderAs: se.addToOverlayRendering.RenderAs,  
    layer: int = None,  
    purpose: int = None)  
Returns: None
```

Summary:

Adds a packet or a specific LPP to SE overlay rendering system.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- packetName – Display resource packet name
- renderAs – Overlay objects rendering to use. ({ OverlayDevices | OverlayWiring })
- layer – Layer number, must be used together with <-purpose>. You can only set one of the following arguments/combination of arguments at a time: <-packetName> or <-layer> and <-purpose>.
- purpose – Purpose number, must be used together with <-layer>. You can only set one of the following arguments/combination of arguments at a time: <-packetName> or <-layer> and <-purpose>.

Returns

None

align

Syntax

```
align(  
  objects: list[deFigure] | list[oaFig] | list[oaFigGroup] |  
           list[oaInst] | list[oaShape],  
  /,  
  *,  
  src: deFigure | oaFig,  
  direction: se.align.Direction = None,  
  stretch: bool = None)  
Returns: list[deFigure] | list[oaFig]
```

Summary:

Align specified objects.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- objects – Set of objects for alignment.
- src – Source object for alignment, can be also a 1x object or partial object.

- direction – Direction of the alignment line towards which the alignment should be made ({ auto | horizontal | vertical } (Default is: 'auto'))
- stretch – Controls how the object motion must be made: as stretch (true) or as move (false) (Default is: True)

Returns

Collection of aligned objects

attachText

Syntax

```
attachText(  
  name: oaText,  
  /,  
  *,  
  to: oaShape)  
Returns: None
```

Summary:

This command attaches given text to given shape.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- name – Attached text.
- to – Shape to attach.

Returns

None

check

Syntax

```
check(  
  none: list[seRule] = None,  
  /,  
  *,
```

```
design: dmCellView | dmCellViewName | oaDesign,  
autoSave: bool = None,  
forceChecks: bool = None,  
checkOnly: bool = None,  
crossViews: bool = None,  
deletePreexistingMarkersMode: se.check.DeletePreexistingMarkersMode =  
None,  
quiet: bool = None,  
viewSearchList: list[str] = None,  
viewStopList: list[str] = None)  
Returns: list[deViolation]
```

Summary:

Run erc/src checks on design.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- none – A collection of rules to invoke on the given design. If not given, all registered rules with severity not set to ignore will be invoked.
- design – design to process
- autoSave – Indicates whether any marker objects for a design should be saved (Default is: False)
- forceChecks – Indicates whether a determination should first be made if the design needs checking
- checkOnly – Indicates whether only checks should be run with no designs being modified
- crossViews – Indicates whether cross view checking is performed before the rules.
- deletePreexistingMarkersMode – Deletes error markers created by all tools or SRC/ERC after performing Check or Check and Save. ({ all | srcErcOnly } (Default is: 'all'))
- quiet – Indicates whether messages or warnings in console are suppressed (Default is: False)
- viewSearchList – View Search List for hierarchical rules
- viewStopList – View Stop List for hierarchical rules

Returns

All violations found from all requested rule checks

chopWires

Syntax

```
chopWires(  
*,  
    design: oaDesign,  
    box: box)  
Returns: None
```

Summary:

This command will chop all wires in the specified area.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- design – Design in which to create the wire chopping.
- box – Wires chopping area.

Returns

None

clearNetColorizations

Syntax

```
clearNetColorizations(  
*,  
    nets: list[str] = None,  
    definitions: list[str] = None,  
    design: oaDesign)  
Returns: None
```

Summary:

Clear Net Colorizations for specified nets or definitions

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- nets – Net Name(s)
- definitions – Colorization Definition
- design – The design to apply a net colorization.

Returns

None

colorizeNets

Syntax

```
colorizeNets(  
names: list[str] = None,  
/,  
*,  
design: oaDesign,  
definitions: list[str] = None)  
Returns: None
```

Summary:

Colorize the specified nets according to color definitions(specified or current active).

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- names – Net Name(s)
- design – The design to apply a net colorization.
- definitions – Colorization Definition

Returns

None

connectWires

Syntax

```
connectWires(  
*,  
    design: oaDesign,  
    wire1: oaLine | oaPath,  
    wire2: oaLine | oaPath)  
Returns: point
```

Summary:

This command will connect two given wire objects if they are orthogonal and intersect perpendicularly. Wire segments will be split at the crossing point and any associated wire names will be re-parented to the new segments. An oaEllopse object (solder dot) will be created at the crossing point using the size in the tool environment.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- design – Design in which to create the wire connecting.
- wire1 – First wire for connection.
- wire2 – Second wire for connection.

Returns

Intersection point of given wires

copy

Syntax

```
copy(  
objects: list[deFigure] | list[oaFig] | list[oaFigGroup] |  
        list[oaInst] | list[oaShape],  
/,  
*,
```

```
anchor: point = None,  
dx: float = None,  
dy: float = None,  
rotate: se.copy.Rotate = None,  
rows: int = None,  
columns: int = None,  
cols: int = None,  
dxRow: float = None,  
dyRow: float = None,  
dxCol: float = None,  
dyCol: float = None,  
keepNets: se.copy.KeepNets = None,  
chop: bool = None,  
destination: oaDesign = None,  
dest: oaDesign = None)  
Returns: list[deFigure] | list[oaFig]
```

Summary:

This command copies the given set of objects relative to the supplied anchor point to the destination point. Optionally, the set of the objects can be copied multiple times if the -rows or -cols option is given. The spacing between rows and columns can be controlled by command line options.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- objects – Set of objects for copying.
- anchor – Anchor point for moving vector start and for rotation.
- dx – X projection of move vector. (Default is: 0)
- dy – Y projection of move vector. (Default is: 0)
- rotate – Object rotation. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 })
- rows – Number of rows. (Default is: 1)
- columns – Number of columns. (Default is: 1)
- cols – Number of columns. (Default is: 1)
- dxRow – X projection of distance between rows. (Default is: 0)
- dyRow – Y projection of distance between rows. (Default is: 0)

- dxCol – X projection of distance between columns. (Default is: 0)
- dyCol – Y projection of distance between columns. (Default is: 0)
- keepNets – Controls how nets and labels are copied. ({ all | labeled | none } (Default is: 'labeled'))
- chop – Chop wires under created instances. (Default is: False)
- destination – Design where objects will be pasted.
- dest – Design where objects will be pasted.

Returns

Collection of newly copied objects

createArc

Syntax

```
createArc(  
box: box,  
/  
*,  
design: oaDesign,  
startAngle: float,  
endAngle: float,  
stipple: str = None,  
lineStyle: str = None,  
color: str = None)  
Returns: oaArc
```

Summary:

Create Arc

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- box – Bounding box of the touching ellipse of the arc.
- design – Design in which to create the arc.
- startAngle – Start angle of arc in positive degrees.

- `endAngle` – End angle of arc in positive degrees
- `stipple` – Stipple of the arc.
- `lineStyle` – Line style of the arc.
- `color` – Color of the arc.

Returns

The new arc

createBlock

Syntax

```
createBlock(  
*,  
  design: oaDesign,  
  cellView: dmCellView,  
  points: list[point] = None,  
  origin: point = None)  
Returns: oaInst
```

Summary:

Create block

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- `design` – The design in which a block should be added.
- `cellView` – Cell view of block to add.
- `points` – The points to use for creating shape of new block.
- `origin` – Location of the origin of block.

Returns

The instance of new created block.

createCheckOption

Syntax

```
createCheckOption(  
    name: str,  
    /,  
    *,  
    description: str,  
    value: bool)  
Returns: seCheckOption
```

Summary:

Creates check option object

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- name – Option name
- description – Option description
- value – Option value

Returns

Created check option

createConnectDef

Syntax

```
createConnectDef(  
    name: str,  
    /,  
    *,  
    object: deFigure | oaPinFig,  
    defaultNet: str,  
    origin: point = None,  
    just: se.createConnectDef.Just = None,  
    orient: se.createConnectDef.Orient = None,  
    font: se.createConnectDef.Font = None,  
    height: float = None)  
Returns: oaEvalText
```

Summary:

This command creates a connection definition.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- **name** – The name of Connection Definition.
- **object** – Object for which a connection definition should be created.
- **defaultNet** – Default net name.
- **origin** – Origin point for annotation label.
- **just** – Justification of the connection definition with respect to the origin. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight } (Default is: 'lowerCenter'))
- **orient** – Orientation of the connection definition. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 } (Default is: 'R0'))
- **font** – Font style with which the connection definition is displayed. ({ euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish } (Default is: 'stick'))
- **height** – Height of the connection definition. (Default is: 0.0625)

Returns

The new connection definition

createEllipse

Syntax

```
createEllipse(  
box: box,  
/,  
*,  
design: oaDesign,  
stipple: str = None,  
lineStyle: str = None,  
color: str = None)  
Returns: oaEllipse
```

Summary:

Create ellipse

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- box – Bounding box of the ellipse.
- design – Design in which to create the ellipse.
- stipple – Stipple of the ellipse.
- lineStyle – Line style of the ellipse.
- color – Color of the ellipse.

Returns

The new ellipse

createInst**Syntax**

```
createInst(  
name: str = None,  
/,  
*,  
design: oaDesign,  
origin: point,  
master: oaDesign = None,  
libName: str = None,  
cellName: str = None,  
viewName: str = None,  
orient: se.createInst.Orient = None,  
params: list[str] = None)  
Returns: oaInst
```

Summary:

Add an instance of the specified master cellview in given parent cell.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- name – Instance name
- design – Design in which to create the instance.
- origin – Origin of instance
- master – The master design to instantiate.
- libName – The master library name to instantiate.
- cellName – The master cell name to instantiate.
- viewName – The master view name to instantiate.
- orient – Orientation of the instance. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 } (Default is: 'R0')
- params – Parameter values provided as an array {name value name value} or as a map {{name value} {name value}}.

Returns

The inst of the specified master added in the parent cell view

createLine

Syntax

```
createLine(  
    __point__: list[point],  
    /,  
    *,  
    design: oaDesign,  
    stipple: str = None,  
    lineStyle: str = None,  
    color: str = None,  
    width: float = None)  
Returns: oaShape
```

Summary:

Create polyline

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- `__point__` – Points of polyline.
- `design` – Design in which to create the polyline.
- `stipple` – Stipple of the polyline.
- `lineStyle` – Line style of the polyline.
- `color` – Color of the polyline.
- `width` – Width of polyline. (Default is: 0)

Returns

The new line or path object added to the design

createMask

Syntax

```
createMask(  
box: box,  
/,  
*,  
design: oaDesign,  
type: se.createMask.Type = None)  
Returns: oaRect
```

Summary:

Create overlay mask

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- `box` – The box of the mask rectangle.
- `design` – Design in which to create the mask.
- `type` – Overlay mask type. ({ clip | mask } (Default is: 'mask'))

Returns

The new overlay mask added to the given design

createNetColorizationDefinition

Syntax

```
createNetColorizationDefinition(  
    name: str,  
    /,  
    *,  
    description: str = None,  
    pattern: str = None,  
    color: str,  
    lineStyle: str = None)  
Returns: seNetColorizationDefinition
```

Summary:

Create Net Colorization Definition

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- name – Definition name.
- description – Definition description. (Default is: “)
- pattern – Definition pattern (Default is: “)
- color – Definition color
- lineStyle – Definition line style (Default is: ‘solid’)

Returns

Net colorization definition object

createPin

Syntax

```
createPin(  
    name: str,
```

```
/,
*,
design: oaDesign,
type: se.createPin.Type,
points: list[point],
shape: se.createPin.Shape = None,
orient: se.createPin.Orient = None,
assignmentName: str = None,
defaultNet: str = None,
optical: bool = None)
Returns: oaPin
```

Summary:

Add a pin to the given design.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- name – Pin name
- design – Design in which to create the pin.
- type – Terminal type for pin. ({ input | inputOutput | output | switch | tristate })
- points – Tcl list of points for pin shape definition.
- shape – Shape type for pin. ({ circle | custom | polygon | square | triangle })
- orient – Orientation of the pin. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 } (Default is: 'R0'))
- assignmentName – Assignment name for connect definition to be added to pin.
- defaultNet – Name of default net for connection definition.
- optical – Create optical pin (Default is: False)

Returns

The new pin in the design

createPolygon

Syntax

```
createPolygon(  
    __point__: list[point],  
    /,  
    *,  
    design: oaDesign,  
    stipple: str = None,  
    lineStyle: str = None,  
    color: str = None)  
Returns: oaPolygon
```

Summary:

Create polygon

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- __point__ – Points of polygon.
- design – Design in which to create the polygon.
- stipple – Stipple of the polygon.
- lineStyle – Line style of the polygon.
- color – Color of the polygon.

Returns

The new polygon in the given design

createRectangle

Syntax

```
createRectangle(  
    box: box,  
    /,  
    *,  
    design: oaDesign,  
    stipple: str = None,
```

```
    lineStyle: str = None,  
    color: str = None)  
Returns: oaRect
```

Summary:

Create rectangle

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- box – The box of the rectangle.
- design – Design in which to create the rectangle.
- stipple – Stipple of the rectangle.
- lineStyle – Line style of the rectangle.
- color – Color of the rectangle.

Returns

The new rectangle added to the given design

createRule

Syntax

```
createRule(  
string: str,  
/,  
*,  
proc: str,  
description: str = None,  
defaultSeverity: se.createRule.DefaultSeverity = None,  
category: str = None,  
allowDynamic: bool = None,  
allowDisable: bool = None,  
forceFatal: bool = None,  
prefName: str,  
isIntegrityCheck: bool = None,  
viewTypes: list[str] = None,  
options: dbCollection = None,  
**kwargs)  
Returns: seRule
```

Summary:

Registers a rule check into the ERC/SRS system.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- string – The unique label of the check rule.
- proc – Tcl procedure name.
- description – Short description of the check rule.
- defaultSeverity – Default severity level. ({ error | message | warning } (Default is: 'message'))
- category – Category name of the check rule. (Default is: 'user')
- allowDynamic – Indicates if the check could be initiated dynamically. (Default is: False)
- allowDisable – Indicates if the check can be disabled. (Default is: True)
- forceFatal – Indicates if violations from this rule should be forced to be fatal severity regardless of the severity settings from the preference. (Default is: False)
- prefName – Preference name corresponding to the check rule.
- isIntegrityCheck – Indicates if the check could be done hierarchically. (Default is: False)
- viewTypes – View types on which rule can be checked.
- options – Rule check options used in TCL procedure

Returns

Newly created rule object

createSelectionShape

Syntax

```
createSelectionShape(  
point: list[point] = None,  
/  
*,  
design: oaDesign,
```

```
    overwrite: bool = None)
Returns: oaShape
```

Summary:

Create selection shape

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- point – Points of the selection shape.
- design – Design in which to create the selection shape.
- overwrite – Overwrite selection shapes. (Default is: False)

Returns

The new shape representing the selection+avoidance region

createTable

Syntax

```
createTable(
*,
    design: oaDesign,
    origin: point,
    rows: int = None,
    columns: int = None,
    name: str = None,
    title: str = None,
    namePosition: se.createTable.NamePosition = None,
    titlePosition: se.createTable.TitlePosition = None,
    rotateFirstRowText: bool = None,
    autoSize: bool = None,
    width: float = None,
    height: float = None,
    color: str = None,
    lineStyle: str = None,
    font: se.createTable.Font = None,
    fonHeight: float = None,
    textJust: se.createTable.TextJust = None,
    textColor: str = None,
    cellData: matrix = None)
Returns: seTable
```

Summary:

Creates a schematic table object

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- design – Design in which to create the table.
- origin – Upper left corner point for table
- rows – Number of rows (Default is: 2)
- columns – Number of columns (Default is: 2)
- name – Table name (Default is: "")
- title – Table title (Default is: "")
- namePosition – Table name label position ({ bottomCenter | bottomLeft | bottomRight | hide | topCenter | topLeft | topRight }) (Default is: 'topLeft')
- titlePosition – Table title position ({ bottomCenter | bottomLeft | bottomRight | hide | topCenter | topLeft | topRight }) (Default is: 'topRight')
- rotateFirstRowText – Indicates whether the first row texts of the table should be rotated 90 degrees (Default is: False)
- autoSize – Indicates whether cell size should be adjusted to table content (Default is: True)
- width – Cell width (Default is: 0.125)
- height – Cell height (Default is: 0.0875)
- color – Color of the table.
- lineStyle – Line style of the table.
- font – Font style with which the table is displayed. ({ euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish }) (Default is: 'stick')
- fontHeight – Font height (Default is: 0.0625)

- **textJust** – Justification of the table cell contents ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight } (Default is: 'centerCenter'))
- **textColor** – Text Color
- **cellData** – List of row contents, which in their turn are lists of strings.

Returns

Created schematic table object

createText

Syntax

```
createText(  
text: str,  
/,  
*,  
design: oaDesign,  
anchor: point,  
noteShape: oaShape = None,  
color: str = None,  
type: se.createText.Type = None,  
just: se.createText.Just = None,  
orient: se.createText.Orient = None,  
drafting: bool = None,  
overbar: bool = None,  
font: se.createText.Font = None,  
height: float = None)  
Returns: oaText
```

Summary:

Create Text

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- **text** – Text of the label or note.
- **design** – Design in which to create the text.
- **anchor** – The position of the label in user units.

- `noteShape` – Shape object for associating with text, only considered if type is note.
- `color` – Color of the text.
- `type` – The text evaluation type. ({ IL | SEL | normal | note } (Default is: 'normal'))
- `just` – Justification of the text with respect to the origin. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight } (Default is: 'lowerCenter'))
- `orient` – Orientation of the text. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 } (Default is: 'R0'))
- `drafting` – Indicates whether the text should not be mirrored or rotated. (Default is: True)
- `overbar` – Indicates whether the text will be rendered with overbar. (Default is: False)
- `font` – Font style with which the text is displayed. ({ euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish } (Default is: 'stick'))
- `height` – The size for displaying the label in user units. (Default is: 0.0625)

Returns

The new text

createViolation

Syntax

```
createViolation(  
string: str,  
/,  
*,  
polygon: list[point] = None,  
design: heContext | oaDesign,  
objects: list[oaBlockObject] = None,  
deleteWhen: se.createViolation.DeleteWhen = None,  
severity: se.createViolation.Severity = None,  
shortMessage: str = None,  
toolName: str = None,  
probeType: se.createViolation.ProbeType = None,  
probeObjectName: str = None,  
**kwargs)  
Returns: deViolation
```

Summary:

Creates a violation container object.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- string – Required description for the violation
- polygon – Required Tcl list of points describing violation region
- design – Design where violation originates or hierarchy context of the violation object
- objects – Tcl list of block objects associated with the violation
- deleteWhen – Controls when the marker object should be removed from the design with respect to the objects associated with the violation ({ never | onFirstRemove | onLastRemove | onModify } (Default is: 'never'))
- severity – Severity level of the violation ({ error | message | warning } (Default is: 'message'))
- shortMessage – Short description of the violation (Default is: "")
- toolName – Name of tool that found the violation (Default is: "")
- probeType – Type of probe to create required for context-based violations ({ cellview | instTerm | instance | net | path | terminal })
- probeObjectName – Probe object name required for context-based violations (Default is: "")

Returns

Newly created violation object

createWire

Syntax

```
createWire(  
points: list[point],  
/,  
*,  
design: oaDesign,  
width: float = None,  
mode: se.createWire.Mode = None,  
avoidSymbols: bool = None,  
avoidShorts: bool = None,  
)
```

```
jogStyle: se.createWire.JogStyle = None,  
color: str = None)  
Returns: list[oaShape]
```

Summary:

This function creates a wire between two points given using the point to point router engine.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- points – Start and end points of wire.
- design – Design in which to create the wire.
- width – Width of the wire. (Default is: 0)
- mode – Angle mode. ({ any | auto | diagonal | guided | orthogonal })
- avoidSymbols – Avoiding symbols.
- avoidShorts – Avoiding name shorts.
- jogStyle – Jog style. ({ auto | horizontal | vertical })
- color – Color of the wire.

Returns

The new wire

createWireName

Syntax

```
createWireName(  
name: str = None,  
/,  
*,  
wire: oaLine | oaPath,  
origin: point = None,  
just: se.createWireName.Just = None,  
orient: se.createWireName.Orient = None,  
font: se.createWireName.Font = None,  
height: float = None,
```

```
abut: bool = None,  
overbar: bool = None,  
color: str = None)  
Returns: oaText
```

Summary:

This function creates a wire name label.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- name – Wire name.
- wire – Wire to which label should be connected.
- origin – Origin point for label.
- just – Justification of the wire name with respect to the origin. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight }) (Default is: 'lowerCenter')
- orient – Orientation of the wire name. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 }) (Default is: 'R0')
- font – Font style with which the wire name is displayed. ({ euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish }) (Default is: 'stick')
- height – Height of the wire name. (Default is: 0.0625)
- abut – Abut of the wire name and wire (Default is: False)
- overbar – Indicates whether the wire name will be rendered with overbar. (Default is: False)
- color – Color of the wire name.

Returns

The wire name

createWireStubs

Syntax

```
createWireStubs(  
  Objects: list[deFigure],  
  /,  
  *,  
  auto: bool = None,  
  size: int = None,  
  wireLabel: se.createWireStubs.WireLabel = None,  
  pinType: se.createWireStubs.PinType = None,  
  userDefinedName: str = None,  
  updateStubs: bool = None)  
Returns: list[oaShape]
```

Summary:

This function creates a wire stub.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- Objects – Set of objects.
- auto – Auto adjust stub to label size. (Default is: False)
- size – Size of the wire stub in snap grids. (Default is: 2)
- wireLabel – Indicated whether the nets should be auto named, should use the pin names or a user defined name. ({ none | pinName | userDefined } (Default is: 'none'))
- pinType – Indicates type of pin which should be created at each end of created stub ({ auto | input | inputOutput | noConnPin | none | output | switch | tristate } (Default is: 'noConnPin'))
- userDefinedName – User defined parameterized name for nets. (Default is: '%I_%P')
- updateStubs – If true the command will update already existing stubs for missed pin or label according to specified label or pin creation policy. (Default is: True)

Returns

The wire stubs created

delete

Syntax

```
delete(  
  objects: list[deFigure] | list[oaFig] | list[oaFigGroup] |  
           list[oaInst] | list[oaShape])  
Returns: None
```

Summary:

This command deletes the set of specified objects.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

objects – Set of objects for deleting.

Returns

None

disconnectWires

Syntax

```
disconnectWires(  
  *,  
  design: oaDesign,  
  at: point)  
Returns: None
```

Summary:

This command will disconnect the set of four wires joining at the given point. The point must be solder dot location and the each segment should be orthogonal and have a collinear counterpart. The corresponding oaEllipse object (solder dot) will be removed and the collinear wire segments will be merged.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- design – Design in which to create the wire disconnecting.
- at – Point of solder for disconnecting.

Returns

None

enableDynamicNetLabels

Syntax

```
enableDynamicNetLabels(  
*,  
    context: deContext,  
    applicationName: str,  
    clientData: str = None,  
    getTextProc: list[variant])  
Returns: seTextResult
```

Summary:

Enable dynamic net labels and register get text procedure.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- context – The design context to enable dynamic net labels.
- applicationName – The application name for this dynamic net label object.
- clientData – The client data pass to get text procedure, typically and SQL handle.
(Default is: “)
- getTextProc – Name of the procedure which evaluates the dynamic net label.

Returns

text result object created

extractConnectivity

Syntax

```
extractConnectivity(  
    *,  
    design: oaDesign,  
    fullExtraction: bool = None)  
Returns: bool
```

Summary:

Forces the SE connectivity engine to process the cellview for connectivity extraction.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- design – The design to extract the SE connectivity.
- fullExtraction – Force to perform the design full connectivity re-extraction regardless of timestamps set. Note that some wiring shapes, instTerms and nets can get removed. (Default is: False)

Returns

Boolean indicating whether is done something.

findReferenceDifferences

Syntax

```
findReferenceDifferences(  
    overlay: oaDesign,  
    /,  
    **kwargs)  
Returns: str
```

Summary:

Compare reference objects in overlay design against the reference design.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

overlay – The overlay design to check.

Returns

{<details_ref_vs_reflOverlay> <details_ref_vs_ovlOverlay>},
where:<details_ref_vs_reflOverlay> contains differences between
reference design and non-modified reference objects in overlay design,
and has format:{<instances_with_changed_masters> <added_instances>
<removed_instances><changed_connection_to_instance_terminals>
<changed_parameters_of_instances> <instances_with_changed_positions>
<added_terminals> <removed_terminals>}<details_ref_vs_ovlOverlay>
contains differences between reference design and overlay or modified reference
objects in overlay design, and has format:{<instances_with_changed_masters>
<removed_instances><changed_connection_to_instance_terminals>
<changed_parameters_of_instances> <instances_with_changed_positions>}

Return type

True if difference found, else false. If there are details of found differences, the true result
is container of string information about these details

generateEstRCSchematic

Syntax

```
generateEstRCSchematic(  
*,  
    src: dmCellView | dmCellViewName,  
    nets: list[oaNet] = None,  
    layout: dmCellView | dmCellViewName,  
    dst: dmCellView | dmCellViewName,  
    estRCLib: str = None,  
    **kwargs)  
Returns: None
```

Summary:

interconnect RC modeling flow command

Category:

Schematic and Symbol Editing

Licenses:

Elite

Parameters

- src – Cell view of the source schematic
- nets – Nets to generate estimated RC schematic
- layout – Cell view of the layout
- dst – Cell view of the destination schematic
- estRCLib – The library for RC pcell generation

Returns

None

generateSchematic

Syntax

```
generateSchematic(  
*,  
    interface: dbInterfaceSet,  
    update: bool,  
    dest: dmCellView | dmCellViewName,  
    options: dbGenOptionSet)  
Returns: None
```

Summary:

Generates a schematic.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- interface – The interface object.
- update – Indicates if the user wishes to update an existing cellview instead of regenerating.

- `dest` – The destination cellview.
- `options` – Object containing the option values to use for schematic generation.

Returns

None

generateSymbol

Syntax

```
generateSymbol(  
*,  
    interface: dbInterfaceSet,  
    update: bool,  
    dest: dmCellView | dmCellViewName,  
    options: dbGenOptionSet,  
    pinMap: list[str] = None)  
Returns: None
```

Summary:

Generates a symbol.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- `interface` – The interface object.
- `update` – Indicates if the user wishes to update an existing cellview instead of regenerating.
- `dest` – The destination cellview.
- `options` – Object containing the option values to use for symbol generation.
- `pinMap` – The pin name mapping items.

Returns

None

getActiveSheet

Syntax

```
getActiveSheet(  
*,  
    window: giWindow | int = None)  
Returns: dbSheet
```

Summary:

Gets the currently active sheet's object or nothing.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

window – The window to get the active sheet from

Returns

Active sheet.

getBillOfMaterials

Syntax

```
getBillOfMaterials(  
*,  
    context: heContext,  
    levels: int = None,  
    excludedLibraries: str = None)  
Returns: str
```

Summary:

Computes flattened instance count

Category:

Auxiliary

Licenses:

None

Parameters

- context – The design context.
- levels – indicates count of hierarchy levels which will processed. (Default is: 0)
- excludedLibraries – List of libraries the cells from which are ignored together with their lower hierarchy. (Default is: "")

Returns

Returns list of {libName, cellName, viewName, count}.

getColorizationDefinitions

Syntax

```
getColorizationDefinitions()  
Returns: dbCollection
```

Arguments:

Summary:

Retrieves all color definitions for schematic nets colorization.

Category:

Schematic and Symbol Editing

Licenses:

None

Returns

Net colorization definitions

getDynamicNetLabels

Syntax

```
getDynamicNetLabels(  
*,  
context: deContext = None)  
Returns: dbCollection
```

Summary:

Look for registered dynamic net labels by given context.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

context – The design context to look for registered dynamic net labels.

Returns

Collection of text result objects

getNetComponents

Syntax

```
getNetComponents (  
  object: deFigure,  
  /,  
  *,  
  of: se.getNetComponents.Of = None)  
Returns: list[oaShape]
```

Summary:

This command returns a set of objects associated with a given net group. The objects will be dependent on the active object as well as the value of enNetGroup.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- object – Wiring object used as base for expansion
- of – Indicates the group of objects of the net associated with given object to return. ({ branch | net | object | subnet })

Returns

The net components

getPinWiringPoint

Syntax

```
getPinWiringPoint(  
    pin: deFigure | oaPin | oaPinFig)  
Returns: point
```

Summary:

Get the preferred wiring point for the given pin.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

pin – A pin, a schematic pin or an instance pin.

Returns

The preferred wiring point for the given pin.

getRules

Syntax

```
getRules(  
    string: str = None,  
    /,  
    *,  
    severity: se.getRules.Severity = None,  
    scope: dbScope | deContext | dmCell | dmCellView | oaLib |  
    str = None)  
Returns: list[seRule]
```

Summary:

Returns a collection of matching rules currently registered in the ERC/SRS system.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- string – Style pattern for matching rule name. (Default is: “)
- severity – Default severity level. ({ any | error | message | warning } (Default is: ‘any’)
- scope – Scope used for severity preference.

Returns

Collection of rules matching the name and filter expression

getShapeStyle

Syntax

```
getShapeStyle(  
    name: oaShape,  
    /,  
    *,  
    attribute: se.getShapeStyle.Attribute)  
Returns: str
```

Summary:

Get the shape style – color, stipple and linestyle.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- name – An oaShape object to get color, linestyle or stipple.
- attribute – Name of the style attribute to retrieve. ({ color | lineStyle | stipple })

Returns

Current color, line style, and fill for the given shape

getSheets

Syntax

```
getSheets(  
    *,  
    /,
```

```
design: oaDesign)
Returns: dbCollection
```

Summary:

Returns all the sheets collected from the multisheet cellview.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

design – The design to get the sheets from

Returns

A list of collected objects. Will be empty for non multisheet design.

getTables

Syntax

```
getTables (
*,
design: oaDesign)
Returns: dbCollection
```

Summary:

Returns the schematic table objects of the design.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

design – Design from which table objects are retrieved

Returns

Collection of table objects.

importConfig

Syntax

```
importConfig(  
*,  
    window:  giWindow  |  int,  
    viewName: str,  
    **kwargs)  
Returns: None
```

Summary:

Import VDRC configuration for VDRC assistant

Category:

Schematic

Licenses:

Frontend

Parameters

- window – Specify the window to load the configuration
- viewName – Specify the config view name. The lib and cell name of the imported config view will be the same as the design by the given window.

Returns

None

importSymbol

Syntax

```
importSymbol(  
*,  
    design: oaDesign,  
    src:    dmCellView | dmCellViewName,  
    point:  point,  
    exclude: list[str] = None)  
Returns: None
```

Summary:

Copies the shapes, pins, labels and selection regions from the specified source cellview into the given design.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- design – The design to contain the new object.
- src – Source cellview with view type of schemaicSymbol to import.
- point – Coordinate in destination design to use as the point to coincide with the origin of the copied objects.
- exclude – TCL list of enums indicating which types of objects to exclude from copying.

Returns

None

isNetlistable**Syntax**

```
isNetlistable(  
  design: dmCellView | dmCellViewName | oaDesign)  
Returns: bool
```

Summary:

Checks whether design is netlistable.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

design – design to process

Returns

Indication whether design is ready for netlisting or not

move

Syntax

```
move (  
objects: list[deFigure] | list[oaFig] | list[oaFigGroup] |  
        list[oaInst] | list[oaShape],  
/,  
*,  
dx: float = None,  
dy: float = None,  
rotate: se.move.Rotate = None,  
anchor: point = None,  
destination: oaDesign = None,  
dest: oaDesign = None)  
Returns: list[deFigure] | list[oaFig]
```

Summary:

This command moves a given set of objects by the supplied X and Y distance. At least one of the two options, -dx or -dy, must be supplied else an error will be generated.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- objects – Set of objects for moving.
- dx – X projection of move vector. (Default is: 0)
- dy – Y projection of move vector. (Default is: 0)
- rotate – Object rotation. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 })
- anchor – Anchor point for rotation.
- destination – Design where objects will be pasted.
- dest – Design where objects will be pasted.

Returns

Collection of the moved objects

moveSheetToPosition

Syntax

```
moveSheetToPosition(  
    none: dbSheet | int,  
    /,  
    *,  
    newPos: int,  
    design: oaDesign)  
Returns: None
```

Summary:

Changes the position of specified sheet.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- none – The sheet to move.
- newPos – The new position of a sheet.
- design – The multisheet design to process.

Returns

None

populateDesignSummary

Syntax

```
populateDesignSummary(  
    *,  
    context: heContext,  
    levels: int = None,  
    excludedLibraries: str = None,  
    mappingFile: str = None,  
    dbHandle: str = None)  
Returns: str
```

Summary:

Process the design hierarchy and populate design data info

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- context – The design context.
- levels – indicates count of hierarchy levels which will processed. (Default is: 0)
- excludedLibraries – List of libraries the cells from which are ignored together with their lower hierarchy. (Default is: “)
- mappingFile – The mapping file to refer for equations. (Default is: “)
- dbHandle – The database handle to populate data. If not set, the output is TCL list. (Default is: “)

Returns

Returns list of data rows if DB handle not provided.

propagatePortOrder

Syntax

```
propagatePortOrder(  
*,  
    destination: oaDesign,  
    value: str = None,  
    source: oaDesign = None)  
Returns: None
```

Summary:

Propagates the given string or the portOrder property value of the given source design to the portOrder property of the destination design.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- destination – Design which portOrder property should be set.
- value – Port order value.
- source – Source design the port order value of which should be propagated to given design.

Returns

None

registerLabelFunction

Syntax

```
registerLabelFunction(  
name: str,  
/,  
*,  
evalType: se.registerLabelFunction.EvalType,  
expr: str,  
layer: int,  
purpose: int,  
position: int = None,  
nameEvalProc: dbExecObject | str = None,  
valueEvalProc: dbExecObject | str = None,  
commitProc: dbExecObject | str = None,  
validateProc: dbExecObject | str = None)  
Returns: None
```

Summary:

Register label function.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- name – Evaluation function name
- evalType – Evaluation type ({ IL | SEL })
- expr – Text expression

- layer – Layer number
- purpose – Purpose number
- position – Position of the newly registered label function
- nameEvalProc – Name of the procedure which evaluates the name part of the eval text
- valueEvalProc – Name of the procedure which evaluates the value of the eval text
- commitProc – Name of the procedure responsible for committing the values during on-canvas editing
- validateProc – Name of the procedure which should validate the values that are added by on-canvas editing

Returns

None

registerParasiticTopology

Syntax

```
registerParasiticTopology(  
none: str,  
/,  
*,  
cellview: dmCellView,  
fromLib: bool = None)  
Returns: None
```

Summary:

Register custom parasitic topology.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- none – Name
- cellview – Cellview
- fromLib – Device is from the lib specified by
seEstimatedParasiticsCustomDeviceLibName (Default is: False)

Returns

None

registerPinShape

Syntax

```
registerPinShape(  
    none: str,  
    /,  
    *,  
    prefix: str)  
Returns: None
```

Summary:

Registers a new pin shape for use when creating pins in a symbol.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- none – Name of pin shape.
- prefix – String used as the prefix for associated preference names.

Returns

None

renameNet

Syntax

```
renameNet(  
    from_: oaNet | str,  
    /,  
    *,  
    to: str,  
    design: oaDesign = None)  
Returns: None
```

Summary:

Rename the specified net according to rules.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- `from_` – Source net or net name to rename
- `to` – Target name to apply
- `design` – Design to be modified

Returns

None

resizeSheets

Syntax

```
resizeSheets(  
  none: dbCollection,  
  /,  
  *,  
  newSize: dmCellView | dmCellViewName,  
  design: oaDesign)  
Returns: None
```

Summary:

Changes the border's size of specified sheets and also updates the placement of sheets.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- none – The collection of sheets to resize.
- newSize – The new border to use.
- design – The multisheet design to process.

Returns

None

setConnectivityValid

Syntax

```
setConnectivityValid(  
    design: oaDesign)  
Returns: bool
```

Summary:

Sets the design's connectivity status to valid.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

design – The design to set the connectivity status to valid.

Returns

Boolean indicating whether or not connectivity status has been set to valid.

setShapeStyle

Syntax

```
setShapeStyle(  
    name: oaShape,  
    /,  
    *,  
    stipple: str = None,  
    lineStyle: str = None,  
    color: str = None)  
Returns: None
```

Summary:

set shape style

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- name – An oaShape object to set color, linestyle or stipple.
- stipple – Stipple of the style.
- lineStyle – Line style of the style.
- color – Color of the style.

Returns

None

setSheetColumns

Syntax

```
setSheetColumns(  
    none: int,  
    /,  
    *,  
    design: oaDesign)  
Returns: None
```

Summary:

Sets the number of sheet columns of multisheet cellview.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- none – The number of columns to be set to multisheet cellview.
- design – The design to process

Returns

None

showCheckOptions

Syntax

```
showCheckOptions(  
    none: giWindow | int = None,  
    /,  
    *,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Brings up the main ERC dialog.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- none – Optional window to associate with dialog specifically to supply scope for the preference values
- parent – Optional window to associate with dialog specifically to supply scope for the preference values

Returns

Dialog window

showEditTable

Syntax

```
showEditTable(  
    *,
```

```
    window: giWindow | int = None,  
    table: seTable | str = None)  
Returns: giDialog
```

Summary:

Opens schematic table editing dialog.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- window – Parent window
- table – Table object

Returns

The created dialog

showLibraryCheck

Syntax

```
showLibraryCheck(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Show a library wide check dialog.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

parent – Optional window to associate with dialog specifically to supply scope for the preference values

Returns

Dialog window

showSchDesignOptions

Syntax

```
showSchDesignOptions(  
*,  
    parent: giWindow | int = None)  
Returns: None
```

Summary:

Shows the Schematic Design Options dialog. If the dialog is already opened for the given window, it will be raised and the existing object returned.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

parent – Parent window.

Returns

None

showSymDesignOptions

Syntax

```
showSymDesignOptions(  
*,  
    parent: giWindow | int = None)  
Returns: None
```

Summary:

Shows the Symbol Design Options dialog. If the dialog is already opened for the given window, it will be raised and the existing object returned.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

parent – Parent window.

Returns

None

showSynchronizePortOrder

Syntax

```
showSynchronizePortOrder(  
    none: dmCellView | dmCellViewName = None,  
    /,  
    *,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Opens the port order synchronizing dialog.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- none – Source cellview
- parent – Parent window

Returns

Schematic Generation Dialog

stretch

Syntax

```
stretch(  
    objects: list[deFigure] | list[oaFig] | list[oaFigGroup] |  
            list[oaInst] | list[oaShape],
```

```
/,
*,
dx: float = None,
dy: float = None,
rotate: se.stretch.Rotate = None,
anchor: point = None,
avoidSymbols: bool = None,
avoidShorts: bool = None)
Returns: list[deFigure] | list[oaFig]
```

Summary:

This command stretches a given set of objects by the supplied X and Y distance and transformation and reroutes broken connections according to avoidances.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- objects – Set of objects for stretching.
- dx – X projection of move vector. (Default is: 0)
- dy – Y projection of move vector. (Default is: 0)
- rotate – Object rotation. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 })
- anchor – Anchor point for rotation.
- avoidSymbols – Avoiding symbols.
- avoidShorts – Avoiding name shorts.

Returns

Collection of stretched objects

syncOverlayDesign

Syntax

```
syncOverlayDesign(
*,
resolveConflicts: se.syncOverlayDesign.ResolveConflicts = None,
force: bool = None,
window: giWindow | int = None)
Returns: None
```

Summary:

Synchronizes the overlay design with its reference cellview.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- `resolveConflicts` – Specifies how the conflicting cases should be handled. (`{ acceptOverlayChanges | acceptReferenceChanges | autoFix }` (Default is: `'autoFix'`)
- `force` – Force the sync without prompts. (Default is: `False`)
- `window` – The editor window which edit cellview need to be synchronized.

Returns

None

unregisterLabelFunction

Syntax

```
unregisterLabelFunction(  
    name: str,  
    /,  
    *,  
    evalType: se.unregisterLabelFunction.EvalType)  
Returns: None
```

Summary:

Unregister label function.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- `name` – Evaluation function name
- `evalType` – Evaluation type (`{ IL | SEL }`)

Returns

None

updatePinLabels

Syntax

```
updatePinLabels(  
    *,  
    design: oaDesign)  
Returns: bool
```

Summary:

Update pins annotation labels in a symbol design to have term names matched.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

design – The design to process for labels fix.

Returns

Boolean indicating if any change was done.

snps.custom.cmd.ise

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform schematic editing. Interactive commands are listed in this chapter.

For details, see:

- [align](#)
- [alignToColumn](#)
- [attachText](#)

- [check](#)
- [chopWires](#)
- [colorizeNets](#)
- [connectWires](#)
- [copy](#)
- [copyToClipboard](#)
- [createBlock](#)
- [createColumn](#)
- [createConnectDef](#)
- [createInst](#)
- [createLabel](#)
- [createMask](#)
- [createNote](#)
- [createOverlayInstanceWithMask](#)
- [createRuler](#)
- [createSchematicPin](#)
- [createSelectionShape](#)
- [createShape](#)
- [createSymbolPin](#)
- [createTable](#)
- [createTemplate](#)
- [createWire](#)
- [createWireName](#)
- [createWireStubs](#)
- [cut](#)
- [delete](#)
- [disconnectWires](#)

- [ignoreInst](#)
- [importSymbol](#)
- [manageLabels](#)
- [move](#)
- [paste](#)
- [renameInst](#)
- [route](#)
- [stretch](#)

align

Syntax

```
align(  
context: deContext | int = None)  
Returns: None
```

Summary:

This command performs alignment of passed target objects to a position of the source one.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

alignToColumn

Syntax

```
alignToColumn(  
context: deContext | int = None)  
Returns: None
```

Summary:

Aligns the selected instances to the left of a selected frame column.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

attachText

Syntax

```
attachText(  
    context: deContext | int = None,  
    /,  
    *,  
    text: oaText = None)  
Returns: None
```

Summary:

Interactive command for text attaching.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- context – Context of activation.
- text – Text object

Returns

None

check

Syntax

```
check(  
  none: giWindow | int = None,  
  /,  
  *,  
  hierarchy: bool = None)  
Returns: None
```

Summary:

Run erc/src checks on the design associated with given Window ID.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- none – Window object or window Id
- hierarchy – Indicates if entire hierarchy should be checked in addition to the given cellview. (Default is: False)

Returns

None

chopWires

Syntax

```
chopWires(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command for chopping wires in the specified area.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

colorizeNets

Syntax

```
colorizeNets(  
context: deContext | int = None)  
Returns: None
```

Summary:

Colorize a net by click, selection or names pattern

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

connectWires

Syntax

```
connectWires(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command for connecting crossing wires.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

copy

Syntax

```
copy(  
context: deContext | int = None,  
/,  
*,  
object: deFigure | oaFig = None,  
point: point = None)  
Returns: None
```

Summary:

This command copies the objects in the active selected set of the cellview associated with the specified window.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- context – Context of activation.
- object – Object for exclusive copy
- point – Specifies the drag start point and enters “one shot mode” where the COT is not updated.

Returns

None

copyToClipboard

Syntax

```
copyToClipboard(  
context: deContext | int = None,  
/,  
*,  
log: ise.copyToClipboard.Log = None)  
Returns: None
```

Summary:

Interactive schematic copy selected objects to clipboard.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- context – Context of activation.
- log – Logging mode ({ auto | explicit } (Default is: 'auto'))

Returns

None

createBlock

Syntax

```
createBlock(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactively creates blocks.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

createColumn

Syntax

```
createColumn(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command for column creation.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

createConnectDef

Syntax

```
createConnectDef(  
context: deContext | int = None)  
Returns: None
```

Summary:

This command will add a connection definition label to the given pin or wiring object and associated it to the pin or wiring object.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

createInst

Syntax

```
createInst(  
context: deContext | int = None,  
/  
*,  
device: deDevice = None,  
hidden: bool = None,  
singleShot: bool = None,  
instNames: str = None,  
deleteShorts: bool = None,  
**kwargs)  
Returns: None
```

Summary:

Interactively creates an instance.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- context – Context of activation.
- device – If specified, populates the Create Instance dialog with the master and parameters from the specified favorite device object

- **hidden** – If true, starts the command in a hidden state without affecting the COT or showing the COD. (Default is: False)
- **singleShot** – If true, the command is terminated after adding one instance. (Default is: False)
- **instNames** – Instance name(s) for the instance(s) to be created.
- **deleteShorts** – If true, after adding an instance any net segments shorting the instance terminals are deleted. (Default is: False)

Returns

None

createLabel

Syntax

```
createLabel(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactively creates a label.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

createMask

Syntax

```
createMask(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command for mask creation.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

createNote

Syntax

```
createNote(  
context: deContext | int = None,  
/,  
*,  
shape: ise.createNote.Shape = None)  
Returns: None
```

Summary:

Interactively creates a note.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- context – Context of activation.
- shape – Shape of drawing object ({ arc | circle | ellipse | line | polygon | rectangle })

Returns

None

createOverlayInstanceWithMask

Syntax

```
createOverlayInstanceWithMask(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command for masked instance creation.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

createRuler

Syntax

```
createRuler(  
context: deContext | int = None)  
Returns: None
```

Summary:

Starts the interactive Create Ruler command in the specified symbol or schematic window.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

createSchematicPin

Syntax

```
createSchematicPin(  
    context: deContext | int = None  
Returns: None
```

Summary:

Interactively adds pins to a schematic design.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

createSelectionShape

Syntax

```
createSelectionShape(  
    context: deContext | int = None,  
    /,  
    *,  
    shape: ise.createSelectionShape.Shape = None)  
Returns: None
```

Summary:

Interactive command for selection_shape creation.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- context – Context of activation.
- shape – Shape of selection region ({ polygon | rectangle })

Returns

None

createShape

Syntax

```
createShape(  
context: deContext | int = None,  
//,  
*,  
shape: ise.createShape.Shape = None)  
Returns: None
```

Summary:

Interactive command for shape creation.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- context – Context of activation.
- shape – Shape of drawing object ({ arc | circle | ellipse | line | polygon | rectangle })

Returns

None

createSymbolPin

Syntax

```
createSymbolPin(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactively adds pins to a symbol design.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

createTable

Syntax

```
createTable(  
context: deContext | int = None)  
Returns: None
```

Summary:

Invokes interactive command for schematic table creation.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

createTemplate

Syntax

```
createTemplate(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactively creates a template.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

createWire

Syntax

```
createWire(  
context: deContext | int = None,  
/,  
*,  
type: ise.createWire.Type = None,  
object: deFigure = None,  
point: point = None)  
Returns: None
```

Summary:

Creates a wire between start and end points using the point-to-point router.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- context – Context of activation.
- type – set seWiringType preference, which determines how the wire width should be calculated ({ auto | fat | thin })
- object – Indicates an object from which to define the first point to begin wiring.
- point – Optional reference point to be used to determine a point on given object to begin a wiring

Returns

None

createWireName

Syntax

```
createWireName(  
context: deContext | int = None)  
Returns: None
```

Summary:

Adds a text string and associates it to a wire. This text string represents the net name to which the wire belongs.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

createWireStubs

Syntax

```
createWireStubs(  
context: deContext | int = None)  
Returns: None
```

Summary:

Adds a wire stub

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

cut

Syntax

```
cut(  
context: deContext | int = None,  
/,  
*,  
log: ise.cut.Log = None)  
Returns: None
```

Summary:

Interactive schematic copy selected objects to clipboard.

Category:

Schematic and Symbol Editing

Licenses:

None

Parameters

- context – Context of activation.
- log – Logging mode ({ auto | explicit } (Default is: 'auto'))

Returns

None

delete

Syntax

```
delete(  
context: deContext | int = None,  
/,  
*,  
object: deFigure | oaFig = None)  
Returns: None
```

Summary:

This command deletes the objects in the active selected set of the cellview associated with the specified window.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- context – Context of activation.
- object – Object for exclusive remove

Returns

None

disconnectWires

Syntax

```
disconnectWires(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive command for disconnecting crossing wires.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

ignoreInst

Syntax

```
ignoreInst(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactively ignores instances.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

importSymbol

Syntax

```
importSymbol(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive import symbol command.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

manageLabels

Syntax

```
manageLabels(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Manages visibility of labels.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

move

Syntax

```
move(  
context: deContext | int = None,  
/,  
*,  
object: deFigure | oaFig = None,  
point: point = None)  
Returns: None
```

Summary:

This command moves the objects in the active selected set of the cellview associated with the specified window.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- context – Context of activation.
- object – Object for exclusive move
- point – Specifies the drag start point and enters “one shot mode” where the COT is not updated.

Returns

None

paste

Syntax

```
paste(  
context: deContext | int = None,  
/,  
*,
```

```
clip: deClipboard = None)
Returns: None
```

Summary:

This command pastes the clipboard content into the design of the given window

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- context – Context of activation.
- clip – Clipboard to paste

Returns

None

renameInst

Syntax

```
renameInst (
context: deContext | int = None)
Returns: None
```

Summary:

Interactively renames instances.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

route

Syntax

```
route(  
context: deContext | int = None)  
Returns: None
```

Summary:

Connects two sets of endpoints pin-to-pin or pin-to-bus.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

context – Context of activation.

Returns

None

stretch

Syntax

```
stretch(  
context: deContext | int = None,  
/,  
*,  
object: deFigure | oaFig = None,  
point: point = None)  
Returns: None
```

Summary:

This command stretches the objects in the active selected set of the cellview associated with the specified window.

Category:

Schematic and Symbol Editing

Licenses:

Frontend

Parameters

- context – Context of activation.
- object – Object for exclusive stretch
- point – Specifies the drag start point and enters “one shot mode” where the COT is not updated.

Returns

None

snps.custom.cmd.slr

This chapter describes how to use Python submodules in the Custom Compiler Synopsys Custom command submodule. Also, describes how to use the Synopsys Live Recorder (SLR) Python submodules to start recording Custom Compiler program runs and save the recordings so that you can pinpoint and troubleshoot problems.

For details, see:

- [flush](#)
- [isActive](#)
- [start](#)
- [stop](#)

flush

Syntax

```
flush()  
Returns: None
```

Arguments:

Summary:

Save SLR recording to the specified file.

Category:

Diagnostics

Licenses:

None

Returns

None

isActive

Syntax

```
isActive()  
Returns: bool
```

Arguments:

Summary:

Check SLR is recording or not.

Category:

Diagnostics

Licenses:

None

Returns

SLR is recording or not

start

Syntax

```
start(  
*,  
  logName: str | None = None,  
  delay: int | None = None)  
Returns: None
```

Summary:

Start SLR recording.

Category:

Diagnostics

Licenses:

None

Parameters

- logName – Specify the log file Name.
- delay – Delayed recording for specified seconds (CPU time).

Returns

None

stop

Syntax

```
stop()  
Returns: None
```

Arguments:

Summary:

Stop SLR recording.

Category:

Diagnostics

Licenses:

None

Returns

None

snps.custom.cmd.te

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you edit text views of designs.

For details, see:

- [bindExternalNetlistView](#)
- [getActiveObject](#)
- [getCursor](#)
- [getTokens](#)
- [insertText](#)
- [removeText](#)
- [setCursor](#)

bindExternalNetlistView

Syntax

```
bindExternalNetlistView(  
*,  
    file: str,  
    language: te.bindExternalNetlistView.Language,  
    module: str = None,  
    cellView: dmCellView | dmCellViewName,  
    overwrite: bool = None,  
    virtualTermMapping: list[str] = None,  
    virtualTermsReferenceView: list[str] = None)  
Returns: dmCellView
```

Summary:

Binds the cellview to an external netlist file

Category:

Data Translation

Licenses:

None

Parameters

- file – Path of the file to bind the cellview to
- language – Simulation language of the netlist file ({ CDL | SPF | SPICE | SystemVerilog | VHDL | Verilog | VerilogA | VerilogAMS })
- module – Module name to bind the cellview to. If not specified, the cell name will be used instead

- `cellView` – `dmCellView` to bind to the external netlist.
- `overwrite` – Indicates if the cellview can be overwritten (Default is: `False`)
- `virtualTermMapping` – Terminal to connection definition mapping
- `virtualTermsReferenceView` – Prioritized list of views for determining if a terminal should be virtual or not, any terminal that is not found in a reference view will be created as virtual.

Returns

Updated `CellView`

getActiveObject

Syntax

```
getActiveObject(  
    window: int = None  
Returns: oaBlockObject
```

Summary:

Returns active OA object in Text Editor

Category:

Text Editing

Licenses:

None

Parameters

`window` – Window Id

Returns

Current active OA object

getCursor

Syntax

```
getCursor(  
    window: int = None  
Returns: point
```

Summary:

Returns (row,column) position of cursor

Category:

Text Editing

Licenses:

None

Parameters

window – Window Id

Returns

Current (row,col) position of cursor

getTokens

Syntax

```
getTokens(  
window: int = None,  
/,  
*,  
position: point = None)  
Returns: teToken
```

Summary:

Returns collection of tokens from text

Category:

Text Editing

Licenses:

None

Parameters

- window – Window Id
- position – Row and column in teText window of token to fetch.

Returns

Collection of tokens

insertText

Syntax

```
insertText(  
    string: str,  
    /,  
    *,  
    window: int = None,  
    position: point)  
Returns: None
```

Summary:

Inserts text into teText canvas

Category:

Text Editing

Licenses:

None

Parameters

- string – Text to insert
- window – Window Id
- position – Text to insert into teText canvas.

Returns

None

removeText

Syntax

```
removeText(  
    *,  
    window: int = None,  
    position: point,  
    count: int = None)  
Returns: None
```

Summary:

Removes text from teText canvas

Category:

Text Editing

Licenses:

None

Parameters

- window – Window Id
- position – Starting position for text removal.
- count – Number of characters to remove (Default is: 1)

Returns

None

setCursor

Syntax

```
setCursor(  
window: int = None,  
/,  
*,  
position: point)  
Returns: point
```

Summary:

Set (row,column) position of cursor

Category:

Text Editing

Licenses:

None

Parameters

- window – Window Id
- position – Row and column position to set cursor.

Returns

Set (row,col) position of cursor

snps.custom.cmd.ue

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform UDD editing.

For details, see:

- [cndbg](#)
- [compile](#)
- [getDesign](#)
- [preview](#)
- [searchAndReplace](#)
- [showOptions](#)
- [showSearchReplaceDialog](#)
- [updateCanvas](#)

cndbg

Syntax

```
cndbg (  
*,  
    window: giWindow | int = None)  
Returns: None
```

Summary:

Launch Ciranova debugger cndbg on the UDD pcell.

Category:

Symbolic Editing

Licenses:

None

Parameters

window – Layout window-id for the UDD session

Returns

None

compile

Syntax

```
compile(  
*,  
    window: giWindow | int = None,  
    cellView: dmCellView | dmCellViewName = None)  
Returns: None
```

Summary:

Compiles a python pcell for the UDD design.

Category:

Symbolic Editing

Licenses:

None

Parameters

- window – Layout window-id for the UDD session
- cellView – The cellview to be compiled

Returns

None

getDesign

Syntax

```
getDesign(  
*,  
    window: giWindow | int = None,  
    type: ue.getDesign.Type = None)  
Returns: oaDesign
```

Summary:

Returns the requested design id

Category:

Symbolic Editing

Licenses:

None

Parameters

- `window` – Layout window-id for the UDD session
- `type` – Type of design used in UDD session ({ `preview` | `source` }) (Default is: 'source')

Returns

OA Design Id

preview

Syntax

```
preview(  
    *,  
    window: giWindow | int = None)  
Returns: None
```

Summary:

Preview an instance of python UDD pcell in a separate design.

Category:

Symbolic Editing

Licenses:

None

Parameters

`window` – Layout window-id for the UDD session

Returns

None

searchAndReplace

Syntax

```
searchAndReplace (  
*,  
    window:  giWindow  |  int,  
    value:    str,  
    newValue: str,  
    scope:    ue.searchAndReplace.Scope  =  None)  
Returns: None
```

Summary:

Finds 'value' in all attribute expressions and replaces it with 'newValue'.

Category:

Symbolic Editing

Licenses:

None

Parameters

- window – Layout window-id for the UDD session
- value – The value to be replaced
- newValue – The new value as a replacement
- scope – The scope of substitution ({ global | objects | operations | userParameters }
(Default is: 'global')

Returns

None

showOptions

Syntax

```
showOptions (  
*,  
    window:  giWindow  |  int  =  None)  
Returns: None
```

Summary:

Open the UDD Options dialog.

Category:

Symbolic Editing

Licenses:

None

Parameters

window – Layout window-id for the UDD session

Returns

None

showSearchReplaceDialog

Syntax

```
showSearchReplaceDialog(  
*,  
    window: giWindow | int = None)  
Returns: None
```

Summary:

Open the UDD Search and Replace dialog.

Category:

Symbolic Editing

Licenses:

None

Parameters

window – Layout window-id for the UDD session

Returns

None

updateCanvas

Syntax

```
updateCanvas(  
*,
```

```
window: giWindow | int = None)
Returns: None
```

Summary:

Update canvas as per object attribute defaults.

Category:

Symbolic Editing

Licenses:

None

Parameters

window – Window-id for the UDD session

Returns

None

snps.custom.cmd.iue

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform UDD editing. Interactive commands are listed in this chapter.

For details, see:

- [align](#)
- [boolean](#)
- [copy](#)
- [debug](#)
- [fill](#)
- [move](#)
- [remove](#)
- [setOrigin](#)
- [size](#)
- [stretch](#)

align

Syntax

```
align(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive udd align command

Category:

UDD Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

boolean

Syntax

```
boolean(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive udd boolean command

Category:

UDD Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

copy

Syntax

```
copy(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive udd copy command

Category:

UDD Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

debug

Syntax

```
debug(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive UDD Debugger

Category:

UDD Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

fill

Syntax

```
fill(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive udd fill command

Category:

UDD Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

move

Syntax

```
move(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactive udd move command

Category:

UDD Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

remove

Syntax

```
remove(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive udd remove command

Category:

UDD Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

setOrigin

Syntax

```
setOrigin(  
context: deContext | int = None)  
Returns: None
```

Summary:

Interactive udd setOrigin command

Category:

UDD Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

size

Syntax

```
size(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive udd size command

Category:

UDD Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

stretch

Syntax

```
stretch(  
  context: deContext | int = None)  
Returns: None
```

Summary:

Interactive udd stretch command

Category:

UDD Editing

Licenses:

None

Parameters

context – Context of activation.

Returns

None

snps.custom.cmd.xt

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform netlisting, general Custom Compiler environment based simulation, parasitic extraction, physical verification, and external tool integration.

For details, see:

Custom Compiler

- [addJobClassPrefix](#)
- [addPlotPacket](#)
- [annotateParasitics](#)
- [clearPlotPackets](#)
- [clearViolations](#)
- [closeTextViewerTab](#)
- [configureQueueEngine](#)
- [createCrossRef](#)
- [createDebugSession](#)
- [createIntegration](#)

- [createJob](#)
- [createJobClass](#)
- [createJobGroup](#)
- [createParDBfromQuery](#)
- [createPDKConfig](#)
- [createPlot](#)
- [createPlotQueueItem](#)
- [createPlotViewer](#)
- [createPlotViewerDef](#)
- [createPlotViewerInst](#)
- [createPveJob](#)
- [createSimRun](#)
- [createSimulator](#)
- [createTool](#)
- [crossProbe](#)
- [findExecutable](#)
- [findFeature](#)
- [findFeatureManifest](#)
- [findJobGroup](#)
- [getCallingJob](#)
- [getCrossRefs](#)
- [getDebugSessions](#)
- [getGPDCorners](#)
- [getGPDCouplingCapacitors](#)
- [getGPDGroundCapacitors](#)
- [getGPDLayers](#)
- [getGPDOObjects](#)

- [getGPDRRefs](#)
- [getGPDRResistors](#)
- [getInputFiles](#)
- [getInputFilesTimeStamps](#)
- [getIntegrations](#)
- [getJobClasses](#)
- [getJobClassOptionPrefName](#)
- [getJobGroups](#)
- [getJobs](#)
- [getNetlistInputFiles](#)
- [getOpenFile](#)
- [getOutOfDateFiles](#)
- [getPackages](#)
- [getPDKConfigs](#)
- [getPDKLibs](#)
- [getPveJobs](#)
- [getQueueEngines](#)
- [getSaveFile](#)
- [getSelectedJobGroups](#)
- [getSelectedJobs](#)
- [getSimRuns](#)
- [getSimulators](#)
- [getTextViewerActiveTab](#)
- [getTextViewerTabs](#)
- [getTools](#)
- [getWalkupPort](#)
- [hideJobMonitorColumn](#)

- [highlightViolation](#)
- [initializeQueueEngine](#)
- [initIntegration](#)
- [isValidRunDir](#)
- [launchJobs](#)
- [loadPackage](#)
- [loadPlotSet](#)
- [loadPlugin](#)
- [loadSimRun](#)
- [openFeatureManifest](#)
- [openTextViewer](#)
- [openTextViewerTab](#)
- [processMarkupTags](#)
- [queryParasitics](#)
- [readGPDParasitics](#)
- [readJob](#)
- [refreshTextViewer](#)
- [reloadJobs](#)
- [removeGPDParasitics](#)
- [removeJobClassPrefix](#)
- [reportGPDConfig](#)
- [reportGPDPProperties](#)
- [resetGPDConfig](#)
- [runNetlisterCbk](#)
- [savePlotSet](#)
- [saveSimRun](#)
- [sendCommand](#)

- [setGPDConfig](#)
- [setSaveRequiredValue](#)
- [showInitLoad](#)
- [showInitNew](#)
- [showJobMonitor](#)
- [showNetlistAndSimulate](#)
- [showQueryParasiticsReport](#)
- [showSetCalibreHighlightColorDialog](#)
- [signalJob](#)
- [testPackage](#)
- [unloadPlugin](#)
- [updateJob](#)
- [wait](#)
- [writeJob](#)
- [writeParasitics](#)
- [snps.custom.cmd.xt.plotQueue](#)
- [snps.custom.cmd.xt.utils](#)

addJobClassPrefix

Syntax

```
addJobClassPrefix(  
    prefix: str,  
    /,  
    *,  
    jobClass: xtJobClass)  
Returns: None
```

Summary:

Add a job prefix to the given job class.

Category:

Tool Integration

Licenses:

None

Parameters

- prefix – The job ID prefix to add to the job class
- jobClass – The xtJobClass object

Returns

None

addPlotPacket

Syntax

```
addPlotPacket(  
    packet: str,  
    /,  
    *,  
    position: int | None = None)  
Returns: None
```

Summary:

Adds another display resource packet to the list of plotting packets.

Category:

Netlist Tools

Licenses:

None

Parameters

- packet – The name of the packet to add to the color wheel. It must exist in the default display.
- position – The position to insert this new packet. Negative numbers indicate to append to the end. The default is to append the packet to the end of the list.

Returns

None

annotateParasitics

Syntax

```
annotateParasitics(  
  enable: bool = None,  
  /,  
  *,  
  window: giWindow | int = None)  
Returns: None
```

Summary:

Enable/Disable parasitic annotations in the current context.

Category:

Parasitics

Licenses:

None

Parameters

- enable – Enable viewing of parasitic annotation values to the current cellview (Default is: True)
- window – Schematic design editor window

Returns

None

clearPlotPackets

Syntax

```
clearPlotPackets()  
Returns: None
```

Arguments:

Summary:

Clears out the list of plotting packets.

Category:

Netlist Tools

Licenses:

None

Returns

None

clearViolations

Syntax

```
clearViolations(  
*,  
    design: oaDesign,  
    toolName: str = None)  
Returns: None
```

Summary:

Removes all violation highlights within the specified design, optionally restricted by a given tool name.

Category:

Physical Verification

Licenses:

None

Parameters

- design – Design in which to highlight the violation.
- toolName – Tool which owns this marker.

Returns

None

closeTextViewerTab

Syntax

```
closeTextViewerTab(  
    tabname: str,  
    /,  
    *,  
    window: giWindow | int)  
Returns: None
```

Category:

Summary:

Closes a tab in a text viewer window, given the name of the tab to close.

Licenses:

None

Parameters

- `tabname` – The filename to close
- `window` – The Text Viewer window

Returns

None

configureQueueEngine

Syntax

```
configureQueueEngine(  
    engine: str | xtQueueEngine,  
    /,  
    *,  
    jobClass: str | xtJobClass,  
    parent: giDialog | giWindow | int,  
    submitHost: str = None)  
Returns: None
```

Summary:

Configures queue engine settings for the given job class.

Category:

Tool Integration

Licenses:

None

Parameters

- `engine` – The queue engine to configure.
- `jobClass` – Configure the settings for this `xtJobClass`

- parent – Parent window for the configuration dialog
- submitHost – Configure the settings for this submit host (Default is: 'localhost')

Returns

None

createCrossRef

Syntax

```
createCrossRef(  
*,  
    schematicContext: heContext,  
    parasiticContext: heContext,  
    layoutContext: heContext = None,  
    schematicWindowID: int,  
    parasiticWindowID: int,  
    layoutWindowID: int = None,  
    parasiticTool: str = None,  
    parasiticRunDir: str = None)  
Returns: xtCrossRef
```

Summary:

Create view association for cross-referencing.

Category:

Parasitics

Licenses:

None

Parameters

- schematicContext – Schematic Hierarchical Context
- parasiticContext – Parasitic Hierarchical Context
- layoutContext – Layout Hierarchical Context
- schematicWindowID – Schematic Window ID
- parasiticWindowID – Parasitic Window ID
- layoutWindowID – Layout Window ID (Default is: -1)
- parasiticTool – Parasitic Tool (Default is: "")
- parasiticRunDir – Parasitic Run Directory (Default is: "")

Returns

The new xtCrossRef object

createDebugSession

Syntax

```
createDebugSession(  
*,  
    tool: str | xtTool,  
    job: str | xtJob = None,  
    clientId: str = None,  
    rundir: str = None,  
    cellMap: str = None,  
    layoutTopCellView: dmCellView = None,  
    layoutEditCellView: dmCellView = None,  
    nameMap: nlNameMap = None,  
    schematicTopCellView: dmCellView = None,  
    schematicEditCellView: dmCellView = None)  
Returns: xtDebugSession
```

Summary:

Create an xtDebugSession object representing the state of an active debugger session.

Category:

Physical Verification

Licenses:

None

Parameters

- tool – The debugger tool.
- job – The job object representing the running debugger.
- clientId – Client identifier used by socket based connections to identify the running debugger.
- rundir – Verification results directory.
- cellMap – Verification layout cell to OpenAccess design map.
- layoutTopCellView – The top verification layout cell view.
- layoutEditCellView – The edit verification layout cell view.
- nameMap – Map object used to store schematic to netlist name mappings.

- schematicTopCellView – The top verification schematic cell view.
- schematicEditCellView – The edit verification schematic cell view.

Returns

Newly created xtDebugSession object.

createIntegration

Syntax

```
createIntegration(  
name: str,  
/,  
*,  
type: xt.createIntegration.Type,  
filePath: str)  
Returns: xtIntegration
```

Summary:

Creates and registers a simulator integration by name and returns the integration object

Category:

Netlist Tools

Licenses:

None

Parameters

- name – The name of the integration
- type – The type of integration to be registered, note only simulator is valid.
({ simulator })
- filePath – The full path to TCL source file that defines the integration

Returns

Newly created xtIntegration object

createJob

Syntax

```
createJob(  
name: str,
```

```
/,
*,
type: xt.createJob.Type,
aimType: str = None,
cmdLine: str,
runDesc: str,
files: list[str] = None,
defaultViewFile: str = None,
readOnlyFiles: list[str] = None,
host: str = None,
data: str = None,
readyProc: dbExecObject | str | xtProcedure = None,
dataReadyProc: dbExecObject | str | xtProcedure = None,
menuProc: dbExecObject | str | xtProcedure = None,
exitProc: dbExecObject | str | xtProcedure = None,
killOnExit: bool = None,
queueEngine: str = None,
group: xtJobGroup = None,
deferred: bool = None,
jobClass: xtJobClass = None,
gridOptions: str = None,
**kwargs)
Returns: xtJob
```

Summary:

Start a job.

Category:

Tool Integration

Licenses:

None

Parameters

- name – Prefix for the name to display in the job monitor.
- type – Job type ({ aim | batch | dp | interactive | tclSocket })
- aimType – Aim job type; integrator defined
- cmdLine – Command used to launch the job (Bourne Shell syntax)
- runDesc – Descriptive text to show in the Job Monitor.
- files – Output files to track; list of pairs of (display name, file name)
- defaultViewFile – Filename to set the default active file tab in view output files action. (Default is: “)

- `readOnlyFiles` – Read-Only Output files to track; list of pairs of (display name, file name)
- `host` – Host on which to run the job. (Default is: “)
- `data` – Arbitrary string data associated with the job. (Default is: “)
- `readyProc` – Procedure to call once the job is initialized and ready for input.
- `dataReadyProc` – Procedure to call whenever there is data in the job’s buffer.
- `menuProc` – Procedure to call when the CSM is displayed.
- `exitProc` – Procedure to call once the job has exited.
- `killOnExit` – Indicates the job should be killed when the application shuts down. Default is ‘false’ for batch jobs and ‘true’ for all others.
- `queueEngine` – Specifies which registered queue engine to use to launch the job. If unspecified, it will default to the value of the `xtJobQueueEngine` preference.
- `group` – The job group under which this job is added.
- `deferred` – Defer the job launch. (Default is: False)
- `jobClass` – Job Class to use for the launch settings
- `gridOptions` – Additional grid options (only used for queue managed jobs)

Returns

Job object for communication and tracking

createJobClass

Syntax

```
createJobClass(  
    name: str,  
    /,  
    *,  
    prefName: str = None,  
    labels: list[str] = None)  
Returns: xtJobClass
```

Summary:

Create a job class.

Category:

Tool Integration

Licenses:

None

Parameters

- name – The name of the job class
- prefName – Name to use in preferences
- labels – TCL list of arbitrary string labels for this job class

Returns

xtJobClass object for the job class.

createJobGroup

Syntax

```
createJobGroup(  
name: str,  
/  
*,  
parent: xtJobGroup = None,  
runDesc: str = None,  
detailedStatusProc: dbExecObject | str | xtProcedure = None,  
expanded: bool = None,  
trackProgress: list[bool] = None,  
showProgress: list[bool] = None,  
initOptionsDialogProc: dbExecObject | str | xtProcedure = None,  
exitProc: dbExecObject | str | xtProcedure = None,  
data: str = None,  
**kwargs)  
Returns: xtJobGroup
```

Summary:

Create a job group.

Category:

Tool Integration

Licenses:

None

Parameters

- name – The name of the job group
- parent – The parent job group

- runDesc – Descriptive string (Default is: “)
- detailedStatusProc – Tcl procedure to produce the detailed text message
- expanded – Group should be expanded in the Job Monitor
- trackProgress – Group should track job progress
- showProgress – Group should display job progress
- initOptionsDialogProc – Tcl procedure to initialize the Job Management Options dialog for this filtered group
- exitProc – Procedure to call once all jobs in the group have exited.
- data – Client data to associate with the job group (Default is: “)

Returns

The new xtJobGroup object.

createParDBfromQuery

Syntax

```
createParDBfromQuery(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia],  
/  
*,  
filename: str = None,  
queried: list[str] = None,  
crossRef: xtCrossRef = None,  
eqvNets: list[str] = None)  
Returns: str
```

Summary:

Create database parasitics from query

Category:

Parasitics

Licenses:

None

Parameters

- objects – Collection of parasitic instances.
- filename – file to store db (Default is: 'parQuery.db')
- queried – List of queried nets.
- crossRef – CrossRef containing schematic and parasitic views to be probed
- eqvNets – tcl array of equivalent nets.

Returns

path to db file

createPDKConfig

Syntax

```
createPDKConfig(  
name: str,  
/,  
*,  
lib: oaLib = None,  
libName: str = None,  
fileName: str = None)  
Returns: xtPDKConfig
```

Summary:

Create an xtPDKConfig object encapsulating the PDK configuration data for the specified PDK library.

Category:

Physical Verification

Licenses:

None

Parameters

- name – Name / identifier.
- lib – PDK library.
- libName – PDK library name.
- fileName – PDK configuration file name. (Default is: 'pdk.xml')

Returns

Newly created xtPDKConfig object.

createPlot

Syntax

```
createPlot(  
*,  
    name: str,  
    type: str,  
    analysis: str = None,  
    packet: str)  
Returns: xtPlot
```

Summary:

Creates an xtPlot object

Category:

Netlist Tools

Licenses:

None

Parameters

- name – Full hierarchical name of the property
- type – Type of property being plotted (“net” or “port”)
- analysis – The name of the analysis to which this plot should apply
- packet – The packet name to use when plotting

Returns

Newly created xtPlot object

createPlotQueueItem

Syntax

```
createPlotQueueItem(  
    signal_plot: xtPlot,  
    /,  
    *,  
    command: str,
```

```
mode: xt.createPlotQueueItem.Mode)
Returns: xtPlotQueueItem
```

Summary:

Creates an xtPlotQueueItem object

Category:

Netlist Tools

Licenses:

None

Parameters

- signal_plot – the wfSignal or xtPlot object to plot
- command – the command to plot
- mode – Plotting mode ({ append | new | replace })

Returns

Newly created xtPlotQueueItem

createPlotViewer

Syntax

```
createPlotViewer(  
name: str)  
Returns: xtPlotViewer
```

Summary:

Creates an xtPlotViewer object

Category:

Netlist Tools

Licenses:

None

Parameters

name – Name of the xtPlotViewer

Returns

Newly created xtViewer

createPlotViewerDef

Syntax

```
createPlotViewerDef(  
    name: str)  
Returns: xtPlotViewerDefinition
```

Summary:

Creates an xtPlotViewerDefinition object

Category:

Netlist Tools

Licenses:

None

Parameters

name – Name of the xtPlotViewerDefinition

Returns

Newly created xtViewerDefinition

createPlotViewerInst

Syntax

```
createPlotViewerInst(  
    viewerDef: xtPlotViewerDefinition)  
Returns: xtPlotViewerInst
```

Summary:

Creates an xtPlotViewerInst object

Category:

Netlist Tools

Licenses:

None

Parameters

viewerDef – xtPlotViewerDefinition associated with the instance

Returns

Newly created xtViewerInst

createPveJob

Syntax

```
createPveJob(  
    name: str,  
    /,  
    *,  
    title: str = None,  
    prefId: str = None,  
    jobType: xt.createPveJob.JobType,  
    toolDefault: str | xtTool,  
    toolList: list[str | xtTool] = None,  
    windowTypes: list[giWindowType | str] = None)  
Returns: xtPveJob
```

Summary:

Create an xtPveJob object.

Category:

Physical Verification

Licenses:

None

Parameters

- name – Name / identifier.
- title – Name as displayed in the interface. (Default is: '<name>')
- prefId – Identifier used to encode preference names. (Default is: '<name>')
- jobType – PVE Job type. ({ DFM | DRC | FILL | IDRC | LPE | LVS | LVSLPE | PERC | XACT })
- toolDefault – Default tool used to run the PVE job.
- toolList – List of tools capable of running the PVE job.
- windowTypes – List of window types in which PVE Job related actions are available.

Returns

Newly created xtPveJob object.

createSimRun

Syntax

```
createSimRun(  
*,  
  dir: str,  
  netlistLanguage: str = None,  
  simulator: str,  
  context: deContext = None,  
  cellView: dmCellView = None)  
Returns: xtSimRun
```

Summary:

Creates an xtSimRun object

Category:

Netlist Tools

Licenses:

None

Parameters

- dir – Path of run dir
- netlistLanguage – Netlist language
- simulator – The simulator to be associated with the new xtSimRun
- context – The current design context
- cellView – Top dmCellView associated with the new xtSimRun

Returns

Newly created xtSimRun

createSimulator

Syntax

```
createSimulator(  
  name: str)  
Returns: xtSimulator
```

Summary:

Creates an xtSimulator

Category:

Netlist Tools

Licenses:

None

Parameters

name – the name of the simulator integration

Returns

Newly created xtSimulator

createTool**Syntax**

```
createTool(  
name: str,  
/,  
*,  
title: str = None,  
prefId: str = None,  
namespace: str,  
binary: str = None,  
walkupToken: str = None,  
aliases: list[str] = None)  
Returns: xtTool
```

Summary:

Create an xtTool object.

Category:

Physical Verification

Licenses:

None

Parameters

- name – Name / identifier.
- title – Name as displayed in the interface. (Default is: '<name>')

- `prefId` – Identifier used to encode preference names. (Default is: '<name>')
- `namespace` – Fully qualified Tcl namespace for named interface procedures.
- `binary` – Command line name used to execute the tool. (Default is: '<name>')
- `walkupToken` – Token used to identify the tool during a walk-up Tcl-DP connection.
- `aliases` – Tcl list of alias names consulted for matching user input.

Returns

Newly created `xtTool` object.

crossProbe

Syntax

```
crossProbe(  
*,  
  type:  xt.crossProbe.Type  =  None,  
  object: oaInst | oaNet  =  None,  
  context: heContext  =  None,  
  clear: bool  =  None,  
  crossRef: xtCrossRef,  
  packet: str  =  None,  
  bus: xt.crossProbe.Bus  =  None)  
Returns: deProbe
```

Summary:

Cross probe between parasitic and schematic views using LVS cross-reference information

Category:

Parasitics

Licenses:

None

Parameters

- `type` – Type of probe to create ({ device | net })
- `object` – Net or instance to use to create the crossprobe.
- `context` – Context of the object to be probed.
- `clear` – Specifies whether existing probes should be cleared. (Default is: False)
- `crossRef` – CrossRef containing schematic and parasitic views to be probed

- packet – Display resource packet alias name.
- bus – Bus selection to cross probe a bus net. ({ all | selectBits })

Returns

Collection of probes created in the schematic and/or parasitic views

findExecutable

Syntax

```
findExecutable(  
executable: str,  
/,  
*,  
all: bool | None = None)  
Returns: str
```

Summary:

Finds the absolute path to the named executable form the user's PATH environment variable

Category:

Tool Integration

Licenses:

None

Parameters

- executable – The name of the executable (matched exactly) to be searched for and the absolute path to the file is return if found in the \$PATH.
- all – option to return all paths for named executables when set to true, by default this option is false and only the first absolute path foundis returned. (Default is: False)

Returns

the absolute path to the specified executable.

findFeature

Syntax

```
findFeature(  
feature: str,  
/,
```



```
*,  
in_: xtFeatureManifest)  
Returns: xtFeature
```

Summary:

Finds the named feature in the given xtFeatureManifest.

Category:

Tool Integration

Licenses:

None

Parameters

- feature – the name of the feature to search for.
- in_ – the xtFeatureManifest to search for the given feature.

Returns

returns a xtFeature for the passed feature name.

findFeatureManifest

Syntax

```
findFeatureManifest(  
executable: str,  
/,  
*,  
version: str = None,  
searchPaths: list[str] = None,  
versionCompareProc: variant = None,  
**kwargs)  
Returns: str
```

Summary:

Finds named executable's manifest file, co-located with the executable, by searching through the user's PATH environment variable or by the nearest matching version manifest in a given search path.

Category:

Tool Integration

Licenses:

None

Parameters

- `executable` – The name of the executable (matched exactly) to be searched for in the user's `$PATH`.
- `version` – The current version of the executable.
- `searchPaths` – The list of paths to search for manifest files with names in the format `<executable>.<version>.manifest`.
- `versionCompareProc` – The Tcl proc to use to compare version strings (must work like Tcl's string compare).

Returns

The absolute path to the manifest file corresponding to the given executable name.

findJobGroup

Syntax

```
findJobGroup(  
  name: str,  
  /,  
  *,  
  parent: xtJobGroup = None)  
Returns: xtJobGroup
```

Summary:

Find a job group by name (does not descend the hierarchy)

Category:

Tool Integration

Licenses:

None

Parameters

- `name` – Name of the job group
- `parent` – Parent to search

Returns

The found job group.

getCallingJob

Syntax

```
getCallingJob()  
Returns: xtJob
```

Arguments:

Summary:

Get the job object for the current externally invoked command. Returns "" if not called from an externally invoked command

Category:

Tool Integration

Licenses:

None

Returns

Calling job object

getCrossRefs

Syntax

```
getCrossRefs(  
*,  
    window: int = None)  
Returns: dbCollection
```

Summary:

Return a list of xtCrossRef objects that currently exist matching specified criteria.

Category:

Parasitics

Licenses:

None

Parameters

window – Get xtCrossRef objects associated with window.

Returns

xtCrossRef objects matching search criteria

getDebugSessions

Syntax

```
getDebugSessions(  
    pattern: str = None)  
Returns: dbCollection
```

Summary:

Get all registered xtDebugSession objects.

Category:

Physical Verification

Licenses:

None

Parameters

pattern – Pattern to match object job name/client ID. Empty pattern matches all.

Returns

xtDebugSession objects matching search criteria.

getGPDCorners

Syntax

```
getGPDCorners(  
    *,  
    gpd: str)  
Returns: str
```

Summary:

This command will return a string of corners for the specified GPD.

Category:

Parasitics

Licenses:

None

Parameters

gpd – Specifies the path to GPD directory

Returns

Corners for the specified GPD.

getGPDCouplingCapacitors

Syntax

```
getGPDCouplingCapacitors(  
*,  
  of: str | None = None,  
  from_: str | None = None,  
  to: str | None = None,  
  corners: str | None = None)  
Returns: str
```

Summary:

This command will return a string of coupling capacitors for the given nets or nodes on the command line.

Category:

Parasitics

Licenses:

None

Parameters

- of – Reports coupling capacitors associated with the specified nets
- from_ – Constrains the selected capacitors to be part of a path starting from the specified node
- to – Constrains the selected capacitors to be part of a path ending with the specified node
- corners – Indicates the parasitic corner ordering for coupling capacitance values

Returns

Coupling capacitors for the given nets or nodes.

getGPDGroundCapacitors

Syntax

```
getGPDGroundCapacitors(  
*,  
  of: str | None = None,  
  from_: str | None = None,  
  to: str | None = None,  
  corners: str | None = None)  
Returns: str
```

Summary:

This command will return a string of ground capacitors for the given nets or nodes on the command line.

Category:

Parasitics

Licenses:

None

Parameters

- of – Reports ground capacitors associated with the specified nets
- from_ – Constrains the selected capacitors to be part of a path starting from the specified node
- to – Constrains the selected capacitors to be part of a path ending with the specified node
- corners – Indicates the parasitic corner ordering for ground capacitance values

Returns

Ground capacitors for the given nets or nodes.

getGPDLayers

Syntax

```
getGPDLayers(  
*,  
  gpd: str,  
  type: xt.getGPDLayers.Type = None)  
Returns: str
```

Summary:

This command will return a string of layers for the specified GPD.

Category:

Parasitics

Licenses:

None

Parameters

- `gpd` – Specifies the path to GPD directory
- `type` – Specifies the type of layers to report ({ `all` | `routing` | `via` })

Returns

Layers for the specified GPD.

getGPDOObjects

Syntax

```
getGPDOObjects(  
*,  
  cell: str | None = None,  
  net: str | None = None,  
  port: str | None = None,  
  pin: str | None = None,  
  exact: bool | None = None,  
  verbose: bool | None = None)  
Returns: str
```

Summary:

This command will return a string of objects for the given type.

Category:

Parasitics

Licenses:

None

Parameters

- `cell` – Specifies pattern for cell object query
- `net` – Specifies pattern for net object query

- port – Specifies pattern for port object query
- pin – Specifies pattern for pin object query
- exact – Disables wildcard pattern matching
- verbose – Indicates report detailed information

Returns

GPD object names for the specified type.

getGPDRefs

Syntax

```
getGPDRefs(  
*,  
    lcvName: str | None = None,  
    **kwargs)  
Returns: str
```

Summary:

Return the associated Golden Parasitic Database that currently exist.

Category:

Parasitics

Licenses:

None

Parameters

lcvName – Lib Cell View Name (Default is: “)

Returns

The associated Golden Parasitic Database.

getGPDResistors

Syntax

```
getGPDResistors(  
*,  
    of: str | None = None,  
    from_: str | None = None,  
    to: str | None = None,  
    corners: str | None = None,
```



```
geometry: bool | None = None)
Returns: str
```

Summary:

This command will return a string of resistors for the given nets or nodes on the command line.

Category:

Parasitics

Licenses:

None

Parameters

- `of` – Reports resistors associated with the specified nets
- `from_` – Constrains the selected resistors to be part of a path starting from the specified node
- `to` – Constrains the selected resistors to be part of a path ending with the specified node
- `corners` – Indicates the parasitic corner ordering for ground capacitance values
- `geometry` – Reports geometry information of resistor

Returns

Resistors for the given nets or nodes.

getInputFiles

Syntax

```
getInputFiles (
*,
file: str)
Returns: str
```

Summary:

Recursively scans the named file and any included files to form a list of all input file dependencies

Category:

Netlist Tools

Licenses:

None

Parameters

file – The name of the file to scan

Returns

List of input files

getInputFilesTimeStamps

Syntax

```
getInputFilesTimeStamps (  
    *,  
    file: str)  
Returns: str
```

Summary:

Recursively scans the named file and any included files to form a time-stamped list of all input file dependencies

Category:

Netlist Tools

Licenses:

None

Parameters

file – The name of the file to scan

Returns

List of input file/timestamp pairs

getIntegrations

Syntax

```
getIntegrations (  
    string: str = None)  
Returns: xtIntegration
```

Summary:

Returns the registered Integrations

Category:

Netlist Tools

Licenses:

None

Parameters

string – Glob style regular expression used to match existing integration names.

Returns

Collection of all integrations matching the given filter

getJobClasses

Syntax

```
getJobClasses(  
    pattern: str = None)  
Returns: dbCollection
```

Summary:

Return a collection of the registered job classes.

Category:

Tool Integration

Licenses:

None

Parameters

pattern – Pattern to match against the object names. Empty pattern matches all.

Returns

The collection of registered job classes.

getJobClassOptionPrefName

Syntax

```
getJobClassOptionPrefName(  
    jobClass: str | xtJobClass,  
    /,  
    *,  
    option: str)  
Returns: str
```

Summary:

Get the preference name of the given job class option.

Category:

Tool Integration

Licenses:

None

Parameters

- jobClass – The xtJobClass object
- option – The option name

Returns

Name of the preference containing the given option

getJobGroups

Syntax

```
getJobGroups(  
    pattern: str = None,  
    /,  
    *,  
    toplevel: bool = None)  
Returns: dbCollection
```

Summary:

Return a collection of the registered job groups.

Category:

Tool Integration

Licenses:

None

Parameters

- pattern – Pattern to match against the object names. Empty pattern matches all.
- toplevel – Flag to request only top-level groups. (Default is: False)

Returns

The collection of registered job groups.

getJobs

Syntax

```
getJobs(  
pattern: str = None,  
/,  
*,  
group: xtJobGroup = None)  
Returns: dbCollection
```

Summary:

Return a list of jobs currently known to the job manager, subject to the filter which may have been provided.

Category:

Tool Integration

Licenses:

None

Parameters

- pattern – Pattern to match the job names against. Empty pattern matches all jobs. (Default is: "")
- group – Group whose jobs to return.

Returns

A collection of xtJobs matching the filter criteria

getNetlistInputFiles

Syntax

```
getNetlistInputFiles(  
    simRun: xtSimRun)  
Returns: str
```

Summary:

Category:

Netlist Tools

Licenses:

None

Parameters

simRun – xtSimRun object

Returns

Names of all files which are referenced through netlist template

getOpenFile

Syntax

```
getOpenFile(  
    *,  
    title: str = None,  
    multiple: bool = None,  
    fileTypes: list[str] = None,  
    initialDir: str = None,  
    initialFile: str = None,  
    directory: bool = None,  
    window: gi.Window | int = None)  
Returns: None
```

Category:

Summary:

Opens a file selection dialog to select an existing file or directory.

Licenses:

None

Parameters

- title – File selector dialog title
- multiple – Allow multiple selection
- fileTypes – File types filters. For example, { “Images (.png *.jpg *.jpeg)” “Text files (.txt)” “All files (*)” }
- initialDir – Initial directory
- initialFile – Initial file
- directory – Toggle directory selection
- window – Window Id

Returns

None

getOutOfDateFiles

Syntax

```
getOutOfDateFiles(  
*,  
    files: str,  
    timestamps: str)  
Returns: str
```

Summary:

Given the list of input files and timestamps, this command will check each file on disk against the associated time stamp. Each out of date file will be returned as a list of files. Any file not found in the list of timestamps is assumed to be out of date.

Category:

Netlist Tools

Licenses:

None

Parameters

- files – The name of the file to check time stamps
- timestamps – -timestamps

Returns

Names of all files changed relative to timestamps passed in

getPackages

Syntax

```
getPackages(  
    pattern: str = None  
Returns: dbCollection
```

Summary:

Returns a collection of packages found in \$PATH, optionally filtered by a glob-style pattern matched against the package name

Category:

Foundation

Licenses:

None

Parameters

pattern – Glob-style pattern matched against the package name. (Default is: '*')

Returns

Collection of xtPackages

getPDKConfigs

Syntax

```
getPDKConfigs(  
    pattern: str = None  
Returns: dbCollection
```

Summary:

Get all registered xtPDKConfig objects.

Category:

Physical Verification

Licenses:

None

Parameters

pattern – Pattern to match object name.

Returns

xtPDKConfig objects matching search criteria.

getPDKLibs

Syntax

```
getPDKLibs()  
Returns: list[oaLib]
```

Arguments:

Summary:

Returns a list of all oaLib objects designated as a PDK library.

Category:

Physical Verification

Licenses:

None

Returns

A collection of PDK libraries

getPveJobs

Syntax

```
getPveJobs(  
    pattern: str = None)  
Returns: dbCollection
```

Summary:

Get all registered xtPveJob objects.

Category:

Physical Verification

Licenses:

None

Parameters

pattern – Pattern to match object name.

Returns

xtPveJob objects matching search criteria.

getQueueEngines

Syntax

```
getQueueEngines(  
    pattern: str = None)  
Returns: dbCollection
```

Summary:

Returns the currently registered queue engines.

Category:

Tool Integration

Licenses:

None

Parameters

pattern – Pattern to match against the object names. Empty pattern matches all.

Returns

Collection of registered objects.

getSaveFile

Syntax

```
getSaveFile(  
    *,  
    title: str = None,  
    fileTypes: list[str] = None,  
    initialDir: str = None,  
    initialFile: str = None,  
    window: gi.Window | int = None,
```

```
confirmOverwrite: bool = None)  
Returns: None
```

Summary:

Opens a file selection dialog to select a filename for saving.

Category:

Netlist Tools

Licenses:

None

Parameters

- title – File selector dialog title
- fileTypes – File types filters. For example, { “Images (.png *.jpg *.jpeg)” “Text files (.txt)” “All files (*)” }
- initialDir – Initial directory
- initialFile – Initial file
- window – Window Id
- confirmOverwrite – Enables optional popup overwrite confirmation dialog for existing files (Default is: True)

Returns

None

getSelectedJobGroups

Syntax

```
getSelectedJobGroups(  
window: giWindow | int,  
/  
*,  
jobSummary: bool = None)  
Returns: dbCollection
```

Summary:

Return a list of groups selected in the given Job Monitor window.

Category:

Tool Integration

Licenses:

None

Parameters

- window – A Job Monitor window or id.
- jobSummary – If it is triggered from job summary. (Default is: False)

Returns

Collection of Groups selected in the given Job Monitor window

getSelectedJobs

Syntax

```
getSelectedJobs(  
window:  giWindow  |  int,  
/,  
*,  
jobSummary: bool = None)  
Returns: dbCollection
```

Summary:

Return a list of jobs selected in the given Job Monitor window.

Category:

Tool Integration

Licenses:

None

Parameters

- window – A Job Monitor window or id.
- jobSummary – If it is triggered from job summary. (Default is: False)

Returns

Collection of Jobs selected in the given Job Monitor window

getSimRuns

Syntax

```
getSimRuns (  
*,  
    context: deContext = None)  
Returns: xtSimRun
```

Summary:

Returns the registered SimRuns

Category:

Netlist Tools

Licenses:

None

Parameters

context – The design context associated with an xtSimRun

Returns

Collection of xtSimRun objects matching filter, or all objects

getSimulators

Syntax

```
getSimulators (  
string: str = None)  
Returns: xtSimulator
```

Summary:

Returns the registered xtSimulator(s)

Category:

Netlist Tools

Licenses:

None

Parameters

string – Glob style regular expression used to match existing simulator names.

Returns

Collection of xtSimulator objects matching the given expression

getTextViewerActiveTab

Syntax

```
getTextViewerActiveTab(  
    *,  
    window: giWindow | int)  
Returns: str
```

Category:

Summary:

Gets the name of the current tab

Licenses:

None

Parameters

window – The Text Viewer window

Returns

Name of the active tab

getTextViewerTabs

Syntax

```
getTextViewerTabs(  
    pattern: str = None,  
    /,  
    *,  
    window: giWindow | int)  
Returns: str
```

Category:

Summary:

Get a list of all tab indices optionally filtered using a pattern

Licenses:

None

Parameters

- pattern – The pattern to match against
- window – The Text Viewer window

Returns

List of tab names matching the pattern

getTools

Syntax

```
getTools(  
pattern: str = None)  
Returns: dbCollection
```

Summary:

Get all registered xtTool objects.

Category:

Physical Verification

Licenses:

None

Parameters

pattern – Pattern to match object name.

Returns

xtTool objects matching search criteria.

getWalkupPort

Syntax

```
getWalkupPort()  
Returns: int
```

Arguments:

Summary:

Returns the port number that Custom Designer is listening on for incoming walkup connections.

Category:

Tool Integration

Licenses:

None

Returns

The port number that Customer Designer is listening on for incoming walkup connections.

hideJobMonitorColumn

Syntax

```
hideJobMonitorColumn(  
    window: giWindow | int,  
    /,  
    *,  
    column: xt.hideJobMonitorColumn.Column,  
    hide: bool = None)  
Returns: None
```

Summary:

Hide or show a table column in a Job Monitor window.

Category:

Tool Integration

Licenses:

None

Parameters

- window – A Job Monitor window or id.
- column – The column to show or hide. ({ creation_time | end_time | host | job | output_files | process_id | run_description | start_time | status | type })
- hide – Hide the column (true) or show it (false). (Default is: True)

Returns

None

highlightViolation

Syntax

```
highlightViolation(  
  points: list[point],  
  /,  
  *,  
  design: oaDesign,  
  isClosed: bool,  
  toolName: str,  
  packet: str = None)  
Returns: None
```

Summary:

Creates a temporary violation shape for the specified design

Category:

Physical Verification

Licenses:

None

Parameters

- points – List of points that forms the polygon.
- design – Design in which to highlight the violation.
- isClosed – Indicates whether the point array is a close polygon or an open polyline.
- toolName – Tool which owns this marker.
- packet – Packet or packet alias name defining how this violation should be drawn.
(Default is: 'markerErr')

Returns

None

initializeQueueEngine

Syntax

```
initializeQueueEngine(  
  engine: str | xtQueueEngine)  
Returns: bool
```

Summary:

Initializes a queue engine for use.

Category:

Tool Integration

Licenses:

None

Parameters

engine – The queue engine to initialize.

Returns

Initialization result

initIntegration**Syntax**

```
initIntegration(  
    name: str,  
    /,  
    *,  
    type: xt.initIntegration.Type)  
Returns: None
```

Summary:

Command loads all code for the named integration

Category:

Netlist Tools

Licenses:

None

Parameters

- name – The name of the simulator integration
- type – The type of integration to be registered, note only simulator is valid.
({ simulator })

Returns

None

isValidRunDir

Syntax

```
isValidRunDir(  
  rundir: str)  
Returns: bool
```

Summary:

Check if the specified directory to is a complete and valid rundirectory. Return true if valid, false otherwise.

Category:

Netlist Tools

Licenses:

None

Parameters

rundir – path to the runDir

Returns

True if the named directory is a valid run directory

launchJobs

Syntax

```
launchJobs(  
  jobs: dbCollection,  
  /,  
  *,  
  arrayName: str = None)  
Returns: None
```

Summary:

Launch deferred jobs.

Category:

Tool Integration

Licenses:

None

Parameters

- jobs – The jobs to launch
- arrayName – Queue engine job array name

Returns

None

loadPackage

Syntax

```
loadPackage(  
  package: list[str | xtPackage])  
Returns: None
```

Summary:

Loads a package

Category:

Foundation

Licenses:

None

Parameters

package – Package name or object to load.

Returns

None

loadPlotSet

Syntax

```
loadPlotSet(  
  *,  
  filePath: str)  
Returns: xtPlot
```

Summary:

Load the plot set from the specified file

Category:

Netlist Tools

Licenses:

None

Parameters

filePath – name of file to load plot set from

Returns

Collection of xtPlot object which were loaded from the file

loadPlugin

Syntax

```
loadPlugin(  
  filePath: str  
Returns: None
```

Summary:

Loads a plugin

Category:

Foundation

Licenses:

None

Parameters

filePath – Path to the shared object to load.

Returns

None

loadSimRun

Syntax

```
loadSimRun(  
  *,  
  dir: str,  
  context: deContext = None,
```

```
cellView: dmCellView = None)  
Returns: xtSimRun
```

Summary:

Loads an xtSimRun object from an existing run directory

Category:

Netlist Tools

Licenses:

None

Parameters

- dir – Path of run dir
- context – The design context to be associated with the xtSimRun
- cellView – Top dmCellView associated with the xtSimRun

Returns

Data object associated with the newly created run directory

openFeatureManifest

Syntax

```
openFeatureManifest(  
filePath: str)  
Returns: xtFeatureManifest
```

Summary:

Reads the manifest file specified by the absolute file path and returns an xtFeatureManifest object.

Category:

Tool Integration

Licenses:

None

Parameters

filePath – the absolute path to a manifest file to be read.

Returns

the xtManifestFeature object for the specified manifest file or an error is thrown.

openTextViewer

Syntax

```
openTextViewer(  
*,  
    text: str = None,  
    files: list[str] = None,  
    titles: list[str] = None,  
    includePaths: list[str] = None)  
Returns: giWindow
```

Category:

Summary:

Opens a text viewer window, displaying the given files

Licenses:

None

Parameters

- text – List of strings to view
- files – List of filenames to open
- titles – List of tab titles to assign
- includePaths – List of include directory paths to search

Returns

The new Text Viewer window

openTextViewerTab

Syntax

```
openTextViewerTab(  
*,  
    filePath: str = None,  
    text: str = None,  
    window: giWindow | int)  
Returns: str
```

Category:

Summary:

Opens a file in a new tab in an existing text viewer window.

Licenses:

None

Parameters

- `filePath` – The filename to open
- `text` – The string to display in the tab
- `window` – The Text Viewer window

Returns

The name of the new tab

processMarkupTags

Syntax

```
processMarkupTags (  
*,  
  inFile: str,  
  outFile: str,  
  map: nlNameMap | nlOANameMap | nlParameterizedNameMap,  
  direction: xt.processMarkupTags.Direction = None,  
  workingDir: str = None)  
Returns: None
```

Summary:

Recursively process the named netlist template for all supported markup tags.

Category:

Netlist Tools

Licenses:

None

Parameters

- `inFile` – The name of the input file to be processed
- `outFile` – The name of the output file to output results. This is relative to the `workingDir` unless it's an absolute path.

- **map** – Name map object to be queried or modified
- **direction** – If set to “Forward”, map names using the -src argument. If set to “Reverse”, map names using the -dst argument. ({ Forward | Reverse } (Default is: ‘Forward’))
- **workingDir** – The directory for output, for SAE iterative runs (Default is: “)

Returns

None

queryParasitics

Syntax

```
queryParasitics(  
*,  
    type: xt.queryParasitics.Type = None,  
    nets: list[oaNet] = None,  
    from_: oaInstTerm | oaNet | oaRect = None,  
    of: oaInstTerm | oaNet | oaRect = None,  
    to: oaInstTerm | oaNet | oaRect = None,  
    context: heContext = None,  
    fromContext: heContext = None,  
    toContext: heContext = None,  
    crossRef: xtCrossRef = None,  
    queryR: bool = None,  
    queryCc: bool = None,  
    queryCg: bool = None,  
    queryL: bool = None,  
    thresholdR: str = None,  
    thresholdCc: str = None,  
    thresholdCg: str = None,  
    thresholdL: str = None,  
    queryRFilter: bool = None,  
    corner: str = None)  
Returns: list[oaInst]
```

Summary:

This command will return a collection of parasitic instances associated with the nets or terminals given on the command line.

Category:

Parasitics

Licenses:

None

Parameters

- **type** – Type of query to perform ({ net | netToNet | pointToPoint })
- **nets** – Collection of nets to query. (Default is: "")
- **of** – Net to use as first net for coupled parasitics or terminal to use as path starting point for parasitics between two points.
- **to** – Net to use as second net for coupled parasitics or terminal to use as path ending point for parasitics between two points.
- **context** – heContext in which the nets occur (use with -nets option)
- **fromContext** – heContext in which the first net/instTerm occurs
- **toContext** – heContext in which the second net/instTerm occurs
- **crossRef** – CrossRef containing schematic and parasitic views to be probed
- **queryR** – Include parasitic resistors in the query.
- **queryCc** – Include parasitic coupled capacitors in the query.
- **queryCg** – Include parasitic grounded capacitors in the query.
- **queryL** – Include parasitic inductors in the query.
- **thresholdR** – Threshold value below which parasitic resistors are ignored
- **thresholdCc** – Threshold value below which parasitic coupled capacitors are ignored
- **thresholdCg** – Threshold value below which parasitic grounded capacitors are ignored
- **thresholdL** – Threshold value below which parasitic inductors are ignored
- **queryRFilter** – Filter parasitic resistors by the threshold value.
- **corner** – Corner map value to calculate the property value of parasitics

Returns

Collection of parasitic instances associated with the given query

readGPDParasitics

Syntax

```
readGPDParasitics(  
*,  
  read: str | None = None,  
  current: bool | None = None,
```

```
keep_capacitive_coupling: bool | None = None)
Returns: str
```

Summary:

This command will read GPD parasitics data with the specified arguments.

Category:

Parasitics

Licenses:

None

Parameters

- read – Read GPD from specified path
- current – Return current GPD directory
- keep_capacitive_coupling – Indicates that the cross capacitors are to be kept in the RC networks data structure

Returns

Result of GPD data.

readJob

Syntax

```
readJob(
    job: xtJob,
    /,
    *,
    timeout: int = None)
Returns: None
```

Summary:

Read stdout from an interactive job.

Category:

Tool Integration

Licenses:

None

Parameters

- job – The job to read data from.
- timeout – Time, in seconds, to wait for a response (return value) to the command. Timeout of 0 will return immediately. (Default is: 0)

Returns

None

refreshTextViewer

Syntax

```
refreshTextViewer(  
window: giWindow | int)  
Returns: None
```

Category:

Summary:

Refresh the text viewer contents from their sources.

Licenses:

None

Parameters

window – The Text Viewer window

Returns

None

reloadJobs

Syntax

```
reloadJobs(  
filePath: str | None = None)  
Returns: None
```

Summary:

Reload the jobs list from disk

Category:

Tool Integration

Licenses:

None

Parameters

filePath – File from which to load jobs into the job manager. (Default is: “)

Returns

None

removeGPDParasitics

Syntax

```
removeGPDParasitics()  
Returns: str
```

Arguments:

Summary:

This command will remove GPD parasitics data.

Category:

Parasitics

Licenses:

None

Returns

Result of removing GPD.

removeJobClassPrefix

Syntax

```
removeJobClassPrefix(  
    prefix: str,  
    /,  
    *,  
    jobClass: xtJobClass)  
Returns: None
```

Summary:

Remove a job prefix from the given job class.

Category:

Tool Integration

Licenses:

None

Parameters

- `prefix` – The prefix to remove from the job class
- `jobClass` – The `xtJobClass` object

Returns

None

reportGPDConfig**Syntax**

```
reportGPDConfig(  
*,  
  gpd: str,  
  type: xt.reportGPDConfig.Type = None,  
  includeStarRCOptions: bool = None)  
Returns: str
```

Summary:

This command will return a string of configurations for the specified GPD.

Category:

Parasitics

Licenses:

None

Parameters

- `gpd` – Specifies the path to GPD directory
- `type` – Specifies the type of GPD configurations to report ({ `active` | `original` | `user_defined` })
- `includeStarRCOptions` – Includes StarRC GPD configuration options in the report

Returns

Configurations for the specified GPD.

reportGPDPProperties

Syntax

```
reportGPDPProperties(  
*,  
  gpd: str,  
  all: bool | None = None,  
  layers: bool | None = None,  
  corners: bool | None = None)  
Returns: str
```

Summary:

This command will return a string of properties for the specified GPD.

Category:

Parasitics

Licenses:

None

Parameters

- gpd – Specifies the path to GPD directory
- all – Reports all types of information
- layers – Reports only the layers
- corners – Reports only the parasitic corners

Returns

Properties for the specified GPD.

resetGPDConfig

Syntax

```
resetGPDConfig(  
*,  
  gpd: str)  
Returns: str
```

Summary:

This command will reset configurations for the specified GPD.

Category:

Parasitics

Licenses:

None

Parameters

gpd – Specifies the path to GPD directory

Returns

None

runNetlisterCbk**Syntax**

```
runNetlisterCbk(  
    simRun: xtSimRun,  
    /,  
    *,  
    generateFinal: bool = None,  
    generateStructural: bool = None)  
Returns: None
```

Summary:

Builds the structural netlist invoking the netlister.

Category:

Netlist Tools

Licenses:

None

Parameters

- simRun – xtSimRun object
- generateFinal – Create the final netlist. (Default is: True)
- generateStructural – Create the structural netlist. (Default is: True)

Returns

None

savePlotSet

Syntax

```
savePlotSet (  
*,  
  plots: list[xtPlot],  
  filePath: str)  
Returns: None
```

Summary:

Save the plot set to the specified file

Category:

Netlist Tools

Licenses:

None

Parameters

- plots – list of plots to save
- filePath – name of file where plot set will be stored

Returns

None

saveSimRun

Syntax

```
saveSimRun (  
simRun: xtSimRun)  
Returns: None
```

Summary:

Save the sim_run data to disk.

Category:

Netlist Tools

Licenses:

None

Parameters

simRun – xtSimRun object

Returns

None

sendCommand

Syntax

```
sendCommand(  
    command: str,  
    /,  
    *,  
    job: xtJob,  
    timeout: int = None)  
Returns: str
```

Summary:

Send a Tcl command to a job.

Category:

Tool Integration

Licenses:

None

Parameters

- command – Command for the interactive job to execute.
- job – The job to execute the command.
- timeout – Time, in seconds, to wait for a response (return value) to the command. Timeout of 0 will return immediately. (Default is: 0)

Returns

Return value of the command which was sent to the interactive job

setGPDConfig

Syntax

```
setGPDConfig(  
*,  
  gpd: str,  
  absoluteCouplingThreshold: float = None,  
  relativeCouplingThreshold: float = None,  
  couplingThresholdOperation: xt.setGPDConfig.CouplingThresholdOperation  
  = None,  
  netlistSelectNets: str = None,  
  netlistType: str = None)  
Returns: str
```

Summary:

This command will set configurations for the specified GPD.

Category:

Parasitics

Licenses:

None

Parameters

- gpd – Specifies the path to GPD directory
- absoluteCouplingThreshold – Specifies the absolute coupling threshold for filtering coupling capacitors. A capacitor having less than this capacitance value is split into two capacitors to ground.
- relativeCouplingThreshold – Includes StarRC GPD configuration options in the report
- couplingThresholdOperation – Specifies the type of GPD configurations to report ({ and | or })
- netlistSelectNets – Specifies a subset of nets to be read from the GPD database
- netlistType – Specifies the parasitics network form to consider when reading the GPD. The allowed settings are no_couple, R, CG, CC, RCG, and RCC

Returns

None

setSaveRequiredValue

Syntax

```
setSaveRequiredValue(  
*,  
    value: bool)  
Returns: None
```

Summary:

Set attr of job manager used to determine if jobs information file required to be saved.

Category:

Tool Integration

Licenses:

None

Parameters

value – Value to be set for attr saveRequired.

Returns

None

showInitLoad

Syntax

```
showInitLoad(  
window: int | None = None)  
Returns: None
```

Summary:

Shows the initialized load dialog

Category:

Netlist Tools

Licenses:

None

Parameters

window – Window Id

Returns

None

showInitNew

Syntax

```
showInitNew(  
    window: int | None = None)  
Returns: None
```

Summary:

Shows the initialized new dialog

Category:

Netlist Tools

Licenses:

None

Parameters

window – Window Id

Returns

None

showJobMonitor

Syntax

```
showJobMonitor(  
    *,  
    group: xtJobGroup = None)  
Returns: giWindow
```

Summary:

Shows the job monitor window.

Category:

Tool Integration

Licenses:

None

Parameters

group – The job group to display in the job monitor.

Returns

The Job Monitor window

showNetlistAndSimulate

Syntax

```
showNetlistAndSimulate(  
window: int | None = None)  
Returns: None
```

Summary:

Shows the netlist and simulation dialog

Category:

Netlist Tools

Licenses:

None

Parameters

window – Window Id

Returns

None

showQueryParasiticsReport

Syntax

```
showQueryParasiticsReport(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
| list[oaVia],  
/,  
*,  
title: str = None,  
window: giWindow | int,  
corner: str = None,  
point1: str = None,
```

```
point2: str = None)
Returns: giDialog
```

Summary:

Open the Query Parasitics Report GUI dialog.

Category:

Parasitics

Licenses:

None

Parameters

- objects – Collection of parasitic instances.
- title – Title to display for the results table. (Default is: 'Parasitic Query Results')
- window – Parent window
- corner – Corner map value to calculate the property value of parasitics
- point1 – Instance terminal or sub net among the collection of parasitic instances to use as the 'Starting' point for calculating equivalent resistance.
- point2 – Instance terminal or sub net among the collection of parasitic instances to use as the 'Ending' point for calculating equivalent resistance.

Returns

The Dialog

showSetCalibreHighlightColorDialog

Syntax

```
showSetCalibreHighlightColorDialog()
Returns: None
```

Arguments:

Summary:

Show "Set Calibre Highlight Color" dialog

Category:

Physical Verification

Licenses:

None

Returns

None

signalJob

Syntax

```
signalJob(  
  job: dbCollection | xtJob,  
  /,  
  *,  
  signal: xt.signalJob.Signal)  
Returns: None
```

Summary:

Either suspend, resume, or kill the remote job depending on the supplied arguments. This is not the same as sending a UNIX signal to the process.

Category:

Tool Integration

Licenses:

None

Parameters

- job – The Job, or collection of jobs to send the signal to.
- signal – Signal to send to the process. ({ forcekill | kill | resume | suspend })

Returns

None

testPackage

Syntax

```
testPackage(  
  filePath: str)  
Returns: bool
```


Summary:

Tests feature manifest file and prints warning or error messages as it tests the specified file.

Category:

Foundation

Licenses:

None

Parameters

filePath – Package manifest file to test.

Returns

Return true if warnings or errors are discovered, false otherwise.

unloadPlugin

Syntax

```
unloadPlugin(  
    filePath: str)  
Returns: None
```

Summary:

Loads a plugin

Category:

Foundation

Licenses:

None

Parameters

filePath – Path to the shared object to unload.

Returns

None

updateJob

Syntax

```
updateJob(  
  none: xtJob,  
  /,  
  *,  
  status: xt.updateJob.Status = None,  
  isQueued: bool = None,  
  host: str = None,  
  procId: int = None,  
  startTime: int = None,  
  endTime: int = None,  
  exitStatus: int = None,  
  progress: str = None)  
Returns: None
```

Summary:

Allows a queue engine integrator to update an xtJob during the queue engine statusProc callback. This command can only be called within the statusProc callback for an xtQueueEngine. It will throw an error if it is called at any other time.

Category:

Tool Integration

Licenses:

None

Parameters

- none – The job to update.
- status – The updated status of the job. ({ FAILED | FINISHED | KILLED | RUNNING | SUSPENDED | UNKNOWN })
- isQueued – Indicates if the job is currently queued by the queuing software.
- host – The host where the job is running
- procId – The PID of the job.
- startTime – The time that the job started in the form of seconds since the epoch (see Tcl's "clock scan" command).
- endTime – The time that the job finished in the form of seconds since the epoch (see Tcl's "clock scan" command).

- `exitStatus` – The exit code of the job.
- `progress` – The progress display of the job.

Returns

None

wait

Syntax

```
wait(  
  target: dbCollection | script | xtJob)  
Returns: str
```

Summary:

Wait for a job to finish.

Category:

Tool Integration

Licenses:

None

Parameters

`target` – The jobs to wait for, or script that creates jobs

Returns

Job status

writeJob

Syntax

```
writeJob(  
  job: xtJob,  
  /,  
  *,  
  data: str,  
  timeout: int = None)  
Returns: None
```

Summary:

Write to stdin of an interactive job.

Category:

Tool Integration

Licenses:

None

Parameters

- `job` – The job to read data from.
- `data` – Data to send to the interactive job.
- `timeout` – Time, in seconds, to wait for the data to be sent. Timeout of 0 will return immediately. (Default is: 0)

Returns

None

writeParasitics

Syntax

```
writeParasitics(  
objects: list[deFigure] | list[deSelectionSet] | list[oaFig] |  
         list[oaFigGroup] | list[oaInst] | list[oaMarker] | list[oaShape]  
         | list[oaVia],  
/,  
*,  
file: str,  
format: xt.writeParasitics.Format = None,  
description: str = None,  
totalResistance: str = None,  
corner: str = None)  
Returns: None
```

Summary:

Write a list of parasitics to an output file

Category:

Parasitics

Licenses:

None

Parameters

- objects – Collection of parasitic instances.
- file – Output file name
- format – Format of the output file ({ ascii | csv | xml } (Default is: 'ascii'))
- description – Description of query details (Default is: "")
- totalResistance – Total resistance or instTerms (Default is: "")
- corner – Corner for parasitic value (Default is: "")

Returns

None

snps.custom.cmd.xt.plotQueue

This chapter lists the commands in the `plotqueue` module.

This section covers the following topics:

- [removeFirstItem](#)
- [addNewItem](#)
- [clear](#)
- [getPlotObj](#)
- [hasNoItems](#)
- [peekFirstItem](#)

removeFirstItem

Syntax

```
removeFirstItem(  
viewInst: xtPlotViewerInst)  
Returns: None
```

Summary:

Removes the first item in the plot command queue.

Category:

Netlist Tools

Licenses:

None

Parameters

viewInst – The plot viewer instance whose first queue item to remove

Returns

None

addNewItem

Syntax

```
addNewItem(  
viewInst:  xtPlotViewerInst,  
/,  
*,  
item:  xtPlotQueueItem)  
Returns: None
```

Summary:

Adds an xtPlotQueueItem to the plot command queue.

Category:

Netlist Tools

Licenses:

None

Parameters

- viewInst – The plot viewer instance for which to queue the item
- item – The plot queue item to be added

Returns

None

clear

Syntax

```
clear(  
viewInst: xtPlotViewerInst)  
Returns: None
```

Summary:

Clears all xtPlotQueueItems from the plot command queue.

Category:

Netlist Tools

Licenses:

None

Parameters

viewInst – The plot viewer instance for which to queue the item

Returns

None

getPlotObj

Syntax

```
getPlotObj(  
queueItem: xtPlotQueueItem)  
Returns: xtPlot
```

Summary:

Returns the plot object of the given plot queue item.

Category:

Netlist Tools

Licenses:

None

Parameters

queueItem – The plot queue item whose plot object to get

Returns

The plot object

hasNoItems

Syntax

```
hasNoItems(  
  viewInst: xtPlotViewerInst)  
Returns: bool
```

Summary:

Returns true if the plot command queue is empty.

Category:

Netlist Tools

Licenses:

None

Parameters

viewInst – The plot viewer instance whose queue to check

Returns

True if the plot command queue is empty

peekFirstItem

Syntax

```
peekFirstItem(  
  viewInst: xtPlotViewerInst)  
Returns: xtPlotQueueItem
```

Summary:

Returns the first plot command in the queue.

Category:

Netlist Tools

Licenses:

None

Parameters

viewInst – The plot viewer instance whose first queue item to get

Returns

The first item on the plot command queue

snps.custom.cmd.xt.utils

This chapter lists the commands in the `utils` module.

This section covers the following topics:

- [showNetlistChanged](#)
- [freshPlotPacket](#)
- [getLineage](#)
- [getPacketRGB](#)
- [loadNameMap](#)
- [removeFromRegistry](#)

showNetlistChanged

Syntax

```
showNetlistChanged(  
    window: int | None = None,  
    /,  
    *,  
    files: str)  
Returns: None
```

Summary:

Shows the util dialog for changed netlists

Category:

Netlist Tools

Licenses:

None

Parameters

- window – Window Id
- files – tcl list of files for the dialog

Returns

None

freshPlotPacket

Syntax

```
freshPlotPacket(  
  simRun: xtSimRun)  
Returns: None
```

Summary:

Gets the name of the next display resource packet from the list of plotting packets. The packet exists in the default display.

Category:

Netlist Tools

Licenses:

None

Parameters

simRun – xtSimRun object

Returns

None

getLineage

Syntax

```
getLineage(  
  oaHierPath: oaHierPath)  
Returns: None
```

Summary:

Convert an oaHierPath into a list of instances representing the lineage

Category:

Netlist Tools

Licenses:

None

Parameters

oaHierPath – The oaHierPath object from which to extract

Returns

None

getPacketRGB

Syntax

```
getPacketRGB(  
  packet: str)  
Returns: None
```

Summary:

Get the red green and blue components of the line style of the named packet. The result is a string like #rrggbb where 'rr' are the hex digits that make up the magnitude of the red component (0-255). Similarly, gg and bb represent green and blue digits respectively.

Category:

Netlist Tools

Licenses:

None

Parameters

packet – The name of a packet in the default display.

Returns

None

loadNameMap

Syntax

```
loadNameMap(  
  simRun: xtSimRun)  
Returns: None
```

Summary:

Load the name map.

Category:

Netlist Tools

Licenses:

None

Parameters

simRun – xtSimRun object

Returns

None

removeFromRegistry

Syntax

```
removeFromRegistry(  
    simRun: xtSimRun)  
Returns: None
```

Summary:

Removes xtSimRun from the internal list of registered NTI sessions.

Category:

Netlist Tools

Licenses:

None

Parameters

simRun – xtSimRun object

Returns

None

snps.custom.cmd.ixt

This chapter describes how to use Python submodules to set constraints and lists commands in the Custom Compiler Synopsys Custom command submodule. These submodules let you perform netlisting, general Custom Compiler environment based

simulation, parasitic extraction, physical verification, and external tool integration. Interactive commands are listed in this chapter.

For details, see:

- [crossProbe](#)
- [editPlotSet](#)
- [plotWaveform](#)
- [queryGPDParasitic](#)
- [showGPDPParasiticsOptions](#)

crossProbe

Syntax

```
crossProbe(  
context: deContext | int = None,  
/,  
*,  
type: ixt.crossProbe.Type = None,  
bus: ixt.crossProbe.Bus = None)  
Returns: None
```

Summary:

Cross probe between schematic and parasitic views using the compare database after an LVS run

Category:

Parasitics

Licenses:

None

Parameters

- context – Context of activation.
- type – Type of probe to create. The user is prompted to select the required input for the type of cross probe specified ({ auto | device | net })
- bus – Mode of bus net probe. The user is prompted to select the optional input for the bus net mode of cross probe specified ({ all | selectBits })

Returns

None

editPlotSet

Syntax

```
editPlotSet(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Add and remove nets and ports from the active plotset.

Category:

Netlist Tools

Licenses:

None

Parameters

context – Context of activation.

Returns

None

plotWaveform

Syntax

```
plotWaveform(  
    context: deContext | int = None)  
Returns: None
```

Summary:

Interactively plot waveforms

Category:

Netlist Tools

Licenses:

None

Parameters

context – Context of activation.

Returns

None

queryGPDParasitic

Syntax

```
queryGPDParasitic(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the LE Query Parasitic dialog.

Category:

Layout Editing

Licenses:

Backend or Elite or Apex

Parameters

parent – Window(id) of layout editor window to query GPD Parasitic

Returns

The Dialog

showGPDParasiticsOptions

Syntax

```
showGPDParasiticsOptions(  
*,  
    parent: giWindow | int = None)  
Returns: giDialog
```

Summary:

Open the LE GPD Parasitic option dialog.

Category:

Layout Editing

Licenses:

Backend or Elite or Apex

Parameters

parent – Window(id) of layout editor window to query GPD Parasitic

Returns

The Dialog